普中 51 单片机开发攻略

开源共享　　携手共进

普中51单片机

目录

普中 51 单片机开发攻略 1
第 1 章 如何使用本攻略 1
产品购买地址： 2
第 2 章 如何学习单片机 3
2.1 学习哪种类型的单片机 4
2.2 学习单片机的最佳方法 4
2.2.1 鹦鹉学舌 5
2.2.2 照葫芦画瓢 5
2.2.3 他山之石，可以攻玉 5
2.2.4 理论实践结合，温故而知新 6
2.3 学习单片机的准备工作 6
2.3.1 要有信心、恒心和耐心 6
2.3.2 完善配套的教程 7
2.3.3 计算机一台 7
2.3.4 51 开发板一块 7
2.4 学习单片机的常见疑问 8
2.4.1 学完单片机能做什么 8
2.4.2 学完单片机能找什么工作，薪资待遇怎么样 8
2.4.3 学习单片机应该学 C 还是汇编语言 10
课后作业 11
第 3 章 开发板功能及使用介绍 11
3.1 开发板功能介绍 12
3.2 开发板使用方法 13
3.2.1 CH340 驱动安装 13
3.2.2 程序的烧录（下载） 15
3.2.3 51 仿真器调试 19
3.2.4 实验现象 20
课后作业 20
第 4 章 51 单片机介绍 21
4.1 51 单片机介绍 22
4.1.1 单片机简介 22
4.1.2 51 单片机简介 22
4.2 STC89Cxx 芯片介绍 24
4.2.1 STC89C51 芯片简介 26
4.2.2 STC89C51 芯片引脚介绍 27
4.2.3 STC89C51 芯片内部结构 32
4.3 51 单片机能做什么 33
4.4 怎样学好 51 单片机 35
课后作业 36
第 5 章 数字电路与 C 语言基础 37
5.1 电平特性 38
5.2 二进制与十六进制 39

普中51单片机开发板

5.2.1 二进制 39
5.2.2 十六进制 41
5.3 二进制的逻辑运算 42
5.3.1 与运算 42
5.3.2 或运算 42
5.3.3 非运算 43
5.3.4 同或运算 43
5.3.5 异或运算 43
5.4 C51 基础知识 44
5.4.1 C 语言简介 44
5.4.2 C51 数据类型 45
5.4.3 C51 运算量 48
5.4.4 C51 运算符 53
5.4.5 C51 表达式及复合语句 58
5.4.6 C51 基本结构和相关语句 59
5.4.7 函数 66
5.4.8 C51 构造数据类型 69
课后作业 71
第 6 章 51 单片机最小系统 71
6.1 51 单片机最小系统构成 72
6.1.1 晶振电路 73
6.1.2 复位电路 74
6.1.3 电源电路 75
6.1.4 下载电路 75
课后作业 77
第 7 章 KEIL C51 软件安装 77
7.1 KEIL C51 软件获取 78
7.2 KEIL C51 软件安装 79
7.3 KEIL C51 软件破解 82
课后作业 86
第 8 章 51 单片机工程模板创建 87
8.1 51 单片机工程模板创建 88
8.1.1 新建工程 88
8.1.2 选择 CPU 型号 89
8.1.3 给工程添加文件 91
8.1.4 配置魔术棒选项卡 93
8.1.5 修改编程区内字体大小 96
8.2 reg52.h 头文件介绍 97
8.2.1 头文件作用 97
8.2.2 头文件内容 98
课后作业 102
第 9 章 点亮第一个 LED 103
9.1 51 单片机 GPIO 介绍 104
9.1.1 GPIO 概念 104

9.1.2 GPIO 结构框图与工作原理 105
9.2 LED 简介 112
9.3 硬件设计 113
9.4 软件设计 114
9.4.1 点亮第一个 LED 114
9.4.2 LED 闪烁实验 115
9.4.3 LED 流水灯实验 120
9.5 实验现象 122
课后作业 123
第 10 章 蜂鸣器实验 124
10.1 蜂鸣器介绍 125
10.2 硬件设计 126
10.3 软件设计 127
10.4 实验现象 128
课后作业 129
第 11 章 静态数码管实验 130
11.1 数码管介绍 131
11.1.1 数码管简介 131
11.1.2 数码管显示原理 132
11.1.3 数码管静态显示原理 134
11.2 硬件设计 135
11.3 软件设计 136
11.4 实验现象 137
课后作业 138
第 12 章 动态数码管实验 139
12.1 数码管介绍 140
12.1.1 多位数码管简介 140
12.1.2 数码管动态显示原理 141
12.2 74HC245 和 74HC138 芯片介绍 141
12.2.1 74HC245 芯片简介 142
12.2.2 74HC138 芯片简介 143
12.3 硬件设计 145
12.4 软件设计 145
12.5 实验现象 148
课后作业 148
第 13 章 独立按键实验 149
13.1 按键介绍 150
13.2 硬件设计 151
13.3 软件设计 152
13.4 实验现象 155
课后作业 155
第 14 章 矩阵按键实验 156
14.1 矩阵按键介绍 157
14.2 硬件设计 158
普中51单片机开发板

14.3 软件设计 159
14.4 实验现象 163
课后作业 164
第 15 章 IO 扩展(串转并)-74HC595 164
15.1 74HC595 芯片介绍 165
15.2 硬件设计 166
15.3 软件设计 167
15.4 实验现象 170
课后作业 171
第 16 章 LED 点阵实验 172
16.1 LED 点阵介绍 173
16.2 硬件设计 174
16.3 软件设计 174
16.3.1 LED 点阵（点亮一个点） 174
16.3.2 LED 点阵（显示数字） 177
16.3.3 LED 点阵（显示图像） 183
16.4 实验现象 186
16.4.1 LED 点阵（点亮一个点） 187
16.4.2 LED 点阵（显示数字） 187
16.4.3 LED 点阵（显示图像） 188
课后作业 190
第 17 章 直流电机实验 190
17.1 直流电机介绍 191
17.2 ULN2003 芯片介绍 192
17.3 硬件设计 193
17.4 软件设计 194
17.5 实验现象 196
课后作业 197
第 18 章 步进电机实验 197
18.1 步进电机简介 198
18.2 步进电机工作原理 199
18.2.1 步进电机极性区分 199
18.2.2 双极性步进电机驱动原理 201
18.2.3 单极性步进电机驱动原理 203
18.2.4 细分驱动原理 205
18.3 步进电机技术指标 206
18.3.1 静态技术指标 206
18.3.2 动态技术指标 206
18.4 28BYJ-48 步进电机简介 207
18.5 硬件设计 208
18.6 软件设计 208
18.7 实验现象 213
课后作业 214
第 19 章 中断系统 215

19.1 中断介绍 216
19.1.1 中断概念 216
19.2 中断结构及相关寄存器 218
19.2.1 中断结构 218
19.2.2 中断相关寄存器 220
课后作业 223
第 20 章 外部中断实验 223
20.1 外部中断介绍 224
20.2 外部中断配置 227
20.3 硬件设计 228
20.4 软件设计 228
20.5 实验现象 230
课后作业 231
第 21 章 定时器中断 231
21.1 定时器介绍 232
21.1.1 51 单片机定时器原理 233
21.1.2 51 单片机定时/计数器结构 234
21.1.3 51 单片机定时/计数器的工作方式 235
21.2 定时器配置 237
21.3 硬件设计 239
21.4 软件设计 239
21.5 实验现象 241
课后作业 242
第 22 章 串口通信实验 243
22.1 通信的基本概念 244
22.1.1 串行通信与并行通信 244
22.1.2 异步通信与同步通信 245
22.1.3 单工、半双工与全双工通信 246
22.1.4 通信速率 246
22.2 51 单片机串口介绍 247
22.2.1 串口通信简介 247
22.2.2 串口相关寄存器 250
22.2.3 串口工作方式 252
22.2.4 串口的使用方法 255
22.3 硬件设计 257
22.4 软件设计 259
22.5 实验现象 260
课后作业 261
第 23 章 I2C-EEPROM 实验 262
23.1 I2C 介绍 263
23.1.1 I2C 物理层 263
23.1.2 I2C 协议层 264
23.2 AT24C02 介绍 268
23.3 硬件设计 270

23.4 软件设计 270
23.4.1 创建多文件工程 270
23.4.2 实验代码 281
23.5 实验现象 292
课后作业 293
第 24 章 DS18B20 温度传感器实验 294
24.1 DS18B20 介绍 295
24.2 硬件设计 300
24.3 软件设计 300
24.3.1 数码管显示函数 301
24.3.2 DS18B20 初始化函数 302
24.3.3 温度读取函数 304
24.3.4 主函数 305
24.4 实验现象 307
课后作业 308
第 25 章 DS1302 时钟实验 309
25.1 DS1302 时钟芯片介绍 310
25.1.1 DS1302 简介 310
25.1.2 DS1302 使用 311
25.2 硬件设计 315
25.3 软件设计 316
25.3.1 DS1302 读写函数 316
25.3.2 主函数 320
25.4 实验现象 321
课后作业 322
第 26 章 红外遥控实验 323
26.1 红外遥控介绍 324
26.1.1 红外线简介 324
26.1.2 红外遥控的原理 324
26.2 硬件设计 327
26.3 软件设计 328
26.3.1 红外初始化函数 328
26.3.2 红外解码函数 329
26.3.3 主函数 331
26.4 实验现象 332
课后作业 333
第 27 章 ADC 模数转换实验 333
27.1 ADC 介绍 334
27.1.1 ADC 简介 334
27.1.2 ADC 转换原理 335
27.2 XPT2046 芯片介绍 337
27.3 硬件设计 340
27.4 软件设计 341
27.4.1 ADC 读写函数 342
普中51单片机开发板

27.4.2 主函数 344
27.5 实验现象 345
课后作业 347
第 28 章 DAC 数模转换实验 348
28.1 DAC 介绍 349
28.1.1 DAC 简介 349
28.1.2 DAC 工作原理 350
28.2 PWM 介绍 351
28.3 硬件设计 352
28.4 软件设计 353
28.4.1 PWM 实现函数 353
28.4.2 主函数 355
28.5 实验现象 356
课后作业 357
第 29 章 LCD1602 液晶显示实验 358
29.1 LCD1602 介绍 359
29.1.1 LCD1602 简介 359
29.1.2 LCD1602 常用指令 361
29.1.3 LCD1602 使用 362
29.2 硬件设计 364
29.3 软件设计 364
29.3.1 LCD1602 驱动函数 365
29.3.2 主函数 370
29.4 实验现象 371
课后作业 372
第 30 章 LCD12864 液晶显示实验 373
30.1 LCD12864 介绍 374
30.1.1 带字库 LCD12864 374
30.1.2 不带字库 LCD12864 377
30.2 硬件设计 378
30.3 软件设计 379
30.3.1 LCD12864 显示函数 379
30.3.2 主函数 382
30.4 实验现象 383
课后作业 384
第 31 章 TFTLCD 显示实验 385
31.1 TFTLCD 介绍 386
31.2 硬件设计 390
31.3 软件设计 391
31.3.1 TFTLCD 初始化函数 392
31.3.2 TFTLCD 显示函数 407
31.3.3 字符汉字图片提取软件使用 420
31.3.4 主函数 428
31.4 实验现象 429

课后作业 431
第 32 章 触摸屏实验 432
32.1 触摸屏介绍 433
32.1.1 电阻式触摸屏介绍 433
32.1.2 电容式触摸屏介绍 435
32.2 硬件设计 437
32.3 软件设计 438
32.3.1 触摸屏扫描函数 438
32.3.2 主函数 449
32.4 实验现象 452
课后作业 455
普中51单片机开发板

第 1 章 如何使用本攻略

学习本开发攻略主要参考的文档有《STC89Cxx 中文参考手册》，这是 STC 官方手册，里面包含了 STC89Cxx 单片机内部所有资源介绍，非常详细。大家在学习 51 单片机的时候可以参考下这个文档，特别是涉及到外设寄存器部分。在后面具体的章节中也会提到所要参考的文档等提示信息。

本攻略编写风格是：

（1）外设介绍，包括外设内部结构框图等

（2）外围芯片使用方法讲解

（3）硬件讲解

（4）软件分析

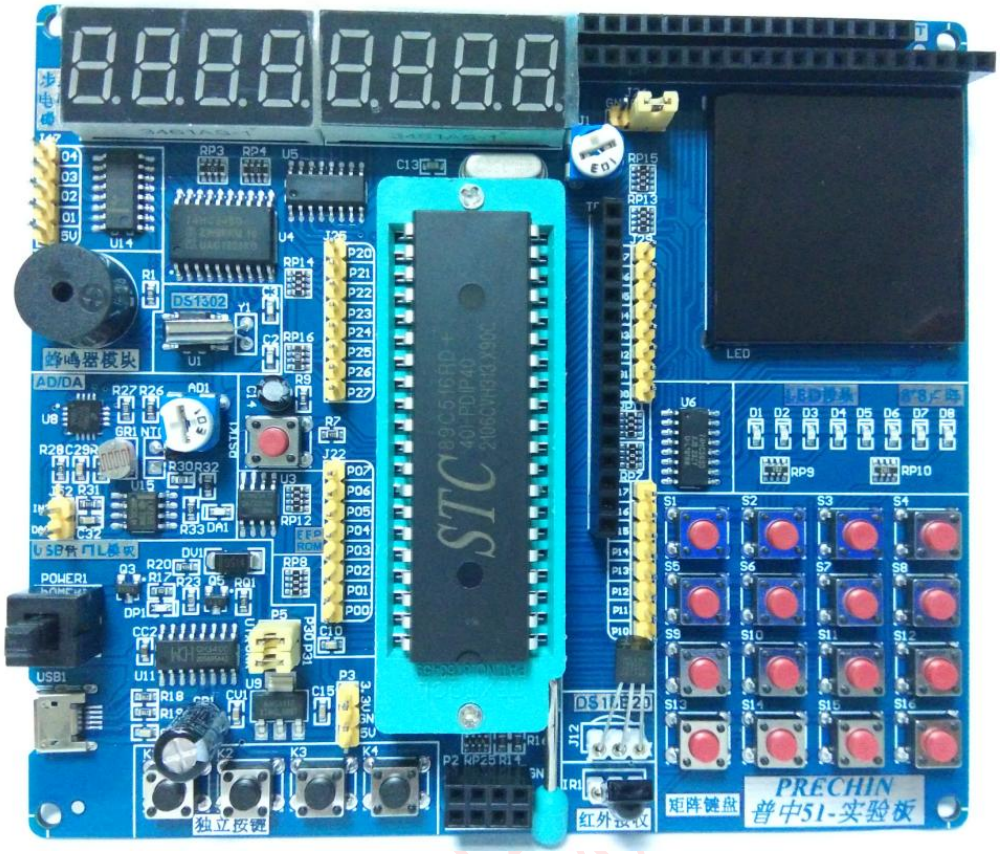
（5）实验现象

通过上述几大块的介绍让您快速掌握 51 单片机开发。

本开发攻略配套的实验平台为：普中 51-实验板，这款开发板出厂搭配的是 STC89C52 或 STC89C516 芯片,学习的时候如果配套该硬件平台做实验，必会达到事半功倍的效果，可以省去中间移植时遇到的各种问题。

普中 51-实验板外观图如下：（此图没有采用任何包装拍摄）

普中51单片机开发板



在学习的过程中，如果遇到什么问题，可以到我们技术论坛：www.prechin.net 发帖交流，也可联系我们技术电话：0755-21509063，大家共同进步。

鉴于作者水平有限，难免会有纰漏，还请热心的读者指正并发到论坛，好让我们改进，祝您生活学习愉快。在 51 单片机的学习过程中，我们与您同行！

产品购买地址：

（1）购买地址（普中授权店铺）

http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=

（2）资料下载

http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1

（3）技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）

第 2 章 如何学习单片机

有很大一部分同学，在看到这类章节会直接跳过，因为大多数类似章节都是废话连篇。但是，这里作者可以很负责任的告诉你，本章讲到的学习单片机的方法，都是作者学习单片机的无数经验和教训总结出来的。希望各位都能站在作者的肩膀上，看得更远！本章主要分以下几部分内容：

2.1 学习哪种类型的单片机

2.2 学习单片机的最佳方法

2.3 学习单片机的准备工作

2.4 学习单片机的常见疑问

普中51单片机开发板

2.1 学习哪种类型的单片机

单片机的型号那么多，该如何选择一款合适的进行学习呢？这里给读者首推的当然是学习 51 单片机。为什么呢？虽然现在单片机种类和型号非常多，每个型号都有一定的市场份额，但是哪一个型号也没有早期 51 单片机那般风光和火爆，虽然现在的地位不是那么高了，但是因为 51 单片机具有先天的简单易用性，而且积累的资料非常多，大家学起来就会拥有非常多的学习资料，以及遇到问题时的各种解决方法，所以上手肯定比其他型号要快很多。如果学习稍微偏门的单片机，可能一个简单的软件问题就要折腾好长时间，最后不仅浪费了宝贵的学习时间，更重要的是打击了学习单片机的信心，最终可能就放弃，可能就损失一位优秀的嵌入式工程师。因此，对于初学者来说，选择好单片机学习是非常重要的。

那么是不是每种单片机都要学习一遍呢？答案当然是否定的。单片机型号那么多，挨个下来完估计头发白了也学不完。所以大家跟着作者学习 51 单片机，必须得学会举一反三、融会贯通的能力。在学习 51 单片机的时，不能仅仅当做 51 来学习，要当做“单片机”来学，要把单片机的内部资源都搞清楚、弄明白，每个内部资源模块的用法理解透彻，这样当遇到一个从没有用过的单片机，也能快速知道如何下手去使用它。

2.2 学习单片机的最佳方法

单片机是一门实用技术，学习它已经不是为了应付考试，不需要死机硬背。下面给大家总结单片机的学习方法：一个要领，四个步骤。

学习单片机的要领就是：在实践中成长，不断的去练习！

学游泳得下水，整天在岸上做模仿活动肯定不行。同样的，学单片机，整天盯着单片机书看也不行，必须得亲自动手去练习。这里不是不让大家看书，而是看了以后要马上去实践验证，然后再回头结合实践的结果理解书上的内容。遇到问题查书比直接看书的效果要好过百倍。

这点相信大家都清楚，可为什么还是会有很多人学单片机的时候始终抱着一本书看呢？第一，学习模式如此，学什么东西主要都是靠看书，应付书面考试。第二，很多人想实践却不知道该如何实践。

下面就是学习单片机的四个步骤：

2.2.1 鹦鹉学舌

刚出生的孩子叫“爸爸”、“妈妈”的时候，他甚至不知道“爸爸”、“妈妈”是什么意思，但是当带着孩子见到爸爸就让他喊“爸爸”，见到妈妈就让他喊“妈妈”，次数多了，孩子就知道谁是爸爸，谁是妈妈了。

大家刚开始接触单片机的时候，也属于单片机行业的新生儿。单片机的样子，单片机内部资源，单片机外围的各种器件，单片机使用 C 语言编程的方法，初学者可能都没有见过，脑子里全无概念。没关系，有些概念和方法不理解也没有关系，甚至不需要去理解，只需要跟着去鹦鹉学舌式的学习，一遍遍去模仿，就比如程序，大家跟着抄下来，甚至抄个两三遍，过段时间会发现，好多东西也认识了，好多概念也慢慢清楚了，也能大概看懂别人的小程序了，**此时切忌觉得自己看会了就简单复制粘贴**。

2.2.2 照葫芦画瓢

很多同学学习的时候喜欢看视频、程序，甚至看别人的程序都能看懂，觉得自己就会了，可是等到自己写程序的时候，却不知道从何下手了，这是初学者很容易犯的“眼高手低”的毛病，所以第二步的内容就非常重要了。

这就要求每一位同学，在学习完当前课程的内容，把第一步顺利完成以后，然后把视频、源代码统统关掉。自己通过原理图和查找非源代码的其他任何资料，把当节课的程序重新默写出来，边写边多少理解那么一点点，而不是纯粹的死记，应该说是记忆加理解的结合。甚至学过几节课以后，可以回头把前面曾经这样实现的课程，再按照这种方法实现一遍。千万不要认为这一步没有必要，这一步是读者能否学会单片机关键的一步，在学完本教程之前，每一课的内容都要这样做，如果每一课都能这样完成，可以说，当节课的百分之七八十内容已经掌握了。

2.2.3 他山之石，可以攻玉

单片机技术的最大特点就是可以通过修改程序来实现不同的功能。因此举一

反三的能力就必不可少了。每学完一节课后都会预留作业，大家尽量去独立完成，可以参考当节课的程序思路，在这个基础上结合自己的思考去完成作业。

在实际产品开发中，很多时候也是如此。比如开发一个产品，从零起步的话，可能会走很多弯路，会遇到前人曾经遇到过的问题，开发周期大大拉长。所以通常的做法是寻找或购买几款同类产品，然后先研究它们各自的优缺点，然后在同类产品的基础上设计自己的产品，这就是“他山之石，可以攻玉”。

初学者在学习的时候往往会遇到很多问题，这时不要慌张，要想到你遇到的问题，可能前辈们早就遇到过，所以可以借助百度或谷歌在网上搜索下是否有解决方法，多多参考别人的东西，只要把别人的东西分析明白，自己用起来就成为自己的知识了。

2.2.4 理论实践结合，温故而知新

当大家把所有的课程按照前面 3 步完成后，这个时候不妨再把书本打开，再看看书，经过自己的实战经历，再看书的时候，对很多知识点会有一种恍然大悟的感觉。甚至可能有的知识点在当时学习时不明白，过了一段时间，回头再学习的时候，一下就明白了。

2.3 学习单片机的准备工作

2.3.1 要有信心、恒心和耐心

有很多同学问过我，单片机这门技术难不难，可以多久学会？我觉得这个问题可以从以下两个方面去分析。

首先，要从战略上藐视它。要想到，有那么多同学跟着老师学习一段时间就可以做出来小车，甚至机器人来。他们又不是三头六臂，他们能做到的，我们也没有做不了的道理。其实当你真正学会了单片机就能明白，实际上单片机的使用是非常简单的，只要大家认真踏实的坚持学下去，肯定是能学好这门技术的。

其次，要从战术上重视它。如果能在短短十天八天就学会了，那么这个技术还能值钱吗？可以这样说，如果一个技术可以很简单的掌握，很多人都会的这种

技术，肯定也没有什么钱途。那究竟多久能学会呢？按照我给大家制定的学习方案，根据每个人的基础不同，平均每天拿出 2 个小时以上的学习时间，大概 1 个月可以入门。入门的概念是给读者一个单片机开发任务，能起码知道要努力的方向和解决问题的大概方法。技术这东西，关键是要坚持做下去，要有恒心和耐心，如果长时间不用的话，肯定还会生疏。所以要想成为单片机高手，起码需要一年左右的单片机开发历练才行。这里所说的单片机高手是指可以根据自己的想法去设计一个电路，根据需要的功能编写代码，做一个产品出来。

2.3.2 完善配套的教程

单片机技术主要涵盖了单片机和 C 语言知识，要学习这门技术，完善配套的良好教程是必不可少的。本套教程将单片机由内到外涉及到的资源，详细的分析讲解，全程以 C 语言由易到难贯穿整个单片机开发。大家可以直接学习本套教程即可，当然对于想深入 C 语言的同学，可以再找本纯 C 语言的教材，当学习单片机时用到的 C 语言知识点，理解的不够透彻时，可以查阅下，当然也可以直接在百度或谷歌上搜索一下，绝大多数情况都会找到满意的答案。

2.3.3 计算机一台

计算机是学习单片机必不可少的工具，在设计仿真电路、编写程序、查阅资料时都会用到它。

2.3.4 51 开发板一块

有同学可能会问，是不是还得准备一块 51 单片机开发板呢？我的回答是，如果条件允许最好买一块开发板，这样编写的代码可直接在硬件上验证，况且现在的 51 单片机开发板售价也非常低，没有理由为了节约这点钱而放弃以后的钱途。当然，有的朋友可能会说可以使用 Proteus 仿真软件来模拟，确实可以通过该软件来仿真，但仿真与实际电路运行效果肯定还是有差别的，特别对于初学者，编写的程序稍微有点漏洞，在仿真软件中运行正常，但在实际硬件中运行却不正常。我们也遇到过很多用户咨询这方面的问题，明明在仿真软件上可以运行，但

实际电路却不正常。所以选择一款开发板还是非常有必要的。

2.4 学习单片机的常见疑问

很多朋友在开始选择学习单片机时会有这些疑问，下面作者就列出供大家参考。

2.4.1 学完单片机能做什么

单片机的应用非常广泛，电子、电气、自动化、通信等领域都有大量的应用。生活中常见的电子产品如：智能小车、LED 点阵屏、四轴飞行器、电子密码锁、门铃、超市储物柜、智能电饭煲、智能台灯、智能风扇等。工业领域的应用主要体现在单片机的控制、通信方面。

2.4.2 学完单片机能找什么工作，薪资待遇怎么样

基于单片机的应用方向，主要可以从事智能电子产品、通信技术、运动控制技术等相关工作，当然单片机本身仅是一个工具，一个产品开发往往还会涉及到周边的一些技术，所以确定好应用方向后，还需要熟悉它的外围技术。比如作者之前做过工业通信方面的产品，使用的是 51 单片机，但要和工业产品 PLC 通信不仅要掌握 51 单片机，还要掌握 Modbus 通信协议的相关知识，所以在确定自己应用方向后，还需要了解这个方向涉及到的外围技术。

相信大家学习一门技术，除了兴趣爱好，最关心的应该是薪资待遇。那么从事单片机相关开发工作，工资待遇怎么样呢？下面作者直接从智联招聘网中截取 2021 年 4 月 8 日最新招聘数据。

单片机软件工程师 立即沟通
15K-20K 深圳-宝安区 1-3年 本科
求贤若渴 员工旅游 节日福利 每年多次调薪 五险一金 年终分红
深圳市航林微电子有限公司 会员
民营 100-299人
最新

单片机软件工程师/单片机工程... 立即沟通
10K-20K 深圳-龙华区 经验不限 大专
求贤若渴 年底双薪 五险一金 节日福利 员工旅游 周末双休
深圳市爵影科技有限公司 会员
民营 100-299人
最新

嵌入式软件单片机工程师 立即沟通
8K-14K 深圳 1-3年 本科
求贤若渴 年底双薪 绩效奖金 弹性工作 节日福利 五险一金
深圳市港盛电子有限公司 会员
外商独资 20人以下
最新

单片机软件工程师 立即沟通
8K-12K 深圳-宝安区 经验不限 本科
深圳逆时针科技有限公司 会员
民营 20-99人

单片机软件工程师 立即沟通
8K-12K 深圳-龙华区 1-3年 本科
求贤若渴 五险一金 弹性工作
深圳市易酷智行科技有限公司 会员
民营 20-99人
最近

单片机软件工程师 立即沟通
15K-20K 深圳 3-5年 大专
求贤若渴 五险一金 员工旅游 节日福利 加班补助 年底双薪
深圳市硅宇电子有限公司 会员
民营 20-99人
最新

单片机软件工程师 立即沟通
10K-15K 深圳 1-3年 本科
工作环境好 同事很nice 人际关系好 管理人性化
艾美特电器(深圳)有限公司 ZP名企
外商独资 1000-9999人
最新

单片机开发工程师/MCU软件开... 立即沟通
5K-8K 深圳-福田区 经验不限 大专
深圳市鹏思电子有限公司 会员
民营 20-99人

从上述数据可以看出，学好单片机技术钱途还是可以的，当然这些公司肯定不会要求只会 51 单片机，但通过学习 51 单片机后，再往高级的 STM32、DSP 等学习会更快。大家如果还不确定学什么好，可以多去招聘网看下对岗位的要求，按照那个要求努力就行。

2.4.3 学习单片机应该学 C 还是汇编语言

汇编语言相比其他高级语言，是最接近单片机底层的，程序执行效率高。如果是几行代码就可实现的功能，使用汇编编写还可以。但如果程序容量达到成百上千甚至上万行后，汇编语言在组织结构、修改维护等方面就会成为你的噩梦。此时 C 语言就展现它不可替代的优势，虽然使用 C 语言开发的程序执行效率相对汇编要低一些，但如今处理器速度越来越快，这点降低的效率完全可忽略。

在实际开发中，目前至少有 95%以上的工程师是使用 C 语言做单片机开发的，只有在一些特殊要求的场合才会用到汇编，所以这里强烈建议大家使用 C 语言开发。

课后作业

第 3 章 开发板功能及使用介绍

本章将向大家介绍普中 51-实验板（以下简称普中-2 或普中-3 或普中-4）的功能及使用方法，通过本章的学习，让大家能快速上手开发板的学习。本章分为如下几部分内容：

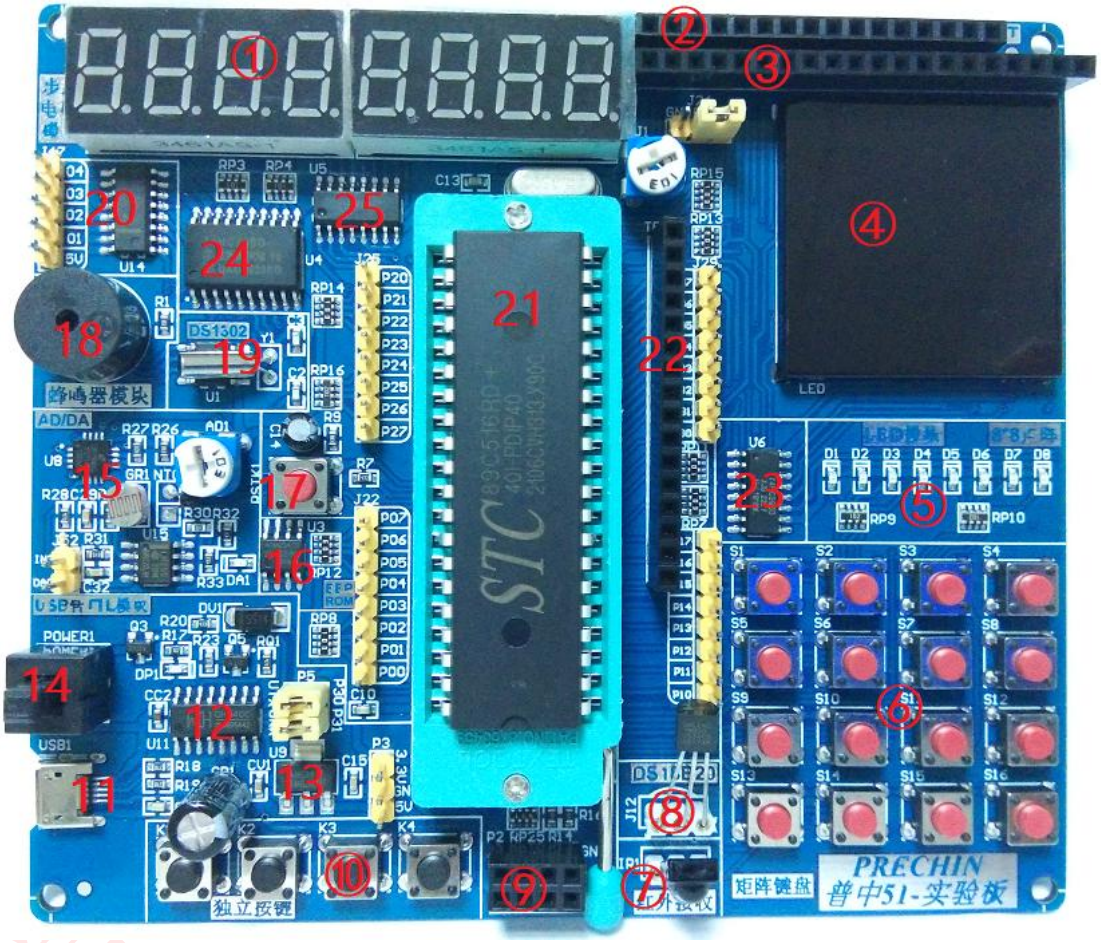
3.1 开发板功能介绍

3.2 开发板使用方法

普中51单片机开发板

3.1 开发板功能介绍

我们先来看下开发板各功能模块，如下：



序号	板载资源	说明
1	数码管模块	2个四位一体共阴数码管，可显示8位数字字符，如时钟、密码等
2	LCD1602接口	兼容LCD1602/LCD9648液晶屏，可显示数字字符等
3	LCD12864接口	兼容LCD9648/MiniLCD12864/带字库LCD12864液晶，可显示汉字/图像等
4	8*8LED点阵	可显示数字/字符/图形等
5	LED模块	8个LED，可实现流水灯等花样显示
6	矩阵按键	4*4矩阵按键，可作计算器、密码锁等应用的输入装置
7	红外接收头	NEC协议，可实现遥控
8	DS18B20温度传感器	可实现温度采集控制等
9	NRF24L01接口	可实现2.4G远程遥控通信
10	独立按键	4个按键，可作按键输入控制
11	MicroUSB接口	可作电源输入、程序下载等
12	USB转TTL模块	CH340C芯片，可作电脑USB与单片机串口下载和通信
13	3.3V电源模块	ASM1117-3.3芯片，将5V转为3.3V供用户使用
14	电源开关	系统电源控制开关
15	ADC/DAC模块	XPT2046芯片作为ADC，LM358+PWM作为DAC，可采集外部模拟信号和输出模拟电压
16	EEPROM模块	AT24C02芯片，可存储256字节数据，掉电不丢失
17	复位按键	系统复位
18	无源蜂鸣器	可作报警提示、音乐等
19	DS1302时钟模块	DS1302芯片，可作时钟发生
20	步进电机驱动模块	ULN2003芯片，可作直流电机、28BYJ48步进电机驱动
21	STC89Cxx单片机座及IO	固定单片机，并将所有IO引出，方便用户二次开发
22	TFTLCD模块接口	与3号接口组合可连接TFTLCD触摸屏，可实现酷炫GUI人机
23	74HC595	扩展IO，控制LED点阵
24	74HC245	驱动数码管段选显示
25	74HC138	驱动数码管位选显示

3.2 开发板使用方法

3.2.1 CH340 驱动安装

上一节，我们已经介绍了开发板各个模块的功能，下面我们看看如何使用这款开发板。

首先，拿到开发板，我们要安装下 USB 转串口 CH340 驱动，对于大多数电脑系统,将 USB 线连接电脑和开发板的USB接口后会自动检测安装 CH340 驱动，如果您的电脑没有自动安装 CH340 驱动，没关系，我们可以手动安装，打开资料目录“\5--开发工具\2-开发板 CH340 驱动”，如下：

双击 SETUP.EXE 应用程序，出现如下界面，点击安装即可。

一段时间后，如果安装成功会显示如下界面：（前提：必须使用 USB 线将电脑 USB 口和开发板 USB 接口连接）

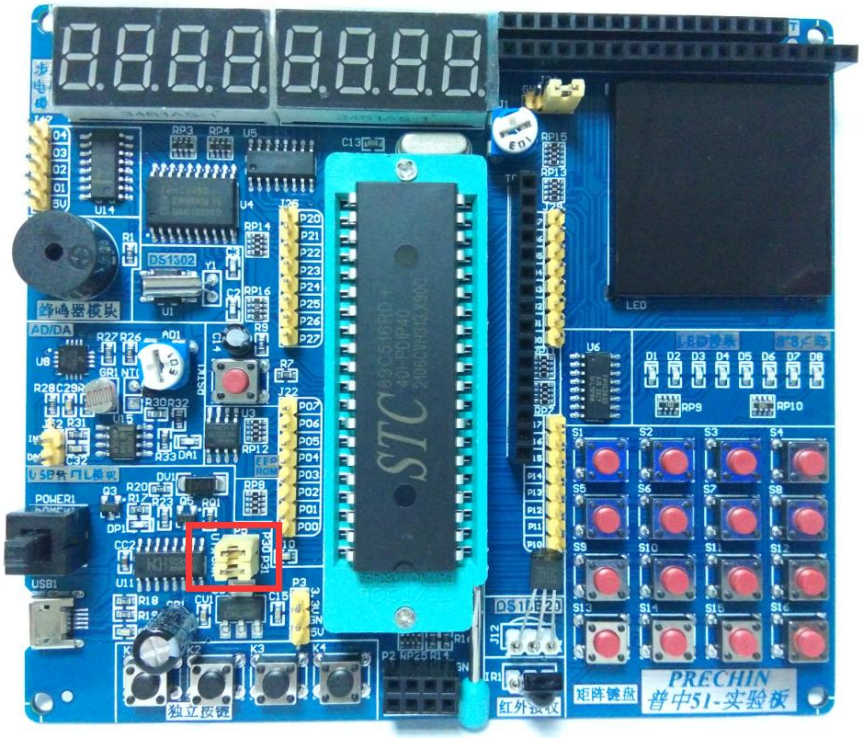
如果显示“驱动预安装成功”或者“驱动安装失败”等提示信息，表明驱动安装不成功。这时可以打开资料目录“\5--开发工具\2-开发板 CH340 驱动\安装失败解决办法”，安装对应的驱动。如果还是安装失败，可以重新换条 USB 线（支持安卓手机数据线）再次安装测试；如果依然安装失败，可以手动将“\5--开发工具\2-开发板 CH340 驱动\安装失败解决办法”对应自己系统的文件夹内 serenum.sys 和 serial.sys 这两个文件，拷贝到 C:\Windows\System32\drivers 文件夹下。如果该文件夹下本来就有这两个文件，提示无法替换，那么请先删除这两个原有的文件，再拷贝过去即可。然后再试试能否成功安装驱动，通过上述操作一般就可以解决串口无法安装/使用的问题了。

如果还安装失败，你的电脑系统是 WIN8 以上，可以试试关闭电脑数字签名，具体方法请百度“数字签名如何关闭”。假如还是安装失败，请联系我们技术电话：0755-21509063，或者到我们公司论坛：www.prechin.net 发帖咨询，我们技术看到后会第一时间给您处理。

驱动安装成功后可以打开打开资料目录“\5--开发工具\3-程序下载软件\PZ-ISP.exe”软件，查看串口号是否显示有“CH340”字样的串口，如果有则证明驱动安装成功，否则失败。如下：

3.2.2 程序的烧录（下载）

安装好 CH340 驱动后，我们就可以下载程序了，在下载程序前先确认下开发板上的 USB 转 TTL 串口模块上的 P5 端子短接片是否短接好（即 P31T 与 URXD 连接，P30R 与 UTXD 连接），出厂的时候该短接片默认已经短接好，如下所示：

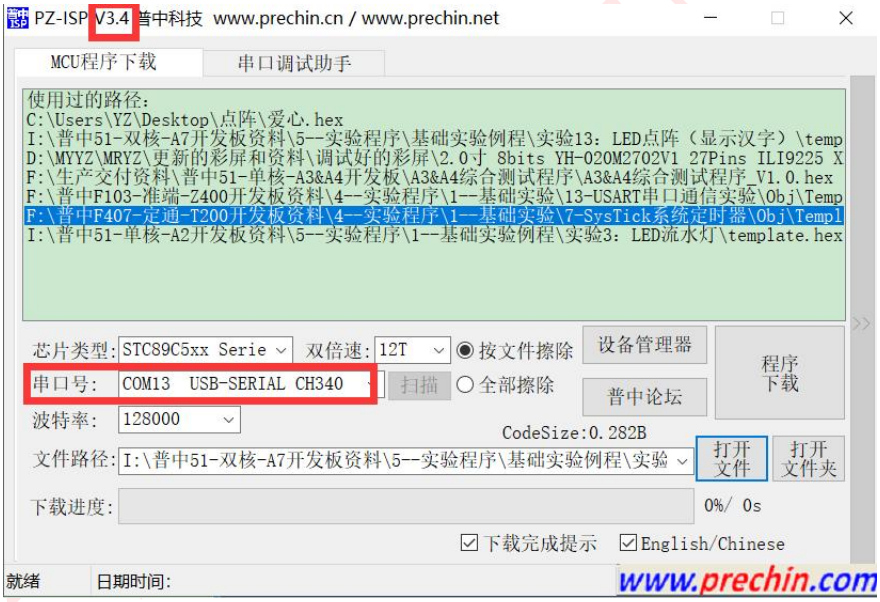


可以使用本公司开发的自动下载软件，软件在资料目录“\5--开发工具\3-程序下载软件”内，可以看到里面有两个烧写软件，一个是“STC-ISP”下载软

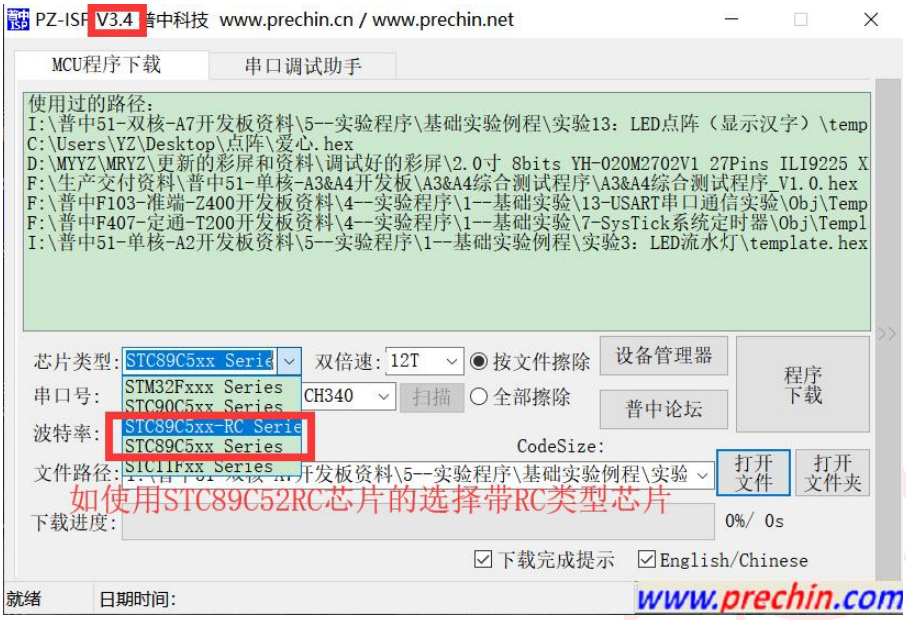
件，另一个是“PZ-ISP”下载软件。STC-ISP 下载软件是需要冷启动，即先点击下载然后开启电源，操作较为复杂，不推荐使用该软件。而 PZ-ISP 是普中自主研发的自动下载软件，可一键下载，操作非常简单，推荐大家使用 PZ-ISP 软件下载程序。打开此软件选择对应的 51 单片机类型进行程序下载。具体操作如下：

1，打开“\5--开发工具\3-程序下载软件\PZ-ISP（推荐使用）”，如下图：

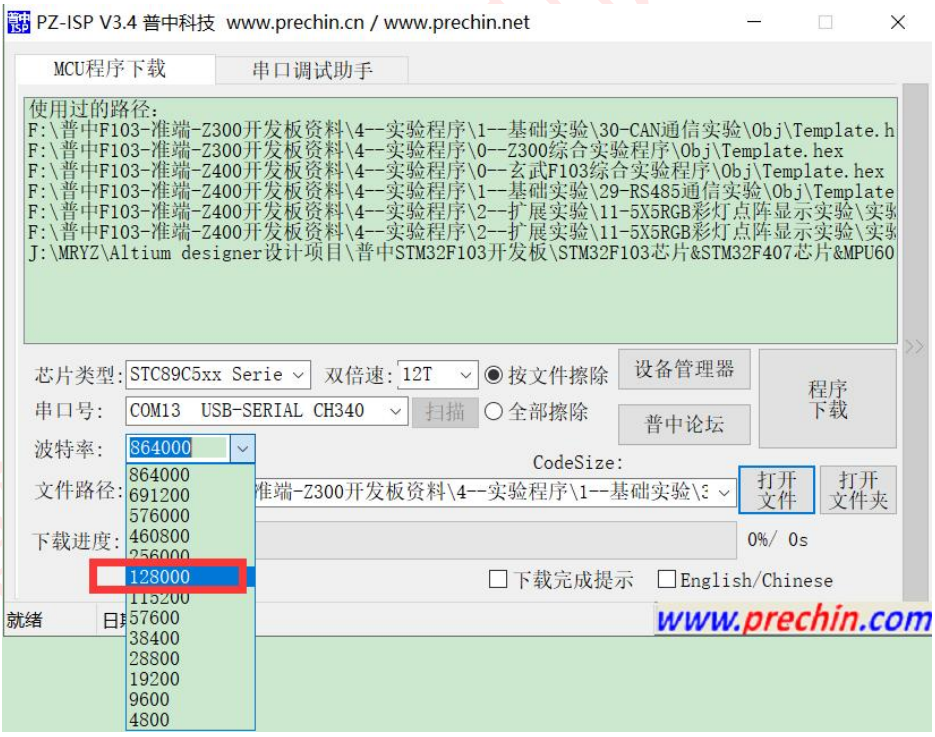
鼠标双击该软件，弹出如下界面：（注意软件的版本号）（注意：此时默认你已经安装好了 CH340 驱动， 此时可以看到对应的串口号，我们这里显示的是“COM13 USB-SERIAL CH340”，你们电脑不一定是这个串口）



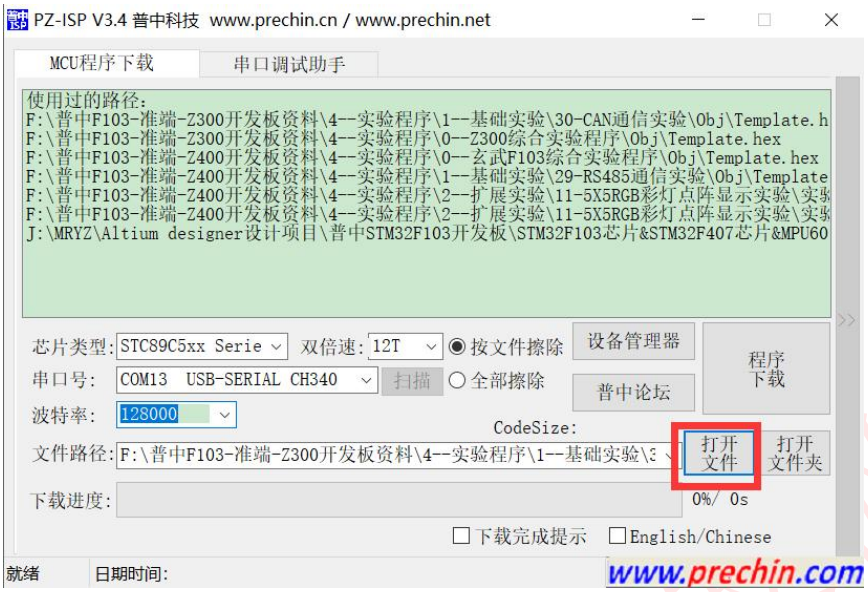
2，根据板载芯片型号是否含有 RC 来选择芯片类型，比如板载芯片使用 STC89C52RC 或 STC89C52RD+或 STC89C516RC 或 STC89C516RD+，则选择芯片类型为“STC89C52xxx-RC”。如果使用 STC89C52，则芯片类型为“STC89C52xx”。如下图：



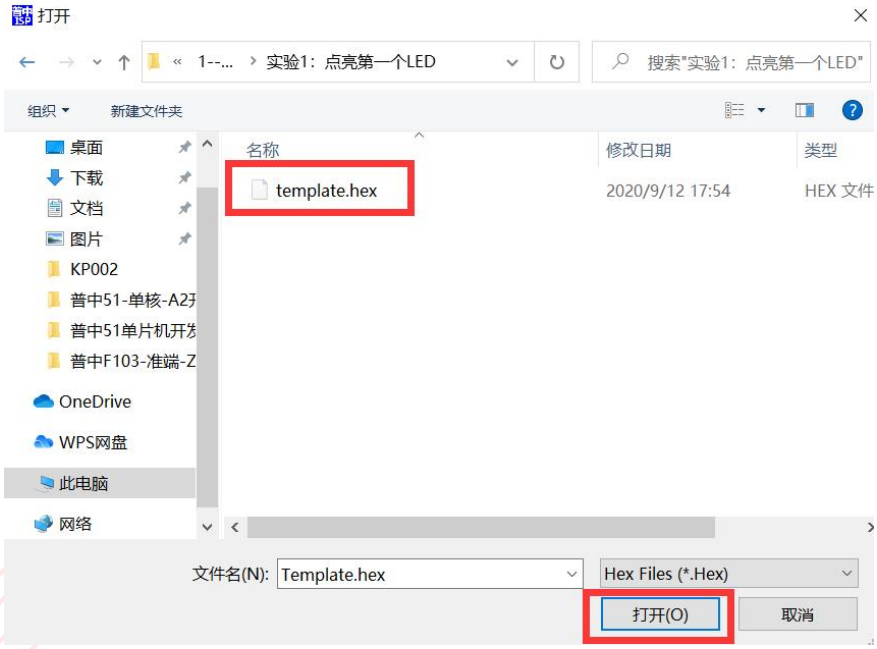
3，将波特率设置为“128000”（如果发现此波特率下载速度比较慢，可以提高波特率，如果下载失败，可以把波特率降低，总之选择一个能下载的波特率），如下图：

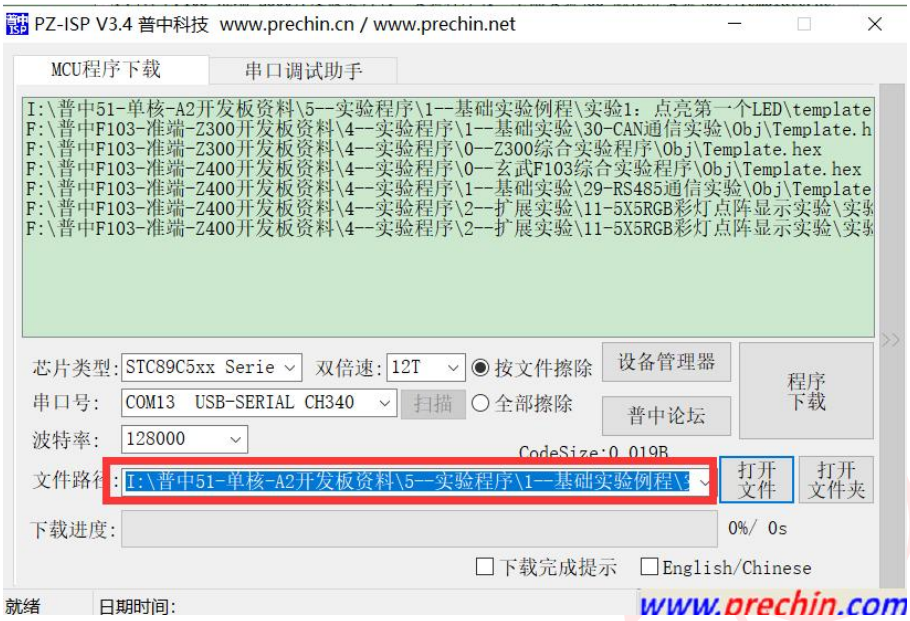


其他的选项我们保持默认设置。下面点击“打开文件”，操作如下：



选择实验程序文件夹内.HEX 文件，点击“打开”即可选择好需要下载的程序，操作如下：





点击“程序下载”按钮即可完成程序下载。当程序下载完成会提示程序下载成功，操作如下：

3.2.3 51 仿真器调试

可使用普中 51 仿真器插入单片机座进行在线调试（前提需购买普中 51 仿真器），但是需要在 KEILC51 软件内设置，在本开发攻略的“51 工程模块创建”章节内有详细的说明，这里就不叙述。

3.2.4 实验现象

最后，程序下载成功后即可观察到实验现象。在后面章节的实验中，本教程都有单独一节“实验现象”介绍，大家在做实验时可以打开看下。

课后作业

普中51单片机开发板

第 4 章 51 单片机介绍

在目前，在电子界，如果想进入微控制器领域的学习，相信绝大多数的人会认为要从 51 单片机开始。不错，在众多微控制器中，51 单片机确实是非常适合初学者入门学习的，因此很有必要先认识和了解 51 单片机。本章将向大家介绍 51 单片机，通过本章的学习，让大家对 51 单片机有一个深入的了解，为后面 51 单片机的学习做好铺垫。本章分为如下几部分内容：

4.1 51 单片机介绍

4.2 STC89Cxx 芯片介绍

4.3 51 单片机能做什么

4.4 怎样学好 51 单片机

普中51单片机开发板

4.1 51 单片机介绍

4.1.1 单片机简介

一台能够工作的计算机要有这样几个部件构成：CPU（进行运算、控制）、 RAM（数据存储）、ROM（程序存储）、输入/输出设备（例如：串行口、并行输出口等）。在个人计算机上这些部件被分成若干块芯片，安装一个称之为主板的印刷线路板上。而在单片机中，这些部件全部被做到一块集成电路芯片中，所以就称为单片机（也称微控制器 MCU），而且有一些单片机中除了上述部份外，还集成了其它部份如 A/D，D/A 等。体积不大，一般用 40 脚封装， 当然功能多一些单片机也有引脚比较多的， 如 68 引脚， 功能少的只有 10 多个或 20 多个引脚，有的甚至只 8 只引脚。

4.1.2 51 单片机简介

51 单片机是对所有兼容 Intel 8031 指令系统的单片机的统称。该系列单片机的始祖是 Intel 的 8004 单片机，后来随着 Flash rom 技术的发展，8004 单片机取得了长足的进展，成为应用最广泛的 8 位单片机之一，其代表型号是 ATMEL 公司的 AT89 系列，它广泛应用于工业测控系统之中。很多公司都有 51 系列的兼容机型推出，今后很长的一段时间内将占有大量市场。51 单片机是基础入门的一个单片机，还是应用最广泛的一种。需要注意的是 51 系列的单片机一般不具备自编程能力。

80C51 是 MCS-51 系列中的一个典型品种；其它厂商以 8051 为基核开发出的 CMOS 工艺单片机产品统称为 80C51 系列。当前常用的 80C51 系列单片机主要产品有：

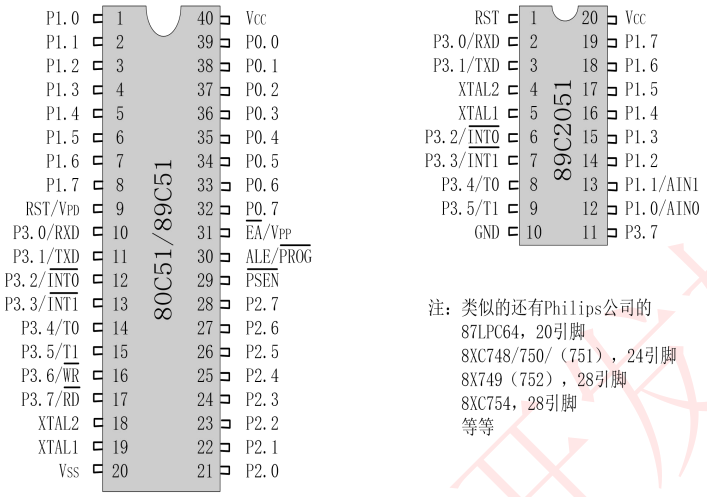
Intel(英特尔)的：i80C31、i80C51、i87C51，i80C32、i80C52、i87C52 等；

ATMEL(艾德梅尔)的：AT89C51、AT89C52、AT89C2051，AT89S51（RC），AT89S52（RC）等；

Philips(飞利浦)、华邦、Dallas(达拉斯)、Siemens(西门子)等公司的许多产品；

STC(国产宏晶)单片机：STC89C51、STC89C52、STC89C516、STC90C516 等众多品牌。

80C51 芯片管脚图如下所示：



上图中左图是 51 单片机非常经典的管脚图，通常采用的是 DIP-40 封装。

其内部资源如下：

- 8 位 CPU
- 4kbytes 程序存储器(ROM) (52 为 8K)
- 128bytes 的数据存储器(RAM) （52 有 256bytes 的 RAM）
- 32 条 I/O 口线
- 111 条指令，大部分为单字节指令
- 21 个专用寄存器
- 2 个可编程定时/计数器
- 5 个中断源，2 个优先级（52 有 6 个）
- 一个全双工串行通信口
- 外部数据存储器寻址空间为 64kB
- 外部程序存储器寻址空间为 64kB
- 逻辑操作位寻址功能
- 双列直插 40PinDIP 封装
- 单一+5V 电源供电

CPU：由运算和控制逻辑组成，同时还包括中断系统和部分外部特殊功能寄存器；

RAM：用以存放可以读写的数据，如运算的中间结果、最终结果以及欲显示的数据；

ROM：用以存放程序、一些原始数据和表格；

I/O 口：四个 8 位并行 I/O 口，既可用作输入，也可用作输出

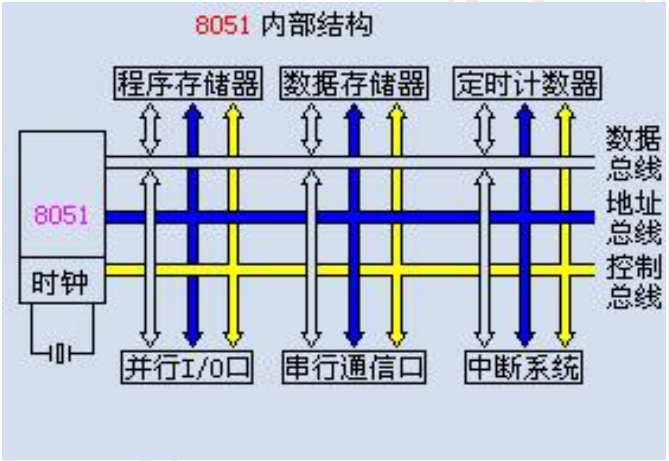
T/C：两个定时/记数器，既可以工作在定时模式，也可以工作在计数模式；

五个中断源的中断控制系统；

一个全双工 UART（通用异步接收发送器）的串行 I/O 口，用于实现单片机之间或单片机与微机之间的串行通信；

片内振荡器和时钟产生电路，石英晶体和微调电容需要外接。最佳振荡频率为 6M—12M。

其内部结构图如下所示：



总线（BUS）是单片机各部件之间传送信息的公共通道。单片机中有内部总线和外部总线两类，内部总线是 CPU 内部之间的连线，外部总线是指 CPU 与其它部件之间的连线；外部总线有三种：数据总线 DB（Data Bus），地址总线 AB（Address Bus）和控制总线 CBControl Bus）。

4.2 STC89Cxx 芯片介绍

前面介绍了很多款基于 MCS-51 内核设计的 51 单片机，这么多款 51 单片机是不是都要去学习呢？当然不是，既然他们的内核指令都是一样的，那我们只需要学会一款典型并具代表性的 51 单片机即可，其他的都是可以兼容的。

早期很长一段时间，51 单片机大部分使用的是 AT89C51 或者 AT89C52。随着

我国芯片技术的快速发展，宏晶公司推出的增强型 51 单片机 STC89Cxx/STC90Cxx 等系列更受大众喜爱，除内部资源及功能大大增强外，还有一个非常重要的是它支持 ISP（在系统可编程）/IAP（在应用可编程），无需专用编程器或专用仿真器。宏晶公司推出的 51 芯片种类非常多，我们只需选择一款经典的学习即可。我们开发板上使用的是 STC89C52 或者 STC89C516，这两款芯片以及其他 STC89Cxx 或者 STC90Cxx 除 Flash 和 RAM 容量差异外，内部资源功能几乎一样，我们只需学习任意一种即可。下面我们看下常见 PDIP 封装（直插封装）的 51 单片机芯片实物图如下所示：

上图可看到芯片表面印有 STC89C52RC 40I-PDIO40 1947H0XN52.X90C 丝印，下面我们了解下芯片表面丝印的含义：

STC--表示芯片为 STC 公司生产的产品，其他公司的也有 AT、i、SST 等。

8--表示该芯片为 8051 内核芯片。

9--表示内部含有 Flash EEPROM 存储器，还有如 80C51 中 0 表内部含有 MaskROM（掩模 ROM）存储器；如 87C51 中 7 表示内部含有 EPROM（紫外线可擦除 ROM）存储器。

C--表示该器件为 CMOS 产品。还有如 89LV52 和 89LE58 中的 LV 和 LE 都表示该芯片为低电压产品（通常为 3.3V 电压供电）；而 89S52 中 S 表示该芯片含有可串行下载功能的 Flash 存储器，即具有 ISP 可在线编程功能。

5--固定不变。

2--表示该芯片内部程序存储（FLASH）空间大小，1 为 4KB，2 为 8KB，3 为 12KB，即该数乘以 4KB 就是芯片内部的程序存储空间大小。程序空间大小决定了一个芯片所能装入执行代码的多少。一般来说，程序存储空间越大，芯片价格也越高，所以我们再选择芯片的时候要根据自己需求选择合适芯片。

RC--STC 单片机内部 RAM（随机读写存储器）为 512B。还有如 RD+表示内部 RAM 为 1280B。还有芯片会省略此部分。

40--表示芯片外部晶振最高可接入 40MHz。对 AT 单片机数值一般为 24，表示外部晶振最高位 24MHz。

I--产品级别，表示芯片使用温度范围。

C 表示商业级，温度范围为 0~+70 度。

I 表示工业级，温度范围为-40~+85 度。

A 表示汽车级，温度范围为-40~+125 度。

M 表示军用级，温度范围为-55~+150 度。

PDIP40--产品封装型号。PDIP 表示双列直插式。

1947--表示本批芯片生产日期为 19 年第 47 周。

HOXN52.X90C--不详，有关资料显示，此标号表示芯片制造工艺或处理工艺。

4.2.1 STC89C51 芯片简介

STC89C51 是 STC 推出的新一代超强抗干扰/高速/低功耗的 51 单片机，它是采用 8051 核的 ISP（In System Programming）在系统可编程芯片，指令代码完全兼容传统的 8051 单片机，最高工作时钟频率为 80MHz，片内含 4K Bytes 的可反复擦写 1000 次的 Flash 只读程序存储器，器件兼容标准 MCS-51 指令系统及 80C51 引脚结构，芯片内集成了通用 8 位中央处理器和 ISP Flash 存储单元，具有在系统可编程（ISP）特性，配合 PC 端的控制程序即可将用户的程序代码下载进单片机内部，省去了购买通用编程器，而且速度更快。STC89C51 系列单片机是单时钟/机器周期(1T)的兼容 8051 内核单片机，是高速/ 低功耗的新一代 8051 单片机，全新的流水线/精简指令集结构,内部集成 MAX810 专用复位电路。其特性功能主要如下：

1：增强型 8051 单片机，6 时钟/机器周期和 12 时钟/机器周期可任意选择，

指令代码完全兼容传统 8051。

2：工作电压： 5.5V - 3.3V（5V 单片机） / 3.6V - 2.0V（3V 单片机）

3：工作频率范围： 0～40MHz，相当于普通 8051 的 0～80MHz，实际工作频率可达 48MHz

4：用户应用程序空间： 4K / 8K / 13K / 16K / 32K / 64K 字节（STC89C516）

5：片上集成 1280 字节或 512 字节或 256 字节 RAM

6：通用 I/O 口(35/39 个)， 复位后为：P1/P2/P3/P4 是准双向口/弱上拉(普通 8051 传统 I/O 口)； P0 口是开漏输出，作为总线扩展用时，不用加上拉电阻，作为 I/O 口用时，需加上拉电阻。

7：ISP（在系统可编程） / IAP（在应用可编程）， 无需专用编程器， 无需专用仿真器，可通过串口（ RxD/P3.0, TxD/P3.1）直接下载用户程序，数秒即可完成一片。

8：有 EEPROM 功能

9：看门狗

10：内部集成 MAX810 专用复位电路（HD 版本和 90C 版本才有），外部晶体 20M 以下时，可省外部复位电路,复位脚可直接接地。

11：有 3 个 16 位定时器/ 计数器，其中定时器 0 还可以当成 2 个 8 位定时器使用。

12：外部中断 4 路,下降沿中断或低电平触发中断,Power Down 模式可由外部中断低电平触发中断方式唤醒

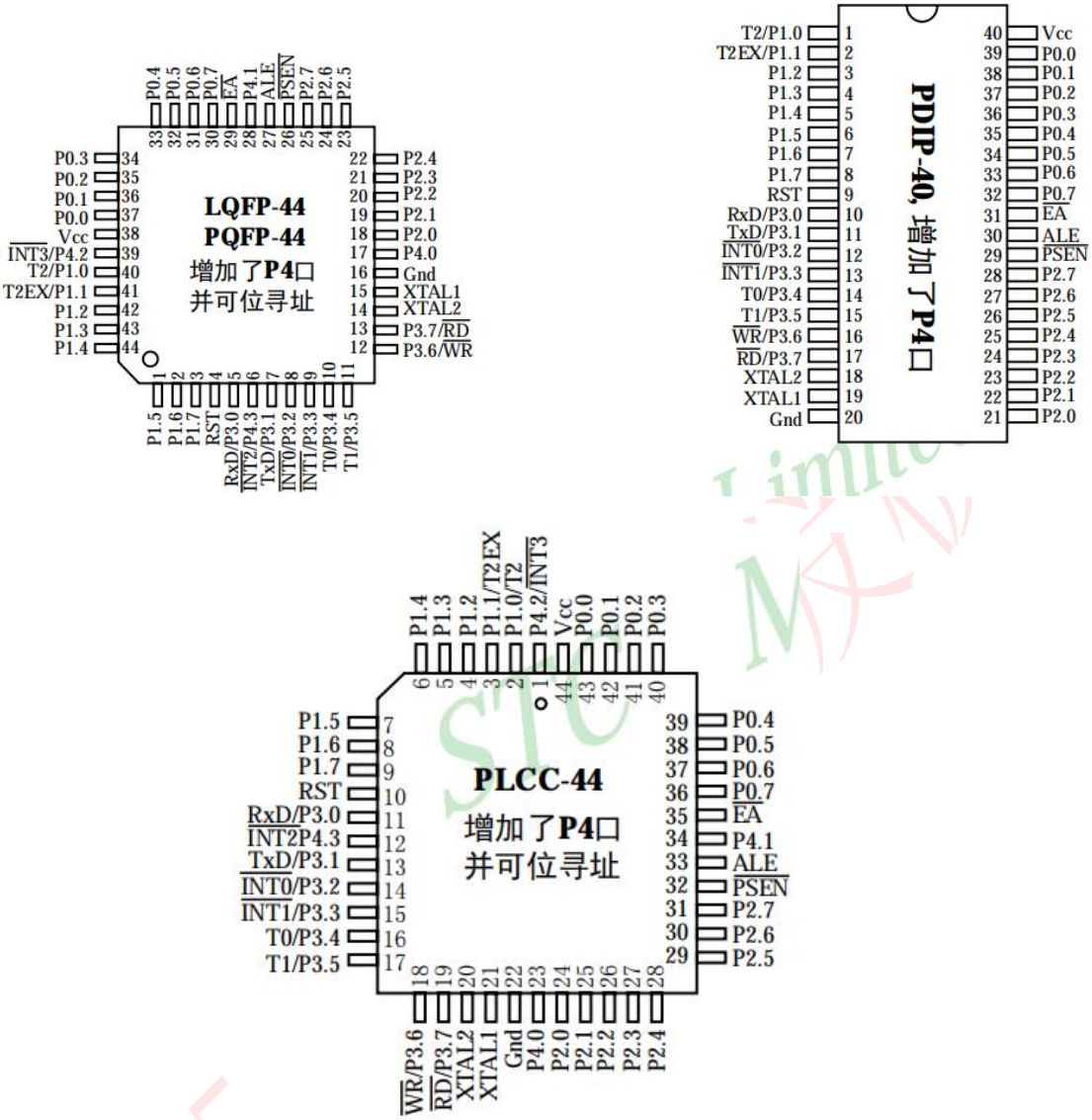
13：通用异步串行口(UART)，还可用定时器软件实现多个 UART

14：工作温度范围： -40 ~ +85℃(工业级) / 0 ~ 75℃(商业级)

15：封装： LQFP-44, PDIP-40, PLCC-44, PQFP-44。

4.2.2 STC89C51 芯片引脚介绍

下图我们可以看到 51 单片机不同封装的引脚图，当大家首次看见这些引脚时，一定会有又多又乱的感觉，而且难以记忆。千万不要着急，对于初学者来说，单纯的记忆引脚标号没有任何意义，最好的方法是边学边记忆。

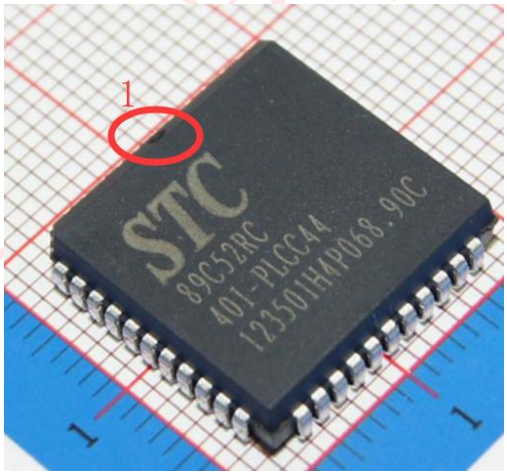


4.2.2.1 单片机脚位判断

在在讲解各引脚含义之前，我们首先应该学会如何在实物上区分引脚序号，基于 8051 内核的单片机，若引脚数相同，或封装相同，它们的引脚功能是相同的，其中用的较多的是 40 脚 DIP 封装的 51 单片机，也有 20,28,32，44 等不同引脚数的 51 单片机，这些大家也要了解，不能只见了 40 脚的芯片才认为它是 51 单片机。

无论哪种芯片，单片机也好，其他不知名的芯片也好，当我们观察它的表面时，大都会找到一个凹进去的小圆坑，或是用颜色标记的一个小标记（圆点或三角或其他小图形），这个小圆坑或者小标记所对应的引脚就是这个芯片的第 1

脚，然后逆时针方向数下去，即 1 到最后一个引脚。如下图所示：



上图中对于 LQFP/PQFP 封装，小圆坑在左下角位置；对于 DIP 封装，小圆坑在芯片上方正中间位置，对应左边第一个脚位为 1；对于 PLCC 封装，小圆坑在芯片上方正中间位置，对应脚位为 1；在实际焊接或绘制电路板时，大家务必要注意它们的脚位标号，否则若焊接错误则导致产品无法正常工作。

4.2.2.2 51 单片机脚位功能

接下来我们以上图 PDIP 封装引脚图为例介绍各个引脚的功能，40 引脚我们按其功能类别可分为四类：

①电源引脚。如 VCC、GND

②时钟引脚。如 XTAL1、XTAL2

③编程控制引脚。如 RST、PSEN、ALE/PROG、EA/Vpp。（此处了解即可）

④I/O 口引脚。如 P0、P1、P2、P3，4 组 8 位 I/O 口。

VCC（40 脚）、GND（20 脚）：电源引脚，不同型号单片机接入对应电压，常压为+5V，低压为+3.3V，大家在使用时要查看其芯片所要求的电压。

XTAL1（19 脚）、XTAL2（18 脚）：外接时钟引脚。XTAL1 为片内振荡电路的输入端，XTAL2 为片内振荡电路的输出端。8051 的时钟有两种方式，一种是片内时钟振荡方式，即需在这两个引脚处外接石英晶振和振荡电容，振荡电容的值一般取 10p~30p；另一种是外部时钟方式，即将 XTAL1 接地，外部时钟信号从 XTAL2 脚输入。通常使用第一种方式。

RST（9 脚）：复位引脚。当输入连续两个机器周期以上高电平时为有效，用来完成单片机的复位初始化操作，即单片机从头开始执行程序。

PSEN（29 脚）：程序存储器允许输出控制端。在读外部程序存储器时 PSEN 低电平有效，以实现外部程序存储器单元的读操作，由于现在我们使用的单片机内部已经有足够大的 ROM，所以几乎没有人再去扩展外部 ROM，因此这个引脚大家只需了解即可。

ALE/PROG（30 脚）：在扩展外部 RAM 时，ALE 用于控制把 P0 口的输出低 8 位地址送锁存器锁存起来，以实现低位地址和数据的隔离。ALE 有可能是高电平也有可能是低电平，当 ALE 为高电平时，允许地址锁存型号，当访问外部存储器时，ALE 信号负跳变（即由正变负）将 P0 口上低 8 位地址信号送入锁存器；当 ALE 为高电平时，P0 口上的内容和锁存器输出一致。关于锁存器后面我们会有介绍。在没有访问外部存储器期间，ALE 以 1/6 振荡周期频率输出（即 6 分频），当访问外部存储器时，以 1/12 振荡周期输出（12 分频）。从这里可以看到，当没有扩展外部 RAM 时，ALE 会以 1/6 振荡周期的固定频率输出，因此可以作为外

部时钟，或作为外部定时脉冲使用。PROG 为编程脉冲的输入端，单片机的内部有程序存储器（ROM），它的作用是用来存放用户需要执行的程序，那么我们怎样才能将写好的程序存入这个 ROM 中呢？实际上，我们是通过编程脉冲输入才写进去的，这个脉冲的输入端口就是 PROG。现在绝大多数单片机都已经不需要编程脉冲引脚往内部写程序了，比如我们使用的 STC 单片机，它可以直接通过串口往里面写程序，只需要三条线与计算机相连即可。而且现在的单片机内部都已经带有丰富的 RAM，所以也不需要再扩展 RAM 了，因此 ALE/PROG 引脚用于不大，大家了解即可。

EA/Vpp（31 脚）：EA 接高电平时，单片机读取内部程序存储器。当扩展有外部 ROM 时，当读取完内部 ROM 后自动读取外部 ROM。EA 接低电平时，单片机直接读取外部 ROM。我们没有外扩 ROM，并且需要单片机直接读取内部程序存储器，因此 EA/Vpp 脚直接接高电平。

P0 口（39 脚~32 脚）：双向 8 位三态 I/O 口，每个口可独立控制。51 单片机 P0 口内部没有上拉电阻，若输出高时为高阻态，不能正常输出高电平，因此该组 I/O 口，每个口可独立控制。51 单片机 P0 口内部没有上拉电阻，若输出高时为高阻态，不能正常输出高电平，因此该组 I/O 口在使用时务必要外接上拉电阻，一般我们选择接入 10K 欧上拉电阻。

P1 口（1 脚~8 脚）：准双向 8 位 I/O 口，每个口可独立控制，内部自带上拉电阻，这种接口输出没有高阻态，输入也不能锁存，故不是真正的双向 I/O 口。之所以称它为“准双向”是因为该口在作为输入使用前，要先向该口进行写 1 操作，然后单片机内部才可正确读出外部信号，也就是要使其先有个“准”备的过程，所以才称为准双向口。对 52 单片机 P1.0 引脚的第二功能未 T2 定时器/计数器的外部输入，P1.1 引脚的第二功能为 T2EX 捕捉、重装触发，即 T2 的外部控制端。

P2 口（21 脚~28 脚）：准双向 8 位 I/O 口，每个口可独立控制，内部自带上拉电阻，与 P1 口相似。

P3 口（10 脚~17 脚）：准双向 8 位 I/O 口，每个口可独立控制，内部自带上拉电阻。作为第一功能使用时就当做普通 I/O 口，与 P1 口相似。作为第二功能使用时，各引脚的定义如下：

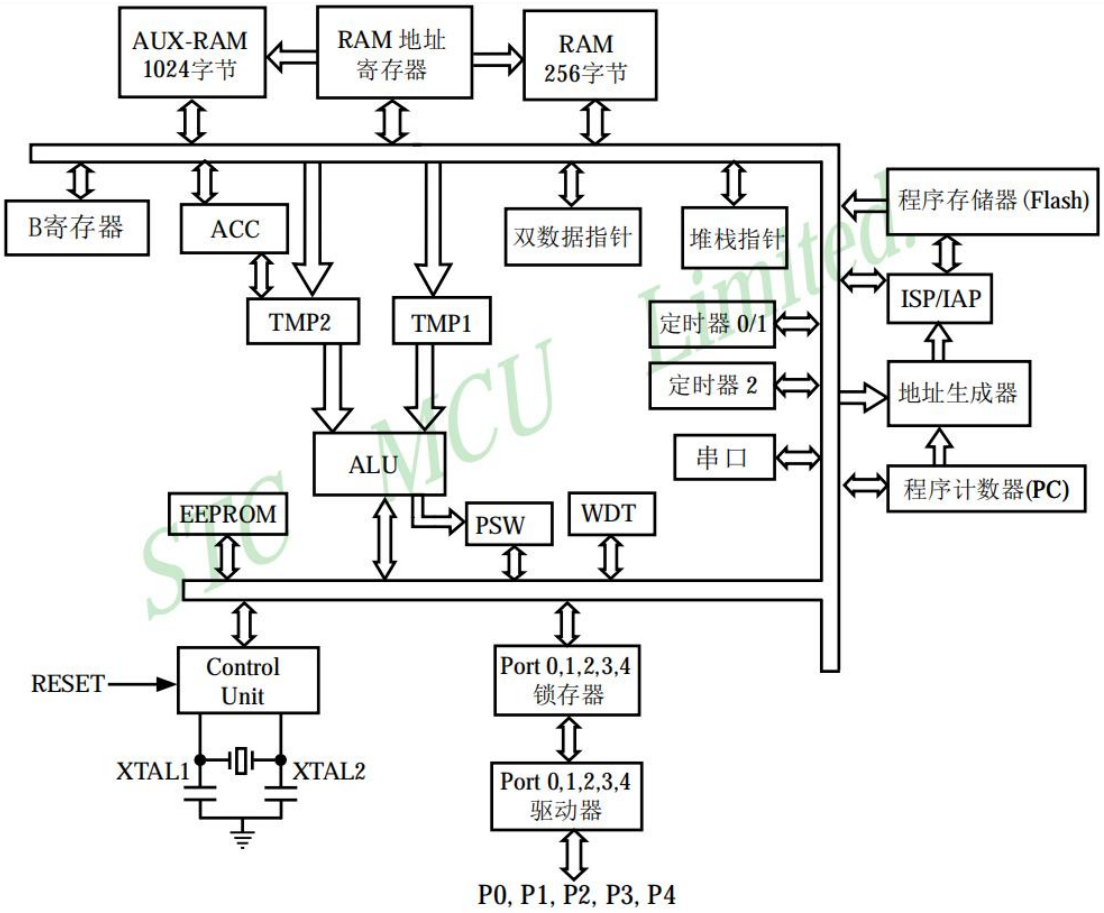
P3.0/RxD	10	P3.0	标准I/O口 PORT3[0]
		RxD	串口1数据接收端
P3.1/TxD	11	P3.1	标准I/O口 PORT3[1]
		TxD	串口1数据发送端
P3.2/$\overline{INT0}$	12	P3.2	标准I/O口 PORT3[2]
		$\overline{INT0}$	外部中断0, 下降沿中断或低电平中断
P3.3/$\overline{INT1}$	13	P3.3	标准I/O口 PORT3[3]
		$\overline{INT1}$	外部中断1, 下降沿中断或低电平中断
P3.4/T0	14	P3.4	标准I/O口 PORT3[4]
		T0	定时器/计数器0的外部输入
P3.5/T1	15	P3.5	标准I/O口 PORT3[5]
		T1	定时器/计数器1的外部输入
P3.6/$\overline{WR}$	16	P3.6	标准I/O口 PORT3[6]
		$\overline{WR}$	外部数据存储器写脉冲
P3.7/$\overline{RD}$	17	P3.7	标准I/O口 PORT3[7]
		$\overline{RD}$	外部数据存储器读脉冲

从图中可知，P3 口的每一个引脚均可独立定义为第一功能的输入/输出或第二功能。

4.2.3 STC89C51 芯片内部结构

STC89C51 系列单片机的内部结构框图如下图所示：

普中51单片机



STC89C51 单片机中包含中央处理器(CPU)、程序存储器(Flash)、数据存储器(SRAM)、定时/计数器、UART 串口、I/O 接口、EEPROM、看门狗等模块。STC89C51 系列单片机几乎包含了数据采集和控制中所需的所有单元模块，可称得上一个片上系统。

想要了解更多 STC89C5xx 系列芯片介绍，可以百度搜索对应数据手册。在我们资料内也提供了该芯片的数据手册，可作为 STC89C5XX 单片机学习参考。

4.3 51 单片机能做什么

单片机说白了，就是一种微型的电脑。只要我们稍加编程，再加上一系列的外围电子设备，就可以发挥强大的功能，接下来给大家简单介绍一下，51 单片机能干什么？

51 单片机主要用于“控制”，通过串口可以和 WFIF/GPS/蓝牙等模块实现无线控制，通过 AD 接口可以采集光敏/烟雾传感器/可燃气体传感器等模拟信号，还可以使用 AD 设计简易示波器。当然还包括直流电机、交流电机、步进电机、

伺服电机、变频电机、电磁铁、电磁阀、LED、LCD 等等，进而由这些去驱动各种设备，应用于家电、机械加工、制造、航空航天等各行各业。

日常生活中可采用 51 单片机设计的电子产品：

智能手环，微型四轴飞行器，平衡车、扫地机、移动 POST 机，智能电饭锅，3D 打印机、机器人等。

总之学好 51 单片机用处非常大，而且学好 51 单片机对后期学习 STM32 或者其他高级单片机帮助是非常大的。在众多 51 芯片中，STC89CXX 或者 STC90CXX 系列芯片是首选。大家只要认真学习本套教程，相信学好 51 并不难！

4.4 怎样学好 51 单片机

（1）基本外设

GPIO 输入输出，外部中断，定时器，串口。理解了这四个外设，基本入门了一款 MCU。

（2）掌握基础的数字电路和模拟电路知识

51 单片机开发中，涉及的电路并不复杂，对于初学者来说只需要一些基础的数电和模电知识即可。比如：二极管导通特性、三极管工作原理等。至于数字芯片，只需学会看对应的数据手册即可。这些在我们后面章节也都会一一介绍。

（3）C 语言能力要加强

C 语言是嵌入式开发基础中的基础。如果 C 语言不过关，大大限制嵌入式学习进度，以及嵌入式学习深度。 嵌入式高手都是 C 高手。选择谭浩强的《 C 程序设计 第四版》 入门及巩固 C 语言基础是不错的，对于 51 单片机的简单开发来说，掌握 C 语言基础部分即可，它对 C 语言功底要求不是很高。但如果想使用 STM32 这类高端单片机开发项目，仅靠一点 C 语言基础知识是不够的，可以参考《 C 与指针》、《 C 指针编程之道》等。我们已经在资料“ \8--51 相关资料\C 语言加油站”内提供给大家。

（4）多动手编程

请一定要记住：编程能力是练出来的，不是看出来的。很多初学者在看视频时，编写的程序都能看懂，等到脱离视频自己动手编写时却发现不知道怎么去写，而且编写的一些语句有错误，比如字母大小写，关键字写错等问题。

（5）遇到问题多百度多浏览论坛

在学习过程中不可能是一帆风顺的，会经常遇到各种各样的问题，有问题了先要学会自己去思考解决，可以多百度或者去一些比较大的技术论坛交流。不要一遇到问题就马上联系售后技术，即使技术帮你解决了，过段时间可能你又忘了，对你的技术成长没有任何帮助。这里推荐几个论坛，大家可以在里面相互交流。

普中科技论坛：www.prechin.net

51 单片机论坛：http://bbs.21ic.com/icfilter-typeid-11-211.html

课后作业

普中51单片机开发板

第 5 章 数字电路与 C 语言基础

在前面章节，我们已经介绍过 51 单片机的学习方法，其中非常重要的一点是要具备一定的数字电路和 C 语言基础知识。考虑到部分初学者的专业问题，可能未学习过数字电路或 C 语言，因此很有必要为大家介绍数字电路和 C 语言基础知识，由于 C51 和 C 几乎类似，因此不去划分 C51 和 C，而统一为 C。如果有 C 语言基础的朋友可以绕过本章内容，当然还是推荐大家看下，就当做是复习。通过本章的学习，让大家掌握 C 语言基础知识，为后面 51 单片机的编程学习做好铺垫。本章分为如下几部分内容：

5.1 电平特性

5.2 二进制与十六进制

5.3 二进制的逻辑运算

5.4 C51 基础知识

普中51单片机开发板

5.1 电平特性

单片机是一种数字集成芯片，数字电路中只有两种电平：高电平和低电平。为了让大家在刚起步的时候对电平特性有一个清晰的认识，我们暂时定义单片机输出与输入为 TTL 电平，其中高电平为+5V，低电平为 0V。计算机的串口为 RS232 电平，其中高电平为-12V，低电平为+12V。这里强调的是，RS232C 电平为负逻辑电平，大家不要认为上面是我写错了。因此当计算机与单片机之间要通信时，需要依靠电平转换芯片，比如 MAX232 电平转换芯片。

常用的逻辑电平还有很多，比如 TTL、CMOS、LVTTL、RS-232、RS-485 等。其中 TTL 和 CMOS 的逻辑电平按典型电压可分为四类：5V 系列（5V TTL 和 5V CMOS）、3.3V 系列，2.5V 系列和 1.8V 系列。

5V TTL 和 5V CMMOS 是通用的逻辑电平。3.3V 及以下的逻辑电平被称为低电压逻辑电平，常用的为 LVTTL 电平。低电压逻辑电平还有 2.5V 和 1.8V 两种。RS-232 和 RS-485 是串口的接口标准，RS-232 是单端输入/输出。RS-485 是差分输入/输出。

TTL 电平信号用的最多，这是因为数据表示通常采用二进制，+5V 等价于逻辑 1，0V 等价于逻辑 0，这被称为 TTL（晶体管-晶体管逻辑电平）信号系统。

CMOS 电平 VCC 可达 12V，CMOS 电路输出高电平约为 0.9VCC，而输出低电平约为 0.1VCC。CMOS 电路中不使用的输入端不能悬空，否则会造成逻辑混乱。另外，CMOS 集成电路电源电压可以在较大范围内变化，因而对电源的要求不像 TTL 集成电路那样严格。

TTL 电路和 CMOS 电路的逻辑电平关系如下：

①VOH：逻辑电平 1 的输出电压。

②VOL：逻辑电平 0 的输出电压。

③VIH：逻辑电平 1 的输入电压。

④VIL：逻辑电平 0 的输入电压。

TTL 电平临界值：

①VOHmin=2.4V，VOLmax=0.4V。

②VIHmin=2.0V，VILmax=0.8V。

CMOS 电平临界值（假设电源电压为+5V）：

①VOHmin=4.99V，VOLmax=0.01V。

②VIHmin=3.5V，VILmax=1.5V。

TTL 和 CMOS 的逻辑电平转换：CMOS 电平能驱动 TTL 电平，但 TTL 电平不能驱动 CMOS 电平，需加上拉电阻。

常用逻辑芯片的特点如下：

74LS 系列：　TTL　输入：TTL，　输出：TTL

74HC 系列：　CMOS　输入：CMOS，　输出：CMOS

74HCT 系列：　CMOS　输入：TTL，　输出：CMOS

CD4000 系列：　CMOS　输入：TTL，　输出：CMOS

通常情况下，单片机、DSP、FPGA 之间引脚能否直接相连要参考以下方法进行判断：一般来说，同电压的是可以相连的，不过最好还是要查看下芯片技术手册上的 VIL、VIH、VOL 和 VOH 的值，看是否能够匹配。有些情况在一般应用中没有问题，但是参数上就是有点不够匹配，在某些情况下运行可能就不够稳定，或者不同批次的器件就不能运行。

5.2 二进制与十六进制

5.2.1 二进制

数字电路中只有两种电平特性，即高电平和低电平，这也就决定了数字电路中使用二进制。十进制数大家应该都不陌生，“逢十进一，借一当十”是十进制数的特点。有了十进制数的基础，我们学习二进制数便非常容易了，“逢二进一，借一当二”便是二进制数的特点。十进制数 1 转换为二进制数是 1B（这里 B 表示二进制数的后缀）；十进制数 2 转换为二进制数时，因为已经到 2，所以需要进 1，那么二进制数即为 10B；十进制数 5 转换为二进制数，2 为 10B，那么 3 即为 10B+1B=11B，4 即为 11B+1B=100B，5 即为 100B+1B=101B。依次类推，当十进制数为 254 时，对应而=二进制数为 1111 1110B（**中间空一格是为了方便阅读，**

实际编写不要空格）。

我们可以找出一般规律，当二进制数转换为十进制数时，从二进制数的最后一位往前看，每一位代表的数为 2 的 n 次幂，这里的 n 表示从最后起的第几位二进制数，n 从 0 算起，若对应该二进制数位上有 1，那么就有值，为 0 则无值。例如，再把二进制数 1111 1110B 反推回十进制数，计算过程如下：

$0*2^0+1*2^1+1*2^2+1*2^3+1*2^4+1*2^5+1*2^6+1*2^7=254$

其中 2^n 称为“位权”。对于十进制与二进制之间的转换，我们能熟练掌握 0-15 以内的数就够用了，为了方便记忆，我们归纳如下：

十进制	二进制
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

实际开发中常常会用到其它比较大的数，这时我们用 Windows 系统自带的计算器，选择程序员模式，可以非常方便的进行二进制、八进制、十进制、十六进制数之间的任意转换，如下图所示：

普中51单片机开发板

5.2.2 十六进制

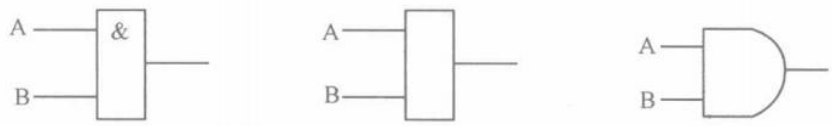
十六进制与二进制大同小异，不同之处就是十六进制是“逢十六进一，借一当十六”。还有一点特别之处需要注意，十进制的 0-15 表示成十六进制为 0~9、A、B、C、D、E、F，即十进制的 10 对应十六进制的 A，11 对应 B，以此类推。我们一般在十六进制数的最后面加上后缀 H，表示该数为十六进制数，如 AH、DEH 等。这里的字母不区分大小写，在 C 语言编程时要写成“0xa，0xde”，在数的最前面加上“0x”表示该数为十六进制数。十进制数与十六进制数之间的转换在这里不再讲解，大家可参考十进制与二进制数之间的转换规则。关于十进制、二进制与十六进制数之间的转换，我们要熟练掌握 0~15 之间的数，因为在以后的单片机 C 语言编程中，我们要大量使用它们。一般的转换规律是，先将二进制数转换为十进制数，再将十进制数转换为十六进制数，这里不用死机，在后面的学习中遇到多了也就记住了。二进制、十进制、十六进制 0~15 的数的转换关系如下：

十进制	二进制	十六进制	十进制	二进制	十六进制
0	0	0	8	1000	8
1	1	1	9	1001	9
2	10	2	10	1010	A
3	11	3	11	1011	B
4	100	4	12	1100	C
5	101	5	13	1101	D
6	110	6	14	1110	E
7	111	7	15	1111	F

5.3 二进制的逻辑运算

5.3.1 与运算

“与”运算是实现“必须都有，否则就没有”这种逻辑关系的一种运算。C 语言中运算符为“&”，其运算规则如下：0&0=0，0&1=0（1&0=0），1&1=1。其运算符号如下图所示：

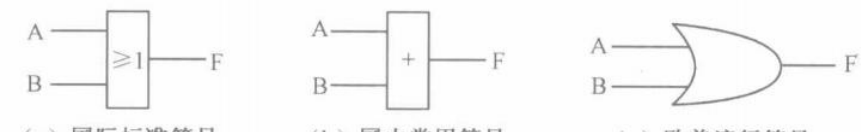


（a）国际标准符号　（b）国内常用符号　（c）欧美流行符号

C 语言中“&&”表示“按位与”运算，意思是变量之间按二进制位数对应关系一一进行“与”运算。如（0101 0101）&&（1010 1010）=0000 0000，而上面讲到的“&”运算只是对单一位进行运算。

5.3.2 或运算

“或”运算是实现“只要其中之一有就有”这种逻辑关系的一种运算。C 语言中运算符为“|”，其运算规则如下：0|0=0，0|1=1（1|0=1），1|1=1。其运算符号如下：



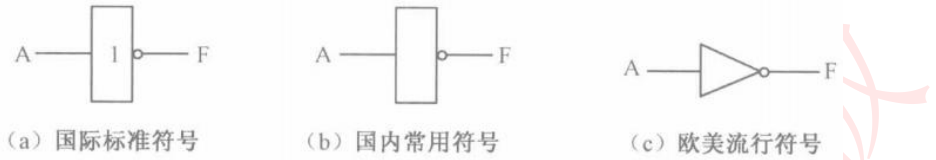
（a）国际标准符号　（b）国内常用符号　（c）欧美流行符号

C 语言中“||”表示“按位或”运算，意思是变量之间按二进制位数对应关

系一一进行“或”运算。如（0101 0101）||（1010 1010）=1111 1111，而上面讲到的“|”运算只是对单一位进行运算。

5.3.3 非运算

“非”运算是实现“求反”这种逻辑关系的一种运算。C 语言中运算符为“!”，其运算规则如下：!0=1，!1=0。其运算符号如下图所示：



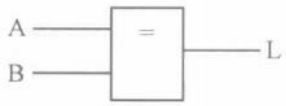
（a）国际标准符号　（b）国内常用符号　（c）欧美流行符号

C 语言中“~”表示“按位取反”运算。如~0101 0101=1010 1010，而上面讲到的“!”运算只是对单一位进行运算。

5.3.4 同或运算

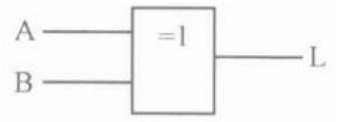
“同或”与“异或”运算使用的较少，我们在这里只做简单了解，大家用到之处可再查找相关资料。“同或”运算是实现“必须相同，否则就没有”这种逻辑关系的一种运算，其逻辑运算符为“⊙”。其运算规则如下：

0⊙0=1，0⊙1=0（1⊙0=0），1⊙1=1。在 C 语言中没有规定符号。其运算符号如下：



5.3.5 异或运算

“异或”运算是实现“必须不同，否则就没有”这种逻辑关系的一种运算，其逻辑运算符为“⊕”。其运算规则如下：0⊕0=0，0⊕1=1（1⊕0=1），1⊕1=0。在 C 语言中有“按位异或”运算“^”。其运算符号如下：



至此，数字电路的基础知识我们就讲解完，如果对上述知识还不了解，可以在网上搜索相关数字电路教程，自行查看。下面我们重点看下 C 语言知识。

5.4 C51 基础知识

5.4.1 C 语言简介

C 语言是在 70 年代初问世的。一九七八年由美国电话电报公司(AT&T)贝尔实验室正式发表了 C 语言。同时由 B.W.Kernighan 和 D.M.Ritchit 合著了著名的“THE C PROGRAMMING LANGUAGE”一书。通常简称为《K&R》，也有人称之为《K&R》标准。但是，在《K&R》中并没有定义一个完整的标准 C 语言，后来由美国国家标准协会（American National Standards Institute）在此基础上制定了一个 C 语言标准，于一九八三年发表。通常称之为 ANSI C。

早期的 C 语言主要是用于 UNIX 系统。由于 C 语言的强大功能和各方面的优点逐渐为人们认识，到了八十年代，C 开始进入其它操作系统，并很快在各类大、中、小和微型计算机上得到了广泛的使用，成为当代最优秀的程序设计语言之一。

C 语言作为一种非常方便的语言而得到广泛的支持，很多硬件开发都用 C 语言编程，如各种单片机、DSP、ARM 等。C 语言程序本身不依赖于机器硬件系统，基本上不做修改或仅做简单的修改就可以将程序从不同的系统移植过来直接使用。C 语言提供了很多数学函数并支持浮点运算，开发效率高，可极大的缩短开发周期，增加程序可读性和可维护性。单片机的 C51 编程与汇编 ASM-51 编程相比，有如下优点：

①对单片机的指令系统不要求有任何的了解，就可以用 C 语言直接编程操作单片机。

②寄存器分配、不同存储器的寻址及数据类型等细节完全有编译器自动管理。

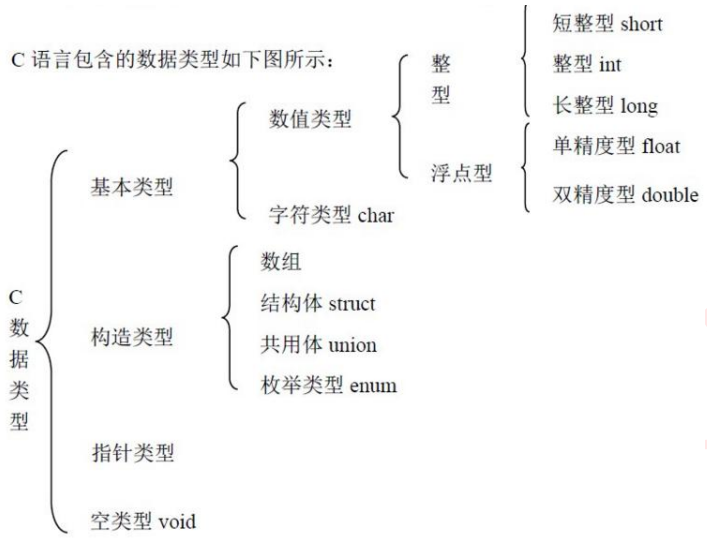
③程序有规范的结构，可分成不同的函数，可使程序结构化。

④库中包含许多标准子程序，具有较强的数据处理能力，使用方便。

⑤具有方便的模块化编程技术，大大提供程序的可移植性。

5.4.2 C51 数据类型

C 语言包含的数据类型如下图所示：



5.4.2.1 C51 基本数据类型

很多初学者对数据类型是什么东西搞不明白，我们举个简单例子帮助大家更好理解。假设 X=10，Y=I，Z=X+Y，求 Z=?在这个例子当中，我们将 10 和 I 分别赋给 X 和 Y，再将 X+Y 赋给 Z，由于 10 已经固定，我们称 X 为“常量”；由于 Y 的值随 I 值的变化而变化，Z 的值随 X+Y 值的变化而变化，我们称 Y 和 Z 为“变量”，本题中 X 的值为 10，而 Y 的值为 I，但 I 的值是不确定的，有可能是 10000，Y 的值也就有可能是别的数了。在我们日常计算时，X 和 Y 的值我们可以赋给它任意大小的数，当我们给单片机编程时，单片机也要运算，而在单片机的运算中，这个“变量”数据的大小是有限制的，我们不能随意给一个变量赋任意的值，因为变量在单片机的内存中是要占据空间的，变量大小不同，所占据的空间就不同，为了合理利用单片机内存空间，我们在编程时就要设定合适的数据类型，不同的数据类型也就代表了十进制中不同的数据大小，所以我们在设定一个变量之前，必须要给编译器声明这个变量的类型，以便让编译器提前从单片机内存中分配给这个变量合适的空间。单片机的 C 语言中常用的基本数据类型如下：

数据类型	关键字	所占位数	表示数的范围
无符号字符型	unsigned char	8	0~255
有符号字符型	char	8	-128~127
无符号整型	unsigned int	16	0~65535
有符号整型	int	16	-32768~32767
无符号长整型	unsigned long	32	$0\sim2^{32}-1$
有符号长整型	long	32	$-2^{31}\sim2^{31}-1$
单精度浮点型	float	32	3.4e-38~3.4e38
双精度浮点型	double	64	1.7e-308~1.7e308

大家在 C 语言的书籍上还能看到有 short int，long int，signed short int 等数据类型，在单片机的 C 语言中我们默认的规则如下：short int 即为 int，long int 即为 long，前面若无 unsigned 符号则一律认为是 signed 型。

有同学可能对上图中所占位数不理解，关于所占位数的解释：在编写程序时，无论是以十进制、十六进制还是二进制表示的数，在单片机中，所有的数据都是以二进制形式存储在存储器中的，既然是二进制，那么就只有两个数，0 和 1，这两个数每一个所占的空间就是 1 位（b），位也是单片机存储器中最小的单位。比位大的单位是字节（B），1 个字节等于 8 位（即 1B=8b 或 1Byte=8bit）。我们从上图中可以看出，字符型占用存储空间最小，为 8 位，双精度浮点型最大，为 64 位。其中 float 型和 double 型是用来表示浮点数的，也就是带小数点的数，如 12.345,0.213 等。在这里要说明的是，在一般的系统中，float 型数据只能提供 7 位有效数字，double 型数据能够提供 15~16 位有效数字，但是这个精度还和编译器有关系，并不是所有的编译器都准守这条原则。当把一个 double 型变量赋给 float 型变量时，系统会截取相应的有效位数，例如：

```
float a; //定义一个 float 型变量
a=123.1234567;
```

由于 float 型变量只能接受 7 位有效数字，因此最后 3 位小数将会被四舍五入截掉，即实际 a 的值将是 123.1235。若将 a 改成 double 型变量，则能全部接收上述 10 位数字并存储在变量 a 中。

5.4.2.2 C51 扩充数据类型

单片机内部有很多的特殊功能寄存器，每个寄存器在单片机内部都分配有唯一的地址，一般我们会根据寄存器功能不同给寄存器赋予各自的名称，当我们需

要在程序中操作这些特殊功能寄存器时，必须要在程序的最前面将这些名称加以声明，声明的过程实际就是将这个寄存器在内存中的地址编号赋给这个名称，这样编译器在以后的程序中才可认识这些名称所对应的寄存器。对于大多数初学者来说，这些寄存器的声明已经完全被包含在 51 单片机的特殊功能寄存器声明头文件“reg51.h”中了，初学者若不想深入了解，完全可以暂不操作它。C51 扩充数据类型主要如下：

类 型	长 度	值 域	说明
bit	位	**0 或 1**	位变量声明
sbit	位	**0 或 1**	特殊功能位声明
sfr	**8 位=1 字节**	**0~255**	特殊功能寄存器声明
Sfr16	**16 位=2 字节**	**0~65535**	**Srf 的 16 位数据声明**
*	**1~3 字节**		对象的地址

例如：sfr SCON=0x98;

SCON 是单片机的串行口控制寄存器，这个寄存器在单片机内存中的地址是 0X98。这样声明后，我们再以后要操作这个控制寄存器时，就可以直接对 SCON 进行操作，这时编译器也会明白，我们实际要操作的是单片机内部 0X98 地址处的这个寄存器，而 SCON 仅仅是这个地址的一个代号或是名称而已，当然，我们也可以定义成其他的名称。

例如：sfr16 T2=0xCC;

声明一个 16 位的特殊功能寄存器，它的起始地址为 0XCC。

例如：sbit TI=SCONˆ1;

SCON 是一个 8 位寄存器，SCONˆ1 表示这个 8 位寄存器的次低位，最低位是 SCONˆ0;SCONˆ7 表示这个寄存器的最高位。该语句的功能就是将 SCON 寄存器的次低位声明为 TI，以后若要对 SCON 寄存器的次低位操作，则可直接操作 TI。

在 C51 语言程序中，有可能会出现在运算中数据类型不一致的情况。C51 允许任何标准数据类型的隐式转换，隐式转换的优先级顺序如下：

bit→char→int→long→float→signed→unsigned

也就是说，当 char 型与 int 型进行运算时，先自动对 char 型扩展为 int 型，然后与 int 型进行运算，运算结果为 int 型。C51 除了支持隐式类型转换外，还可以通过强制类型转换符“（）”对数据类型进行人为的强制转换。

C51 编译器除了能支持以上这些基本数据类型之外，还能支持一些复杂的组

合型数据类型，如数组类型、指针类型、结构类型、联合类型等这些复杂的数据类型。对于初学者我们要求先掌握 C 语言基础知识，把基础的掌握了在学习复杂的。

5.4.3 C51 运算量

5.4.3.1 常量

常量是指在程序执行过程中其值不能改变的量。在 C51 中支持整型常量、浮点型常量、字符型常量和字符串型常量。

1，整型常量

整型常量也就是整型常数，根据其值范围在计算机中分配不同的字节数来存放。在 C51 中它可以表示成以下几种形式：

十进制整数。如 234、-56、0 等。

十六进制整数。以 0x 开头表示，如 0x12 表示十六进制数 12H。

长整数。在 C51 中当一个整数的值达到长整型的范围，则该数按长整型存放，在存储器中占四个字节，另外，如一个整数后面加一个字母 L，这个数在存储器中也按长整型存放。如 123L 在存储器中占四个字节。

2，浮点型常量

浮点型常量也就是实型常数。有十进制表示形式和指数表示形式。十进制表示形式又称定点表示形式，由数字和小数点组成。如 0.123、34.645 等都是十进制数表示形式的浮点型常量。指数表示形式为： [] 数字 [.数字] e []数字

例如：123.456e-3、-3.123e2 等都是指数形式的浮点型常量。

3，字符型常量

字符型常量是用单引号引起的字符，如‘a’、‘1’、‘F’等。可以是可显示的 ASCII 字符，也可以是不可显示的控制字符。对不可显示的控制字符须在前面加上反斜杠“\”组成转义字符。利用它可以完成一些特殊功能和输出时的格式控制。常用的转义字符如下表所示。

转义字符	含　义	ASCII 码（十六进制数）

\o	空字符（null）	00H
\n	换行符（LF）	0AH
\r	回车符（CR）	0DH
\t	水平制表符（HT）	09H
\b	退格符（BS）	08H
\f	换页符（FF）	0CH
\‘	单引号	27H
\”	双引号	22H
\\	反斜杠	5CH

4，字符串型常量

字符串型常量由双引号“”括起的字符组成。如“D”、“1234”、“ABCD”等。注意字符串常量与字符常量是不一样，一个字符常量在计算机内只用一个字节存放，而一个字符串常量在内存中存放时不仅双引号内的字符一个占一个字节，而且系统会自动的在后面加一个转义字符“\o”作为字符串结束符。因此不要将字符常量和字符串常量混淆，如字符常量‘A’和字符串常量“A”是不一样的。

5.4.3.2 变量

变量是在程序运行过程中其值可以改变的量。一个变量由两部分组成：变量名和变量值。在 C51 中，变量在使用前必须对变量进行定义，指出变量的数据类型和存储模式。以便编译系统为它分配相应的存储单元。定义的格式如下：

[存储种类] 数据类型说明符 [存储器类型] 变量名 1[=初值]，变量名 2[初值]…；

1，数据类型说明符

在定义变量时，必须通过数据类型说明符指明变量的数据类型，指明变量在存储器中占用的字节数。可以是基本数据类型说明符，也可以是组合数据类型说

普中51单片机开发板

明符，还可以是用 typedef 定义的类型别名。

在 C51 中，为了增加程序的可读性，允许用户为系统固有的数据类型说明符用 typedef 起别名，格式如下：

```
typedef  c51 固有的数据类型说明符  别名;
```

定义别名后，就可以用别名代替数据类型说明符对变量进行定义。别名可以用大写，也可以用小写，为了区别一般用大写字母表示。

【例】 typedef 的使用。

```
typedef  unsigned  int   u8;
typedef  unsigned  char  u16;
u8 a1=0x12;
u16 a2=0x1234;
```

2，变量名

变量名是 C51 区分不同变量，为不同变量取的名称。在 C51 中规定变量名可以由字母、数字和下划线三种字符组成，且第一个位置必须为字母或下划线。变量名有两种：普通变量名和指针变量名。它们的区别是指针变量名前面要带“*”号。

3，存储种类

存储种类是指变量在程序执行过程中的作用范围。C51 变量的存储种类有四种，分别是自动(auto)、外部(extern)、静态(static)和寄存器(register)。

a. auto:

使用 auto 定义的变量称为自动变量，其作用范围在定义它的函数体或复合语句内部，当定义它的函数体或复合语句执行时，C51 才为该变量分配内存空间，结束时占用的内存空间释放。自动变量一般分配在内存的堆栈空间中。定义变量时，如果省略存储种类，则该变量默认为自动(auto)变量。

b. extern:

使用 extern 定义的变量称为外部变量。在一个函数体内，要使用一个已在该函数体外或别的程序中定义过的外部变量时，该变量在该函数体内要用 extern 说明。外部变量被定义后分配固定的内存空间，在程序整个执行时间内都有效，直到程序结束才释放。

c．static：

使用 static 定义的变量称为静态变量。它又分为内部静态变量和外部静态变量。在函数体内部定义的静态变量为内部静态变量，它在对应的函数体内有效，一直存在，但在函数体外不可见，这样不仅使变量在定义它的函数体外被保护，还可以实现当离开函数时值不被改变。外部静态变量上在函数外部定义的静态变量。它在程序中一直存在，但在定义的范围之外是不可见的。如在多文件或多模块处理中，外部静态变量只在文件内部或模块内部有效。

d．register：

使用 register 定义的变量称为寄存器变量。它定义的变量存放在 CPU 内部的寄存器中，处理速度快，但数目少。C51 编译器编译时能自动识别程序中使用频率最高的变量，并自动将其作为寄存器变量，用户可以无需专门声明。

4，存储器类型

存储器类型是用于指明变量所处的单片机的存储器区域情况。存储器类型与存储种类完全不同。C51 编译器能识别的存储器类型有以下几种，见表所示。

存储器类型	描　述
data	直接寻址的片内 RAM 低 128B，访问速度快
bdata	片内 RAM 的可位寻址区（20H~2FH），允许字节和位混合访问
idata	间接寻址访问的片内 RAM，允许访问全部片内 RAM
pdata	用 Ri 间接访问的片外 RAM 的低 256B
xdata	用 DPTR 间接访问的片外 RAM，允许访问全部 64k 片外 RAM
code	程序存储器 ROM 64k 空间

定义变量时也可以省“存储器类型”，省时 C51 编译器将按编译模式默认存储器类型。

5，特殊功能寄存器变量

51 系列单片机片内有许多特殊功能寄存器，通过这些特殊功能寄存器可以控制 51 系列单片机的定时器、计数器、串口、I/O 及其它功能部件，每一个特殊功能寄存器在片内 RAM 中都对应于一个字节单元或两个字节单元。

在 C51 中，允许用户对这些特殊功能寄存器进行访问，访问时须通过 sfr 或 sfr16 类型说明符进行定义，定义时须指明它们所对应的片内 RAM 单元的地址。格式如下：

```
sfr 或 sfr16   特殊功能寄存器名=地址;
```

sfr 用于对 51 单片机中单字节的特殊功能寄存器进行定义，sfr16 用于对双字节特殊功能寄存器进行定义。特殊功能寄存器名一般用大写字母表示。地址一般用直接地址形式。

【例】特殊功能寄存器的定义。

```
sfr   PSW=0xd0;
sfr   SCON=0x98;
sfr   TMOD=0x89;
sfr   P1=0x90;
sfr16   DPTR=0x82;
sfr16   T1=0X8A;
```

6，位变量

在 C51 中，允许用户通过位类型符定义位变量。位类型符有两个：bit 和 sbit。可以定义两种位变量。

bit 位类型符用于定义一般的可位处理位变量。它的格式如下：

```
bit   位变量名;
```

在格式中可以加上各种修饰，但注意存储器类型只能是 bdata、data、idata。只能是片内 RAM 的可位寻址区，严格来说只能是 bdata。

sbit 位类型符用于定义在可位寻址字节或特殊功能寄存器中的位，定义时须指明其位地址，可以是位直接地址，可以是可位寻址变量带位号，也可以是特殊功能寄存器名带位号。格式如下：

```
sbit   位变量名=位地址;
```

如位地址为位直接地址，其取值范围为 0x00~0xff；如位地址是可位寻址变量带位号或特殊功能寄存器名带位号，则在它前面须对可位寻址变量或特殊功能寄存器进行定义。字节地址与位号之间、特殊功能寄存器与位号之间一般用“^”作间隔。如定义 51 单片机管脚：

```
sbit  LED=P1^0;
```

在 C51 中，为了用户操作方便，C51 编译器把 51 单片机的常用的特殊功能寄存器和特殊位进行了定义，放在一个“reg51.h”或“reg52.h”的头文件中，当用户要使用时，只须要在使用之前用一条预处理命令“#include <reg51.h>”或“#include <reg52.h>”把这个头文件包含到程序最开始位置，然后就可使用殊功能寄存器名和特殊位名称。

5.4.4 C51 运算符

5.4.4.1 赋值运算符

赋值运算符“=”，在 C51 中，它的功能是将一个数据的值赋给一个变量，如 x=10。利用赋值运算符将一个变量与一个表达式连接起来的式子称为赋值表达式，在赋值表达式的后面加一个分号“;”就构成了赋值语句，一个赋值语句的格式如下：

```
变量=表达式;
```

执行时先计算出右边表达式的值，然后赋给左边的变量。例如：

```
x=8+9;      /*将 8+9 的值赋给变量 x*/
x=y=5;      /*将常数 5 同时赋给变量 x 和 y*/
```

在 C51 中，允许在一个语句中同时给多个变量赋值，赋值顺序自右向左。

5.4.4.2 算术运算符

C51 中支持的算术运算符有：

```
+     加或取正值运算符
-     减或取负值运算符
*     乘运算符
/     除运算符
%     取余运算符
```

加、减、乘运算相对比较简单，而对于除运算，如相除的两个数为浮点数，

则运算的结果也为浮点数，如相除的两个数为整数，则运算的结果也为整数，即为整除。如 25.0/20.0 结果为 1.25，而 25/20 结果为 1。

对于取余运算，则要求参加运算的两个数必须为整数，运算结果为它们的余数。例如：x=5%3，结果 x 的值为 2。

5.4.4.3 关系运算符

C51 中有 6 种关系运算符：

```
>     大于
<     小于
>=    大于等于
<=    小于等于
==    等于
!=    不等于
```

关系运算用于比较两个数的大小，用关系运算符将两个表达式连接起来形成的式子称为关系表达式。关系表达式通常用来作为判别条件构造分支或循环程序。关系表达式的一般形式如下：

```
表达式 1  关系运算符  表达式 2
```

关系运算的结果为逻辑量，成立为真（1），不成立为假（0）。其结果可以作为一个逻辑量参与逻辑运算。例如：5>3，结果为真（1），而 10==100，结果为假（0）。

注意：关系运算符等于“==”是由两个“=”组成。

5.4.4.4 逻辑运算符

C51 有 3 种逻辑运算符：

```
||     逻辑或
&&     逻辑与
!      逻辑非
```

关系运算符用于反映两个表达式之间的大小关系，逻辑运算符则用于求条件

式的逻辑值，用逻辑运算符将关系表达式或逻辑量连接起来的式子就是逻辑表达式。

逻辑与，格式：

```
条件式 1 && 条件式 2
```

当条件式 1 与条件式 2 都为真时结果为真（非 0 值），否则为假（0 值）。

逻辑或，格式：

```
条件式 1 || 条件式 2
```

当条件式 1 与条件式 2 都为假时结果为假（0 值），否则为真（非 0 值）。

逻辑非，格式：

```
！条件式
```

当条件式原来为真（非 0 值），逻辑非后结果为假（0 值）。当条件式原来为假（0 值），逻辑非后结果为真（非 0 值）。

例如：若 a=8，b=3，c=0，则！a 为假，a && b 为真，b && c 为假。

5.4.4.5 位运算符

C51 语言能对运算对象按位进行操作，它与汇编语言使用一样方便。位运算是按位对变量进行运算，但并不改变参与运算的变量的值。如果要求按位改变变量的值，则要利用相应的赋值运算。C51 中位运算符只能对整数进行操作，不能对浮点数进行操作。C51 中的位运算符有：

```
&     按位与
|     按位或
^     按位异或
~     按位取反
<<    左移
>>    右移
```

【例】设 a=0x45=01010100B，b=0x3b=00111011B，则 a&b、a|b、a^b、~a、a<<2、b>>2 分别为多少？

a&b=00010000B=0x10。

a|b=01111111B=0x7f。

a^b=01101111B=0x6f。

~a=10101011B=0xab。

a<<2=01010000B=0x50。

b>>2=00001110B=0x0e。

5.4.4.6 复合赋值运算符

C51 语言中支持在赋值运算符“=”的前面加上其它运算符，组成复合赋值运算符。下面是 C51 中支持的复合赋值运算符

+=	加法赋值	-=	减法赋值
*=	乘法赋值	/=	除法赋值
%=	取模赋值	&=	逻辑与赋值
|=	逻辑或赋值	^=	逻辑异或赋值
~=	逻辑非赋值	>>=	右移位赋值
<<=	左移位赋值		

复合赋值运算的一般格式如下：

```
变量  复合运算赋值符  表达式
```

它的处理过程：先把变量与后面的表达式进行某种运算，然后将运算的结果赋给前面的变量。其实这是 C51 语言中简化程序的一种方法，大多数二目运算都可以用复合赋值运算符简化表示。例如：a+=6 相当于 a=a+6；a*=5 相当于 a=a*5；b&=0x55 相当于 b=b&0x55；x>>=2 相当于 x=x>>2。

5.4.4.7 逗号运算符

在 C51 语言中，逗号“,”是一个特殊的运算符，可以用它将两个或两个以上的表达式连接起来，称为逗号表达式。逗号表达式的一般格式为：

```
表达式 1，表达式 2，……，表达式 n
```

程序执行时对逗号表达式的处理：按从左至右的顺序依次计算出各个表达式的值，而整个逗号表达式的值是最右边的表达式（表达式 n）的值。例如：

x=(a=3,6*3)结果 x 的值为 18。

5.4.4.8 条件运算符

条件运算符“？：”是 C51 语言中唯一的一个三目运算符，它要求有三个运算对象，用它可以将三个表达式连接在一起构成一个条件表达式。条件表达式的一般格式为：

逻辑表达式？表达式 1：表达式 2

其功能是先计算逻辑表达式的值，当逻辑表达式的值为真（非 0 值）时，将计算的表达式 1 的值作为整个条件表达式的值；当逻辑表达式的值为假（0 值）时，将计算的表达式 2 的值作为整个条件表达式的值。例如：条件表达式 max=(a>b)?a:b 的执行结果是将 a 和 b 中较大的数赋值给变量 max。

5.4.4.9 指针与地址运算符

指针是 C51 语言中的一个十分重要的概念，在 C51 中的数据类型中专门有一种指针类型。指针为变量的访问提供了另一种方式，变量的指针就是该变量的地址，还可以定义一个专门指向某个变量的地址的指针变量。

为了表示指针变量和它所指向的变量地址之间的关系，C51 中提供了两个专门的运算符：

*	指针运算符
&	取地址运算符

指针运算符“*”放在指针变量前面，通过它实现访问以指针变量的内容为地址所指向的存储单元。例如：指针变量 p 中的地址为 2000H，则*p 所访问的是地址为 2000H 的存储单元，x=*p，实现把地址为 2000H 的存储单元的内容送给变量 x。

取地址运算符“&”放在变量的前面，通过它取得变量的地址，变量的地址通常送给指针变量。例如：设变量 x 的内容为 12H，地址为 2000H，则&x 的值为 2000H，如有一指针变量 p，则通常用 p=&x，实现将 x 变量的地址送给指针变量 p，指针变量 p 指向变量 x，以后可以通过*p 访问变量 x。

5.4.5 C51 表达式及复合语句

5.4.5.1 表达式语句

在表达式的后边加一个分号“;”就构成了表达式语句 ,如:

```
a=++b*9;
x=8;y=7;
++k;
```

可以一行放一个表达式形成表达式语句，也可以一行放多个表达式形成表达式语句，这时每个表达式后面都必须带“;”号，另外，还可以仅由一个分号“;”占一行形成一个表达式语句，这种语句称为空语句。

空语句在程序设计中通常用于两种情况：

①在程序中为有关语句提供标号，用以标记程序执行的位置。例如采用下面的语句可以构成一个循环。

```
repeat：语句
    ...功能
goto  repeat;
```

②在用 while 语句构成的循环语句后面加一个分号，形成一个不执行其它操作的空循环体。这种结构通常用于对某位进行判断，当不满足条件则等待，满足条件则执行。

5.4.5.2 复合语句

复合语句是由若干条语句组合而成的一种语句，在 C51 中，用一个大括号“{ }”将若干条语句括在一起就形成了一个复合语句，复合语句最后不需要以分号“；”结束，但它内部的各条语句仍需以分号“；”结束。复合语句的一般形式为：

```
{
    局部变量定义;
    语句 1;
```

```
    语句 2;
}
```

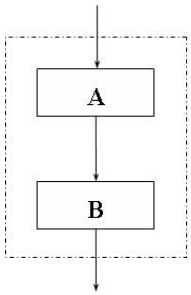
复合语句在执行时，其中的各条单语句按顺序依次执行，整个复合语句在语法上等价于一条单语句，因此在 C51 中可以将复合语句视为一条单语句。通常复合语句出现在函数中，实际上，函数的执行部分（即函数体）就是一个复合语句；复合语句中的单语句一般是可执行语句，此外还可以是变量的定义语句（说明变量的数据类型）。在复合语句内部语句所定义的变量，称为该复合语句中的局部变量，它仅在当前这个复合语句中有效。利用复合语句将多条单语句组合在一起，以及在复合语句中进行局部变量定义是 C51 语言的一个重要特征。

5.4.6 C51 基本结构和相关语句

5.4.6.1 C51 的基本结构

5.4.6.1.1 顺序结构

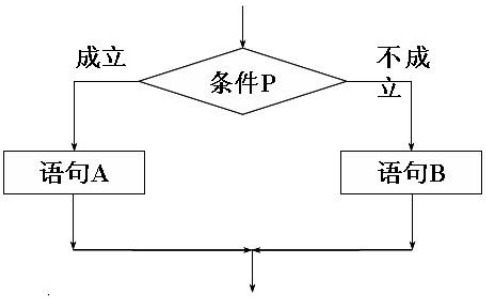
顺序结构是最基本、最简单的结构，在这种结构中，程序由低地址到高地址依次执行，如图给出顺序结构流程图，程序先执行 A 操作，然后再执行 B 操作。



5.4.6.1.2 选择结构

选择结构可使程序根据不同的情况，选择执行不同的分支，在选择结构中，程序先都对一个条件进行判断。当条件成立，即条件语句为“真”时，执行一个分支，当条件不成立时，即条件语句为“假”时，执行另一个分支。如图：当条件 S 成立时，执行分支 A，当条件 P 不成立时，执行分支 B。

普中51单片机开发板



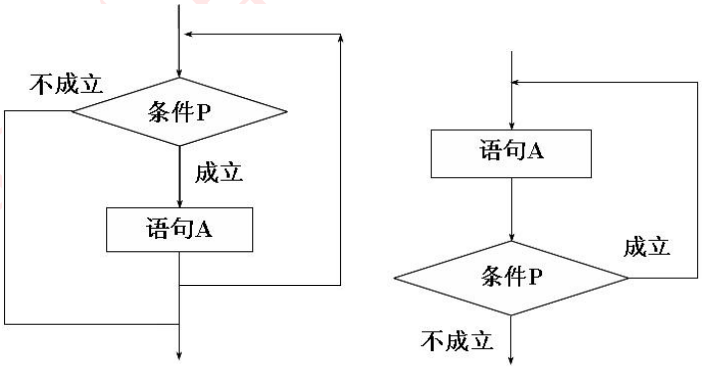
在 C51 中，实现选择结构的语句为 if/else，if/else if 语句。另外在 C51 中还支持多分支结构，多分支结构既可以通过 if 和 else if 语句嵌套实现，可用 swith/case 语句实现。

5.4.6.1.3 循环结构

在程序处理过程中，有时需要某一段程序重复执行多次，这时就需要循环结构来实现，循环结构就是能够使程序段重复执行的结构。循环结构又分为两种：当（while）型循环结构和直到（do...while）型循环结构。

①当型循环结构

当型循环结构如图：当条件 P 成立（为“真”）时，重复执行语句 A，当条件不成立（为“假”）时才停止重复，执行后面的程序。



②直到型循环结构

直到型循环结构，先执行语句 A，再判断条件 P，当条件成立（为“真”）时，再重复执行语句 A，直到条件不成立（为“假”）时才停止重复，执行后面的程序。构成循环结构的语句主要有：while、do while、for、goto。

5.4.6.2 C51 的相关语句

5.4.6.2.1 if 语句

if 语句是 C51 中的一个基本条件选择语句，它通常有三种格式：

```
①if （表达式）｛语句；｝
②if （表达式）｛语句 1；｝ else ｛语句 2；｝
③if （表达式 1）｛语句 1；｝
   else if （表达式 2） （语句 2；）
   else if （表达式 3） （语句 3；）
   ……
   else if （表达式 n-1） （语句 n-1；）
   else ｛语句 n｝
```

【例】 if 语句的用法。

（1）if (x!=y) printf(“x=%d,y=%d\n”,x,y);

执行上面语句时，如果 x 不等于 y，则输出 x 的值和 y 的值。

（2）if (x>y) max=x;
   else max=y;

执行上面语句时，如 x 大于 y 成立，则把 x 送给最大值变量 max，如 x 大于 y 不成立，则把 y 送给最大值变量 max。使 max 变量得到 x、y 中的大数。

（3）if (score>=90) printf(“Your result is an A\n”);
   else if (score>=80) printf(“Your result is an B\n”);
   else if (score>=70) printf(“Your result is an C\n”);
   else if (score>=60) printf(“Your result is an D\n”);
   else printf(“Your result is an E\n”);

执行上面语句后，能够根据分数 score 分别打出 A、B、C、D、E 五个等级。

5.4.6.2.2 switch/case 语句

if 语句通过嵌套可以实现多分支结构，但结构复杂。switch 是 C51 中提供

的专门处理多分支结构的多分支选择语句。它的格式如下：

```
switch （表达式）
{
    case  常量表达式 1：{语句 1；}break；
    case  常量表达式 2：{语句 2；}break；
    ……
    case  常量表达式 n：{语句 n；}break；
    default：{语句 n+1；}
}
```

说明如下：

（1）switch 后面括号内的表达式，可以是整型或字符型表达式。

（2）当该表达式的值与某一“case”后面的常量表达式的值相等时，就执行该“case”后面的语句，然后遇到 break 语句退出 switch 语句。若表达式的值与所有 case 后的常量表达式的值都不相同，则执行 default 后面的语句，然后退出 switch 结构。

（3）每一个 case 常量表达式的值必须不同否则会出现自相矛盾的现象。

（4）case 语句和 default 语句的出现次序对执行过程没有影响。

（5）每个 case 语句后面可以有“break”，也可以没有。有 break 语句，执行到 break 则退出 switch 结构，若没有，则会顺次执行后面的语句，直到遇到 break 或结束。

（6）每一个 case 语句后面可以带一个语句，也可以带多个语句，还可以不带。语句可以用花括号括起，也可以不括。

（7）多个 case 可以共用一组执行语句。

【例】 switch/case 语句的用法。

对学生成绩划分为 A~D，对应不同的百分制分数，要求根据不同的等级打印出它的对应百分数。可以通过下面的 switch/case 语句实现。

```
……
switch（grade）
{
```

```
    case   'A' ; printf (" 90~100\n" ) ; break;
    case   'B' ; printf (" 80~90\n" ) ; break;
    case   'C' ; printf (" 70~80\n" ) ; break;
    case   'D' ; printf (" 60~70\n" ) ; break;
    case   'E' ; printf (" <60\n" ) ; break;
    default; printf (" error" \n)
}
```

5.4.6.2.3 while 语句

while 语句在 C51 中用于实现当型循环结构，它的格式如下：

```
while（表达式）
{
   语句;
} /*循环体*/
```

while 语句后面的表达式是能否循环的条件，后面的语句是循环体。当表达式为非 0（真）时，就重复执行循环体内的语句；当表达式为 0（假），则中止 while 循环，程序将执行循环结构之外的下一条语句。它的特点是：先判断条件，后执行循环体。在循环体中对条件进行改变，然后再判断条件，如条件成立，则再执行循环体，如条件不成立，则退出循环。如条件第一次就不成立，则循环体一次也不执行。

5.4.6.2.4 do...while 语句

do...while 语句在 C51 中用于实现直到型循环结构，它的格式如下：

```
do
{
   语句;
} /*循环体*/
while（表达式）;
```

它的特点是：先执行循环体中的语句，后判断表达式。如表达式成立（真），则再执行循环体，然后又判断，直到有表达式不成立（假）时，退出循环，执行 do while 结构的下一条语句。do while 语句在执行时，循环体内的语句至少会被执行一次。

5.4.6.2.5 for 语句

```
for（表达式 1；表达式 2；表达式 3）
{
    语句；
}  /*循环体*/
```

for 语句后面带三个表达式，它的执行过程如下：

（1）先求解表达式 1 的值。

（2）求解表达式 2 的值，如表达式 2 的值为真，则执行循环休中的语句，然后执行下一步（3）的操作，如表达式 2 的值为假，则结束 for 循环，转到最后一步。

（3）若表达式 2 的值为真，则执行完循环体中的语句后，求解表达式 3，然后转到第四步。

（4）转到（2）继续执行。

（5）退出 for 循环，执行下面的一条语句。

在 for 循环中，一般表达式 1 为初值表达式，用于给循环变量赋初值；表达式 2 为条件表达式，对循环变量进行判断；表达式 3 为循环变量更新表达式，用于对循环变量的值进行更新，使循环变量能不满足条件而退出循环。

5.4.6.2.6 循环的嵌套

在一个循环的循环体中允许又包含一个完整的循环结构，这种结构称为循环的嵌套。外面的循环称为外循环，里面的循环称为内循环，如果在内循环的循环体内又包含循环结构，就构成了多重循环。在 C51 中，允许三种循环结构相互嵌套。

【例】用嵌套结构构造一个延时程序。

```
void  delay(unsigned  int  x)
{
    unsigned  char j;
    while(x--)
    {for (j=0;j<125;j++);}
}
```

这里，用内循环构造一个基准的延时，调用时通过参数设置外循环的次数，这样就可以形成各种延时关系。

5.4.6.2.7 break 和 continue 语句

break 和 continue 语句通常用于循环结构中，用来跳出循环结构。但是二者又有所不同，下面分别介绍。

（1）break 语句

前面已介绍过用 break 语句可以跳出 switch 结构，使程序继续执行 switch 结构后面的一个语句。使用 break 语句还可以从循环体中跳出循环，提前结束循环而接着执行循环结构下面的语句。它不能用在除了循环语句和 switch 语句之外的任何其它语句中。

【例】下面一段程序用于计算圆的面积，当计算到面积大于 100 时，由 break 语句跳出循环。

```
for (r=1; r<=10; r++)
{
    area=pi*r*r;
    if (area>100) break;
    printf("%f\n", area);
}
```

（2）continue 语句

continue 语句用在循环结构中，用于结束本次循环，跳过循环体中 continue 下面尚未执行的语句，直接进行下一次是否执行循环的判定。

continue 语句和 break 语句的区别在于：continue 语句只是结束本次循环而不是终止整个循环；break 语句则是结束循环，不再进行条件判断。

【例】 输出 100~200 间不能被 3 整除的数。

```
for (i=100; i<=200; i++)
{
    if  (i%3= =0)  continue;
    printf(“%d  ”; i);
}
```

在程序中，当 i 能被 3 整除时，执行 continue 语句，结束本次循环，跳过 printf 函数，只有能被 3 整除时才执行 printf 函数。

5.4.6.2.8 return 语句

return 语句一般放在函数的最后位置，用于终止函数的执行，并控制程序返回调用该函数时所处的位置。返回时还可以通过 return 语句带回返回值。return 语句格式有两种：

（1）return;

（2）return (表达式);

如果 return 语句后面带有表达式，则要计算表达式的值，并将表达式的值作为函数的返回值。若不带表达式，则函数返回时将返回一个不确定的值。通常我们用 return 语句把调用函数取得的值返回给主调用函数。

5.4.7 函数

5.4.7.1 函数的定义

函数定义的一般格式如下：

```
函数类型  函数名(形式参数表)  [reentrant][interrupt  m][using  n]
形式参数说明
{
```

```
    局部变量定义
    函数体
}
```

前面部件称为函数的首部，后面称为函数的尾部，格式说明：

1．函数类型

函数类型说明了函数返回值的类型。

2．函数名

函数名是用户为自定义函数取的名字以便调用函数时使用。

3．形式参数表

形式参数表用于列录在主调函数与被调用函数之间进行数据传递的形式参数。

【例】定义一个返回两个整数的最大值的函数 max()。

```
int max(int  x,int  y)
{
    int  z;
    z=x>y?x：y;
    return（z）;
}
```

4．reentrant 修饰符

这个修饰符用于把函数定义为可重入函数。所谓可重入函数就是允许被递归调用的函数。函数的递归调用是指当一个函数正被调用尚未返回时，又直接或间接调用函数本身。一般的函数不能做到这样，只有重入函数才允许递归调用。

关于重入函数，注意以下几点：

（1）用 reentrant 修饰的重入函数被调用时，实参表内不允许使用 bit 类型的参数。函数体内也不允许存在任何关于位变量的操作，更不能返回 bit 类型的值。

（2）编译时，系统为重入函数在内部或外部存储器中建立一个模拟堆栈区，称为重入栈。重入函数的局部变量及参数被放在重入栈中，使重入函数可以实现递归调用。

（3）在参数的传递上，实际参数可以传递给间接调用的重入函数。无重入属性的间接调用函数不能包含调用参数，但是可以使用定义的全局变量来进行参数传递。

5. interrupt m 修饰符

interrupt m 是 C51 函数中非常重要的一个修饰符，这是因为中断函数必须通过它进行修饰。在 C51 程序设计中，当函数定义时用了 interrupt m 修饰符，系统编译时把对应函数转化为中断函数，自动加上程序头段和尾段，并按 51 系统中断的处理方式自动把它安排在程序存储器中的相应位置。

在该修饰符中，m 的取值为 0~31，对应的中断情况如下：

0——外部中断 0

1——定时/计数器 T0

2——外部中断 1

3——定时/计数器 T1

4——串行口中断

5——定时/计数器 T2

其它值预留。

5.4.7.2 函数的调用与声明

5.4.7.2.1 函数的调用

函数调用的一般形式如下：

```
函数名（实参列表）；
```

对于有参数的函数调用，若实参列表包含多个实参，则各个实参之间用逗号隔开。

按照函数调用在主调函数中出现的位置，函数调用方式有以下三种：

（1）函数语句。把被调用函数作为主调用函数的一个语句。

（2）函数表达式。函数被放在一个表达式中，以一个运算对象的方式出现。这时的被调用函数要求带有返回语句，以返回一个明确的数值参加表达式的运算。

（3）函数参数。被调用函数作为另一个函数的参数。

5.4.7.2.2 自定义函数的声明

在 C51 中，函数原型一般形式如下：

```
[extern]  函数类型  函数名（形式参数表）；
```

函数的声明是把函数的名字、函数类型以及形参的类型、个数和顺序通知编译系统，以便调用函数时系统进行对照检查。函数的声明后面要加分号。

如果声明的函数在文件内部，则声明时不用 extern，如果声明的函数不在文件内部，而在另一个文件中，声明时须带 extern，指明使用的函数在另一个文件中。

5.4.8 C51 构造数据类型

5.4.8.1 数组

5.4.8.1.1 一维数组

一维数组只有一个下标，定义的形式如下：

```
数据类型说明符  数组名[常量表达式][={初值，初值……}]
```

各部分说明如下：

（1）“数据类型说明符”说明了数组中各个元素存储的数据的类型。

（2）“数组名”是整个数组的标识符，它的取名方法与变量的取名方法相同。

（3）“常量表达式”，常量表达式要求取值要为整型常量，必须用方括号“[]”括起来。用于说明该数组的长度，即该数组元素的个数。

（4）“初值部分”用于给数组元素赋初值，这部分在数组定义时属于可选项。对数组元素赋值，可以在定义时赋值，也可以定义之后赋值。在定义时赋值，后面须带等号，初值须用花括号括起来，括号内的初值两两之间用逗号间隔，可以对数组的全部元素赋值，也可以只对部分元素赋值。初值为 0 的元素可以只用

逗号占位而不写初值 0。

例如：下面是定义数组的两个例子。

```
unsigned  char  x[5];
unsigned  int  y[3]={1,2,3};
```

第一句定义了一个无符号字符数组，数组名为 x，数组中的元素个数为 5。

第二句定义了一个无符号整型数组，数组名为 y，数组中元素个数为 3，定义的同时给数组中的三个元素赋初值，赋初值分别为 1、2、3。

需要注意的是，C51 语言中数组的下标是从 0 开始的，因此上面第一句定义的 5 个元素分别是：x[0]、x[1]、x[2]、x[3]、x[4]。第二句定义的 3 个元素分别是：y[0]、y[1]、y[2]。赋值情况为：y[0]=1；y[1]=2；y[2]=3。

C51 规定在引用数组时，只能逐个引用数组中的各个元素，而不能一次引用整个数组。但如果是字符数组则可以一次引用整个数组。

5.4.8.1.2 字符数组

用来存放字符数据的数组称为字符数组，它是 C 语言中常用的一种数组。字符数组中的每一个元素都用来存放一个字符，也可用字符数组来存放字符串。字符数组的定义下一般数组相同，只是在定义时把数据类型定义为 char 型。

例如：char string1[10];

```
char  string2[20];
```

上面定义了两个字符数组，分别定义了 10 个元素和 20 个元素。

在 C51 语言中，字符数组用于存放一组字符或字符串，字符串以“\0”作为结束符，只存放一般字符的字符数组的赋值与使用和一般的数组完全相同。对于存放字符串的字符数组。既可以对字符数组的元素逐个进行访问，也可以对整个数组按字符串的方式进行处理。

至此我们把 51 单片机涉及到的 C 语言基础知识简单介绍完毕，大家还可以打开我们的资料资料“\7--51 相关资料\C 语言加油站\[我的第①本 c 语言编程书：C 语言从入门到精通].国家 863 中部软件孵化器.扫描版.pdf”来全面的学习 C 语言基础知识。

课后作业

（1）花一点时间重新学习本章内容或资料“\7--单片机相关资料\C 语言加油站\[我的第①本 c 语言编程书：C 语言从入门到精通].国家 863 中部软件孵化器.扫描版.pdf”文档，查找变量命名的规则、if/switch 语句、for 语句、while 语句、一维数组的使用方法。（温馨提示：这些知识也在是 51 单片机开发过程中应用最多的）

第 6 章 51 单片机最小系统

在前面章节，我们已经认识了 51 单片机芯片，51 单片机要工作，光靠一个芯片肯定是不行的，它必须搭配相应的外围电路，我们把能使 51 单片机工作的最简单基础的电路统称为 51 单片机最小系统。相信很多学完 51 单片机的朋友在后期可能会自己做一些具体的综合项目，在项目中涉及到的硬件电路各种各样，但唯一不变的是最小系统，因此有些朋友就会自己去制作最小系统。本章就来介绍 51 单片机最小系统构成，通过本章的学习，让大家能够掌握 51 单片机最小系统的构成，为后期自己搭建最小系统提供理论基础。本章分为如下几部分内容：

6.1 51 最小系统构成

普中51单片机开发板

6.1 51 单片机最小系统构成

若要使系统正常运行，必须确保单片机的最小系统稳定工作。51 单片机的最小系统由以下几部分组成：

（1）晶振电路

（2）复位电路

（3）电源电路

（4）下载电路

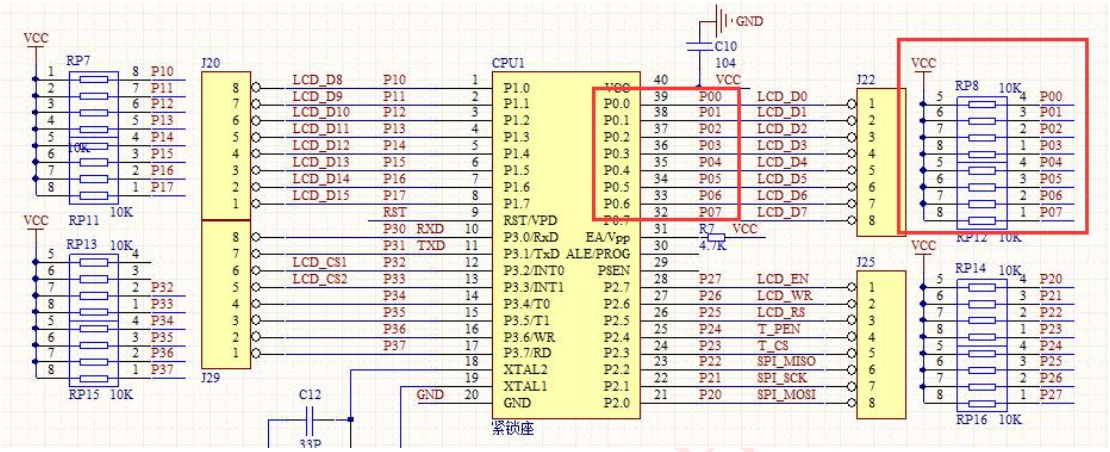
第（4）是我们添加进来的，实际上最小系统只由前面 3 个部分组成。为什么要加入第（4）部分呢？仅靠前面 3 个部分电路只能使单片机正常运行，但如果我们要给系统更新程序即烧入程序时就没有办法了，所以我们将第（4）部分电路也加入到最小系统当中，这时候我们就可以给系统自由烧入程序了。

晶振电路提供时钟给单片机工作，犹如人的心脏。复位电路提供系统复位操作，当系统出现运行不正常或者死机等情况时，可以通过复位按键重新启动系统。电源电路也是非常关键的一个部分，因为单片机对供电电压是有要求的，如果电

普中51单片机开发板

压过大将烧坏芯片，如果电压过小系统将运行不了。所以选择一个合适稳定的电源电路是非常关键的。

还有一点非常重要，由于 51 单片机的 P0 口是漏极开路，即输出高电平会导致高阻态，要让它输出高电平就必须外接上拉电阻，如下：

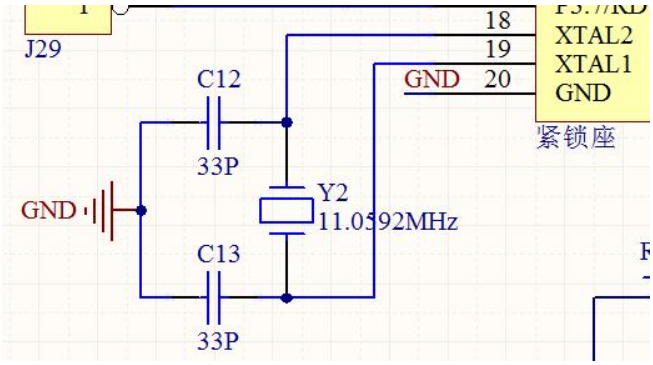


相同网络标号的表示其线路是连接在一起的，这个在后面章节会介绍。为了增强其他 IO 口的驱动能力，开发板上也都为其外接了上拉电阻（除了串口），通常上拉电阻选择 10K 左右。

下面就来介绍下这 4 部分的电路。

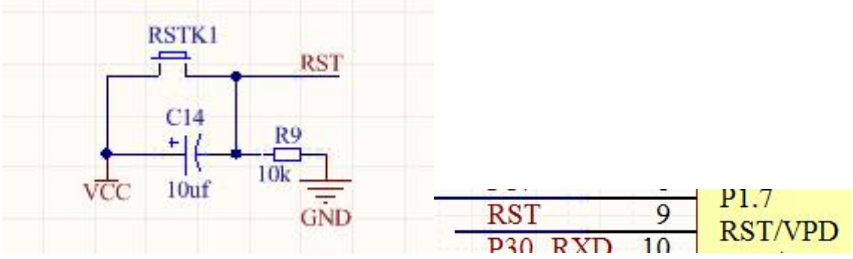
6.1.1 晶振电路

由于单片机正常工作需要一个时钟，因此就需要在其晶振引脚上外接晶振（我们使用的 STC89CXX 单片机晶振引脚是 18 和 19 脚），至于需要多大晶振这就取决于你所使用的单片机，由于我们使用的是 51 单片机，其时钟频率可在 0-40MHZ 上运行，一般情况下我们建议选择 12M（适合计算延时时间）或者是 11.0592M（适合串口通信）。若直接将此晶振接入单片机晶振引脚，会发现系统工作不稳定，这是因为晶振起振的一瞬间会产生一些电感，为了消除这个电感所带来的干扰，可以在此晶振两端分别加上一个电容，电容的选取需要无极性的，另一端需要共地。根据选取的晶振大小决定电容值，通常电容可在 10-33PF 值范围内选取。我们使用的是 33PF 电容。这样一来就构成了晶振电路。只有保证晶振电路稳定，单片机才能继续工作。其电路如下所示：



6.1.2 复位电路

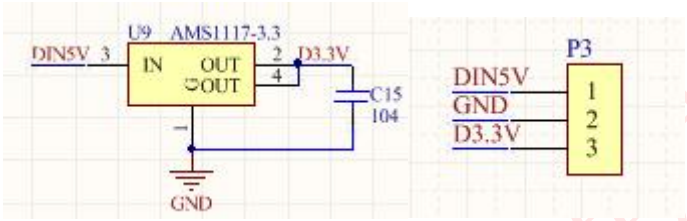
前面我们已经将晶振电路搭建完成，我们知道晶振犹如人的心脏，需要无时无刻给单片机提供运行周期。但即使时钟周期在不停的运行，系统也有可能会出现崩溃或者瘫痪状态。这就好比人会生病一样，人一生病就得看医生，服用医生开的药后重新获得正常状态。那么单片机呢，它是如何获取重生的？这就需要设计一个复位电路来实现此功能。我们知道单片机引脚当中就有一个 RST 复位引脚，而 STC89CXX 单片机又是高电平复位，所以只需要让这个引脚保持一段时间高电平就可以。要实现此功能通常有两种方式，一种是通过按键进行手动复位，还有一种是上电复位，即电源开启后自动复位。手动复位是通过一个按键及电容电阻所组成，利用按键的开关功能实现复位，按键按下后 VCC 直接进入到单片机 RST 引脚，松开后 VCC 断开，RST 被电阻拉为低电平。这一合一开就实现了手动复位。而自动复位主要是利用 RC 充放电功能，电源已开启，由于电容隔直，VCC 直接进入 RST，然后电容开始慢慢充电，直到充电完成，此时 RST 被电阻拉低。这样就起到上电复位的效果。这里我们采用手动复位。不到系统崩溃，我们几乎不会操作复位。复位电路如下图所示：



当按下按键 RSTK1，VCC 直接连接到 RST 复位脚，VCC 是高电平所以系统复位。

6.1.3 电源电路

任何电子器件都需要有一个合适的电源进行供电，这就好比人要吃饭一样，没有电源，系统是不会工作的。STC89CXX 单片机的工作电压是 3.3-5.5V 范围，通常我们使用 5V 直流。将电源接入到各芯片电源引脚即可。开发板电源电路如下图所示：



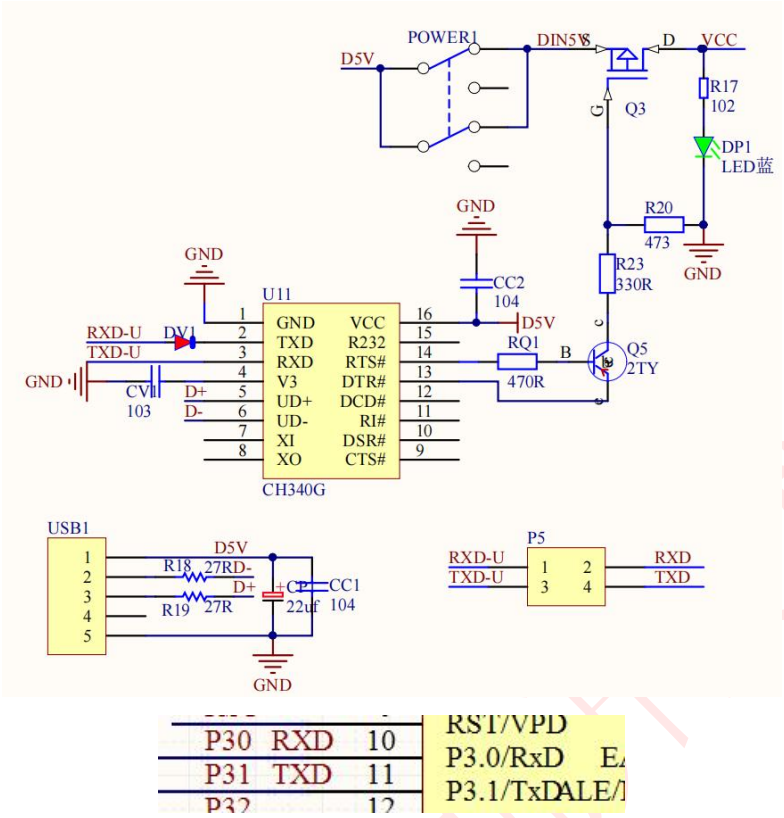
开发板上还预留了 P3（5V/3V）端子，这些端子可以很方便的给外部模块连接电源。

6.1.4 下载电路

在前面已经介绍过为什么要加下载电路到最小系统中，这里就不再重复。程序要烧写到单片机内是通过上位机（PC 机）及对应的软件将编译器生成的 xxx.HEX 文件通过单片机串口写入进去。

我们知道现在的笔记本电脑没有 RS232 接口，所以要使用 USB 转 TTL 串口电平芯片来建立 PC 机和单片机数据传输通路。通常使用 CH340G 或者 CH340C 芯片来完成电平转换。CH340G 需外接 12M 晶振，而 CH340C 内部自带晶振，所以可以不接外部 12M 晶振。开发板上使用的是 CH340C 芯片。

开发板下载电路如下图所示：



本电路是我们自主研发的一键自动下载电路，无需冷启动。主要依靠上位机软件（PZ-ISP）控制 CH340 芯片的 RTS 和 DTR 脚输出，进而使 MOS 管 Q3 自动断电后上电，完成冷启动工作。

从上图可以知道，USB1 接口即为程序下载接口，D-和 D+连接到 CH340 芯片的 D-和 D+，然后 CH340 芯片的串口 TXD 和 RXD 管脚就连接到单片机的串口（P3.0、P3.1）上，这里不是直接连接到单片机串口，而是通过 P5 端子进行转接。这样做不仅可以让开发板 USB 接口下载程序，还可以让单片机串口不受 CH340 干扰，这样开发板也可以当成 USB 转 TTL 模块使用，该设计也是考虑 WIFI、蓝牙等模块与串口通信不受干扰问题。

USB1 接口不仅可以作为程序下载口，还可以作为串口通信口，因为它本身就是实现串口下载。同时还可以作为电源供电口，可以看到 USB1 的管脚 1 就是 5V 电源脚，所以开发板的可以直接使用 USB 线来供电，如果提供的 USB 线有问题，可以使用安卓手机数据线，接口都是兼容的。当电源开关打开后，电源指示灯 DP1 即会点亮，表明系统电源正常。

至此，我们就把 51 单片机的最小系统介绍完，大家在制作最小系统时，可以参考我们的电路，这样成功几率会更高。

课后作业

普中51单片机开发板

第 7 章 KEIL C51 软件安装

本章将向大家介绍 KEIL C51 软件的安装，通过本章的学习，我们可以在电脑上安装一个 KEIL C51 软件，为后面学习程序的开发做好铺垫。本章分为如下几部分内容：

7.1 KEIL C51 软件获取

7.2 KEIL C51 软件安装

7.3 KEIL C51 软件破解

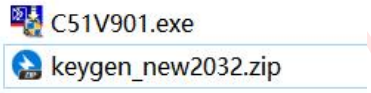
普中51单片机开发板

7.1 KEIL C51 软件获取

要在电脑上成功安装 KEIL C51，首先必须要有安装包，我们可以通过万能的百度，搜索关键字“KEIL C51 下载”，上面会有很多下载渠道，也可以在 KEIL 的官网上下载： https://www.keil.com/download/product/，打开界面如下图所示。

不过我们资料内已经给大家提供了下载好的软件，在资料“\5--开发工具\1-KEILC51 软件”内，大家直接使用即可，省去了查找下载的时间。我们使用 KEIL C51 是 9.01 版本， 如果后面出了更高的版本选择性升级即可，不过也没有必要使用最新的，用习惯了一个软件就行。

使用我们提供下载好的软件包，其内部含有如下图所示文件。

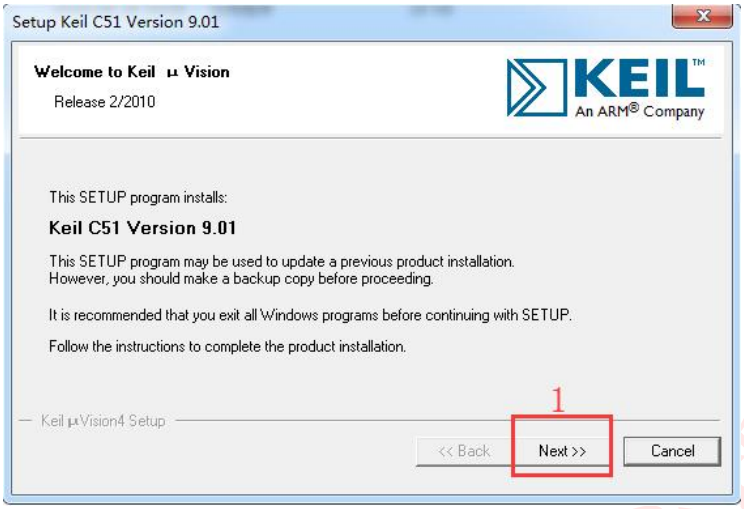


C51V901.exe：是 KEIL C51 软件的安装文件。

keygen_new2032.zip：KEIL C51 破解软件，在 KEIL C51 不破解的情况下只能编译 2K 大小文件，如果工程超过 2K，编译即会报错，因此需要破解。有的电脑系统会误认为该文件是病毒文件，所以这里把它压缩存放。

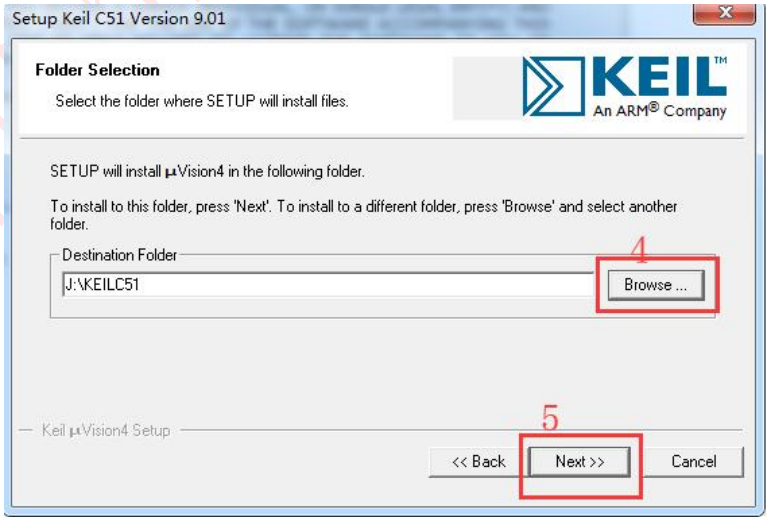
7.2 KEIL C51 软件安装

软件包下载完成之后，我们双击 C51V901.exe 这个应用程序，弹出如下所示对话框。



点击 Next 按钮。 弹出如下对话框。

在红框 2 中勾选上，点击 Next 按钮。弹出如下对话框。

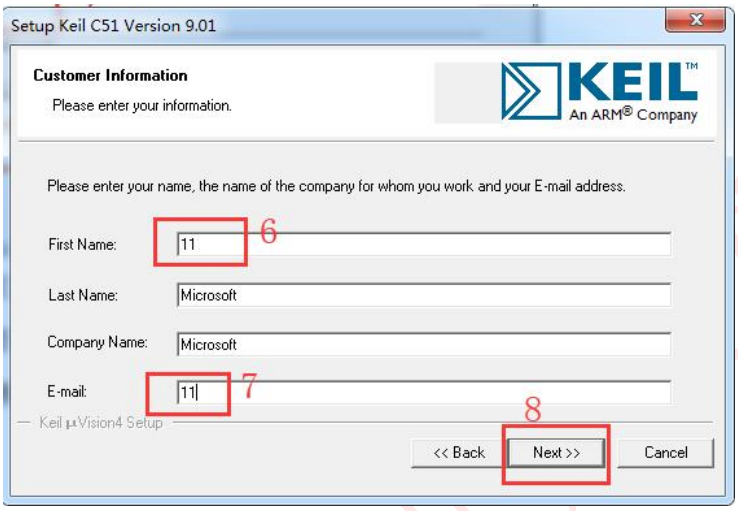


点击红框 4 选择软件安装的路径，我们这里在 J 盘内新建一个文件夹，命名为 KEILC51，将其安装到该文件夹内，方便软件的管理。**特别要注意：**

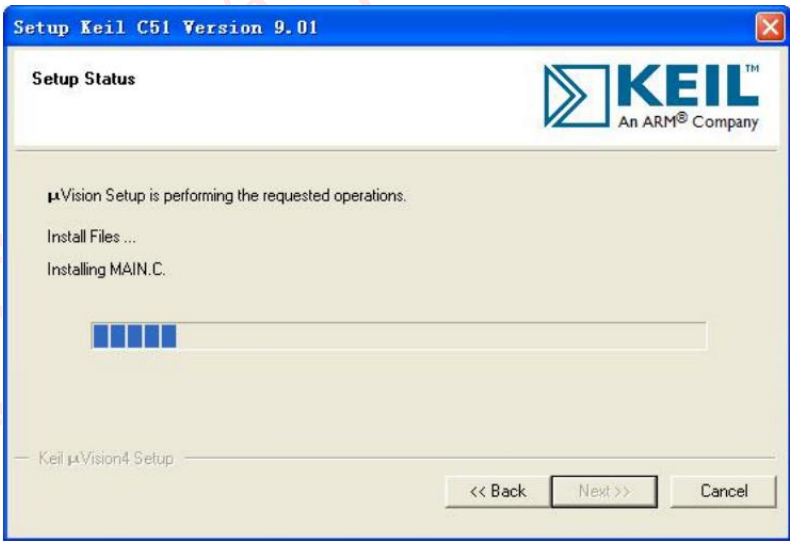
（1）软件安装保存路径不能出现中文或者特殊字符，否则会出现很多奇怪的错误，到时候很难找问题。

（2）不要将 KEIL5 软件和 KEIL4 或者 51 的 KEIL 安装在一个文件夹内。

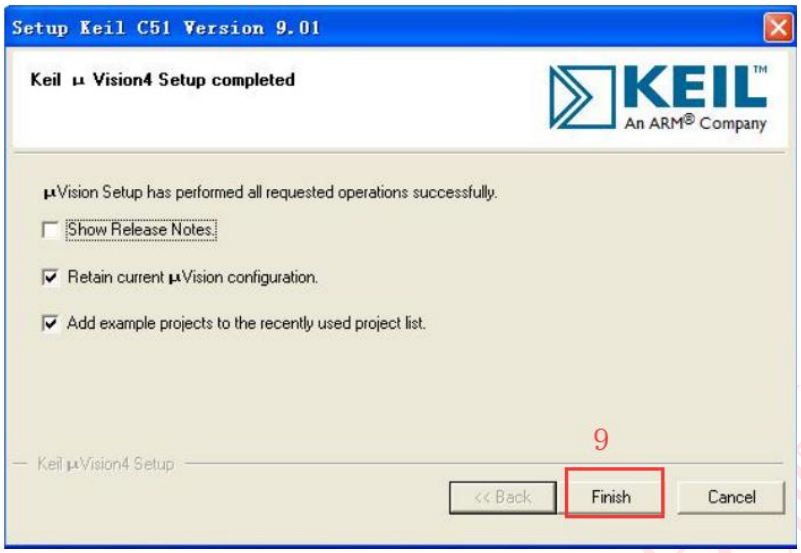
然后点击 Next。弹出如下对话框。



在红色框 6 和 7 中我们随便输入一些东西， 我们这里输入数字 11， 当然也可以输入空格， 但是一定要输入， 否则红色框 8 就一直是灰色状态， 输入完成后点击 Next， 弹出如下对话框。



说明 KEIL C51 软件正在安装，只要等一段时间即可，安装完成以后会出现如下界面。

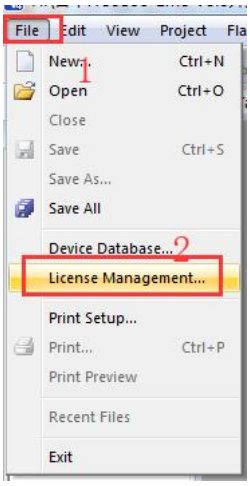


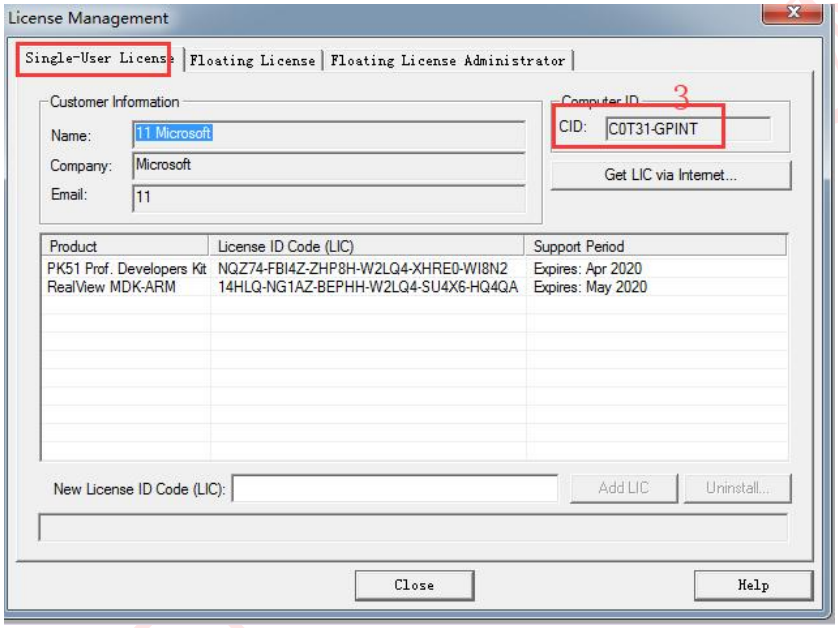
点击 Finish 按钮完成 KEIL C51 软件的安装过程。KEIL C51 安装完成后在电脑桌面上一般会有一个快捷方式，如下所示：

7.3 KEIL C51 软件破解

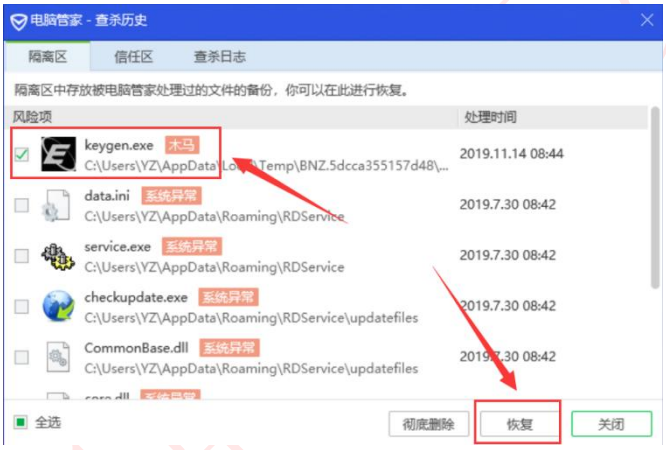
上一节我们已经安装好了 KEIL C51 编程软件，可以使用它来编译 51 单片机一些代码量小于 2K 的工程。但是有时候我们开发的项目可能比较大，程序代码超过了 2K，编译后会报错，因此就需要破解 KEIL C51。下面就来介绍下如何破解。

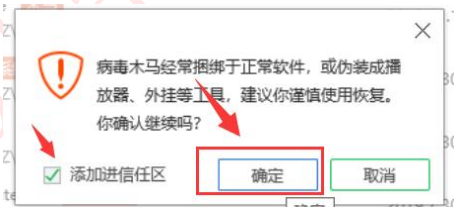
（1）首先打开 KEIL C51 软件，点击 File/License Management..,复制红色框 3 中的 CID。 操作步骤如下：



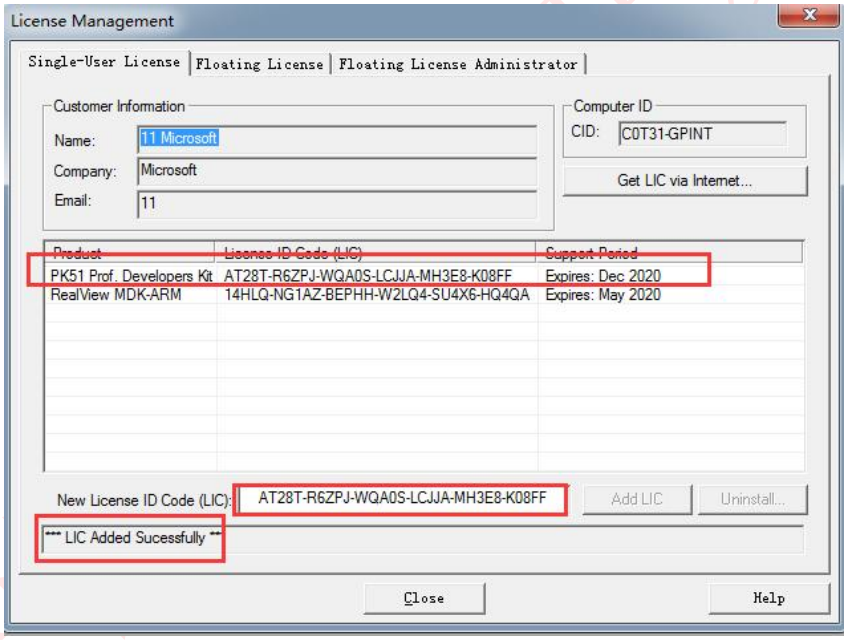


然后我们打开之前下载的 KEIL C51 软件包，里面还有一个 keygen 压缩文件，该压缩文件内含有 keygen_new2032.exe 文件，此文件用于破解。因此需要解压，解压后电脑会误认为该文件为病毒文件，会隔离掉，因此我们需要在弹出病毒隔离窗口时点击找回文件即可。如下所示：





以**管理员模式**打开此破解软件 keygen_new2032.exe，将刚才从 KEIL C51 中复制的 CID 码粘贴到下面红色框 4 中，并且在红色框 5 选择 C51，然后点击红色框 6，生成破解码，将此破解码即红色框 7 内容复制到 KEIL C51 软件内 LIC 框中，点击 Add LIC 即可破解，如果破解成功则会显示。操作步骤如下：



如果破解失败，请确认是否使用管理员模式打开 KEIL C51 软件和 keygen_new2032.exe 破解工具。管理员模式打开方式：选择右键： Run As Administrator ，中文系统应该是“以管理员权限运行”，然后就可以正常加 licence 了。建议所有用户，都以管理员身份运行，以避免重复破解。

至此，我们就将 KEIL C51 软件安装和破解成功。后面就可以使用该软件来编写我们的 51 单片机程序。

课后作业

普中51单片机开发板

第 8 章 51 单片机工程模板创建

本章将向大家介绍如何使用 KEIL C51 软件以及如何创建 51 单片机工程。通过本章的学习，让大家创建属于自己的 51 单片机工程模板，为后面学习 51 单片机编程做好铺垫。本章分为如下几部分内容：

8.1 51 单片机工程模板创建

8.2 reg52.h 头文件介绍

普中51单片机开发板

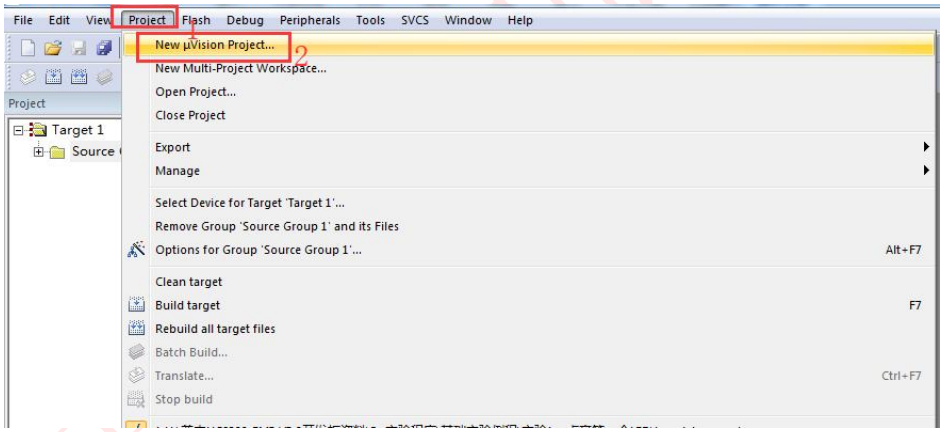
8.1 51 单片机工程模板创建

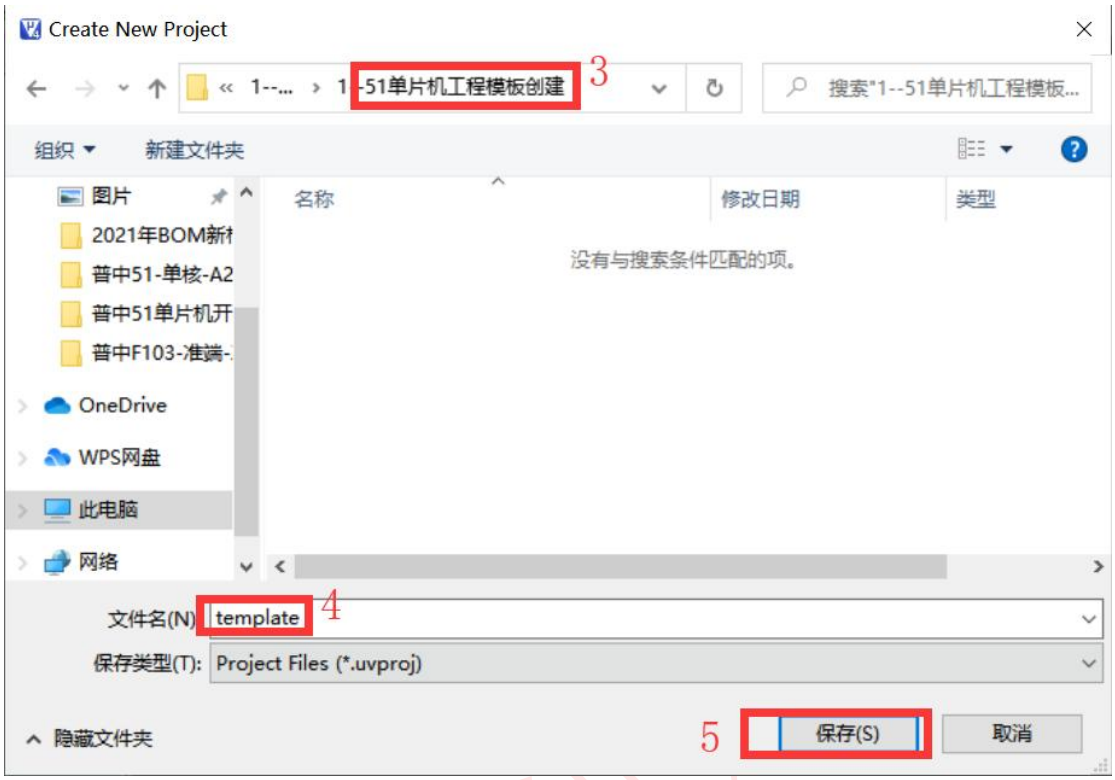
前面章节我们介绍了如何安装和破解 KEIL C51 软件，这一节我们来介绍下如何使用该软件创建一个 51 单片机工程模板，为后面章节程序的编写带来方便。

8.1.1 新建工程

首先在电脑任意位置新建一个文件夹，命名为“51 单片机工程模板创建”。

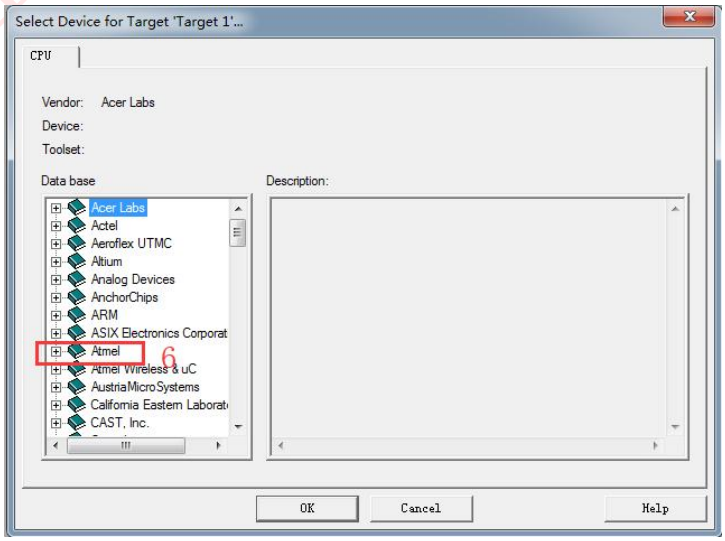
打开 KEIL C51 软件，新建一个工程，工程名根据喜好命名，但是要注意使用英文，如果使用中文名可能会出现一些奇怪的错误，这里我们命名为 template，直接保存在刚才创建的“51 单片机工程模板创建”文件夹下。具体步骤如下：

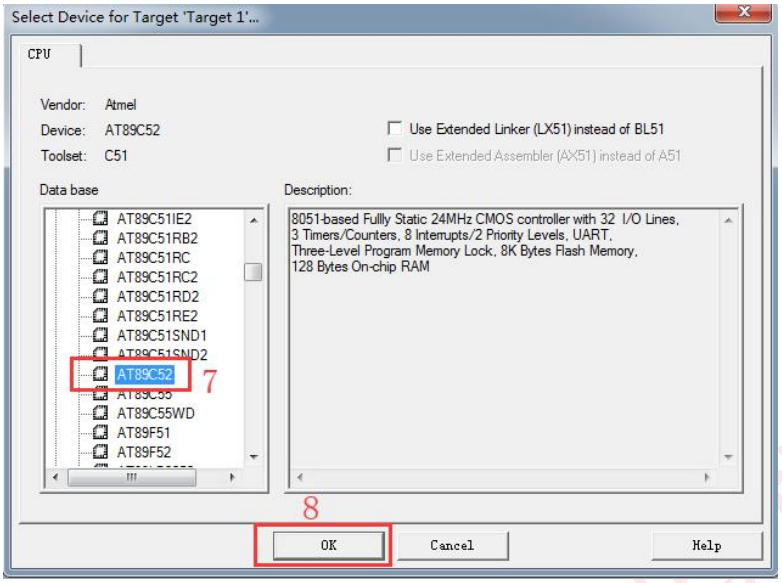




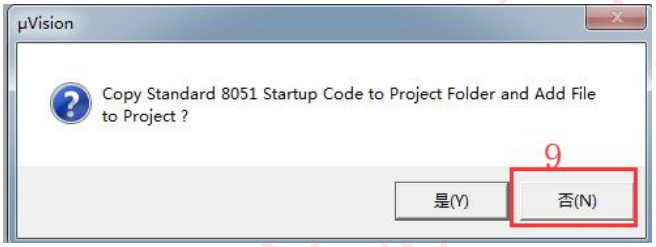
8.1.2 选择 CPU 型号

这个根据开发板使用的 CPU 具体的型号来选择，我们开发板使用的是 STC89C52 或 STC89C516 芯片。在 KEIL C51 软件内找不到这种型号的单片机，因为 51 内核单片机具有通用性，所以在这里可以任选一款 89C52 就行，Keil 软件的关键是程序代码的编写，而非用户选择什么硬件，在这里我们选择 Ateml 的 89C52 来说明。具体操作如下：

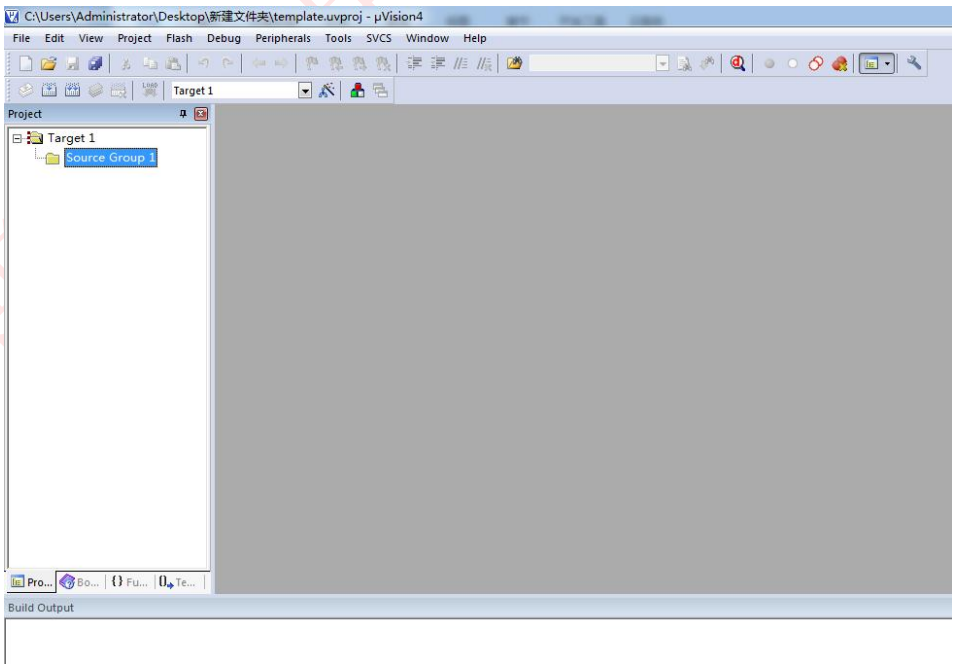




点击 OK 键后，弹出如下对话框：



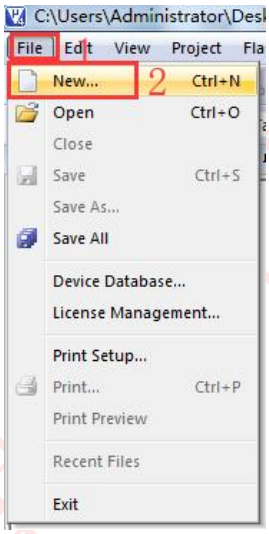
该对话框内容是提示我们是否要将 8051 启动文件添加到工程中，这里我们选择“否”，原因是 KEIL C51 内已经帮我们完成了启动，所以只需要编写应用程序即可。选择“否”后界面如下：

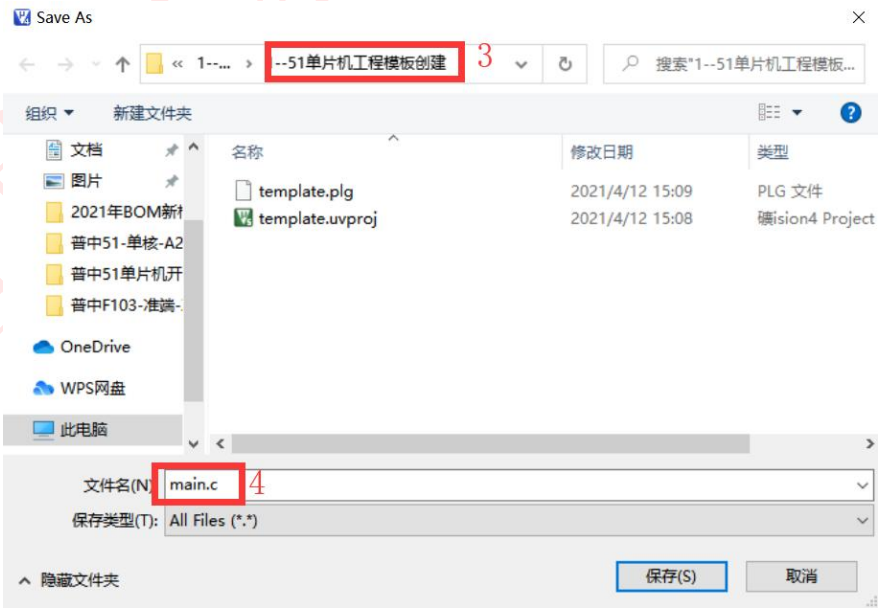


8.1.3 给工程添加文件

选择 File/New...或者使用工具栏的图标

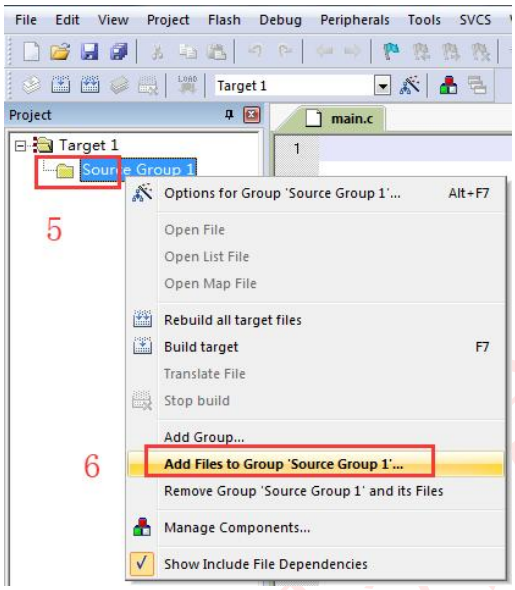
来新建一个文件后点击保存，系统会自动定位到我们工程目录，只需要在文件名栏输入新建的文件名即可，一个 51 单片机工程必须含有且仅有一个 main 函数，因此将新建的文件命名为 main.c（当然也可以根据个人习惯自定义命名，但必须是英文），点击保存。具体操作步骤如下：

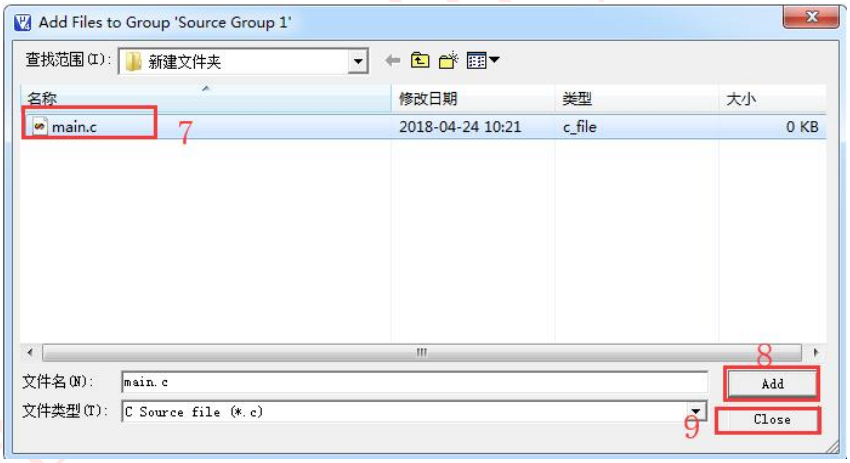


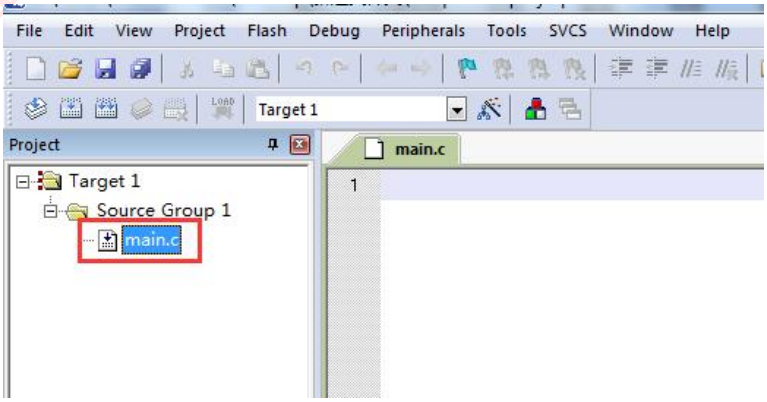


然后将新建的 main.c 文件添加到工程中，选择工程组“Source Group 1”，鼠标右键选择“Add Files to Group ‘Source Group 1’...”，然后选择

对应的 xxx.c 文件，点击 Add 键后在点击 Close 关闭。这时工程中就显示已加入的文件。具体操作如下：



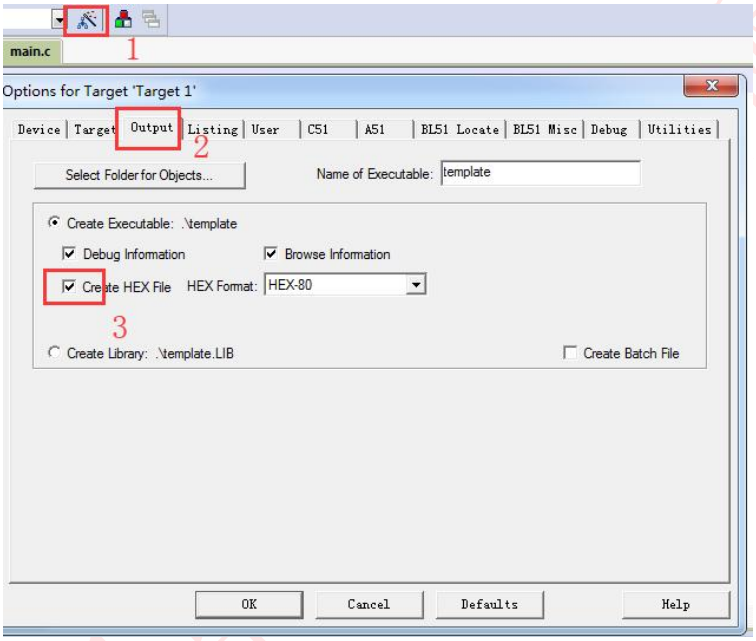




8.1.4 配置魔术棒选项卡

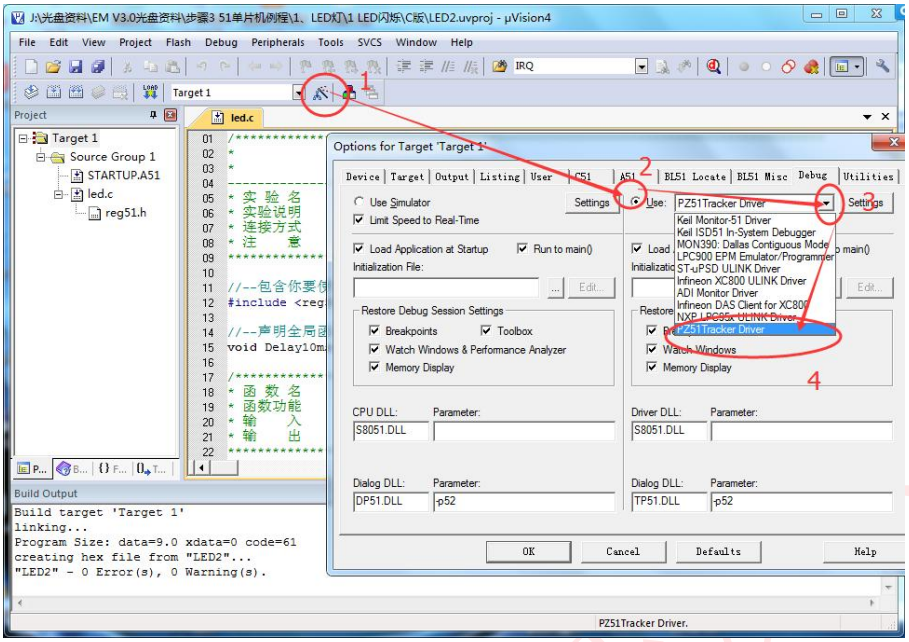
这一步的配置工作非常重要，很多人自己编写程序编译后发现找不到 .HEX 文件，或者使用我们 51 仿真器的时候出现仿真失败，这些问题都是在这个地方没有配置好导致的。

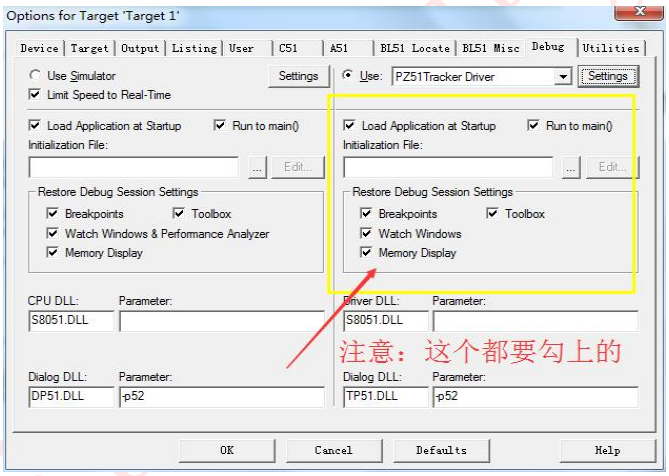
1，选择魔术棒工具的 Output 选项卡，勾选红色框 3，即程序编译成功后会产生 HEX 文件。具体操作如下：

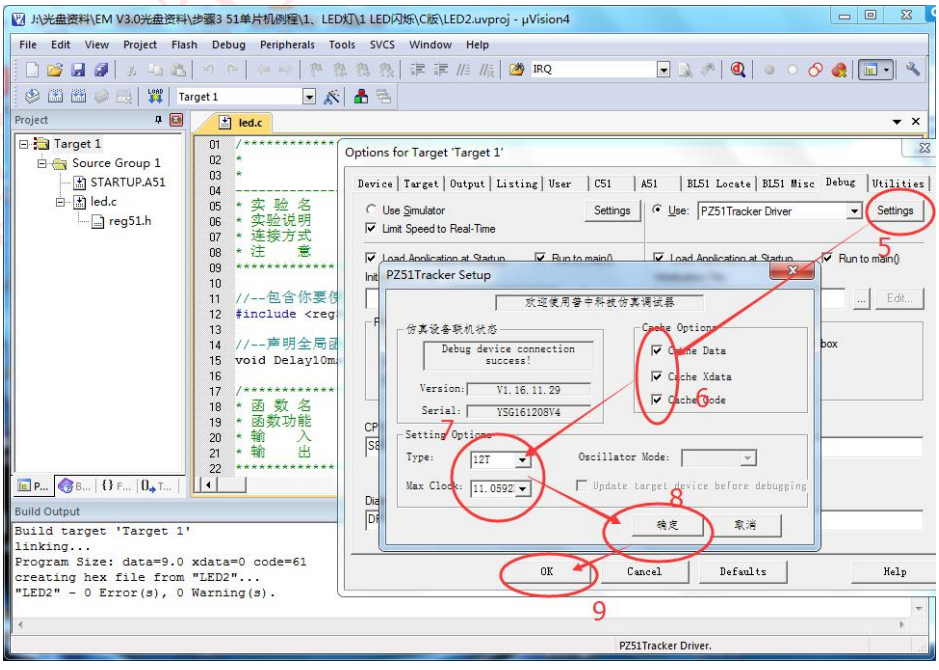


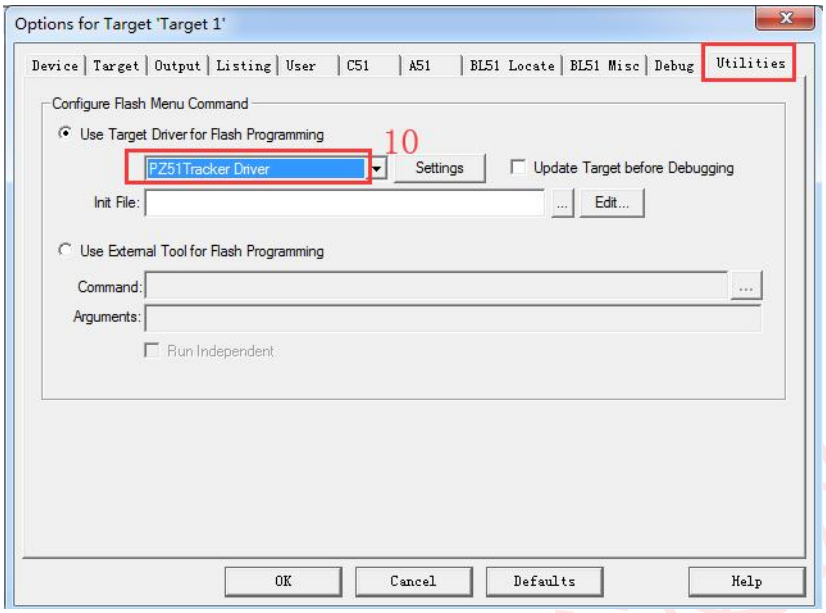
2，51 仿真器配置

当购买了 51 仿真器且你的仿真器安装好驱动并且连接了电脑和板子，那么就可以开始配置。如果使用我们公司的 51 仿真器的朋友，相关驱动的安装请查看“\1--用户必看\4--普中 51 仿真器使用说明”教程。将 51 仿真器直接插在单片机卡座上，仿真器采用防呆设计，方向错了没法卡紧。仿真器一端连接电脑，同时仿真器也给开发板供电，所以使用仿真器时可不接电源线。要使用 51 仿真器在线调试程序，需要在 KEIL 软件内进行相应的配置。具体的配置如下（这里已经将普中 51 仿真器连接至开发板）：


J:\光盘资料\EM V3.0光盘资料\步骤3 51单片机例程\1、LED灯\1 LED闪烁\C版\LED2.uvproj - µVision4
Options for Target 'Target 1'
Use Simulator
Limit Speed to Real-Time
Use: PZ51Tracker Driver
Settings
Load Application at Startup
Run to main()
Initialization File:
Restore Debug Session Settings
Breakpoints
Toolbox
Watch Windows & Perfomance Analyzer
Memory Display
CPU DLL:
Parameter:
S8051.DLL
Driver DLL:
Dialog DLL:
DP51.DLL
TP51.DLL
-p52
OK
Cancel
Defaults
Help
Build Output
Build target 'Target 1'
linking...
Program Size: data=9.0 xdata=0 code=61
creating hex file from "LED2"...
"LED2" - 0 Error(s), 0 Warning(s).
PZ51Tracker Driver.


Options for Target 'Target 1'
Use Simulator
Limit Speed to Real-Time
Use: PZ51Tracker Driver
Settings
Load Application at Startup
Run to main()
Initialization File:
Restore Debug Session Settings
Breakpoints
Toolbox
Watch Windows & Perfomance Analyzer
Watch Windows
Memory Display
CPU DLL:
Parameter:
S8051.DLL
Driver DLL:
DP51.DLL
TP51.DLL
-p52
注意：这个都要勾上的
OK
Cancel
Defaults
Help


J:\光盘资料\EM V3.0光盘资料\步骤3 51单片机例程\1、LED灯\1 LED闪烁\C版\LED2.uvproj - µVision4
Options for Target 'Target 1'
PZ51Tracker Setup
Debug device connection success!
Version:
Serial:
Cache Data
Cache Xdata
Cache Code
Type:
Max Clock:
Oscillator Mode:
OK
Cancel
Defaults
Help
Build Output
Build target 'Target 1'
linking...
Program Size: data=9.0 xdata=0 code=61
creating hex file from "LED2"...
"LED2" - 0 Error(s), 0 Warning(s).
PZ51Tracker Driver.

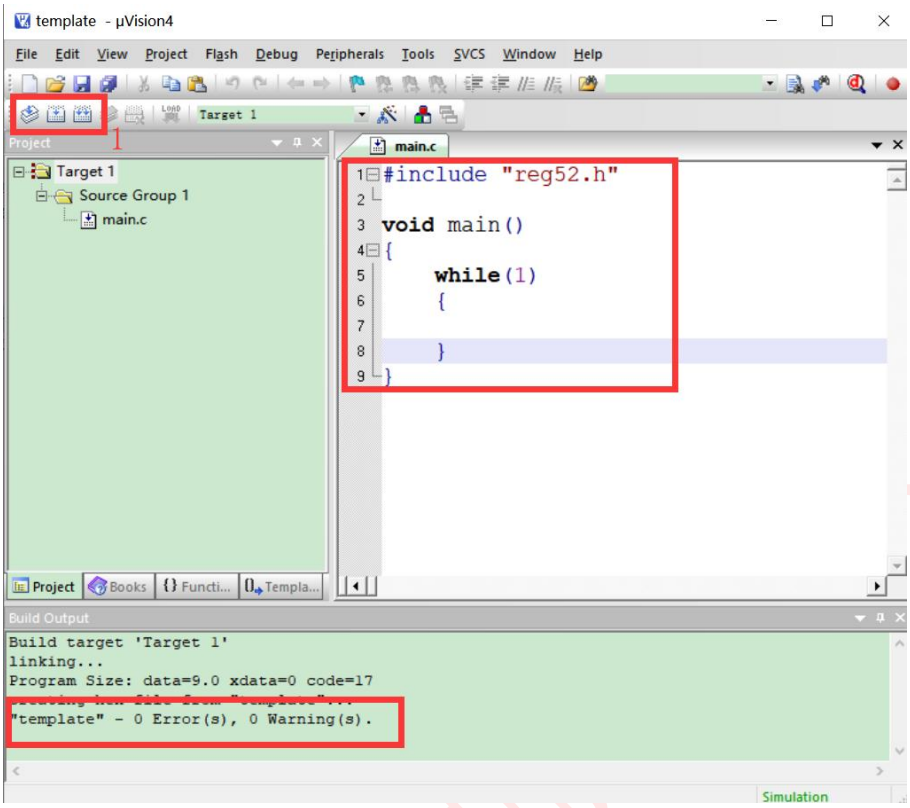


根据上图从 1 到 10 步骤设置完成之后就可以进行在线仿真调试了。

最后点击 OK 按钮即可。然后双击工程组中的 main.c 文件， 我们输入如下内容：

```
#include "reg52.h"

void main()
{
    while(1)
    {

    }
}
```

编译后结果 0 错误 0 警告，表明我们创建的 51 单片机工程完全正确。如下：



上图红色标记 1 中三个按钮都可以对代码编译，但其功能不同：

第一个：它是编译当前界面所在的一个 C 文件。

第二个：联合编译整个工程，发生修改的文件重新编译，并生成可执行文件。

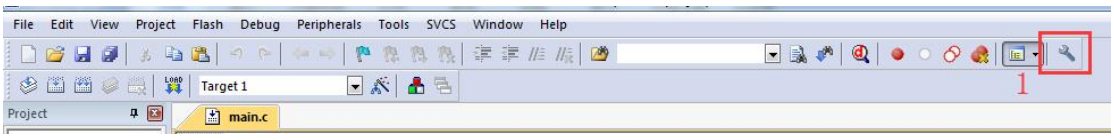
第三个：联合编译整个工程，所有文件都重新编译，并生成可执行文件。

从上面说明可以看出，一般情况下，我们不使用第三个编译按钮，第二个最常用。因为第三个每次都把所有的文件都重新编译，需要消耗大量的时间。 当工程比较小的时候，感觉不到，但是工程较大的时候就不行了。

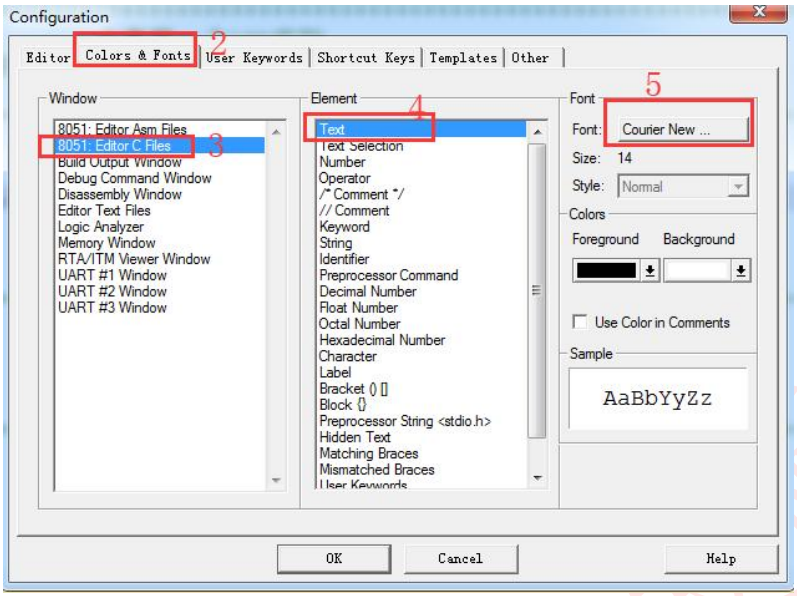
8.1.5 修改编程区内字体大小

有朋友可能会觉得，KEIL C51 默认提供的字体比较小，看起来非常不舒服。下面我们就来介绍如何在 KEIL C51 软件内修改编程区字体大小问题。

1，首先在 KEIL C51 软件的工具栏中内找到一个“小扳手”，如下所示：



2，然后点击它，弹出如下设置界面，按照设置界面的红色标号顺序进入。



3，点击标号 5 按钮时用于选择设置当前字体大小等信息的，选择适合自己的字体大小即可，然后点击 OK，设置完成。如下所示：

最后设置完成后，编程区内的字体就会发生改变。

更多有关 KEIL C51 软件的使用小技巧大家可以通过百度了解学习。

8.2 reg52.h 头文件介绍

8.2.1 头文件作用

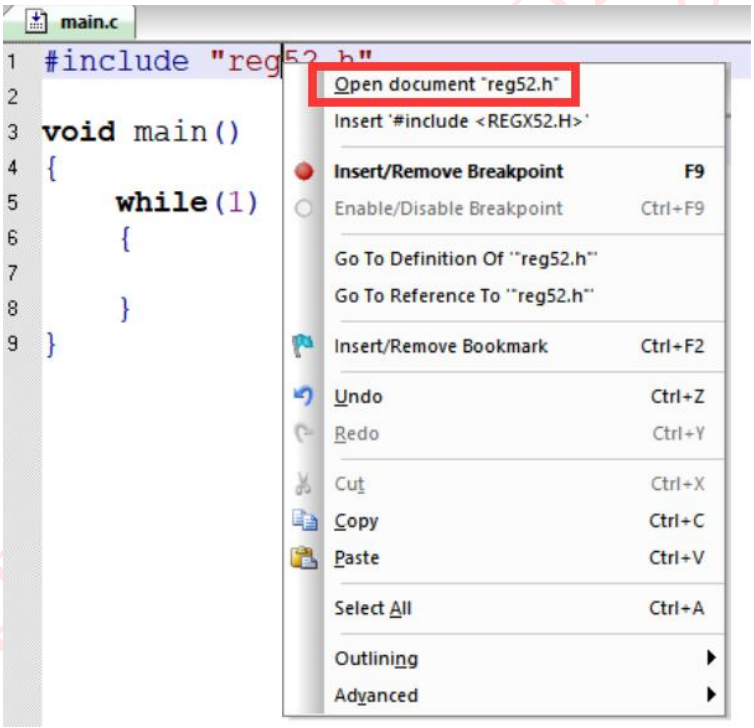
在代码中引用头文件，其实际意义就是将这个头文件中的全部内容放到引用头文件的位置处，免去我们每次编写同类程序都要将头文件中的语句重复编写。在代码中加入头文件有两种书写方法，分别为#include <reg52.h>和#include "reg52.h"，包含头文件时都不需要在后面加分号。两种写法的区别如下：

①当使用<>包含头文件时，编译器先进入到软件安装文件夹处开始搜索这个头文件，也就是 KEILC51/C51/INC 这个文件夹下，如果这个文件夹没有引用的头文件，编译器将会报错。

②当使用""包含头文件时，编译器先进入到当前工程所在文件夹处开始搜索该头文件，如果当前工程所在文件夹下没有该头文件，编译器将继续回到软件安装文件夹处搜索这个头文件，若找不到该头文件，编译器将报错。

8.2.2 头文件内容

打开该头文件查看其内容，将鼠标移动到 reg52.h 上，单击右键，选择“Open document <reg52.h>”，即可打开该头文件，如下：



以后若需打开工程中的其他头文件，也可采用这种方式，或者将工程编译后，通过右边工程组“+”下拉找到对应头文件，双击打开。如下：

头文件内容如下：

```
/*--------------------------------------------------------------------------
REG52.H

Header file for generic 80C52 and 80C32 microcontroller.
Copyright (c) 1988-2002 Keil Elektronik GmbH and Keil Software, Inc.
All rights reserved.
--------------------------------------------------------------------------*/

#ifndef __REG52_H__
#define __REG52_H__

/*  BYTE Registers  */
sfr P0    = 0x80;
sfr P1    = 0x90;
sfr P2    = 0xA0;
sfr P3    = 0xB0;
sfr PSW   = 0xD0;
sfr ACC   = 0xE0;
sfr B     = 0xF0;
sfr SP    = 0x81;
sfr DPL   = 0x82;
sfr DPH   = 0x83;
sfr PCON  = 0x87;
sfr TCON  = 0x88;
sfr TMOD  = 0x89;
sfr TL0   = 0x8A;
sfr TL1   = 0x8B;
sfr TH0   = 0x8C;
sfr TH1   = 0x8D;
sfr IE    = 0xA8;
sfr IP    = 0xB8;
sfr SCON  = 0x98;
sfr SBUF  = 0x99;

/*  8052 Extensions  */
sfr T2CON  = 0xC8;
sfr RCAP2L = 0xCA;
sfr RCAP2H = 0xCB;
sfr TL2    = 0xCC;
sfr TH2    = 0xCD;


/*  BIT Registers  */
/*  PSW  */
```

```
sbit CY    = PSW^7;
sbit AC    = PSW^6;
sbit F0    = PSW^5;
sbit RS1   = PSW^4;
sbit RS0   = PSW^3;
sbit OV    = PSW^2;
sbit P     = PSW^0; //8052 only

/*  TCON  */
sbit TF1   = TCON^7;
sbit TR1   = TCON^6;
sbit TF0   = TCON^5;
sbit TR0   = TCON^4;
sbit IE1   = TCON^3;
sbit IT1   = TCON^2;
sbit IE0   = TCON^1;
sbit IT0   = TCON^0;

/*  IE  */
sbit EA    = IE^7;
sbit ET2   = IE^5; //8052 only
sbit ES    = IE^4;
sbit ET1   = IE^3;
sbit EX1   = IE^2;
sbit ET0   = IE^1;
sbit EX0   = IE^0;

/*  IP  */
sbit PT2   = IP^5;
sbit PS    = IP^4;
sbit PT1   = IP^3;
sbit PX1   = IP^2;
sbit PT0   = IP^1;
sbit PX0   = IP^0;

/*  P3  */
sbit RD    = P3^7;
sbit WR    = P3^6;
sbit T1    = P3^5;
sbit T0    = P3^4;
sbit INT1  = P3^3;
sbit INT0  = P3^2;
sbit TXD   = P3^1;
sbit RXD   = P3^0;
```

```
/*  SCON  */
sbit SM0   = SCON^7;
sbit SM1   = SCON^6;
sbit SM2   = SCON^5;
sbit REN   = SCON^4;
sbit TB8   = SCON^3;
sbit RB8   = SCON^2;
sbit TI    = SCON^1;
sbit RI    = SCON^0;

/*  P1  */
sbit T2EX  = P1^1; // 8052 only
sbit T2    = P1^0; // 8052 only

/*  T2CON  */
sbit TF2    = T2CON^7;
sbit EXF2   = T2CON^6;
sbit RCLK   = T2CON^5;
sbit TCLK   = T2CON^4;
sbit EXEN2  = T2CON^3;
sbit TR2    = T2CON^2;
sbit C_T2   = T2CON^1;
sbit CP_RL2 = T2CON^0;

#endif
```

从上面代码中可以看到，该头文件中定义了 52 系列单片机内部所有的功能寄存器，用到了前面讲到的 sfr 和 sbit 这两个关键字，“sfr P0=0x80;”语句的意义是，把单片机内部地址 0x80 处的这个寄存器重新起名叫 P0，以后我们在程序中可直接操作 P0，就相当于直接对单片机内部的 0x80 地址处的寄存器进行操作。通俗点讲就是通过 sfr 这个关键字，让 Keil 编译器在单片机与人之间搭建一条可以进行沟通的桥梁，我们操作的是 P0 口，而单片机本身并不知道什么是 P0 口，但是它知道它内部地址 0x80 是什么东西。说道这我想大家应该明白了，以后凡是编写 51 内核单片机程序时，我们再源代码的第一行就可以直接包含该头文件。

在上面我们还看到，“sbit CY=PSW^7;”语句的意思是，将 PSW 这个寄存器的最高位重新命名为 CY，以后我们要单独操作 PSW 寄存器的最高位时，便可直

接操作 CY，其他雷同。

课后作业

（1）按照上述方法创建一个自己的 51 单片机工程模板。

普中51单片机开发板

第 9 章 点亮第一个 LED

不论学习什么单片机，最简单的外设莫过于 IO 口的高低电平控制，本章将向大家介绍如何在创建好的工程模板上，通过控制 51 单片机的 GPIO 使开发板上的 LED 灯点亮。通过本章的学习，让大家学会如何在程序中操作 51 单片机的 GPIO 口输出高低电平。本章分为如下几部分内容：

9.1 51 单片机 GPIO 介绍

9.2 LED 简介

9.3 硬件设计

9.4 软件设计

9.5 实验现象

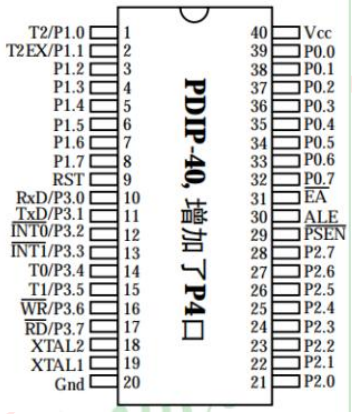
普中51单片机开发板

9.1 51 单片机 GPIO 介绍

9.1.1 GPIO 概念

GPIO（general purpose intput output）是通用输入输出端口的简称，可以通过软件来控制其输入和输出。51 单片机芯片的 GPIO 引脚与外部设备连接起来，从而实现与外部通讯、控制以及数据采集的功能。不过 GPIO 最简单的应用还属点亮 LED 灯了，只需通过软件控制 GPIO 输出高低电平即可。当然 GPIO 还可以作为输入控制，比如在引脚上接入一个按键，通过电平的高低判断按键是否按下。

我们开发板上使用的 51 单片机型号是 STC89C52 或 STC89C516，此芯片共有 40 引脚，芯片引脚图如下图所示：



那么是不是所有引脚都是 GPIO 呢？当然不是，51 单片机引脚可以分为这么几大类：

（1）电源引脚：引脚图中的 VCC、 GND 都属于电源引脚。

（2）晶振引脚：引脚图中的 XTAL1、XTAL2 都属于晶振引脚。

（3）复位引脚：引脚图中的 RST/VPD 属于复位引脚，不做其他功能使用。

（4）下载引脚：51 单片机的串口功能引脚（TXD、RXD）可以作为下载引脚使用。

（5） GPIO 引脚：引脚图中带有 Px.x 等字样的均属于 GPIO 引脚。从引脚图可以看出，GPIO 占用了芯片大部分的引脚，共达 32 个，分为了 4 组，P0、P1、P2、P3，每组为 8 个 IO，而且在 P3 组中每个 IO 都具备额外功能，只要通过相应的寄存器设置即可配置对应的附加功能，同一时刻，每个引脚只能使用该引脚

的一个功能。

对于这么多 GPIO 管脚，我们怎么知道具体某个引脚有什么功能呢？很简单，可以查阅 STC89CXX 芯片数据手册获取信息，数据手册在我们资料“\6--芯片资料\开发板芯片数据手册”内，里面有一个 STC89CXX 数据手册.pdf。里面的第 23 页中就有介绍，我们截取了一部分内容如下图所示：

管脚	管脚编号			说明	
	LQFP44	PDIP40	PLCC44		
P0.0 ~ P0.7	37-30	39-32	43~36	P0:P0口既可作为输入/输出口，也可作为地址/数据复用总线使用。当P0口作为输入/输出口时，P0是一个8位准双向口，上电复位后处于开漏模式。P0口内部无上拉电阻，所以作I/O口必须外接10K-4.7K的上拉电阻。当P0作为地址/数据复用总线使用时，是低8位地址线[A0~A7]，数据线的[D0~D7]，此时无需外接上拉电阻。	
P1.0/T2	40	1	2	P1.0	标准I/O口 PORT1[0]
				T2	定时器/计数器2的外部输入
P1.1/T2EX	41	2	3	P1.1	标准I/O口 PORT1[1]
				T2EX	定时器/计数器2捕捉/重装方式的触发控制
P1.2	42	3	4	标准I/O口 PORT1[2]	
P1.3	43	4	5	标准I/O口 PORT1[3]	
P1.4	44	5	6	标准I/O口 PORT1[4]	
P1.5	1	6	7	标准I/O口 PORT1[5]	
P1.6	2	7	8	标准I/O口 PORT1[6]	

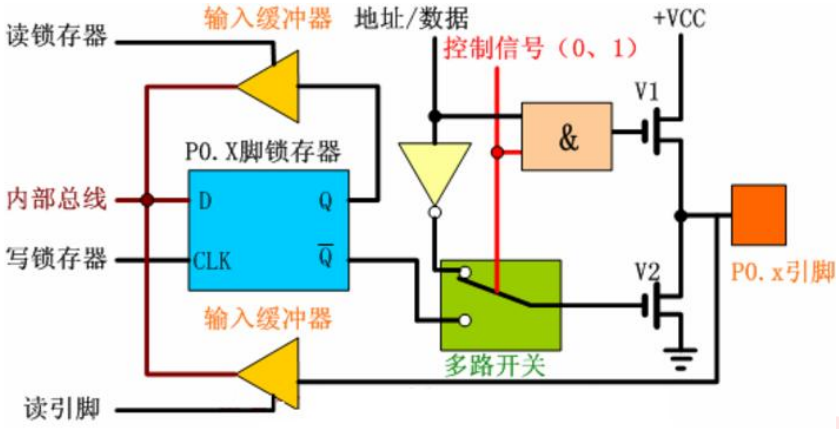
从上图中我们可以获取引脚的名字和引脚功能等信息。这个我们开发板芯片原理图内已经将引脚所有功能都标进去了，所以后面也不需要查找具体引脚有什么功能，直接看原理图即可。

9.1.2 GPIO 结构框图与工作原理

我们使用的 51 单片机 GPIO 分为 P0、P1、P2 和 P3 口，下面分别来介绍其内部结构框图与工作原理。

9.1.2.1 P0 端口

P0 端口含有 8 位引脚，下图为其中一个，其它几个与之完全一致，因此只需了解当中一个即可。如下图所示：



由上图可见，P0 端口由锁存器、输入缓冲器、切换开关、一个非门、一个与非门及场效应管驱动电路构成。再看图的最右边，标号为 P0.x 引脚的图标，也就是说 P0.x 引脚可以是 P0.0 到 P0.7 的任何一位，即在 P0 口有 8 个与上图相同的电路组成。

下面，我们先就组成 P0 口的每个单元部份跟大家介绍一下：

①输入缓冲器

在 P0 口中，有两个三态的缓冲器，在学数字电路时，我们已知道，三态门有三个状态，即在输出端可以是高电平、低电平，同时还有一种就是高阻状态（或称为禁止状态），大家看上图，上面一个是读锁存器的缓冲器，也就是说，要读取 D 锁存器输出端 Q 的数据，那就得使读锁存器的这个缓冲器的三态控制端（上图中标号为‘读锁存器’端）有效。下面一个是读引脚的缓冲器，要读取 P0.x 引脚上的数据，也要使标号为‘读引脚’的这个三态缓冲器的控制端有效，引脚上的数据才会传输到我们单片机的内部数据总线上。

②D 锁存器

构成一个锁存器，通常要用一个时序电路，时序的单元电路在学数字电路时我们已知道，一个触发器可以保存一位的二进制数（即具有保持功能），在 51 单片机的 32 根 I/O 口线中都是用一个 D 触发器来构成锁存器的。大家看上图中的 D 锁存器，D 端是数据输入端，CP（CLK）是控制端（也就是时序控制信号输入端），Q 是输出端，Q 非是反向输出端。

对于 D 触发器来讲，当 D 输入端有一个输入信号，如果这时控制端 CP 没有信号（也就是时序脉冲没有到来），这时输入端 D 的数据是无法传输到输出端 Q 及反向输出端 Q 非的。如果时序控制端 CP 的时序脉冲一旦到了，这时 D 端输入

的数据就会传输到 Q 及 Q 非端。数据传送过来后，当 CP 时序控制端的时序信号消失了，这时，输出端还会保持着上次输入端 D 的数据（即把上次的数据锁存起来了）。如果下一个时序控制脉冲信号来了，这时 D 端的数据才再次传送到 Q 端，从而改变 Q 端的状态。

③多路开关

在 51 单片机中，当内部的存储器够用（也就是不需要外扩展存储器时，这里讲的存储器包括数据存储器及程序存储器）时，P0 口可以作为通用的输入输出端口（即 I/O）使用，对于 8031（内部没有 ROM）的单片机或者编写的程序超过了单片机内部的存储器容量，需要外扩存储器时，P0 口就作为‘地址/数据’总线使用。那么这个多路选择开关就是用于选择是做为普通 I/O 口使用还是作为‘数据/地址’总线使用的选择开关了。大家看上图，当多路开关与下面接通时，P0 口是作为普通的 I/O 口使用的，当多路开关是与上面接通时，P0 口是作为‘地址/数据’总线使用的。

④场效应管输出驱动

从上图中可以看出，P0 口的输出是由两个 MOS 管组成的推拉式结构，也就是说，这两个 MOS 管一次只能导通一个，当 V1 导通时，V2 就截止，当 V2 导通时，V1 截止。

⑤与非门、非门

这个在学习数字电路时也很好理解，如果没有数字电路基础的用户，可以百度查找与非门、非门以及前面的 D 触发器详细了解，这里就不再过多叙述。当然如果搞不明白这些也不会影响后续我们学习 51 单片机编程，大家也可以忽略。

前面我们已将 P0 口的各单元部件进行了一个详细的讲解，下面我们就来研究一下 P0 口做为 I/O 口及地址/数据总线使用时的具体工作过程。

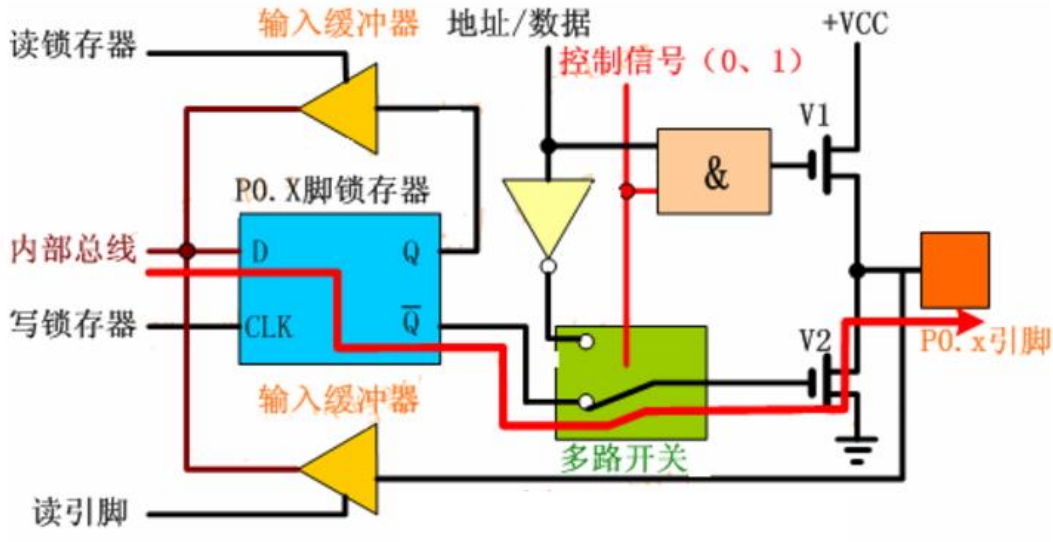
（1）作为 I/O 端口输出使用时的工作原理

P0 口作为 I/O 端口使用时，多路开关的控制信号为 0（低电平），看上图中的红线部份，多路开关的控制信号同时与与非门的一个输入端是相接的，我们知道与门的逻辑特点是“全 1 出 1，有 0 出 0”那么控制信号是 0 的话，这时与门输出的也是一个 0（低电平），与门的输出是 0，V1 管就截止，在多路控制开关的控制信号是 0（低电平）时，多路开关是与锁存器的 Q 非端相接的（即 P0 口

作为 I/O 口线使用）。

P0 口用作 I/O 口线，其由数据总线向引脚输出（即输出状态 Output）的工作过程：当写锁存器信号 CP 有效，数据总线的信号→锁存器的输入端→D 锁存器的反向输出 Q 非端→多路开关→V2 管的栅极→V2 的漏极到输出端 P0.X。前面我们已讲了，当多路开关的控制信号为低电平 0 时，与门输出为低电平，V1 管是截止的，所以作为输出口时，P0 是漏极开路输出，类似于 OC 门，当驱动上接电流负载时，需要外接上拉电阻。

下图就是由内部数据总线向 P0 口输出数据的流程图（红色箭头）：

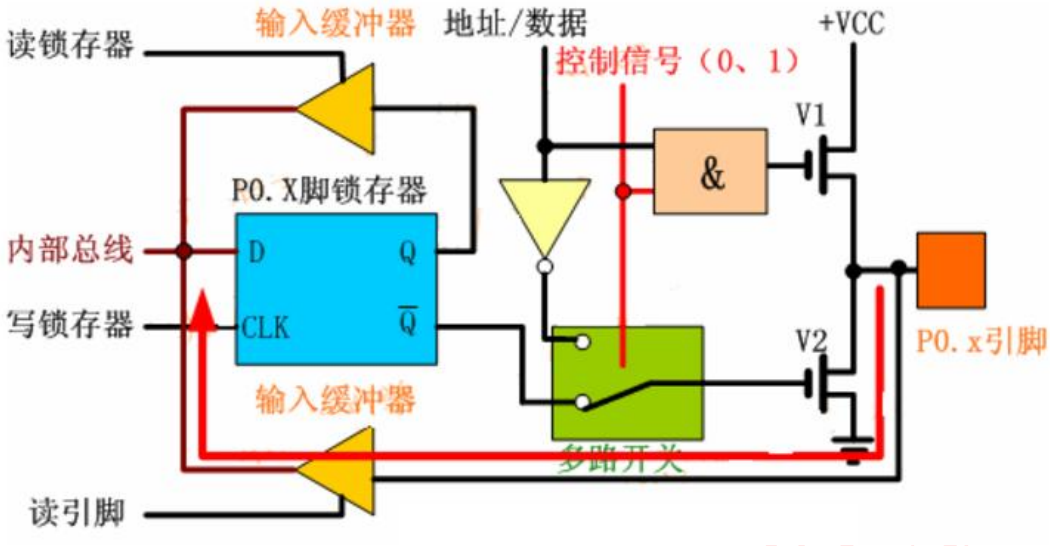


（2）作为 I/O 端口输入使用时的工作原理

数据输入时（读 P0 口）有两种情况：

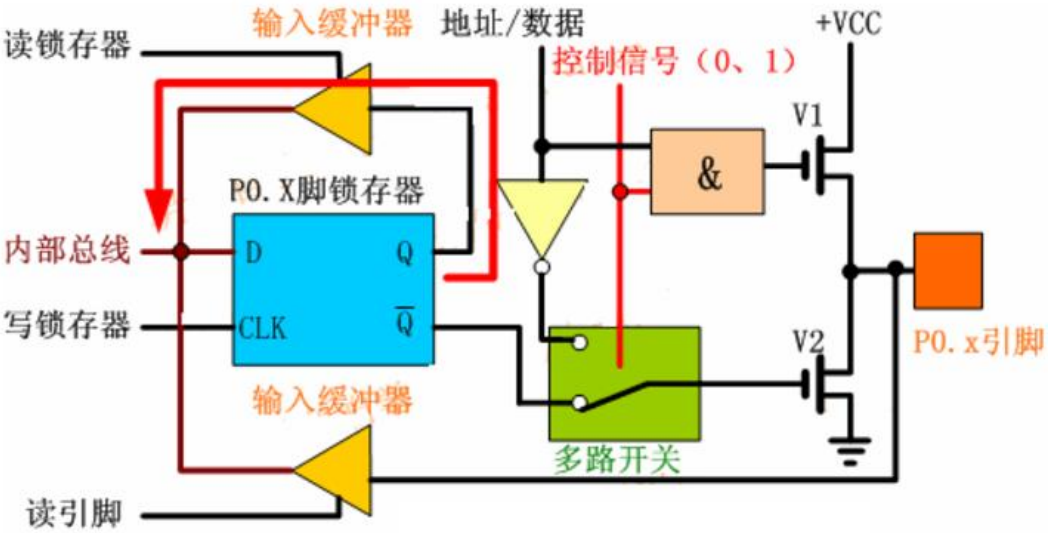
1、读引脚

读芯片引脚上的数据，读引脚数时，读引脚缓冲器打开（即三态缓冲器的控制端要有效），通过内部数据总线输入，请看下图（红色箭头）。



2、读锁存器

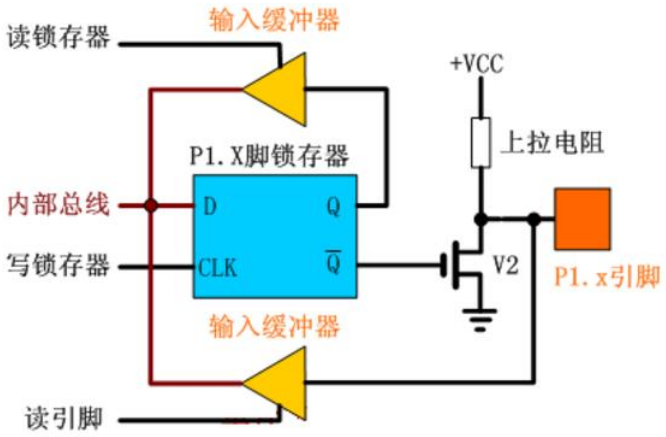
通过打开读锁存器三态缓冲器读取锁存器输出端 Q 的状态，请看下图（红色箭头）



因为现在 STC 51 单片机内存已经足够使用，所以也用不到通过 P0 口外扩存储器，对于 P0 口作为外扩存储器时的工作原理这里就不叙述，如需了解的朋友可以上网百度。

9.1.2.2 P1 端口

P1 口的结构最简单，用途也单一，仅作为数据输入/输出端口使用。输出的信息有锁存，输入有读引脚和读锁存器之分。P1 端口的一位结构见下图：



由图可见，P1 端口与 P0 端口的主要差别在于，P1 端口用内部上拉电阻 R 代替了 P0 端口的场效应管 V1，并且输出的信息仅来自内部总线。由内部总线输出的数据经锁存器反相和场效应管反相后，锁存在端口线上，所以，P1 端口是具有输出锁存的静态口。

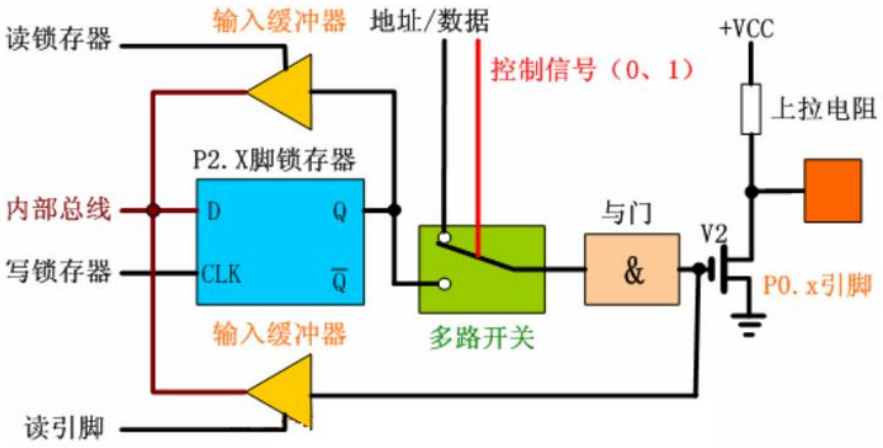
由上图可见，要正确地从引脚上读入外部信息，必须先使场效应管关断，以便由外部输入的信息确定引脚的状态。为此，在作引脚读入前，必须先对该端口写入 1。具有这种操作特点的输入/输出端口，称为准双向 I/O 口。8051 单片机的 P1、P2、P3 都是准双向口。P0 端口由于输出有三态功能，输入前，端口线已处于高阻态，无需先写入 1 后再作读操作。

P1 口的结构相对简单，前面我们已详细的分析了 P0 口，只要大家认真的分析了 P0 口的工作原理，P1 口我想大家都有能力去分析，这里我就不多论述了。

单片机复位后，各个端口已自动地被写入了 1，此时，可直接作输入操作。如果在应用端口的过程中，已向 P1 一 P3 端口线输出过 0，则再要输入时，必须先写 1 后再读引脚，才能得到正确的信息。此外，随输入指令的不同，P1 端口也有读锁存器与读引脚之分。

9.1.2.3 P2 端口

P2 端口的一位结构见下图：



由图可见，P2 端口在片内既有上拉电阻，又有切换开关 MUX，所以 P2 端口在功能上兼有 P0 端口和 P1 端口的特点。这主要表现在输出功能上，当切换开关向下接通时，从内部总线输出的一位数据经反相器和场效应管反相后，输出在端口引脚线上；当多路开关向上时，输出的一位地址信号也经反相器和场效应管反相后，输出在端口引脚线上。

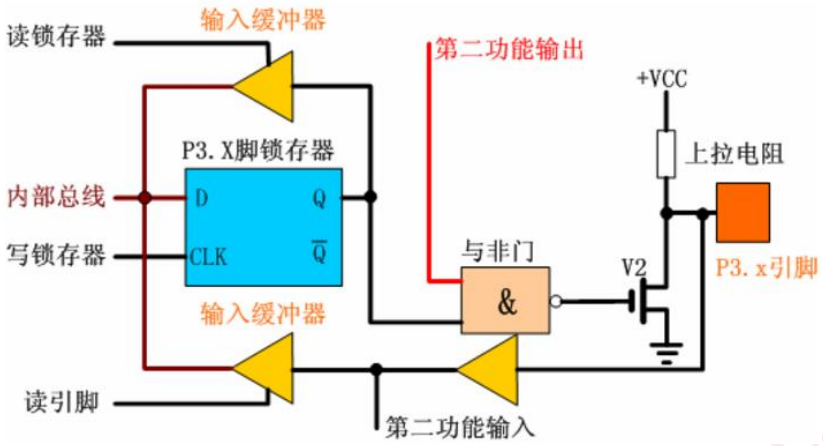
对于 8031 单片机必须外接程序存储器才能构成应用电路（或者我们的应用电路扩展了外部存储器），而 P2 端口就是用来周期性地输出从外存中取指令的地址(高 8 位地址)，因此，P2 端口的多路开关总是在进行切换，分时地输出从内部总线来的数据和从地址信号线上来的地址。因此 P2 端口是动态的 I/O 端口。输出数据虽被锁存，但不是稳定地出现在端口线上。其实，这里输出的数据往往也是一种地址，只不过是外部 RAM 的高 8 位地址。

P2 口既可作为 I/O 口使用，也可作为地址总线使用，通常主要用作 I/O 口使用，地址总线使用不作分析。

P2 口的结构相对简单，前面我们已详细的分析了 P0 和 P1 口，只要大家认真的分析了它们的工作原理，P2 口我想大家都有能力去分析，这里我就不多论述了。

9.1.2.4 P3 端口

P3 口是一个多功能口，它除了可以作为 I/O 口外，还具有第二功能，P3 端口的一位结构见下图：



由上图可见，P3 端口和 P1 端口的结构相似，区别仅在于 P3 端口的各端口线有两种功能选择。当处于第一功能时，第二输出功能线为 1，此时，内部总线信号经锁存器和场效应管输入/输出，其作用与 P1 端口作用相同，也是静态准双向 I/O 端口。当处于第二功能时，锁存器输出 1，通过第二输出功能线输出特定的内含信号，在输入方面，即可以通过缓冲器读入引脚信号，还可以通过替代输入功能读入片内的特定第二功能信号。由于输出信号锁存并且有双重功能，故 P3 端口为静态双功能端口。有关 P3 口第二功能，在前面章节芯片管脚功能定义已经讲解过，此处不再重复。

至此，我们就把 51 单片机的 P0、P1、P2 和 P3 口内部结构及原理讲解完，可能有的朋友会很懵、看不懂，没关系，这些都不会影响你编写单片机应用程序，大家记住以下几点即可：

①P0 口是漏极开路，要使其输出高电平，必须外接上拉电阻，通常选择 4.7K~10K 阻值。

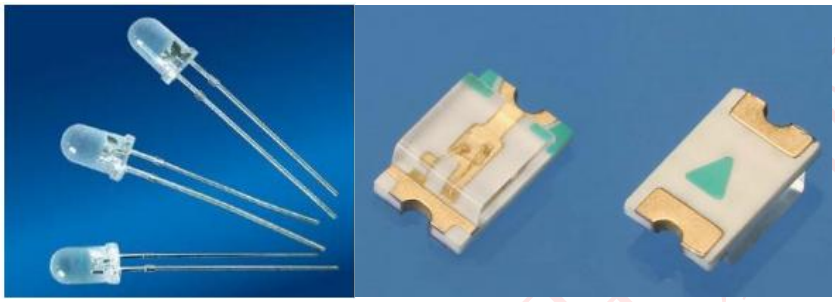
②P0、P1、P2 几乎都用作普通 I/O 口使用，既可作为输入，又可作为输出。

③P3 口既可用作普通 I/O 口，又可作为第二功能使用，比如串口、外部中断、计数器等。

9.2 LED 简介

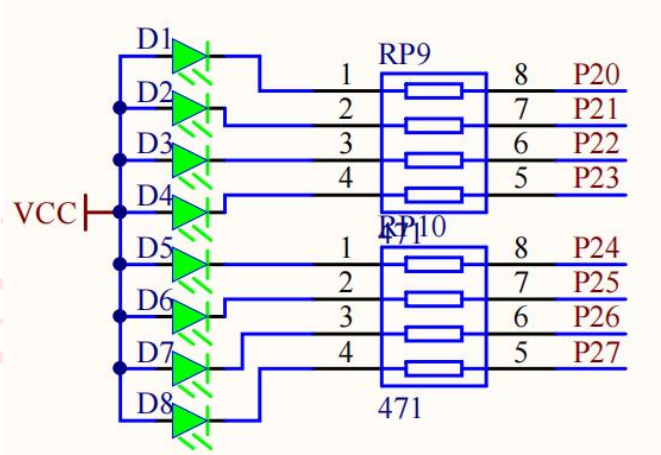
LED 即发光二极管。它具有单向导电性，通过 5mA 左右电流即可发光，电流越大，其亮度越强，但若电流过大，会烧毁二极管，一般我们控制在 3 mA-20mA 之间，通常我们会在 LED 管脚上串联一个电阻，目的就是为了限制通过发光二极

管的电流不要太大，因此这些电阻又可以称为“限流电阻”。当发光二极管发光时，测量它两端电压约为 1.7V，这个电压又叫做发光二极管的“导通压降”。下图左右分别为直插式发光二极管和贴片式发光二极管实物图。发光二极管正极又称阳极，负极又称阴极，电流只能从阳极流向阴极。直插式发光二极管长脚为阳极，短脚为阴极。仔细观察贴片式发光二极管正面的一端有彩色标记，通常有标记的一端为阴极。



9.3 硬件设计

开发板上 LED 模块电路如下图所示：



在前面我们介绍过如何查看原理图，相同网络标号表示它们是连接在一起的，因此 D1-D8 连接到单片机的 P20-P27 口。图中 LED 采用共阳接法，即所有 LED 阳极管脚接电源 VCC，阴极管脚通过一个 470 欧的限流电阻接到 P2 口上。根据前面 LED 的介绍我们知道，要让 LED 发光即对应的阴极管脚应该为低电平，若为高电平则熄灭。

如果要想 51 单片机控制 LED，就必须通过单片机管脚在 P2 口上输出低电平。本章所要实现的功能是：点亮 D1 指示灯。

9.4 软件设计

9.4.1 点亮第一个 LED

本章所要实现的功能是：点亮 D1 指示灯，即让 P2.0 管脚输出一个低电平。完成后可再控制 D1 指示灯闪烁，即间隔一段时间点亮和熄灭 D1 指示灯。

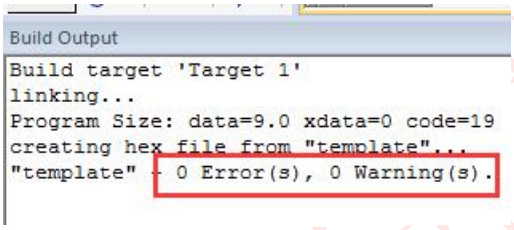
我们直接复制前面创建好的工程模板，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“2-点亮第一个 LED”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*************************************************************************
**********************
实验名称：点亮第一个 LED
接线说明：
实验现象：下载程序后“LED 模块”的 D1 指示灯点亮
注意事项：

*************************************************************************
**********************/
#include "reg52.h"

sbit LED1=P2^0;  //将 P2.0 管脚定义为 LED1

/*************************************************************************
***************
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*************************************************************************
**************/
void main()
{
    LED1=0;    //LED1 端口设置为低电平
    while(1)
    {

    }
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，

然后使用 sbit 关键字来定义 P2.0 管脚，定义好后即可使用 LED1 来替代 P2.0 口的操作。主函数功能非常简单，首先让 LED1 为低电平，即 P2.0 口输出为低电平，D1 指示灯即点亮，然后进入 while 循环，单片机此时一直在 while 内循环操作。当然该条语句也可以放在 while 循环语句内，同样会点亮 D1 指示灯。在 51 单片机程序开发中，main 函数内通常都会有一个 while 循环，在循环体内不断执行我们所要实现的功能。对于仅点亮 LED，可以把控制语句放在 while 之前来执行。

至此，整个程序就编写完成，我们编译一下，如下图所示



可以看到没有错误，也没有警告。从编译信息可以看出，我们的代码占用 FLASH 大小为：19 字节，所用的 SRAM 大小为：9 个字节（9+0）。这里我们解释一下，编译结果里面的几个数据的意义：

code：表示程序所占用 FLASH 的大小。

data：数据储存器内部 RAM 占用大小。

xdata：数据储存器外部 RAM 占用大小。

有了这个就可以知道你当前使用的 flash 和 sram 大小了。一定要注意的是程序的大小不是.hex 文件的大小，而是编译后的 code 和 data 之和。

9.4.2 LED 闪烁实验

如果要实现 LED 闪烁，只需循环让 D1 指示灯先亮一会后熄灭。这里就有一个延时问题，如何来产生延时呢？我们知道单片机执行每条代码指令都是需要时间的，在前面介绍 C 语言时讲解过循环语句，因此只需编写一个循环函数，让 CPU 不干其它事，专门在那循环运行即可实现延时功能。依据人的肉眼余晖效应，延时时间不能太短，否则就无法观察到 LED 闪烁。打开“\4--实验程序\1--基础实验\3-LED 闪烁实验”例程内延时函数如下：

```
/*******************************************************************************
```

```
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*****************************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}
```

上述代码即为延时函数，通过 while 循环来实现。函数入口有一个形式参数 ten_us，如果 ten_us 等于 1，则 while 循环执行一次，调用该函数延时时间大约 10us，当然使用循环来实现延时，这种延时是不精确的，目前我们先得到个大概的时间即可。

细心的朋友可能会看到函数形参 ten_us 是 u16 类型的，这个似乎不是 C 语言数据类型关键字，这是我们重定义的数据类型，如下：

```
typedef unsigned int u16;    //对系统默认数据类型进行重命名
typedef unsigned char u8;
```

使用关键字 typedef 对系统默认数据类型 unsigned int 和 unsigned char 重新命名，主要是方便我们代码的书写和变量类型的查看。u16 即代表该变量是 16 位的无符号整型数据，u8 代表该变量是 8 位的无符号字符型数据。有了这个就知道参数的传送范围，不能超过形参定义的范围。

下面看下 main 函数代码，如下：

```
void main()
{
    while(1)
    {
        LED1=0;    //点亮
        delay_10us(50000); //大约延时 450ms
        LED1=1;    //熄灭
        delay_10us(50000);
    }
}
```

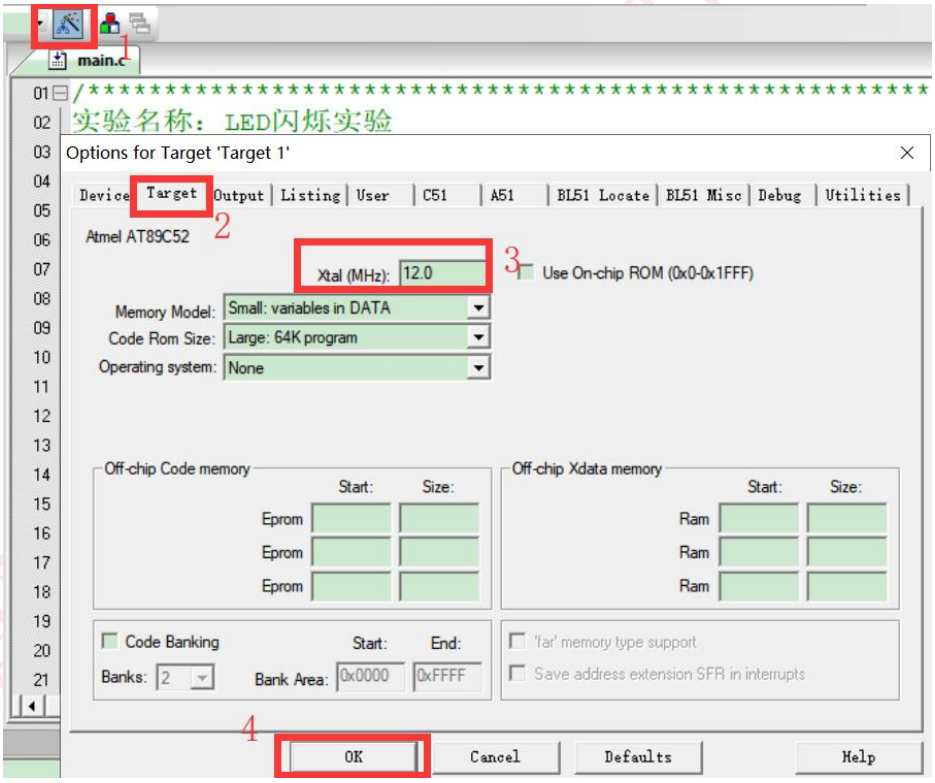
main 函数内实现功能很简单，在 while 循环内不断间隔一定时间点亮 LED1 和熄灭 LED1，这样即可实现 D1 指示灯闪烁。细心的朋友可能会问，前面

delay_10us 函数形参为 1 时大约是 10us，那现在实参传输 50000，不应该是 500ms 吗，为什么注释写的是 450ms 呢？这里还得回到刚才话题，使用循环来延时只是获得一个大概的时间，并不能精确，如需精确延时，后期我们会学习定时器。此处就不用纠结这个问题。

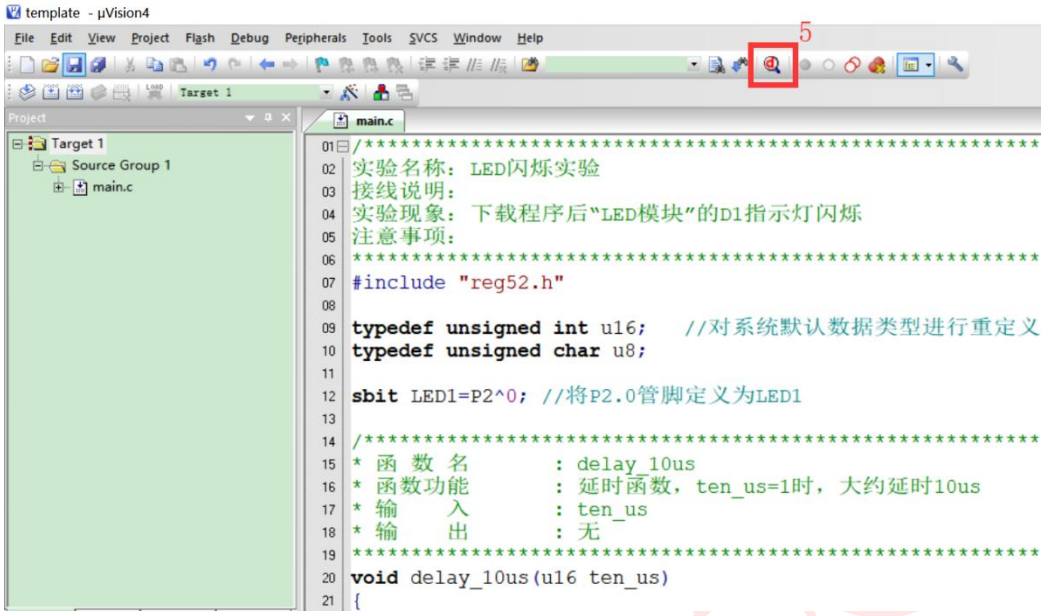
9.4.2.1 通过 KEIL 软件仿真查看延时时间

上述代码中我们传递实参是 50000，得到的延时大约是 450ms，如何来验证呢？可以通过 KEIL 自带的软件仿真功能，操作如下：

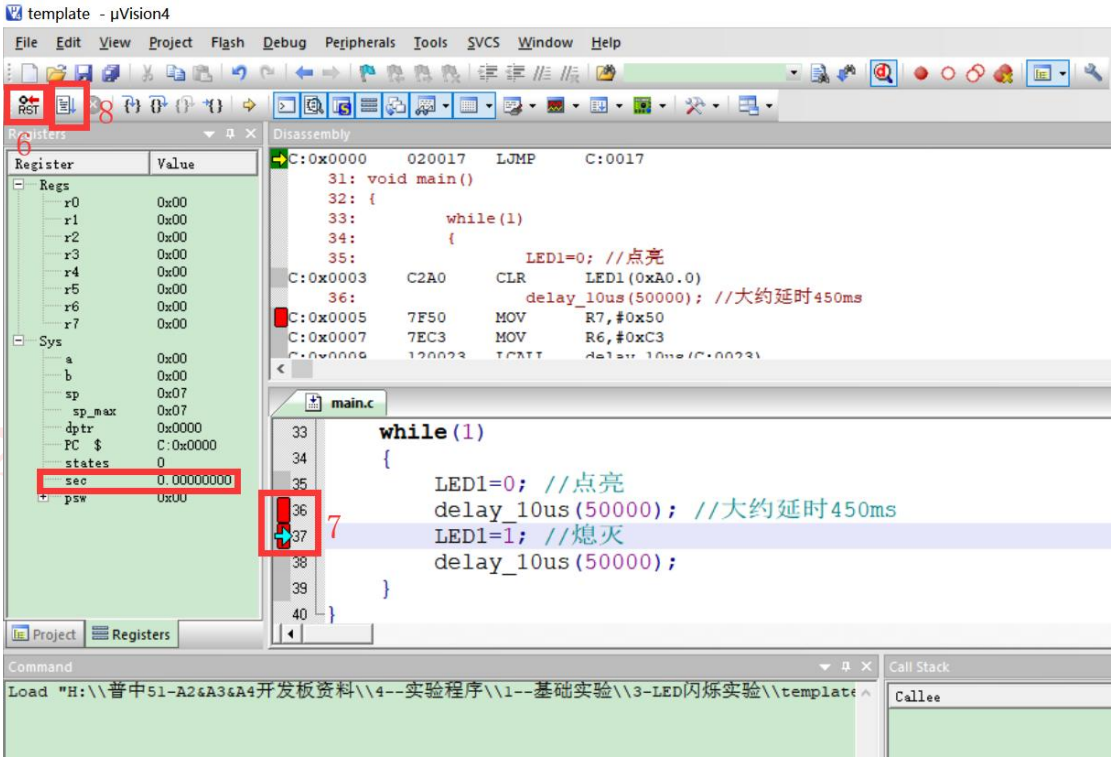
①打开实验工程，点击魔术棒，选择“Target”选项卡，在 Xtal(MHz)文本框中输入 12M，该值表示开发板上实际使用外部晶振大小，如果开发板上使用外部晶振是 11.0592M，则修改为对应值。然后点击 OK。



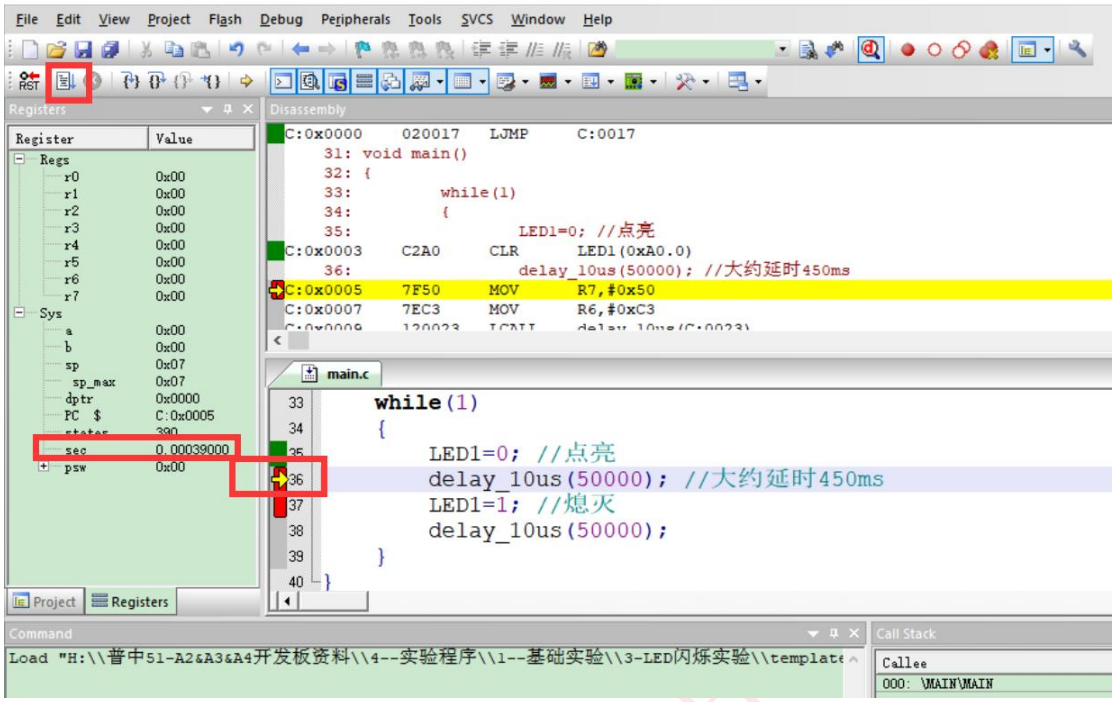
②点击仿真按钮，进入仿真界面，如下所示：



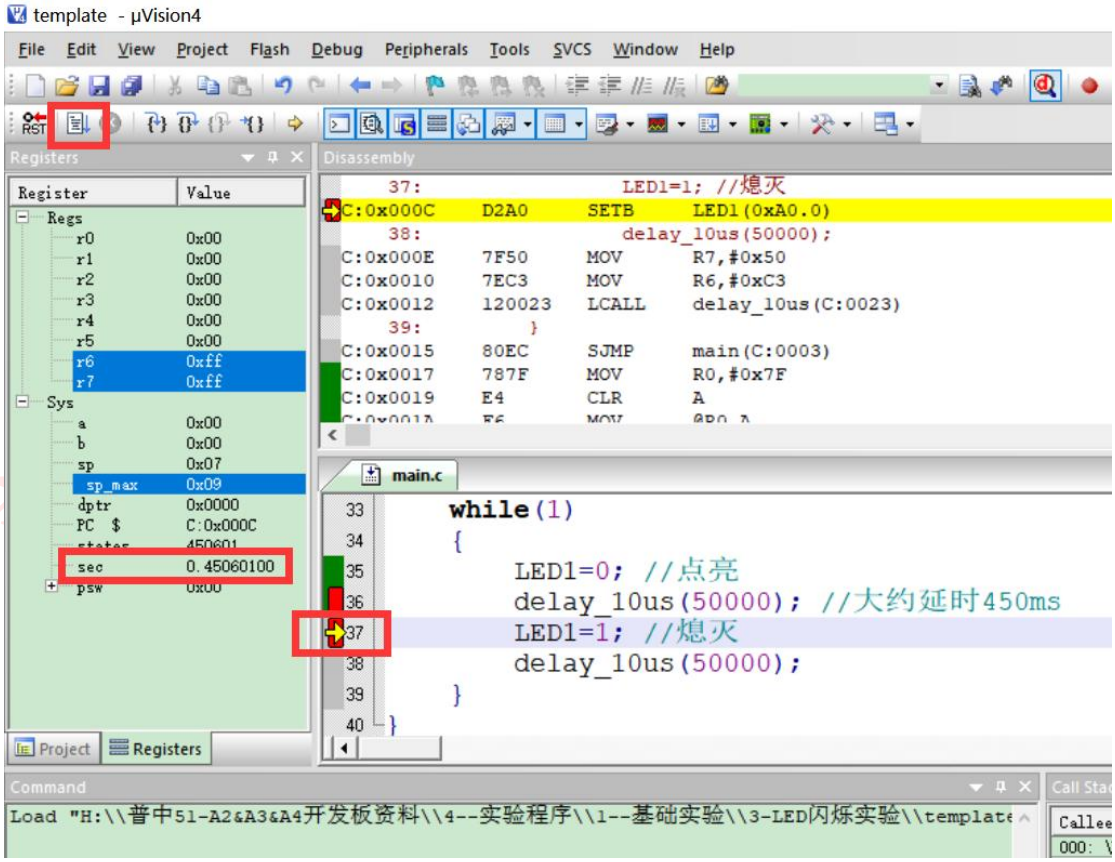
③点击 RST 按钮，重新复位系统参数，此时参数列表中 sec 则为 0，然后在所要查看调试的代码数字前面用鼠标左键双击即可出现“红色块”，我们称之为断点。如果再次双击，即可取消该断点。当点击红色标记 8 运行的时候就能直接运行到我们设置的断点处。



④当点击红色标记 8 运行时，可以看到黄色箭头直接定位到 36 行代码，也就是我们刚才设置的第一个断点位置，此时参数列表中 sec 时间是 0.00039s。



⑤再次点击红色标记 8 运行时，此时黄色箭头指向第 37 行代码，此时 sec 为 0.450601s。



将现在这个时间减去上一步的 sec 时间就可以得到 delay_10us(50000)运行的实际时间了。再次点击仿真按钮则可退出仿真界面，回到程序编辑界面。

KEIL 软件的仿真功能非常强大，里面有很多功能，包括在仿真时使用单步调试观察变量参数等都有，这里就不过多介绍，如果对这个感兴趣的可以自行百度搜索 KEILC51 软件仿真的使用。

9.4.3 LED 流水灯实验

9.4.3.1 使用移位和循环实现

如果要实现 LED 流水灯，只需循环让 D1-D8 指示灯逐个点亮。同样本实验也需要延时，这个在前面已介绍，这里就不多说。要实现循环点亮，可以使用最容易理解的方法：点亮 D1 且把 D2-D8 熄灭，延时一段时间后再点亮 D2 且把 D1、D3-D8 熄灭，延时一段时间后再点亮 D3 且把 D1-D2、D4-D8 熄灭，如此循环，这样就可以很简单的实现 LED 流水灯实验，当然我们不推荐此种方法。在前面我们学习了 C 语言相关的基础知识，里面有移位以及循环语句操作，根据流水灯实现原理，即 IO 口由低往高或者由高往低逐个输出低电平特点，那么我们可以将移位操作以及循环结合进来。实现代码如下：

```
#define LED_PORT  P2  //使用宏定义 P2 端口
void main()
{
    u8 i=0;

    while(1)
    {
        for(i=0;i<8;i++)
        {
            LED_PORT=~(0x01<<i); //将 1 右移 i 位,然后取反将结果赋值到 LED_PORT
            delay_10us(50000);
        }
    }
}
```

进入 main 函数后首先定义一个变量 i，然后进入 while 循环，由于要实现 8 个 LED 从 D1->D8 循环点亮，因此可以使用 for 循环语句循环 8 次，每循环一次，点亮的小灯向右移动一个，而 D1-D8 是连接到 P2.0-P2.7 的，因此输出的低电平要左移一位，因此可以使用 LED_PORT=~(0x01<<i);语句实现。0X01<<i 表示 i

增加 1 次，0x01 中的 1 就移动多少位，因为 1（高电平）不会让 LED 点亮，需要取反后变为低电平 0 才能点亮，所以最后的结果需要取反后给 LED_PORT 口，并且每次循环都要延时一段时间，这样才能分辨出来 LED 在流水。

9.4.3.2 使用左移_crol_、右移_cror_函数

除了使用 for 循环语句实现移位，KEIL C51 软件内还有对应的移位库函数，左移函数是_crol_()，右移函数是_cror_()，要使用这两个函数在我们的程序中必须包含 intrins.h 头文件。这两个移位函数大家可以百度了解下，其内部实现过程是看不到的，该移位函数实现的移位功能就相当于一个队列内循环移动，如果是左移，那么最高位就被移到最低位了，次高位变为最高位，依次类推。使用左移、右移函数实现的流水灯操作代码如下：

```
#include "reg52.h"
#include "intrins.h"

void main()
{
    u8 i=0;

    LED_PORT=~0x01;
    delay_10us(50000);
    while(1)
    {
        for(i=0;i<7;i++)   //将 led 左移一位
        {
            LED_PORT=_crol_(LED_PORT,1);
            delay_10us(50000);
        }
        for(i=0;i<7;i++)  //将 led 右移一位
        {
            LED_PORT=_cror_(LED_PORT,1);
            delay_10us(50000);
        }
    }
}
```

进入 main 函数后首先定义一个变量 i，LED_PORT=~0x01，因为 LED 是低电平点亮，所以 0X01 取反后的结果是 0XFE，对应二进制数为 1111 1110，即最低位

为 0，因此最开始的 D1 指示灯会点亮，然后进入 while 循环，使用 for 循环、_crol_和_cror_移位函数实现 LED 左右流水显示。

细心的朋友可能会发现此处每个 for 循环只有 7 次，为什么不是 8 次呢，这是因为在进入 main 开始，就已经将 LED_PORT 端口设置了一次状态，即让 D1 点亮，并且我们是想让 LED 从左至右依次点亮，然后继续又从右至左依次点亮，这样形成左右流水效果。假如将循环次数改为 8 次，我们列举下第一个 for 循环的 LED_PORT 端口状态值，如下所示：

```
初始状态：LED_PORT=1111 1110
i=0：LED_PORT=1111 1101
i=1：LED_PORT=1111 1011
i=2：LED_PORT=1111 0111
i=3：LED_PORT=1110 1111
i=4：LED_PORT=1101 1111
i=5：LED_PORT=1011 1111
i=6：LED_PORT=0111 1111
i=7：LED_PORT=1111 1110
```

通过上述列举，可以非常清楚的了解端口 LED 状态，所以此处应该将循环次数设置为 7 次，即 i 最大等于 6。这样到下一个循环右移时就可以从最高位开始往低位移动，从而实现左右流水灯效果。

9.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内（程序的烧入具体操作步骤可参考“第 3 章 开发板功能及使用介绍”的程序烧入（下载）章节），下载“\4--实验程序\1--基础实验\2-点亮第一个 LED”实验程序，可以看到开发板上 LED 模块 D1 指示灯点亮。

下载“\4--实验程序\1--基础实验\3-LED 闪烁实验”实验程序，可以看到开发板上 LED 模块 D1 指示灯闪烁。

下载“\4--实验程序\1--基础实验\4-LED 流水灯实验”实验程序，可以看到

开发板上 LED 模块 D1-D8 指示灯左右循环流水。

课后作业

普中51单片机开发板

第 10 章 蜂鸣器实验

前面章节我们已经介绍了如何控制 51 单片机的 IO 口输出高低电平，本章我们通过另外一个实验来讲述 51 单片机 IO 口的输出。通过单片机的一个 IO 口控制板载无源蜂鸣器，实现蜂鸣器控制。学习本章可以参考前面 LED 实验章节内容。本章分为如下几部分内容：

10.1 蜂鸣器介绍

10.2 硬件设计

10.3 软件设计

10.4 实验现象

普中51单片机开发板

10.1 蜂鸣器介绍

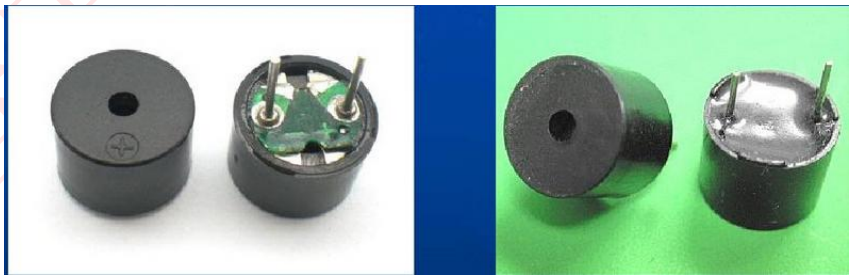
蜂鸣器是一种一体化结构的电子讯响器，采用直流电压供电，广泛应用于计算机、 打印机、 复印机、 报警器、 电子玩具、 汽车电子设备、 电话机、 定时器等电子产品中作发声器件。蜂鸣器主要分为压电式蜂鸣器和电磁式蜂鸣器两种类型。

压电式蜂鸣器主要由多谐振荡器、压电蜂鸣片、阻抗匹配器及共鸣箱、外壳等组成。多谐振荡器由晶体管或集成电路构成，当接通电源后（1.5~15V 直流工作电压），多谐振荡器起振,输出 1.5～5kHZ 的音频信号，阻抗匹配器推动压电蜂鸣片发声。

电磁式蜂鸣器由振荡器、电磁线圈、磁铁、振动膜片及外壳等组成。接通电源后，振荡器产生的音频信号电流通过电磁线圈，使电磁线圈产生磁场，振动膜片在电磁线圈和磁铁的相互作用下，周期性地振动发声。

其实一句话就可概括它们之间的区别，想要压电式蜂鸣器发声，需提供一定频率的脉冲信号；想要电磁式蜂鸣器发声，只需提供电源即可。

我们开发板上使用的蜂鸣器是无源蜂鸣器，属于压电式蜂鸣器类型。这里说的有源和无源，并不是指电源的意思，而是指蜂鸣器内部是否含有振荡电路，有源蜂鸣器内部自带振荡电路，只需提供电源即可发声，而无源蜂鸣器则需提供一定频率的脉冲信号才能发声，频率大小通常在 1.5-5KHz 之间。蜂鸣器实物图如下图所示：



对于无源蜂鸣器，如果改变频率就可以调节蜂鸣器音调，产生各种不同音色、音调的声音。如果改变输出电平的高低电平占空比，则可以改变蜂鸣器的声音大小。

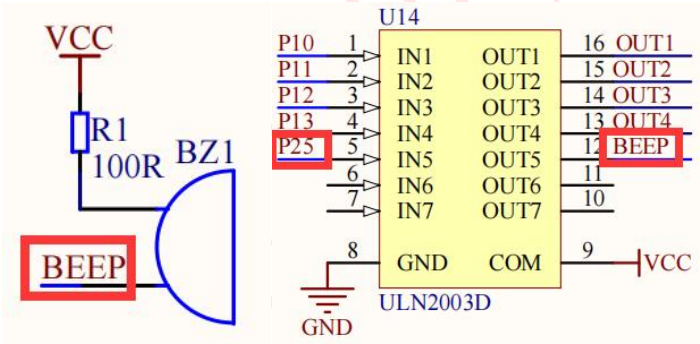
对于有源蜂鸣器，通常内部已经固定了频率，对于调节频率或占空比可能改变不了蜂鸣器的音调和音量，当然也有的有源蜂鸣器可以实现和无源蜂鸣器一样

的效果。

10.2 硬件设计

在前面章节中我们已经对 51 单片机的 GPIO 做了简单介绍，并且还使用了其中 IO 口直接控制开发板上的 LED。对于本章要实现蜂鸣器的控制，我们能否直接使用单片机的 IO 口驱动呢？答案是否定的，因为 51 单片机 IO 口的驱动能力较弱（即使外接上拉电阻），而蜂鸣器驱动需要约 30mA，所以非常困难，即使可以驱动，那对于整个芯片的其它 IO 剩下驱动能力就更加弱甚至无法工作。所以我们不会直接使用 IO 口驱动蜂鸣器，而是通过三极管把电流放大后再驱动蜂鸣器，这样 51 单片机的 IO 口只需要提供不到 1mA 的电流就可控制蜂鸣器。所以我们也经常说到 51 单片机是用来做控制的，而不是驱动。

我们开发板上的蜂鸣器模块电路如下图所示：



从图中可以看出，蜂鸣器控制管脚直接连接到 51 单片机的 P2.5 管脚上。图中并没有使用三极管进行电流放大，而是使用 ULN2003 芯片来驱动，有关 ULN2003 芯片的使用此处先不作介绍，后面章节会有，大家暂时只需知道当 P25 输出高电平，BEEP 则输出低电平；当 P25 输出低电平，BEEP 则输出高电平，类似一个非门。

开发板上使用的是无源蜂鸣器，它需要一定频率的脉冲（高低电平）才会发声，因此需要让 P25 脚以一定频率不断输出高低电平信号才能控制蜂鸣器发出声音。

10.3 软件设计

本章所要实现的功能是：让蜂鸣器发出声音，一段时间后再关闭，即让 P2.5 管脚输出一定频率的脉冲信号（高低电平）控制无源蜂鸣器。

我们打开“\4--实验程序\1--基础实验\5-蜂鸣器实验”工程，控制代码全部都在 main.c 中，代码如下：

```
/************************************************************************
******************
实验名称：蜂鸣器实验
接线说明：
实验现象：下载程序后蜂鸣器发出声音，一段时间后关闭
注意事项：

************************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

sbit BEEP=P2^5;   //将 P2.5 管脚定义为 BEEP

/************************************************************************
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
************************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
```

```
*************************************************************************
**********/
void main()
{
    u16 i=2000;

    while(1)
    {
        while(i--)//循环 2000 次
        {
            BEEP=!BEEP;//产生一定频率的脉冲信号
            delay_10us(100);
        }
        i=0;//清零
        BEEP=0;//关闭蜂鸣器
    }
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后使用 sbit 关键字来定义 P2.5 管脚，定义好后即可使用 BEEP 来替代 P2.5 口的操作。主函数功能非常简单，直接进入 while 循环，在循环内再次套用了一个 while 循环，只不过这里并非死循环，而是通过变量 i 值来决定何时退出，i 值初始化为 2000，即该循环会执行 2000 次，此循环内不断对 BEEP 取反，然后延时一定时间，即 P2.5 间隔一定时间输出高低电平，这样就会产生脉冲信号控制蜂鸣器发出声音，当 i 值递减到 0 时则退出 while 循环，然后将 i 值清零，且将 BEEP 输出 0。若修改变量 i 的值可以改变蜂鸣器发声时间。

若要改变音调可以修改延时时间，但要注意频率不能太大或者太小，具体大家可以试着调试。若要改变音量，可以修改 BEEP 输出高电平时间，如下：

```
BEEP=1;
delay_10us(190);
BEEP=0;
delay_10us(10);
```

10.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），

把编译后产生的.hex 文件烧入到芯片内，可以听到开发板上蜂鸣器发出美妙的声音，一段时间后关闭。

课后作业

普中51单片机开发板

第 11 章 静态数码管实验

前面章节我们已经介绍了如何控制 51 单片机的 IO 口输出高低电平，本章我们通过另外一个实验来讲述 51 单片机 IO 口的输出。通过单片机的 IO 口控制板载数码管显示。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

11.1 数码管介绍

11.2 硬件设计

11.3 软件设计

11.4 实验现象

普中51单片机开发板

11.1 数码管介绍

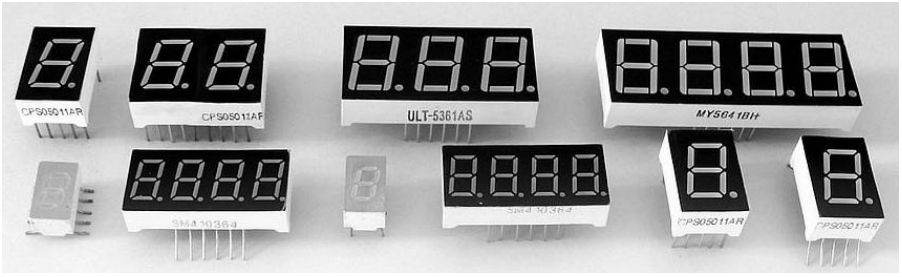
11.1.1 数码管简介

数码管是一种半导体发光器件，其基本单元是发光二极管。数码管也称 LED 数码管，不同行业人士对数码管的称呼不一样，其实都是同样的产品。数码管按段数可分为七段数码管和八段数码管，八段数码管比七段数码管多一个发光二极管单元，也就是多一个小数点（DP），这个小数点可以更精确的表示数码管想要显示的内容；按能显示多少个（8）可分为 1 位、2 位、3 位、4 位、5 位、6 位、7 位等数码管。按发光二极管单元连接方式可分为共阳极数码管和共阴极数码管。

共阳数码管是指将所有发光二极管的阳极接到一起形成公共阳极(COM)的数码管，共阳数码管在应用时应将公共极 COM 接到+5V，当某一字段发光二极管的阴极为低电平时，相应字段就点亮，当某一字段的阴极为高电平时，相应字段就不亮。

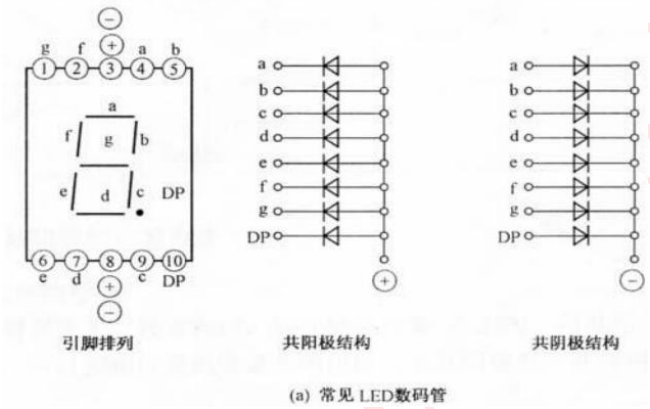
共阴数码管是指将所有发光二极管的阴极接到一起形成公共阴极(COM)的数码管，共阴数码管在应用时应将公共极 COM 接到地线 GND 上，当某一字段发光二极管的阳极为高电平时，相应字段就点亮，当某一字段的阳极为低电平时，相应字段就不亮。

不同位数的数码管实物图如下所示：



11.1.2 数码管显示原理

不管将几位数码管连在一起，数码管的显示原理都是一样的，都是靠点亮内部的发光二极管来发光，下面我们就来讲解一个数码管是如何亮起来的。数码管内部电路如下图所示：



(a) 常见 LED数码管

从上图可看出，一位数码管的引脚是 10 个，显示一个 8 字需要 7 个小段，另外还有一个小数点，所以其内部一共有 8 个小的发光二极管，最后还有一个公共端，多数生产商为了封装统一，单位数码管都封装 10 个引脚，其中第 3 和第 8 引脚是连接在一起的。而它们的公共端又可分为共阳极和共阴极，图中间为共阳极内部原理图，右图为共阴极内部原理图。

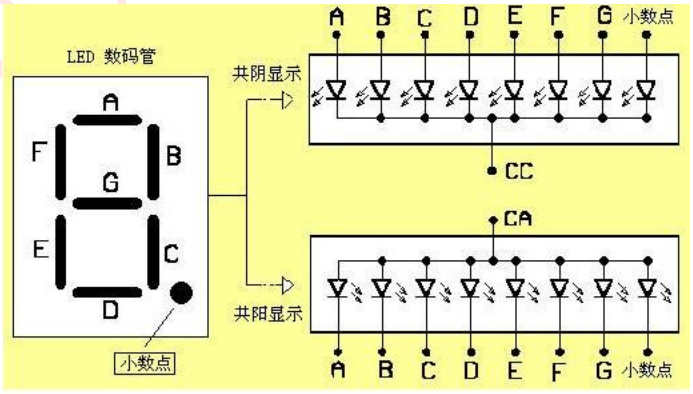
对共阴极数码来说，其 8 个发光二极管的阴极在数码管内部全部连接在一起，所以称“共阴”，而它们的阳极是独立的，通常在设计电路时一般把阴极接地。当我们给数码管的任意一个阳极加一个高电平时，对应的这个发光二极管就点亮了。如果想要显示出一个 8 字，并且把右下角的小数点也点亮的话，可以给 8 个阳极全部送高电平，如果想让它显示出一个 0 字，那么我们可以除了给第“g, dp” 这两位送低电平外，其余引脚全部都送高电平，这样它就显示出 0 字了。

如果使用共阴数码管，需要注意增加单片机 IO 口驱动电流，因为共阴数码管是要靠单片机 IO 口输出电流来点亮的，但单片机 I/O 口难以输出稳定的、如此大的电流，所以数码管与单片机连接时需要加驱动电路，可以用上拉电阻的方法或使用专门的数码管驱动芯片，比如 74HC573、74HC245 等，其输出电流较大，电路接口简单。

共阳极数码管其内部 8 个发光二极管的所有阳极全部连接在一起，电路连接时，公共端接高电平，因此我们要点亮哪个发光管二极管就需要给阴极送低电平，此时显示数字的编码与共阴极编码是相反的关系，数码管内部发光二极管点亮时，也需要 5mA 以上的电流，而且电流不可过大，否则会烧坏发光二极管。因此不仅要防止数码管电流过大，同时要防止流经数码管的电流集中到单片机时电流不能过大，否则会损坏主芯片。

一般共阳极数码管更为常用，为什么呢？ 这是因为数码管的非公共端往往接在 IC 芯片的 I/O 上，而 IC 芯片的驱动能力往往是比较小的，如果采用共阴极数码管，它的驱动端在非公共端， 就有可能受限于 IC 芯片输出电流不够而显示昏暗，要外加上拉电阻或者是增加三极管加大驱动能力。但是 IC 芯片的灌电流，即输入电流范围比较大。所以使用共阳极数码管的好处是：将驱动数码管的工作交到公共端（一般接驱动电源），加大驱动电源的功率自然要比加大 IC 芯片 I/O 口的驱动电流简单许多。另一方面，这样也能减轻主芯片的负担。

我们开发板上使用的数码管是 2 个四位一体的共阴极数码管（即 8 个 LED 的阳极全部并联一起引出，阴极分别引出如 A、 B...DP），本章实验也是在该数码管上实现单个的静态显示。如果要让共阴数码管显示数字 0，即对应的段 ABCDEF 要点亮即给它高电平，其他的段熄灭即给它低电平。其他的数字显示方式一样，这里就不多说。下面给出共阴和共阳数码管的 0-F 段码数据表，如下所示：



①共阴数码管码表

```
0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d,
 0     1     2     3     4     5
0x7d, 0x07, 0x7f, 0x6f, 0x77, 0x7c,
```

```
 6     7     8     9     A     B
0x39, 0x5e, 0x79, 0x71, 0x00,
 C     D     E     F   无显示
```

②共阳数码管码表

```
0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92,
 0     1     2     3     4     5
0x82, 0xF8, 0x80, 0x90, 0x88, 0x83,
 6     7     8     9     A     B
0xC6, 0xA1, 0x86, 0x8E, 0xFF,
 C     D     E     F   无显示
```

从上述共阳和共阴码表中不难发现，它们的数据正好是相互取反的值。比如共阴数码管数字 0 段码：0x3f，其二进制是：0011 1111，取反后为：1100 0000，转换成 16 进制即为 0XC0。其他段码依此类推。该段码数据由来，是将 a 段作为最低位，b 段作为次低位，其他按顺序类推，dp 段为最高位，共 8 位，正好和 51 单片机的一组端口数一样，因此可以直接使用某一组端口控制数码管的段选数据口，比如 P0 口。

11.1.3 数码管静态显示原理

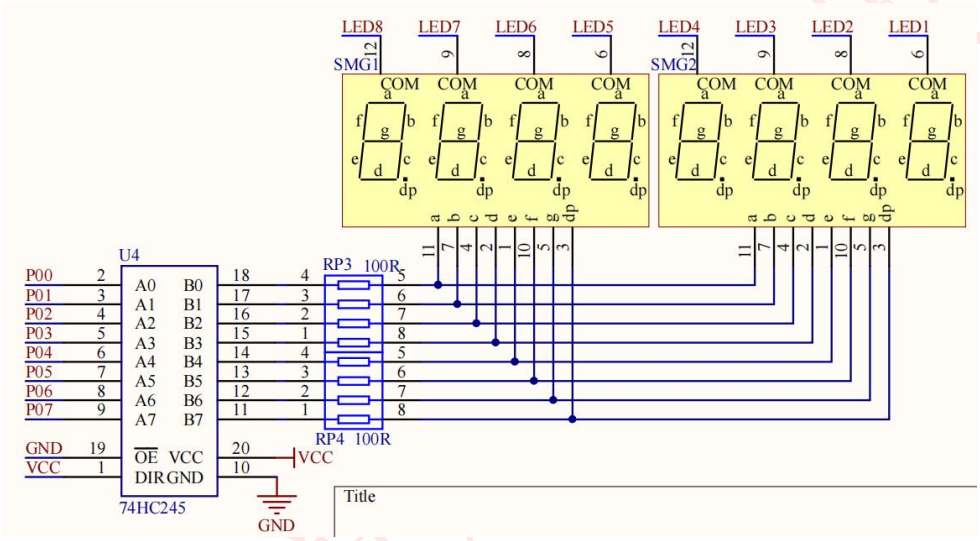
LED 数码管显示器工作方式有两种：静态显示方式和动态显示方式。静态显示的特点是每个数码管的段选必须接一个 8 位数据线来保持显示的字形码。当送入一次字形码后，显示字形可一直保持，直到送入新字形码为止。这种方法的优点是占用 CPU 时间少，显示便于监测和控制。缺点是硬件电路比较复杂，成本较高，比如使用 4 个静态数码管，那么就得 32 个 IO 来控制，这对 51 单片机来说是无法承受的，正因为如此才会有后面章节动态数码实验的讲解。

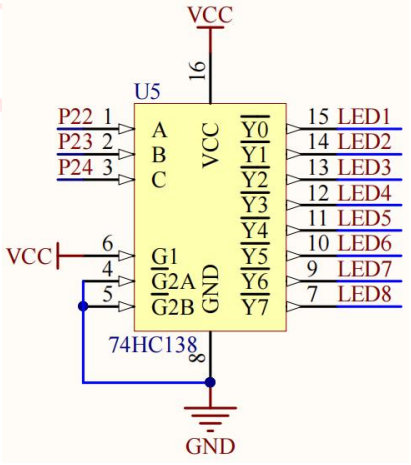
动态显示的特点是将所有数码管的段选线并联在一起，由位选线控制是哪一位数码管有效。选亮数码管采用动态扫描显示。所谓动态扫描显示即轮流向各位数码管送出字形码和相应的位选，利用发光管的余辉和人眼视觉暂留作用，使人的感觉好像各位数码管同时都在显示。动态显示的亮度比静态显示要差一些，所以在选择限流电阻时应略小于静态显示电路中的。

本章实验使用的数码管虽然是动态数码管电路，但我们依然可以使用数码管其中一位来学习静态数码管知识。本章实验主要介绍静态数码管的控制，有关动态数码管控制将在下一章节介绍。有关静态数码管的详细介绍，大家可以在百度上查找了解。

11.2 硬件设计

开发板上的静态数码管模块电路如下图所示：





上图电路实际上是动态数码管电路，使用的是 2 个四位一体的共阴数码管组成，即 8 位数码管的段选数据 a-dp 全部并联一起引出，每位数码管的位选即公共端引出，前面我们也说了，虽然开发板上没有单个的静态数码管，但依然可以在动态数码管电路中使用其中一个来学习静态数码管显示。本实验使用 SMG1 最左边那个数码管作为静态数码管，因为单片机 IO 口外部都增加了外部上拉电阻，

因此 P22、P23、P24 引脚默认就是高电平，根据 38 译码器输出特点，此时 Y7 脚（LED8）输出有效，即低电平。而数码管的段选 a-dp 连接在 74HC245 驱动芯片输出口，由 P0 端口控制。所以只要控制 P0 口输出高电平，SMG1 最左边那个数码管默认就可以显示。此处暂时不去理会 38 译码器的工作原理，后面动态数码管实验会专门介绍。74HC245 作为驱动芯片使用，目的是让数码管能获得更大的电流，为防止因电流过大烧坏数码管，在 74HC245 芯片输出管脚又串联了 2 个 4 位的 100 欧排阻后连接数码管段码 a-dp 脚。本章暂时不必了解 74HC245 芯片使用，只要知道 P0 口输出什么，74HC245 芯片就输出什么。

11.3 软件设计

本章所要实现的功能是：控制静态数码管显示数字 0，即让 P0 端口输出数字 0 的段码 0x3f（共阴）。

我们打开“\4--实验程序\1--基础实验\6-静态数码管实验”工程，控制代码全部都在 main.c 中，代码如下：

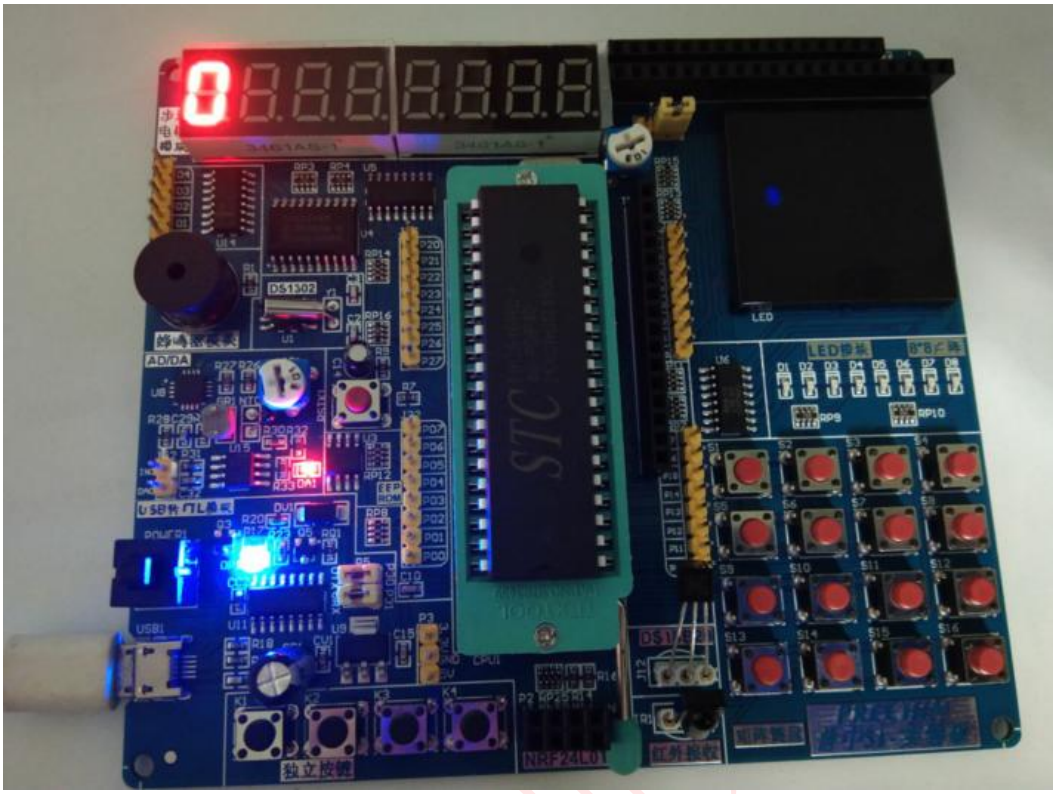
```
/************************************************************************
******************
实验名称：静态数码管实验
接线说明：
实验现象：下载程序后“数码管模块”最左边数码管显示数字 0
注意事项：

************************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

#define SMG_A_DP_PORT    P0  //使用宏定义数码管段码口

//共阴极数码管显示 0~F 的段码数据
u8 gsmg_code[17]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,
                  0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x79,0x71};

/************************************************************************
***********
```

```
* 函 数 名         : main
* 函数功能         : 主函数
* 输    入         : 无
* 输    出         : 无
*******************************************************************************
**********/
void main()
{
    SMG_A_DP_PORT=gsmg_code[0];//将数组第 1 个数据赋值给数码管段选口
    while(1)
    {

    }
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义一个全局数组变量 gsmg_code 存放共阴数码管 0-F 段码数据。主函数功能也很简单，首先将数组的第 1 个数据赋值给 SMG_A_DP_PORT，因为数组内定义的是共阴数码管段码，数组角标为 0 存储的就是第一个数据 0X3F。然后进入 while 循环，单片机此时一直在 while 内循环操作。当然该条语句也可以放在 while 循环语句内，同样会让静态数码管显示 0。

11.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：最左边那位数码管显示数字 0。



课后作业

普中51单片机

第 12 章 动态数码管实验

前面章节我们已经介绍了如何使用 51 单片机控制静态数码管显示，在实际应用中通常都需要显示多位数值，如果采用静态数码管显示就不够好，因此就需要采用另外一种显示方式，即数码管动态显示。开发板上板载 2 个四位一体的共阴数码管，本章我们就来介绍下如何使用 51 单片机控制动态数码管显示。本章所要实现的功能是：控制动态数码管从左至右显示数字 0-7。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

12.1 数码管介绍

12.2 74HC245 和 74HC138 芯片介绍

12.3 硬件设计

12.4 软件设计

12.5 实验现象

普中51单片机开发板

12.1 数码管介绍

上一章我们主要是介绍一位数码管的内部结构及控制原理。下面我们再来介绍下多位数码管及动态显示原理的相关知识。

12.1.1 多位数码管简介

多位数码管，即两个或两个以上单个数码管并列集中在一起形成一体的数码管。当多位一体时，它们内部的公共端是独立的，而负责显示什么数字的段线（a-dp）全部是连接在一起的，独立的公共端可以控制多位一体中的哪一位数码管点亮，而连接在一起的段线可以控制这个能点亮数码管亮什么数字，通常我们把公共端叫做“位选线”，连接在一起的段线叫做“段选线”，有了这两个线后，通过单片机及外部驱动电路就可以控制任意的数码管显示任意的数字了。

一般一位数码管有 10 个引脚， 二位数码管也是 10 个引脚， 四位数码管是 12 个引脚，关于具体的引脚及段、位标号大家可以查询相关资料，最简单的办

普中51单片机开发板

法就是用数字万用表测量，若没有数字万用表也可用 5V 直流电源串接 1k 电阻后测量，将测量结果记录，通过统计便可绘制出引脚标号。多位数码管有许多是按一定要求设计的，引脚不完全按照一般规则设定，所以需要在使用时查找手册，最直接的办法就是按照数码管上的标示向生产商要求。

我们开发板上使用了 2 个四位一体的共阴数码管，这样可在上面同时显示 8 个数值。

12.1.2 数码管动态显示原理

多位数码管依然可以静态显示，但是显示时要么只显示一位数码管，要么多位同时显示相同内容。当多位数码管应用于某一系统时，它们的“位选”是可独立控制的，而“段选”是连接在一起的，我们可以通过位选信号控制哪几个数码管亮，而在同一时刻，位选选通的所有数码管上显示的数字始终都是一样的，因为它们的段选是连接在一起的，送入所有数码管的段选信号都是相同的，所以它们显示的数字必定一样，数码管的这种显示方法叫做静态显示。

而动态显示，就是利用减少段选线，分开位选线，利用位选线不同时选择通断，改变段选数据来实现的。比如在第一次选中第一位数码管时，给段选数据 0，下一次位选中第二位数码管时显示 1。为了在显示 1 的时候，0 不会消失（当然实际上是消失了），必须在人肉眼观察不到的时间里再次点亮第一次点亮的 0。而这时就需要记住，人的肉眼正常情况下只能分辨变化超过 24ms 间隔的运动。也就是说，在下一次点亮 0 这个数字的时间差不得大于 24ms。这时就会发现，数码管点亮是在向右或者向左一位一位点亮，形成了动态效果。如果把间隔时间改长就能直接展现这一现象。

数码管动态显示的应用非常多，所以大家一要认真学好数码管的动态显示方法。

12.2 74HC245 和 74HC138 芯片介绍

通过前面内容的介绍我们知道，要使单片机能控制开发板上 2 位一体的共阴数码管显示，仅靠单片机 IO 口来驱动是不行的，这里就需要增加外部驱动芯片，

开发板上使用的是 74HC245 芯片。2 个 4 位一体的共阴数码管的位选线有 8 根，直接让单片机 IO 口控制是没有任何问题的，但考虑到 51 单片机 IO 口资源的限制，通常我们会使用一种 IO 扩展芯片，比如 74HC138、74HC164、74HC595 芯片等，只需要很少的单片机 IO 口就可以扩展出 8 个控制口，通过级联方式甚至可扩展出更多的控制口（这个在后面 LED 点阵实验章节中会有详细介绍）。我们开发板上使用的是 74HC138 译码器芯片，只需单片机 3 个 IO 口就可以实现 8 个位选管脚的控制，节省了芯片的 IO 资源。

下面就来看看这两个芯片功能及使用方法。

12.2.1 74HC245 芯片简介

74HC245 是一种三态输出、八路信号收发器，主要应用于大屏显示，以及其它的消费类电子产品中增加驱动。

（1）主要特性

①采用 CMOS 工艺

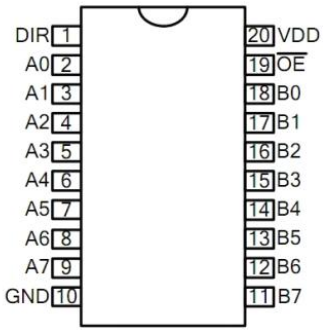
②宽电压工作范围：3.0V-5.0V

③双向三态输出

④八线双向收发器

⑤封装形式：SOP20、SOP20-2、TSSOP20、DIP20

（2）管脚功能定义



管脚定义说明

符号	管脚名称	管脚号	说明
A0——A7	数据输入/输出	2—9	
B0——B7	数据输入/输出	18—11	
$\overline{OE}$	输出使能	19	
DIR	方向控制	1	DIR=1,A——>B；DIR=0,B——>A
GND	逻辑地	20	逻辑地
VDD	逻辑电源	10	电源端

功能真值表

输出使能	输出控制	工作状态
$\overline{OE}$	DIR	
L	L	Bn 输入 An 输出
L	H	An 输入 Bn 输出
H	X	高阻态

从上面的管脚功能定义说明及真值表可以知道该芯片使用方法很简单，给 OE 使能管脚低电平，DIR 管脚为高电平传输方向是 A->B 输出，DIR 为低电平传输方向是 B->A，至于输出高电平还是输出低电平取决于输入端的状态，如果输入为低电平，输出即为低；输入为高电平，输出即为高。如果 OE 使能管脚为高电平，不论 DIR 管脚是高还是低，输出是高组态。

通常我们使用 74HC245 芯片用作驱动只会让其在一个方向输出，即 DIR 管脚为高电平，传输方向是 A->B。

12.2.2 74HC138 芯片简介

74HC138D 是一种三通道输入、八通道输出译码器，主要应用于消费类电子产品。

（1）主要特性

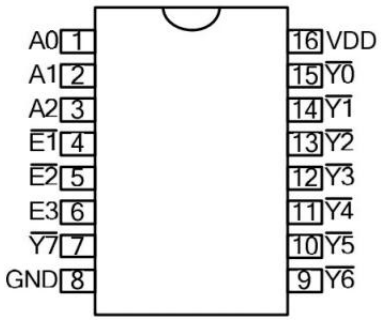
①采用 CMOS 工艺

②低功耗

③工作电压：3.0V-5.0V

④封装形式：SOP16

（2）管脚功能定义



管脚说明

名称	功能说明	管脚号
$\overline{Y0}$ —$\overline{Y6}$,$\overline{Y7}$	数据输出	15—9,7
A0——A2	数据输入	1—3
$\overline{E1}$,$\overline{E2}$,E3	使能控制	4—6
VDD	逻辑电源	16
GND	逻辑地	8

真值表：

输入						输出							
$\overline{E1}$	$\overline{E2}$	E3	A0	A1	A2	$\overline{Y0}$	$\overline{Y1}$	$\overline{Y2}$	$\overline{Y3}$	$\overline{Y4}$	$\overline{Y5}$	$\overline{Y6}$	$\overline{Y7}$
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

从上面的管脚功能定义说明及真值表可以知道该芯片使用方法很简单，给 E1、E2 使能管脚低电平，E3 管脚为高电平，至于哪个管脚输出有效电平（低电平），要看 A0，A1，A2 输入管脚的电平状态。如果 A0，A1，A2 都为低电平，则 Y0 输出有效电平（低电平），其他管脚均输出高电平。如果 A0 为高电平，A1，A2 都为低电平，则 Y1 输出有效电平（低电平），其他管脚均输出高电平。其他几种输出大家可以对照真值表查看。如果 E1、E2 使能管脚任意一个为高电平或者 E3 为低电平，不论输入是什么，输出都为高电平。

这里给大家总结一个方法：A0、A1、A2 输入就相当于 3 位 2 进制数，A0 是低位，A1 是次高位，A2 是高位。而 Y0-Y7 具体哪一个输出有效电平，就看输入二进制对应的十进制数值。比如输入是 101（A2，A1，A0），其对应的十进制数是 5，所以 Y5 输出有效电平（低电平）。

有关 74HC245 和 74HC138 芯片的更多详细介绍，大家可以在资料“\6--芯片资料\开发板芯片数据手册”内查看。

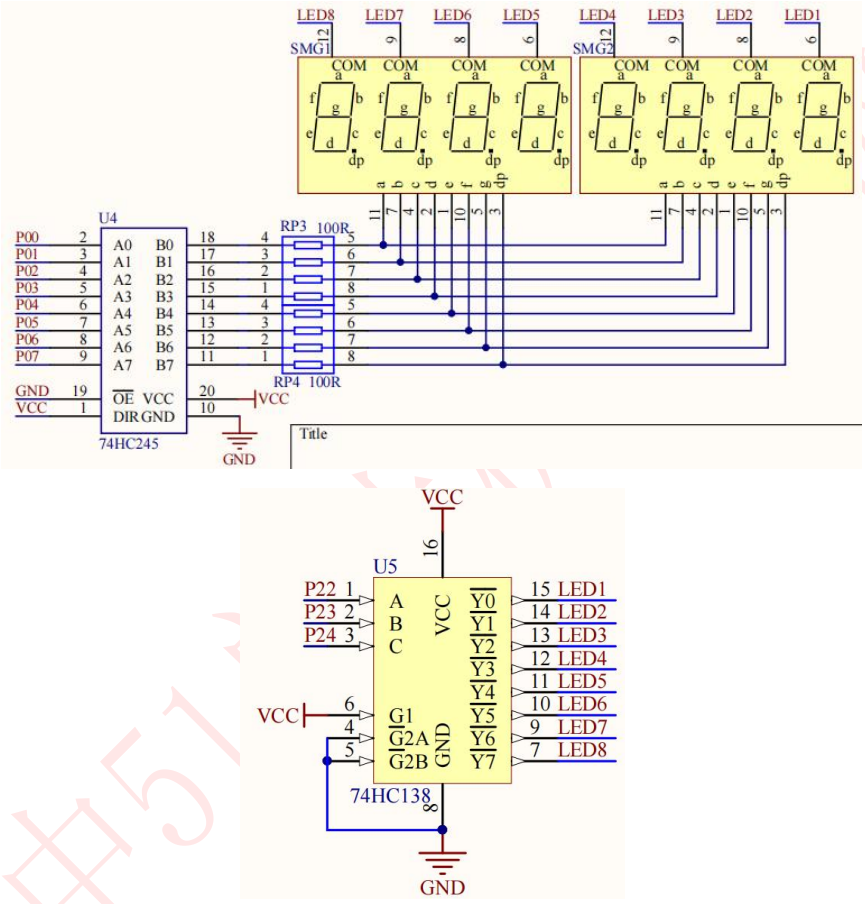
12.3 硬件设计

本实验使用到硬件资源如下：

（1）动态数码管模块

（2）74HC138

开发板上的动态数码管模块电路如下图所示：



上图电路使用的是 2 个四位一体的共阴数码管组成，即 8 位数码管的段选数据 a-dp 全部并联一起引出，每位数码管的位选即公共端引出。数码管的段选 a-dp 连接在 74HC245 驱动芯片输出口，由 P0 端口控制。由 P2.2、P2.3、P2.4 管脚控制 74HC138 译码器输入从而控制数码管位选。

12.4 软件设计

本章所要实现的功能是：控制动态数码管从左至右显示数字 0-7。

我们打开“\4--实验程序\1--基础实验\7-动态数码管实验”工程，控制代

码全部都在 main.c 中，代码如下：

```
/*********************************************************************************************
****
实验名称：动态数码管实验
接线说明：
实验现象：下载程序后“数码管模块”显示 01234567
注意事项：

*********************************************************************************************
****/
#include "reg52.h"

typedef unsigned int u16;//对系统默认数据类型进行重定义
typedef unsigned char u8;

#define SMG_A_DP_PORT    P0  //使用宏定义数码管段码口

//定义数码管位选信号控制脚
sbit LSA=P2^2;
sbit LSB=P2^3;
sbit LSC=P2^4;

//共阴极数码管显示 0~F 的段码数据
u8 gsmg_code[17]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,
                  0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x79,0x71};

/*********************************************************************************
* 函 数 名         : delay_10us
* 函数功能         : 延时函数，ten_us=1 时，大约延时 10us
* 输    入         : ten_us
* 输    出         : 无
*********************************************************************************/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*********************************************************************************
* 函 数 名         : smg_display
* 函数功能         : 动态数码管显示
* 输    入         : 无
* 输    出         : 无
*********************************************************************************/
```

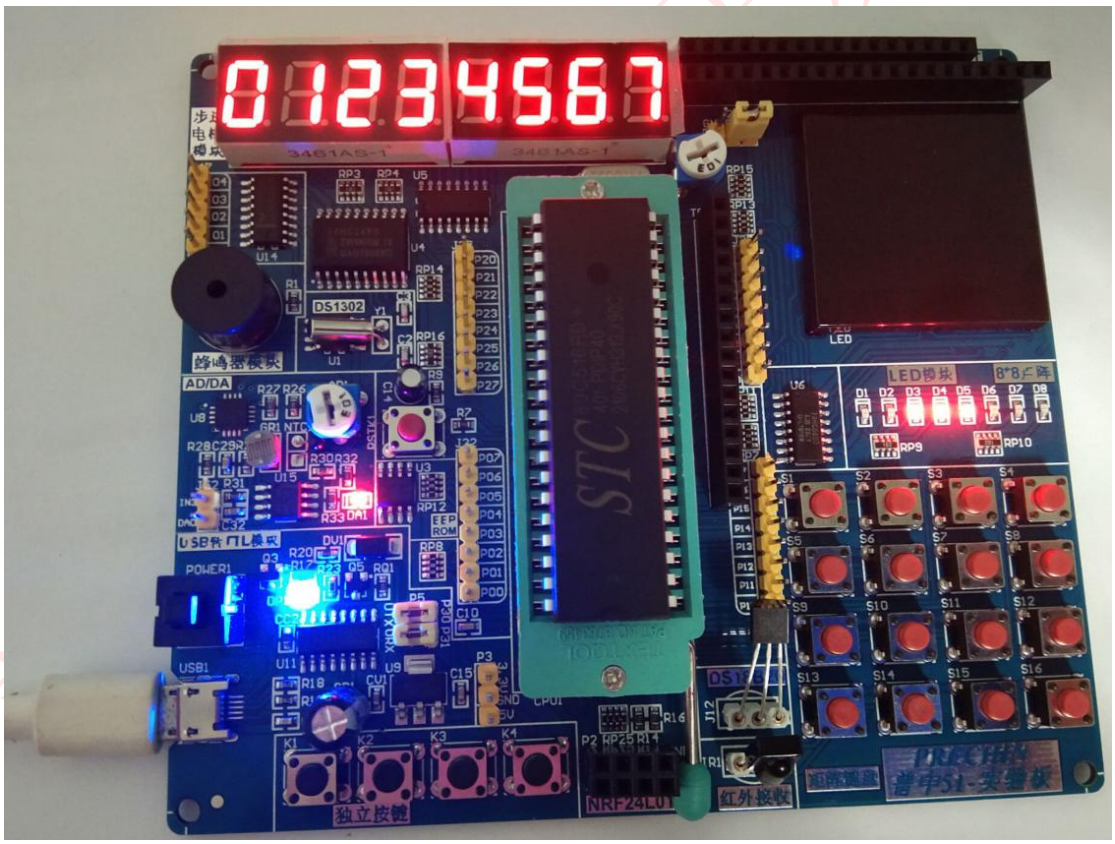
```
void smg_display(void)
{
    u8 i=0;

    for(i=0;i<8;i++)
    {
        switch(i)//位选
        {
            case 0: LSC=1;LSB=1;LSA=1;break;
            case 1: LSC=1;LSB=1;LSA=0;break;
            case 2: LSC=1;LSB=0;LSA=1;break;
            case 3: LSC=1;LSB=0;LSA=0;break;
            case 4: LSC=0;LSB=1;LSA=1;break;
            case 5: LSC=0;LSB=1;LSA=0;break;
            case 6: LSC=0;LSB=0;LSA=1;break;
            case 7: LSC=0;LSB=0;LSA=0;break;
        }
        SMG_A_DP_PORT=gsmg_code[i];//传送段选数据
        delay_10us(100);//延时一段时间，等待显示稳定
        SMG_A_DP_PORT=0x00;//消音
    }
}

/*******************************************************************************
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*******************************************************************************/
void main()
{
    while(1)
    {
        smg_display();
    }
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义 38 译码器的控制引脚，并将共阴数码管 0-F 段码数据使用数组定义好。主函数功能也很简单，直接进入 while 循环，在循环体内执行 smg_display()数码管动态显示函数。该函数是根据动态数码管显示原理所编写，即选中所要显示的那位数码管，然后发送在该位数码管上所要显示的段码数据，延时一定时间后

在将段选口清零即消隐（消除之前的显示），如此循环 8 次即可实现 8 位数码管显示。在动态显示中，要注意延时时间，只要保证在人肉眼所能感觉时间之内即可，让我们看到所要显示的内容就像是同时显示一样。如果时间过长或者过短都可能会影响数码管的显示效果，大家可以在例程基础上试着修改延时时间观察效果。

12.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：动态数码管显示 01234567。



课后作业

第 13 章 独立按键实验

前几章介绍的都是 IO 口输出的使用，这一章我们通过独立按键实验来介绍 IO 口作为输入的使用。开发板上板载 4 个独立按键。本章所要实现的功能是：通过开发板上的独立按键 K1 控制 D1 指示灯亮灭。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

13.1 按键介绍

13.2 硬件设计

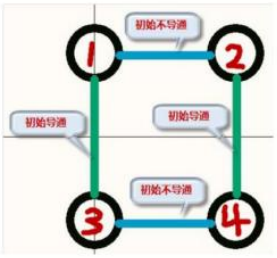
13.3 软件设计

13.4 实验现象

普中51单片机开发板

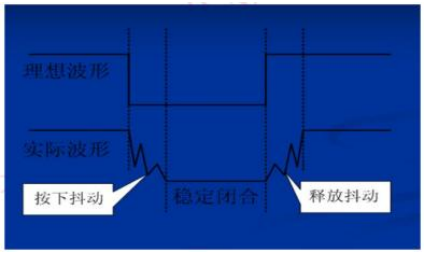
13.1 按键介绍

按键是一种电子开关，使用时轻轻按开关按钮就可使开关接通，当松开手时，开关断开。开发板上使用的按键及内部简易图如下图所示：



按键管脚两端距离长的表示默认是导通状态，距离短的默认是断开状态， 如果按键按下，初始导通状态变为断开，初始断开状态变为导通。

通常的按键所用开关为机械弹性开关，当机械触点断开、闭合时，电压信号如下图所示：



由于机械点的弹性作用，按键开关在闭合时不会马上稳定的接通，在断开时

也不会一下子断开，因而在闭合和断开的瞬间均伴随着一连串的抖动。抖动时间的长短由按键的机械特性决定的，一般为 5ms 到 10ms。按键稳定闭合时间的长短则由操作人员的按键动作决定的，一般为零点几秒至数秒。按键抖动会引起按键被误读多次。为了确保 CPU 对按键的一次闭合仅作一次处理，必须进行消抖。

按键消抖有两种方式，一种是硬件消抖，另一种是软件消抖。为了使电路更加简单，通常采用软件消抖。我们开发板也是采用软件消抖，一般来说一个简单的按键消抖就是先读取按键的状态，如果得到按键按下之后，延时 10ms，再次读取按键的状态，如果按键还是按下状态，那么说明按键已经按下。其中延时 10ms 就是软件消抖处理，至于硬件消抖，大家可以百度了解下，网上都有非常详细的介绍。这里给大家列出单片机常用的软件去抖动方法：

1，先设置 IO 口为高电平（由于开发板 IO 都有上拉电阻，所以默认 IO 为高电平）。

2，读取 IO 口电平确认是否有按键按下。

3，如有 IO 电平为低电平后，延时几个毫秒。

4，再读取该 IO 电平，如果仍然为低电平，说明按键按下。

5，执行按键控制程序。

独立按键电路构成是由各个按键的一个管脚连接在一起接地，按键其他引脚分别接到单片机 IO 口。

我们知道单片机的 IO 口既可作为输出也可作为输入使用，当检测按键时用的是它的输入功能，独立按键的一端接地， 另一端与单片机的 I/O 口相连，开始时先给该 IO 口赋一高电平，然后让单片机不断地检测该 I/O 口是否变为低电平，当按键闭合时，即相当于该 I/O 口通过按键与地相连，变成低电平，程序一旦检测到 I/O 口变为低电平则说明按键被按下，然后执行相应的指令。

13.2 硬件设计

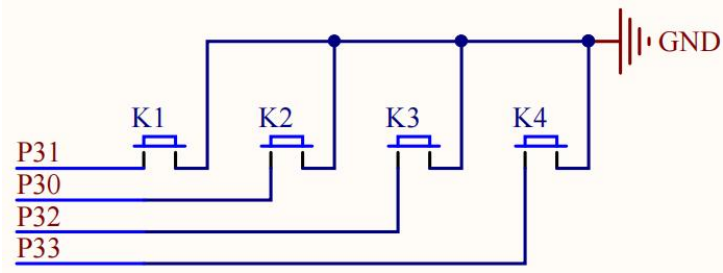
本实验使用到硬件资源如下：

（1）LED 模块中 D1 指示灯

（2）K1 按键

LED 模块电路在前面章节都介绍过，这里就不多说，开发板上的独立按键模

块电路如下图所示：



从上图中可以看出，4 个独立按键的控制管脚连接到 51 单片机的 P3.0-P3.3 脚上。其中 K1 连接在 P3.1 上，K2 连接在 P3.0 上，K3 连接在 P3.2 上，K4 连接在 P3.3 上。4 个按键另一端全部连接在 GND，当按键按下后，对应 IO 口即为低电平。

13.3 软件设计

本章所要实现的功能是：通过开发板上的独立按键 K1 控制 D1 指示灯亮灭。

我们打开“\4--实验程序\1--基础实验\8-独立按键实验”工程，控制代码全部都在 main.c 中，代码如下：

```
/*********************************************************************
******************
实验名称：独立按键实验
接线说明：
实验现象：下载程序后，按下“独立按键”模块中 K1 键，控制 D1 指示灯亮灭
注意事项：

*********************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义独立按键控制脚
sbit KEY1=P3^1;
sbit KEY2=P3^0;
sbit KEY3=P3^2;
sbit KEY4=P3^3;

//定义 LED1 控制脚
```

```
sbit LED1=P2^0;

//使用宏定义独立按键按下的键值
#define KEY1_PRESS    1
#define KEY2_PRESS    2
#define KEY3_PRESS    3
#define KEY4_PRESS    4
#define KEY_UNPRESS   0

/*********************************************************************
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*********************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*********************************************************************
***********
* 函 数 名       : key_scan
* 函数功能       : 检测独立按键是否按下，按下则返回对应键值
* 输    入       : mode=0：单次扫描按键
                   mode=1：连续扫描按键
* 输    出       : KEY1_PRESS：K1 按下
                   KEY2_PRESS：K2 按下
                   KEY3_PRESS：K3 按下
                   KEY4_PRESS：K4 按下
                   KEY_UNPRESS：未有按键按下
*********************************************************************
**********/
u8 key_scan(u8 mode)
{
    static u8 key=1;

    if(mode)key=1;//连续扫描按键
    if(key==1&&(KEY1==0||KEY2==0||KEY3==0||KEY4==0))//任意按键按下
    {
        delay_10us(1000);//消抖
        key=0;
```

```
        if(KEY1==0)
            return KEY1_PRESS;
        else if(KEY2==0)
            return KEY2_PRESS;
        else if(KEY3==0)
            return KEY3_PRESS;
        else if(KEY4==0)
            return KEY4_PRESS;
    }
    else if(KEY1==1&&KEY2==1&&KEY3==1&&KEY4==1)    //无按键按下
    {
        key=1;
    }
    return KEY_UNPRESS;
}
/*************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*************************************************************************
**********/
void main()
{
    u8 key=0;

    while(1)
    {
        key=key_scan(0);
        if(key==KEY1_PRESS)//检测按键 K1 是否按下
            LED1=!LED1;//LED1 状态翻转
    }
}
```

key_scan 函数带一个形参 mode，该参数用来设定是否连续扫描按键，如果 mode 为 0，只能操作一次按键，只有当按键松开后才能触发下次的扫描，这样做的好处是可以防止按下一次出现多次触发的情况。如果 mode 为 1，函数是支持连续扫描的，即使按键未松开，在函数内部有 if(mode==1)这条判断语句，因此 key 始终是等于 1 的，所以可以连续扫描按键，当按下某个按键，会一直返回这 个按键的键值，这样做的好处是可以很方便实现连按操作。函数内的

delay_10us(1000)即为软件消抖处理，通常延时 10ms 即可。

key_scan 函数还带有一个返回值，如果未有按键按下，返回值即为 KEY_UNPRESS，否则返回值即为对应按键的键值，如 KEY1_PRESS、KEY2_PRESS、KEY3_PRESS、 KEY4_PRESS，这都是程序开头定义好的宏，方便大家理解和使用。函数内定义了一个 static 变量 key，相当于全局变量，所以该函数不是一个可重入函数。还有一点要注意的就是该函数按键的扫描是有优先级的，因为函数内用了 if...else if...else 格式，所以最先扫描处理的按键是 KEY1，其次是 KEY2，然后是 KEY3，最后是 KEY4。如果需要将其优先级设置一样，那么可以全部用 if 语句。

main 函数中主要就是调用 key_scan 函数用于检测按键，此时传入的 mode 值为 0，表示单次扫描按键，然后将扫描按键的值保存在变量 key 中，最后通过 if 判断语句控制 LED1 状态。

13.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：当按下 K1 键，D1 指示灯亮，再按下 K1 键，D1 指示灯灭，如此循环。

课后作业

普中51单片机开发板

第 14 章 矩阵按键实验

上一章我们介绍了独立按键的构成以及使用方法，本章我们介绍按键电路的另一种模型：矩阵按键。开发板上板载了一个 4*4 矩阵键盘。本章所要实现的功能是：通过开发板上的矩阵键盘控制静态数码管显示对应的键值 0-F。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

14.1 矩阵按键介绍

14.2 硬件设计

14.3 软件设计

14.4 实验现象

普中51单片机开发板

14.1 矩阵按键介绍

独立按键与单片机连接时，每一个按键都需要单片机的一个 I/O 口，若某单片机系统需较多按键，如果用独立按键便会占用过多的 I/O 口资源。单片机系统中 I/O 口资源往往比较宝贵，当用到多个按键时为了减少 I/O 口引脚，引入了矩阵按键。

本章以 4*4 矩阵键盘为例讲解其工作原理和检测方法。开发板上将 16 个按键排成 4 行 4 列，第一行将每个按键的一端连接在一起构成行线，第一列将每个按键的另一端连接在一起构成列线，这样便一共有 4 行 4 列共 8 根线，我们将这 8 根线连接到单片机的 8 个 I/O 口上，通过程序扫描键盘就可检测 16 个键。用这种方法我们也可实现 3 行 3 列 9 个键、 5 行 5 列 25 个键、 6 行 6 列 36 个键甚至更多。

无论是独立键盘还是矩阵键盘，单片机检测其是否被按下的依据都是一样的，也就是检测与该键对应的 I/O 口是否为低电平。独立键盘有一端固定为低电平，此种方式编程比较简单。 而矩阵键盘两端都与单片机 I/O 口相连，因此在检测时需编程通过单片机 I/O 口送出低电平。检测方法有多种，最常用的是行列扫描和线翻转法。

行列扫描法检测时，先送一列为低电平，其余几列全为高电平(此时我们确定了列数)，然后立即轮流检测一次各行是否有低电平，若检测到某一行为低电平(这时我们又确定了行数)，则我们便可确认当前被按下的键是哪一行哪一列的，用同样方法轮流送各列一次低电平，再轮流检测一次各行是否变为低电平，这样即可检测完所有的按键，当有键被按下时便可判断出按下的键是哪一个键。当然我们也可以将行线置低电平，扫描列是否有低电平。从而达到整个键盘的检测。

线翻转法，就是使所有行线为低电平时，检测所有列线是否有低电平，如果有，就记录列线值；然后再翻转，使所有列线都为低电平，检测所有行线的值，由于有按键按下，行线的值也会有变化，记录行线的值。从而就可以检测到全部按键。

矩阵键盘也少不了按键消抖的环节，本章实验中采用的是行列扫描法来检测哪个按键按下。

14.2 硬件设计

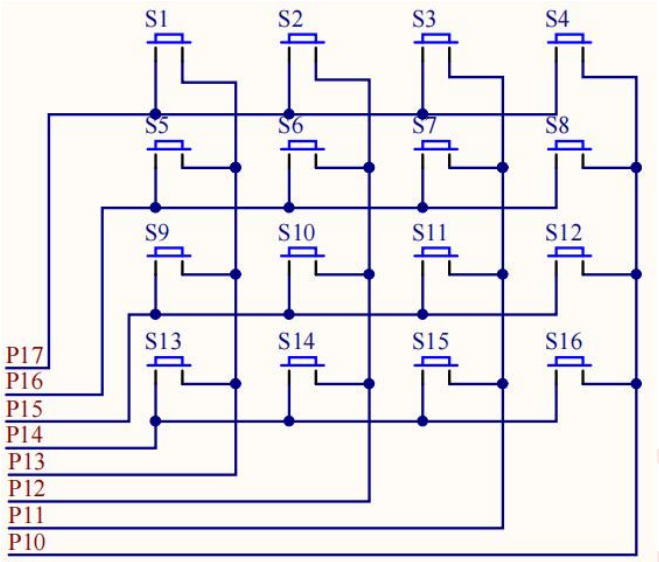
本实验使用到硬件资源如下：

（1）静态数码管

（2）4*4 矩阵按键

静态数码管模块电路在前面章节都介绍过，这里就不多说，开发板上的矩阵按键模块电路如下图所示：

普中51单片机开发板



从上图中可以看出，4*4 矩阵按键引出的 8 根控制线直接连接到 51 单片机的 P1 口上。电路中的 P17 连接矩阵键盘的第 1 行，P13 连接矩阵键盘第 1 列。

14.3 软件设计

本章所要实现的功能是：通过数码管显示矩阵按键 S1-S16 按下后键值 0-F。

我们打开“\4--实验程序\1--基础实验\9-矩阵按键实验”工程，控制代码全部都在 main.c 中，代码如下：

```
/*********************************************************************
******************
实验名称：矩阵按键实验
接线说明：
实验现象：下载程序后，按下“矩阵按键”模块中 S1-S16 键，对应数码管最左边显示 0-F
注意事项：

*********************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

#define KEY_MATRIX_PORT  P1  //使用宏定义矩阵按键控制口

#define SMG_A_DP_PORT    P0  //使用宏定义数码管段码口

//共阴极数码管显示 0~F 的段码数据
```

```
u8 gsmg_code[17]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,
                  0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x79,0x71};

/*************************************************************************
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*************************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*************************************************************************
***********
* 函 数 名       : key_matrix_ranks_scan
* 函数功能       : 使用行列式扫描方法，检测矩阵按键是否按下，按下则返回对应键值
* 输    入       : 无
* 输    出       : key_value：1-16，对应 S1-S16 键，
                   0：按键未按下
*************************************************************************
**********/
u8 key_matrix_ranks_scan(void)
{
    u8 key_value=0;

    KEY_MATRIX_PORT=0xf7;//给第一列赋值 0，其余全为 1
    if(KEY_MATRIX_PORT!=0xf7)//判断第一列按键是否按下
    {
        delay_10us(1000);//消抖
        switch(KEY_MATRIX_PORT)//保存第一列按键按下后的键值
        {
            case 0x77: key_value=1;break;
            case 0xb7: key_value=5;break;
            case 0xd7: key_value=9;break;
            case 0xe7: key_value=13;break;
        }
    }
    while(KEY_MATRIX_PORT!=0xf7);//等待按键松开

    KEY_MATRIX_PORT=0xfb;//给第二列赋值 0，其余全为 1
```

```
    if(KEY_MATRIX_PORT!=0xfb)//判断第二列按键是否按下
    {
        delay_10us(1000);//消抖
        switch(KEY_MATRIX_PORT)//保存第二列按键按下后的键值
        {
            case 0x7b: key_value=2;break;
            case 0xbb: key_value=6;break;
            case 0xdb: key_value=10;break;
            case 0xeb: key_value=14;break;
        }
    }
    while(KEY_MATRIX_PORT!=0xfb);//等待按键松开

    KEY_MATRIX_PORT=0xfd;//给第三列赋值 0，其余全为 1
    if(KEY_MATRIX_PORT!=0xfd)//判断第三列按键是否按下
    {
        delay_10us(1000);//消抖
        switch(KEY_MATRIX_PORT)//保存第三列按键按下后的键值
        {
            case 0x7d: key_value=3;break;
            case 0xbd: key_value=7;break;
            case 0xdd: key_value=11;break;
            case 0xed: key_value=15;break;
        }
    }
    while(KEY_MATRIX_PORT!=0xfd);//等待按键松开

    KEY_MATRIX_PORT=0xfe;//给第四列赋值 0，其余全为 1
    if(KEY_MATRIX_PORT!=0xfe)//判断第四列按键是否按下
    {
        delay_10us(1000);//消抖
        switch(KEY_MATRIX_PORT)//保存第四列按键按下后的键值
        {
            case 0x7e: key_value=4;break;
            case 0xbe: key_value=8;break;
            case 0xde: key_value=12;break;
            case 0xee: key_value=16;break;
        }
    }
    while(KEY_MATRIX_PORT!=0xfe);//等待按键松开

    return key_value;
}
```

```
/*********************************************************************
***********
* 函 数 名         : key_matrix_flip_scan
* 函数功能         : 使用线翻转扫描方法，检测矩阵按键是否按下，按下则返回对应键值
* 输    入         : 无
* 输    出         : key_value：1-16，对应 S1-S16 键，
                     0：按键未按下
*********************************************************************
**********/
u8 key_matrix_flip_scan(void)
{
    static u8 key_value=0;

    KEY_MATRIX_PORT=0x0f;//给所有行赋值 0，列全为 1
    if(KEY_MATRIX_PORT!=0x0f)//判断按键是否按下
    {
        delay_10us(1000);//消抖
        if(KEY_MATRIX_PORT!=0x0f)
        {
            //测试列
            KEY_MATRIX_PORT=0x0f;
            switch(KEY_MATRIX_PORT)//保存行为 0，按键按下后的列值
            {
                case 0x07: key_value=1;break;
                case 0x0b: key_value=2;break;
                case 0x0d: key_value=3;break;
                case 0x0e: key_value=4;break;
            }
            //测试行
            KEY_MATRIX_PORT=0xf0;
            switch(KEY_MATRIX_PORT)//保存列为 0，按键按下后的键值
            {
                case 0x70: key_value=key_value;break;
                case 0xb0: key_value=key_value+4;break;
                case 0xd0: key_value=key_value+8;break;
                case 0xe0: key_value=key_value+12;break;
            }
            while(KEY_MATRIX_PORT!=0xf0);//等待按键松开
        }
    }
    else
        key_value=0;

    return key_value;
```

```
}

/*********************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*********************************************************************
**********/
void main()
{
    u8 key=0;

    while(1)
    {
        key=key_matrix_ranks_scan();
        if(key!=0)
            SMG_A_DP_PORT=gsmg_code[key-1];//得到的按键值减 1 换算成数组下标
对应 0-F 段码
    }
}
```

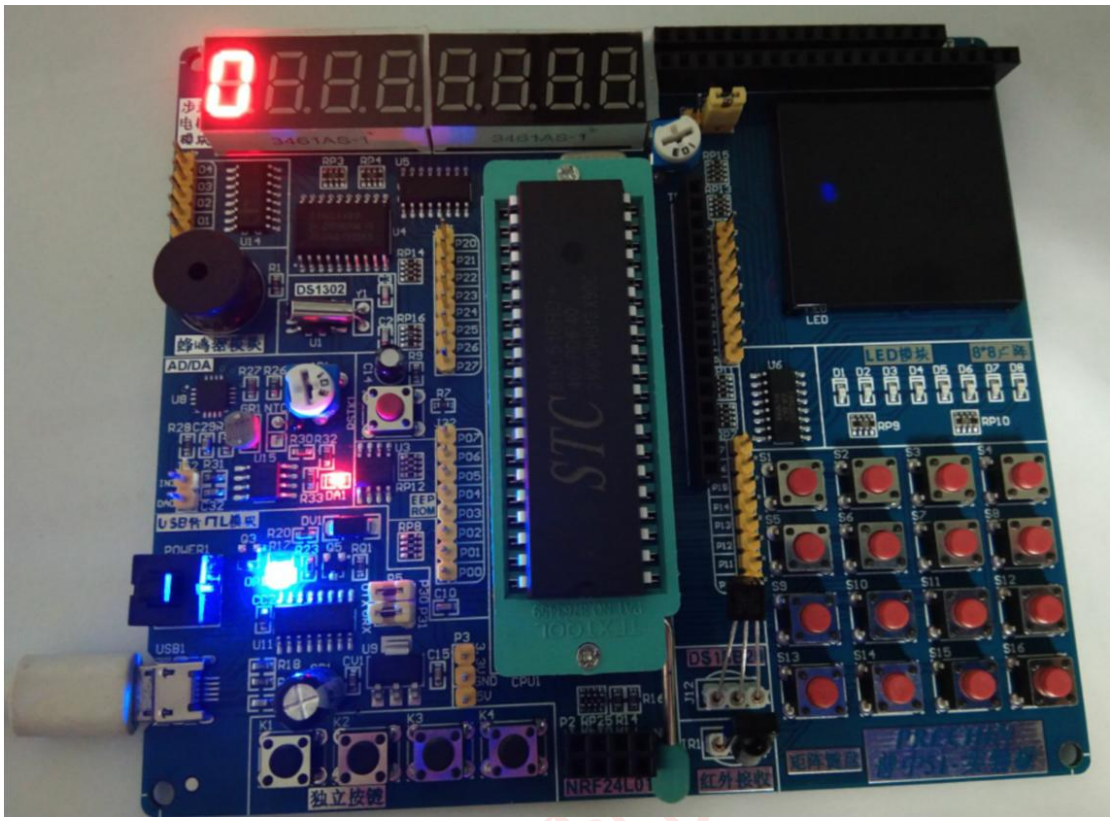
本实验核心代码为 key_matrix_ranks_scan 函数和 key_matrix_flip_scan 函数，前者是使用行列式扫描方式实现，而后者是使用线翻转式扫描方式实现，实现功能一致，二者可选其一。对于初学者，可能行列式扫描方式更易于理解，因为比较接近独立按键的编程方式。

行列式扫描原理比较简单，与独立式按键操作类似，即给每一列赋值 0，此时的矩阵按键就被分割成独立按键，然后再判断每一列中的按键按下情况，并返回对应的键值。如此循环 4 组，就可将 4 列 4 行按键按下键值全部得到。

而线翻转式扫描相对较难理解，不过静下心，在纸上画画，列举几个数据也是比较容易理解的。

14.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：当按下 S1-S16 键，最左边数码管对应显示 0-F。



课后作业

第 15 章 IO 扩展(串转并)-74HC595

在前面章节实验中，我们是直接使用单片机 IO 口控制外围设备，从 LED 流水灯到动态数码管显示，可以看到这些外围设备已经占据了很多的 IO 口，而 51 单片机 IO 口非常有限，如果想要连接更多外围设备，此时可以通过 IO 扩展来实

现。本章就来介绍另外一种 IO 口扩展方式-串转并，使用的芯片是 74HC595。开发板板载 1 个 74HC595 芯片，仅需单片机 3 个 IO 口即可扩展 8 个，如果需要还可以将 2 个 74HC595 级联扩展出 16 个 IO，这就实现用少数 IO 资源控制多个设备。通过本章的学习，让大家学会使用 74HC595 芯片来扩展 IO 口。本章所要实现的功能是：通过 74HC595 模块控制 LED 点阵以一行循环滚动显示。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

15.1 74HC595 芯片介绍

15.2 硬件设计

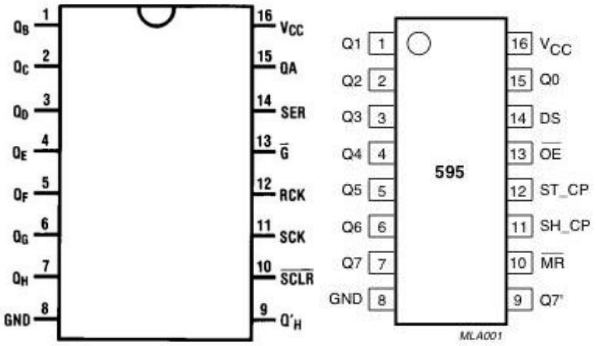
15.3 软件设计

15.4 实验现象

普中51单片机开发板

15.1 74HC595 芯片介绍

74HC595 是一个 8 位串行输入、并行输出的位移缓存器，其中并行输出为三态输出（即高电平、低电平和高阻抗）。芯片管脚及功能说明如下：



上面两张都是 74HC595 芯片管脚图，细心的朋友就会发现左侧的 1 脚是 QB，而右侧芯片的 1 脚是 Q1，左侧芯片的 11 脚是 SCK，而右侧芯片的 11 脚是 SH_CP，还有很多其他管脚不一样，其实这个都没有什么，每个人在绘制芯片管脚图时命名可能不一样而已，看一个芯片重点是管脚功能。

15 和 1 到 7 脚 QA--QH：并行数据输出

9 脚 QH 非：串行数据输出

10 脚 SCLK 非（ MR） ： 低电平复位引脚

11 脚 SCK（ SHCP） ： 移位寄存器时钟输入

12 脚 RCK（ STCP） ： 存储寄存器时钟输入

13 脚 G 非（ OE） ： 输出有效

14 脚 SER（ DS） ： 串行数据输入

74HC595 是具有 8 位移位寄存器和一个存储器，三态输出功能。移位寄存器和存储器是单独的时钟。数据在 SCK 的上升沿输入，在 RCK 的上升沿进入到存储器中。如果两个时钟连在一起，则移位寄存器总是比存储器早一个脉冲。移位寄存器有一个串行输入（DS），和一个串行输出（Q7 非），和一个异步的低电平复位，存储寄存器有一个并行 8 位的，具有三态的总线输出，当 MR 为高电平，OE 为低电平时，数据在 SHCP 上升沿进入移位寄存器，在 STCP 上升沿输出到并行端口。

有关 74HC595 芯片的更多详细介绍，大家可以在资料“\6--芯片资料\开发板芯片数据手册”内查看。

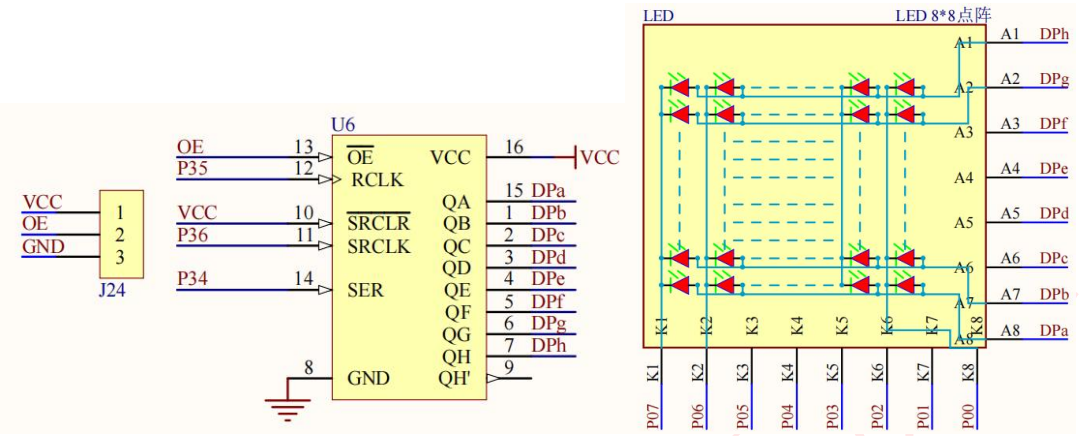
15.2 硬件设计

本实验使用到硬件资源如下：

（1）8*8LED 点阵模块

（2）74HC595 模块

开发板上的 74HC595 模块电路如下图所示：



从上图中可以看出，74HC595 需要用到的控制管脚 SER、RCLK、SRCLK 直接连接到 51 单片机的 P3.4-P3.6 IO 口上，输出端则是直接连接到 LED 点阵模块的行端口上，即为 LED 发光二极管的阳极，LED 点阵的列则为发光二极管的阴极。

要想控制 LED 点阵，可以将单片机管脚按照 74HC595 芯片的通信时序要求来传输数据，这样即可控制 LED 点阵的行数据。根据 LED 发光二极管导通原理，当阳极为高电平，阴极为低电平则点亮，否则熄灭。因此通过单片机 P0 口可控制点阵列，74HC595 可控制点阵行。

15.3 软件设计

本章所要实现的功能是：通过 74HC595 模块控制 LED 点阵以一行循环滚动显示。

我们打开“\4--实验程序\1--基础实验\10-IO 扩展(串转并)实验-74HC595”工程，控制代码全部都在 main.c 中，代码如下：

```
/************************************************************************
******************
实验名称：IO 扩展(串转并)实验-74HC595
接线说明：
实验现象：下载程序后，8*8LED 点阵以一行循环滚动显示
注意事项：LED 点阵旁的 J24 黄色跳线帽短接到 GND 一端

************************************************************************
```

```
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义 74HC595 控制管脚
sbit SRCLK=P3^6;  //移位寄存器时钟输入
sbit RCLK=P3^5;       //存储寄存器时钟输入
sbit SER=P3^4;        //串行数据输入

#define LEDDZ_COL_PORT   P0  //点阵列控制端口

u8 ghc595_buf[8]={0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80};

/*************************************************************************
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*************************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*************************************************************************
***********
* 函 数 名       : delay_ms
* 函数功能       : ms 延时函数，ms=1 时，大约延时 1ms
* 输    入       : ten_us
* 输    出       : 无
*************************************************************************
**********/
void delay_ms(u16 ms)
{
    u16 i,j;
    for(i=ms;i>0;i--)
        for(j=110;j>0;j--);
}

/*************************************************************************
```

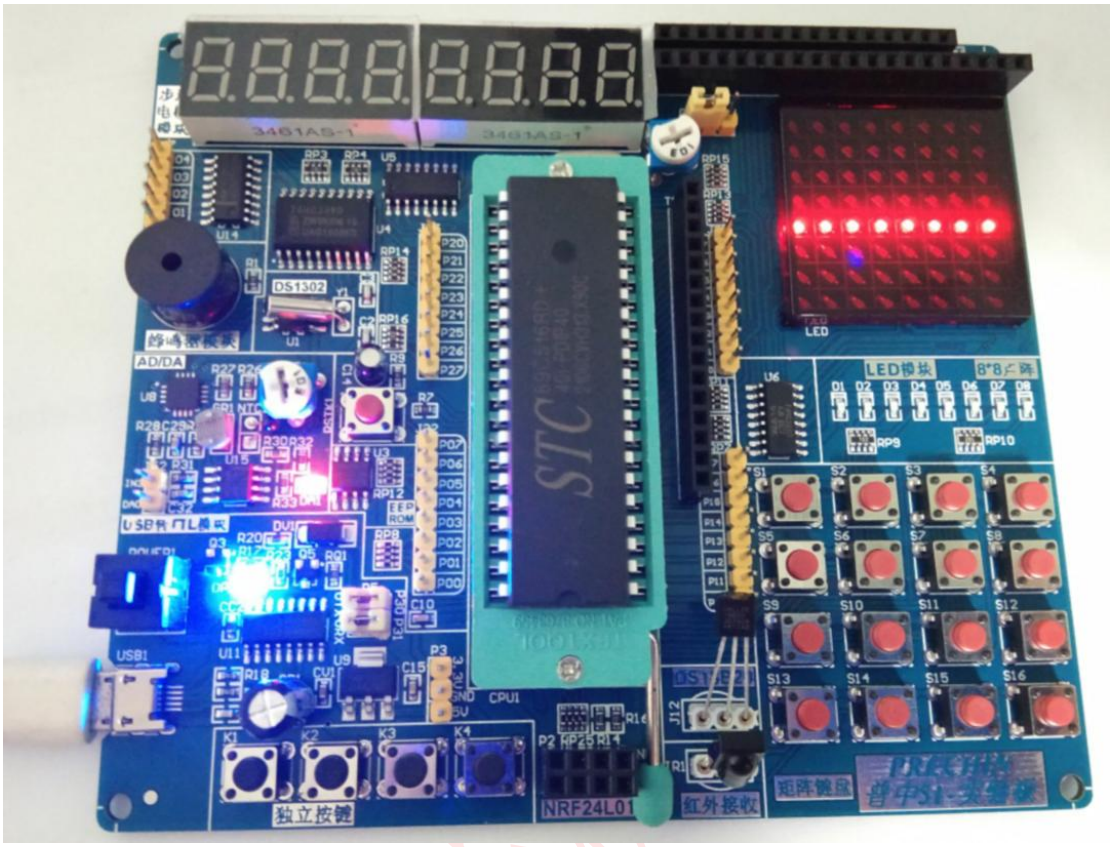
```
***********
* 函 数 名         : hc595_write_data(u8 dat)
* 函数功能         : 向 74HC595 写入一个字节的数据
* 输    入         : dat：数据
* 输    出         : 无
***********************************************************************
**********/
void hc595_write_data(u8 dat)
{
    u8 i=0;

    for(i=0;i<8;i++)//循环 8 次即可将一个字节写入寄存器中
    {
        SER=dat>>7;//优先传输一个字节中的高位
        dat<<=1;//将低位移动到高位
        SRCLK=0;
        delay_10us(1);
        SRCLK=1;
        delay_10us(1);//移位寄存器时钟上升沿将端口数据送入寄存器中
    }
    RCLK=0;
    delay_10us(1);
    RCLK=1;//存储寄存器时钟上升沿将前面写入到寄存器的数据输出
}



/***********************************************************************
***********
* 函 数 名       : main
* 函数功能        : 主函数
* 输    入       : 无
* 输    出        : 无
***********************************************************************
**********/
void main()
{
    u8 i=0;

    LEDDZ_COL_PORT=0x00;//将 LED 点阵列全部设置为 0，即 LED 阴极为低电平
    while(1)
    {
        for(i=0;i<8;i++)
        {
            hc595_write_data(0x00);//消除前面寄存器缓存数据
```

```
            hc595_write_data(ghc595_buf[i]);//写入新的数据
            delay_ms(500);//延时 500ms
        }
    }
}
```

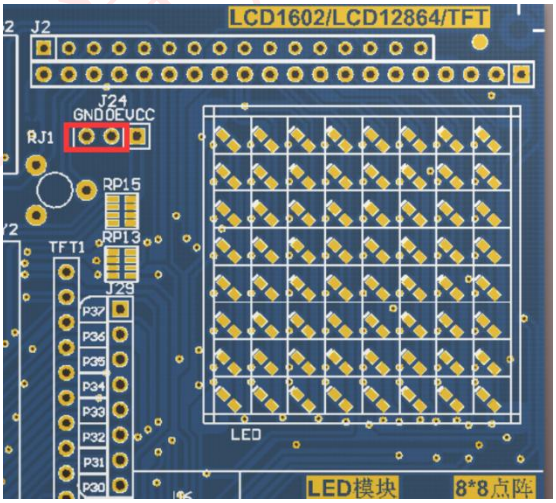
代码还是比较简单易懂的，首先定义好 74HC595 控制管脚，以及点阵列控制口，代码中重新定义了一个 ms 级延时函数 delay_ms，该函数与前面 delay_10us 类似，都是利用循环占用 CPU 起到延时效果。然后又定义了 74HC595 的控制函数 hc595_write_data，该函数完全按照 74HC595 的通信时序要求编写，主要要注意的是 74HC595 是先传输字节的高位后传输低位，所以需要将字节低位移动到高位传输，在传输数据时，要注意移位寄存器时钟和存储寄存器时钟的先后顺序，将要写入的数据先传输到 74HC595 寄存器中，即在准备好每位数据时要将 SRCLK 进行一个上升沿变化，此时即可将数据传输到寄存器内，待循环 8 次即一个字节传输到寄存器中时，就可以来一个存储时钟上升沿，此时就可以将 74HC595 寄存器中的数据全部一次传输到 595 端口输出。最后就是在 main 函数中调用 74HC595 的控制函数，将实验中要实现的效果数据写入进去，从而控制 LED 点阵的阳极，而阴极由 P0 口控制，默认初始化时已经设置为 0，也就是说只要 595 输出高电平，那么对应的行就会点亮。

15.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：8*8LED 点阵以一行循环滚动显示。



注意：做 LED 点阵实验时，一定要将 LED 点阵旁的 J24 黄色跳线帽短接到 GND 一端。如下所示：



课后作业

第 16 章 LED 点阵实验

在前面章节，我们介绍过静态数码管和动态数码管显示，其中动态数码管是一种应用非常多的显示设备，除此之外还有很多应用广泛的显示装置，比如 LED 点阵屏、LCD 液晶显示器等。我们开发板上使用了 64 个红色 LED 按照行列排布组成的 8*8LED 点阵。本章就来介绍 LED 点阵的使用，通过本章的学习，让大家能够在 8*8LED 点阵屏上显示信息。本章所要实现的功能是：在 8*8LED 点阵屏上点亮一个点，上述功能实现后再显示数字图形等。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

16.1 LED 点阵介绍

16.2 硬件设计

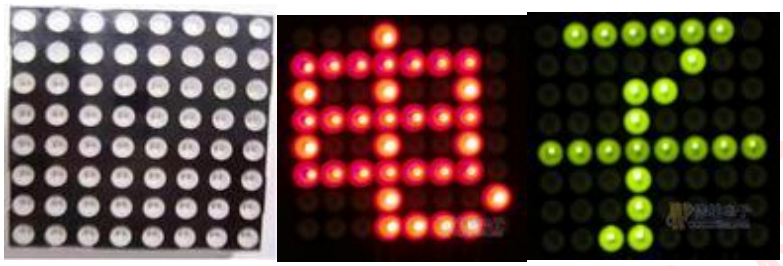
16.3 软件设计

16.4 实验现象

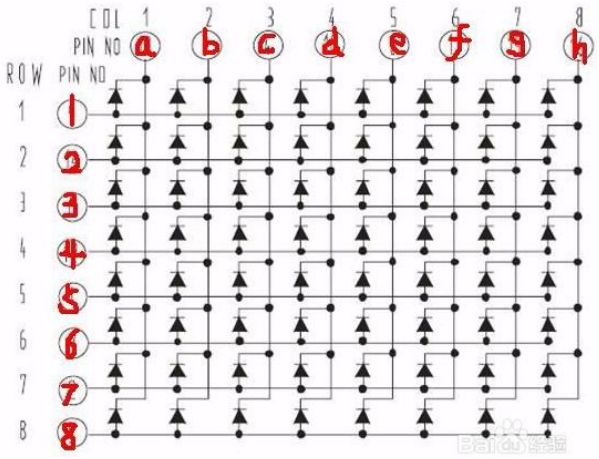
普中51单片机开发板

16.1 LED 点阵介绍

LED 点阵是由发光二极管排列组成的显示器件,在我们日常生活的电器中随处可见，被广泛应用于汽车报站器，广告屏等。如下所示：



通常应用较多的是 8*8 点阵,然后使用多个 8*8 点阵可组成不同分辨率的 LED 点阵显示屏，比如 16*16 点阵可以使用 4 个 8*8 点阵构成。因此理解了 8*8LED 点阵的工作原理，其他分辨率的 LED 点阵显示屏都是一样的。这里以 8*8LED 点阵来做介绍。其内部结构图如下所示：



8*8 点阵共由 64 个发光二极管组成，且每个发光二极管是放置在行线和列线的交叉点上，当对应的某一行置 1 电平，某一列置 0 电平，则相应的二极管就亮；如要将第一个点点亮，则 1 脚接高电平 a 脚接低电平，则第一个点就亮了；如果要将第一行点亮，则第 1 脚要接高电平，而（a、b、c、d、e、f、g、h ）这些引脚接低电平，那么第一行就会点亮；如要将第一列点亮，则第 a 脚接低电平，而（1、2、3、4、5、6、7、8）接高电平，那么第一列就会点亮。由此可见，LED 点阵的使用也是非常简单的。

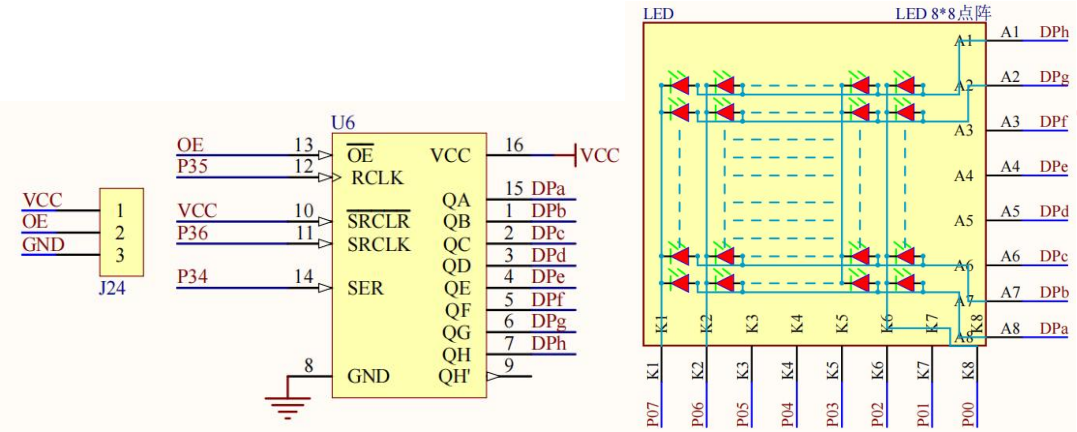
16.2 硬件设计

本实验使用到硬件资源如下：

（1）8*8LED 点阵模块

（2）74HC595 模块

74HC595 模块、8*8LED 点阵模块电路在前面章节都介绍过，电路如下：



从上图中可以看出，74HC595 需要用到的控制管脚 SER、RCLK、SRCLK 直接连接到 51 单片机的 P3.4-P3.6 IO 口上，输出端则是直接连接到 LED 点阵模块的行端口上，即为 LED 发光二极管的阳极，LED 点阵的列则为发光二极管的阴极。

要想控制 LED 点阵，可以将单片机管脚按照 74HC595 芯片的通信时序要求来传输数据，这样即可控制 LED 点阵的行数据。根据 LED 发光二极管导通原理，当阳极为高电平，阴极为低电平则点亮，否则熄灭。因此通过单片机 P0 口可控制点阵列，74HC595 可控制点阵行。

16.3 软件设计

本章所要实现的功能是：在点阵屏上点亮一个点，上述功能实现后再显示数字和图形。

16.3.1 LED 点阵（点亮一个点）

我们打开“\4--实验程序\1--基础实验\11-LED 点阵实验(点亮一个点)”工程，控制代码全部都在 main.c 中，代码如下：

```
/*****************************************************************
******************
实验名称：LED 点阵实验(点亮一个点)
接线说明：
实验现象：下载程序后，8*8LED 点阵点亮左上角第一个点
注意事项：LED 点阵旁的 J24 黄色跳线帽短接到 GND 一端

*****************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义 74HC595 控制管脚
sbit SRCLK=P3^6;  //移位寄存器时钟输入
sbit RCLK=P3^5;       //存储寄存器时钟输入
sbit SER=P3^4;        //串行数据输入

#define LEDDZ_COL_PORT   P0  //点阵列控制端口


/*****************************************************************
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*****************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*****************************************************************
***********
* 函 数 名         : hc595_write_data(u8 dat)
* 函数功能         : 向 74HC595 写入一个字节的数据
* 输    入         : dat：数据
* 输    出         : 无
*****************************************************************
**********/
void hc595_write_data(u8 dat)
```

```
{
    u8 i=0;

    for(i=0;i<8;i++)//循环 8 次即可将一个字节写入寄存器中
    {
        SER=dat>>7;//优先传输一个字节中的高位
        dat<<=1;//将低位移动到高位
        SRCLK=0;
        delay_10us(1);
        SRCLK=1;
        delay_10us(1);//移位寄存器时钟上升沿将端口数据送入寄存器中
    }
    RCLK=0;
    delay_10us(1);
    RCLK=1;//存储寄存器时钟上升沿将前面写入到寄存器的数据输出
}


/**********************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
**********************************************************************
**********/
void main()
{
    u8 i=0;

    LEDDZ_COL_PORT=0x7f;//将 LED 点阵左边第一列设置为 0，即 LED 阴极为低电平，其
余列为 1，即高电平
    while(1)
    {
        hc595_write_data(0x80);//将 LED 点阵上边第一行设置为 1，即 LED 阳极为高
电平，其余行为 0，即低电平
    }
}
```

代码很简单，与上一章核心代码是一样的，这里主要是理解如何让 LED 点阵的左上角第一个点点亮，实际上就是将第一个点对应的行为高电平，列为低电平即可。也就是让 74HC595 输出 0X80（1000 0000），这样点阵第一行就是高电平，而 P0 口输出 0X7F（0111 1111），这样点阵第一列就是低电平，从而让 LED 点

阵第一个点点亮。

16.3.2 LED 点阵（显示数字）

点亮一个点很简单，可是如何点亮多个点呢？如果需要一次显示多个怎么办？从原理图上可以看到每一行上都连接着多个 LED 灯，每一列上也都连接着多个 LED 灯，如果要点亮一个，按照上面原理可以，但是要同时点亮多个怎么办？

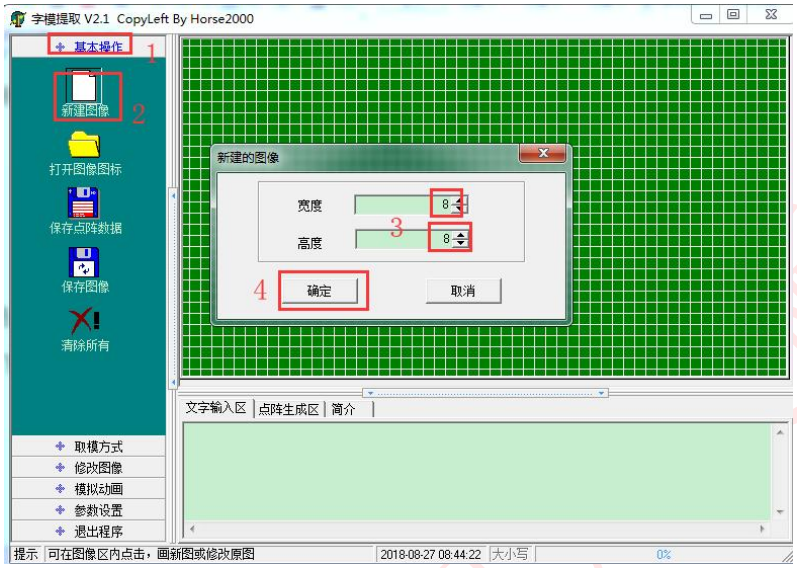
那么就需要用到动态数码管的动态扫描原理。首先如何点亮一行上面多个灯或者一列上面多个灯？明显需要某行或某列有效，同时使多列或多行有效。比如在第一行有效（输出高电平）的情况下，有效列（输出低电平）与这一行交点上的 LED 灯就会被点亮。那么实现一行或一列点亮会比较容易。如何实现不同行不同列上的灯被多个点亮呢？ 是否是行有效，列有效就可以？并不是！

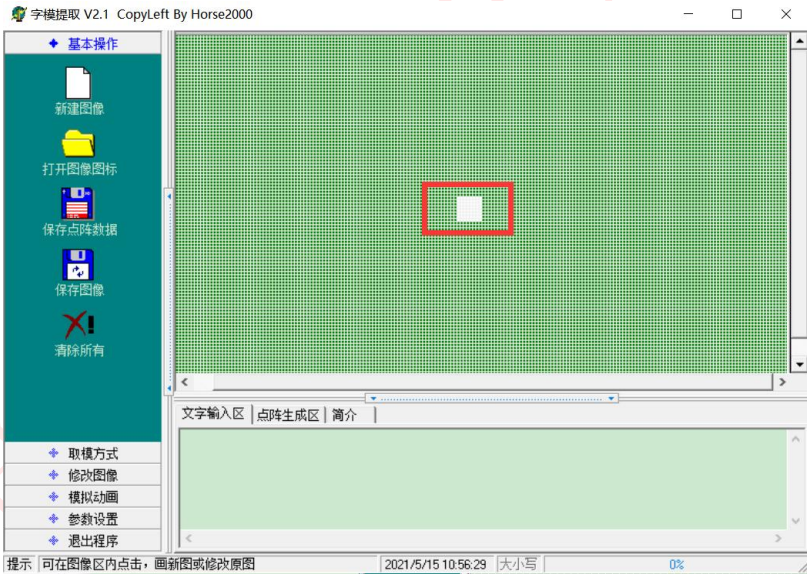
要实现行列不同位置亮灯，需要使用动态显示的方法，也要结合扫描的方法。在第一行亮灯一段时间以后灭掉，点亮第二行一段时间以后灭掉，点亮第三行一段时间以后灭掉，如此点亮，直到八行全部点亮一次，在第一行点亮到最后一行灭掉的总时间不能超过人肉眼可识别的时间，即 24 毫秒。在每一行点亮的时候，给列一个新的数据，此时对应列的数据就可以体现在这行上要点亮的灯上。这样就和动态数码管的显示一样，只不过数码管的 LED 灯是段值。这里使用 LED 点阵显示数字，也是多个 LED 同时点亮。

要想在点阵上显示数字等字符，首先要获取在 LED 点阵上显示数字字符所需的数据，即一个数字字符在 LED 点阵上显示，对应的每行每列都会有一些灯点亮或者熄灭，这样就会构成一组数据，也就是数字字符的显示数据，我们只要将这些数据通过 74HC595 发送到点阵对应的行或列就能显示数字字符。

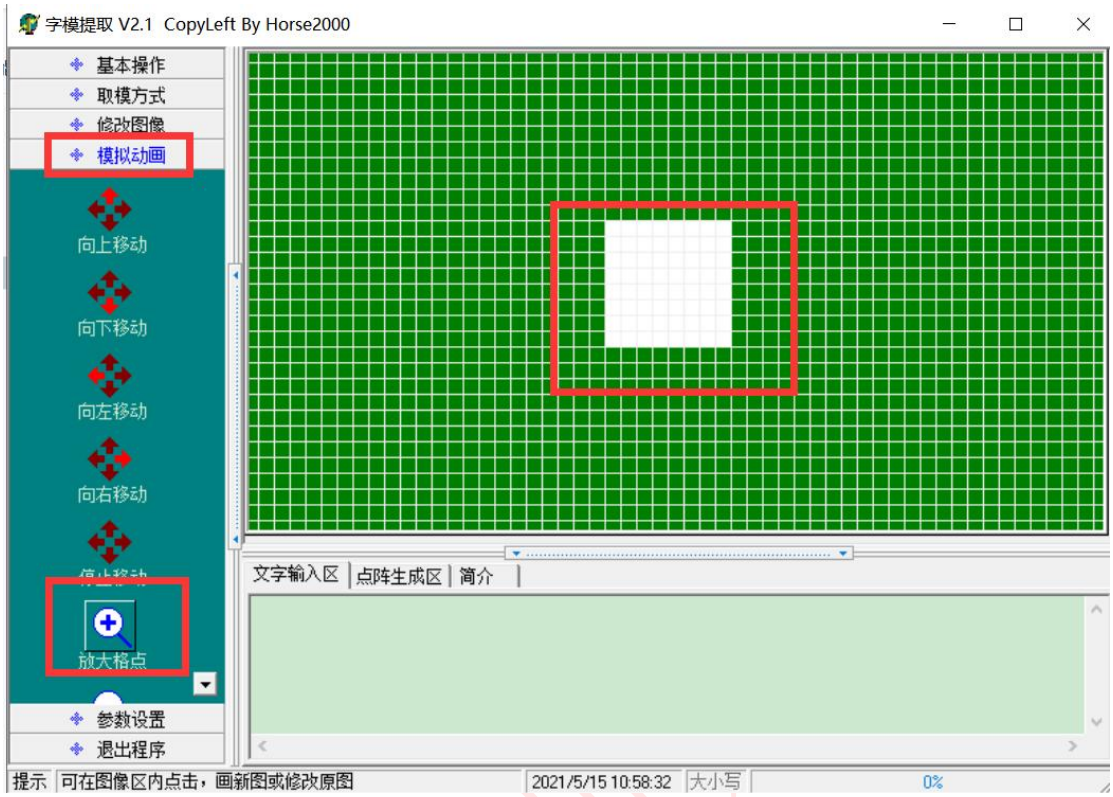
数字字符数据如何获取呢？这里给大家介绍一个非常好用的工具-取字模软件。该软件在“\5--开发工具\4-常用辅助开发软件\文字取模软件”内，如下所示：

双击打开该软件，首先选择“基本操作->新建图像”，设置图像的宽度和高度为8，点击确定后将在显示窗口出现一个8*8的白色格子，这个就类似于8*8LED点阵，具体操作如下：



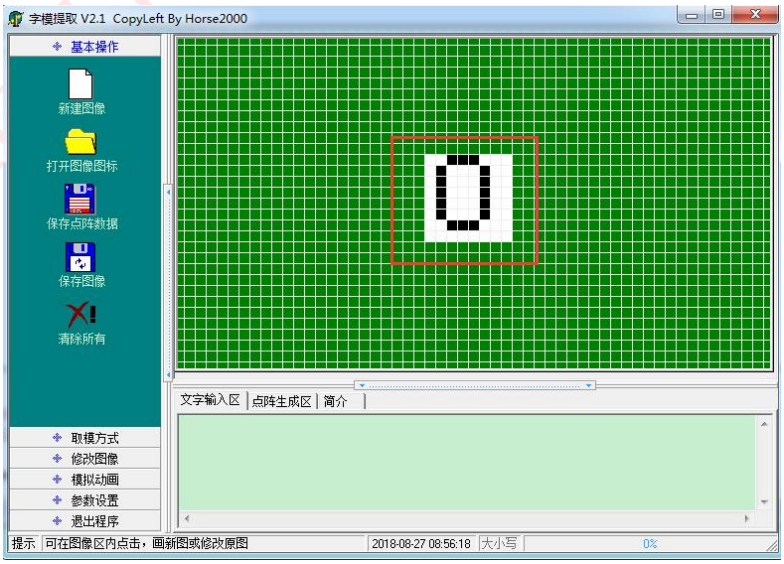


可以看到上图8*8点阵区域非常小，我们可以将其放大，选择“模拟动画”，后点击“放大格点”，如下所示：

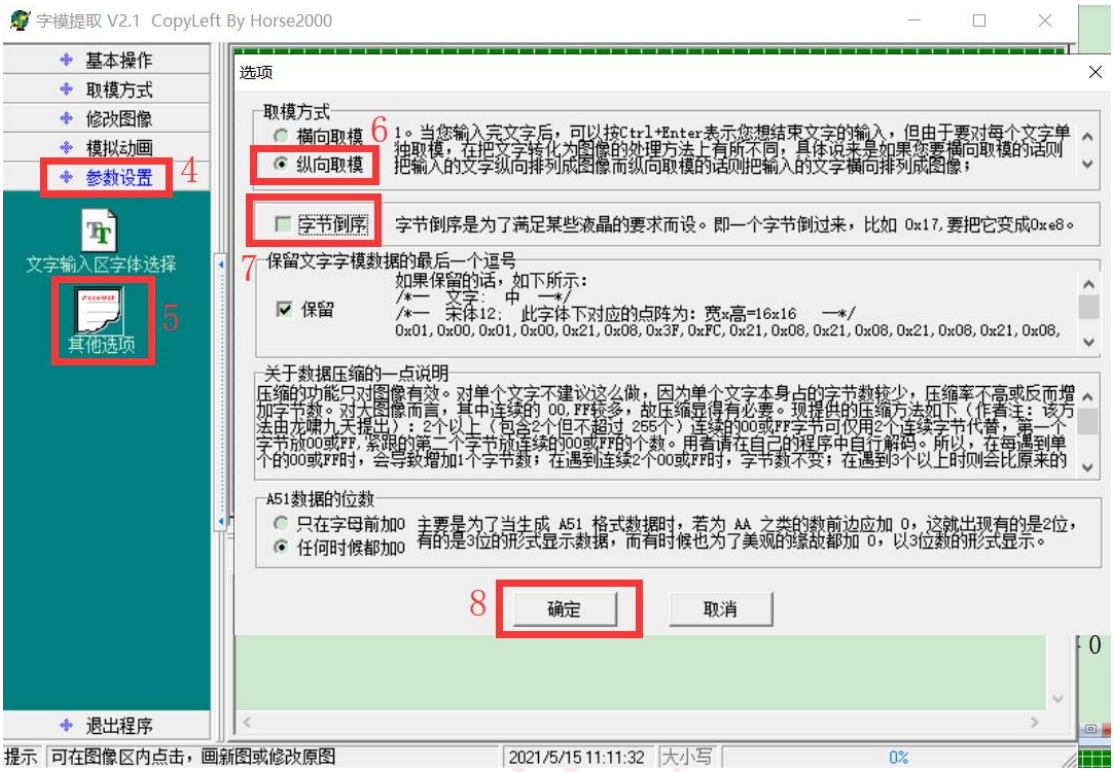


然后可以在这个 8*8 白色格子里面点击，点击后即会在对应位置出现一个黑点，表示在 LED 点阵对应位置的 LED 灯点亮，未点击位置（白色）表示 LED 点阵对应位置的 LED 灯熄灭。

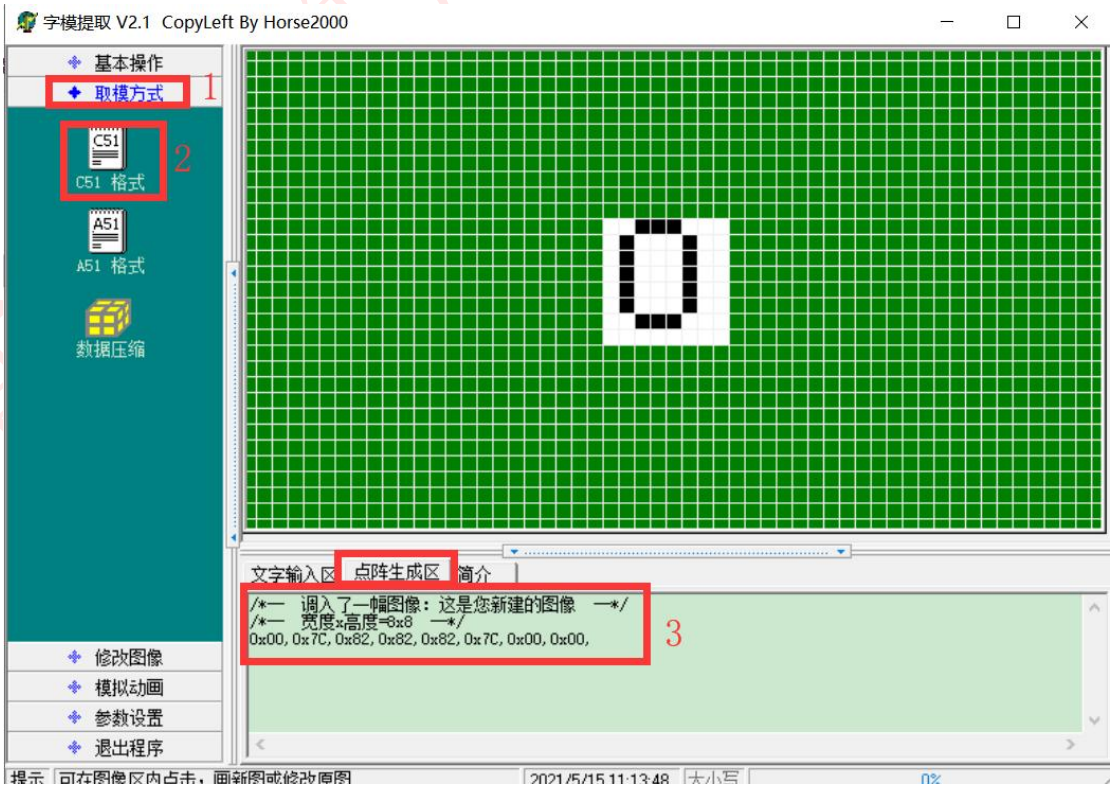
比如在 8*8LED 点阵上显示数字 0，那么可以在图中 8*8 白色框内通过点击对应位置描出一个数字 0 的外形，如下所示：



然后设置取模数据的取模方式等内容，选择“参数设置”后点击“其他选项”，具体操作如下：



然后点击“取模方式”，选择 C51 格式选项，然后在点阵生成区自动会生成数字字符对应的数据（如果是使用汇编编程，那么汇编对应的汉字数据可选择 A51 格式）。如下所示：



到这里我们就将数字 0 的数据生成了，然后将生成的数据复制到我们程序定

义的数组中，如下所示：

```
    u8 gled_row[8]={0x00,0x7C,0x82,0x82,0x82,0x7C,0x00,0x00};//LED 点阵显示数字 0 的行数据
```

这些数据其实就是上述描绘的数字 0 从左到右依次每列对应的行数据。

至于其它数字或字符及简单图形的显示取模方法与上述类似。

既然是动态扫描，就需要不断的扫描每列，因此可以把 LED 点阵的列控制也用数组存储起来，为后面循环调用提供方便。根据数字 0 取模的数据特点是从左至右每列对应的行数据，因此扫描时也应该从左至右的顺序，如下：

```
    u8 gled_col[8]={0x7f,0xbf,0xdf,0xef,0xf7,0xfb,0xfd,0xfe};//LED 点阵显示数字 0 的列数据
```

我们打开“\4--实验程序\1--基础实验\12-LED 点阵实验(显示数字)”工程，控制代码全部都在 main.c 中，代码如下：

```
/*********************************************************************
******************
实验名称：LED 点阵实验(显示数字)
接线说明：
实验现象：下载程序后，8*8LED 点阵显示数字 0
注意事项：LED 点阵旁的 J24 黄色跳线帽短接到 GND 一端

*********************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义 74HC595 控制管脚
sbit SRCLK=P3^6;  //移位寄存器时钟输入
sbit RCLK=P3^5;       //存储寄存器时钟输入
sbit SER=P3^4;        //串行数据输入

#define LEDDZ_COL_PORT   P0  //点阵列控制端口

u8 gled_row[8]={0x00,0x7C,0x82,0x82,0x82,0x7C,0x00,0x00};//LED 点阵显示数字 0 的行数据
u8 gled_col[8]={0x7f,0xbf,0xdf,0xef,0xf7,0xfb,0xfd,0xfe};//LED 点阵显示数字 0 的列数据
```

```
/*************************************************************************
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*************************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*************************************************************************
***********
* 函 数 名         : hc595_write_data(u8 dat)
* 函数功能         : 向 74HC595 写入一个字节的数据
* 输    入         : dat：数据
* 输    出         : 无
*************************************************************************
**********/
void hc595_write_data(u8 dat)
{
    u8 i=0;

    for(i=0;i<8;i++)//循环 8 次即可将一个字节写入寄存器中
    {
        SER=dat>>7;//优先传输一个字节中的高位
        dat<<=1;//将低位移动到高位
        SRCLK=0;
        delay_10us(1);
        SRCLK=1;
        delay_10us(1);//移位寄存器时钟上升沿将端口数据送入寄存器中
    }
    RCLK=0;
    delay_10us(1);
    RCLK=1;//存储寄存器时钟上升沿将前面写入到寄存器的数据输出
}


/*************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
```

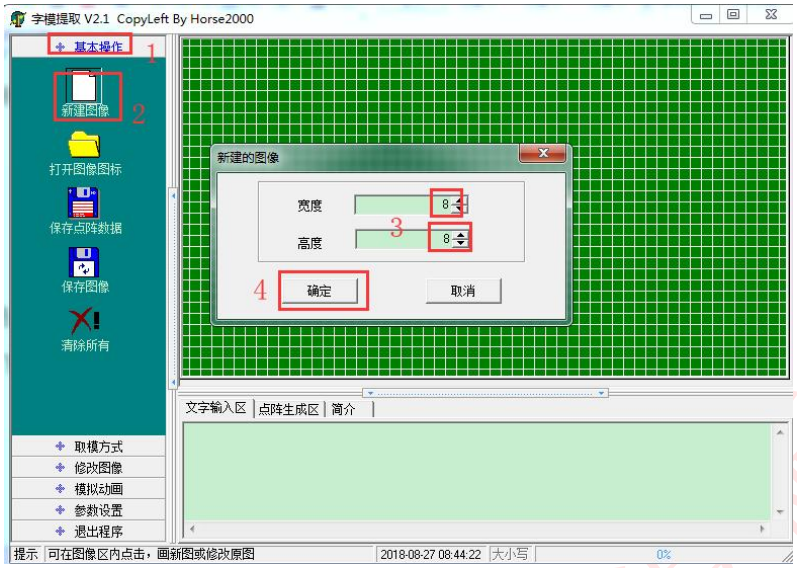
```
* 输    入         : 无
* 输    出         : 无
*******************************************************************************
**********/
void main()
{
    u8 i=0;

    while(1)
    {
        for(i=0;i<8;i++)//循环 8 次扫描 8 行、列
        {
            LEDDZ_COL_PORT=gled_col[i];//传送列选数据
            hc595_write_data(gled_row[i]);//传送行选数据
            delay_10us(100);//延时一段时间，等待显示稳定
            hc595_write_data(0x00);//消影
        }
    }
}
```

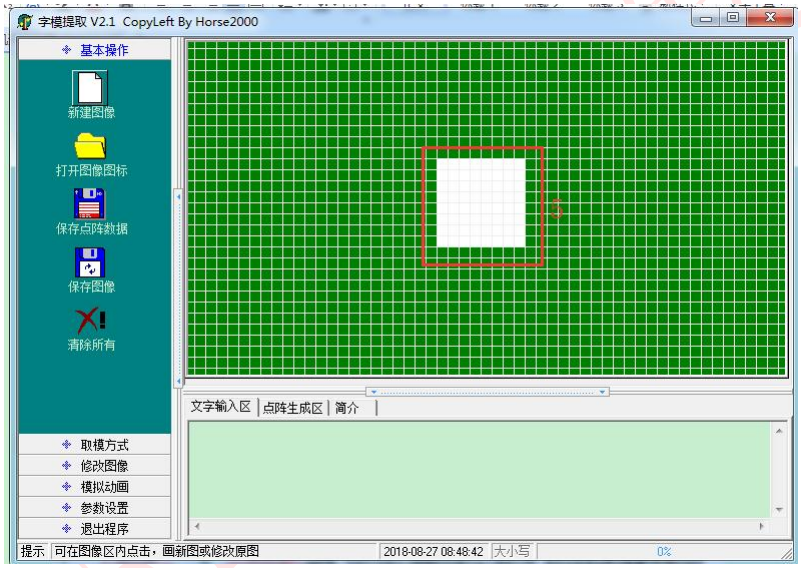
实验代码与前面点亮一个点的核心代码是一样的，这里重点要理解动态扫描，如果动态数码管代码能理解，这个也就没有任何问题。main 函数中主要是在 while 循环内从上至下，从左至右不断扫描 8 行、列，即首先设置左边第一列有效（P0.7 输出低电平），其余列无效（P0.6-P0.0 输出高电平），然后通过 74HC595 输出该列对应的行数据，延时一段时间等待显示稳定，最后清除列对应的行数据，即消影。从整个流程下来与动态数码管显示程序是很相似的。

16.3.3 LED 点阵（显示图像）

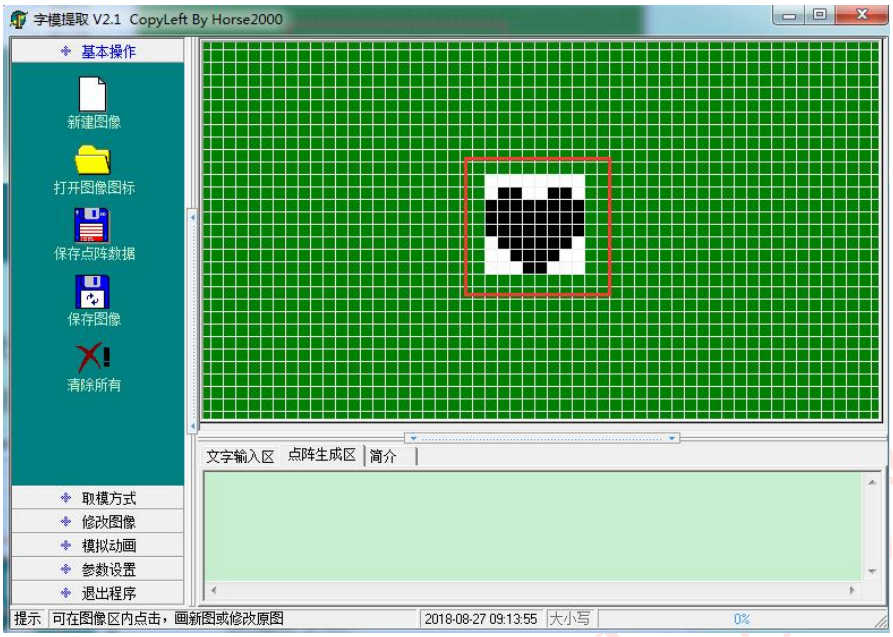
该取模软件不仅可以用来取汉字、数字等字符数据，还可以用来取图像数据，这里以“心”形图像为例，教大家如何使用该软件来获取图像数据。

（1）新建图像。弹出选择框，开发板 LED 点阵是 8X8 的，就建一个高 8 宽 8 的。

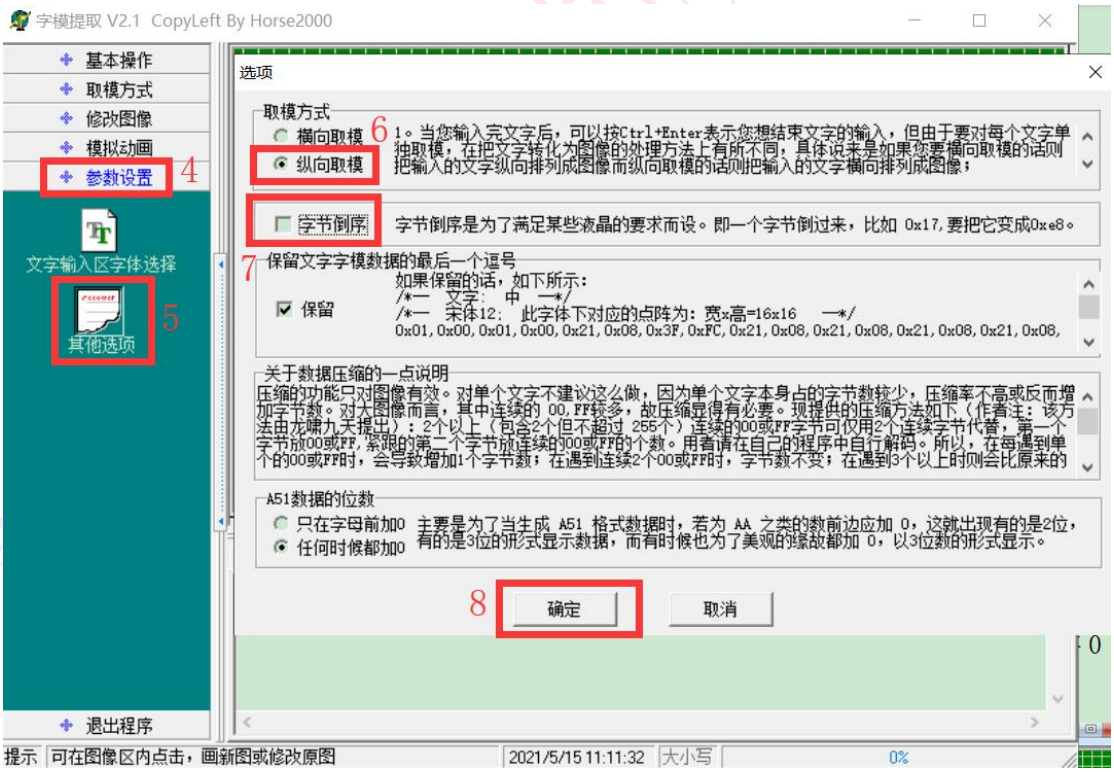




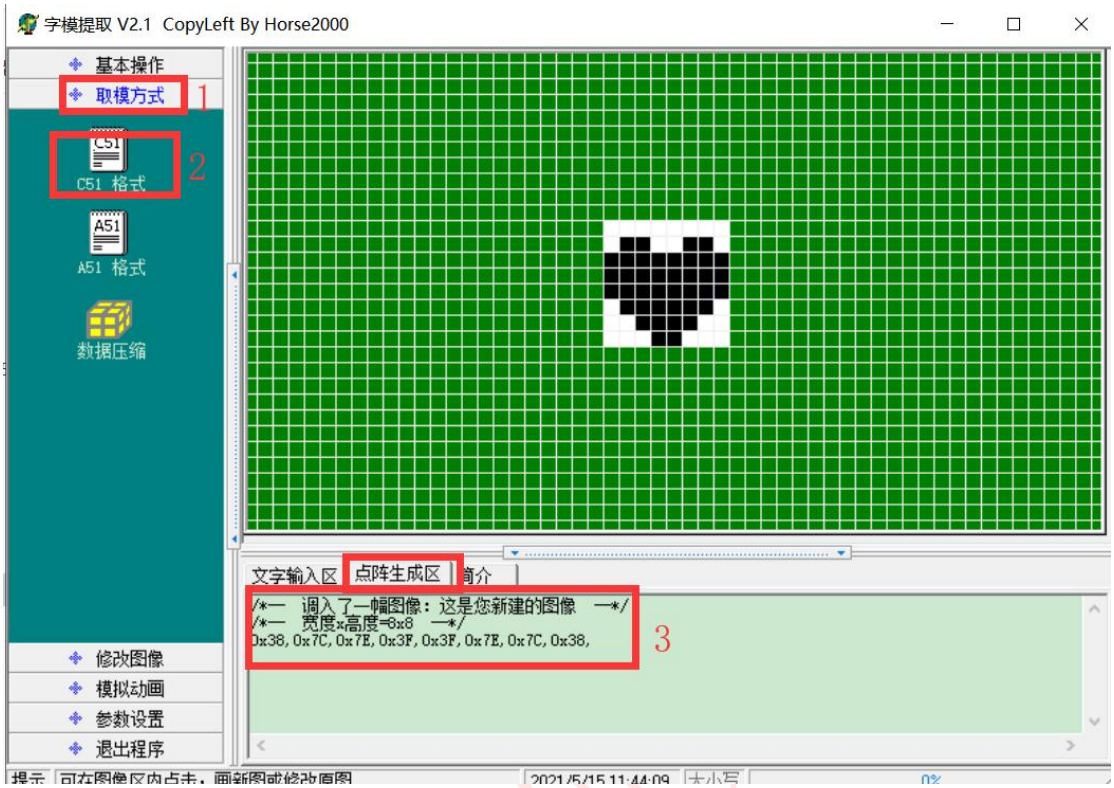
（2）选中图像需用格点。这里每一个格点代表一个 LED 灯，点击可选中，再次点击也可取消。在界面上点出爱心。如下图：



（3）参数设置。“参数设置” 选项中有许多可选项，可以自己进行设置，得到自己想要的结果。可参考前面取模里面的设置。如下所示：



（4）生成数据。设计好图案和参数以后，就需要生成可用数据。当点击格式以后会出现相应数据。如下图所示



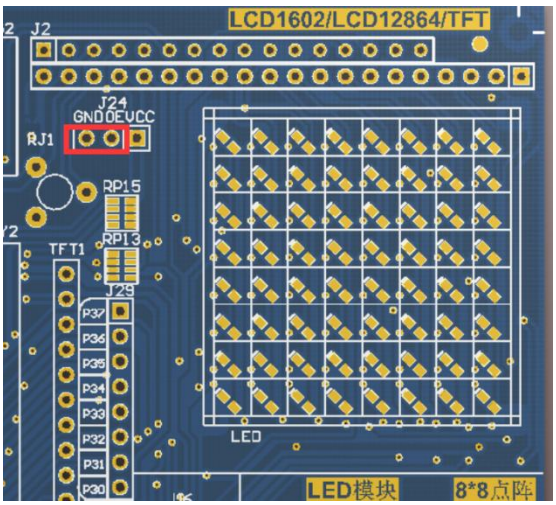
这里将生成的图形数据复制出来粘贴到例程中，替换数字 0 的数据，如下：

```
u8 gled_row[8]={0x38,0x7C,0x7E,0x3F,0x3F,0x7E,0x7C,0x38};//LED 点阵显示图像的行数据
```

其余代码与显示数字完全一样，这里就不再列出。

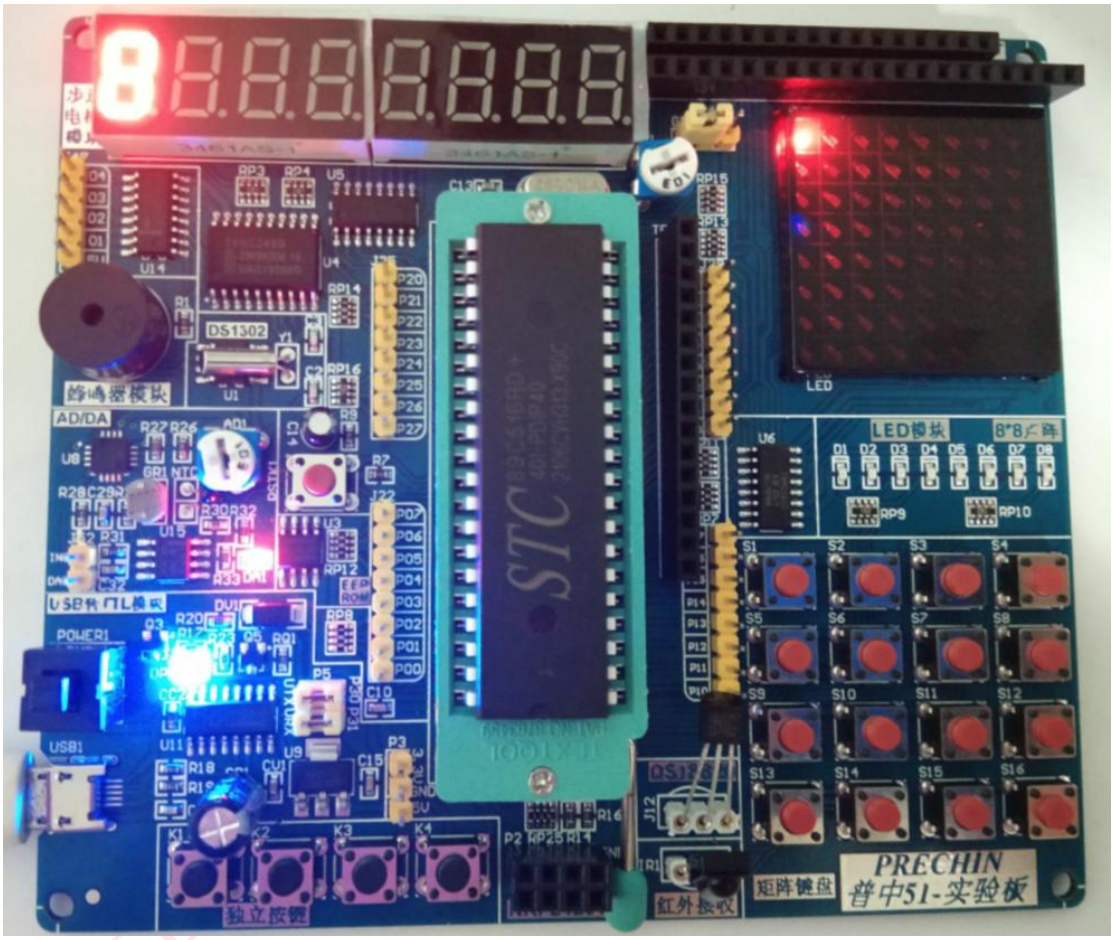
16.4 实验现象

注意：做 LED 点阵实验时，一定要将 LED 点阵旁的 J24 黄色跳线帽短接到 GND 一端。如下所示：



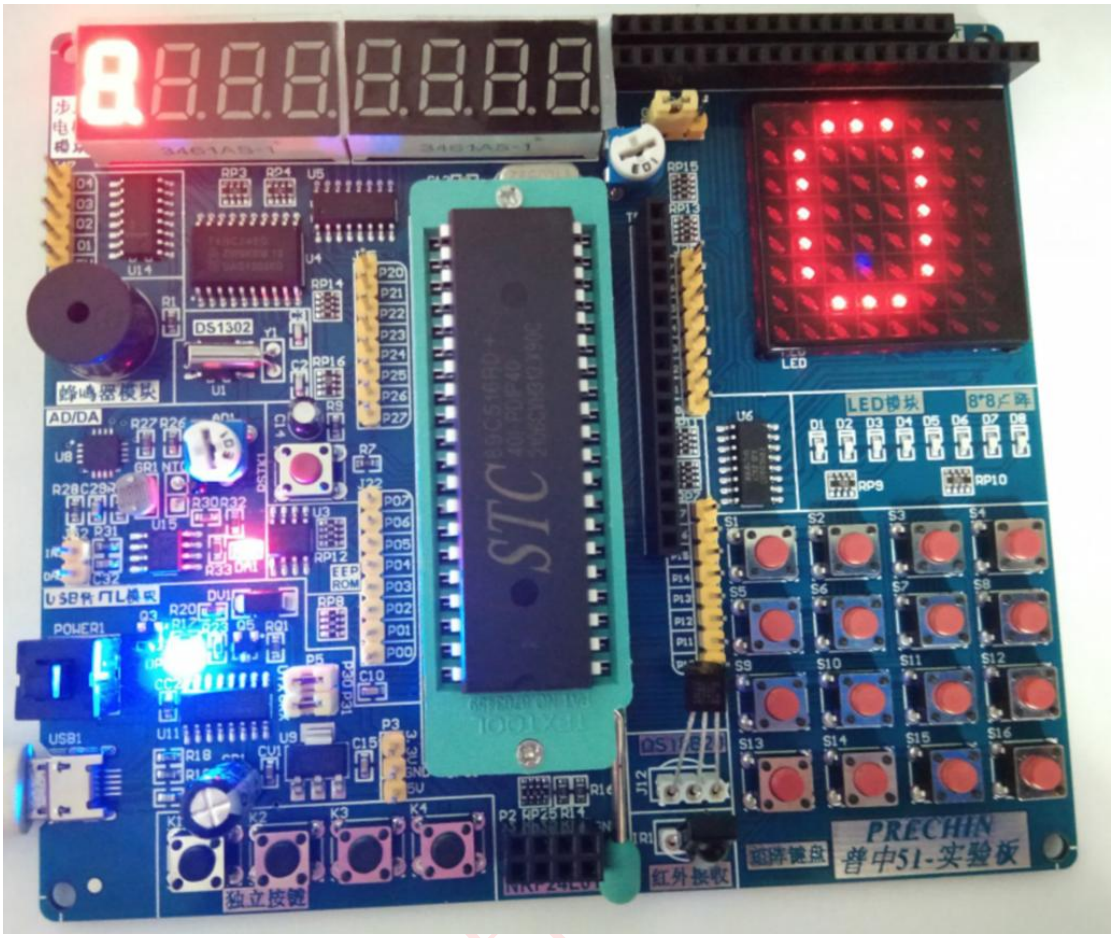
16.4.1 LED 点阵（点亮一个点）

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：8*8LED 点阵左上角显示第一个点。



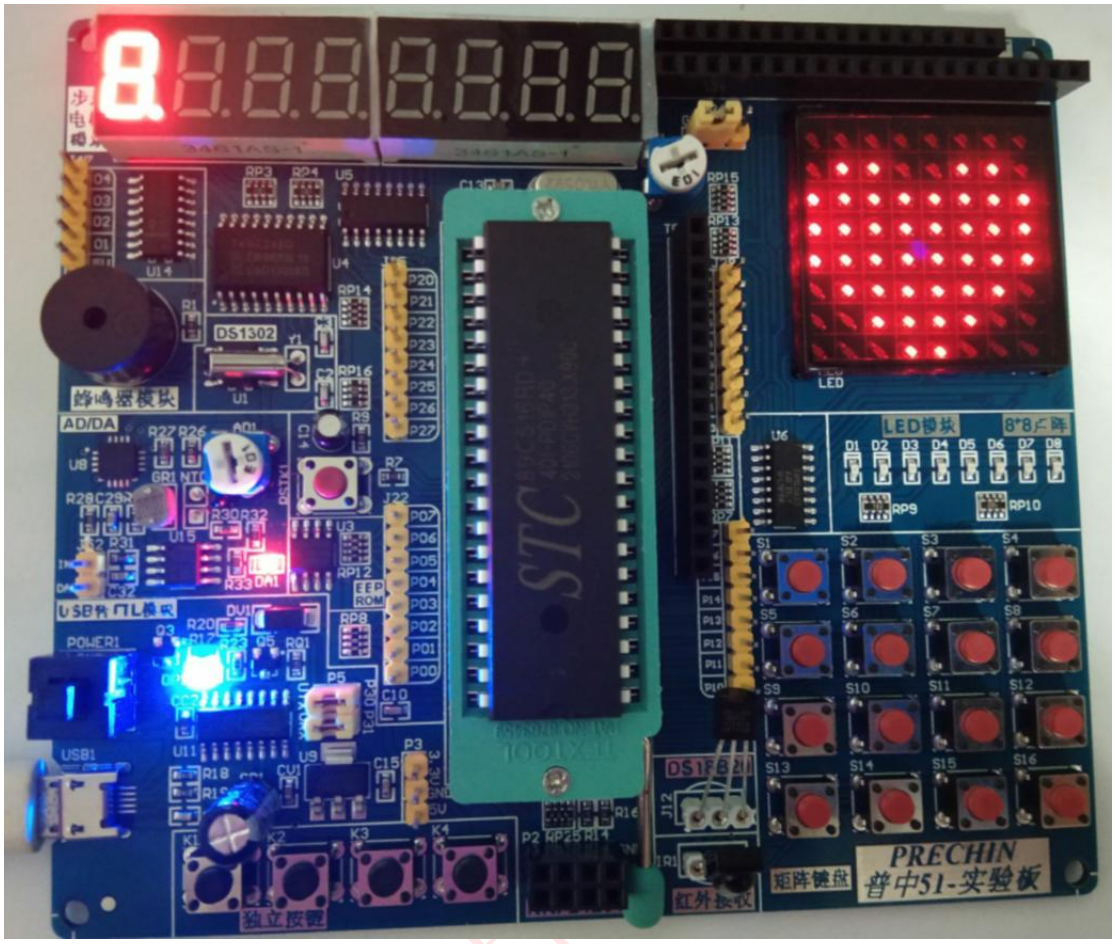
16.4.2 LED 点阵（显示数字）

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：8*8LED 点阵显示数字 0。

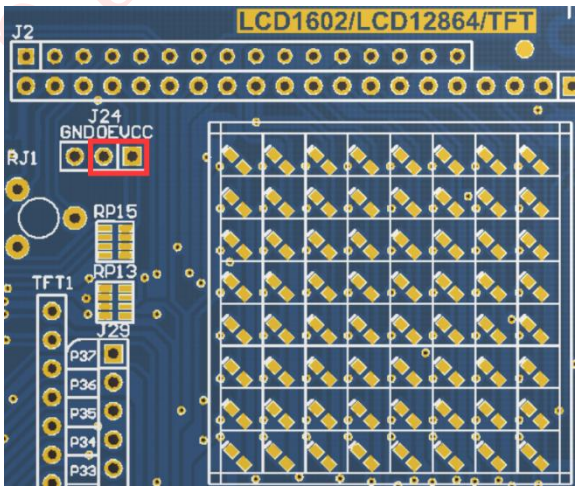


16.4.3 LED 点阵（显示图像）

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：8*8LED 点阵显示心形。



注意：当不使用 LED 点阵时，将 LED 点阵旁的 J24 黄色跳线帽短接到 VCC 一端。这是因为 LED 点阵使用到了 P0 口，而数码管也使用到 P0 口，为了不干扰后续数码管显示的一些实验，所以短接到 VCC 一端，如下所示：



课后作业

第 17 章 直流电机实验

在 51 单片机应用中，电机控制的应用也非常多。所以有必要先来了解下电机相关的知识以及如何使用 51 单片机控制电机，在未学习 PWM 之前，我们先简单使用 GPIO 来控制电机的旋转和停止。不能直接使用 GPIO 来驱动电机，而需要相应的驱动芯片来驱动电机，开发板板载了一个 ULN2003 驱动芯片，该芯片是一个单片高电压、高电流的达林顿晶体管阵列集成电路。不仅可以用来驱动我们的直流电机，还可用来驱动五线四相步进电机，比如 28BYJ-48 步进电机。本章使用的是 ULN2003 芯片来驱动，本章所要实现的功能是：直流电机工作约 5S 后停止。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

17.1 直流电机介绍

17.2 ULN2003 芯片介绍

17.3 硬件设计

17.4 软件设计

17.5 实验现象

普中51单片机开发板

17.1 直流电机介绍

直流电机是指能将直流电能转换成机械能（直流电动机）或将机械能转换成直流电能（直流发电机）的旋转电机。它是能实现直流电能和机械能互相转换的电机。当它作电动机运行时是直流电动机，将电能转换为机械能；作发电机运行时是直流发电机，将机械能转换为电能。

直流电机的结构应由定子和转子两大部分组成。直流电机运行时静止不动的部分称为定子，定子的主要作用是产生磁场，由机座、主磁极、 换向极、 端盖、轴承和电刷装置等组成。运行时转动的部分称为转子，其主要作用是产生电磁转矩和感应电动势，是直流电机进行能量转换的枢纽，所以通常又称为电枢，由转轴、电枢铁心、电枢绕组、换向器和风扇等组成。

直流电机没有正负之分，在两端加上直流电就能工作。需要知道直流电机的额定电压和额定功率，不能使之长时间超负荷运作。 在交换接线后，可以形成正反转。

开发板配置的直流电机为 5V 直流电机，其主要参数如下：

轴长：8mm

轴径：2mm

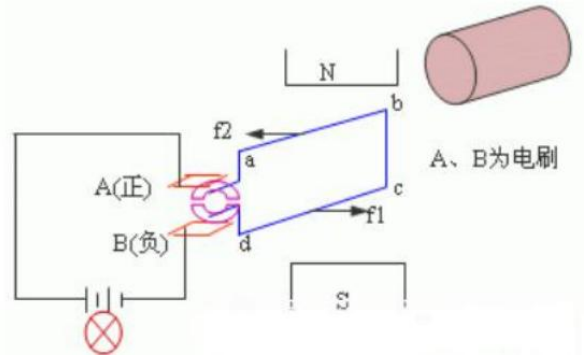
电压：1-6V

参考电流：0.35-0.4A

3V 转速：17000-18000 转每分钟

外观实物图如下：

其内部结构如下图所示：



17.2 ULN2003 芯片介绍

前面讲过，51 单片机主要是用来控制而非驱动，如果直接使用芯片的 GPIO 管脚去驱动大功率器件，要么将芯片烧坏，要么就驱动不起来。所以要驱动大功率器件，比如电机。就必须搭建驱动电路，开发板上板载的驱动芯片是 ULN2003，该芯片是一个单片高电压、高电流的达林顿晶体管阵列集成电路。不仅可以用来驱动直流电机，还可用来驱动五线四相步进电机，比如 28BYJ-48 步进电机。本章我们使用 ULN2003 来驱动直流电机，下面来具体介绍下 ULN2003 芯片的使用。

ULN2003 是一个单片高电压、高电流的达林顿晶体管阵列集成电路。它是由 7 对 NPN 达林顿管组成的，它的高电压输出特性和阴极箝位二极管可以转换感应负载。单个达林顿对的集电极电流是 500mA。达林顿管并联可以承受更大的电流。此电路主要应用于继电器驱动器，字锤驱动器，灯驱动器，显示驱动器（LED 气体放电），线路驱动器和逻辑缓冲器。ULN2003 的每对达林顿管都有一个 2.7k 串联电阻，可以直接和 TTL 或 5V CMOS 装置。

（1）主要特点

①500mA 额定集电极电流（单个输出）

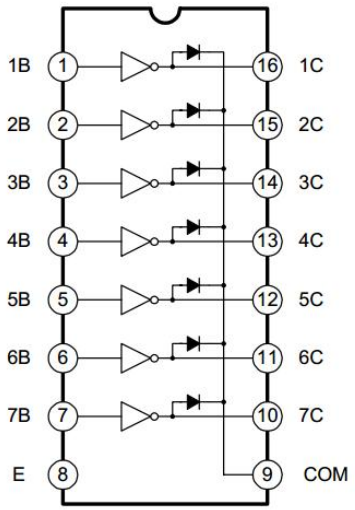
②高电压输出：50V

③输入和各种逻辑类型兼容

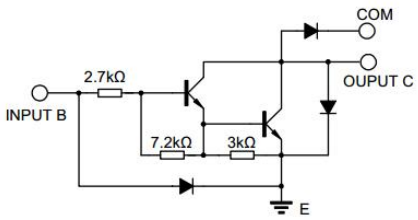
④继电器驱动器

（2）逻辑框图

逻辑框图



示意图(每对达林顿管)



从上图可以很容易理解该芯片的使用方法，其内部相当于非门电路，即输入高输出为低，输入为低输出是高，这里要注意：**因为 ULN2003 的输出是集电极开路，ULN2003 要输出高电平，必须在输出口外接上拉电阻。这也就能解释在后面连接直流电机时为什么不能直接将 ULN2003 的 2 个输出口接电机线，而必须一根线接电源，另一个才接 ULN2003 输出口。**

若使用该芯片驱动直流电机，只可实现单方向控制，电机一端接电源正极，另一端接芯片的输出口。若想控制五线四相步进电机，则可将四路输出接到步进电机的四相上，电机另一条线接电源正。芯片的详细介绍可参考资料资料“\6--开发板芯片资料\ULN2003 中文手册.pdf”。

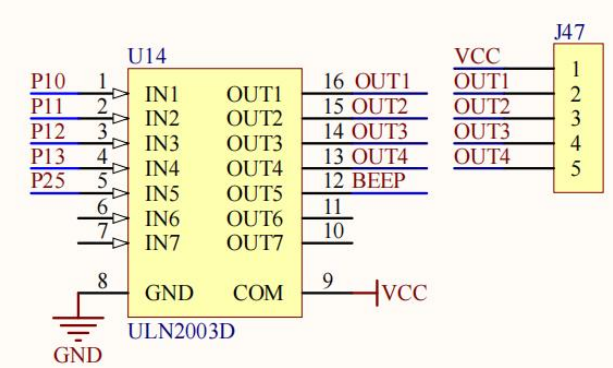
17.3 硬件设计

本实验使用到硬件资源如下：

（1）步进电机驱动模块

（2）直流电机

开发板上的步进电机驱动模块电路如下图所示：



从上图可知，ULN2003 的输入口与单片机的 P1.0-P1.3 连接，对应输出则是 OUT1-OUT4，而 J47 则是提供给外部连接电机的接口，可以支持直流电机、五线四相步进电机 28BYJ-48 连接。本实验使用的是直流电机，电机的一根线连接在 VCC 上，另一根连接在 OUT1 上，因此可通过单片机 P1.0 口输出高电平来控制电机旋转，输出低电源控制电机停止。注意：单片机 P1.0 输出低电平时，ULN2003 的 OUT1 并不会输出高电平导致停止，而是因为集电极开路，导致电机无电流流入致使停止。

17.4 软件设计

本章所要实现的功能是：直流电机工作约 5S 后停止。

我们打开“\4--实验程序\1--基础实验\14-直流电机实验”工程，控制代码全部都在 main.c 中，代码如下：

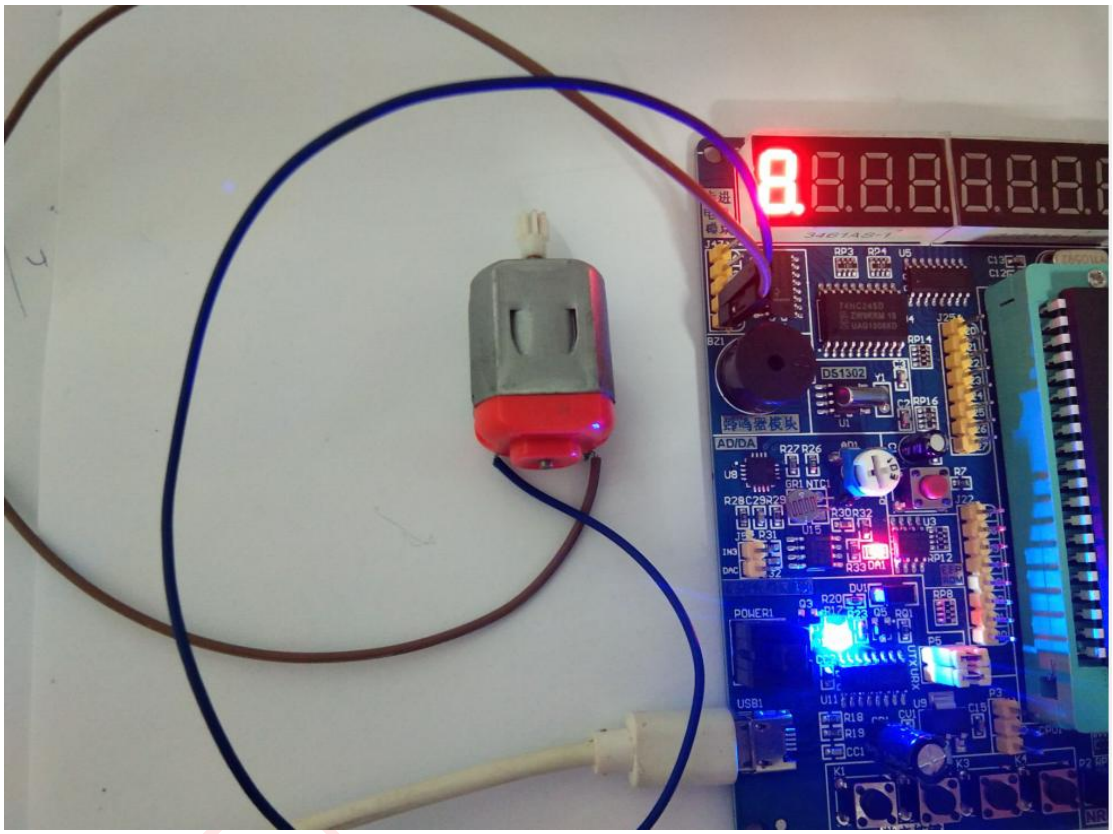
```
/************************************************************************
******************
实验名称：直流电机实验
接线说明：
实验现象：下载程序后，直流电机旋转 5S 后停止
注意事项：将直流电机两根线分别连接到“步进电机模块”输出端子 J47 的 5V 和 O1 上。

************************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义直流电机控制管脚
sbit DC_Motor=P1^0;
```

```
#define DC_MOTOR_RUN_TIME    5000    //定义直流电机运行时间为 5000ms


/*************************************************************************
***********
* 函 数 名       : delay_ms
* 函数功能       : ms 延时函数，ms=1 时，大约延时 1ms
* 输    入       : ten_us
* 输    出       : 无
*************************************************************************
**********/
void delay_ms(u16 ms)
{
    u16 i,j;
    for(i=ms;i>0;i--)
        for(j=110;j>0;j--);
}

/*************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*************************************************************************
**********/
void main()
{
    DC_Motor=1;//开启电机
    delay_ms(DC_MOTOR_RUN_TIME);
    DC_Motor=0;//关闭电机
    while(1)
    {

    }
}
```

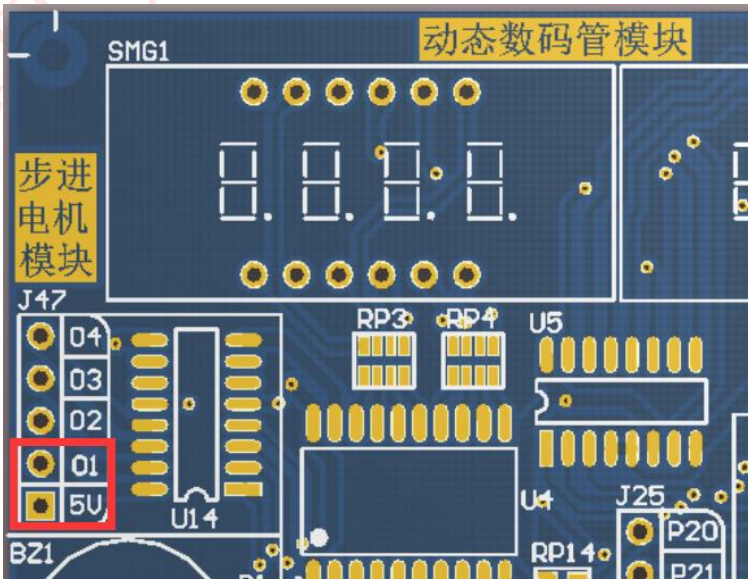
实验代码比较简单，主要就是理解 ULN2003 的电平输出关系，然后就像控制 LED 一样。这里就不再详细分析了。

17.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：直流电机工作约 5S 后停止。



注意：直流电机的两根线要连接在 J47 端子的 01 和 5V 上。如下所示：



课后作业

第 18 章 步进电机实验

前面章节，我们介绍了直流电机的控制，本章将向大家介绍步进电机，步进电机是将电脉冲信号转变为角位移或线位移的开环控制元件。本章主要从步进电机的结构、工作原理、电机参数分别介绍，最后通过实验来实现步进电机运动的简单控制。本章所要实现的功能是：通过 ULN2003 驱动模块控制 28BYJ48 步进电机运行方向及速度，当按下 KEY1 键可调节电机旋转方向；当按下 KEY2 键，电机加速；当按下 KEY3 键，电机减速。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

18.1 步进电机简介

18.2 步进电机工作原理

18.3 步进电机技术指标

18.4 28BYJ-48 步进电机简介

18.5 硬件设计

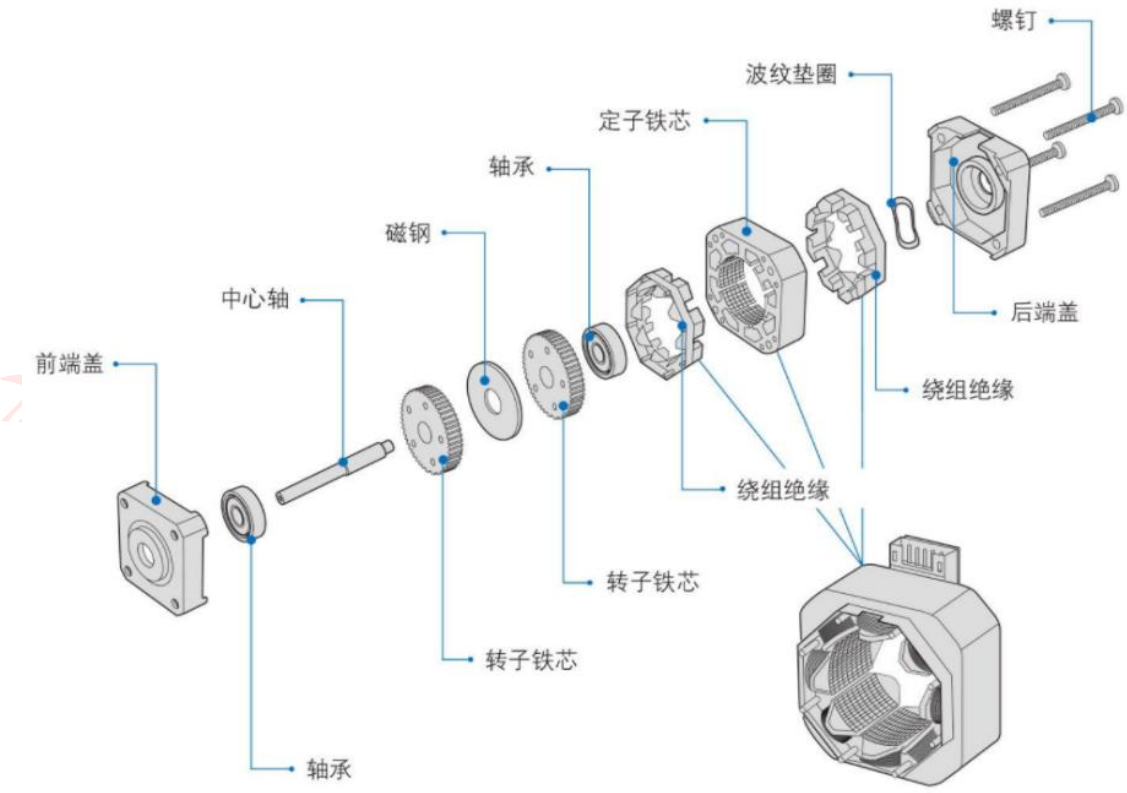
18.6 软件设计

18.7 实验现象

普中51单片机开发板

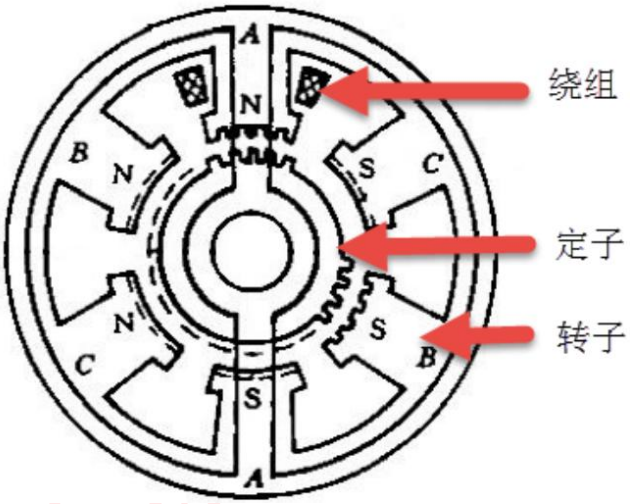
18.1 步进电机简介

步进电机是将电脉冲信号转变为角位移或线位移的开环控制元件。在非超载的情况下，电机的转速、停止的位置只取决于脉冲信号的频率和脉冲数，而不受负载变化的影响，即给电机加一个脉冲信号，电机则转过一个步距角。这一线性关系的存在，加上步进电机只有周期性的误差而无累计误差等特点。使得在速度、位置等控制领域用步进电机来控制变的非常的简单。虽然步进电机已被广泛的应用，但步进电机并不能像普通的直流电机，交流电机在常规下使用。它必须由双环形脉冲信号、功率驱动电路等组成控制系统方可使用。因此用好步进电机也并非易事，它涉及到机械、电机、电子及计算机等多专业知识。下图即为混合式步进电机组成图。



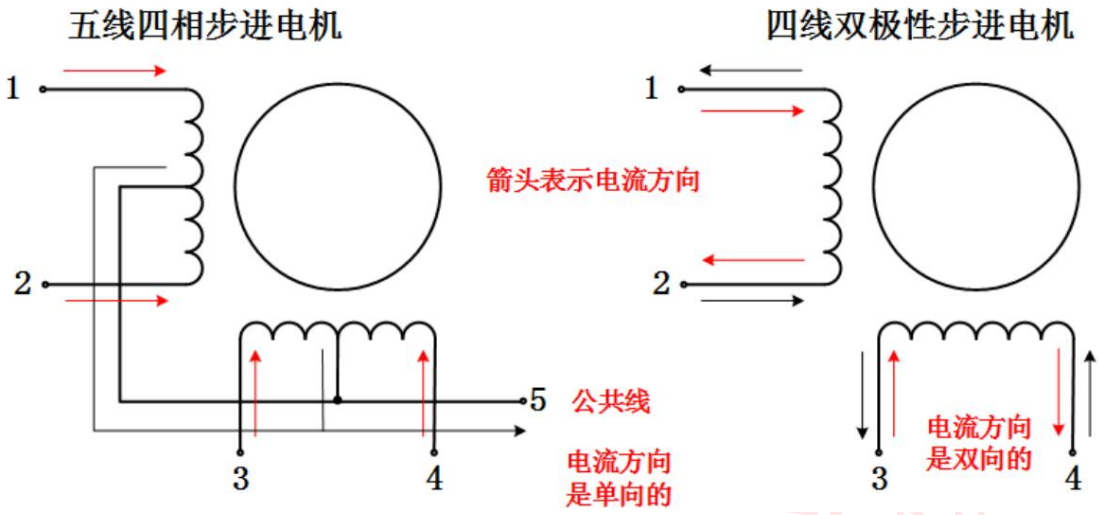
18.2 步进电机工作原理

通常步进电机的转子为永磁体，当电流流过定子绕组时，定子绕组产生一矢量磁场。磁场会带动转子旋转一定的角度，使得转子的一对磁场方向与定子的磁场方向一致。当定子的矢量磁场旋转一个角度。转子也随着该磁场转步距角。每输入一个电脉冲，电动机转动一个角度前进一步。它输出的角位移与输入的脉冲数成正比、转速与脉冲频率成正比。改变绕组通电的顺序，电机就会反转。所以可以控制脉冲数量、频率及电动机各相绕组的通电顺序来控制步进电机的转动。具体看下图：



18.2.1 步进电机极性区分

步进电机又分为单极性的步进电机和双极性的步进电机；具体简易图如下图所示：



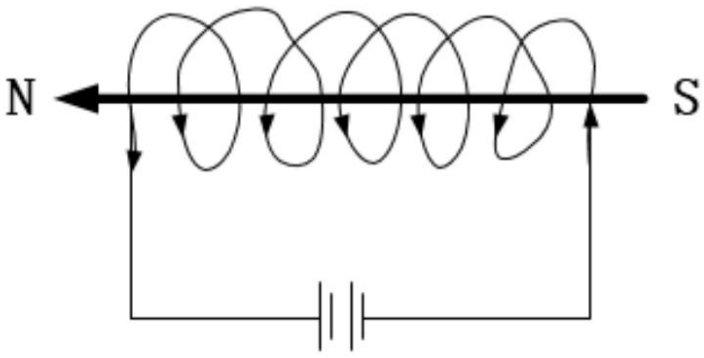
其中左侧为单极性步进电机，右侧为双极性的步进电机，从上图中不难看出区别是什么。单双极性是指一个步进电机里面有几种电流的流向，左侧的五线四相步进电机就是单极性的步进电机，图中的红色箭头为电流的走向，四根线的电流走向汇总到公共线，所以称之为单极性电机；但是右侧则不同，电机中有两个电流的回路，两个电流的回路自然就是双极性，所以称之为双极性电机。

单极性绕组

单极性步进电机使用的是单极性绕组。其一个电极上有两个绕组，这种联接方式为当一个绕组通电时，产生一个北极磁场；另一个绕组通电，则产生一个南极磁场。因为从驱动器到线圈的电流不会反向，所以可称其为单极绕组。

双极性绕组

双极性步进电机使用的是双极性绕组。每相用一个绕组，通过将绕组中电流反向，电磁极性被反向。典型的两相双极驱动的输出步骤在电气原理图和下图中的步进顺序中进一步阐述。按图所示，转换只利用绕组简单地改变电流的方向，就能改变该组的极性。

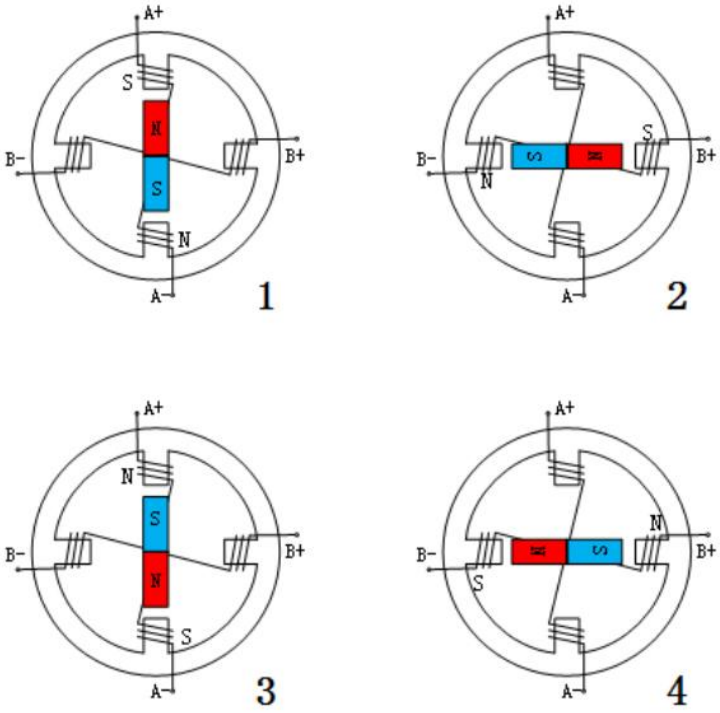


永磁步进电机包括一个永磁转子、线圈绕组和导磁定子，只要将线圈通电根

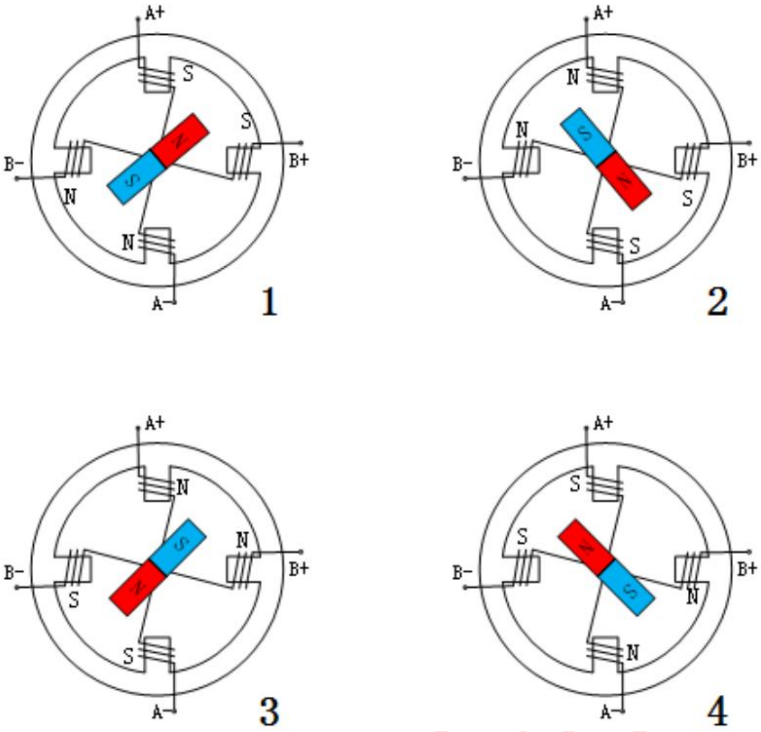
据电磁铁的原理就会产生磁场，分为南北极，见上图所示；通过改变步进电机定子的磁场，转子就会因磁场的变化而发生转动，同理，依次改变通电的顺序就可以使得电机转动起来。

18.2.2 双极性步进电机驱动原理

下图是一个双极性的步进电机整步，步进顺序。在第一步中：将 A 相通电，根据电磁铁原理，产生磁性，并且因异性相吸，所以磁场将转子固定在第一步的位置；第二步：当 A 相关闭，B 相通电时，转子会旋转 90°；第三步：B 相关闭、A 相通电，但极性与第 1 步相反，这促使转子再次旋转 90°。在第四步中：A 相关闭、B 相通电，极性与第 2 步相反。重复该顺序促使转子按 90° 的步距角顺时针旋转。

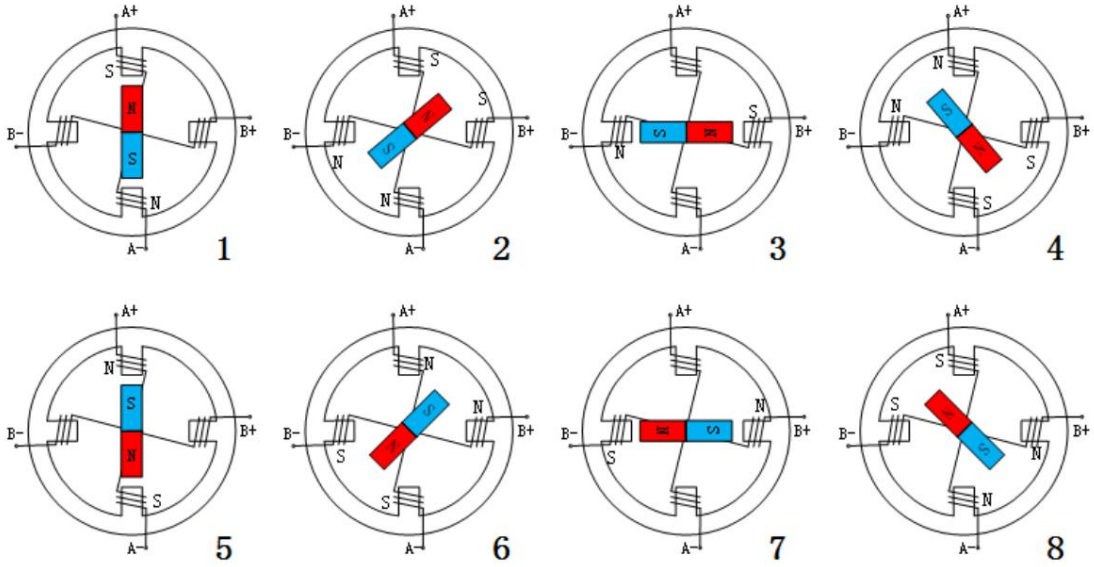


上图中显示的步进顺序是单相激励步进，也可以理解为每次通电产生磁性的相只有一个，要么 A 相，要么 B 相；但是更常用的是双相激励，但是在转换时，一次只能换相一次，具体详见下图：



上图是两相同时通电的旋转顺序，与单相激励不同的是，单相通电后被固定在了与定子正对着的绕组极性，但是双相同时激励时转子却被固定在两个绕阻的极性中间；此时通电顺序就变成了 AB 相同时通电即可。

在双相激励的过程中，也可以在换相位时加一个关闭相位的状态而产生走半步的现象，这将步进电机的整个步距角一分为二，例如，一个 90° 的步进电机将每半步移动 45° ，具体见下图。



1. A 相通电，B 相不通电
2. A、B 相全部通电，且电流相同，产生相同磁性

3. B 相通电，A 断电

4. B 相通电，A 相通电，且电流相等，产生相同磁性

5. A 相通电，B 断电

6. A、B 相全部通电，且电流相同，产生相同磁性

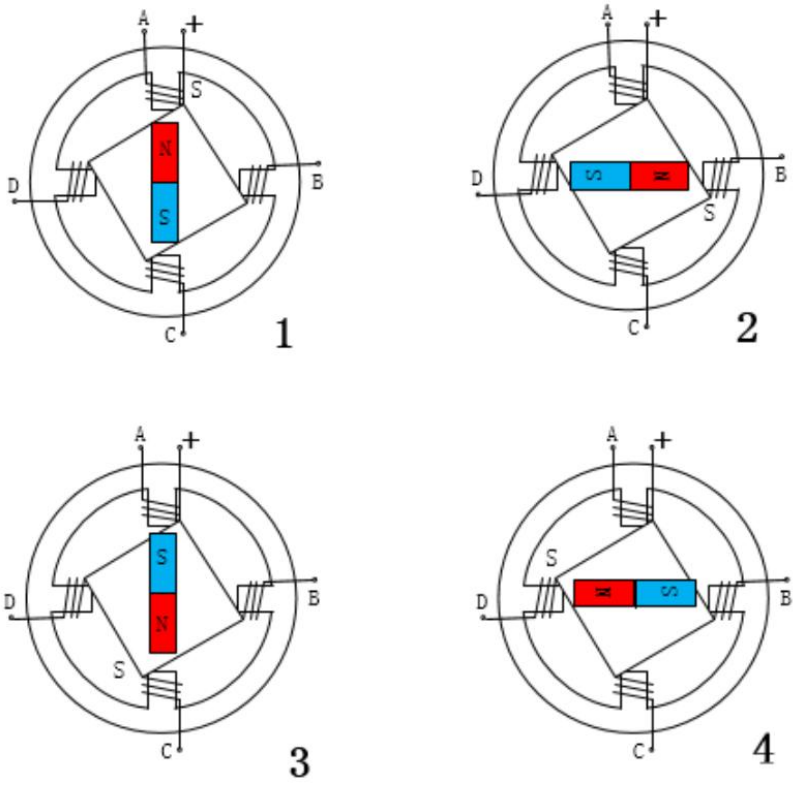
7. B 相通电，A 断电

8. B 相通电，A 相通电，且电流相等，产生相同磁性

其中 1~4 步与 5~8 步的电流方向相反（电流相反，电磁的极性就相反）这样就产生了顺时针旋转，同理逆时针是将通电顺序反过来即可。

18.2.3 单极性步进电机驱动原理

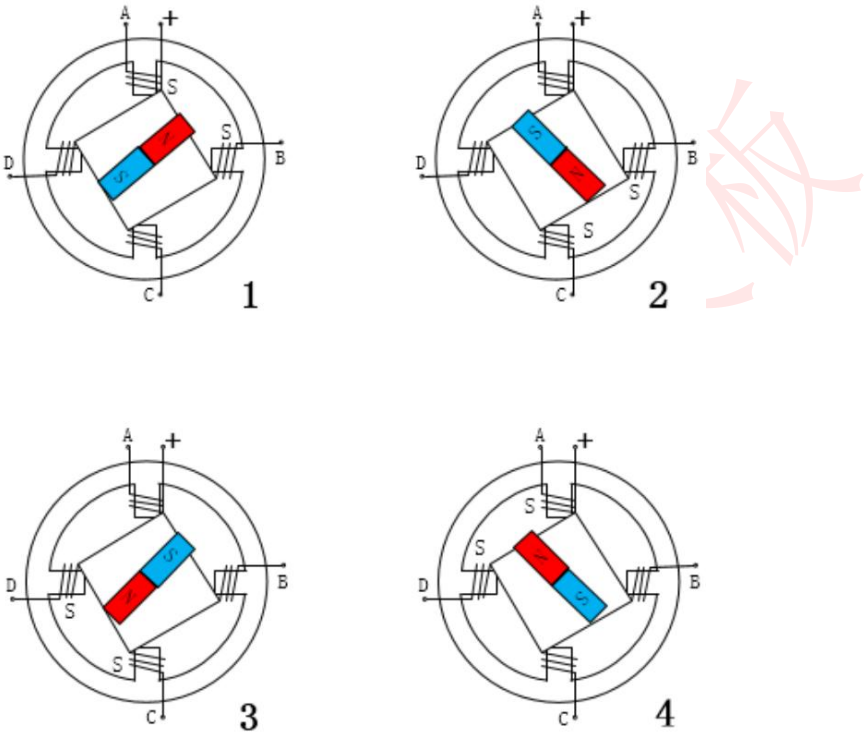
单极性与双极性步进电机驱动类似，都可以分为整步与半步的驱动方式，不同的是，双极性的步进电机可以通过改变电流的方向来改变每相的磁场方向，但是单极性的就不可以了，它有一个公共端，这就直接决定了，电流方向。具体旋转顺序详见下图：



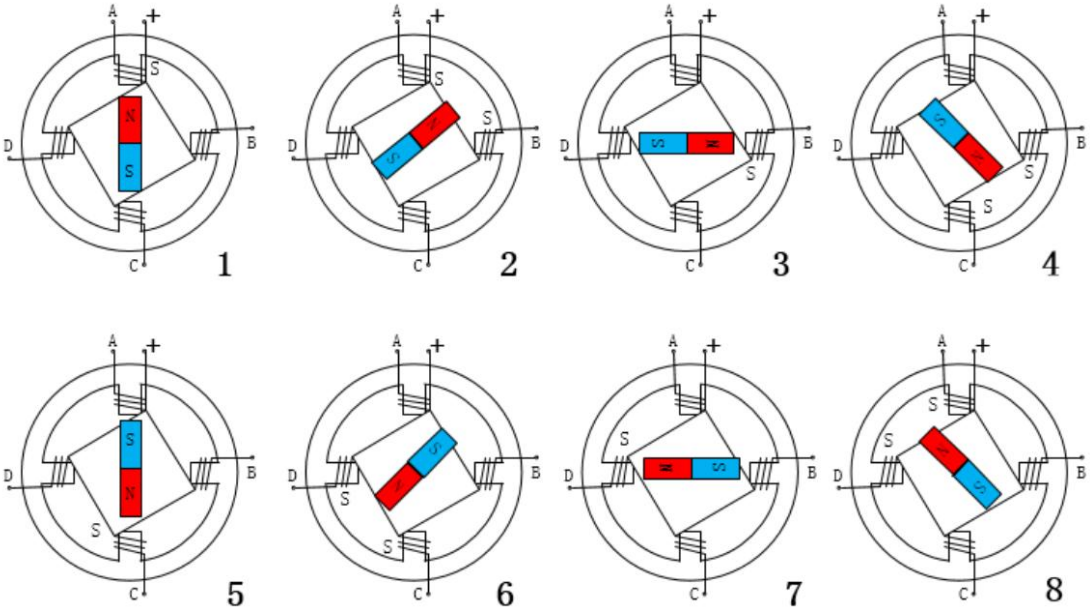
上图是单极性步进电机整步旋转的过程，其中，在图示中分为 5 根线，分别为 A、B、C、D 和公共端（+），公共端需要一直通电，剩下 ABCD 相中只要

有一个相通电，即可形成回路产生磁场，图中的通电顺序为 A->B->C->D，即可完成上图中的顺时针旋转，如果想要逆时针旋转只需要将其倒序即可。

以上是单相通电产生的整步旋转，两相通电也可以产生，两个相邻的相通电，这样相邻的两个相就都产生了回路，也就产生了磁场，图中的通电顺序为 AB->BC->CD->DA，同理逆时针旋转的顺序为逆序。具体看下图：



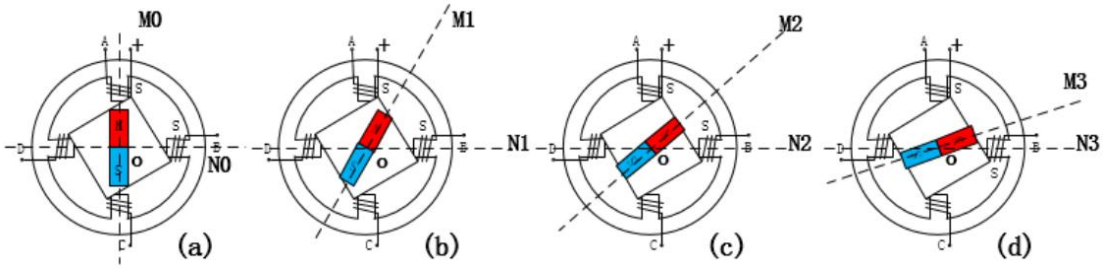
上面两张图清晰的描述了单极性步进电机的通电顺序与旋转的过程，综合这两张图就是单极性步进电机半步的通电顺序，具体看下图：



上图兼容了前两张图的所有特点，也可以说前两张图是这张图的子集，图中的通电顺序为：A->AB->B->BC->C->CD->D->DA 转子每次只走半步 45 度，所以这也被称为半步驱动，与整步相比半步的旋转方式旋转起来更加的顺滑。

18.2.4 细分驱动原理

对于细分驱动的原理，不分单双极性步进电机，下图以单极性为例：



在上图中均为双相激励；其中图（a）为 A 相电流很大，B 相的电流极其微弱，接近 0；图 (C) 为 A 相和 B 相的电流相同，电流决定磁场，所以说 A 相和 B 相的磁场也是相同的，(a) 和（c）可以是极限特殊的情况，再看图（b）和图（d）这两个是由于 A 相和 B 相的电流不同产生位置情况；由此可以得出改变定子的电流比例就可以使得转子在任意角度停住。细分的原理就是：通过改变定子的电流比例，改变转子在一个整步中的不同位置，可以将一个整步分成多个小步来运行。

在上图中就是一个整步分成了 4 步来跑，从（a）~（d）是 A 相的电流逐渐减小，B 相电流逐渐增大的过程，如果驱动器的细分能力很强，可以将其分成 32 细分、64 细分等；这不仅提高了步进电机旋转的顺畅度而且提高了每步的精度。细分驱动具有转动顺畅、精度高、转矩大的特点，但控制复杂，一般需要专用芯片来实现，例如东芝公司的 TB67S10xA 系列步进电机细分驱动芯片，最多可以把 1 个整步分为 32 个小步。

18.3 步进电机技术指标

18.3.1 静态技术指标

• 相数：产生不同对极 N、S 磁场的激磁线圈对数，也可以理解为步进电机中线圈的组数，其中两相步进电机步距角为 1.8°，三相的步进电机步距角为 1.5°，相数越多的步进电机，其步距角就越小。

• 拍数：完成一个磁场周期性变化所需脉冲数或导电状态用 n 表示，或指电机转过一个齿距角所需脉冲数，以四相电机为例，有四相四拍运行方式即 AB-BC-CD-DA-AB，四相八拍运行方式即 A-AB-B-BC-C-CD-D-DA-A。

• 步距角：一个脉冲信号所对应的电机转动的角度，可以简单理解为一个脉冲信号驱动的角度，电机上都有写，一般 42 步进电机的步距角为 1.8°

• 定位转矩：电机在不通电状态下，电机转子自身的锁定力矩（由磁场齿形的谐波以及机械误差造成的）。

• 静转矩：电机在额定静态电压作用下，电机不作旋转运动时，电机转轴的锁定力矩。此力矩是衡量电机体积的标准，与驱动电压及驱动电源等无关。

18.3.2 动态技术指标

• 步距角精度：步进电机转动一个步距角度的理论值与实际值的误差。用百分比表示：误差/步距角 *100%。

• 失步：电机运转时运转的步数，不等于理论上的步数。也可以叫做丢步，一般都是因负载太大或者是频率过快。

• 失调角：转子齿轴线偏移定子齿轴线的角度，电机运转必存在失调角，由失调角产生的误差，采用细分驱动是不能解决的。

• 最大空载起动频率：在不加负载的情况下，能够直接起动的最大频率。

• 最大空载的运行频率：电机不带负载的最高转速频率。

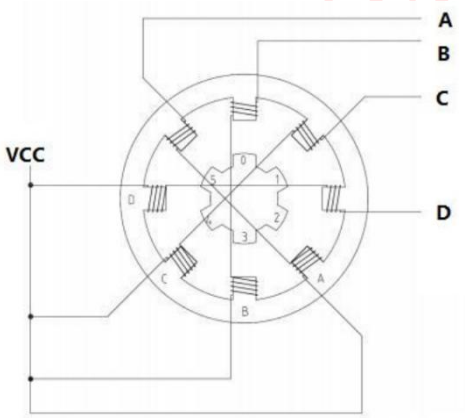
• 运行转矩特性：电机的动态力矩取决于电机运行时的平均电流（而非静态电流），平均电流越大，电机输出力矩越大，即电机的频率特性越硬。

• 电机正反转控制：通过改变通电顺序而改变电机的正反转。

18.4 28BYJ-48 步进电机简介

28BYJ48 步进电机自带减速器，为四相无线步进电机，直径为 28mm，实物如下所示：

28BYJ48 电机内部结构等效图如下所示：



28BYJ48 步进电机旋转驱动方式如下表：

导线颜色	第1步	第2步	第3步	第4步	第5步	第6步	第7步	第8步
VCC 红	5V	5V	5V	5V	5V	5V	5V	5V
D 橙	GND	GND						GND
C 黄		GND	GND	GND				
B 粉				GND	GND	GND		
A 蓝						GND	GND	GND

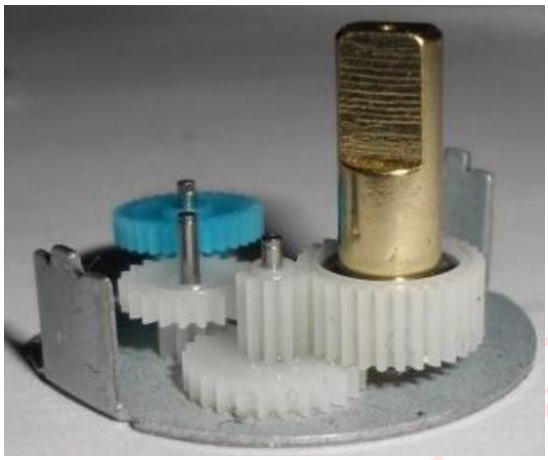
28BYJ48 步进电机主要参数如下所示：

电机型号	电压 V	相数	相电阻 ±10%	步进角度	减速比	起动转矩 100PPS g.cm	起动频率 PPS	定位转矩 g.cm	绝缘耐压 VAC/s
28BYJ48	5	4	300殴	5.625/64	1:64	≥300	≥500	≥300	600

在上图中 28BYJ48 步进电机主要参数中可以看到有一个减速比：1:64，步进角为 5.625/64 度，如果需要转动一圈，那么需要 360/5.625*64=4096 个脉冲信

号。

减速比这个和之前介绍的直流减速电机有点类似，所以 28BYJ48 步进电机实际上是：减速齿轮+步进电机组成，28BYJ48 步进电机减速齿轮实物图如下所示：



减速齿轮计算方法如下所示：

$$\frac{9}{32}\times\frac{11}{22}\times\frac{9}{26}\times\frac{10}{31}=\frac{8910}{567424}\approx\frac{1}{64}$$

18.5 硬件设计

本实验使用到硬件资源如下：

（1）步进电机驱动模块

（2）28BYJ-48 步进电机

该电路在前面直流电机实验中已介绍，此处不再重复。

18.6 软件设计

本章所要实现的功能是：通过 ULN2003 驱动模块控制 28BYJ48 步进电机运行方向及速度，当按下 KEY1 键可调节电机旋转方向；当按下 KEY2 键，电机加速；

当按下 KEY3 键，电机减速。

我们打开“\4--实验程序\1--基础实验\15-步进电机实验”工程，控制代码全部都在 main.c 中，代码如下：

```
/*********************************************************************
******************
实验名称：步进电机实验
接线说明：
实验现象：下载程序后，当按下 KEY1 键可调节电机旋转方向；当按下 KEY2 键，电机加速；
          当按下 KEY3 键，电机减速
注意事项：将步进电机红色线对接到“步进电机模块”输出端子 J47 的 5V 上，其它相序依次接
入。

*********************************************************************
******************/
#include "reg52.h"

typedef unsigned int u16;    //对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义 ULN2003 控制步进电机管脚
sbit IN1_A=P1^0;
sbit IN2_B=P1^1;
sbit IN3_C=P1^2;
sbit IN4_D=P1^3;

//定义独立按键控制脚
sbit KEY1=P3^1;
sbit KEY2=P3^0;
sbit KEY3=P3^2;
sbit KEY4=P3^3;

//使用宏定义独立按键按下的键值
#define KEY1_PRESS    1
#define KEY2_PRESS    2
#define KEY3_PRESS    3
#define KEY4_PRESS    4
#define KEY_UNPRESS   0

// 定义步进电机速度，值越小，速度越快
// 最小不能小于 1
#define STEPMOTOR_MAXSPEED       1
#define STEPMOTOR_MINSPEED       5
```

```
/*****************************************************************
***********
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*****************************************************************
**********/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*****************************************************************
***********
* 函 数 名       : delay_ms
* 函数功能       : ms 延时函数，ms=1 时，大约延时 1ms
* 输    入       : ten_us
* 输    出       : 无
*****************************************************************
**********/
void delay_ms(u16 ms)
{
    u16 i,j;
    for(i=ms;i>0;i--)
        for(j=110;j>0;j--);
}

/*****************************************************************
***********
* 函 数 名       : step_motor_28BYJ48_send_pulse
* 函数功能       : 输出一个数据给 ULN2003 从而实现向步进电机发送一个脉冲
* 输    入       : step：指定步进序号，可选值 0~7
                    dir：方向选择,1：顺时针,0：逆时针
* 输    出       : 无
*****************************************************************
**********/
void step_motor_28BYJ48_send_pulse(u8 step,u8 dir)
{
    u8 temp=step;

    if(dir==0) //如果为逆时针旋转
```

```
        temp=7-step;//调换节拍信号
    switch(temp)//8 个节拍控制：A->AB->B->BC->C->CD->D->DA
    {
        case 0: IN1_A=1;IN2_B=0;IN3_C=0;IN4_D=0;break;
        case 1: IN1_A=1;IN2_B=1;IN3_C=0;IN4_D=0;break;
        case 2: IN1_A=0;IN2_B=1;IN3_C=0;IN4_D=0;break;
        case 3: IN1_A=0;IN2_B=1;IN3_C=1;IN4_D=0;break;
        case 4: IN1_A=0;IN2_B=0;IN3_C=1;IN4_D=0;break;
        case 5: IN1_A=0;IN2_B=0;IN3_C=1;IN4_D=1;break;
        case 6: IN1_A=0;IN2_B=0;IN3_C=0;IN4_D=1;break;
        case 7: IN1_A=1;IN2_B=0;IN3_C=0;IN4_D=1;break;
    }
}

/**********************************************************************
***********
* 函 数 名         : key_scan
* 函数功能         : 检测独立按键是否按下，按下则返回对应键值
* 输    入         : mode=0：单次扫描按键
                     mode=1：连续扫描按键
* 输    出         : KEY1_PRESS：K1 按下
                     KEY2_PRESS：K2 按下
                     KEY3_PRESS：K3 按下
                     KEY4_PRESS：K4 按下
                     KEY_UNPRESS：未有按键按下
**********************************************************************
**********/
u8 key_scan(u8 mode)
{
    static u8 key=1;

    if(mode)key=1;//连续扫描按键
    if(key==1&&(KEY1==0||KEY2==0||KEY3==0||KEY4==0))//任意按键按下
    {
        delay_10us(1000);//消抖
        key=0;
        if(KEY1==0)
            return KEY1_PRESS;
        else if(KEY2==0)
            return KEY2_PRESS;
        else if(KEY3==0)
            return KEY3_PRESS;
        else if(KEY4==0)
            return KEY4_PRESS;
```

```
    }
    else if(KEY1==1&&KEY2==1&&KEY3==1&&KEY4==1)    //无按键按下
    {
        key=1;
    }
    return KEY_UNPRESS;
}

/*********************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*********************************************************************
**********/
void main()
{
    u8 key=0;
    u8 dir=0;//默认逆时针方向
    u8 speed=STEPMOTOR_MAXSPEED;//默认最大速度旋转
    u8 step=0;

    while(1)
    {
        key=key_scan(0);
        if(key==KEY1_PRESS)//换向
        {
            dir=!dir;
        }
        else if(key==KEY2_PRESS)//加速
        {
            if(speed>STEPMOTOR_MAXSPEED)
                speed-=1;
        }
        else if(key==KEY3_PRESS)//减速
        {
            if(speed<STEPMOTOR_MINSPEED)
                speed+=1;
        }
        step_motor_28BYJ48_send_pulse(step++,dir);
        if(step==8)step=0;
        delay_ms(speed);
    }
```

```
}
```

step_motor_28BYJ48_send_pulse 函数用于输出一个数据给 ULN2003，从而实现向步进电机发送一个脉冲信号。它有两个形参，第一个为 step，指定步进序号，可选值为 0-7，代表步进电机控制信号的 8 个节拍。第二个为 dir，指定电机的旋转方向，可选 1:顺时针，0:逆时针。这里指的顺时针逆时针是两个不同方向。

函数先判断 dir 参数值，如果为 0，则为逆时针旋转，将 step 值变为 8 的互补数，其实就是将 8 个节拍反向输出。

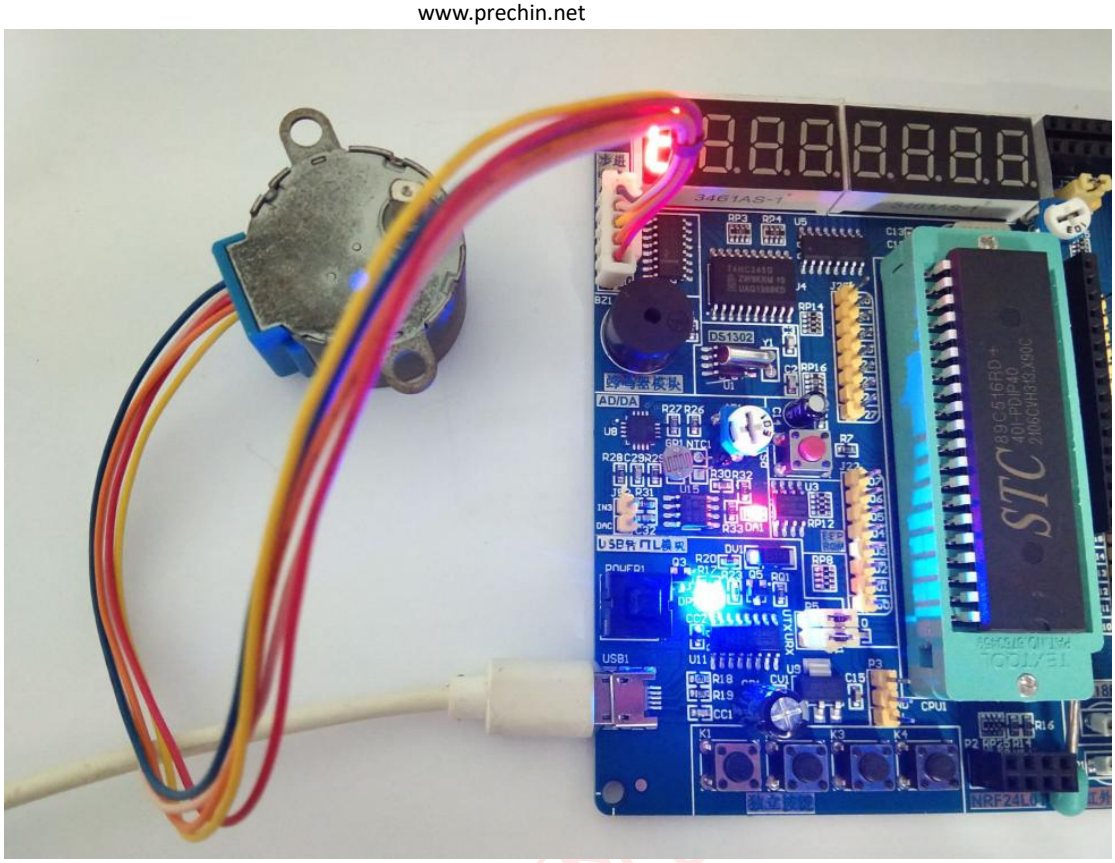
主函数实现的功能很简单，首先定义一些变量，进入循环，检测是否有按键按下，当 KEY1 按下，切换步进电机方向；当 KEY2 按下，使电机加速；当 KEY3 按下，使电机减速。

程序中所用 STEPMOTOR_MAXSPEED 和 STEPMOTOR_MINSPEED 在开头已定义，表示步进电机最大运行速度和最低运行速度，当然可以根据实际现象适当调整这两个值。

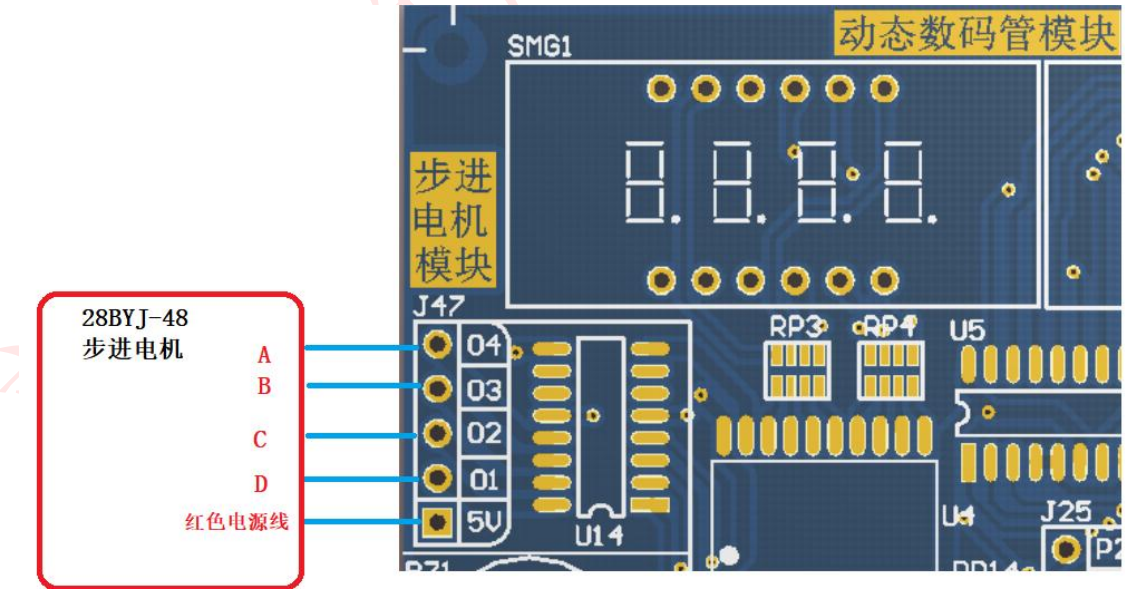
18.7 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：当按下 KEY1 键可调节电机旋转方向；当按下 KEY2 键，电机加速；当按下 KEY3 键，电机减速。

普中51单片机开发板



注意：将步进电机红色线对接到“步进电机模块”输出端子 J47 的 5V 上，其它相序依次接入。如下所示：



课后作业

第 19 章 中断系统

在前面章节，我们通过介绍 51 单片机 IO 口学习了很多相关的实验。从最简单的 LED 到数码管动态扫描再到 LED 点阵以及电机控制，无不体现学习 51 单片机 IO 口操作的重要性。在前面章节，我们还提到了学习 51 单片机的重点及难点主要有中断、定时器、串口等内容，这部分内容大家一定要认真掌握，这部分没有学好就不能说学会了 51 单片机。因此专门用一章来介绍 51 单片机的中断系统，为后面学习外部中断、定时器中断、串口中断做好铺垫。本章分为如下几部分内容：

19.1 中断介绍

19.2 中断结构及相关寄存器

普中51单片机开发板

普中51单片机开发板

19.1 中断介绍

19.1.1 中断概念

中断是为使单片机具有对外部或内部随机发生的事件实时处理而设置的，中断功能的存在，很大程度上提高了单片机处理外部或内部事件的能力。它也是单片机最重要的功能之一，是我们学习单片机必须要掌握的。很多初学者被困在中断中，学了很久仍然不知道中断究竟是如何运作的。千万不要认为它有多难，其实只要掌握正确的学习方法，没有哪个知识点是学不会的。

为了能让大家更容易理解中断概念， 我们先来举一个生活事例： 你打开火，烧上一壶水。然后去洗衣服，在洗衣服的过程中，突然听到水壶发出水开的报警声，这时，你停止洗衣服动作，立即去关掉火，然后将开水灌入暖水瓶中，灌完开水后，你又回去继续洗衣服。这个过程中实际上就发生了一次中断。

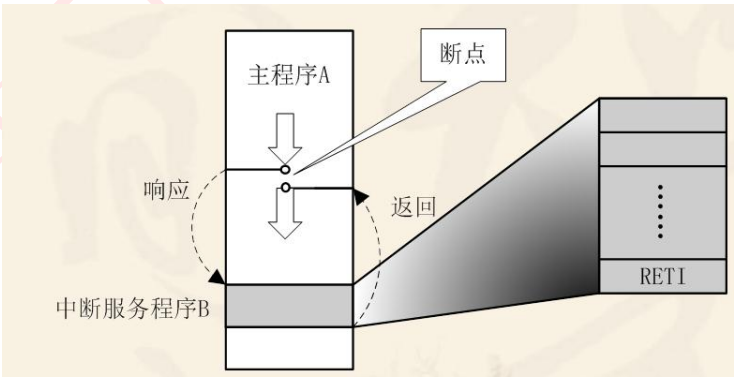
对于单片机来讲，中断是指 CPU 在处理某一事件 A 时，发生了另一事件 B，

请求 CPU 迅速去处理(中断发生)；CPU 暂时停止当前的工作(中断响应)， 转去处理事件 B(中断服务)；待 CPU 将事件 B 处理完毕后，再回到原来事件 A 被中断的地方继续处理事件 A(中断返回)，这一过程称为中断。

生活中的实例　　单片机处理中断的过程

再回来看前面讲的生活事例，与单片机中断结合分析，你的主任务是洗衣服，水开报警这是一个中断请求，这一时刻相当于断点处，你响应中断去关火，然后将开水灌入暖水瓶中，这一动作实际上就是处理中断程序，灌完开水后再回去继续洗衣服，相当于处理完中断程序后再返回主程序继续执行主程序。这里需要注意的是，水开是随时都有可能的，但是无论什么时候开，只要一开你将立即去处理它，处理完后再回来继续接着洗刚才那件衣服。单片机在执行程序时，中断也随时有可能发生，但无论何时发生，只要一旦发生，单片机将立即暂停当前程序，赶去处理中断程序， 处理完中断程序后再返回刚才暂停处接着执行原来的程序。

单片机在执行程序时其程序流程图如下：



引起 CPU 中断的根源称为中断源。中断源向 CPU 提出中断请求，CPU 暂时中断原来的事务 A，转去处理事件 B，对事件 B 处理完毕后，再回到原来被中断的地方(即断点)，称为中断返回。实现上述中断功能的部件称为中断系统(中断机构)。

当中央处理机 CPU 正在处理某件事的时候外界发生了紧急事件请求，要求

CPU 暂停当前的工作，转而去处理这个紧急事件，处理完以后，再回到原来被中断的地方，继续原来的工作，这样的过程称为中断。实现这种功能的部件称为中断系统，请示 CPU 中断的请求源称为中断源。微型机的中断系统一般允许多个中断源，当几个中断源同时向 CPU 请求中断，要求为它服务的时候，这就存在 CPU 优先响应哪一个中断源请求的问题。通常根据中断源的轻重缓急排队，优先处理最紧急事件的中断请求源，即规定每一个中断源有一个优先级别。CPU 总是先响应优先级别最高的中断请求。

当 CPU 正在处理一个中断源请求的时候(执行相应的中断服务程序)，发生了另外一个优先级比它还高的中断源请求。如果 CPU 能够暂停对原来中断源的服务程序，转而去处理优先级更高的中断请求源，处理完以后，再回到原低级中断服务程序，这样的过程称为中断嵌套。这样的中断系统称为多级中断系统，没有中断嵌套功能的中断系统称为单级中断系统。

中断的开启与关闭、设置启用哪一个中断等都是由单片机内部的一些特殊功能寄存器来决定的，在前面章节的学习中我们仅对单片机 IO 口操作过（实际上操作 IO 口即操作 IO 口寄存器，只不过编译器已经帮我们把 IO 口寄存器封装好直接操作 IO 即可，这些可在 51 单片机头文件内查看），从本章开始就会介绍单片机内部更多的特殊功能寄存器以及如何配置它实现相应的功能。

随着计算机技术的应用，人们发现中断技术不仅解决了快速主机与慢速 I/O 设备的数据传送问题，而且还具有如下优点：

①分时操作。CPU 可以分时为多个 I/O 设备服务，提高了计算机的利用率；

②实时响应。CPU 能够及时处理应用系统的随机事件，系统的实时性大大增强；

③可靠性高。CPU 具有处理设备故障及掉电等突发性事件能力，从而使系统可靠性提高。

19.2 中断结构及相关寄存器

19.2.1 中断结构

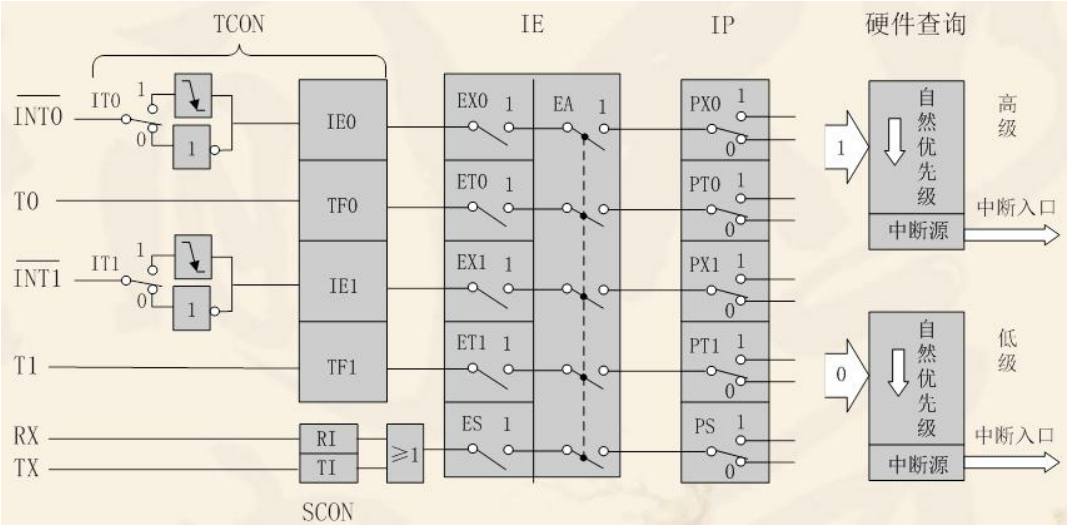
STC89C5X 系列单片机提供了 8 个中断请求源，它们分别是：外部中断

0(INT0)、外部中断 1(INT1)、外部中断 2(INT2)、外部中断 3(INT3)、定时器 0 中断、定时器 1 中断、定时器 2 中断、串口(UART)中断。（**注意：51 系列单片机一定有基本的 5 个中断，但不全有 8 个中断，需要查看芯片手册，通常我们使用的都是基本的 5 个中断：INT0、INT1、定时器 0/1，串口中断**）。所有的中断都具有四个中断优先级（基本型只有两个）。用户可以用关总中断允许位(EA/IE.7)或相应中断的允许位来屏蔽所有的中断请求，也可以用打开相应的中断允许位来使 CPU 响应相应的中断申请。其中有些中断源可以用软件独立地控制为开中断或关中断状态。每一个中断的优先级别均可用软件设置。高优先级的中断请求可以打断低优先级的中断，反之，低优先级的中断请求不可以打断高优先级及同优先级的中断。当两个相同优先级的中断同时产生时，将由查询次序来决定系统先响应哪个中断。 STC89C5X 系列单片机的各个中断查询次序表如下图所示：

中断源	中断向量地址	相同优先级内的查询次序	中断优先级设置(IPH,IP)	优先级0(最低)	优先级1	优先级2	优先级3(最高)	中断请求标志位	中断允许控制位
$\overline{\text{INT0}}$(外部中断 0)	0003H	0(highest)	PX0H, PX0	0, 0	0, 1	1, 0	1, 1	IE0	EX0/EA
Timer 0	000BH	1	PT0H, PT0	0, 0	0, 1	1, 0	1, 1	TF0	ET0/EA
$\overline{\text{INT1}}$(外部中断1)	0013H	2	PX1H, PX1	0, 0	0, 1	1, 0	1, 1	IE1	EX1/EA
Timer1	001BH	3	PT1H, PT1	0, 0	0, 1	1, 0	1, 1	TF1	ET1/EA
UART	0023H	4	PSH, PS	0, 0	0, 1	1, 0	1, 1	RI+TI	
Timer2	002BH	5	PT2H, PT2	0, 0	0, 1	1, 0	1, 1	TF2 + EXF2	(ET2)/EA
$\overline{\text{INT2}}$(外部中断2)	0033H	6	PX2H, PX2	0, 0	0, 1	1, 0	1, 1	IE2	EX2/EA
$\overline{\text{INT3}}$(外部中断3)	003BH	7(lowest)	PX3H, PX3	0, 0	0, 1	1, 0	1, 1	IE3	EX3/EA

通过设置新增加的特殊功能寄存器 IPH 中的相应位，可将中断优先级设为四级，如果只设置 IP 或 XICON，那么中断优先级就只有两级，与传统 8051 单片机两级中断优先级完全兼容。上图中的中断查询次序即为中断号，这个中断号在编程时非常重要，当中断来临时，只有中断号正确才能进入中断。

下面我们以 51 单片机均有的 5 个中断来介绍，其内部结构框图如下所示：



①INT0 对应的是 P3.2 口的附加功能，可由 IT0(TCON.0)选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.2 引脚上出现有效的中断信号时，中断标志 IE0(TCON.1)置 1，向 CPU 申请中断。

②INT1 对应的是 P3.3 口的附加功能，可由 IT1(TCON.2)选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.3 引脚上出现有效的中断信号时，中断标志 IE1(TCON.3)置 1,向 CPU 申请中断。

③T0 对应的是 P3.4 口的附加功能，TF0（TCON.5）,片内定时/计数器 T0 溢出中断请求标志。当定时/计数器 T0 发生溢出时，置位 TF0，并向 CPU 申请中断。

④T1 对应的是 P3.5 口的附加功能，TF1（TCON.7），片内定时/计数器 T1 溢出中断请求标志。当定时/计数器 T1 发生溢出时，置位 TF1，并向 CPU 申请中断。

⑤RXD 和 TXD 对应的是 P3.0 和 P3.1 口的附加功能，RI（SCON.0）或 TI（SCON.1），串行口中断请求标志。当串行口接收完一帧串行数据时置位 RI 或当串行口发送完一帧串行数据时置位 TI，向 CPU 申请中断。

19.2.2 中断相关寄存器

（1）中断允许控制

CPU 对中断系统所有中断以及某个中断源的开放和屏蔽是由中断允许寄存器 IE 控制的。

位	7	6	5	4	3	2	1	0	
字节地址：A8H	EA			ES	ET1	EX1	ET0	EX0	IE

EX0(IE.0)，外部中断 0 允许位；

ET0(IE.1)，定时/计数器 T0 中断允许位；

EX1(IE.2)，外部中断 0 允许位；

ET1(IE.3)，定时/计数器 T1 中断允许位；

ES（IE.4)，串行口中断允许位；

EA（IE.7)， CPU 中断允许（总允许）位。

（2）中断请求标志 TCON

位	7	6	5	4	3	2	1	0	
字节地址：88H	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	TCON

IT0（TCON.0），外部中断 0 触发方式控制位。

当 IT0=0 时，为电平触发方式。

当 IT0=1 时，为边沿触发方式（下降沿有效）。

IE0（TCON.1），外部中断 0 中断请求标志位。

IT1（TCON.2），外部中断 1 触发方式控制位。

IE1（TCON.3），外部中断 1 中断请求标志位。

TF0（TCON.5），定时/计数器 T0 溢出中断请求标志位。

TF1（TCON.7），定时/计数器 T1 溢出中断请求标志位。

（3）中断优先级

同一优先级中的中断申请不止一个时，则有中断优先权排队问题。同一优先级的中断优先权排队，由中断系统硬件确定的自然优先级形成，其排列如所示：

各中断源响应优先级及中断服务程序入口表

中断源	中断标志	中断服务程序入口	优先级顺序
外部中断 0（$\overline{INT0}$）	IE0	0003H	高
定时/计数器 0（T0）	TF0	000BH	↓
外部中断 1（$\overline{INT1}$）	IE1	0013H	↓
定时/计数器 1（T1）	TF1	001BH	↓
串行口	RI 或 TI	0023H	低

（4）中断号

中断源符号	名称	中断引起原因	中断号
/INT0	外部中断0	P3.2引脚低电平或下降沿信号	0
T0	定时器0中断	定时/计数器0计数回0溢出	1
/INT1	外部中断1	P3.3引脚低电平或下降沿信号	2
T1	定时器1中断	定时/计数器1计数回0溢出	3
TI/RI	串行口中断	串行通信完成一帧数据发送或接收引起中断	4

（5）中断响应条件

①中断源有中断请求；

②此中断源的中断允许位为 1；

③CPU 开中断（即 EA=1）。

以上三条同时满足时，CPU 才有可能响应中断。在使用中断时我们需要做什么呢？

①你想使用的中断是哪个？选择相应的中断号；

②你所希望的触发条件是什么？

③你希望在中断之后干什么？

我们以外部中断 0 为例，如下：

主程序中需要有以下代码：

```
EA=1；//打开总中断开关
EX0=1；//开外部中断 0
IT0=0/1；//设置外部中断的触发方式
```

中断服务函数：

```
void int0() interrupt 0 using 1
{
    //编写用户所需的功能代码
}
```

在中断函数中 int0 是函数名，可自定义，但必须符合 C 语言标识符定义规则，interrupt 是一个关键字，表示 51 单片机中断。后面的“0”是中断号，外部中断 0 中断号为 0，这个可参考前面的内容。后面的 using 1 可省略不写。

至此我们就把 51 单片机的中断简单介绍完，如果大家还有不明白的可先暂时放下，先通过实验掌握 51 单片机中断的使用方法，对于理论方面的知识后续可以深入研究。

课后作业

普中51单片机开发板

第 20 章 外部中断实验

上一章我们介绍了 51 单片机的中断系统，这一章就来学习 51 单片机的外部中断，通过上一章的介绍我们知道，51 单片机外部中断有 2 个，外部中断 0 和

外部中断 1，它们的使用方法是一样的，所以只要学会一个即可掌握所有外部中断使用。本章所要实现的功能是：使用独立按键 K3 控制 LED 亮灭，K3 连接外部中断 0（P3.2）管脚。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

20.1 外部中断介绍

20.2 外部中断配置

20.3 硬件设计

20.4 软件设计

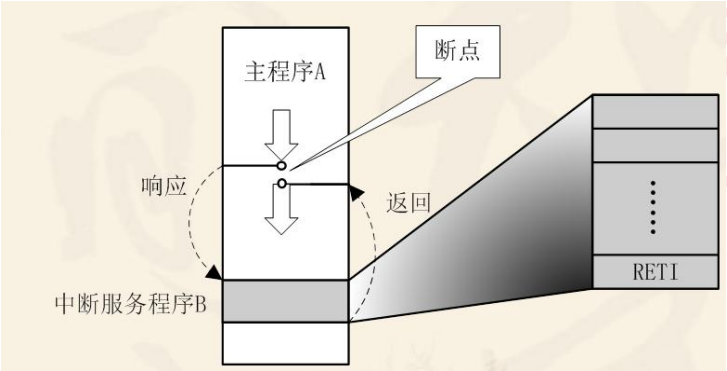
20.5 实验现象

普中51单片机开发板

20.1 外部中断介绍

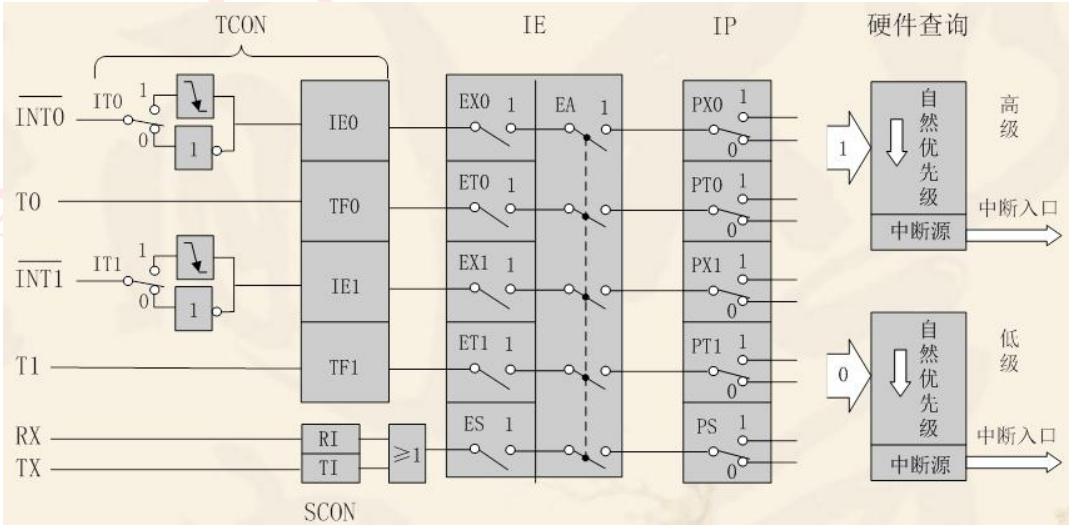
上一章我们介绍了 51 单片机的中断系统,我们再回顾一下 51 单片机的中断。当中央处理机 CPU 正在处理某件事的时候外界发生了紧急事件请求，要求

CPU 暂停当前的工作，转而去处理这个紧急事件，处理完以后，再回到原来被中断的地方，继续原来的工作，这样的过程称为中断。实现这种功能的部件称为中断系统，请示 CPU 中断的请求源称为中断源。微型机的中断系统一般允许多个中断源，当几个中断源同时向 CPU 请求中断，要求为它服务的时候，这就存在 CPU 优先响应哪一个中断源请求的问题。通常根据中断源的轻重缓急排队，优先处理最紧急事件的中断请求源，即规定每一个中断源有一个优先级别。CPU 总是先响应优先级别最高的中断请求。中断示意图如下所示：



STC89C5X 系列单片机提供了 4 个外部中断：外部中断 0(INT0)、外部中断 1(INT1)、外部中断 2(INT2)、外部中断 3(INT3)。（**注意：51 系列单片机一定有基本的 2 个外部中断，但不全有 4 个中断，需要查看芯片手册，通常我们都是使用基本的 2 个外部中断：INT0 和 INT1**）。

下面我们来看下外部中断结构图，如下所示：



图中 INT0 和 INT1 即为外部中断 0 和外部中断 1 输入口。

①INT0 对应的是 P3.2 口的附加功能，可由 IT0(TCON.0)选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.2 引脚上出现有效的中断信号时，中断标

志 IE0(TCON.1)置 1，向 CPU 申请中断。

②INT1 对应的是 P3.3 口的附加功能，可由 IT1(TCON.2)选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.3 引脚上出现有效的中断信号时，中断标志 IE1(TCON.3)置 1,向 CPU 申请中断。

（1）中断允许控制

CPU 对中断系统所有中断以及某个中断源的开放和屏蔽是由中断允许寄存器 IE 控制的。

位	7	6	5	4	3	2	1	0	
字节地址：A8H	EA			ES	ET1	EX1	ET0	EX0	IE

EX0(IE.0)，外部中断 0 允许位；

ET0(IE.1)，定时/计数器 T0 中断允许位；

EX1(IE.2)，外部中断 0 允许位；

ET1(IE.3)，定时/计数器 T1 中断允许位；

ES（IE.4)，串行口中断允许位；

EA（IE.7)， CPU 中断允许（总允许）位。

（2）中断请求标志 TCON

位	7	6	5	4	3	2	1	0	
字节地址：88H	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	TCON

IT0（TCON.0），外部中断 0 触发方式控制位。

当 IT0=0 时，为电平触发方式。

当 IT0=1 时，为边沿触发方式（下降沿有效）。

IE0（TCON.1），外部中断 0 中断请求标志位。

IT1（TCON.2），外部中断 1 触发方式控制位。

IE1（TCON.3），外部中断 1 中断请求标志位。

TF0（TCON.5），定时/计数器 T0 溢出中断请求标志位。

TF1（TCON.7），定时/计数器 T1 溢出中断请求标志位。

20.2 外部中断配置

我们知道要让 51 单片机发生中断必须要满足以下 3 个条件，这 3 个条件的顺序可以任意：

①中断源有中断请求；

②此中断源的中断允许位为 1；

③CPU 开中断（即 EA=1）。

比如我们配置外部中断 0，对应的配置程序如下：

```
EA=1；//打开总中断开关
EX0=1；//开外部中断 0
IT0=0/1；//设置外部中断的触发方式
```

如果要配置的是外部中断 1，只需将 EX0 改为 EX1，IT0 改为 IT1。

因为独立按键一端是共地的，当按下后对应单片机 IO 口被拉低，而默认单片机 IO 口是高电平，这样就有一个下降沿过程，所以通常使用外部中断都是配置为下降沿触发，即 IT0=1；

在编写程序时通常我们会将外部中断的配置放到一个自定义函数内便于管理维护。如下所示：

```
void exti0_init(void)
{
    IT0=1;//跳变沿触发方式（下降沿）
    EX0=1;//打开 INT0 的中断允许
    EA=1;//打开总中断
}
```

当触发中断后即会进入中断服务函数，外部中断 0 中断服务函数如下：

```
void exti0() interrupt 0 //外部中断 0 中断函数
{
    //执行所需的功能
}
```

在中断函数中 exti0 是函数名，可自定义，但必须符合 C 语言标识符定义规则，interrupt 是一个关键字，表示 51 单片机中断。后面的“0”是中断号，外部中断 0 中断号为 0，如果是外部中断 1，则中断号为 2，这个可参考中断章节的内容。

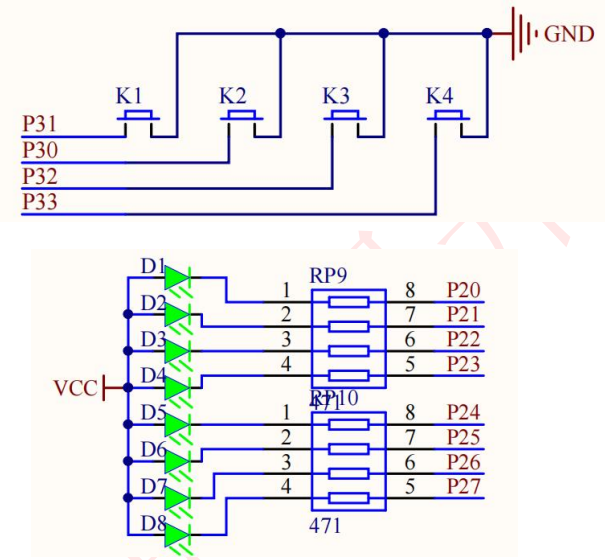
20.3 硬件设计

本实验使用到硬件资源如下：

（1）独立按键模块（K3）

（2）LED 模块（D1）

LED 模块和独立按键模块电路在前面章节已介绍过，这里就不多说。原理图中 K3 键是连接在单片机 P3.2 口（外部中断 0），K4 按键是连接在 P3.3 口（外部中断 1）。



20.4 软件设计

本章所要实现的功能是：使用独立按键 K3 控制 LED 亮灭。

我们打开“\4--实验程序\1--基础实验\16-外部中断实验\16.1-外部中断 0 实验”工程，控制代码全部都在 main.c 中，代码如下：

```
/*********************************************************************************************
实验名称：外部中断 0 实验
接线说明：
实验现象：下载程序后，当按下 K3 键可控制 D1 指示灯亮灭
注意事项：将红外接收传感器取下，防止对 P3.2 口干扰

*********************************************************************************************/
#include "reg52.h"
```

```
typedef unsigned int u16;//对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义 LED1 管脚
sbit LED1=P2^0;

//定义独立按键 K3 控制脚
sbit KEY3=P3^2;


/*******************************************************************************
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*******************************************************************************/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*******************************************************************************
* 函 数 名       : exti0_init
* 函数功能       : 外部中断 0 配置函数
* 输    入       : 无
* 输    出       : 无
*******************************************************************************/
void exti0_init(void)
{
    IT0=1;//跳变沿触发方式（下降沿）
    EX0=1;//打开 INT0 的中断允许
    EA=1;//打开总中断
}

/*******************************************************************************
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*******************************************************************************/
void main()
{
    exti0_init();//外部中断 0 配置
```

```
    while(1)
    {

    }
}

void exti0() interrupt 0 //外部中断 0 中断函数
{
    delay_10us(1000);//消斗
    if(KEY3==0)//再次判断 K3 键是否按下
        LED1=!LED1;//LED1 状态翻转
}
```

程序代码比较简单，首先定义 K3 键与 LED1 的控制管脚，然后定义了外部中断 0 配置函数 exti0_init，该函数内容是按照前面介绍的配置方法实现，即开启总中断、外部中断 0 功能，设置外部中断 0 为下降沿触发。然后进入 while 循环，在循环体内没有执行任何功能程序。

有人就会问，在主函数中怎么没有看到按键对 LED 的控制呢？因为我们在 exti0_init()函数内就已经把按键管脚配置为外部中断 0 下降沿触发，当有按键按下，即会进入对应中断服务函数执行相应的功能程序，LED 的控制就在中断函数内完成的。

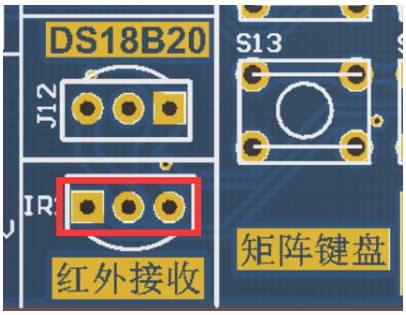
外部中断 1 的使用与外部中断 0 是一样的，大家可以打开“16.2-外部中断 1 实验”程序查看。这里就不再重复。

20.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：当按下 K3 键，D1 指示灯亮，再次按下 K3 键，D1 指示灯灭，如此循环。

如果下载的是外部中断 1 实验程序，实验现象如下：当按下 K4 键，D1 指示灯亮，再次按下 K4 键，D1 指示灯灭，如此循环

注意：由于红外接收传感器与 K3 共用 P3.2 口，因此在做外部中断 0 实验时，将红外接收传感器从开发板取下，防止干扰。



课后作业

普中51单片机开发板

第 21 章 定时器中断

上一章我们介绍了 51 单片机的外部中断，学习了如何配置 51 单片机的外部中断。这一章我们来学习 51 单片机的定时器中断。STC89C5X 含有 3 个定时器：定时器 0、定时器 1、定时器 2（**注意：51 系列单片机一定有基本的 2 个定时器（定时器 0 和定时器 1），但不全有 3 个中断，需要查看芯片手册，通常我们使用的是基本的 2 个定时器：定时器 0/1**）。本章以定时器 0 为例进行讲解，让大家学会 51 单片机定时器的使用，定时器 1 的使用方法与定时器 0 一样。本章要实现的功能是：通过定时器 0 中断控制 D1 指示灯间隔 1 秒闪烁。学习本章可以

参考前面的实验章节内容。本章分为如下几部分内容：

21.1 定时器介绍

21.2 定时器配置

21.3 硬件设计

21.4 软件设计

21.5 实验现象

普中51单片机开发板

21.1 定时器介绍

在介绍定时器之前先科普下几个知识：

1，CPU 时序的有关知识

①振荡周期：为单片机提供定时信号的振荡源的周期（晶振周期或外加振荡周期）。

②状态周期：2 个振荡周期为 1 个状态周期，用 S 表示。振荡周期又称 S 周期或时钟周期。

③机器周期：1 个机器周期含 6 个状态周期，12 个振荡周期。

④指令周期：完成 1 条指令所占用的全部时间，它以机器周期为单位。

例如：外接晶振为 12MHz 时，51 单片机相关周期的具体值为：

振荡周期=1/12us；

状态周期=1/6us；

机器周期=1us；

指令周期=1~4us；

2，学习定时器前需要明白的几点

①51 单片机有两组定时器/计数器，因为既可以定时，又可以计数，故称之为定时器/计数器。

②定时器/计数器和单片机的 CPU 是相互独立的。定时器/计数器工作的过程是自动完成的，不需要 CPU 的参与。

③51 单片机中的定时器/计数器是根据机器内部的时钟或者是外部的脉冲信号对寄存器中的数据加 1。

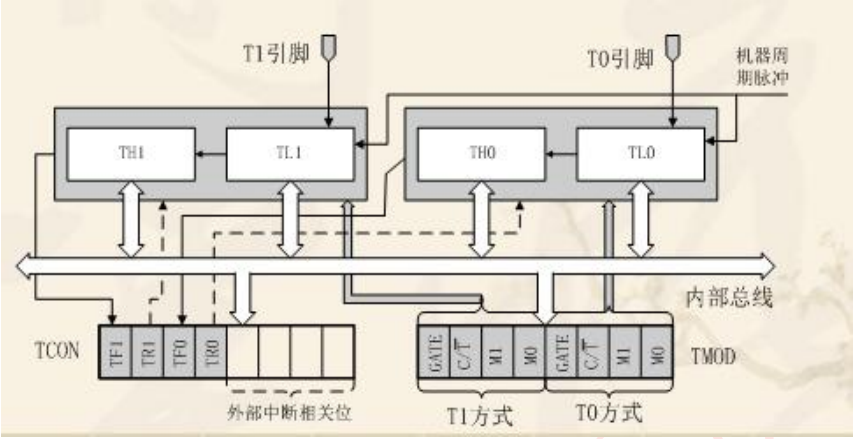
有了定时器/计数器之后，可以增加单片机的效率，一些简单的重复加 1 的工作可以交给定时器/计数器处理。CPU 转而处理一些复杂的事情。同时可以实现精确定时作用。

21.1.1 51 单片机定时器原理

STC89C5X 单片机内有两个可编程的定时/计数器 T0、T1 和一个特殊功能定时器 T2。定时/计数器的实质是加 1 计数器（16 位），由高 8 位和低 8 位两个寄存器 THx 和 TLx 组成。它随着计数器的输入脉冲进行自加 1，也就是每来一个脉冲，计数器就自动加 1，当加到计数器为全 1 时，再输入一个脉冲就使计数器回零，且计数器的溢出使相应的中断标志位置 1，向 CPU 发出中断请求（定时/计数器中断允许时）。如果定时/计数器工作于定时模式，则表示定时时间已到；如果工作于计数模式，则表示计数值已满。可见，由溢出时计数器的值减去计数初值才是加 1 计数器的计数值。

21.1.2 51 单片机定时/计数器结构

51 单片机定时器/计数器内部结构如下所示：



上图中的 T0 和 T1 引脚对应的是单片机 P3.4 和 P3.5 管脚。51 单片机定时/计数器的工作由两个特殊功能寄存器控制。TMOD 是定时/计数器的工作方式寄存器，确定工作方式和功能；TCON 是控制寄存器，控制 T0、T1 的启动和停止及设置溢出标志。

1，工作方式寄存器 TMOD

工作方式寄存器 TMOD 用于设置定时/计数器的工作方式，低四位用于 T0，高四位用于 T1。其格式如下：

位	7	6	5	4	3	2	1	0	
字节地址：89H	GATE	C/$\overline{T}$	M1	M0	GATE	C/$\overline{T}$	M1	M0	TMOD

GATE 是门控位，GATE＝0 时，用于控制定时器的启动是否受外部中断源信号的影响。只要用软件使 TCON 中的 TR0 或 TR1 为 1，就可以启动定时/计数器工作；GATA＝1 时，要用软件使 TR0 或 TR1 为 1，同时外部中断引脚 INT0/1 也为高电平时，才能启动定时/计数器工作。即此时定时器的启动条件，加上了 INT0/1 引脚为高电平这一条件。

C/T ：定时/计数模式选择位。C/T ＝0 为定时模式;C/T =1 为计数模式。

M1M0：工作方式设置位。定时/计数器有四种工作方式。

定时/计数器工作方式设置表

M1M0	工作方式	说　　明
00	方式 0	13 位定时/计数器
01	方式 1	16 位定时/计数器
10	方式 2	8 位自动重装定时/计数器
11	方式 3	T0 分成两个独立的 8 位定时/计数器；T1 此方式停止计数

2，控制寄存器 TCON

TCON 的低 4 位用于控制外部中断,已在前面介绍。TCON 的高 4 位用于控制定时/计数器的启动和中断申请。其格式如下：

位	7	6	5	4	3	2	1	0	
字节地址：88H	TF1	TR1	TF0	TR0					TCON

TF1（TCON.7）：T1 溢出中断请求标志位。T1 计数溢出时由硬件自动置 TF1 为 1。CPU 响应中断后 TF1 由硬件自动清 0。T1 工作时，CPU 可随时查询 TF1 的状态。所以，TF1 可用作查询测试的标志。TF1 也可以用软件置 1 或清 0，同硬件置 1 或清 0 的效果一样。

TR1（TCON.6）：T1 运行控制位。TR1 置 1 时，T1 开始工作；TR1 置 0 时，T1 停止工作。TR1 由软件置 1 或清 0。所以，用软件可控制定时/计数器的启动与停止。

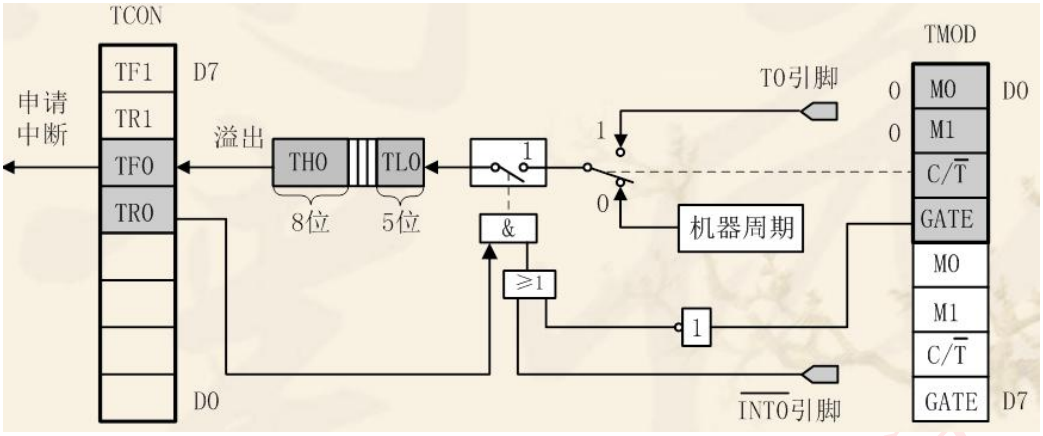
TF0（TCON.5）：T0 溢出中断请求标志位，其功能与 TF1 类同。

TR0（TCON.4）：T0 运行控制位，其功能与 TR1 类同。

21.1.3 51 单片机定时/计数器的工作方式

1，方式 0

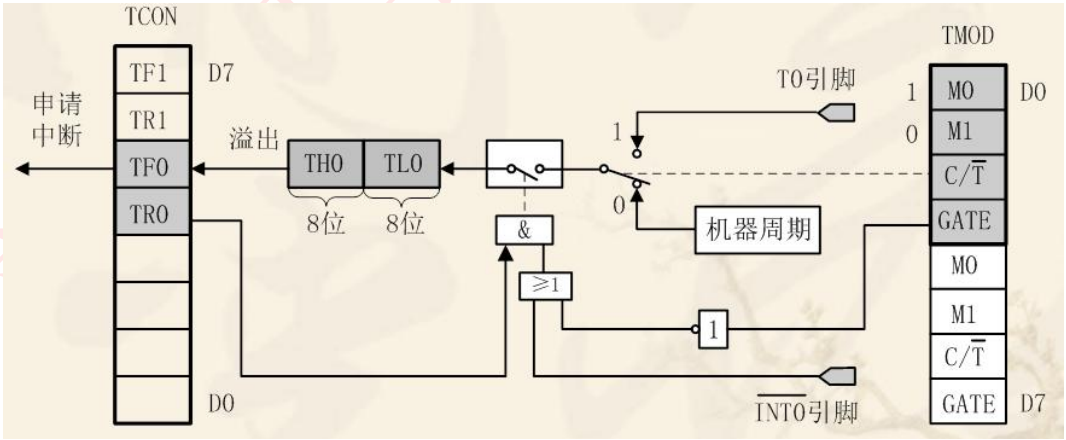
方式 0 为 13 位计数，由 TL0 的低 5 位（高 3 位未用）和 TH0 的 8 位组成。TL0 的低 5 位溢出时向 TH0 进位，TH0 溢出时，置位 TCON 中的 TF0 标志，向 CPU 发出中断请求。其结构图如下所示：



门控位 GATE 具有特殊的作用。当 GATE=0 时，经反相后使或门输出为 1，此时仅由 TR0 控制与门的开启，与门输出 1 时，控制开关接通，计数开始；当 GATE=1 时，由外中断引脚信号控制或门的输出，此时控制与门的开启由外中断引脚信号和 TR0 共同控制。当 TR0=1 时，外中断引脚信号引脚的高电平启动计数，外中断引脚信号引脚的低电平停止计数。这种方式常用来测量外中断引脚上正脉冲的宽度。计数模式时，计数脉冲是 T0 引脚上的外部脉冲。计数初值与计数个数的关系为：$X=2^{13}-N$。

2，方式 1

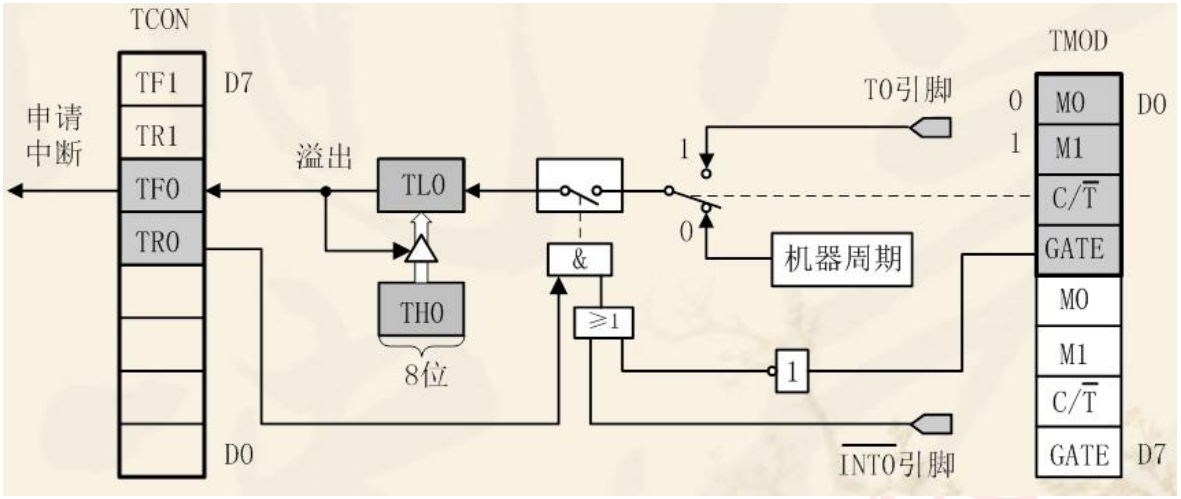
方式 1 的计数位数是 16 位，由 TL0 作为低 8 位，TH0 作为高 8 位，组成了 16 位加 1 计数器。其结构图如下所示：



计数初值与计数个数的关系为：$X=2^{16}-N$。

3，方式 2

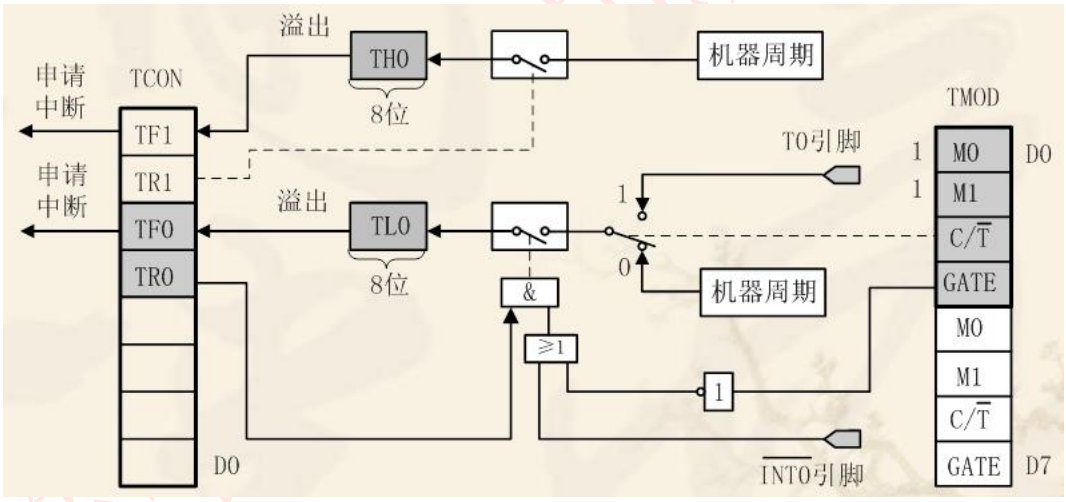
方式 2 为自动重装初值的 8 位计数方式。工作方式 2 特别适合于用作较精确的脉冲信号发生器。其结构图如下所示：



计数初值与计数个数的关系为：$X=2^8-N$。

4，方式 3

方式 3 只适用于定时/计数器 T0，定时器 T1 处于方式 3 时相当于 TR1=0，停止计数。工作方式 3 将 T0 分成为两个独立的 8 位计数器 TL0 和 TH0。其结构如下所示：



这几种工作方式中应用较多的是方式 1 和方式 2。定时器中通常使用定时器方式 1，串口通信中通常使用方式 2。

21.2 定时器配置

在使用定时器时，应该如何配置使其工作？其步骤如下（各步骤顺序可任意）：

①对 TMOD 赋值，以确定 T0 和 T1 的工作方式，如果使用定时器 0 即对 T0 配置，如果使用定时器 1 即对 T1 配置。

②根据所要定时的时间计算初值,并将其写入 TH0、TL0 或 TH1、TL1。

③如果使用中断，则对 EA 赋值，开放定时器中断。

④使 TR0 或 TR1 置位，启动定时/计数器定时或计数。

上述中有一个定时/计数器初值的计算，下面我们来看下如何计算定时/计数器初值。

前面我们介绍过机器周期的概念,它是 CPU 完成一个基本操作所需要的时间。其计算公式是：机器周期=1/单片机的时钟频率。51 单片机内部时钟频率是外部时钟的 12 分频，也就是说当外部晶振的频率输入到单片机里面的时候要进行 12 分频。比如说你用的是 12MHZ 晶振，那么单片机内部的时钟频率就是 12/12MHZ，当你使用 12MHZ 的外部晶振的时候，机器周期=1/1M=1us。如果我们想定时 1ms 的初值是多少呢？1ms/1us=1000。也就是要计数 1000 个，初值=65535-1000+1（因为实际上计数器计数到 65536（2 的 16 次方）才溢出，所以后面要加 1）=64536=FC18H，所以初值即为 THx=0XFC，TLx=0X18。

知道了如何计算定时/计数器初值，那么想定时多长时间都可以计算出，当然由于定时计数器位数有限，我们不可能直接通过初值定时很长时间，如果要实现很长时间的定时，比如定时 1 秒钟。可以通过初值设置定时 1ms，每当定时 1ms 结束后又重新赋初值，并且设定一个全局变量累计定时 1ms 的次数，当累计到 1000 次，表示已经定时 1 秒了。需要其他定时时间类似操作，这样我们就可以使用定时器来实现精确延时来替代之前的 delay 函数。

这里以定时器 0 为例介绍配置定时器工作方式 1、设定 1ms 初值，开启定时器计数功能以及总中断，如下：

```
void time0_init(void)
{
    TMOD|=0X01;//选择为定时器 0 模式，工作方式 1
    TH0=0XFC;  //给定时器赋初值，定时 1ms
    TL0=0X18;
    ET0=1;//打开定时器 0 中断允许
    EA=1;//打开总中断
    TR0=1;//打开定时器
```

```
}
```

对于定时器 1 的使用方法是一样的，只是将上述的 0 变为 1 即可，具体可参考我们定时器 1 实验例程。

21.3 硬件设计

本实验使用到硬件资源如下：

（1）LED 模块（D1）

（2）定时器 0

本章硬件电路非常简单，只使用到开发板上 LED 模块的 D1，至于定时器 0 它属于 51 单片机内部资源，只需通过软件配置即可使用。开发板上 LED 模块电路在前面已经介绍，这里就不多说。

21.4 软件设计

本章所要实现的功能是：通过定时器 0 中断控制 D1 指示灯间隔 1 秒闪烁。

我们打开“\4--实验程序\1--基础实验\17-定时器实验\17.1-定时器 0 实验”工程，控制代码全部都在 main.c 中，代码如下：

```
/*********************************************************************************
****
实验名称：定时器 0 实验
接线说明：
实验现象：下载程序后，当按下 D1 指示灯间隔 1s 闪烁
注意事项：

*********************************************************************************
****/
#include "reg52.h"

typedef unsigned int u16;//对系统默认数据类型进行重定义
typedef unsigned char u8;

//定义 LED1 管脚
sbit LED1=P2^0;
```

```
/*******************************************************************************
* 函 数 名       : delay_10us
* 函数功能       : 延时函数，ten_us=1 时，大约延时 10us
* 输    入       : ten_us
* 输    出       : 无
*******************************************************************************/
void delay_10us(u16 ten_us)
{
    while(ten_us--);
}

/*******************************************************************************
* 函 数 名       : time0_init
* 函数功能       : 定时器 0 中断配置函数，通过设置 TH 和 TL 即可确定定时时间
* 输    入       : 无
* 输    出       : 无
*******************************************************************************/
void time0_init(void)
{
    TMOD|=0X01;//选择为定时器 0 模式，工作方式 1
    TH0=0XFC;    //给定时器赋初值，定时 1ms
    TL0=0X18;
    ET0=1;//打开定时器 0 中断允许
    EA=1;//打开总中断
    TR0=1;//打开定时器
}

/*******************************************************************************
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*******************************************************************************/
void main()
{
    time0_init();//定时器 0 中断配置

    while(1)
    {

    }
}

void time0() interrupt 1 //定时器 0 中断函数
```

```
{
    static u16 i;//定义静态变量 i
    TH0=0XFC;    //给定时器赋初值，定时 1ms
    TL0=0X18;
    i++;
    if(i==1000)
    {
        i=0;
        LED1=!LED1;
    }
}
```

实验代码非常简单，首先定义 LED1 指示灯控制管脚，然后定义定时器 0 中断配置函数 time0_init，该函数配置内容就是按照前面介绍的配置方法所写，即选择定时器工作模式 0、工作方式 1、设置定时 1ms 初值、打开定时器计数功能和开启总中断功能。然后进入 while 循环，在循环体内没有执行任何功能程序。当定时时间到达即会进入定时器 0 中断，在中断服务函数内，重新赋初值准备下次计数，并且定义一个静态变量来累计定时 1ms 次数，当变量等于 1000 时，表示定时时间达 1 秒，然后清零变量以及控制 LED 状态翻转。执行完成后退出中断返回主函数，当时间到达又进入中断，如此循环。

为什么要使用关键字 static 将 i 定义为静态变量呢？我们希望每次进入中断函数时，i 保存的是上次累加值，使用了 static 关键字，就可以让变量 i 实现这种功能，即不会每次进入中断函数后被初始化为 0。假如去掉 static 关键字，那么变量 i 就是一个局部变量，每次进入中断函数后，变量 i 初始值都是 0，也就是说它的值永远也不会递增到 1000，从而实现不了 1s 定时。可以这样理解，使用了 static 关键字就相当于将 i 变成了一个全局变量功能。

对于定时器 1 的使用方法是一样的，只是将上述的 0 变为 1 即可，具体可参考我们定时器 1 实验例程。

21.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：当 D1 指示灯间隔 1s 闪烁。

注意：实验中我们是以 12M 晶振为例讲解初值的计算，假如外部晶振为 11.0592M，那么使用前面的计算公式可能就比较繁琐，这里给大家推荐一个非常好用的小工具“\5--开发工具\4-常用辅助开发软件\51 定时器计算.exe”，打开小工具，选择好定时器工作方式、晶振频率、定时时长即可得出 TH 和 TL 的初值。假如现在使用的晶振是 11.0592M，使用定时器方式 1，定时时间为 1ms，设置界面如下：

定时器初值为 TH0=0XFC，TL0=0X66。该值与前面 12M 晶振得到的初值是有差别的，这也就是很多同学咨询为什么开发板上定时器工作慢或者快的原因，一定要注意开发板上使用的外部晶振大小，晶振大小不同，换算的初值是不一样的。

课后作业

第 22 章 串口通信实验

随着单片机系统的广泛应用和计算机网络技术的普及，单片机的通信功能愈来愈显得重要。在 WIFI、蓝牙、GPS、GSM/GPRS 等应用的控制无不体现串口通信的重要性。在前面章节我们就说过，学习 51 单片机重点及难点内容主要有中断、定时/计数器、串口通信。这一章我们就来学习 51 单片机的串口通信。开发板上集成了 1 个串口通信电路，是 USB 转串口模块，它既可下载程序也可实现串口通信功能。本章要实现的功能是：51 单片机通过串口（UART）实现与 PC 机对话，51 单片机的串口收到 PC 机发来的数据后原封不动的返回给 PC 机显示。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

22.1 通信的基本概念
22.2 51 单片机串口介绍
22.3 硬件设计
22.4 软件设计
22.5 实验现象

普中51单片机开发板

22.1 通信的基本概念

我们知道 51 单片机不仅可以实现串口通信，还可以通过 IO 口模拟实现多种其他通信，比如 SPI、IIC 等，学习这些通信前，我们很有必要了解下通信的基本概念。通信的方式可以分为多种，按照数据传送方式可分为串行通信和并行通信。按照通信的数据同步方式，可分为异同通信和同步通信。按照数据的传输方向又可分为单工、半双工和全双工通信。下面我们就来简单介绍这几种通信方式。

22.1.1 串行通信与并行通信

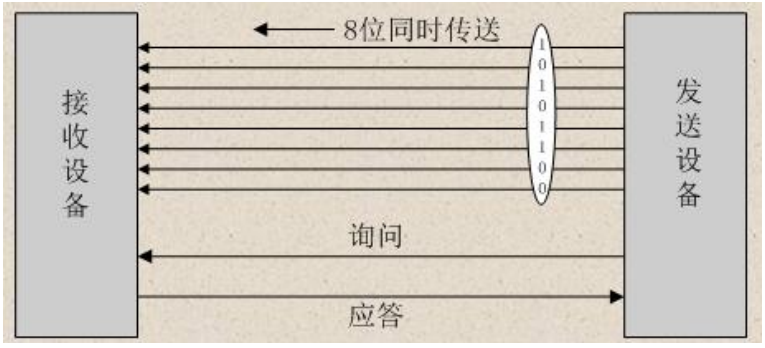
（1）串行通信

串行通信是指使用一条数据线，将数据一位一位地依次传输，每一位数据占据一个固定的时间长度。其只需要少数几条线就可以在系统间交换信息，特别适用于计算机与计算机、计算机与外设之间的远距离通信。如下图所示：

串行通信的特点：传输线少，长距离传送时成本低，且可以利用电话网等现成的设备，但数据的传送控制比并行通信复杂。

（2）并行通信

并行通信通常是将数据字节的各位用多条数据线同时进行传送，通常是 8 位、16 位、32 位等数据一起传输。如下图所示：



并行通信的特点：控制简单、传输速度快；由于传输线较多，长距离传送时

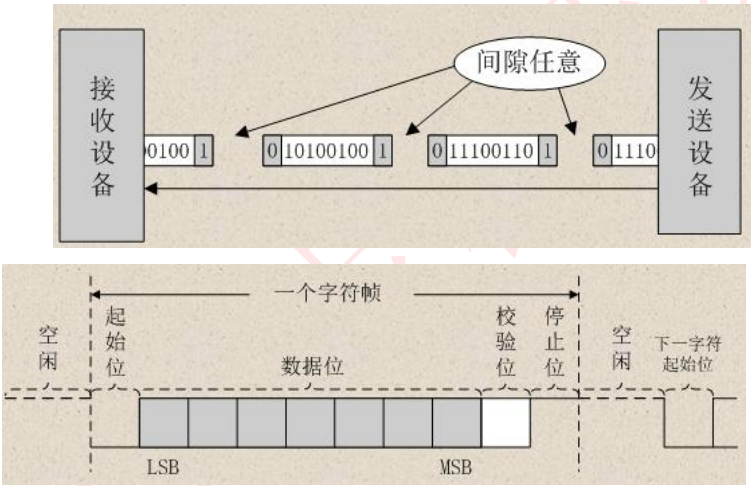
成本高且接收方的各位同时接收存在困难，抗干扰能力差。

22.1.2 异步通信与同步通信

（1）异步通信

异步通信是指通信的发送与接收设备使用各自的时钟控制数据的发送和接收过程。为使双方的收发协调，要求发送和接收设备的时钟尽可能一致。

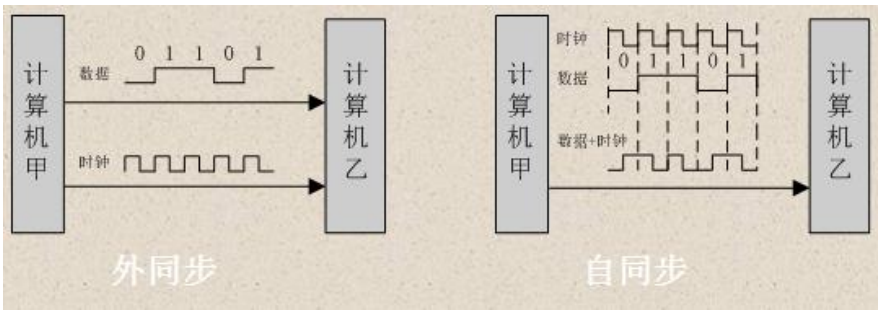
异步通信是以字符（构成的帧）为单位进行传输，字符与字符之间的间隙（时间间隔）是任意的，但每个字符中的各位是以固定的时间传送的，即字符之间不一定有“位间隔”的整数倍的关系，但同一字符内的各位之间的距离均为“ 位间隔”的整数倍。如下图所示：



异步通信的特点：不要求收发双方时钟的严格一致，实现容易，设备开销较小，但每个字符要附加 2～3 位用于起止位，各帧之间还有间隔，因此传输效率不高。

（2）同步通信

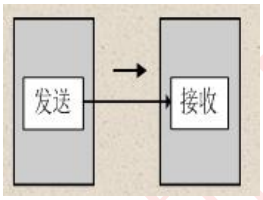
同步通信时要建立发送方时钟对接收方时钟的直接控制，使双方达到完全同步。此时，传输数据的位之间的距离均为“位间隔”的整数倍，同时传送的字符间不留间隙，即保持位同步关系，也保持字符同步关系。发送方对接收方的同步可以通过两种方法实现。如下图所示：



22.1.3 单工、半双工与全双工通信

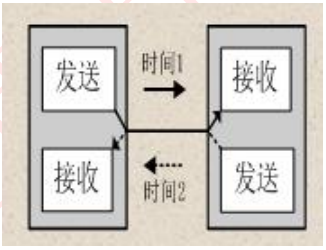
（1）单工通信

单工是指数据传输仅能沿一个方向，不能实现反向传输。如下图所示：



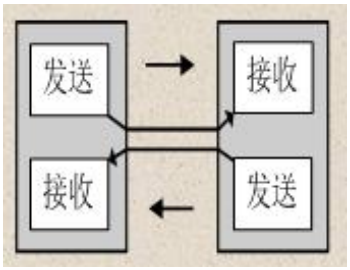
（2）半双工通信

半双工是指数据传输可以沿两个方向，但需要分时进行。如下图所示：



（3）全双工通信

全双工是指数据可以同时进行双向传输。如下图所示：



22.1.4 通信速率

衡量通信性能的一个非常重要的参数就是通信速率，通常以比特率(Bitrate)

来表示。比特率是每秒钟传输二进制代码的位数，单位是：位 / 秒（ bps）。如每秒钟传送 240 个字符，而每个字符格式包含 10 位(1 个起始位、1 个停止位、8 个数据位)，这时的比特率为：

10 位×240 个/秒 = 2400 bps

在后面会遇到一个“波特率”的概念，它表示每秒钟传输了多少个码元。而码元是通信信号调制的概念，通信中常用时间间隔相同的符号来表示一个二进制数字，这样的信号称为码元。如常见的通信传输中，用 0V 表示数字 0，5V 表示数字 1，那么一个码元可以表示两种状态 0 和 1，所以一个码元等于一个二进制比特位，此时波特率的大小与比特率一致；如果在通信传输中，有 0V、2V、4V 以及 6V 分别表示二进制数 00、01、10、11，那么每个码元可以表示四种状态，即两个二进制比特位，所以码元数是二进制比特位数的一半，这个时候的波特率为比特率的一半。由于很多常见的通信中一个码元都是表示两种状态，所以我们常常直接以波特率来表示比特率。

22.2 51 单片机串口介绍

22.2.1 串口通信简介

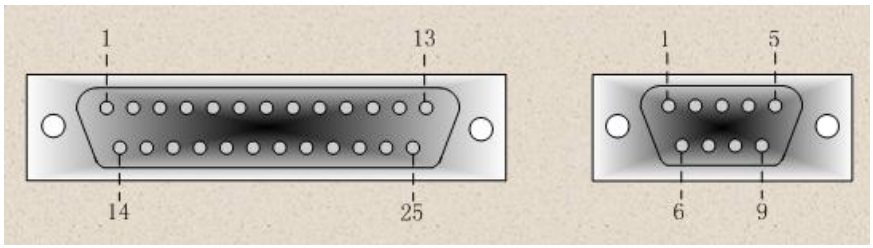
串口通信(Serial Communication)，是指外设和计算机间通过数据信号线、地线等按位进行传输数据的一种通信方式，属于串行通信方式。串口是一种接口标准，它规定了接口的电气标准，没有规定接口插件电缆以及使用的协议。

（1）接口标准

串口通信的接口标准有很多，有 RS-232C、RS-232、RS-422A、RS-485 等。常用的是 RS-232 和 RS-485。RS-232 其实是 RS-232C 的改进，原理是一样的。这里我们就以 RS-232C 接口进行讲解。

RS-232C 是 EIA（美国电子工业协会）1969 年修订 RS-232C 标准。RS-232C 定义了数据终端设备（DTE）与数据通信设备（DCE）之间的物理接口标准。

RS-232C 接口规定使用 25 针连接器，简称 DB25，连接器的尺寸及每个插针的排列位置都有明确的定义，如下图所示：



RS-232C 还有一种 9 针的非标准连接器接口，简称 DB9。串口通信使用的大多都是 DB9 接口。DB25 和 DB9 接头有公头和母头之分，其中带针状的接头是公头，而带孔状的接头是母头。9 针串口线的外观图如下图所示：

从上图中可以看到公头和母头的管脚定义顺序是不一样，这一点需要特别注意。这些管脚都有什么作用呢？ 9 针串口和 25 针串口常用管脚的功能说明如下图所示：

RS-232C 标准接口主要引脚定义

插针序号	信号名称	功能	信号方向
1	PGND	保护接地	
2（3）	TXD	发送数据（串行输出）	DTE→DCE
3（2）	RXD	接收数据（串行输入）	DTE←DCE
4（7）	RTS	请求发送	DTE→DCE
5（8）	CTS	允许发送	DTE←DCE
6（6）	DSR	DCE 就绪（数据建立就绪）	DTE←DCE
7（5）	SGND	信号接地	
8（1）	DCD	载波检测	DTE←DCE
20（4）	DTR	DTE 就绪（数据终端准备就绪）	DTE→DCE
22（9）	RI	振铃指示	DTE←DCE

注：插针序号（）内为 9 针非标准连接器的引脚号

在串口通信中，通常我们只使用 2、3、5 三个管脚，即 TXD、RXD、SGND，其他管脚功能大家看不明白也没关系。

RS-232C 对逻辑电平也做了规定，如下：

在 TXD 和 RXD 数据线上：

1.逻辑 1 为-3~-15V 的电压

2.逻辑 0 为 3~15V 的电压

在 RTS、CTS、DSR、DTR 和 DCD 等控制线上：

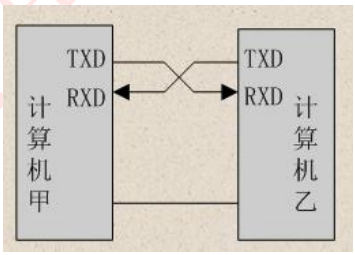
1.信号有效（ ON 状态） 为 3~15V 的电压

2.信号无效（ OFF 状态） 为-3~-15V 的电压

由此可见，RS-232C 是用正负电压来表示逻辑状态，与晶体管-晶体管逻辑集成电路（TTL）以高低电平表示逻辑状态的规定正好相反。而我们 51 单片机使用的就是 TTL 电平，所以要实现 51 单片机与计算机的串口通信，需要进行 TTL 与 RS-232C 电平转换，通常使用的电平转换芯片是 MAX232。

在串口通信中通常 PC 机的 DB9 为公头，单片机上使用的串口 DB9 为母头，通过一根直通串口线进行相连。在 9 针串口线实物图即为直通型串口线，串口线（COM）母头连接计算机 DB9 的公头，串口线公头连接单片机上使用的 DB9 母头，这样就是将 2、3、5 管脚直接相连。如果你要实现两台计算机串口通信，那么就需要一根交叉串口线，将 2 对 3、3 对 2、5 对 5 连接，交叉串口线一般两头都是母头。

串口通信中还需要注意的是，串口数据收发线要交叉连接，计算机的 TXD 要对应单片机的 RXD，计算机的 RXD 要对应单片机的 TXD，并且共 GND，如下图：



有的朋友就会问了，在计算机与单片机进行串口通信时，使用的不是直通线吗，这时候怎么让 TXD 与 RXD 交叉连接？前面我们说了单片机处理的是 TTL 电平，需要使用 RS232 电平转换芯片，将 RS232 电平转换芯片串行数据输出管脚交叉连接在 DB9 母头上即可，本章后面硬件设计部分会介绍。

（2）通信协议

RS232 的通信协议比较简单，通常遵循 96-N-8-1 格式。

“96”表示的是通信波特率为 9600。串口通信中通常使用的是异步串口通信，即没有时钟线，所以两个设备要通信，必须要保持一致的波特率，当然，波

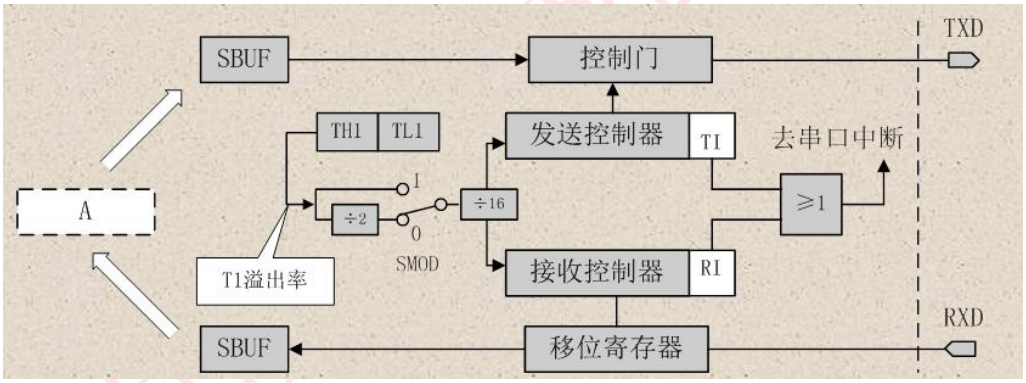
特率常用值还有 4800、 115200 等。

“N”表示的是无校验位，由于串口通信相对更容易受到外部干扰导致传输数据出现偏差，可以在传输过程加上校验位来解决这个问题。校验方法有奇校验(odd)、偶校验(even)、0 校验(space)、1 校验(mark)以及无校验(noparity)。具体的介绍，大家可以百度串口通信了解。

“8”表示的是数据位数为 8 位，其数据格式在前面介绍异步通信中已讲过。当然数据位数还可以为 5、6、7 位长度。

“1”表示的是 1 位停止位，串口通讯的一个数据包从起始信号开始，直到停止信号结束。数据包的起始信号由一个逻辑 0 的数据位表示，而数据包的停止信号可由 0.5、1、1.5 或 2 个逻辑 1 的数据位表示，只要双方约定一致即可。

（3）串口内部结构



上图中右边的 TXD 和 RXD 为单片机 IO 口，TXD 对应的是 P3.1 管脚，RXD 对应的是 P3.0 管脚。其内部工作方式在后面小节会介绍。

22.2.2 串口相关寄存器

（1）串口控制寄存器 SCON

位	7	6	5	4	3	2	1	0	
字节地址：98H	SM0	SM1	SM2	REN	TB8	RB8	TI	RI	SCON

SM0 和 SM1 为工作方式选择位：

串行口的工作方式

SM0	SM1	方　式	说　　明	波特率
0	0	0	移位寄存器	$f_{osc}/12$
0	1	1	10 位异步收发器（8 位数据）	可变
1	0	2	11 位异步收发器（9 位数据）	$f_{osc}/64$ 或 $f_{osc}/32$
1	1	3	11 位异步收发器（9 位数据）	可变

SM2：多机通信控制位，主要用于方式 2 和方式 3。当 SM2=1 时可以利用收到的 RB8 来控制是否激活 RI（RB8＝0 时不激活 RI，收到的信息丢弃；RB8＝1 时收到的数据进入 SBUF，并激活 RI，进而在中断服务中将数据从 SBUF 读走）。当 SM2=0 时，不论收到的 RB8 为 0 和 1，均可以使收到的数据进入 SBUF，并激活 RI（即此时 RB8 不具有控制 RI 激活的功能）。通过控制 SM2，可以实现多机通信。

REN：允许串行接收位。由软件置 REN=1，则启动串行口接收数据；若软件置 REN=0，则禁止接收。

TB8：在方式 2 或方式 3 中，是发送数据的第 9 位，可以用软件规定其作用。可以用作数据的奇偶校验位，或在多机通信中，作为地址帧/数据帧的标志位。在方式 0 和方式 1 中，该位未用到。

RB8：在方式 2 或方式 3 中，是接收到数据的第 9 位，作为奇偶校验位或地址帧/数据帧的标志位。在方式 1 时，若 SM2=0，则 RB8 是接收到的停止位。

TI：发送中断标志位。在方式 0 时，当串行发送第 8 位数据结束时，或在其它方式，串行发送停止位的开始时，由内部硬件使 TI 置 1，向 CPU 发中断申请。在中断服务程序中，必须用软件将其清 0，取消此中断申请。

RI：接收中断标志位。在方式 0 时，当串行接收第 8 位数据结束时，或在其它方式，串行接收停止位的中间时，由内部硬件使 RI 置 1，向 CPU 发中断申请。也必须在中断服务程序中，用软件将其清 0，取消此中断申请。

（2）电源控制寄存器 PCON

位	7	6	5	4	3	2	1	0	
字节地址：97H	SMOD								PCON

SMOD：波特率倍增位。在串口方式 1、方式 2、方式 3 时，波特率与 SMOD 有关，当 SMOD=1 时，波特率提高一倍。复位时，SMOD=0。

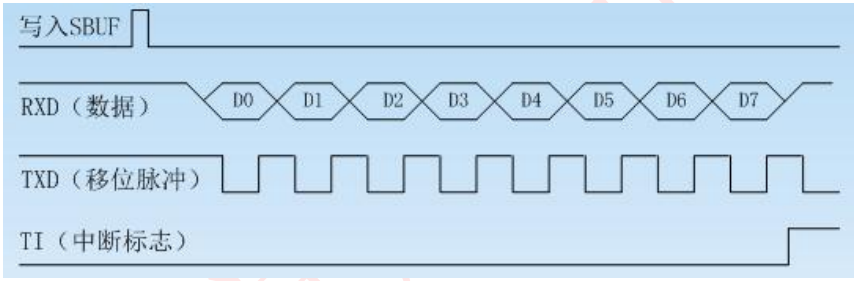
22.2.3 串口工作方式

前面在介绍寄存器时已经知道了 51 单片机串口的几种工作方式，下面分别来对它介绍。

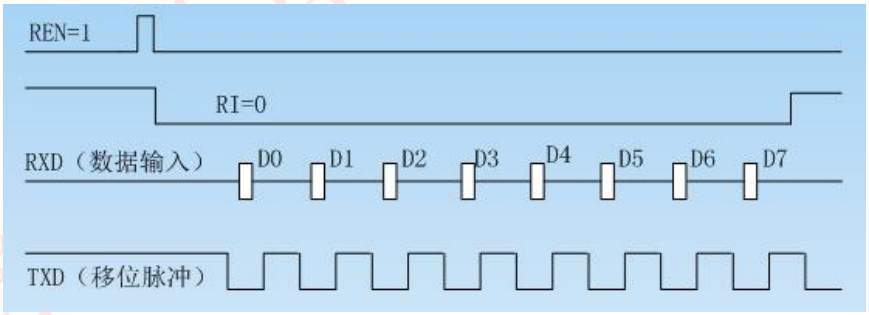
22.2.3.1 方式 0

方式 0 时，串行口为同步移位寄存器的输入输出方式。主要用于扩展并行输入或输出口。数据由 RXD（P3.0）引脚输入或输出，同步移位脉冲由 TXD（P3.1）引脚输出。发送和接收均为 8 位数据，低位在先，高位在后。波特率固定为 fosc/12。对应的输入输出时序图如下所示：

①方式 0 输出

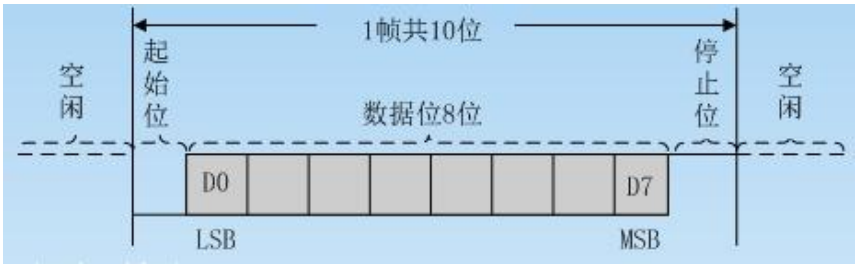


②方式 0 输入



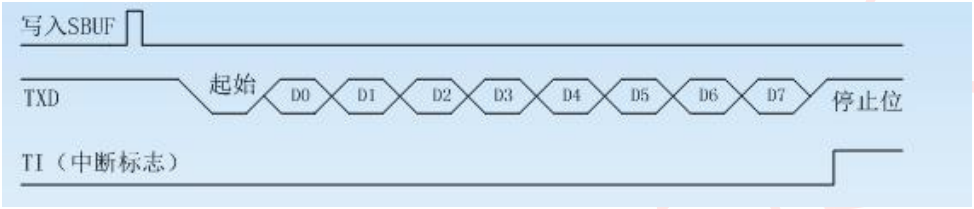
22.2.3.2 方式 1

方式 1 是 10 位数据的异步通信口。TXD 为数据发送引脚，RXD 为数据接收引脚，传送一帧数据的格式如下所示。其中 1 位起始位，8 位数据位，1 位停止位。

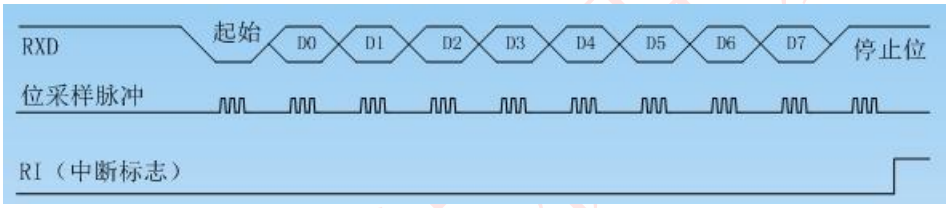


对应的输入输出时序图如下所示：

①方式 1 输出



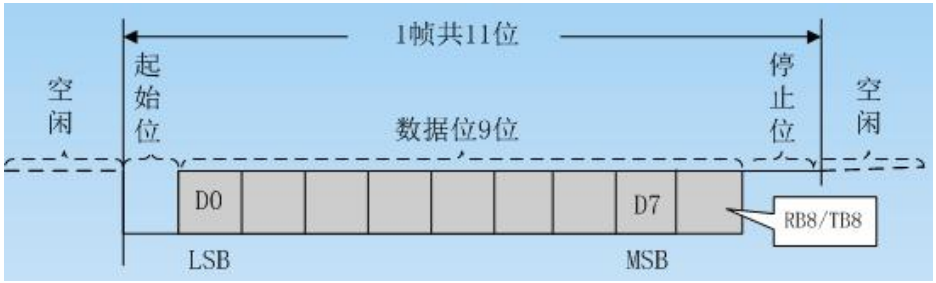
②方式 1 输入



用软件置 REN 为 1 时，接收器以所选择波特率的 16 倍速率采样 RXD 引脚电平，检测到 RXD 引脚输入电平发生负跳变时，则说明起始位有效，将其移入输入移位寄存器，并开始接收这一帧信息的其余位。接收过程中，数据从输入移位寄存器右边移入，起始位移至输入移位寄存器最左边时，控制电路进行最后一次移位。当 RI=0，且 SM2=0（或接收到的停止位为 1）时，将接收到的 9 位数据的前 8 位数据装入接收 SBUF，第 9 位（停止位）进入 RB8，并置 RI=1，向 CPU 请求中断。

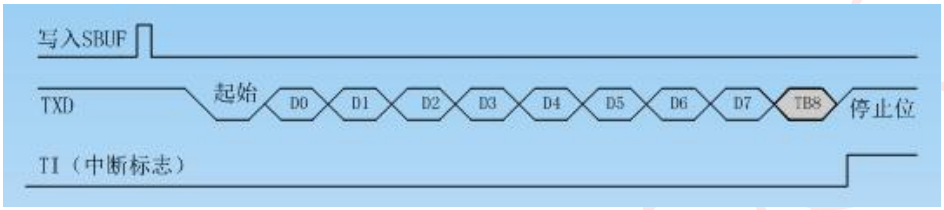
22.2.3.3 方式 2 和方式 3

方式 2 或方式 3 时为 11 位数据的异步通信口。TXD 为数据发送引脚，RXD 为数据接收引脚。其数据格式如下所示：



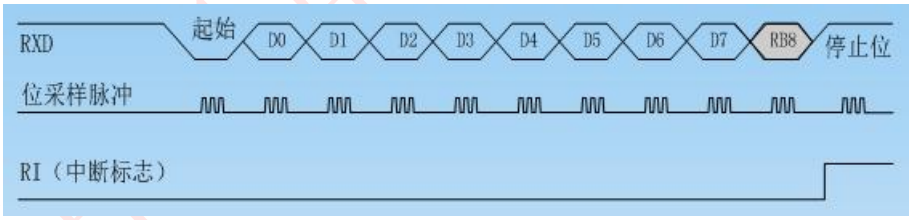
对应的输入输出时序图如下所示：

①方式 2、方式 3 输出



发送开始时，先把起始位 0 输出到 TXD 引脚，然后发送移位寄存器的输出位（D0）到 TXD 引脚。每一个移位脉冲都使输出移位寄存器的各位右移一位，并由 TXD 引脚输出。第一次移位时，停止位“1”移入输出移位寄存器的第 9 位上，以后每次移位，左边都移入 0。当停止位移至输出位时，左边其余位全为 0，检测电路检测到这一条件时，使控制电路进行最后一次移位，并置 TI=1，向 CPU 请求中断。

②方式 2、方式 3 输入



接收时，数据从右边移入输入移位寄存器，在起始位 0 移到最左边时，控制电路进行最后一次移位。当 RI=0，且 SM2=0（或接收到的第 9 位数据为 1）时，接收到的数据装入接收缓冲器 SBUF 和 RB8（接收数据的第 9 位），置 RI=1，向 CPU 请求中断。如果条件不满足，则数据丢失，且不置位 RI，继续搜索 RXD 引脚的负跳变。

22.2.4 串口的使用方法

（1）如何计算波特率

在学习 51 单片机串口时，非常重要的一点是学会如何计算波特率。以下列出了几种方式下波特率的计算公式：

方式 0 的波特率 = fosc/12

方式 2 的波特率 =（2^{SMOD}/64）• fosc

方式 1 的波特率 =（2^{SMOD}/32）•（T1 溢出率）

方式 3 的波特率 =（2^{SMOD}/32）•（T1 溢出率）

其中 T1 溢出率 = fosc /{12×[256 −（TH1）]}，当然还可以使用资料“\5--开发工具\4-常用辅助开发软件\51 波特率初值设定.exe”内附带的小工具自动生成波特率。其使用方法如下：

①双击打开该工具，其界面如下：

②选择定时器工作方式，输入开发板上使用的晶振频率大小，选择所要使用的波特率，SMOD 为是否倍频，这个在前面介绍寄存器时说过，下面的误差大小可以反映出通信时是否出现乱码。在使用串口通信时，定时器 1 工作方式为 2，串口工作方式为 1，以开发板晶振是 11.0592Mh 为例，假如晶振频率是 12M，那么在生成的波特率就会有误差而导致通信出错。为什么替换可以从误差值反映出来。在本章实验中波特率选择 9600，使用 SMOD，即值为 1，点击确定后即会自动生成定时/计数器 THx 的值，如下所示：

从上图可知，使用 11.0592M 晶振时，误差为 0。我们对比下当外部晶振使用 12Mh 时，波特率误差多大，如下所示：

从上图可知，当使用 12M 晶振时，波特率误差有 6.98%，是比较大的，会导致在通信过程中出现乱码等错误信息。这是我们不希望看到的，所以再次说明下，**在做串口通信实验时，一定要确认外部晶振是否是 11.0592M。**

对于波特率的计算方法大家要知道，开发中建议大家使用小工具，它会给你带来很多便利。

（2）串口初始化步骤

如何使用串口，大家可以按照以下几个步骤配置。

①确定 T1 的工作方式（TMOD 寄存器）；

②确定串口工作方式（SCON 寄存器）；

③计算 T1 的初值（设定波特率），装载 TH1、TL1；

④启动 T1（TCON 中的 TR1 位）；

⑤如果使用中断，需开启串口中断控制位（IE 寄存器）。

例如：设置串口为工作方式 1、波特率为 9600、波特率加倍、使用中断。其配置程序如下：

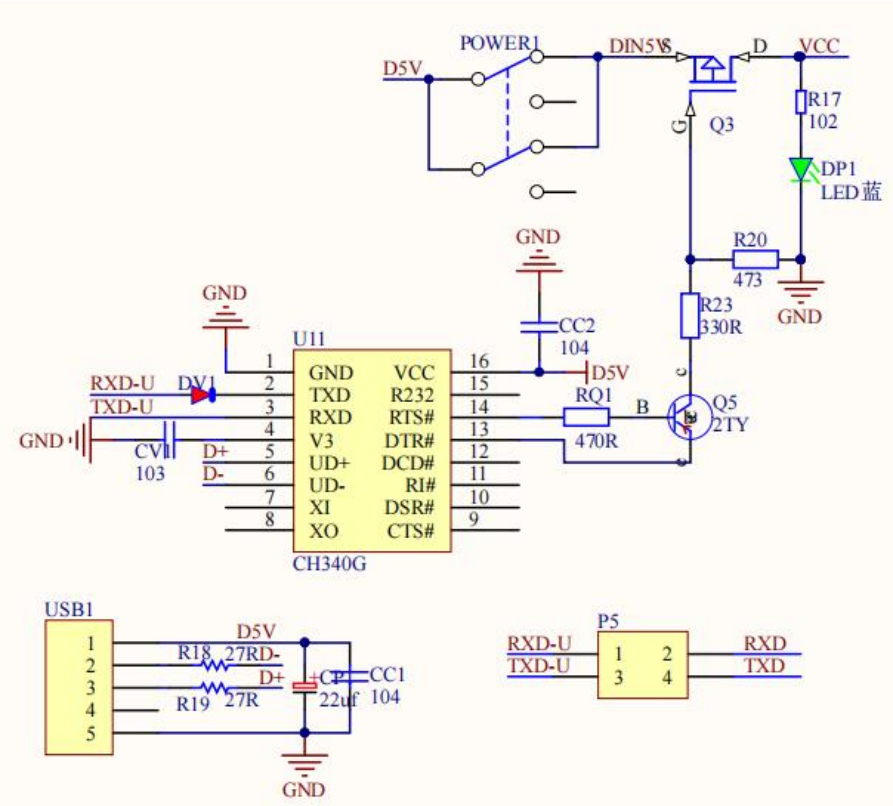
```
/*******************************************************************************
* 函 数 名         : uart_init
* 函数功能         : 串口通信中断配置函数，通过设置 TH 和 TL 即可确定定时时间
* 输    入         : baud：波特率对应的 TH、TL 装载值
* 输    出         : 无
*******************************************************************************/
void uart_init(u8 baud)
{
    TMOD|=0X20;  //设置计数器工作方式 2
    SCON=0X50;   //设置为工作方式 1
    PCON=0X80;   //波特率加倍
    TH1=baud;    //计数器初始值设置
    TL1=baud;
    ES=1;        //打开接收中断
    EA=1;        //打开总中断
    TR1=1;       //打开计数器
}
```

在主函数中调用该函数并传入 0XFA 值即可，如下：

```
    uart_init(0XFA);//波特率为 9600
```

22.3 硬件设计

开发板上板载一个 USB 转串口模块。其硬件电路如下所示：



从上图中可以看出，通过 CH340 芯片把 51 单片机的串口与 PC 机的 USB 口进行连接，不仅可以实现程序的烧入，还可实现串口通信功能。根据前面介绍，串口通信需将数据收发管脚交叉连接，所以可以看到在 CH340 芯片的 2 和 3 脚已做处理。电路中其他部分是自动下载电路部分，目的是控制单片机的电源，无需冷启动。使用 USB 转串口芯片，免去了一根串口线，使用普通 USB 数据线（支持安卓手机数据线）就可以进行串口通信。

从上图中可以看到 CH340 的 2、3 脚串口并非直接连接到单片机串口，而是连接在 P5 端子上，这样就把 CH340 的串口与单片机串口独立出来，为什么不直接连接而要使用这个 P5 端子呢？这是方便用户可以使用开发板上的 USB 转 TTL 模块（也就是 CH340 转串口模块）做一些串口类模块的调试，比如：WIFI、蓝牙、GPS、GPRS 等，直接利用 PC 上位机来调试模块。同时也方便用户使用板载 USB 转 TTL 模块给其它类型单片机下载程序。

如果使用黄色跳线帽将 P5 端子的 1、2 短接，3、4 短接，那么 CH340 串口与单片机串口是连接一起的，此时即可实现程序的下载或串口通信。

22.4 软件设计

本章所要实现的功能是：当串口助手发送数据给单片机，单片机原封不动转发给串口助手显示。

我们打开“\4--实验程序\1--基础实验\18-串口通信实验”工程，控制代码全部都在 main.c 中，代码如下：

```
/*********************************************************************************************
实验名称：串口通信实验
接线说明：
实验现象：下载程序后，当串口助手发送数据给单片机，单片机原封不动转发给串口助手显示
注意事项：使用黄色跳线帽将 CH340 旁的 P5 端子的 UTX 和 P30 短接，URX 和 P31 短接，出厂默认已短接好

**********************************************************************************************/
#include "reg52.h"

typedef unsigned int u16;//对系统默认数据类型进行重定义
typedef unsigned char u8;


/*********************************************************************************
* 函 数 名       : uart_init
* 函数功能       : 串口通信中断配置函数，通过设置 TH 和 TL 即可确定定时时间
* 输    入       : baud：波特率对应的 TH、TL 装载值
* 输    出       : 无
*********************************************************************************/
void uart_init(u8 baud)
{
    TMOD|=0X20;  //设置计数器工作方式 2
    SCON=0X50;   //设置为工作方式 1
    PCON=0X80;   //波特率加倍
    TH1=baud;    //计数器初始值设置
    TL1=baud;
    ES=1;        //打开接收中断
    EA=1;        //打开总中断
    TR1=1;       //打开计数器
}

/*********************************************************************************
```

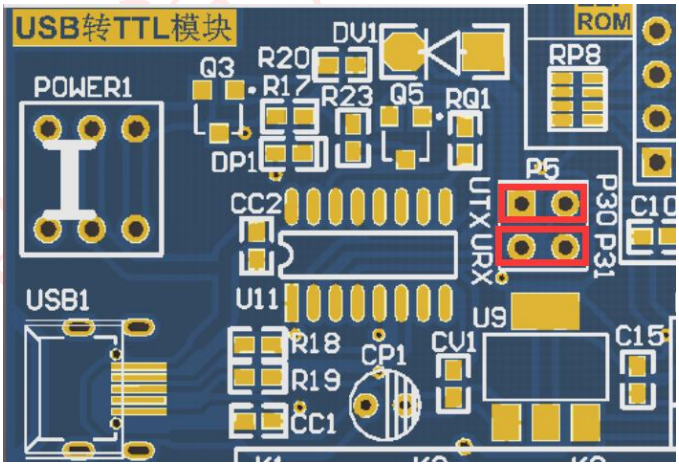
```
* 函 数 名         : main
* 函数功能         : 主函数
* 输    入         : 无
* 输    出         : 无
*******************************************************************************/
void main()
{
    uart_init(0XFA);//波特率为 9600

    while(1)
    {

    }
}

void uart() interrupt 4 //串口通信中断函数
{
    u8 rec_data;

    RI = 0;          //清除接收中断标志位
    rec_data=SBUF;   //存储接收到的数据
    SBUF=rec_data;   //将接收到的数据放入到发送寄存器
    while(!TI);      //等待发送数据完成
    TI=0;            //清除发送完成标志位
}
```

实验代码比较简单，首先定义了串口通信中断配置函数 uart_init，该函数有一个入口参数 baud，该值可改变通信波特率。该函数的实现即是按照前面介绍的串口配置步骤。最后进入 while 循环，在循环体内不执行任何功能程序。如果发生接收中断，即会进入串口中断执行，执行完后回到主函数内继续运行，如此循环。

22.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：当串口助手发送数据给单片机，单片机原封不动转发给串口助手显示。

注意：使用黄色跳线帽将 CH340 旁的 P5 端子的 UTX 和 P30 短接，URX 和 P31 短接，出厂默认已短接好。如下所示：



课后作业

第 23 章 I2C-EEPROM 实验

这一章我们来学习如何使用 51 单片机的 IO 口模拟 I2C 时序，并实现与 AT24C02（EEPROM）之间的双向通信。开发板板载了 1 个 EEPROM 模块，可实现 IIC 通信。本章要实现的功能是：系统运行时，数码管右 3 位显示 0，按 K1 键将数据写入到 EEPROM 内保存，按 K2 键读取 EEPROM 内保存的数据，按 K3 键显示数据加 1，按 K4 键显示数据清零，最大能写入的数据是 255。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

23.1 I2C 介绍

23.2 AT24C02 介绍

23.3 硬件设计

23.4 软件设计

23.5 实验现象

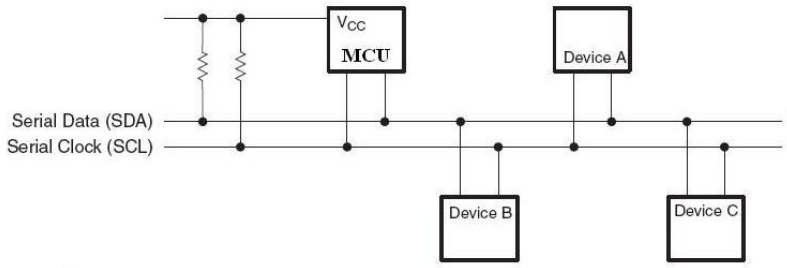
普中51单片机开发板

23.1 I2C 介绍

I2C（Inter－Integrated Circuit）总线是由 PHILIPS 公司开发的两线式串行总线，用于连接微控制器及其外围设备。是微电子通信控制领域广泛采用的一种总线标准。它是同步通信的一种特殊形式，具有接口线少，控制方式简单，器件封装形式小，通信速率较高等优点。I2C 总线只有两根双向信号线。一根是数据线 SDA，另一根是时钟线 SCL。由于其管脚少，硬件实现简单，可扩展性强等特点，因此被广泛的使用在各大集成芯片内。下面我们就从 I2C 的物理层与协议层来了解 I2C。

23.1.1 I2C 物理层

I2C 通信设备常用的连接方式如下图所示：



它的物理层有如下特点：

（1）它是一个支持多设备的总线。“总线”指多个设备共用的信号线。在一个 I2C 通讯总线中，可连接多个 I2C 通讯设备，支持多个通讯主机及多个通讯从机。

（2）一个 I2C 总线只使用两条总线线路，一条双向串行数据线(SDA)，一条串行时钟线(SCL)。数据线即用来表示数据，时钟线用于数据收发同步。

（3）每个连接到总线的设备都有一个独立的地址，主机可以利用这个地址进行不同设备之间的访问。

（4）总线通过上拉电阻接到电源。当 I2C 设备空闲时，会输出高阻态，而当所有设备都空闲，都输出高阻态时，由上拉电阻把总线拉成高电平。

（5）多个主机同时使用总线时，为了防止数据冲突，会利用仲裁方式决定由哪个设备占用总线。

（6）具有三种传输模式：标准模式传输速率为 100kbit/s，快速模式为 400kbit/s，高速模式下可达 3.4Mbit/s，但目前大多 I2C 设备尚不支持高速模式。

（7）连接到相同总线的 IC 数量受到总线的最大电容 400pF 限制。

下面我们来了解下 I2C 总线常用的一些术语：

主机：启动数据传送并产生时钟信号的设备；

从机：被主机寻址的器件；

多主机：同时有多于一个主机尝试控制总线但不破坏传输；

主模式：用 I2CNDAT 支持自动字节计数的模式； 位 I2CRM,I2CSTT,I2CSTP 控制数据的接收和发送；

从模式：发送和接收操作都是由 I2C 模块自动控制的；

仲裁：是一个在有多个主机同时尝试控制总线但只允许其中一个控制总线并使传输不被破坏的过程；

同步：两个或多个器件同步时钟信号的过程；

发送器：发送数据到总线的器件；

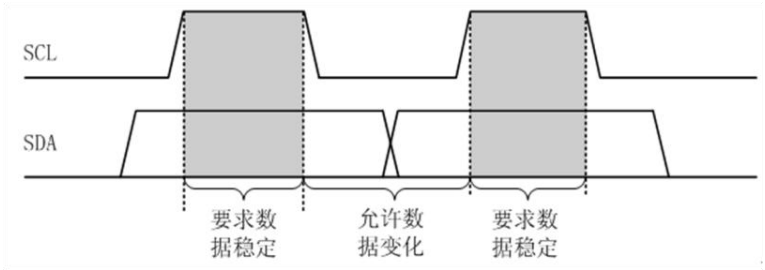
接收器：从总线接收数据的器件。

23.1.2 I2C 协议层

I2C 的协议定义了通信的起始和停止信号、数据有效性、响应、仲裁、时钟同步和地址广播等环节。下面我们就来简单介绍下。

（1）数据有效性规定

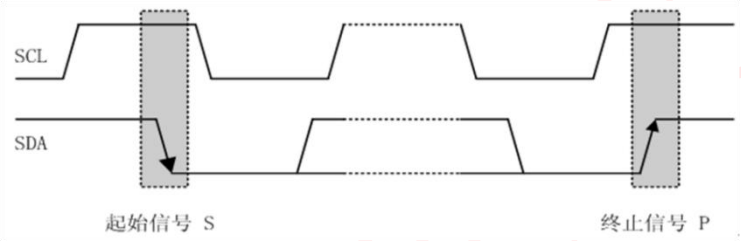
I2C 总线进行数据传送时，时钟信号为高电平期间，数据线上的数据必须保持稳定，只有在时钟线上的信号为低电平期间，数据线上的高电平或低电平状态才允许变化。如下图：



每次数据传输都以字节为单位，每次传输的字节数不受限制。

（2）起始和停止信号

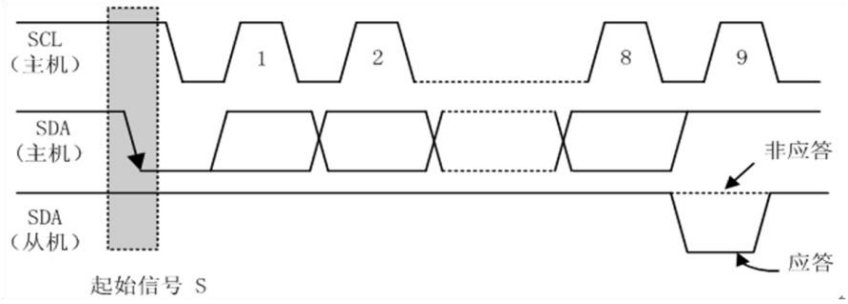
SCL 线为高电平期间，SDA 线由高电平向低电平的变化表示起始信号；SCL 线为高电平期间，SDA 线由低电平向高电平的变化表示终止信号。如下图：



起始和终止信号都是由主机发出的，在起始信号产生后，总线就处于被占用的状态；在终止信号产生后，总线就处于空闲状态。

（3）应答响应

每当发送器件传输完一个字节的数据后，后面必须紧跟一个校验位，这个校验位是接收端通过控制 SDA（数据线）来实现的，以提醒发送端数据我这边已经接收完成，数据传送可以继续进行。这个校验位其实就是数据或地址传输过程中的响应。响应包括“应答(ACK)”和“非应答(NACK)”两种信号。作为数据接收端时，当设备(无论主从机)接收到 I2C 传输的一个字节数据或地址后，若希望对方继续发送数据，则需要向对方发送“应答(ACK)”信号即特定的低电平脉冲，发送方会继续发送下一个数据；若接收端希望结束数据传输，则向对方发送“非应答(NACK)”信号即特定的高电平脉冲，发送方接收到该信号后会产生一个停止信号，结束信号传输。应答响应时序图如下：



每一个字节必须保证是 8 位长度。**数据传送时，先传送最高位（MSB），每一个被传送的字节后面都必须跟随一位应答位（即一帧共有 9 位）。**

由于某种原因从机不对主机寻址信号应答时（如从机正在进行实时性的处理工作而无法接收总线上的数据），它必须将数据线置于高电平，而由主机产生一个终止信号以结束总线的数据传送。

如果从机对主机进行了应答，但在数据传送一段时间后无法继续接收更多的数据时，从机可以通过对无法接收的第一个数据字节的“非应答”通知主机，主机则应发出终止信号以结束数据的继续传送。

当主机接收数据时，它收到最后一个数据字节后，必须向从机发出一个结束传送的信号。这个信号是由对从机的“非应答”来实现的。然后，从机释放 SDA 线，以允许主机产生终止信号。

这些信号中，起始信号是必需的，结束信号和应答信号都可以不要。

（4）总线的寻址方式

I2C 总线寻址按照从机地址位数可分为两种，一种是 7 位，另一种是 10 位。采用 7 位的寻址字节（寻址字节是起始信号后的第一个字节）的位定义如下：

位：	7	6	5	4	3	2	1	0
	从机地址							R/$\overline{W}$

D7～D1 位组成从机的地址。D0 位是数据传送方向位，为“ 0”时表示主机向从机写数据，为“1”时表示主机由从机读数据。

10 位寻址和 7 位寻址兼容，而且可以结合使用。10 位寻址不会影响已有的 7 位寻址，有 7 位和 10 位地址的器件可以连接到相同的 I2C 总线。我们就以 7 位寻址为例进行介绍。

当主机发送了一个地址后，总线上的每个器件都将头 7 位与它自己的地址比较，如果一样，器件会判定它被主机寻址，其他地址不同的器件将被忽略后面的数据信号。至于是从机接收器还是从机发送器，都由 R/W 位决定的。从机的地址由固定部分和可编程部分组成。在一个系统中可能希望接入多个相同的从机，从机地址中可编程部分决定了可接入总线该类器件的最大数目。如一个从机的 7 位寻址位有 4 位是固定位，3 位是可编程位，这时仅能寻址 8 个同样的器件，即可以有 8 个同样的器件接入到该 I2C 总线系统中。

（5）数据传输

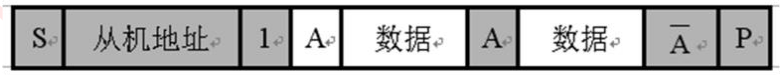
I2C 总线上传送的数据信号是广义的，既包括地址信号，又包括真正的数据信号。在起始信号后必须传送一个从机的地址（7 位），第 8 位是数据的传送方向位（R/W），用“ 0”表示主机发送（写）数据（W），“ 1”表示主机接收数据（R）。每次数据传送总是由主机产生的终止信号结束。但是，若主机希望继续占用总线进行新的数据传送，则可以不产生终止信号，马上再次发出起始信号对另一从机进行寻址。

在总线的一次数据传送过程中，可以有以下几种组合方式：

a、主机向从机发送数据，数据传送方向在整个传送过程中不变

注意：有阴影部分表示数据由主机向从机传送，无阴影部分则表示数据由从机向主机传送。A 表示应答，A 非表示非应答（高电平）。S 表示起始信号，P 表示终止信号。

b、主机在第一个字节后，立即从从机读数据



c、在传送过程中，当需要改变传送方向时，起始信号和从机地址都被重复产生一次，但两次读/写方向位正好相反

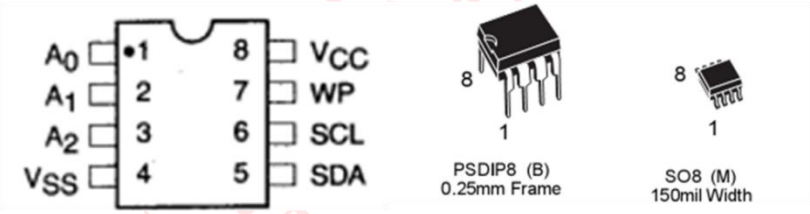
到这里我们就介绍完 I2C 总线，由于 51 单片机没有硬件 IIC 接口，即使有硬件接口我们通常还是采用软件模拟 I2C。主要原因是硬件 IIC 设计的比较复

杂，而且稳定性不怎么好，程序移植比较麻烦，而用软件模拟 IIC，最大的好处就是移植方便，同一个代码兼容所有单片机，任何一个单片机只要有 IO 口（不需要特定 IO），都可以很快的移植过去。

下面我们再来了解下开发板上的 AT24C02 芯片。

23.2 AT24C02 介绍

AT24C01/02/04/08/16...是一个 1K/2K/4K/8K/16K 位串行 CMOS，内部含有 128/256/512/1024/2048 个 8 位字节，AT24C01 有一个 8 字节页写缓冲器，AT24C02/04/08/16 有一个 16 字节页写缓冲器。该器件通过 I2C 总线接口进行操作，它有一个专门的写保护功能。我们开发板上使用的是 AT24C02（EEPROM）芯片，此芯片具有 I2C 通信接口，芯片内保存的数据在掉电情况下都不丢失，所以通常用于存放一些比较重要的数据等。AT24C02 芯片管脚及外观图如下图所示：



芯片管脚说明如下图所示：

引脚号	引脚名称	功能说明
1	A0	地址输入。A2、A1和A0是器件地址输入引脚。 24C02/32/64使用A2、A1和A0输入引脚作为硬件地址，总线上可同时级联8个24C02/32/64器件（详见器件寻址）。 24C04使用A2和A1输入引脚作为硬件地址，总线上可同时级联4个24C04器件，A0为空脚，可接地。 24C08使用A2输入引脚作为硬件地址，总线上可同时级联2个24C08器件，A0和A1为空脚，可接地。 24C16未使用器件地址引脚，总线上最多只可连接一个16K器件，A2、A1和A0为空脚，可接地。
2	A1	
3	A2	
5	SDA	串行地址和数据输入/输出。SDA是双向串行数据传输引脚，漏极开路，需外接上拉电阻到Vcc（典型值10kΩ）。
6	SCL	串行时钟输入。SCL同步数据传输，上升沿数据写入，下降沿数据读出。
7	WP	写保护。WP引脚提供硬件数据保护。当WP接地时，允许数据正常读写操作；当WP接Vcc时，写保护，只读。
4	GND	地
8	Vcc	正电源

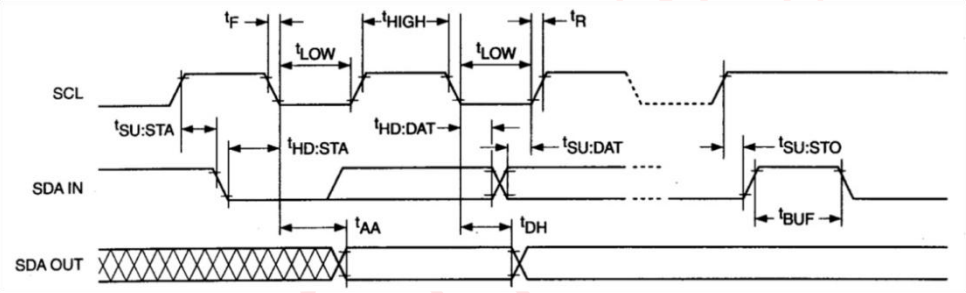
AT24C02 器件地址为 7 位，高 4 位固定为 1010，低 3 位由 A0/A1/A2 信

号线的电平决定。 因为传输地址或数据是以字节为单位传送的，当传送地址时，器件地址占 7 位，还有最后一位（最低位 R/W）用来选择读写方向，它与地址无关。其格式如下：

24WC01/02	1	0	1	0	A2	A1	A0	R/$\overline{W}$

我们开发板已经将芯片的 A0/A1/A2 连接到 GND，所以器件地址为 1010000，即 0x50（未计算最低位）。如果要对芯片进行写操作时，R/W 即为 0，写器件地址即为 0XA0；如果要对芯片进行读操作时，R/W 即为 1，此时读器件地址为 0XA1。开发板上也将 WP 引脚直接接在 GND 上，此时芯片允许数据正常读写。

I2C 总线时序如下图所示：



读写周期范围

符号	参数	1.8 V，2.5 V		4.5V～5.5V		单位
		最小	最大	最小	最大	
F_{SCL}	时钟频率		100		400	KHz
T_I	SCL,SDA 输入的噪声抑制时间		200		200	ns
t_{AA}	SCL 变低至 SDA 数据输出及应答信号		3.5		1	μs
t_{BUF}	新的发送开始前总线空闲时间	4.7		1.2		μs
$t_{HD:STA}$	起始信号保持时间	4		0.6		μs
t_{LOW}	时钟低电平周期	4.7		1.2		μs
t_{HIGH}	时钟高电平周期	4		0.6		μs
$t_{SU:STA}$	起始信号建立时间	4.7		0.6		μs
$t_{HD:DAT}$	数据输入保持时间	0		0		ns
$t_{SUI:DAT}$	数据输入建立时间	50		50		ns
t_R	SDA 及 SCL 上升时间		1		0.3	μs
t_F	SDA 及 SCL 下降时间		300		300	ns
$t_{SU:STO}$	停止信号建立时间	4		0.6		μs
t_{DH}	数据输出保持时间	100		100		ns

关于 AT24C02（EEPROM）的更多信息，可参考“\6--芯片资料\开发板芯片数据手册”内 24C02 数据手册来了解。

23.3 硬件设计

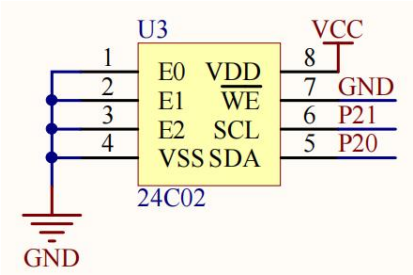
本实验使用到硬件资源如下：

（1）独立按键（K1-K4）

（2）动态数码管

（3）EEPROM 模块电路（AT24C02）

独立按键和动态数码管电路在前面章节都介绍过，这里就不再重复。下面我们来看下开发板上 EEPROM 模块电路，如下图所示：



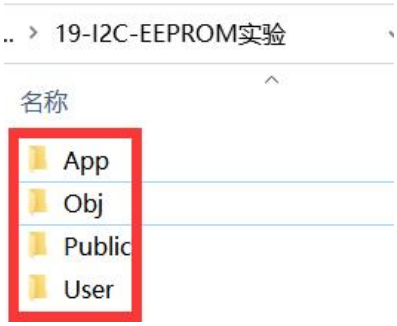
从图中可以看出，芯片的 SCL 和 SDA 管脚是连接在单片机的 P2.1 和 P2.0 上，在介绍 IIC 总线的时候我们说过，为了让 IIC 总线默认为高电平，通常会在 IIC 总线上接上拉电阻，在图中并没有看到 SCL 和 SDA 管脚有上拉电阻，这是因为开发板单片机 IO 都外接了 10K 上拉电阻，当单片机 IO 口连接到芯片的 SCL 和 SDA 脚时即相当于它们外接上拉电阻，所以此处可以省去。

23.4 软件设计

23.4.1 创建多文件工程

本章带领大家创建一个全新的工程，目的是方便日后工程管理、程序移植和维护、易读等。下面开始：

在电脑上创建一个实验文件夹，为了与教程配套，这里命名为“I2C-EEPROM 实验”，然后在该文件夹内新建 App、Obj、Public、User 四个文件夹，如下所示：



App 文件夹：用于存放外设驱动文件，如 LED、数码管、定时器等。

Obj 文件夹：用于存放编译产生的 c/汇编/链接的列表清单、调试信息、hex 文件、预览信息、封装库等文件。

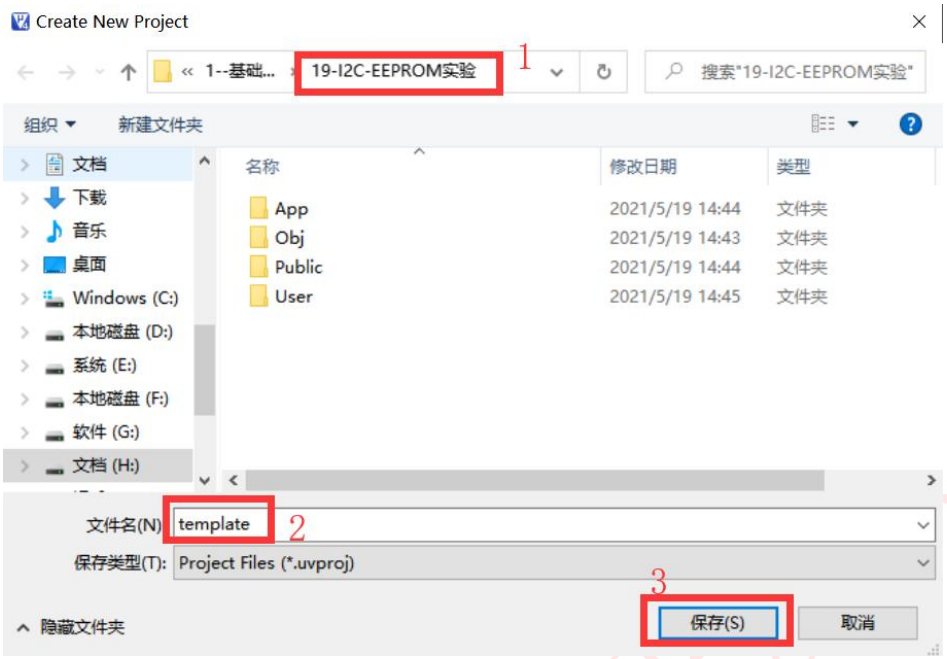
Public 文件夹：用于存放 51 单片机公共的文件，如延时、51 头文件、变量类型重定义等。

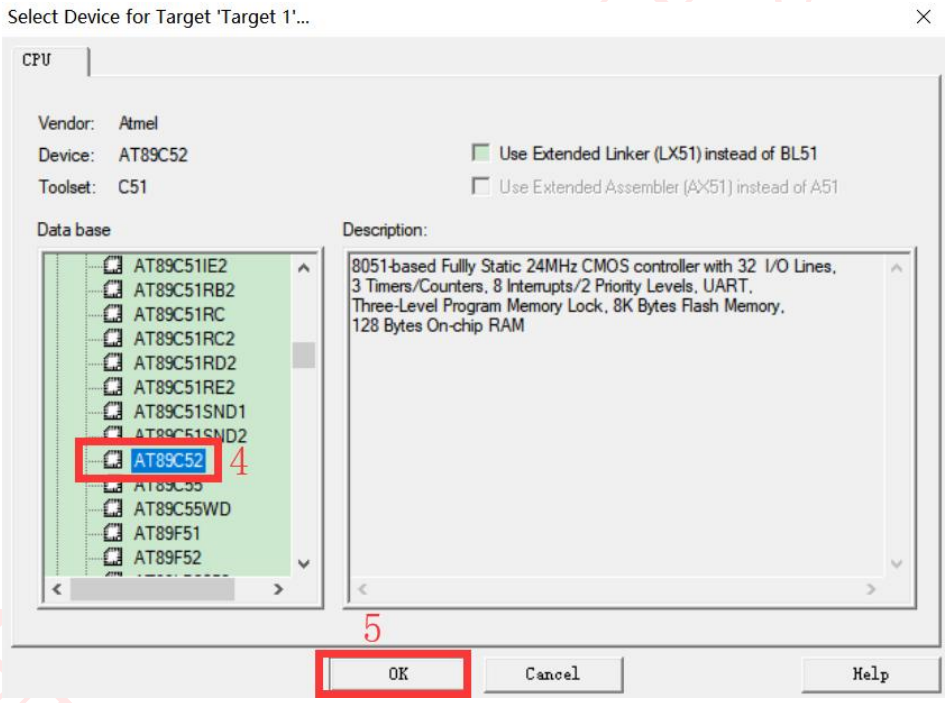
User 文件夹：用于存放用户主函数文件，如 main.c。

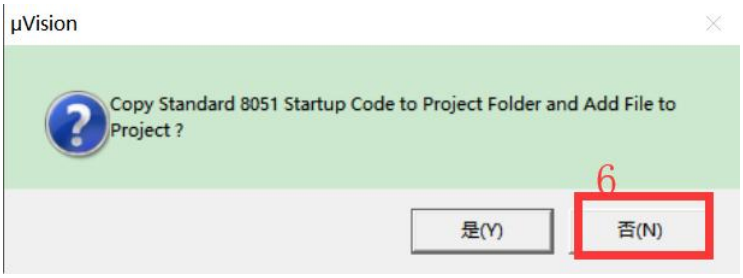
为何要定义这么多文件夹呢？为何不按照之前实验那样直接在 main.c 文件里面写代码？我们先来分析本章实验所需用到哪些功能程序，要用到独立按键、数码管、EEPROM，所以需要将前面实验编写的独立按键、数码管、延时等程序全部放到 main.c 中来，此时在不增加 EEPROM 驱动程序时，main.c 文件中的代码量已经非常冗长，极其不便于程序的阅读、移植和维护，因此可用多文件管理的办法来使工程易于阅读、移植和维护。操作方法如下：

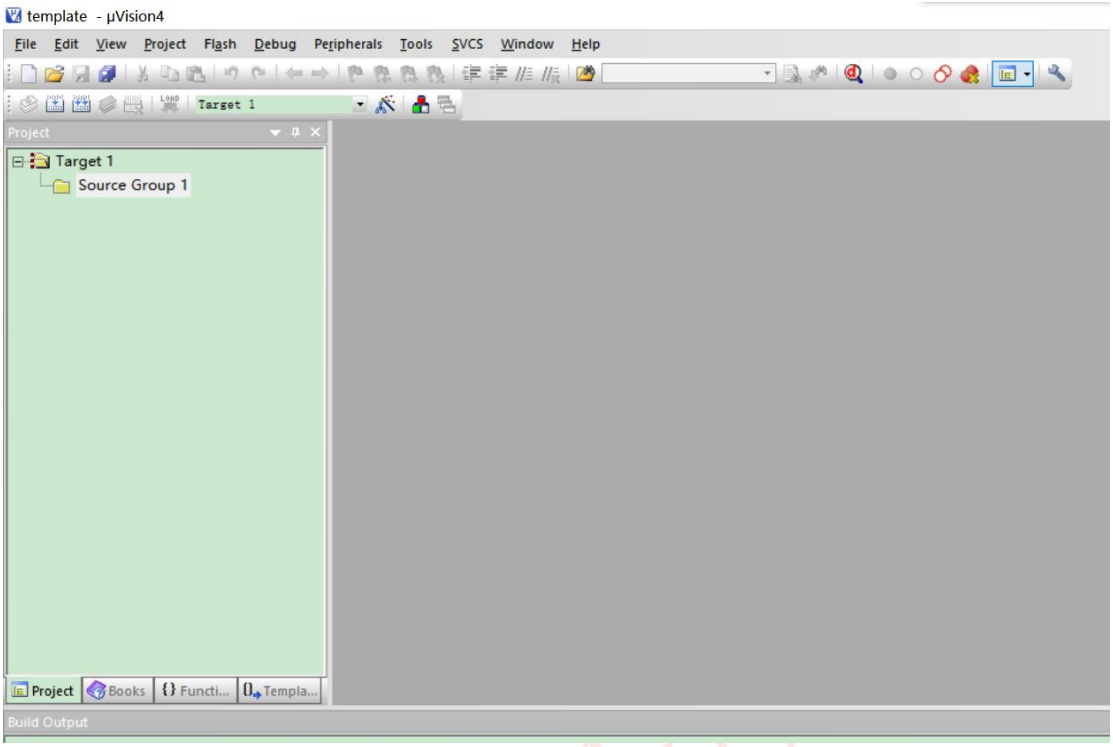
23.4.1.1 新建工程

首先打开 KEILC51 软件，新建一个工程，这个在前面章节已经讲解过，这里不再重复。将工程命名为 template 并保存在“I2C-EEPROM 实验”文件夹下，然后选择芯片类型为“AT89C52”，不使用系统创建启动文件，如下所示：


Create New Project
1--基础...
19-I2C-EEPROM实验
1
搜索"19-I2C-EEPROM实验"
组织
新建文件夹
文档
下载
音乐
桌面
Windows (C:)
本地磁盘 (D:)
系统 (E:)
本地磁盘 (F:)
软件 (G:)
文档 (H:)
名称
修改日期
类型
App 2021/5/19 14:44 文件夹
Obj 2021/5/19 14:43 文件夹
Public 2021/5/19 14:44 文件夹
User 2021/5/19 14:45 文件夹
文件名(N): template
2
保存类型(T): Project Files (*.uvproj)
3
隐藏文件夹
保存(S)
取消


Select Device for Target 'Target 1'...
CPU
Vendor: Atmel
Device: AT89C52
Toolset: C51
Use Extended Linker (LX51) instead of BL51
Use Extended Assembler (AX51) instead of A51
Data base
Description:
AT89C51IE2
AT89C51RB2
AT89C51RC
AT89C51RC2
AT89C51RD2
AT89C51RE2
AT89C51SND1
AT89C52
4
AT89C55WD
AT89F51
AT89F52
8051-based Fully Static 24MHz CMOS controller with 32 I/O Lines,
3 Timers/Counters, 8 Interrupts/2 Priority Levels, UART,
Three-Level Program Memory Lock, 8K Bytes Flash Memory,
128 Bytes On-chip RAM
5
OK
Cancel
Help

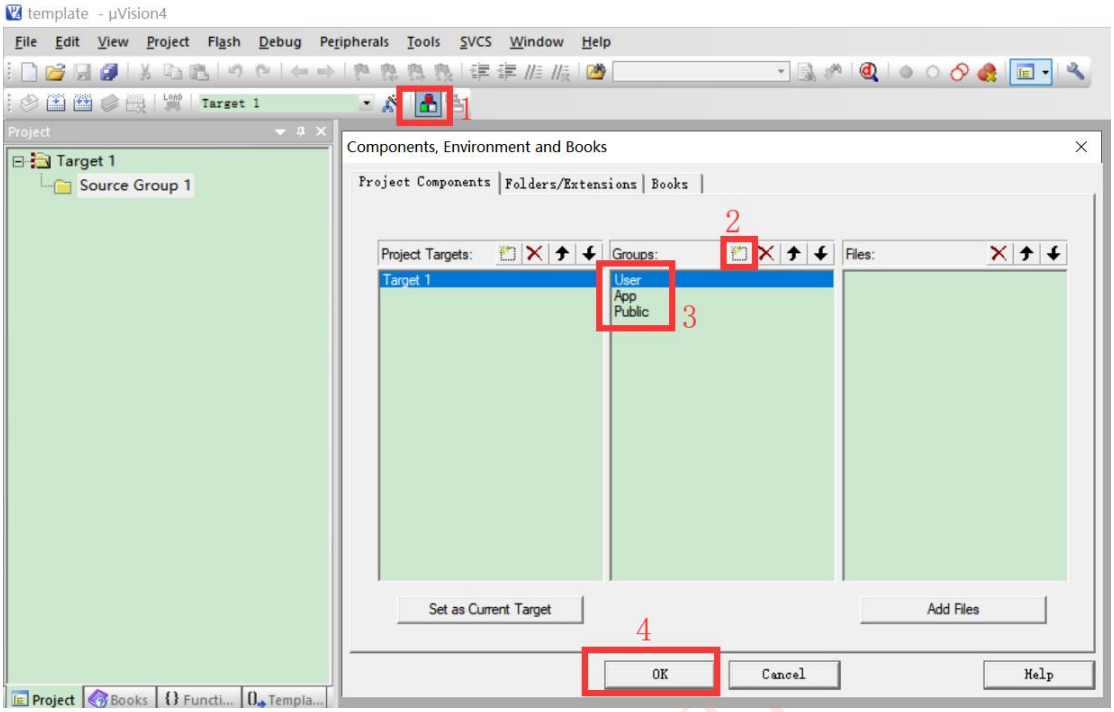

µVision
Copy Standard 8051 Startup Code to Project Folder and Add File to Project ?
6
是(Y)
否(N)



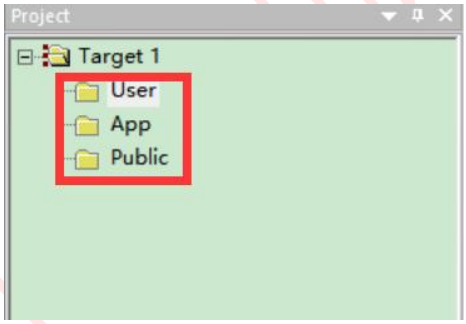
上述操作与前面介绍的创建工程模板是一样的，下面才开始在工程中进行分组管理。

23.4.1.2 向工程添加文件

按照需要给工程分组并添加对应文件，这里我们在工程中分 3 组，User、App、Publi，至于前面创建的 Obj 文件夹是在工程中无需体现，因为只是编译器生成的一些中间文件和.hex 执行文件。通常在工程组的命名与创建的文件夹名保持一致，方便查找到源文件位置。如下所示：



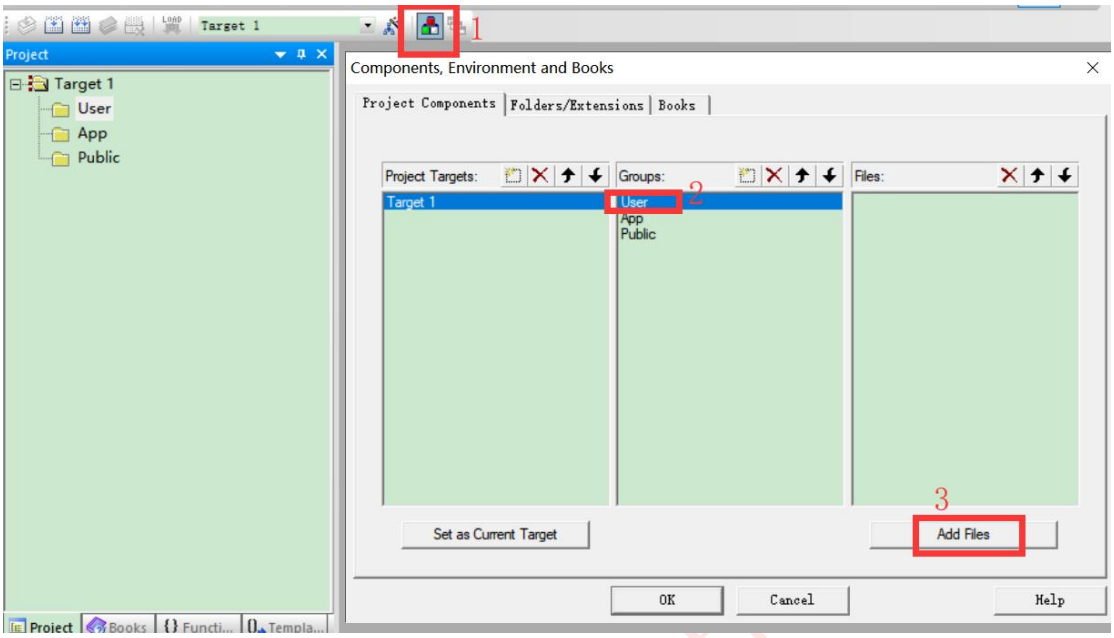
分组后，在工程中就会出现刚才的分组列表，如下所示：

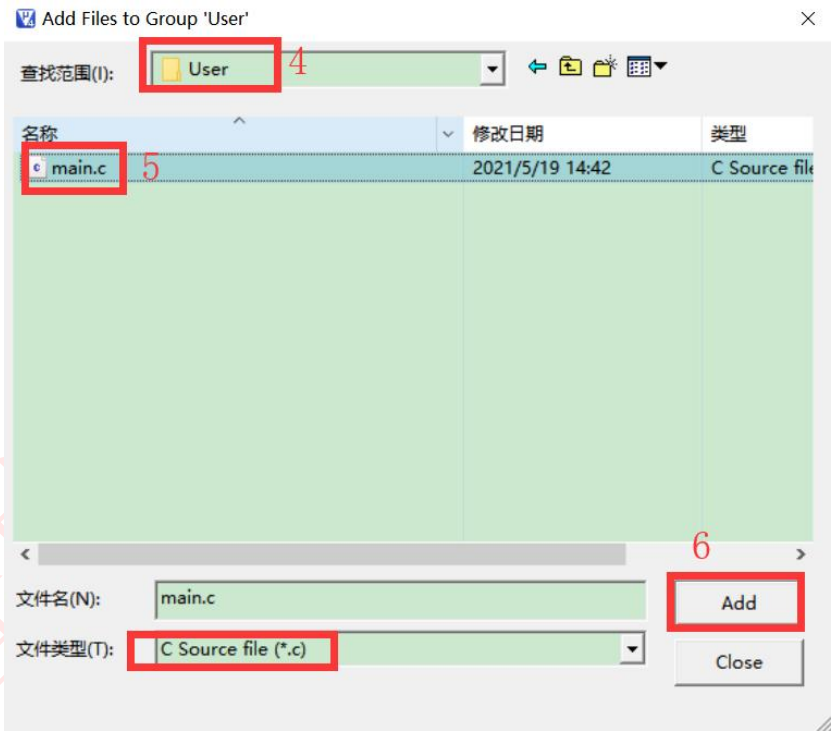


然后就是给每个组添加对应的.c 源文件，这些源文件和头文件在实验例程中已经给出，大家可直接从那复制到新建的文件夹内，复制后如下所示：

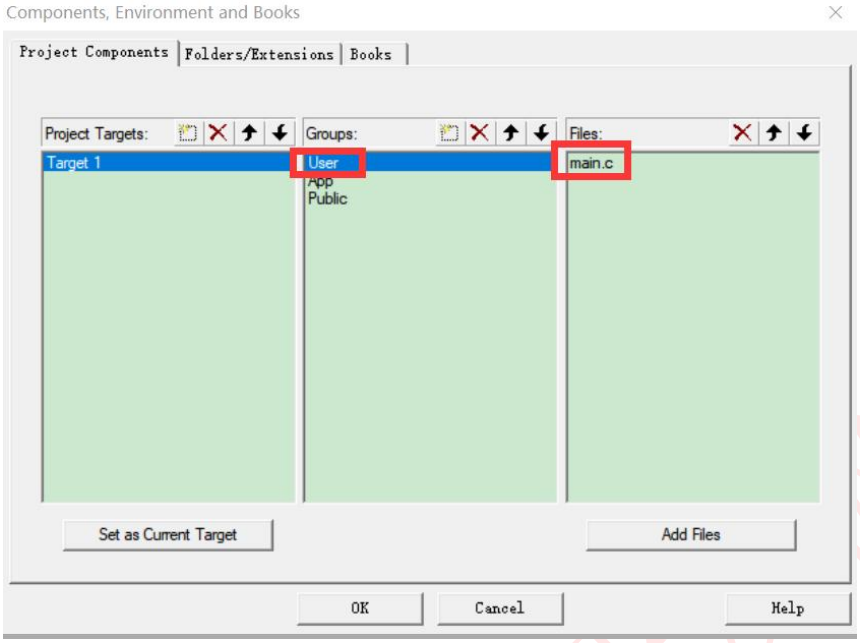
在 App 文件夹中可以看到根据不同外设分别创建了文件夹保存各自驱动文件，这样非常方便程序移植和维护。

然后就将复制过来的文件添加到刚才的工程分组中，如下所示：

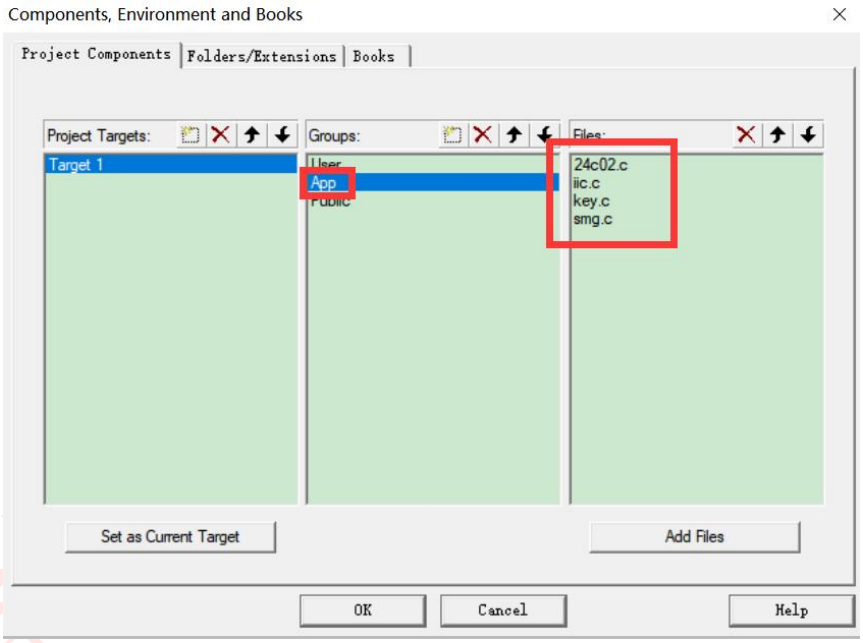


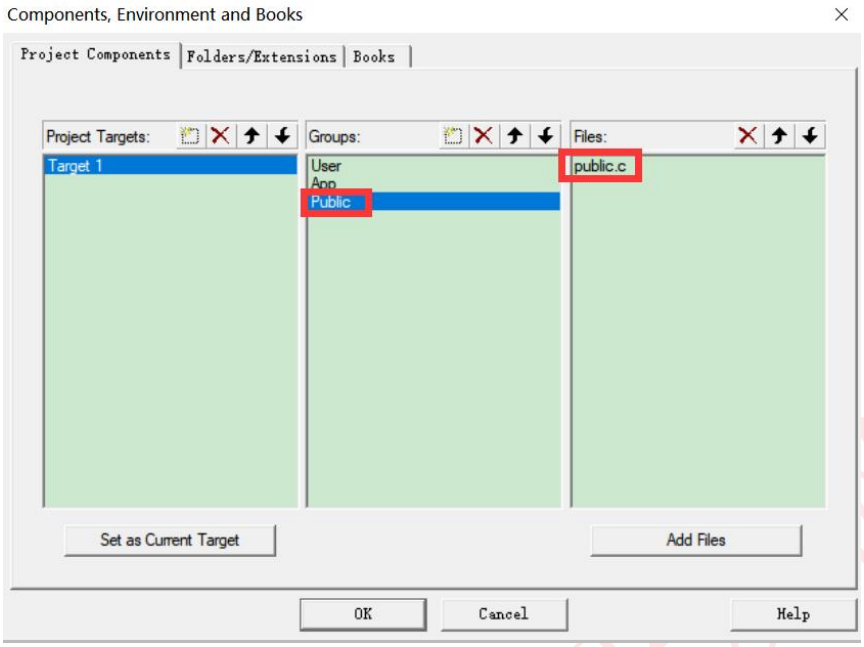


可以选择到要添加的.c 文件（红色标记 5），然后鼠标左键双击也可直接添加进去，就免去了点击红色标记 6 这步。添加好后，在右侧就会显示对应组中已添加的文件，如下所示：

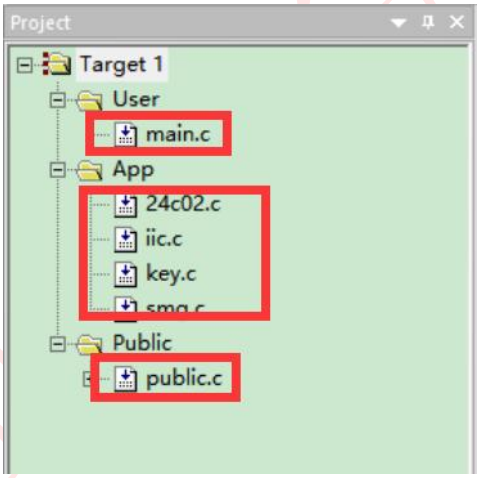


同样的方法，将 App、Public 工程组中文件也添加进去。如下所示：





此时可以看到，在工程栏下每组都已经加入了刚才添加的源文件，如下所示：



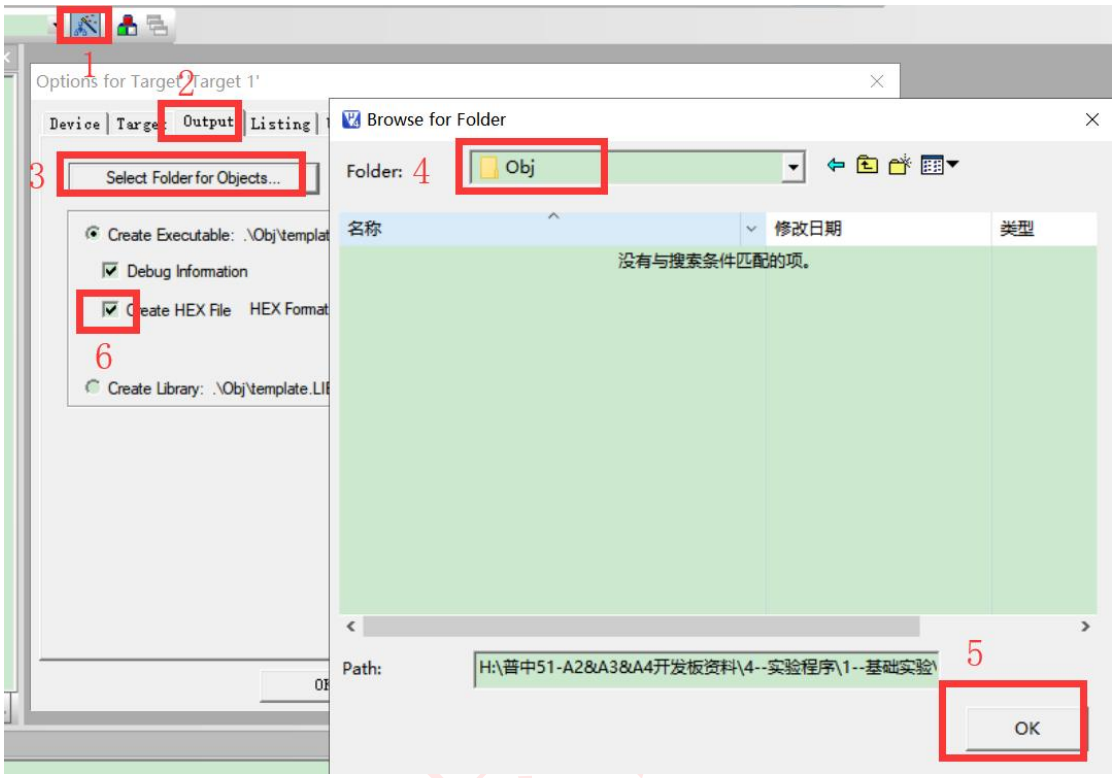
如果有遗漏的文件未添加进来，可按照上述方法重新添加。接下来如果直接编译工程肯定会出现一大堆错误，因为在 KEIL 中指定那些文件夹中的头文件路径。

23.4.1.3 配置魔术棒选项卡

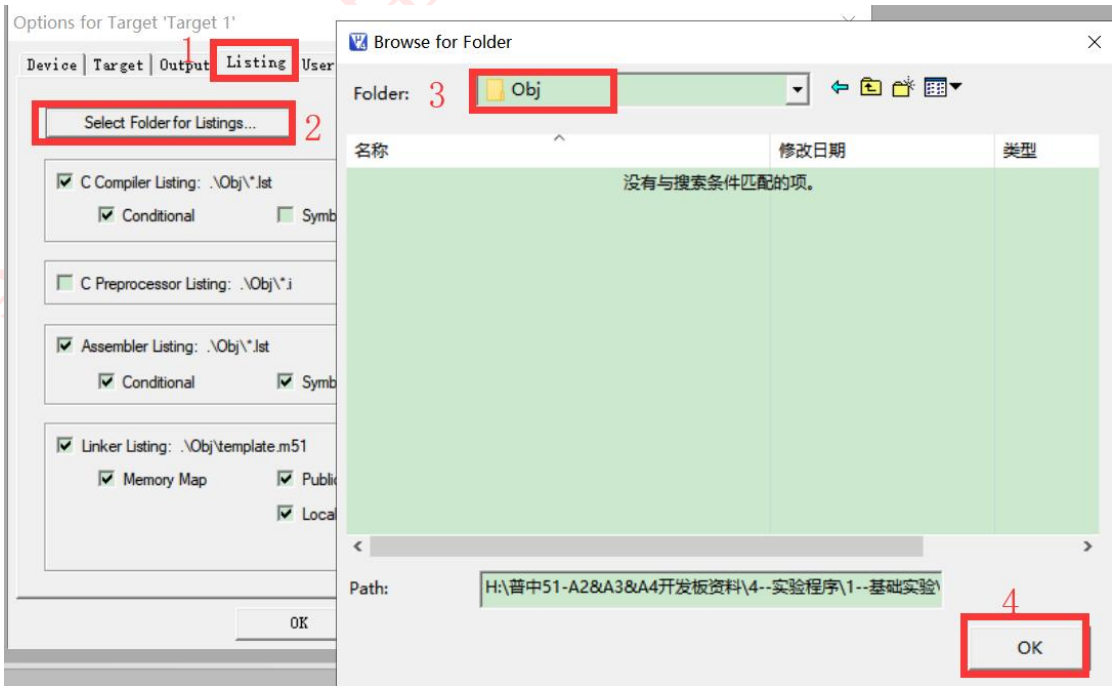
这一步的配置工作非常重要，很多人编写完程序编译后发现找不到 HEX 文件，还有的人直接编译前面添加好文件的工程出现报错，这些问题都是在这个地方没有配置好导致的。

（1）Output 选项卡中把输出文件夹定位到我们实验目录下的 Obj 文件夹，

如果想在编译的过程中生成 hex 文件，那么 Create HEX File 选项勾上。配置如下：

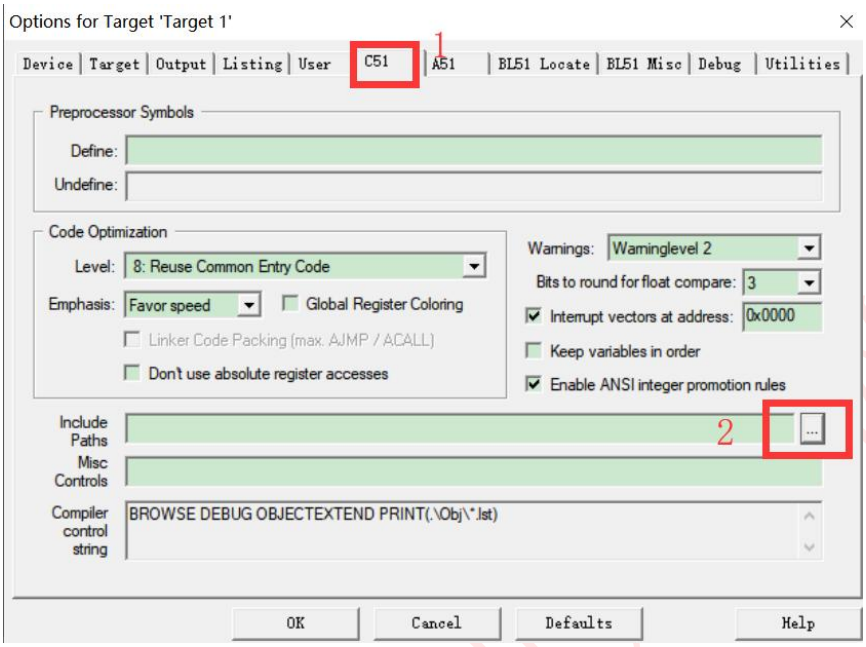


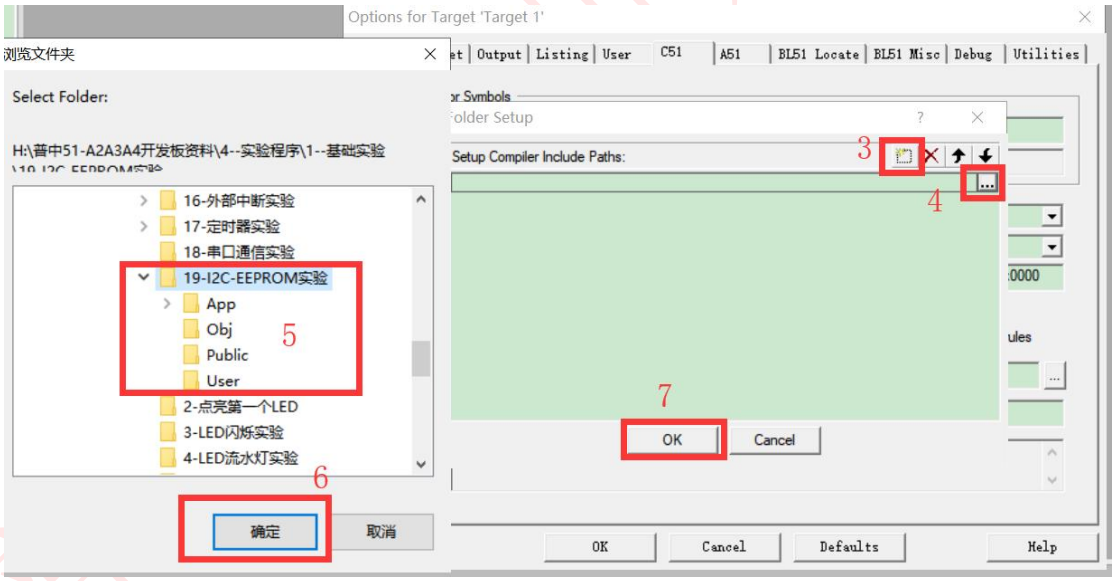
（3）Listing 选项卡中把输出文件夹也定位到我们实验目录下的 Obj 文件夹。其它设置默认，配置如下：



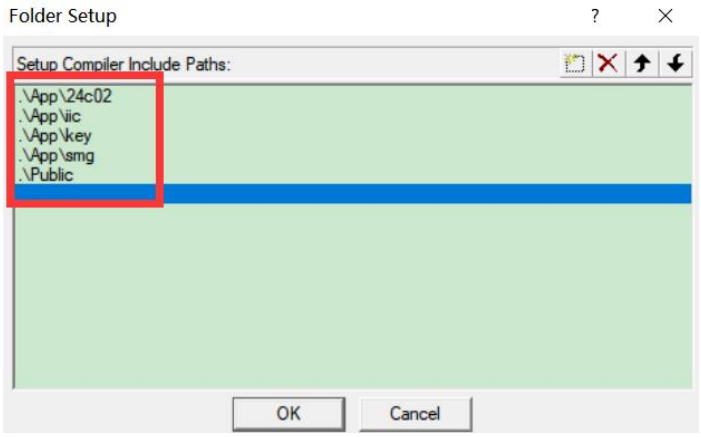
（4）C51 选项卡配置，此处目的是将我们前面添加到工程组中的文件路径包

括进来，否则程序中调用其他文件夹的头文件则会报错找不到头文件路径，具体步骤如下：





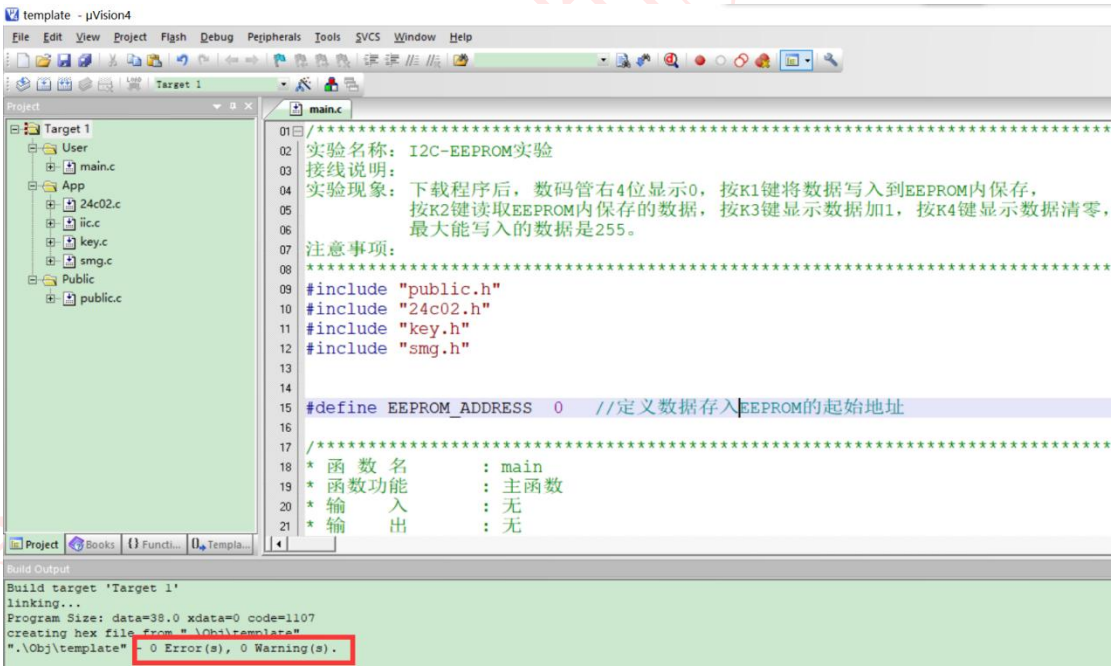
添加的头文件路径是指，在 I2C-EEPROM 实验文件夹下里面，哪些文件夹内含有.h 头文件，并且需要被调用到的，通常我们会把只要含有头文件的文件夹都选择进去。比如本例程中 App 内含有很多子文件夹，它们里面都含有头文件，因此要分别添加，Public 也含有头文件，所以也要添加。添加完成后如下：



23.4.1.4 51 仿真器配置

有关 51 仿真器配置可参考前面章节内容，此处不再重复。

然后编译一下工程，编译后结果 0 错误 0 警告，表明我们创建的多文件工程没有问题。如下：



至此，我们就成功创建好一个多文件工程模板。**在以后的实验中，凡是涉及到多个外设资源模块的都可以使用该工程模板，尤其是对重复利用已编写过的外设驱动。**

23.4.2 实验代码

本章所要实现的功能是：系统运行时，数码管右 3 位显示 0，按 K1 键将数据写入到 EEPROM 内保存，按 K2 键读取 EEPROM 内保存的数据，按 K3 键显示数据加 1，按 K4 键显示数据清零，最大能写入的数据是 255。

程序框架如下：

（1）编写按键检测功能

（2）编写数码管显示功能

（3）编写 IIC 驱动，包括起始、停止、应答信号等

（4）编写 AT24C02 读写功能

（5）编写主函数

前面的实验章节都已编写过按键检测、数码管显示功能，现在对大家来说应该不是问题，所以本章软件的重点在 I2C 时序的模拟及数据的读写上。下面我们打开“\4--实验程序\1--基础实验\19-I2C-EEPROM 实验”工程，在 App 工程组中可以看到添加了 iic.c、24c02.c、key.c 和 smg.c 文件（iic.c 里面包含了 I2C 驱动程序，24c02.c 里面包含了 AT24C02 驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

23.4.2.1 按键检测函数

打开 key.c 文件，代码如下：

```
#include "key.h"

/*************************************************************************
***********
* 函 数 名         : key_scan
* 函数功能         : 检测独立按键是否按下，按下则返回对应键值
* 输    入         : mode=0：单次扫描按键
                     mode=1：连续扫描按键
* 输    出         : KEY1_PRESS：K1 按下
                     KEY2_PRESS：K2 按下
                     KEY3_PRESS：K3 按下
                     KEY4_PRESS：K4 按下
```

```
                KEY_UNPRESS：未有按键按下
*****************************************************************************
**********/
u8 key_scan(u8 mode)
{
    static u8 key=1;

    if(mode)key=1;//连续扫描按键
    if(key==1&&(KEY1==0||KEY2==0||KEY3==0||KEY4==0))//任意按键按下
    {
        delay_10us(1000);//消抖
        key=0;
        if(KEY1==0)
            return KEY1_PRESS;
        else if(KEY2==0)
            return KEY2_PRESS;
        else if(KEY3==0)
            return KEY3_PRESS;
        else if(KEY4==0)
            return KEY4_PRESS;
    }
    else if(KEY1==1&&KEY2==1&&KEY3==1&&KEY4==1)    //无按键按下
    {
        key=1;
    }
    return KEY_UNPRESS;
}
```

上述代码为按键检测函数，调用此函数后若有按键按下，会返回对应键值。该函数在前面独立按键实验章节已详细介绍，此处不再重复。

23.4.2.1.1 .c 与.h 的格式及含义

通常.c 源文件开始会调用与之匹配的.h 头文件，这样在.c 源文件中即可使用.h 头文件的内容。与 key.c 对应的头文件是 key.h，一般我们以文件形式存放对应功能的驱动程序时，会创建 2 个文件，一个是.c 源文件，另一个是.h 头文件。源文件通常存放的是外设的驱动程序，比如按键检测函数；而头文件通常用来存放管脚定义、变量声明、函数声明等。

我们可以看下 key.h 头文件内容如下所示：

```
#ifndef _key_H
```

```
#define _key_H

#include "public.h"

//定义独立按键控制脚
sbit KEY1=P3^1;
sbit KEY2=P3^0;
sbit KEY3=P3^2;
sbit KEY4=P3^3;


//使用宏定义独立按键按下的键值
#define KEY1_PRESS   1
#define KEY2_PRESS   2
#define KEY3_PRESS   3
#define KEY4_PRESS   4
#define KEY_UNPRESS  0


u8 key_scan(u8 mode);

#endif
```

可以看到在 key.h 中定义了独立按键的控制管脚以及按键按下后的键值，这些内容在前面独立按键实验已介绍过。细心的朋友可能会发现在开始和结束位置有如下这种结构：

```
#ifndef _key_H
#define _key_H

...（此处省略头文件定义的内容）

#endif
```

它的功能是防止头文件被重复包含，避免引起编译错误。在头文件的开头，使用“ #ifndef”关键字，判断标号“_key_H”是否被定义，若没有被定义，则从“#ifndef”至“ #endif”关键字之间的内容都有效，也就是说，这个头文件若被其它文件“#include”，它就会被包含到其该文件中，且头文件中紧接着使用“#define”关键字定义上面判断的标号“ _key_H”。当这个头文件被同一个

文件第二次“#include”包含的时候，由于有了第一次包含中的“#define _key_H”定义，这时再判断“#ifndef _led_H”，判断的结果就是假了，从“#ifndef”至“#endif”之间的内容都无效，从而防止了同一个头文件被包含多次，编译时就不会出现“redefine（重复定义）”的错误了。

一般来说，我们不会直接在 C 的源文件写两个“#include”来包含同一个头文件，但可能因为头文件内部的包含导致重复，这种代码主要是避免这样的问题。如“key.h”文件中调用了#include “public.h”头文件，可能我们写主程序的时候会在 main 文件开始处调用#include “public.h”和“key.h”，这个时候“public.h”文件就被包含两次了，如果在头文件中没有这种机制，编译器就会报错。

key.h 头文件中包含的 public.h 头文件是我们定义的，存放在 Public 文件夹中，在前面也介绍其作用，主要用于存放 51 单片机开发中共用的一些程序或变量类型的重定义等，比如延时函数，u16、u8 类型的定义等。其中最关键的是包含了#include “reg52.h”，只要调用 public.h 即可包含 reg52.h。

23.4.2.2 数码管显示函数

打开 smg.c 文件，代码如下：

```
#include "smg.h"

//共阴极数码管显示 0~F 的段码数据
u8 gsmg_code[17]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,
                  0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x79,0x71};

/*******************************************************************
***********
* 函 数 名       : smg_display
* 函数功能       : 动态数码管显示
* 输    入       : dat：要显示的数据
                   pos：从左开始第几个位置开始显示，范围 1-8
* 输    出       : 无
*******************************************************************
**********/
void smg_display(u8 dat[],u8 pos)
{
    u8 i=0;
```

```
    u8 pos_temp=pos-1;

    for(i=pos_temp;i<8;i++)
    {
        switch(i)//位选
        {
            case 0: LSC=1;LSB=1;LSA=1;break;
            case 1: LSC=1;LSB=1;LSA=0;break;
            case 2: LSC=1;LSB=0;LSA=1;break;
            case 3: LSC=1;LSB=0;LSA=0;break;
            case 4: LSC=0;LSB=1;LSA=1;break;
            case 5: LSC=0;LSB=1;LSA=0;break;
            case 6: LSC=0;LSB=0;LSA=1;break;
            case 7: LSC=0;LSB=0;LSA=0;break;
        }
        SMG_A_DP_PORT=gsmg_code[dat[i-pos_temp]];//传送段选数据
        delay_10us(100);//延时一段时间，等待显示稳定
        SMG_A_DP_PORT=0x00;//消音
    }
}
```

该代码也很好理解，在前面动态数码管实验章节已介绍。不同的是，此处我们做了一些改良，即给 smg_display 函数增加了 2 个形参，dat 和 pos，dat 是一个 u8 类型的数组，方便外部传入要显示的数据；pos 是数码管从左开始第几个位置开始显示，取值范围是 1-8，如果为 1，即从第一位数码管开始显示，最多显示 8 位数据；如果取 2，即从第二位数码管开始显示，最多显示 7 位数据。

由于给函数增加了 2 个形参，所以该条语句就变为如下形式：

```
    SMG_A_DP_PORT=gsmg_code[dat[i-pos_temp]];
```

当确定好显示位置，pos_temp 值就确定，如果要显示的数据也确定，dat 数组内的值就直接可以通过数码管段选数据赋值给 SMG_A_DP_PORT。

有些朋友可能看不懂为什么 dat[i-pos_temp]含义，因为动态扫描，for 循环从 i 起始值开始，而一般外部传入的数组 dat 是从下标 0 开始，所以为 i-pos_temp。

23.4.2.3 I2C 读写字节函数

通过前面 I2C 的介绍，我们知道，要进行 I2C 通信，需要编写起始信号、停

止信号、应答和非应答信号。因为 I2C 通信是以字节为单位进行传输的，所以还需要编写 I2C 读写字节的函数。打开 iic.c 具体代码如下：

```
#include "iic.h"

/*********************************************************************
***********
* 函 数 名       : iic_start
* 函数功能       : 产生 IIC 起始信号
* 输    入       : 无
* 输    出       : 无
*********************************************************************
**********/
void iic_start(void)
{
    IIC_SCL=1;
    IIC_SDA=1;
    delay_10us(1);
    IIC_SDA=0; //当 SCL 为高电平时，SDA 由高变为低
    delay_10us(1);
    IIC_SCL=0;//钳住 I2C 总线，准备发送或接收数据
}

/*********************************************************************
***********
* 函 数 名         : iic_stop
* 函数功能         : 产生 IIC 停止信号
* 输    入         : 无
* 输    出         : 无
*********************************************************************
**********/
void iic_stop(void)
{
    IIC_SCL=1;
    IIC_SDA=0;
    delay_10us(1);
    IIC_SDA=1; //当 SCL 为高电平时，SDA 由低变为高
    delay_10us(1);
}

/*********************************************************************
***********
* 函 数 名         : iic_ack
* 函数功能         : 产生 ACK 应答
```

```
* 输    入         : 无
* 输    出         : 无
*******************************************************************************
**********/
void iic_ack(void)
{
    IIC_SCL=0;
    IIC_SDA=0; //SDA 为低电平
    delay_10us(1);
    IIC_SCL=1;
    delay_10us(1);
    IIC_SCL=0;
}

/*******************************************************************************
***********
* 函 数 名         : iic_nack
* 函数功能         : 产生 NACK 非应答
* 输    入         : 无
* 输    出         : 无
*******************************************************************************
**********/
void iic_nack(void)
{
    IIC_SCL=0;
    IIC_SDA=1; //SDA 为高电平
    delay_10us(1);
    IIC_SCL=1;
    delay_10us(1);
    IIC_SCL=0;
}

/*******************************************************************************
***********
* 函 数 名         : iic_wait_ack
* 函数功能         : 等待应答信号到来
* 输    入         : 无
* 输    出         : 1，接收应答失败
                0，接收应答成功
*******************************************************************************
**********/
u8 iic_wait_ack(void)
{
    u8 time_temp=0;
```

```
    IIC_SCL=1;
    delay_10us(1);
    while(IIC_SDA)     //等待 SDA 为低电平
    {
        time_temp++;
        if(time_temp>100)//超时则强制结束 IIC 通信
        {
            iic_stop();
            return 1;
        }
    }
    IIC_SCL=0;
    return 0;
}

/*****************************************************************************
***********
* 函 数 名         : iic_write_byte
* 函数功能         : IIC 发送一个字节
* 输    入         : dat：发送一个字节
* 输    出         : 无
*****************************************************************************
**********/
void iic_write_byte(u8 dat)
{
    u8 i=0;

    IIC_SCL=0;
    for(i=0;i<8;i++)  //循环 8 次将一个字节传出，先传高再传低位
    {
        if((dat&0x80)>0)
            IIC_SDA=1;
        else
            IIC_SDA=0;
        dat<<=1;
        delay_10us(1);
        IIC_SCL=1;
        delay_10us(1);
        IIC_SCL=0;
        delay_10us(1);
    }
}
```

```
/*****************************************************************
***********
* 函 数 名         : iic_read_byte
* 函数功能         : IIC 读一个字节
* 输    入         : ack=1 时，发送 ACK，ack=0，发送 nACK
* 输    出         : 应答或非应答
*****************************************************************
**********/
u8 iic_read_byte(u8 ack)
{
    u8 i=0,receive=0;

    for(i=0;i<8;i++ ) //循环 8 次将一个字节读出，先读高再传低位
    {
        IIC_SCL=0;
        delay_10us(1);
        IIC_SCL=1;
        receive<<=1;
        if(IIC_SDA)receive++;
        delay_10us(1);
    }
    if (!ack)
        iic_nack();
    else
        iic_ack();

    return receive;
}
```

上述代码是基于 I2C 时序编写，具体可以参考时序图和程序注释来看，非常好理解。程序中 IIC_SDA 和 IIC_SCL 是 EEPROM 的控制引脚，在 iic.h 头文件已定义。

23.4.2.4 AT24C02 读写字节函数

打开 24c02.c 文件，代码如下：

```
#include "24c02.h"
#include "iic.h"


/*****************************************************************
**********
```

```
* 函 数 名         : at24c02_write_one_byte
* 函数功能         : 在 AT24CXX 指定地址写入一个数据
* 输    入         : addr:写入数据的目的地址
                     dat:要写入的数据
* 输    出         : 无
*******************************************************************************
**********/
void at24c02_write_one_byte(u8 addr,u8 dat)
{

    iic_start();
    iic_write_byte(0XA0);    //发送写命令
    iic_wait_ack();
    iic_write_byte(addr);//发送写地址
    iic_wait_ack();
    iic_write_byte(dat); //发送字节
    iic_wait_ack();
    iic_stop();              //产生一个停止条件
    delay_ms(10);
}

/*******************************************************************************
**********
* 函 数 名         : at24c02_read_one_byte
* 函数功能         : 在 AT24CXX 指定地址读出一个数据
* 输    入         : addr:开始读数的地址
* 输    出         : 读到的数据
*******************************************************************************
**********/
u8 at24c02_read_one_byte(u8 addr)
{
    u8 temp=0;

    iic_start();
    iic_write_byte(0XA0);    //发送写命令
    iic_wait_ack();
    iic_write_byte(addr);    //发送写地址
    iic_wait_ack();
    iic_start();
    iic_write_byte(0XA1);    //进入接收模式
    iic_wait_ack();
    temp=iic_read_byte(0);   //读取字节
    iic_stop();              //产生一个停止条件
    return temp;          //返回读取的数据
```

```
}
```

上述代码也是基于 AT24C02 数据手册中读写数据格式实现，在前面介绍 I2C 协议时有介绍。此处不再重复。

23.4.2.5 主函数

打开 main.c 文件，代码如下：

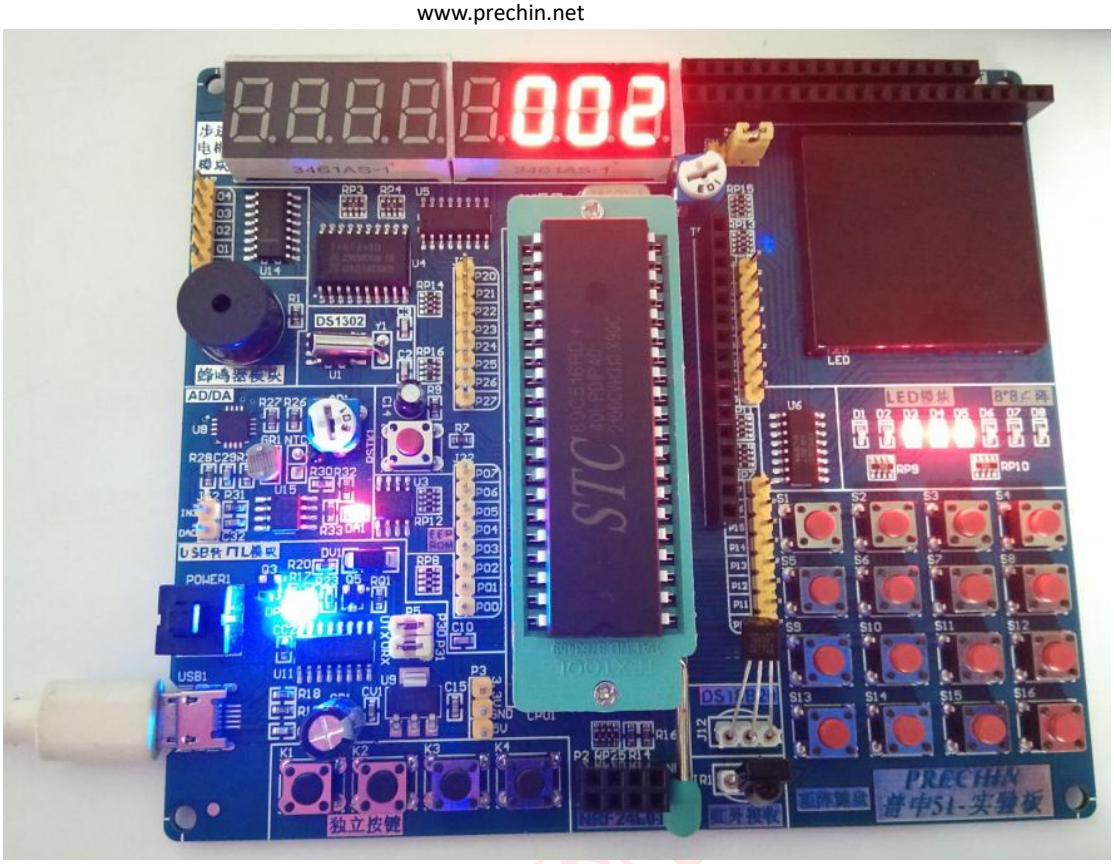
```
/**********************************************************************
******************
实验名称：I2C-EEPROM 实验
接线说明：
实验现象：下载程序后，数码管右 4 位显示 0，按 K1 键将数据写入到 EEPROM 内保存，
          按 K2 键读取 EEPROM 内保存的数据，按 K3 键显示数据加 1，按 K4 键显示数据清
零，
          最大能写入的数据是 255。
注意事项：

**********************************************************************
******************/
#include "public.h"
#include "24c02.h"
#include "key.h"
#include "smg.h"


#define EEPROM_ADDRESS   0   //定义数据存入 EEPROM 的起始地址

/**********************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
**********************************************************************
**********/
void main()
{
    u8 key_temp=0;
    u8 save_value=0;
    u8 save_buf[3];
```

```
    while(1)
    {
        key_temp=key_scan(0);
        if(key_temp==KEY1_PRESS)
        {
            at24c02_write_one_byte(EEPROM_ADDRESS,save_value);
        }
        else if(key_temp==KEY2_PRESS)
        {
            save_value=at24c02_read_one_byte(EEPROM_ADDRESS);
        }
        else if(key_temp==KEY3_PRESS)
        {
            save_value++;
            if(save_value==255)save_value=255;
        }
        else if(key_temp==KEY4_PRESS)
        {
            save_value=0;
        }
        save_buf[0]=save_value/100;
        save_buf[1]=save_value%100/10;
        save_buf[2]=save_value%100%10;
        smg_display(save_buf,6);
    }
}
```

上述代码很简单，首先将所需要用到的外设驱动程序的头文件包含进来，这样在 main.c 中才能使用，然后在 main 中定义所需变量，进入 while 循环后，检测按键是否按下，然后根据返回的键值操作对应功能，最后是将按键调节的值显示在后 3 位数码管上。

23.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：码管右 3 位显示 0，按 K1 键将数据写入到 EEPROM 内保存，按 K2 键读取 EEPROM 内保存的数据，按 K3 键显示数据加 1，按 K4 键显示数据清零，最大能写入的数据是 255。



课后作业

普中51单片机

第 24 章 DS18B20 温度传感器实验

在前面章节，我们介绍了如何使用单片机 IO 口模拟 IIC 总线时序。这一章我们来学习精度较高的外部 DS18B20 数字温度传感器，由于此传感器是单总线接口，所以需要使用 51 单片机的一个 IO 口模拟单总线时序与 DS18B20 通信，将检测的环境温度读取出来。开发板上集成了 1 个 DS18B20 温度传感器接口，需插上 DS18B20 温度传感器后才能测试温度。本章要实现的功能是：系统运行时，插上 DS18B20 温度传感器，数码管显示检测的温度值。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

24.1 DS18B20 介绍

24.2 硬件设计

24.3 软件设计

24.4 实验现象

普中51单片机开发板

24.1 DS18B20 介绍

DS18B20 是由 DALLAS 半导体公司推出的一种的“一线总线（单总线）”接口的温度传感器。与传统的热敏电阻等测温元件相比，它是一种新型的体积小、适用电压宽、与微处理器接口简单的数字化温度传感器。

DS18B20 温度传感器具有如下特点：

1、适应电压范围更宽，电压范围：3.0～5.5V，在寄生电源方式下可由数据线供电。

2、独特的单线接口方式，DS18B20 在与微处理器连接时仅需要一条口线即可实现微处理器与 DS18B20 的双向通讯。

3、DS18B20 支持多点组网功能，多个 DS18B20 可以并联在唯一的三线上，实现组网多点测温。

4、DS18B20 在使用中不需要任何外围元件，全部传感元件及转换电路集成在形如一只三极管的集成电路内。

5、温范围－55℃～+125℃，在-10～+85℃时精度为±0.5℃

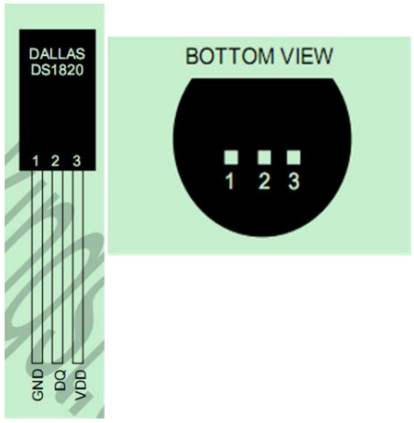
6、可编程的分辨率为 9～12 位，对应的可分辨温度分别为 0.5℃、0.25℃、0.125℃ 和 0.0625℃，可实现高精度测温。

7、在 9 位分辨率时最多在 93.75ms 内把温度转换为数字，12 位分辨率时最多在 750ms 内把温度值转换为数字，速度更快。

8、测量结果直接输出数字温度信号，以"一根总线"串行传送给 CPU，同时可传送 CRC 校验码，具有极强的抗干扰纠错能力。

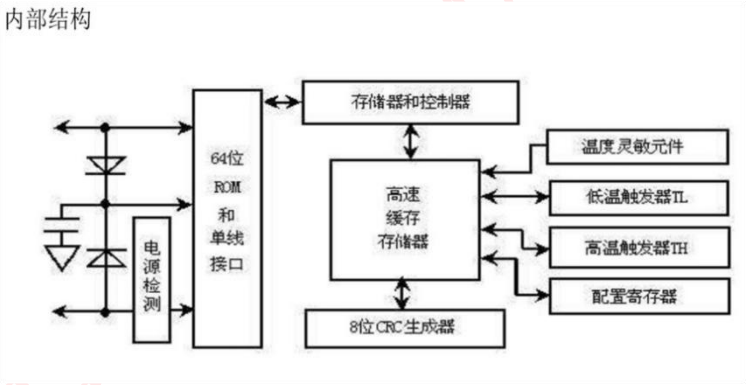
9、负压特性：电源极性接反时，芯片不会因发热而烧毁，但不能正常工作。

DS18B20 外观实物如下图所示：



从 DS18B20 外观图可以看到，当我们正对传感器切面（传感器型号字符那一面）时，传感器的管脚顺序是从左到右排列。管脚 1 为 GND，管脚 2 为数据 DQ，管脚 3 为 VDD。如果把传感器插反，那么电源将短路，传感器就会发烫，很容易损坏，所以一定要注意传感器方向，通常我们在开发板上都会标出传感器的凸起出，所以只需要把传感器凸起的方向对着开发板凸起方向插入即可。

DS18B20 内部结构如下图所示：



ROM 中的 64 位序列号是出厂前被光刻好的，它可以看作是该 DS18B20 的地址序列号。64 位光刻 ROM 的排列是：开始 8 位（28H）是产品类型标号，接着的 48 位是该 DS18B20 自身的序列号，最后 8 位是前面 56 位的循环冗余校验码。光刻 ROM 的作用是使每一个 DS18B20 都各不相同，这样就可以实现一根总线上挂接多个 DS18B20 的目的。

DS18B20 温度传感器的内部存储器包括一个高速的暂存器 RAM 和一个非易失性的可电擦除的 EEPROM,后者存放高温度和低温度触发器 TH、TL 和配置寄存器。

配置寄存器是配置不同的位数来确定温度和数字的转化，配置寄存器结构如

下：

TM	R1	R0	1	1	1	1	1

低五位一直都是"1"，TM 是测试模式位，用于设置 DS18B20 在工作模式还是在测试模式。在 DS18B20 出厂时该位被设置为 0，用户不需要去改动。R1 和 R0 用来设置 DS18B20 的精度（分辨率），可设置为 9，10，11 或 12 位，对应的分辨率温度是 0.5℃，0.25℃，0.125℃和 0.0625℃。R0 和 R1 配置如下图：

R1	R0	精度	最大转换时间	
0	0	9-bit	93.75 ms	(t_{CONV}/8)
0	1	10-bit	187.5 ms	(t_{CONV}/4)
1	0	11-bit	375 ms	(t_{CONV}/2)
1	1	12-bit	750 ms	(t_{CONV})

在初始状态下默认的精度是 12 位，即 R0=1、R1=1。高速暂存存储器由 9 个字节组成，其分配如下：

寄存器内容	字节地址
温度值低位（LS Byte）	0
温度值高位（MS Byte）	1
高温限值（TH）	2
低温限值（TL）	3
配置寄存器	4
保留	5
保留	6
保留	7
CRC校验值	8

当温度转换命令（44H）发布后，经转换所得的温度值以二字节补码形式存放在高速暂存存储器的第 0 和第 1 个字节。存储的两个字节，高字节的前 5 位是符号位 S，单片机可通过单线接口读到该数据，读取时低位在前，高位在后，数据格式如下：

温度寄存器格式 图 2

	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
LS Byte	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	bit 15	bit 14	bit 13	bit 12	bit 11	bit 10	bit 9	bit 8
MS Byte	S	S	S	S	S	2^6	2^5	2^4

如果测得的温度大于 0，这 5 位为‘0’，只要将测到的数值乘以 0.0625（默认精度是 12 位）即可得到实际温度；如果温度小于 0，这 5 位为‘1’，测到的数值需要取反加 1 再乘以 0.0625 即可得到实际温度。温度与数据对应关系如下：

温度/数据关系 表 2

温度 ℃	数据输出（二进制）	数据输出（十六进制）
+125	0000 0111 1101 0000	07D0h
+85	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FFF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Eh
-55	1111 1100 1001 0000	FC90h

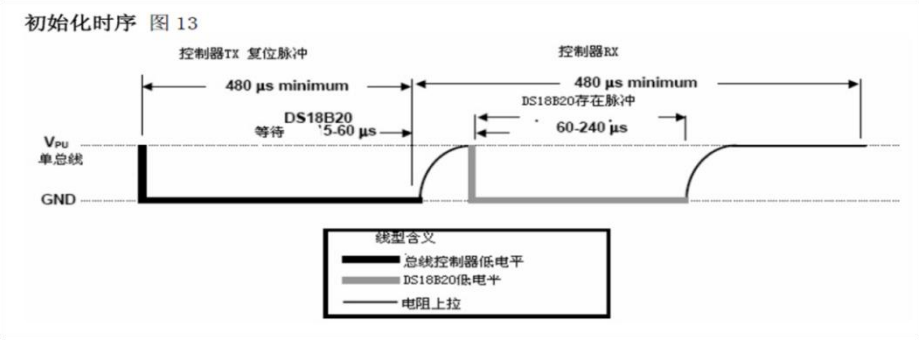
*上电复位时温度寄存器默认值为+85℃

比如我们要计算+85 度，数据输出十六进制是 0X0550，因为高字节的高 5 位为 0，表明检测的温度是正温度，0X0550 对应的十进制为 1360，将这个值乘以 12 位精度 0.0625，所以可以得到+85 度。

知道了怎么计算温度，接下来我们就来看看如何读取温度数据，由于 DS18B20 是单总线器件，所有的单总线器件都要求采用严格的信号时序，以保证 数据的完整性。DS18B20 时序包括如下几种：初始化时序、写（0 和 1）时序、 读（0 和 1）时序。 DS18B20 发送所有的命令和数据都是字节的低位在前。这里我们简单介绍这几个信号的时序：

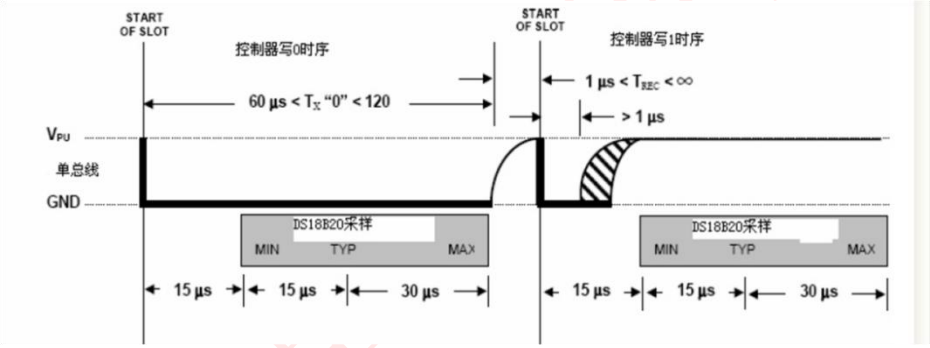
（1）初始化时序

单总线上的所有通信都是以初始化序列开始。主机输出低电平，保持低电平时间至少 480us（该时间的时间范围可以从 480 到 960 微妙），以产生复位脉冲。接着主机释放总线，外部的上拉电阻将单总线拉高，延时 15～60 us，并进入接收模式。接着 DS18B20 拉低总线 60~240 us，以产生低电平应答脉冲，若为低电平，还要做延时，其延时的时间从外部上拉电阻将单总线拉高算起最少要 480 微妙。初始化时序图如下：



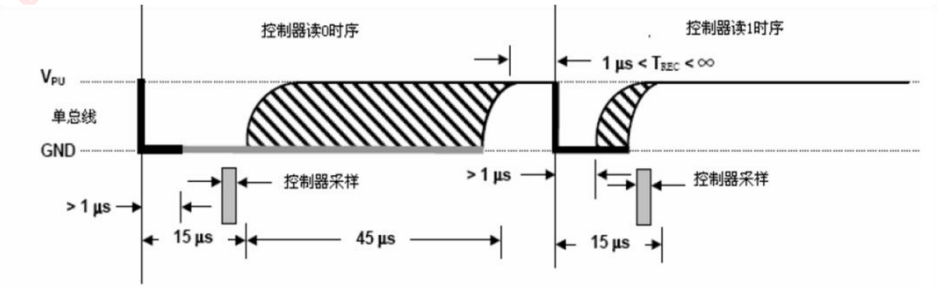
（2）写时序

写时序包括写 0 时序和写 1 时序。所有写时序至少需要 60us，且在 2 次独立的写时序之间至少需要 1us 的恢复时间，两种写时序均起始于主机拉低总线。写 1 时序：主机输出低电平，延时 2us，然后释放总线，延时 60us。写 0 时序：主机输出低电平，延时 60us，然后释放总线，延时 2us。写时序图如下：



（3）读时序

单总线器件仅在主机发出读时序时，才向主机传输数据，所以，在主机发出读数据命令后，必须马上产生读时序，以便从机能够传输数据。所有读时序至少需要 60us，且在 2 次独立的读时序之间至少需要 1us 的恢复时间。每个读时序都由主机发起，至少拉低总线 1us。主机在读时序期间必须释放总线，并且在时序起始后的 15us 之内采样总线状态。读时序图如下：



典型的读时序过程为：主机输出低电平延时 2us，然后主机转入输入模式延时 12us，然后读取单总线当前的电平，然后延时 50us。

在了解了单总线时序之后，我们来看看 DS18B20 的典型温度读取过程，DS18B20 的典型温度读取过程为：复位→发 SKIP ROM 命令（0XCC）→发开始转换命令（0X44）→延时→复位→发送 SKIP ROM 命令（0XCC）→发读存储器命令（0XBE）→连续读出两个字节数据(即温度)→结束。

至此我们就介绍完 DS18B20，如需更详细的介绍，请大家参考“\6--芯片资料\开发板芯片数据手册\DS18B20 中文.pdf”。

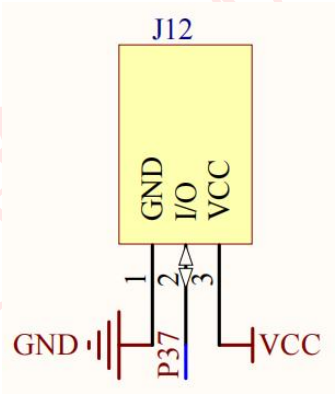
24.2 硬件设计

本实验使用到硬件资源如下：

（1）动态数码管

（2）DS18B20

动态数码管电路在前面章节已介绍，这里就不再重复。下面来看下开发板上 DS18B20 模块电路，如下图所示：



从上图可以看出，传感器接口的单总线管脚接至单片机 P3.7 IO 口上，在介绍单总线的时候我们说过，为了让单总线默认为高电平，通常会在单总线上接上拉电阻，在图中并没有看到有上拉电阻，这是因为单片机 IO 都外接了 10K 上拉电阻，当单片机 IO 口连接到传感器的总线管脚时即相当于它们外接上拉电阻，所以此处可以省去。

24.3 软件设计

本章所要实现的功能是：插上 DS18B20 温度传感器，数码管显示检测的温度值。

程序框架如下：

（1）编写数码管显示功能

（2）编写 DS18B20 读取温度功能

（3）编写主函数

前面的实验章节都已编写过数码管显示功能，现在对大家来说应该不是问题，所以本章软件的重点在 DS18B20 时序的模拟及数据的读写上。下面我们打开“\4--实验程序\1--基础实验\20-DS18B20 温度传感器实验”工程，在 App 工程组中可以看到新添加了 ds18b20.c 文件（里面包含了 DS18B20 驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

24.3.1 数码管显示函数

本实验要显示温度值，就存在小数和符号位，所以不能直接使用前面的数码管显示函数，需要做一点点小改动，代码如下：

```
/*********************************************************************
***********
* 函 数 名       : smg_display
* 函数功能       : 动态数码管显示
* 输    入       : dat：要显示的数据
                   pos：从左开始第几个位置开始显示，范围 1-8
* 输    出       : 无
*********************************************************************
**********/
void smg_display(u8 dat[],u8 pos)
{
    u8 i=0;
    u8 pos_temp=pos-1;

    for(i=pos_temp;i<8;i++)
    {
        switch(i)//位选
        {
            case 0: LSC=1;LSB=1;LSA=1;break;
            case 1: LSC=1;LSB=1;LSA=0;break;
            case 2: LSC=1;LSB=0;LSA=1;break;
            case 3: LSC=1;LSB=0;LSA=0;break;
```

```
            case 4: LSC=0;LSB=1;LSA=1;break;
            case 5: LSC=0;LSB=1;LSA=0;break;
            case 6: LSC=0;LSB=0;LSA=1;break;
            case 7: LSC=0;LSB=0;LSA=0;break;
        }
        SMG_A_DP_PORT=dat[i-pos_temp];//传送段选数据
        delay_10us(100);//延时一段时间，等待显示稳定
        SMG_A_DP_PORT=0x00;//消音
    }
}
```

与之前数码管显示驱动程序不同的就是红色加粗标记的那条语句，可以看到此处没有直接使用数码管段码数据，而是通过外部提供处理好的数码管段码数据传入进来，这样可使数码管显示更加灵活，后面实验我们都将采用这种方式。因为要让外部提前处理好数码管段码数据，所以段码数组 gsmg_code 就要定义为全局变量，需在 smg.h 头文件中声明下即可：extern u8 gsmg_code[17];

24.3.2 DS18B20 初始化函数

初始化代码如下：

```
/*******************************************************************************
***********
* 函 数 名         : ds18b20_init
* 函数功能         : 初始化 DS18B20 的 IO 口 DQ 同时检测 DS 的存在
* 输    入         : 无
* 输    出         : 1:不存在，0:存在
*******************************************************************************
**********/
u8 ds18b20_init(void)
{
    ds18b20_reset();
    return ds18b20_check();
}
```

该函数带有一个返回值，如果返回值为 1 表示 DS18B20 初始化失败，返回值为 0 表示初始化成功。函数返回值其实就是通过调用 ds18b20_check 函数获得，此函数用来检测 DS18B20 是否存在。初始化函数内还调用了 ds18b20_reset 函数，这两个函数其实就是根据前面介绍的初始化时序编写，对应的代码如下：

```
/*******************************************************************************
```

```
***********
* 函 数 名         : ds18b20_reset
* 函数功能         : 复位 DS18B20
* 输    入         : 无
* 输    出         : 无
*******************************************************************************
**********/
void ds18b20_reset(void)
{
    DS18B20_PORT=0;   //拉低 DQ
    delay_10us(75);   //拉低 750us
    DS18B20_PORT=1;   //DQ=1
    delay_10us(2);    //20US
}

/*******************************************************************************
***********
* 函 数 名         : ds18b20_check
* 函数功能         : 检测 DS18B20 是否存在
* 输    入         : 无
* 输    出         : 1:未检测到 DS18B20 的存在，0:存在
*******************************************************************************
**********/
u8 ds18b20_check(void)
{
    u8 time_temp=0;

    while(DS18B20_PORT&&time_temp<20)   //等待 DQ 为低电平
    {
        time_temp++;
        delay_10us(1);
    }
    if(time_temp>=20)return 1;   //如果超时则强制返回 1
    else time_temp=0;
    while((!DS18B20_PORT)&&time_temp<20)    //等待 DQ 为高电平
    {
        time_temp++;
        delay_10us(1);
    }
    if(time_temp>=20)return 1;   //如果超时则强制返回 1
    return 0;
}
```

24.3.3 温度读取函数

初始化 DS18B20 后，我们就可以按照前面介绍的 DS18B20 的典型温度读取过程来编写温度读取函数，代码如下：

```
/*************************************************************************
***********
* 函 数 名         : ds18b20_read_temperture
* 函数功能         : 从 ds18b20 得到温度值
* 输    入         : 无
* 输    出         : 温度数据
*************************************************************************
**********/
float ds18b20_read_temperture(void)
{
    float temp;
    u8 dath=0;
    u8 datl=0;
    u16 value=0;

    ds18b20_start();//开始转换
    ds18b20_reset();//复位
    ds18b20_check();
    ds18b20_write_byte(0xcc);//SKIP ROM
    ds18b20_write_byte(0xbe);//读存储器

    datl=ds18b20_read_byte();//低字节
    dath=ds18b20_read_byte();//高字节
    value=(dath<<8)+datl;//合并为 16 位数据

    if((value&0xf800)==0xf800)//判断符号位，负温度
    {
        value=(~value)+1; //数据取反再加 1
        temp=value*(-0.0625);//乘以精度
    }
    else //正温度
    {
        temp=value*0.0625;
    }
    return temp;
}
```

该函数首先调用了 ds18b20_start 函数用来开始 DS18B20 的温度转换，其代

码如下：

```
/*************************************************************************
***********
* 函 数 名         : ds18b20_start
* 函数功能         : 开始温度转换
* 输    入         : 无
* 输    出         : 无
*************************************************************************
**********/
void ds18b20_start(void)
{
    ds18b20_reset();//复位
    ds18b20_check();//检查 DS18B20
    ds18b20_write_byte(0xcc);//SKIP ROM
    ds18b20_write_byte(0x44);//转换命令
}
```

最终将 2 个字节的温度数据读取出来，判断最高字节的高 5 位是否为 0，如果为 0 表明读取的温度值为正温度，直接乘以 0.0625 即可，否则为负温度，需取反后加 1 再乘以 0.0625。

24.3.4 主函数

编写好 DS18B20 初始化和温度读取函数后，接下来就可以编写主函数了，代码如下：

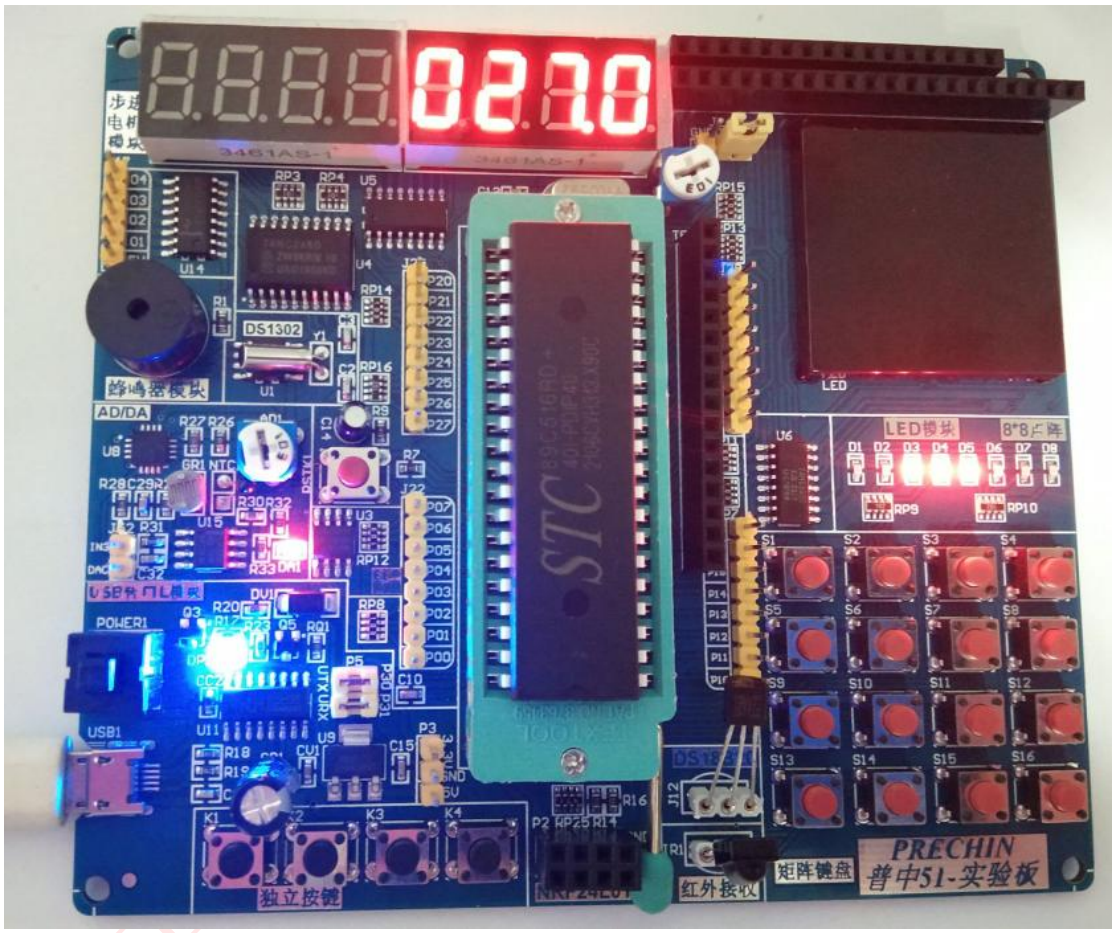
```
/*************************************************************************
******************
实验名称：DS18B20 温度传感器实验
接线说明：
实验现象：下载程序后，插上 DS18B20 温度传感器，数码管显示检测的温度值
注意事项：注意温度传感器的方向，在接口处我们已经用丝印画了一个凸起，
          所以只需要将温度传感器对应插入即可

*************************************************************************
******************/
#include "public.h"
#include "smg.h"
#include "ds18b20.h"


/*************************************************************************
```

```
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*******************************************************************************
**********/
void main()
{
    u8 i=0;
    int temp_value;
    u8 temp_buf[5];

    ds18b20_init();//初始化 DS18B20

    while(1)
    {
        i++;
        if(i%50==0)//间隔一段时间读取温度值,间隔时间要大于温度传感器转换温度时间
            temp_value=ds18b20_read_temperture()*10;//保留温度值小数后一位
        if(temp_value<0)//负温度
        {
            temp_value=-temp_value;
            temp_buf[0]=0x40;//显示负号
        }
        else
            temp_buf[0]=0x00;//不显示
        temp_buf[1]=gsmg_code[temp_value/1000];//百位
        temp_buf[2]=gsmg_code[temp_value%1000/100];//十位
        temp_buf[3]=gsmg_code[temp_value%1000%100/10]|0x80;//个位+小数
点
        temp_buf[4]=gsmg_code[temp_value%1000%100%10];//小数点后一位
        smg_display(temp_buf,4);
    }
}
```

主函数代码很简单,首先调用所需外设头文件,然后初始化DS18B20,在while循环中间隔一定时间读取温度数据，并将温度值保留小数点后一位，然后将读取的温度数据转换为数码管可显示的段码，最后调用数码管显示函数。

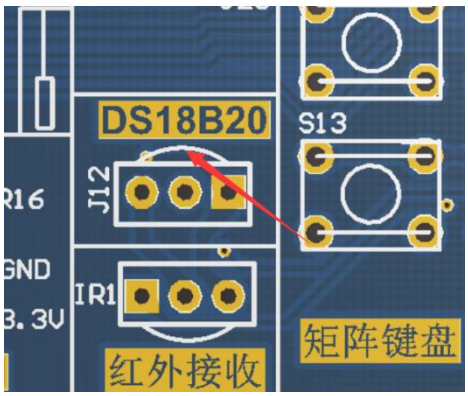
24.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：插上 DS18B20 温度传感器，数码管显示检测的温度值。



实验说明：DS18B20 默认起始温度是 85 度，之后就会进入到正常温度检测，如果条件允许可以在零下温度下检测看看。

注意：一定要注意温度传感器的方向，在接口处我们已经用丝印画了一个凸起，所以只需要将温度传感器对应插入即可。



课后作业

普中51单片机开发板

第 25 章 DS1302 时钟实验

在前面章节，我们介绍了如何使用单片机 IO 口模拟 IIC 总线、单总线时序。这一章我们来学习 DS1302 时钟芯片，该芯片是 3 线 SPI 接口，所以需要使用 51 单片机的 3 个 IO 口模拟 SPI 时序与 DS1302 时钟芯片通信，将时钟日历数据读取出来。开发板上集成了 1 个 DS1302 时钟模块，可使用它设计一个多功能电子时钟。本章要实现的功能是：系统运行时，数码管上显示电子时钟时分秒，格式为“XX-XX-XX”。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

25.1 DS1302 时钟芯片介绍

25.2 硬件设计

25.3 软件设计

25.4 实验现象

普中51单片机开发板

25.1 DS1302 时钟芯片介绍

25.1.1 DS1302 简介

DS1302 是 DALLAS 公司推出的涓流充电时钟芯片，内含有一个实时时钟/日历和 31 字节静态 RAM，通过简单的串行接口与单片机进行通信。实时时钟/日历电路提供秒、分、时、日、周、月、年的信息，每月的天数和闰年的天数可自动调整。时钟操作可通过 AM/PM 指示决定采用 24 或 12 小时格式。DS1302 与单片机之间能简单地采用同步串行的方式进行通信，仅需用到三根通信线：①RES 复位②I/O 数据线③SCLK 串行时钟。时钟/RAM 的读/写数据以一个字节或多达 31 个字节的字符组方式通信。DS1302 工作时功耗很低保持数据和时钟信息时功率小于 1mW。

DS1302 由 DS1202 改进而来增加了以下的特性：双电源管脚用于主电源和备份电源供应，Vcc1 为可编程涓流充电电源，附加七个字节存储器。它广泛应用于电话、传真、便携式仪器以及电池供电的仪器仪表等产品领域下面。

主要的性能指标：

★ 实时时钟具有能计算 2100 年之前的秒、分、时、日、星期、月、年的能力，还有闰年调整的能力；

★ 31 个 8 位暂存数据存储 RAM；

★ 串行 I/O 口方式使得管脚数量最少；

★ 宽范围工作电压 2.0~5.5V；

★ 工作在 2.0V 时，电流小于 300nA；

★ 读/写时钟或 RAM 数据时有两种传送方式单字节传送和多字节传送字符组方式；

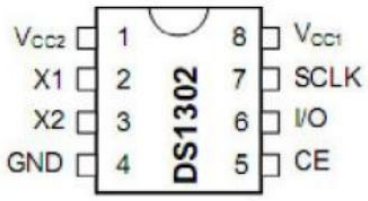
★ 8 脚 DIP 封装或可选的 8 脚 SOIC 封装根据表面装配；

★ 简单 3 线接口；

★ 与 TTL 兼容 Vcc=5V；

★ 可选工业级温度范围-40~+85；

下面来看下 DS1302 芯片的管脚及功能。



1，VCC2：主电源引脚

2，X1、X2：DS1302 外部晶振引脚，通常需外接 32.768K 晶振

3，GND：电源地

4，CE：使能引脚，也是复位引脚（新版本功能变）。

5，I/O：串行数据引脚，数据输出或者输入都从这个引脚

6，SCLK：串行时钟引脚

7，VCC1：备用电源

25.1.2 DS1302 使用

操作 DS1302 的大致过程，就是将各种数据写入 DS1302 的寄存器，以设置它当前的时间的格式。然后使 DS1302 开始运作，DS1302 时钟会按照设置情况运转，再用单片机将其寄存器内的数据读出。再用液晶显示，就是我们常说的简易电子钟。所以总的来说 DS1302 的操作分 2 步（显示部分属于液晶显示的内容，不属于 DS1302 本身的内容），但是在讲述操作时序之前，我们要先看看寄存器，DS1302 有一个控制寄存器、12 个日历、时钟寄存器和 31 个 RAM。

（1）控制寄存器

控制寄存器用于存放 DS1302 的控制命令字，DS1302 的 RST 引脚回到高电平后写入的第一个字节就为控制命令。它用于对 DS1302 读写过程进行控制，格式如下：

7	6	5	4	3	2	1	0
1	RAM $\overline{CK}$	A4	A3	A2	A1	A0	RD $\overline{WR}$

上图是 DS1302 的寄存器样式，我们看到：

1、第 7 位永远都是 1；

2、第 6 位，1 表示 RAM，寻址内部存储器地址；0 表示 CK，寻址内部寄存器；

3、第 5 到第 1 位，为 RAM 或者寄存器的地址；

4、最低位，高电平表示 RD，即下一步操作将要“读”；低电平表示 W，即下一步操作将要“写”。（与 AT24C02 寄存器类似，这点要理解好）。

寄存器名称	D7	D6	D5	D4	D3	D2	D1	D0
	1	RAM/CK	A4	A3	A2	A1	A0	R/W
秒寄存器	1	0	0	0	0	0	0	0或1
分寄存器	1	0	0	0	0	0	1	0或1
小时寄存器	1	0	0	0	0	1	0	0或1
日寄存器	1	0	0	0	0	1	1	0或1
月寄存器	1	0	0	0	1	0	0	0或1
星期寄存器	1	0	0	0	1	0	1	0或1
年寄存器	1	0	0	0	1	1	0	0或1

寄存器名称	D7	D6	D5	D4	D3	D2	D1	D0
	1	RAM/CK	A4	A3	A2	A1	A0	R/W
写保护寄存器	1	0	0	0	1	1	1	0或1
慢充电寄存器	1	0	0	1	0	0	0	0或1
时钟突发模式	1	0	1	1	1	1	1	0或1
RAM0	1	1	0	0	0	0	0	0或1
…	1	1	…	…	…	…	…	0或1
RAM30	1	1	1	1	1	1	0	0或1
RAM突发模式	1	1	1	1	1	1	1	0或1

比如要读秒寄存器则命令为 1000 0001，反之写为 1000 0000，要注意其含义。

（2）日历/时钟寄存器

DS1302 共有 12 个寄存器，其中有 7 个与日历、时钟相关，存放的数据为 BCD 码形式。格式如下：

寄存器名称	取值范围	D7	D6	D5	D4	D3	D2	D1	D0
秒寄存器	00~59	CH	秒的十位			秒的个位			
分寄存器	00~59	0	分的十位			分的个位			
小时寄存器	01~12或 00~23	12/24	0	A/P	HR	小时的个位			
日寄存器	01~31	0	0	日的十位		日的个位			

寄存器名称	取值范围	D7	D6	D5	D4	D3	D2	D1	D0
月寄存器	01~12	0	0	0	1或0	月的个位			
星期寄存器	01~07	0	0	0	0	星期几			
年寄存器	01~99	年的十位				年的个位			
写保护寄存器		WP	0	0	0	0	0	0	0
慢充电寄存器		TCS	TCS	TCS	TCS	DS	DS	RS	RS
时钟突发寄存器									

下面对几个寄存器做下说明：

秒寄存器：低四位为秒的个位，高的次三位为秒的十位。最高位 CH 为 DS1302 的运行标志，当 CH=0 时，DS1302 内部时钟运行，反之 CH=1 时停止；

小时寄存器：时寄存器。最高位为 12/24 小时的格式选择位，该位为 1 时表示 12 小时格式。当设置为 12 小时显示格式时，第 5 位的高电平表示下午（PM）；而当设置为 24 小时格式时，第 5 位位具体的时间数据。

写保护寄存器：当该寄存器最高位 WP 为 1 时，DS1302 只读不写，所以要在往 DS1302 写数据之前确保 WP 为 0。

慢充电寄存器（涓细电流充电）寄存器：我们知道，当 DS1302 掉电时，可以马上调用外部电源保护时间数据。该寄存器就是配置备用电源的充电选项的。其中高四位（4 个 TCS）只有在 1010 的情况下才能使用充电选项；低四位的情况与 DS1302 内部电路有关，具体大家可以查看“\6--开发板芯片资料\开发板芯片数据手册\DS1302 中文手册.pdf”。

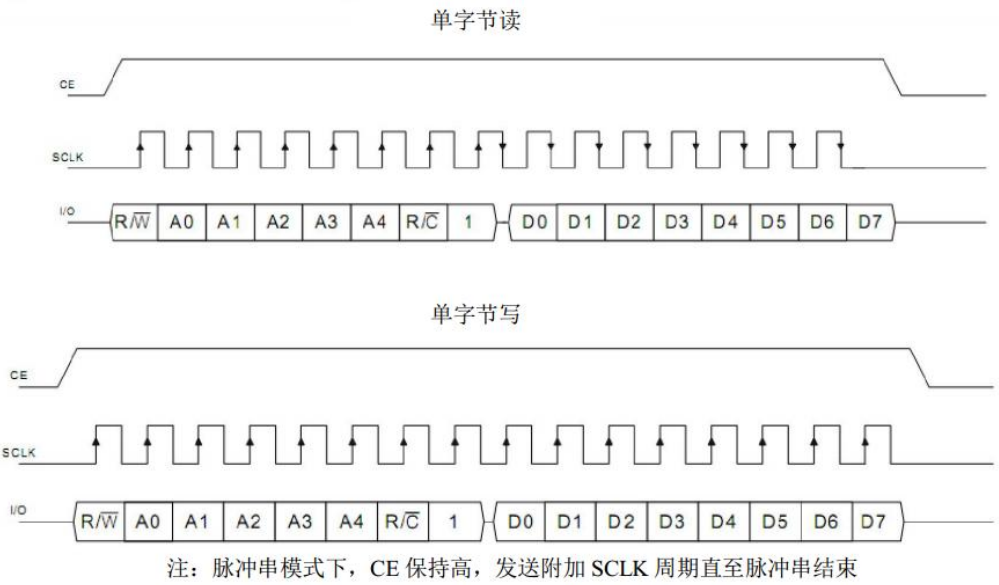
前面我们提到在日历/时钟寄存器中都是以 BCD 码存放数据，那么 BCD 码是什么呢？BCD 码是通过 4 位二进制码来表示 1 位十进制中的 0~9 这 10 个数码。如下所示：

0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

所以从 DS1302 中读取出来的时钟数据均为 BCD 码格式，需转换为我们习惯的 10 进制，转换方法在源程序里，后面我们会介绍。

（3）DS1302 的读写时序

在控制指令字输入后的下一个 SCLK 时钟的上升沿时，数据被写入 DS1302，数据输入从低位（位 0）开始。同样，在紧跟 8 位的控制指令字后的下一个 SCLK 脉冲的下降沿读出 DS1302 的数据，读出数据时从低位 0 位到高位 7。其时序图如下所示：



上图就是 DS1302 的三个时序：复位时序，单字节写时序，单字节读时序；

CE（RST）：复位时序，即在 RST 引脚产生一个正脉冲，在整个读写器件，RST 要保持高电平，一次字节读写完毕之后，要注意把 RST 返回低电平准备下次读写周期；

单字节读时序：注意读之前还是要先对寄存器写命令，从最低位开始写；可以看到，写数据是在 SCLK 的上升沿实现，而读数据在 SCLK 的下降沿实现。所以，在单字节读时序中，写命令的第八个上升沿结束后紧接着的第八个下降沿就将要读寄存器的第一位数据读到数据线上了！这个就是 DS1302 操作中最特别的地方。当然读出来的数据也是最低位开始。

单字节写时序：两个字节的数据配合 16 个上升沿将数据写入即可。

程序注意事项：

★要记得在操作 DS1302 之前关闭写保护；

★注意用延时来降低单片机的速度以配合器件时序；

★DS1302 读出来的数据是 BCD 码形式，要转换成我们习惯的 10 进制，转换方法在源程序里；

★读取字节之前，将 IO 设置为输入口，读取完之后，要将其改回输出口；

★在写程序的时候，建议实现开辟数组（内存空间）来集中放置 DS1302 的一系列数据，方便以后扩展键盘输入。

到这里我们就介绍完 DS1302 时钟芯片，如需更详细的介绍，请大家参考“\6--开发板芯片资料\开发板芯片数据手册\DS1302 中文手册.pdf”。

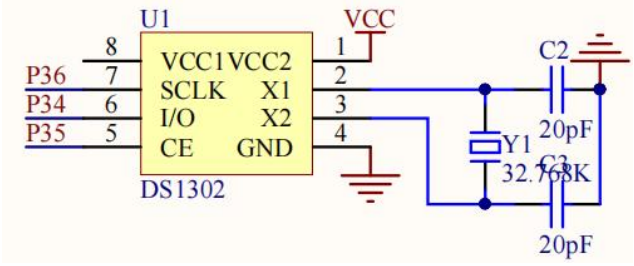
25.2 硬件设计

本实验使用到硬件资源如下：

（1）动态数码管

（2）DS1302

动态数码管电路在前面章节都介绍过，这里就不再重复。下面我们来看下开发板上 DS1302 时钟模块电路，如下图所示：



从上图中可知，DS1302 芯片的控制管脚接至单片机 P3.4-P3.6 上，在芯片的

X1、X2 管脚处外接了一个 32.768KHZ 晶振，为时钟运行提供一个稳定的时钟频率，C2 和 C3 为旁路电容，目的是消除晶振起振时产生的电感干扰。对于本开发板无外接备用电池，如果需要可自行将外部备用电源接入第 8 脚 VCC1。

25.3 软件设计

本章所要实现的功能是：数码管上显示电子时钟时分秒，格式为“XX-XX-XX”。

程序框架如下：

（1）编写数码管显示功能

（2）编写 DS1302 时钟读写功能

（3）编写主函数

前面的实验章节都已编写过数码管显示功能，现在对大家来说应该不是问题，所以本章软件的重点在 DS1302 时序的模拟及数据的读写上。下面我们打开“\4--实验程序\1--基础实验\21-DS1302 时钟实验”工程，在 App 工程组中可以看到新添加了 ds1302.c 文件（里面包含了 DS1302 驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

25.3.1 DS1302 读写函数

打开 ds1302.c 文件，代码如下：

```
#include "ds1302.h"
#include "intrins.h"

//---DS1302 写入和读取时分秒的地址命令---//
//---秒分时日月周年 最低位读写位;-------//
u8 gREAD_RTC_ADDR[7] = {0x81, 0x83, 0x85, 0x87, 0x89, 0x8b, 0x8d};
u8 gWRITE_RTC_ADDR[7] = {0x80, 0x82, 0x84, 0x86, 0x88, 0x8a, 0x8c};

//---DS1302 时钟初始化 2021 年 5 月 20 日星期四 13 点 51 分 47 秒。---//
//---存储顺序是秒分时日月周年,存储格式是用 BCD 码---//
u8 gDS1302_TIME[7] = {0x47, 0x51, 0x13, 0x20, 0x04, 0x05, 0x21};


/*********************************************************************
```

```
***********
* 函 数 名       : ds1302_write_byte
* 函数功能       : DS1302 写单字节
* 输    入       : addr：地址/命令
                   dat：数据
* 输    出       : 无
*******************************************************************
**********/
void ds1302_write_byte(u8 addr,u8 dat)
{
    u8 i=0;

    DS1302_RST=0;
    _nop_();
    DS1302_CLK=0;//CLK 低电平
    _nop_();
    DS1302_RST=1;//RST 由低到高变化
    _nop_();

    for(i=0;i<8;i++)//循环 8 次，每次写 1 位，先写低位再写高位
    {
        DS1302_IO=addr&0x01;
        addr>>=1;
        DS1302_CLK=1;
        _nop_();
        DS1302_CLK=0;//CLK 由低到高产生一个上升沿，从而写入数据
        _nop_();
    }
    for(i=0;i<8;i++)//循环 8 次，每次写 1 位，先写低位再写高位
    {
        DS1302_IO=dat&0x01;
        dat>>=1;
        DS1302_CLK=1;
        _nop_();
        DS1302_CLK=0;
        _nop_();
    }
    DS1302_RST=0;//RST 拉低
    _nop_();
}

/*******************************************************************
***********
* 函 数 名       : ds1302_read_byte
```

```
* 函数功能        : DS1302 读单字节
* 输    入        : addr：地址/命令
* 输    出        : 读取的数据
*******************************************************************************
**********/
u8 ds1302_read_byte(u8 addr)
{
    u8 i=0;
    u8 temp=0;
    u8 value=0;

    DS1302_RST=0;
    _nop_();
    DS1302_CLK=0;//CLK 低电平
    _nop_();
    DS1302_RST=1;//RST 由低到高变化
    _nop_();

    for(i=0;i<8;i++)//循环 8 次，每次写 1 位，先写低位再写高位
    {
        DS1302_IO=addr&0x01;
        addr>>=1;
        DS1302_CLK=1;
        _nop_();
        DS1302_CLK=0;//CLK 由低到高产生一个上升沿，从而写入数据
        _nop_();
    }
    for(i=0;i<8;i++)//循环 8 次，每次读 1 位，先读低位再读高位
    {
        temp=DS1302_IO;
        value=(temp<<7)|(value>>1);//先将 value 右移 1 位，然后 temp 左移 7 位，
最后或运算
        DS1302_CLK=1;
        _nop_();
        DS1302_CLK=0;
        _nop_();
    }
    DS1302_RST=0;//RST 拉低
    _nop_();
    DS1302_CLK=1;//对于实物中，P3.4 口没有外接上拉电阻的，此处代码需要添加，使数
据口有一个上升沿脉冲。
    _nop_();
    DS1302_IO = 0;
    _nop_();
```

```
    DS1302_IO = 1;
    _nop_();
    return value;
}

/*********************************************************************
***********
* 函 数 名       : ds1302_init
* 函数功能       : DS1302 初始化时间
* 输    入       : 无
* 输    出       : 无
*********************************************************************
**********/
void ds1302_init(void)
{
    u8 i=0;
    ds1302_write_byte(0x8E,0X00);
    for(i=0;i<7;i++)
    {
        ds1302_write_byte(gWRITE_RTC_ADDR[i],gDS1302_TIME[i]);
    }
    ds1302_write_byte(0x8E,0X80);
}

/*********************************************************************
***********
* 函 数 名       : ds1302_read_time
* 函数功能       : DS1302 读取时间
* 输    入       : 无
* 输    出       : 无
*********************************************************************
**********/
void ds1302_read_time(void)
{
    u8 i=0;
    for(i=0;i<7;i++)
    {
        gDS1302_TIME[i]=ds1302_read_byte(gREAD_RTC_ADDR[i]);
    }
}
```

各函数功能在代码内有详细注释，大家可以参考注释理解。程序开头定义了 3 个数组：gREAD_RTC_ADDR、gWRITE_RTC_ADDR 和 gDS1302_TIME。gREAD_RTC_ADDR 和 gWRITE_RTC_ADDR 数组内存储的是 DS1302 写入和读取时分秒的地址命令，这

个可以对照前面介绍的寄存器看。TIME 数组是用来存储初始化时间，每个数据对应含义在代码内已做注释。在使用中主要调用 ds1302_init()函数和 ds1302_read_time()函数来初始化 DS1302 时钟数据以及读取 DS1302 时钟数据。在初始化时钟数据时，首先要禁止写保护即关闭写保护功能，然后将所需设置的时钟数据写入到对应的时钟寄存器地址内，最后打开写保护功能，以防止意外修改 DS1302 内部寄存器。对于读取时钟数据其实很简单，只要从对应的时钟寄存器地址内读取数据即可，然后将读取的数据存储到一个缓存数组中，方便数据的处理与显示，这个操作在后面 main.c 文件内会有。

25.3.2 主函数

打开 main.c，代码如下：

```
/**************************************************************************
******************
实验名称：DS1302 时钟实验
接线说明：
实验现象：下载程序后，数码管上显示电子时钟时分秒，格式为“XX-XX-XX”
注意事项：

**************************************************************************
******************/
#include "public.h"
#include "smg.h"
#include "ds1302.h"


/**************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
**************************************************************************
**********/
void main()
{
    u8 time_buf[8];

    ds1302_init();//初始化 DS1302
```

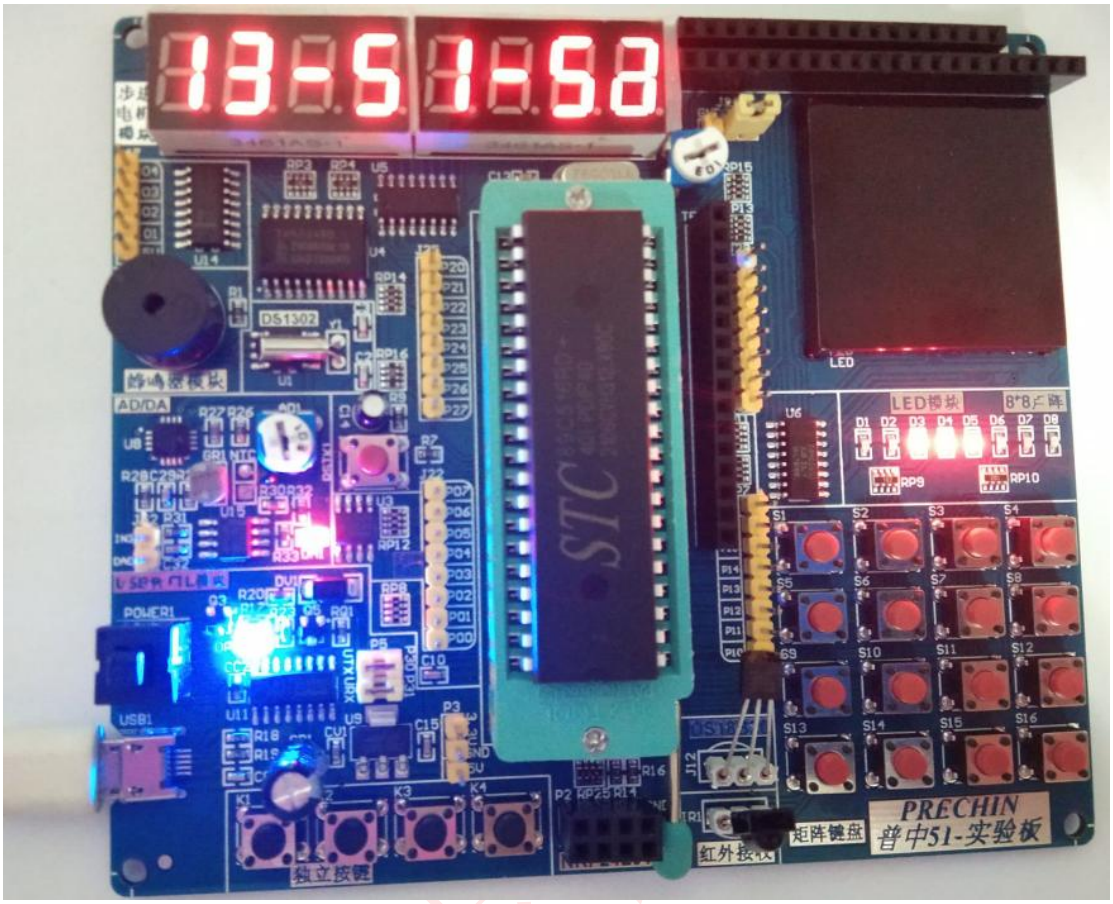
```
    while(1)
    {
        ds1302_read_time();
        time_buf[0]=gsmg_code[gDS1302_TIME[2]/16];
        time_buf[1]=gsmg_code[gDS1302_TIME[2]&0x0f];
        time_buf[2]=0x40;
        time_buf[3]=gsmg_code[gDS1302_TIME[1]/16];
        time_buf[4]=gsmg_code[gDS1302_TIME[1]&0x0f];
        time_buf[5]=0x40;
        time_buf[6]=gsmg_code[gDS1302_TIME[0]/16];
        time_buf[7]=gsmg_code[gDS1302_TIME[0]&0x0f];
        smg_display(time_buf,1);
    }
}
```

主函数代码非常简单，首先调用外设头文件，然后初始化 DS1302 并设定好初始时间，进入 while 循环，读取 DS1302 时钟数据存储至全局变量数组 gDS1302_TIME 中，最后将读取的数据转换为数码管可显示的段码数据并调用数码管显示函数显示时间。

细心的朋友可能发现，在处理 DS1302 读取的数据时，取高低位是使用除 16 和取余 16，并非之前的除 10 和取余 10。这是因为写入进 DS1302 时是 BCD 码，读取出的数据也是 BCD 码，而 BCD 码即是 4 位表示一个十进制数，类似于一个字节的十六进制数据的高 4 位和低 4 位一样，所以这里是除 16 和取余 16。

25.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：数码管上显示电子时钟时分秒，格式为“XX-XX-XX”。



课后作业

普中51单片

第 26 章 红外遥控实验

在前面章节，我们介绍的通信，如 SPI、IIC 等通信均为有线，这一章我们来介绍一种无线通信技术--红外遥控通信。我们开发板标配了一个一体化红外接收头和红外遥控器，我们来学习如何使用 51 单片机解码红外遥控器的信号。本章使用 51 单片机的外部中断功能来解码红外遥控器的编码信号。本章要实现的功能是：使用外部中断功能将遥控器键值编码数据解码后通过数码管显示。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

26.1 红外遥控介绍

26.2 硬件设计

26.3 软件设计

26.4 实验现象

普中51单片机开发板

普中51单片机开发板

26.1 红外遥控介绍

26.1.1 红外线简介

人的眼睛能看到的可见光按波长从长到短排列，依次为红、橙、黄、绿、青、蓝、紫。其中红光的波长范围为 0.62～0.76μm；紫光的波长范围为 0.38～0.46μm。比紫光波长还短的光叫紫外线，比红光波长还长的光叫红外线。红外线遥控就是利用波长为 0.76～1.5μm 之间的近红外线来传送控制信号的。

26.1.2 红外遥控的原理

红外遥控是一种无线、非接触控制技术，具有抗干扰能力强，信息传输可靠，功耗低，成本低，易实现等显著优点，被诸多电子设备特别是家用电器广泛采用，

并越来越多的应用到计算机系统中。

由于红外线遥控不具有像无线电遥控那样穿过障碍物去控制被控对象的能力，所以，在设计红外线遥控器时，不必要像无线电遥控器那样，每套(发射器和接收器)要有不同的遥控频率或编码(否则，就会隔墙控制或干扰邻居的家用电器)，所以同类产品的红外线遥控器，可以有相同的遥控频率或编码，而不会出 现遥控信号“串门”的情况。这对于大批量生产以及在家用电器上普及红外线遥控提供了极大的方便。由于红外线为不可见光，因此对环境影响很小，再由红外光波动波长远小于无线电波的波长，所以红外线遥控不会影响其他家用电器，也不会影响临近的无线电设备。

红外遥控通信系统一般由红外发射装置和红外接收设备两大部分组成。

26.1.2.1 红外发射装置

红外发射装置，也就是通常我们说的红外遥控器是由键盘电路、红外编码电路、电源电路和红外发射电路组成。红外发射电路的主要元件为红外发光二极管。它实际上是一只特殊的发光二极管；由于其内部材料不同于普通发光二极管，因而在其两端施加一定电压时，它便发出的是红外线而不是可见光。目前大量的使用的红外发光二极管发出的红外线波长为 940nm 左右，外形与普通发光二极管相同。红外发光二极管有透明的，还有不透明的，在我们的红外遥控器上可以看到这个红外发光二极管。红外遥控器和红外发光二极管如下图所示：

通常红外遥控为了提高抗干扰性能和降低电源消耗，红外遥控器常用载波的方式传送二进制编码，常用的载波频率为 38kHz，这是由发射端所使用的 455kHz 晶振来决定的。在发射端要对晶振进行整数分频，分频系数一般取 12，所以 455kHz÷12≈37.9kHz≈38kHz。也有一些遥控系统采用 36kHz、 40 kHz、 56 kHz

等，一般由发射端晶振的振荡频率来决定。所以，通常的红外遥控器是将遥控信号（二进制脉冲码）调制在 38KHz 的载波上，经缓冲放大后送至红外发光二极管，转化为红外信号发射出去的。

二进制脉冲码的形式有多种，其中最为常用的是 NEC Protocol 的 PWM 码(脉冲宽度调制)和 Philips RC-5 Protocol 的 PPM 码(脉冲位置调制码，脉冲串之间的时间间隔来实现信号调制)。如果要开发红外接收设备，一定要知道红外遥控器的编码方式和载波频率，我们才可以选取一体化红外接收头和制定解码方案。我们配套的红外遥控器使用的是 NEC 协议，其特征如下：

1、8 位地址和 8 位指令长度；

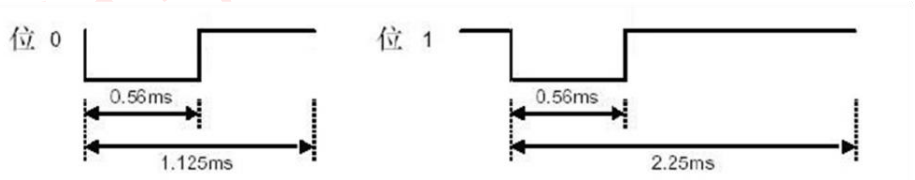
2、地址和命令 2 次传输（确保可靠性）

3、PWM 脉冲位置调制，以发射红外载波的占空比代表“0”和“1”；

4、载波频率为 38Khz；

5、位时间为 1.125ms 或 2.25ms；

NEC 码的位定义：一个脉冲对应 560us 的连续载波，一个逻辑 1 传输需要 2.25ms(560us 脉冲+1680us 低电平)，一个逻辑 0 的传输需要 1.125ms（560us 脉冲+560us 低电平）。而红外接收头在收到脉冲的时候为低电平，在没有脉冲的时候为高电平，这样，我们在接收头端收到的信号为：逻辑 1 应该是 560us 低+1680us 高，逻辑 0 应该是 560us 低+560us 高。所以可以通过计算高电平时间判断接收到的数据是 0 还是 1。NEC 码位定义时序图如下图所示：



NEC 遥控指令的数据格式为：引导码、地址码、地址反码、控制码、控制反码。引导码由一个 9ms 的低电平和一个 4.5ms 的高电平组成，地址码、地址反码、控制码、控制反码均是 8 位数据格式。按照低位在前，高位在后的顺序发送。采用反码是为了增加传输的可靠性（可用于校验）。数据格式如下：

NEC 码还规定了连发码(由 9ms 低电平+2.5m 高电平+0.56ms 低电平+97.94ms 高电平组成)，如果在一帧数据发送完毕之后，红外遥控器按键仍然没有放开，则发射连发码，可以通过统计连发码的次数来标记按键按下的长短或次数。

26.1.2.2 红外接收设备

红外接收设备是由红外接收电路、红外解码、电源和应用电路组成。红外遥控接收器的主要作用是将遥控发射器发来的红外光信好转换成电信号，再放大、限幅、检波、整形，形成遥控指令脉冲，输出至遥控微处理器。近几年不论是业余制作还是正式产品，大多都采用成品红外接收头。成品红外接收头的封装大致有两种:一种采用铁皮屏蔽;一种是塑料封装。均有三只引脚，即电源正(VDD)、电源负（GND）和数据输出（VOUT）。其外观实物图如下图所示：

正对接收头的凸起处看，从左至右，管脚依次是 1：VOUT，2：GND，3：VDD。

由于红外接收头在没有脉冲的时候为高电平，当收到脉冲的时候为低电平，所以可以通过外部中断的下降沿触发中断，在中断内通过计算高电平时间来判断接收到的数据是 0 还是 1。外部中断的相关知识，大家可以参考前面的“外部中断实验”，如果想要更加深入的了解红外遥控通信，大家可以百度，网上这方面的资料可谓是铺天盖地

26.2 硬件设计

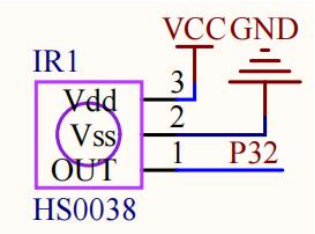
本实验使用到硬件资源如下：

（1）动态数码管

（2）红外接收头和遥控器

动态数码管电路在前面章节都介绍过，这里就不再重复。红外遥控器和接收

头是一体的，内部结构不用去管它。下面来看下开发板上红外接收模块电路，如下图所示：



从上图中可知，红外接收头的输出管脚接至单片机 P3.2 管脚上，为了保证红外接收头输出管脚默认为高电平，需外接一个 10K 上拉电阻，可在上图并没有看到有上拉电阻，在前面介绍最小系统时已知道，单片机 IO 口都增加了 10K 上拉电阻，所以此处可省略。

26.3 软件设计

本章所要实现的功能是：数码管上显示红外解码遥控器键值。

程序框架如下：

（1）编写数码管显示功能

（2）编写红外解码函数

（3）编写主函数

前面的实验章节都已编写过数码管显示功能，现在对大家来说应该不是问题，所以本章软件的重点在红外解码上。下面我们打开“\4--实验程序\1--基础实验\22-红外遥控实验”工程，在 App 工程组中可以看到新添加了 ired.c 文件（里面包含了红外解码驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

26.3.1 红外初始化函数

打开 ired.c 文件，红外初始化代码如下：

```
/*************************************************************************
***********
* 函 数 名         : ired_init
* 函数功能         : 红外端口初始化函数，外部中断 0 配置
* 输    入         : 无
```

```
* 输    出         : 无
*****************************************************************************
**********/
void ired_init(void)
{
    IT0=1; //下降沿触发
    EX0=1; //打开中断 0 允许
    EA=1;  //打开总中断
    IRED=1;    //初始化端口
}
```

因为我们使用外部中断 0 来解码红外遥控数据，所以需初始化配置外部中断 0。初始化配置在外部中断实验章节已详细介绍，此处不再重复。

26.3.2 红外解码函数

初始化外部中断后，中断就已经开启了，当 P32 引脚来一个下降沿，就会触发一次中断，在中断内我们可以计算高电平时间，通过高电平时间判断是否进入引导码及数据 0 和 1。具体代码如下：

```
void ired() interrupt 0  //外部中断 0 服务函数
{
    u8 ired_high_time=0;
    u16 time_cnt=0;
    u8 i=0,j=0;

    if(IRED==0)
    {
        time_cnt=1000;
        while((!IRED)&&(time_cnt))//等待引导信号 9ms 低电平结束，若超过 10ms
强制退出
        {
            delay_10us(1);//延时约 10us
            time_cnt--;
            if(time_cnt==0)return;
        }
        if(IRED)//引导信号 9ms 低电平已过，进入 4.5ms 高电平
        {
            time_cnt=500;
            while(IRED&&time_cnt)//等待引导信号 4.5ms 高电平结束，若超过 5ms 强
制退出
            {
```

```
                delay_10us(1);
                time_cnt--;
                if(time_cnt==0)return;
            }
            for(i=0;i<4;i++)//循环 4 次，读取 4 个字节数据
            {
                for(j=0;j<8;j++)//循环 8 次读取每位数据即一个字节
                {
                    time_cnt=600;
                    while((IRED==0)&&time_cnt)//等待数据 1 或 0 前面的 0.56ms
结束，若超过 6ms 强制退出
                    {
                        delay_10us(1);
                        time_cnt--;
                        if(time_cnt==0)return;
                    }
                    time_cnt=20;
                    while(IRED)//等待数据 1 或 0 后面的高电平结束，若超过 2ms 强制
退出
                    {
                        delay_10us(10);//约 0.1ms
                        ired_high_time++;
                        if(ired_high_time>20)return;
                    }
                    gired_data[i]>>=1;//先读取的为低位，然后是高位
                    if(ired_high_time>=8)//如果高电平时间大于 0.8ms，数据则为
1，否则为 0
                        gired_data[i]|=0x80;
                    ired_high_time=0;//重新清零，等待下一次计算时间
                }
            }
        }
        if(gired_data[2]!=~gired_data[3])//校验控制码与反码，错误则返回
        {
            for(i=0;i<4;i++)
                gired_data[i]=0;
            return;
        }
    }
}
```

进入中断函数，表示以来下降沿，然后判断管脚是否为低电平，如果为低电平则首先判断引导信号，根据前面 NEC 协议可知，引导信号有 9ms 的低电平和

4.5ms 的高电平，因此通过 time_cnt 赋值 1000，然后在 while 循环内判断，time_cnt 每递减一次约 10us，1000 次则为 10ms，在解码时，这个时间要适当放宽一点范围，因为不同传感器性能会有差异，所以此处以 10ms 的低电平为界限，如果超过 10ms 则强制退出，防止系统死机。判断完引导信号的低电平，接着判断高电平，实现方法一样。当引导信号判断完成后进入地址码、地址反码、控制码及控制反码共 4 个字节的数据判断，也就是数据 0 和 1 的判断，实现方法也是和前面判断引导信号一样，这里使用到了嵌套循环，外层循环次数是 4，表示读取 4 个字节，内层循环次数是 8，表示读取每个字节的 8 位。注意，红外遥控解码数据是从低位开始，最后是高位。

最后将读取的 4 个字节数据存储在全局变量数组 gired_data 中，外部可直接使用这四个字节。

26.3.3 主函数

打开 main.c 文件，代码如下：

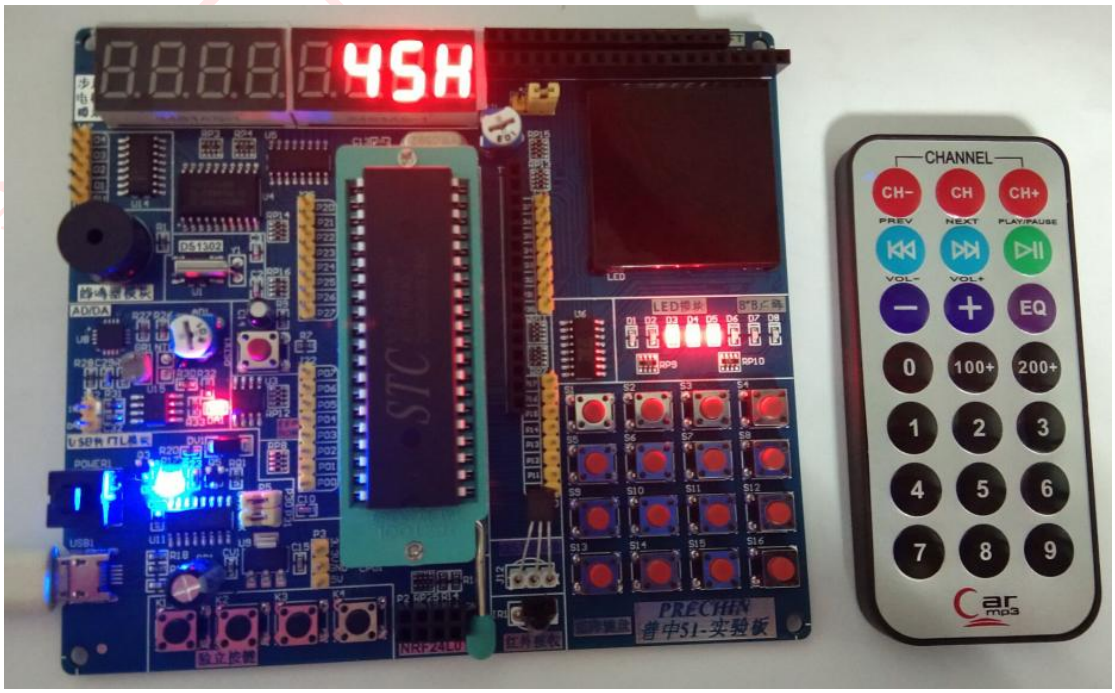
```
/**********************************************************************
******************
实验名称：红外遥控实验
接线说明：
实验现象：下载程序后，数码管上显示数码管上显示红外解码遥控器键值
注意事项：红外接收头凸起处要与 PCB 板接口凸起丝印处对应

**********************************************************************
******************/
#include "public.h"
#include "smg.h"
#include "ired.h"


/**********************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
**********************************************************************
**********/
```

```
void main()
{
    u8 ired_buf[3];

    ired_init();//红外初始化

    while(1)
    {
        ired_buf[0]=gsmg_code[gired_data[2]/16];//将控制码高 4 位转换为数码管段码
        ired_buf[1]=gsmg_code[gired_data[2]%16];//将控制码低 4 位转换为数码管段码
        ired_buf[2]=0X76;//显示 H 的段码
        smg_display(ired_buf,6);
    }
}
```

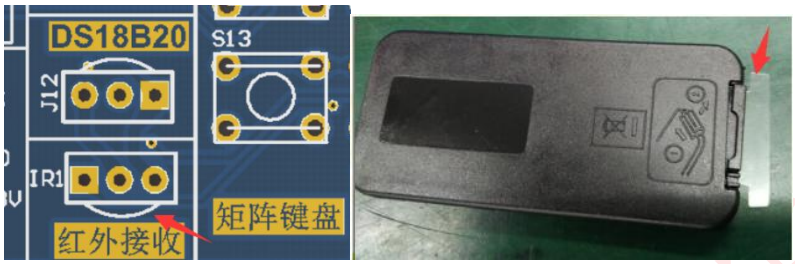
主函数代码很简单，首先调用使用的外设头文件，然后初始化红外，进入 while 循环将红外解码数据转换为数码管段码数据，然后在数码管上显示。

26.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：数码管上显示红外解码遥控器键值。



注意：红外接收头的凸起位置需对到 PCB 板丝印凸出位置，否则功能无法正常实现。出厂时的遥控器电池盖内都有一张绝缘片，使用时必须将此绝缘片拿掉。还有键值码不是大家所想的按键上的数字字符码，这个要区分开。



实验说明：我们配的遥控器所有键的地址码与反码都是相同的，不同的是控制码和控制反码，其实知道了控制码就知道了控制反码，所以如果要使用红外遥控控制其他设备，可以通过区分控制码来实现。

如果程序下载了，能显示“00H”，按键时却没有反应，首先检查遥控器是否拔下绝缘片。如果要检查遥控器好坏，可以在光线暗的地方，打开手机摄像机，将遥控器前方的发射灯对准手机摄像头，按下按键，在手机上是否能看见灯闪，来判断遥控器好坏。

课后作业

第 27 章 ADC 模数转换实验

在前面章节，我们介绍的都是对数字信号的操作也就是 0 和 1 操作，本章我们就来讲解如何检测外部模拟信号，因为我们使用的 STC89C5x 单片机内部不含 ADC 接口，所以需要外接一个 ADC 转换芯片将模拟信号转换成数字信号供单片机处理。我们开发板上集成了一个 ADC 模数转换电路，选用的 ADC 芯片是 12 位的 AD 芯片-XPT2046。本章要实现的功能是：通过 ADC 转换电路采集电位器电压值，将采集转换后的 AD 值通过数码管显示。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

27.1 ADC 介绍

27.2 XPT2046 芯片介绍

27.3 硬件设计

27.4 软件设计

27.5 实验现象

普中51单片机开发板

27.1 ADC 介绍

我们知道 51 单片机系统内部运算时用的全部是数字量，即 0 和 1，因此对单片机系统而言，无法直接操作模拟量，必须将模拟量转换成数字量。所谓数字量，就是用一系列 0 和 1 组成的二进制代码表示某个信号大小的量。用数字量表示同一个模拟量时，数字位数可以多也可以少，位数越多则表示的精度越高，位数越少表示的精度就越低。

27.1.1 ADC 简介

ADC（analog to digital converter）也称为模数转换器，是指一个将模拟

信号转变为数字信号。单片机在采集模拟信号时，通常都需要在前端加上 A/D 芯片。

下面我们看下 ADC 的主要技术指标：

（1）分辨率

ADC 的分辨率是指对于允许范围内的模拟信号，它能输出离散数字信号值的个数。这些信号值通常用二进制数来存储，因此分辨率经常用比特作为单位，且这些离散值的个数是 2 的幂指数。

例如：12 位 ADC 的分辨率就是 12 位，或者说分辨率为满刻度的 1/(2^12)。一个 10V 满刻度的 12 位 ADC 能分辨输入电压变化最小值是

$$10V \times 1/(2^{12}) = 2.4mV。$$

（2）转换误差

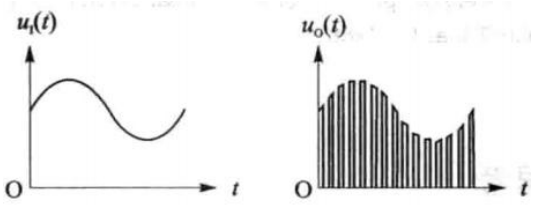
转换误差通常是以输出误差的最大值形式给出。它表示 A/D 转换器实际输出的数字量和理论上的输出数字量之间的差别。常用最低有效位的倍数表示。例如给出相对误差≤±LSB/2，这就表明实际输出的数字量和理论上应得到的输出数字量之间的误差小于最低位的半个字。

（3）转换速率

ADC 的转换速率是能够重复进行数据转换的速度，即每秒转换的次数。而完成一次 A/D 转换所需的时间（包括稳定时间），则是转换速率的倒数。

27.1.2 ADC 转换原理

AD 转换器（ADC）将模拟量转换为数字量通常要经过 4 个步骤：采样、保持、量化和编码。所谓采样即是将一个时间上连续变化的模拟量转换为时间上离散变化的模拟量。如下图所示：



将采样结果存储起来，直到下次采样，这个过程叫做保持。一般采样器和保持电路一起总称为采样保持电路。将采样电平归化为与之接近的离散数字电平，

这个过程叫做量化。将量化后的结果按照一定数制形式表示就是编码。将采样电平（模拟值）转换为数字值时，主要有两类方法：直接比较型与间接比较型。

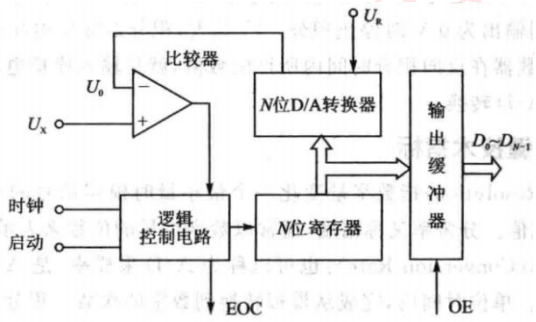
--直接比较型：就是将输入模拟信号直接与标准的参考电压比较，从而得到数字量。常见的有并行 ADC 和逐次比较型 ADC。

--间接比较型：输入模拟量不是直接与参考电压比较，而是将二者变为中间的某种物理量在进行比较，然后将比较所得的结果进行数字编码。常见的有双积分型 ADC。

下面就以常用的逐次比较型 ADC 和双积分型 ADC 介绍其工作原理。

（1）逐次逼近型 ADC

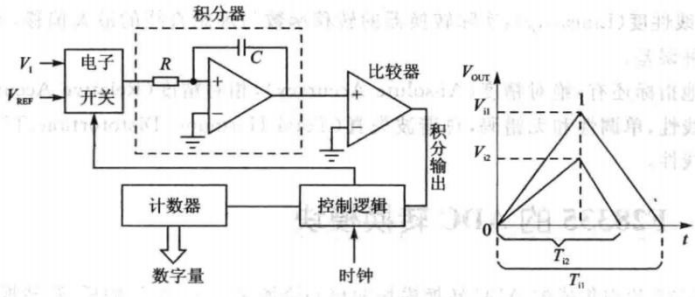
采用逐次逼近法的 AD 转换器是有一个比较器、DA 转换器、缓冲寄存器和控制逻辑电路组成，如下图所示：



基本原理是：从高位到低位逐次试探比较，就像用天平秤物体，从重到轻逐级增减砝码进行试探。逐次逼近法的转换过程是：初始化时将逐次逼近寄存器各位清零，转换开始时，先将逐次逼近寄存器最高位置 1，送入 DA 转换器，经 DA 转换后生成的模拟量送入比较器，称为 UO，与送入比较器的待转换的模拟量 Ux 进行比较，若 UO<Ux，该位 1 被保留，否则被清除。然后再将逐次逼近寄存器次高位置 1,将寄存器中新的数字量送 DA 转换器,输出的 UO 再与 Ux 比较,若 UO<Ux，该位 1 被保留，否则被清除。重复此过程，直至逼近寄存器最低位。转换结束后，将逐次逼近寄存器中的数字量送入缓冲寄存器，得到数字量的输出。逐次逼近的操作过程是在一个控制电路的控制下进行的。

（2）双积分型 ADC

采用双积分法的 AD 转换器由电子开关、积分器、比较器和控制逻辑等部件组成。如下图所示：



其基本原理：将输入电压变换成与其平均值成正比的时间间隔，再把此时间间隔转换成数字量，属于间接转换。双积分法 AD 转换的过程是：先将开关接通待转换的模拟量 Vi，Vi 采样输入到积分器，积分器从零开始进行固定时间 T 的正向积分，时间 T 到后，开关再接通与 Vi 极性相反的基准电压 Vref，将 Vref 输入到积分器，进行反向积分，直到输出为 0V 时停止积分。Vi 越大，积分器输出电压越大，反向积分时间也越长。计数器在反向积分时间内所计的数值，就是输入模拟电压 Vi 所对应的数字量，实现了 AD 转换。

27.2 XPT2046 芯片介绍

XPT2046 是一款 4 线制电阻式触摸屏控制器，内含 12 位分辨率 125KHz 转换速率逐步逼近型 A/D 转换器。XPT2046 支持从 1.5V 到 5.25V 的低电压 I/O 接口。XPT2046 能通过执行两次 A/D 转换查出被按的屏幕位置，除此之外，还可以测量加在触摸屏上的压力。内部自带 2.5V 参考电压，可以作为辅助输入、温度测量和电池监测之用，电池监测的电压范围可以从 0V 到 6V。XPT2046 片内集成有一个温度传感器。在 2.7V 的典型工作状态下，关闭参考电压，功耗可小于 0.75mW。XPT2046 采用微小的封装形式：TSSOP-16,QFN-16 和 VFBGA－48。工作温度范围为-40℃～+85℃。与 ADS7846、TSC2046、AK4182A 完全兼容。

（1）主要特性

①工作电压范围为 1.5V～5.25V

②支持 1.5V～5.25V 的数字 I/O 口

③内建 2.5V 参考电压源

④电源电压测量（ 0V~6V）

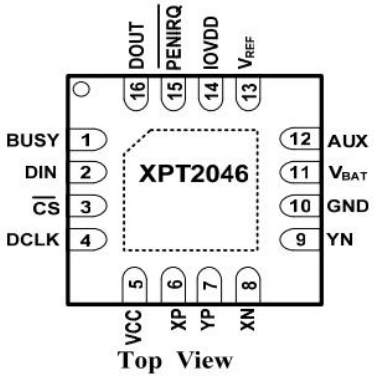
⑤内建结温测量功能

⑥触摸压力测量

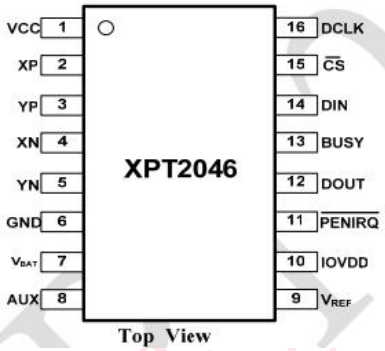
⑦采用 3 线制 SPI 通信接口

⑧具有自动省电功能

（2）芯片管脚说明



Top View



Top View

QFN引脚号	TSSOP引脚号	VFBGA引脚号	名称	说明
1	13	A5	BUSY	忙时信号线。当CS为高电平时为高阻状态
2	14	A4	DIN	串行数据输入端。当CS为低电平时，数据在DCLK上升沿锁存进来
3	15	A3	CS	片选信号。控制转换时序和使能串行输入输出寄存器，高电平时ADC掉电
4	16	A2	DCLK	外部时钟信号输入
5	1	B1和C1	VCC	电源输入端
6	2	D1	XP	XP位置输入端
7	3	E1	YP	YP位置输入端
8	4	G2	XN	XN位置输入端
9	5	G3	YN	YN位置输入端
10	6	G4和G5	GND	接地
11	7	G6	V_{BAT}	电池监视输入端
12	8	E7	AUX	ADC辅助输入通道
13	9	D7	V_{REF}	参考电压输入/输出
14	10	C7	IOVDD	数字电源输入端
15	11	B7	PENIRQ	笔接触中断引脚
16	12	A6	DOUT	串行数据输出端。数据在DCLK的下降沿移出，当CS高电平时为高阻状态

XPT2046 是一种典型的逐次逼近型模数转换器（SAR ADC），包含了采样/保持、模数转换、串口数据 输出等功能。同时芯片集成有一个 2.5V 的内部参考电压源、温度检测电路，工作时使用外部时钟。XPT2046 可以单电源供电，电源电压范围为 2.7V～5.5V。参考电压值直接决定 ADC 的输入范围，参考电压可以使用内部参考电压，也可以从外部直接输入 1V～VCC 范围内的参考电压（要求外部参考电压源输出阻抗低）。X、Y、Z、VBAT、Temp 和 AUX 模拟信号经过片内的控制寄存器选择后进入 ADC，ADC 可以配置为单端或差分模式。选择 VBAT、Temp

和 AUX 时应该配置为单端模式；作为触摸屏应用时，应该配置为差分模式，这可有效消除由于驱动开关的寄生电阻及外部的干扰带来的测量误差，提高转换精度。单端和差分模式输入配置如下图所示：

表3 单端模式输入配置（SER/$\overline{DFR}$=1）

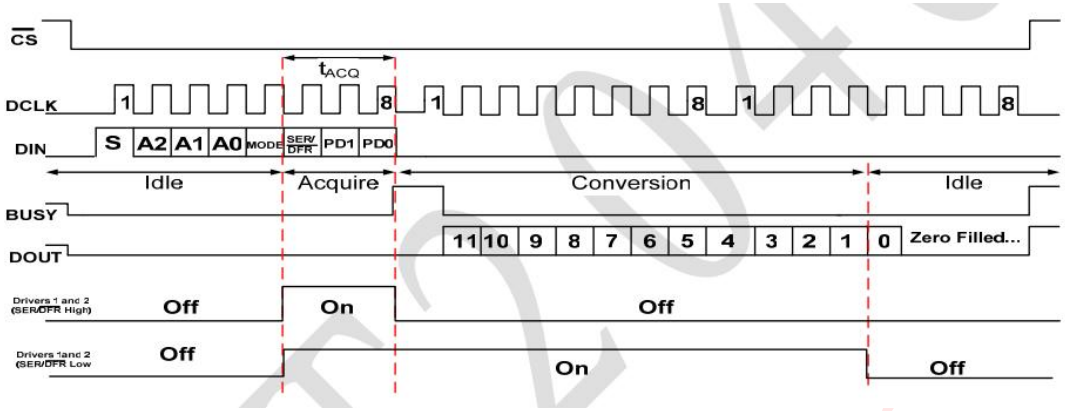
A2	A1	A0	V_{BAT}	AUX_{IN}	TEMP	YN	XP	YP	Y-位置	X-位置	Z_1-位置	Z_2-位置	X-驱动	Y-驱动
0	0	0			+IN (TEMP0)								off	off
0	0	1					+IN		测量				Off	On
0	1	0	+IN										Off	Off
0	1	1					+IN				测量		XN, On	YP, On
1	0	0				+IN						测量	XN, On	YP, On
1	0	1						+IN		测量			On	Off
1	1	0		+IN									Off	Off
1	1	1			+IN (TEMP1)								Off	Off

表4 差分模式输入配置（SER/$\overline{DFR}$=0）

A2	A1	A0	+REF	-REF	YN	XP	YP	Y-位置	X-位置	Z_1-位置	Z_2-位置	驱动
0	0	1	YP	YN		+IN		测量				YP, YN
0	1	1	YP	XN		+IN				测量		YP, XN
1	0	0	YP	XN	+IN						测量	YP, XN
1	0	1	XP	XN			+IN		测量			XP, XN

XPT2046 数据接口是串行接口，其典型工作时序如下图所示，图中展示的信号来自带有基本串行接口的单片机或数据信号处理器。处理器和转换器之间的的通信需要 8 个时钟周期，可采用 SPI、SSI 和 Microwire 等同步串行接口。一次完整的转换需要 24 个串行同步时钟（DCLK）来完成。

前 8 个时钟用来通过 DIN 引脚输入控制字节。当转换器获取有关下一次转换的足够信息后，接着根据获得的信息设置输入多路选择器和参考源输入，并进入采样模式，如果需要，将启动触摸面板驱动器。3 个多时钟周期后，控制字节设置完成，转换器进入转换状态。这时，输入采样－保持器进入保持状态，触摸面板驱动器停止工作（单端工作模式）。接着的 12 个时钟周期将完成真正的模数转换。如果是度量比率转换方式（SER/DFR＝0），驱动器在转换过程中将一直工作，第 13 个时钟将输出转换结果的最后一位。剩下的 3 个多时钟周期将用来完成被转换器忽略的最后字节（DOUT 置低）。



在对 XPT2046 进行控制时，控制字节由 DIN 输入的控制字命令格式如下所示：

表5 制字的控制位命令

位 7 (MSB)	位 6	位 5	位 4	位 3	位 2	位 1	位 0(LSB)
S	A2	A1	A0	MODE	SER/$\overline{DFR}$	PD1	PD0

表6 控制字节各位描述

位	名称	功能描述
7	S	开始位。为 1 表示一个新的控制字节到来，为 0 则忽略 PIN 引脚上数据
6-4	A2-A0	通道选择位。参见表 1 和表 2
3	MODE	12 位/8 位转换分辨率选择位。为 1 选择 8 位为转换分辨率，为 0 选择 12 位分辨率
2	SER/$\overline{DFR}$	单端输入方式/ 差分输入方式选择位。为 1 是单端输入方式，为 0 是差分输入方式
1-0	PD1-PD0	低功率模式选择位。若为 11，器件总处于供电状态;若为 00，器件在变换之间处于低功率模式

注意：差分模式仅用于 X 坐标、Y 坐标和触摸压力的测量，其它测量要求采用单端模式。

表7 掉电和内部参考电压选择

PD1	PD0	$\overline{PENIRQ}$	功能说明
0	0	使能	在两次 A / D 转换之间掉电，下次转换一开始，芯片立即进入完全上电状态，而无需额外的延时。在这种模式下，YN 开关一直处于 ON 状态
0	1	禁止	参考电压关闭，ADC 打开
1	0	使能	参考电压打开，ADC 关闭
1	1	禁止	芯片处于上电状态，参考电压和 ADC 总是打开

至此，我们就对 ADC 的基本原理及 XPT2046 芯片介绍完，大家如果想要更详细的了解其工作原理，可以查看资料“\6--开发板芯片资料\开发板芯片数据手册\xpt2046 中文.pdf”。

27.3 硬件设计

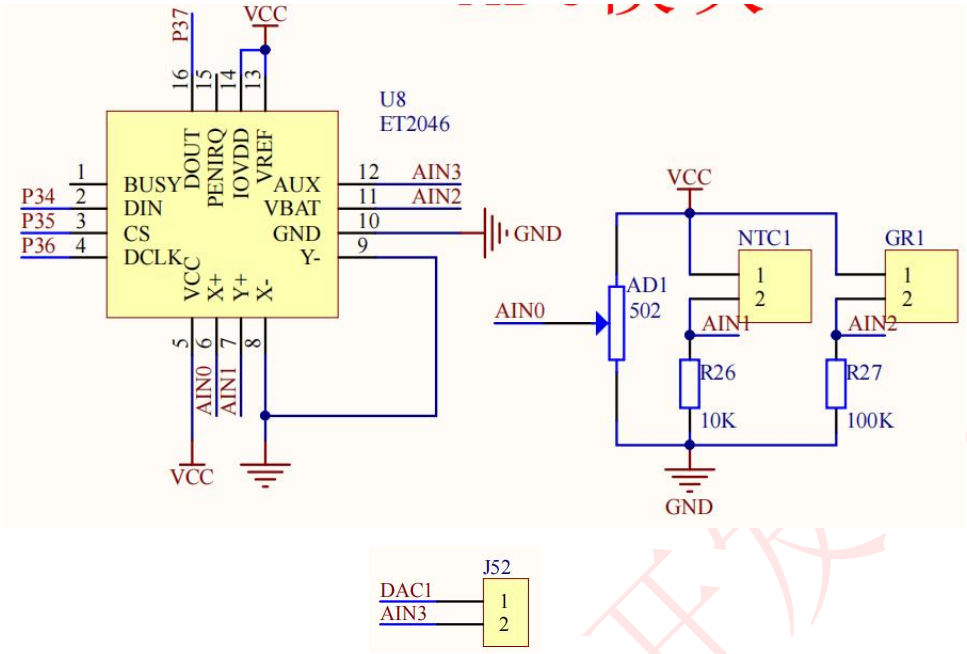
本实验使用到硬件资源如下：

（1）动态数码管

（2）ADC 模块

动态数码管电路在前面章节都介绍过，这里就不再重复。下面来看下开发板

上 ADC 模块电路，如下图所示：



从上图中可以看出，该电路是集成的，XPT2046 芯片的控制管脚接至单片机 P3.4~P3.7 管脚上，XPT2046 芯片的 ADC 输入转换通道分别接入了 AD1 电位器、NTC1 热敏传感器、GR1 光敏传感器，还有一个外接通道 AIN3 接在 DAC（PWM）模块的 J52 端子上供外部模拟信号检测。

注意：上图中 XPT2046 的 DOUT 脚与 DS18B20 温度传感器均连接到单片机的 P3.7，因此该两个外设资源不能同时使用，可以分时复用。

27.4 软件设计

本章所要实现的功能是：数码管上显示 AD 模块采集电位器的电压值。

程序框架如下：

（1）编写数码管显示功能

（2）编写 ADC 转换函数

（3）编写主函数

前面的实验章节都已编写过数码管显示功能，现在对大家来说应该不是问题，所以本章软件的重点在 XPT2046 读写数据上。下面我们打开“\4--实验程序\1--基础实验\23-ADC 模数转换实验\23.1-电位器电压采集”工程，在 App 工程组中可以看到新添加了 xpt2046.c 文件（里面包含了 XPT2046 驱动程序），还要

包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

27.4.1 ADC 读写函数

打开 xpt2046.c 文件，代码如下：

```
#include "xpt2046.h"
#include "intrins.h"

/*****************************************************************************
***********
* 函 数 名       : xpt2046_wirte_data
* 函数功能       : XPT2046 写数据
* 输    入       : dat：写入的数据
* 输    出       : 无
*****************************************************************************
**********/
void xpt2046_wirte_data(u8 dat)
{
    u8 i;

    CLK = 0;
    _nop_();
    for(i=0;i<8;i++)//循环 8 次，每次传输一位，共一个字节
    {
        DIN = dat >> 7;//先传高位再传低位
        dat <<= 1;//将低位移到高位
        CLK = 1;//CLK 由低到高产生一个上升沿，从而写入数据
        _nop_();
        CLK = 0;
        _nop_();
    }
}

/*****************************************************************************
***********
* 函 数 名       : xpt2046_read_data
* 函数功能       : XPT2046 读数据
* 输    入       : 无
* 输    出       : XPT2046 返回 12 位数据
*****************************************************************************
**********/
```

```
u16 xpt2046_read_data(void)
{
    u8 i;
    u16 dat=0;

    CLK = 0;
    _nop_();
    for(i=0;i<12;i++)//循环 12 次，每次读取一位，大于一个字节数，所以返回值类型
是 u16
    {
        dat <<= 1;
        CLK = 1;
        _nop_();
        CLK = 0; //CLK 由高到低产生一个下降沿，从而读取数据
        _nop_();
        dat |= DOUT;//先读取高位，再读取低位。
    }
    return dat;
}

/**********************************************************************
***********
* 函 数 名       : xpt2046_read_adc_value
* 函数功能       : XPT2046 读 AD 数据
* 输    入       : cmd：指令
* 输    出       : XPT2046 返回 AD 值
**********************************************************************
**********/
u16 xpt2046_read_adc_value(u8 cmd)
{
    u8 i;
    u16 adc_value=0;

    CLK = 0;//先拉低时钟
    CS  = 0;//使能 XPT2046
    xpt2046_wirte_data(cmd);//发送命令字
    for(i=6; i>0; i--);//延时等待转换结果
    CLK = 0;
    _nop_();
    CLK = 1;//发送一个时钟，清除 BUSY
    _nop_();
    adc_value=xpt2046_read_data();
    CS = 1;//关闭 XPT2046
    return adc_value;
```

```
}
```

根据前面 XPT2046 时序图可知，XPT2046 完成一个完整的转换需要 24 个串行时钟，也就是需要 3 个字节的 SPI 时钟。XPT2046 前 8 个串行时钟，是接收 1 个字节的转换命令，接收到转换命令了之后，然后使用 1 个串行时钟的时间来完成数据转换（当然在编写程序的时候，为了得到精确的数据，你可以适当的延时一下），然后返回 12 个字节长度（12 个字节长度也计时 12 个串行时钟）的转换结果。然后最后 4 个串行时钟返回 4 个无效数据，可以忽略。xpt2046_read_adc_value 函数就是按照这个时序实现。至于 xpt2046_wirte_data 和 xpt2046_read_data 函数是 IO 口模拟的 SPI 读写时序，与 DS1302 的读写是一样的。

上述代码使用的管脚在 xpt2046 头文件已定义，这里就不列出。

27.4.2 主函数

打开 main.c 文件，代码如下：

```
/**************************************************************************
******************
实验名称：ADC 模数转换实验--电位器电压采集
接线说明：
实验现象：下载程序后，数码管上显示 AD 模块采集电位器的电压值
注意事项：

**************************************************************************
******************/
#include "public.h"
#include "smg.h"
#include "xpt2046.h"


/**************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
**************************************************************************
**********/
void main()
```

```
{
    u16 adc_value=0;
    float adc_vol;//ADC 电压值
    u8 adc_buf[3];

    while(1)
    {
        adc_value=xpt2046_read_adc_value(0x94);//测量电位器
        adc_vol=5.0*adc_value/4096;//将读取的 AD 值转换为电压
        adc_value=adc_vol*10;//放大 10 倍，即保留小数点后一位
        adc_buf[0]=gsmg_code[adc_value/10]|0x80;
        adc_buf[1]=gsmg_code[adc_value%10];
        adc_buf[2]=0x3e;//显示单位 V
        smg_display(adc_buf,6);
    }
}
```

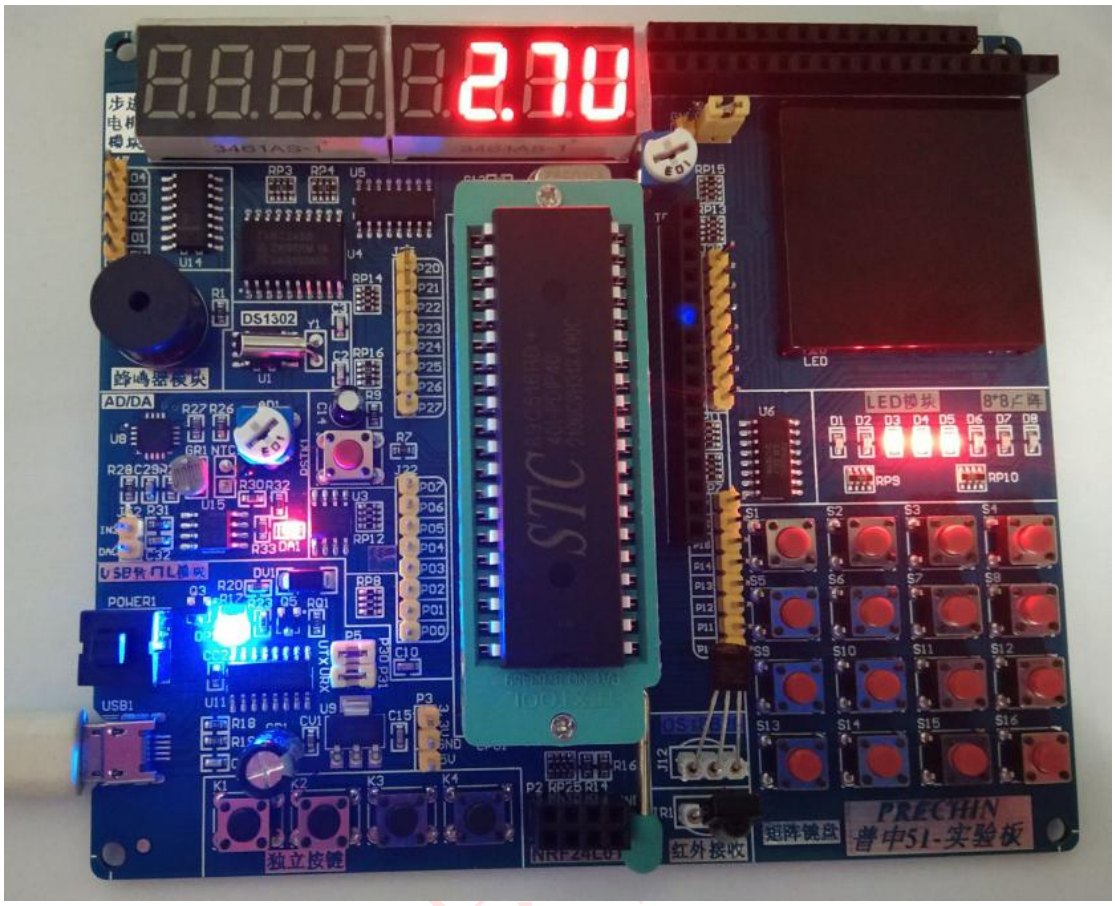
主函数代码非常简单，首先调用外设头文件，然后定义一些变量存储 AD 值和电压值，进入 while 循环，读取电位器的 AD 值，根据 XPT2046 单端模式输入配置表可知，要采集 X+脚通道的信号，则配置值为 0X94。得到 AD 值后，可按照如下公式计算电压值：Vref*ADC_Value/分辨率。

其中 Vref 是参考电压，XPT2046 接入的是 5V，ADC_Value 是读取的 AD 值，分辨率是 ADC 的位数（2^{12}=4096）。

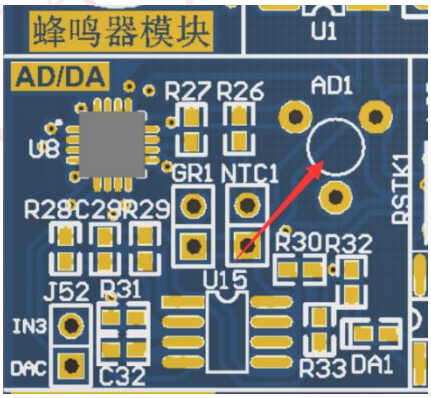
经过公式计算后会得到小数，所以定义了一个 float 型变量 adc_vol 存储，然后将该值放大 10 倍，目的是保留小数后一位。最后将得到的电压值转换为数码管段码数据显示。

27.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：数码管上显示 AD 模块采集电位器的电压值。



注意：调节电位器 AD1 时，数码管上显示的电压值也将变化。



对于光敏电阻和热敏电阻以及 AIN3 外部采集通道实验，大家可以直接打开实验工程，它们与电位器电压采集程序相比仅仅是配置命令不同而已。

电位器对应的采集通道命令：0X94

光敏电阻对应的采集通道命令：0XA4

热敏电阻对应的采集通道命令：0XD4

外部输入 AIN3 对应的采集通道命令：0XE4

课后作业

普中51单片机开发板

第 28 章 DAC 数模转换实验

在前面章节，我们介绍了如何使用 51 单片机来采集外部模拟信号，这一章来介绍下如何使用 51 单片机输出模拟信号，要让 51 单片机输出模拟信号，同样需要相应的转换器，这种转换器我们称之为 DAC 数模转换器，但 DAC 专用芯片价格是比较高的，在实际应用中通常使用 PWM 技术来模拟 DAC 输出。我们开发板上集成了一个 DAC(PWM)模块电路，通过单片机 IO 口生成 PWM 波来模拟 DAC 输出。本章要实现的功能是：系统运行时，DAC(PWM)模块上的指示灯 DA1 呈呼吸灯效果，由暗变亮再由亮变暗。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

28.1 DAC 介绍

28.2 PWM 介绍

28.3 硬件设计

28.4 软件设计

28.5 实验现象

普中51单片机开发板

28.1 DAC 介绍

28.1.1 DAC 简介

DAC（Digital to analog converter）即数字模拟转换器，它可以将数字信号转换为模拟信号。它的功能与 ADC 相反。在常见的数字信号系统中，大部分传感器信号被转化成电压信号，而 ADC 把电压模拟信号转换成易于计算机存储、处理的数字编码，由计算机处理完成后，再由 DAC 输出电压模拟信号，该电压模拟信号常常用来驱动某些执行器件，使人类易于感知。如音频信号的采集及还原就是这样一个过程。

DAC 的主要技术指标如下：

（1）分辨率

DAC 的分辨率是输入数字量的最低有效位（LSB）发生变化时，所对应的输出模拟量（电压或电流）的变化量。它反映了输出模拟量的最小变化值。分辨率与输入数字量的位数有确定的关系，可以表示成 FS／（2^n）。FS 表示满量程输入值，n 为二进制位数。对于 5V 的满量程，采用 8 位的 DAC 时，分辨率为 5V/256＝19.5mV；当采用 12 位的 DAC 时，分辨率则为 5V/4096＝1.22mV。显然，位数越多分辨率就越高。

（2）线性度

线性度（也称非线性误差）是实际转换特性曲线与理想直线特性之间的最大偏差。常以相对于满量程的百分数表示。如± 1 %是指实际输出值与理论值之差在满刻度的± 1 %以内。

（3）绝对精度和相对精度

绝对精度（简称精度）是指在整个刻度范围内，任一输入数码所对应的模拟量实际输出值与理论值之间的最大误差。绝对精度是由 DAC 的增益误差（当输入数码为全 1 时，实际输出值与理想输出值之差）、零点误差（数码输入为全 0 时，DAC 的非零输出值）、非线性误差和噪声等引起的。绝对精度（即最大误差）应小于 1 个 LSB。相对精度与绝对精度表示同一含义，用最大误差相对于满刻度的

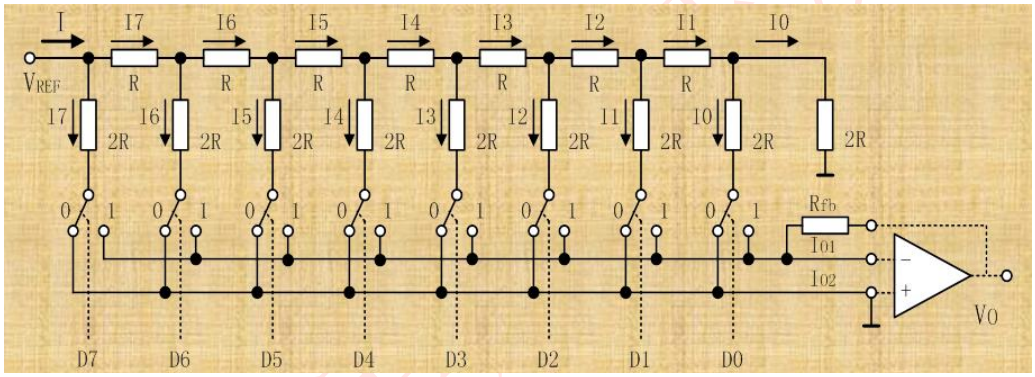
百分比表示。

（4）建立时间

建立时间是指输入的数字量发生满刻度变化时，输出模拟信号达到满刻度值的±1/2LSB 所需的时间。是描述 D/A 转换速率的一个动态指标。根据建立时间的长短，可以将 DAC 分成超高速（＜1μS)、高速（10～1μS）、中速（100～10μS）、低速（≥100μS）几档。

28.1.2 DAC 工作原理

了解了 DAC 基本概念及特性后，再来看下其工作原理，下面以 T 型电阻网络 DAC 来介绍。其内部结构图如下所示：



DAC 输出电压计算公式：VO=Vref*z/256

公式中的 z 表示单片机给的数字量，vref 为参考电压，通常我们是接在系统电源上，即 5V，数值 256 表示 DAC 精度为 8 位。

DAC 主要由数字寄存器、模拟电子开关、位权网络、求和运算放大器和基准电压源（或恒流源）组成。用存于数字寄存器的数字量的各位数码，分别控制对应位的模拟电子开关，使数码为 1 的位在位权网络上产生与其位权成正比的电流值，再由运算放大器对各电流值求和，并转换成电压值。

上述的模拟电子开关都分别接着一个分压的器件，比如说电阻。模拟开关的个数取决于 DAC 的精度。那么 N 个电子开关就把基准电压分为 N 份（并不是平均分哦），而这些开关根据输入的二进制每一位数据对应开启或者关闭，把分压的器件上的电压引入输出电路中。

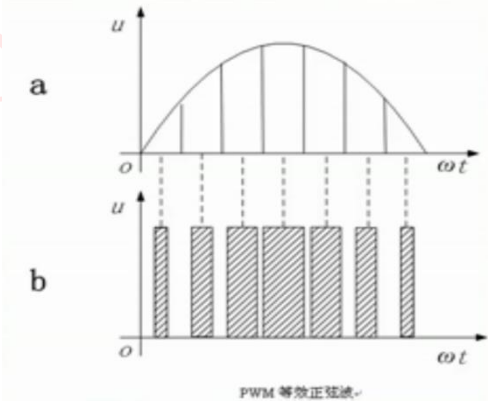
28.2 PWM 介绍

出于成本考虑，在实际开发应用中，使用较多的是通过 PWM 来模拟 DAC 输出。下面就先来了解下 PWM 相关概念。

PWM 是 Pulse Width Modulation 的缩写，中文意思就是脉冲宽度调制，简称脉宽调制。它是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，其控制简单、灵活和动态响应好等优点而成为电力电子技术最广泛应用的控制方式，其应用领域包括测量，通信，功率控制与变换，电动机控制、伺服控制、调光、开关电源，甚至某些音频放大器，因此学习 PWM 具有十分重要的现实意义。

其实我们也可以这样理解，PWM 是一种对模拟信号电平进行数字编码的方法。通过高分辨率计数器的使用，方波的占空比被调制用来对一个具体模拟信号的电平进行编码。PWM 信号仍然是数字的，因为在给定的任何时刻，满幅值的直流供电要么完全有(ON)，要么完全无(OFF)。电压或电流源是以一种通(ON)或断(OFF)的重复脉冲序列被加到模拟负载上去的。通的时候即是直流供电被加到负载上的时候，断的时候即是供电被断开的时候。只要带宽足够，任何模拟值都可以使用 PWM 进行编码。

PWM 对应模拟信号的等效图，如下图所示：

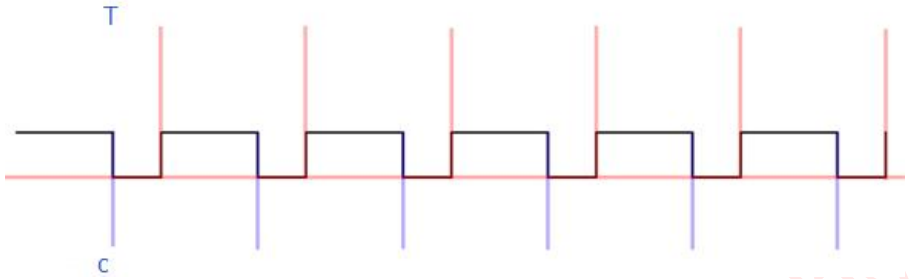


PWM 等效正弦波

从图中可以看到，上图 a 是一个正弦波即模拟信号，b 是一个数字脉冲波形即数字信号。我们知道在计算机系统中只能识别是 1 和 0，对于 51 单片机芯片，要么输出高电平（5V），要么输出低电平（0），假如要输出 1.5V 的电压，那么就必须通过相应的处理，比如本章所要讲解的 PWM 输出，其实从上图也可以看到，只要保证数字信号脉宽足够就可以使用 PWM 进行编码，从而输出

1.5V 的电压。

PWM 的输出其实就是对外输出脉宽可调（即占空比调节）的方波信号，信号频率是由 T 的值决定，占空比由 C 的值决定。其示意图如图所示：



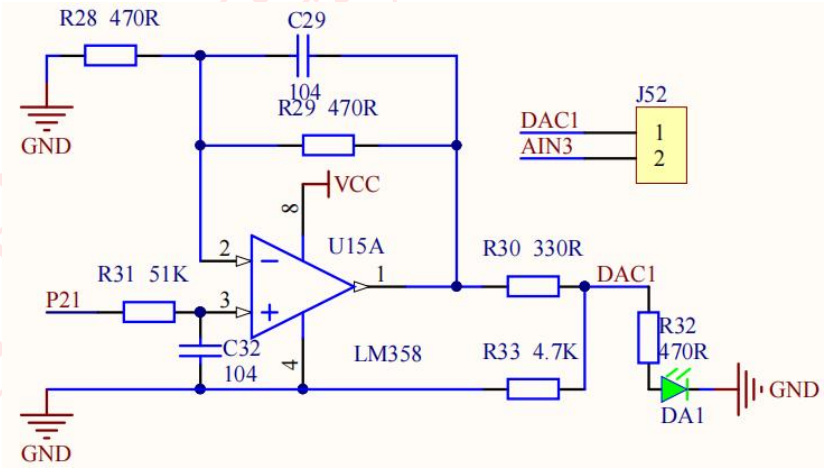
从上图中可以看到，PWM 输出频率是不变的，改变的是 C 的值，此值的改变将导致 PWM 输出信号占空比的改变。占空比其实就是一个周期内高电平时间与周期的比值。而频率的话可以使用 51 单片机的定时器确定。

28.3 硬件设计

本实验使用到硬件资源如下：

（1）DAC(PWM)模块

下面来看下开发板上 DAC(PWM)模块电路，如下图所示：



从上图中可知，PWM 输出控制管脚接在单片机 P2.1 管脚上，DAC1 为 PWM 输出信号，将其连接一个 LED，这样可以通过指示灯的状态直观的反映出 PWM 输出电压值变化。LM358 芯片与这些电容电阻构成了一个跟随电路，即输入是多少，输出即为多大电压，输出电压范围是 0-5V。输出信号由 J52 端子的 DAC1 引出，在其端子上还有一个 AIN3 脚，它是上一章介绍 ADC 时的外部模拟信号输入通道。

如果使用短接片将 DAC1 和 AIN3 短接，这样就可以使用 XPT2046 芯片采集检测 PWM 输出信号。

28.4 软件设计

本章所要实现的功能是：DAC(PWM)模块上的指示灯 DA1 呈呼吸灯效果，由暗变亮再由亮变暗。

程序框架如下：

（1）编写 PWM 函数

（2）编写主函数

本章软件的重点是如何让单片机 IO 口输出 PWM。下面我们打开“\4--实验程序\1--基础实验\24-DAC 数模转换实验”工程，在 App 工程组中可以看到新添加了 pwm.c 文件（里面包含了 PWM 驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

28.4.1 PWM 实现函数

PWM 的实现均在 pwm.c 文件中，代码如下：

```
#include "pwm.h"

//全局变量定义
u8 gtim_h=0;//保存定时器初值高 8 位
u8 gtim_l=0;//保存定时器初值低 8 位
u8 gduty=0;//保存 PWM 占空比
u8 gtim_scale=0;//保存 PWM 周期=定时器初值*tim_scale


/*********************************************************************
***********
* 函 数 名       : pwm_init
* 函数功能       : PWM 初始化函数
* 输    入       : tim_h：定时器高 8 位
                   tim_l：定时器低 8 位
                   tim_scale：PWM 周期倍数：定时器初值*tim_scale
                   duty：PWM 占空比（要小于等于 tim_scale）
* 输    出       : 无
*********************************************************************
```

```
**********/
void pwm_init(u8 tim_h,u8 tim_l,u16 tim_scale,u8 duty)
{
    gtim_h=tim_h;//将传入的初值保存在全局变量中，方便中断函数继续调用
    gtim_l=tim_l;
    gduty=duty;
    gtim_scale=tim_scale;

    TMOD|=0X01;    //选择为定时器 0 模式，工作方式 1
    TH0 = gtim_h; //定时初值设置
    TL0 = gtim_l;
    ET0=1;//打开定时器 0 中断允许
    EA=1;//打开总中断
    TR0=1;//打开定时器
}

/*****************************************************************************
***********
* 函 数 名       : pwm_set_duty_cycle
* 函数功能       : PWM 设置占空比
* 输    入       : duty：PWM 占空比（要小于等于 tim_scale）
* 输    出       : 无
*****************************************************************************
**********/
void pwm_set_duty_cycle(u8 duty)
{
    gduty=duty;
}

void pwm(void) interrupt 1   //定时器 0 中断函数
{
    static u16 time=0;

    TH0 = gtim_h; //定时初值设置
    TL0 = gtim_l;

    time++;
    if(time>=gtim_scale)//PWM 周期=定时器初值*gtim_scale，重新开始计数
        time=0;
    if(time<=gduty)//占空比
        PWM=1;
    else
        PWM=0;
}
```

上述代码主要是基于定时器实现 PWM 输出，PWM 初始化实际上为定时器 0 初始化，pwm_init 函数有 4 个入口参数，tim_h 和 tim_l 为定时器定时初值，即进入中断时间；tim_scale 参数为 PWM 的周期倍数，使用该值乘以定时器初值可得出 PWM 的周期；duty 参数为 PWM 占空比，即一个周期内高电平所占的时间比例。在 pwm 初始化函数内，将函数入口参数通过全局变量保存，方便在后续中断函数内使用。

pwm_set_duty_cycle 函数是占空比设置函数，该函数有一个入口参数，用于设置 PWM 占空比，注意，该值不能超过初始化中的 PWM 的周期倍数值。

最后就是定时器 0 的中断服务函数，在中断内定义了一个静态变量用于统计进入中断的次数时间，当进入中断次数时间大于等于 gtim_scale 周期倍数，则重新开始计数，表示 PWM 周期为定时器初值*gtim_scale；然后当计数次数时间小于等于设置的占空比次数时间，则使对应 IO 输出高电平，否则输出低电平。

28.4.2 主函数

打开 main.c 文件，代码如下：

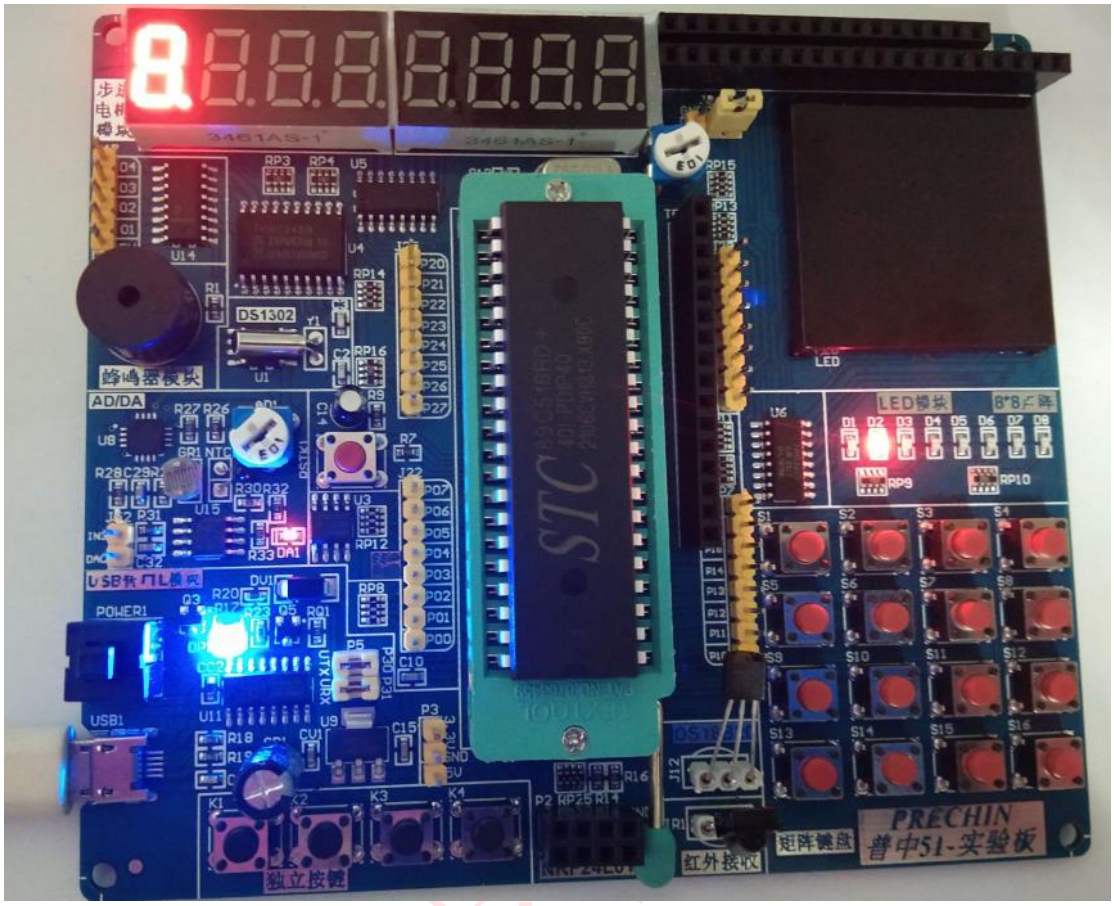
```
/**************************************************************************
******************
实验名称：DAC 模数转换实验
接线说明：
实验现象：下载程序后，DAC(PWM)模块上的指示灯 DA1 呈呼吸灯效果，由暗变亮再由亮变暗
注意事项：

**************************************************************************
******************/
#include "public.h"
#include "pwm.h"


/**************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
**************************************************************************
**********/
```

```
void main()
{
    u8 dir=0;//默认为 0
    u8 duty=0;

    pwm_init(0XFF,0XF6,100,0);//定时时间为 0.01ms，PWM 周期是
100*0.01ms=1ms，占空比为 0%

    while(1)
    {
        if(dir==0)//当 dir 为递增方向
        {
            duty++;//占空比递增
            if(duty==70)dir=1;//当到达一定值切换方向，占空比最大能到 100，但到
达 70 左右再递增，
                                //肉眼也分辨不出亮度变化
        }
        else
        {
            duty--;
            if(duty==0)dir=0;//当到达一定值切换方向
        }
        pwm_set_duty_cycle(duty);//设置占空比
        delay_ms(1);//短暂延时，让呼吸灯有一个流畅的效果
    }
}
```

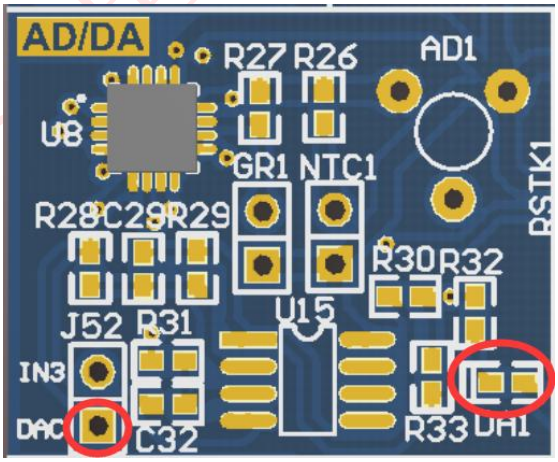
主函数比较简单，首先调用外设驱动头文件，然后进入主函数初始化 PWM，将定时器设置为 0.01ms，初值为 0XFFF6，即每隔 0.01ms 进入一次中断。PWM 周期倍数设置为 100，即 PWM 周期为 1ms，占空比设置为 0。最后进入 while 循环，通过 dir 切换方向实现 duty 值的自增和自减来调节占空比，将该值传入到占空比调节函数 pwm_set_duty_cycl。为了使呼吸灯流畅，每调节占空比短暂延时一下。

28.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：DAC(PWM)模块上的指示灯 DA1 呈呼吸灯效果，由暗变亮再由亮变暗。



注意：实验中可通过 DA1 指示灯观察 PWM 输出，也可使用万用表测量 DAC 和 GND 电压，观察电压变化。



课后作业

第 29 章 LCD1602 液晶显示实验

在前面章节，我们已经学习过几种显示装置，例如静态数码管、动态数码管、8*8LED 点阵、串口通信。使用它们可以直观显示一些字符数据，但是它们也有各种局限性，比如显示字符数据太少，硬件设计复杂、代码编写难度大等。这一章就来介绍一种非常简单且常用的显示装置--LCD1602 液晶显示器，使用它可以显示更多的字符数字。我们开发板上集成了一个 LCD1602 液晶显示器接口，将 LCD1602 液晶对应插入即可。本章要实现的功能是：系统运行时，在 LCD1602 液晶上显示字符信息。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

29.1 LCD1602 介绍

29.2 硬件设计

29.3 软件设计

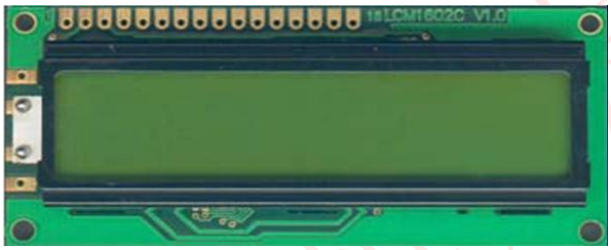
29.4 实验现象

普中51单片机开发板

29.1 LCD1602 介绍

29.1.1 LCD1602 简介

1602 液晶也叫 1602 字符型液晶，它能显示 2 行字符信息，每行又能显示 16 个字符。它是一种专门用来显示字母、数字、符号的点阵型液晶模块。它是由若干个 5x7 或者 5x10 的点阵字符位组成，每个点阵字符位都可以用显示一个字符，每位之间有一个点距的间隔，每行之间也有间隔，起到了字符间距和行间距的作用，正因为如此，所以它不能很好的显示图片。其实物图如下所示：



大家手上拿到的 LCD1602 外观可能和上图不一样，这是由于不同厂家设计所致，但使用方法是一样的。在上图中可以看到有 16 个管脚孔，从左至右管脚编号顺序是 1-16，其功能定义如下所示：

编号	符号	引脚说明	编号	符号	引脚说明
1	VSS	电源地	9	D2	Data I/O
2	VDD	电源正极	10	D3	Data I/O
3	VL	液晶显示偏压信号	11	D4	Data I/O
4	RS	数据/命令选择端（H/L）	12	D5	Data I/O
5	R/W	读/写选择端（H/L）	13	D6	Data I/O
6	E	使能信号	14	D7	Data I/O
7	D0	Data I/O	15	BLA	背光源正极
8	D1	Data I/O	16	BLK	背光源负极

下面对几个管脚做下说明：

3 脚：VL，液晶显示偏压信号，用于调整 LCD1602 的显示对比度，一般会外接电位器用以调整偏压信号，注意此脚电压为 0 时可以得到最强的对比度。

4 脚：RS，数据/命令选择端，当此脚为高电平时，可以对 1602 进行数据字节的传输操作，而为电平时，则是进行命令字节的传输操作。命令字节，即是用

来对 LCD1602 的一些工作方式作设置的字节；数据字节，即使用以在 1602 上显示的字节。值得一提的是，LCD1602 的数据是 8 位的。

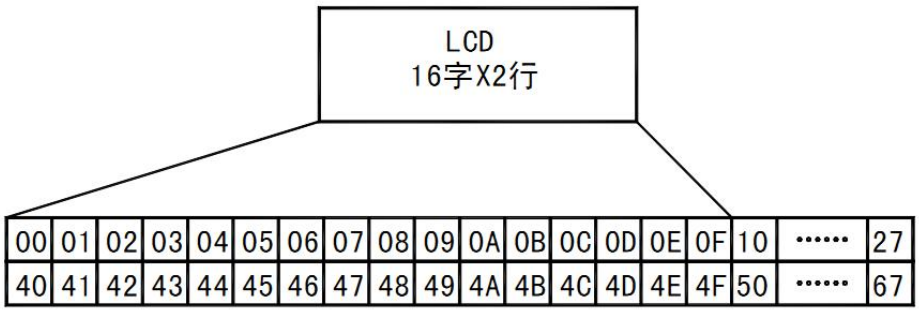
5 脚：R/W，读写选择端。当此脚为高电平可对 LCD1602 进行读数据操作，反之进行写数据操作。

6 脚：E，使能信号，其实是 LCD1602 的数据控制时钟信号，利用该信号的上升沿实现对 LCD1602 的数据传输。

7~14 脚：8 位并行数据口，而 51 单片机一组 IO 也是 8 位，使得对 LCD1602 的数据读写大为方便。

在 LCD1602 内部含有 80 个字节的 DDRAM，它是用来寄存显示字符的。其地址和屏幕的对应关系如下表：

	显示位置	1	2	3	4	5	6	7	……	40
DDRAM 地 址	第一行	00H	01H	02H	03H	04H	05H	06H	……	27H
	第二行	40H	41H	42H	43H	44H	45H	46H	……	67H



从上图可知，不是所有的地址都可以直接用来显示字符数据，只有第一行中的 00-0F，第二行中的 40-4F 才能显示，其他地址只能用于存储。要显示字符时要先输入显示字符地址，也就是告诉模块在哪里显示字符，例如第二行第一个字符的地址是 40H，那么是否直接写入 40H 就可以将光标定位在第二行第一个字符的位置呢？这样不行，因为写入显示地址时要求最高位 D7 恒定为高电平 1 所以实际写入的数据应该是 01000000B(40H）+10000000B(80H)=11000000B(C0H)。在 1602 中我们就用前 16 个就行了。第二行也一样用前 16 个地址。

29.1.2 LCD1602 常用指令

在使用 LCD1602 时，我们需要掌握一些常用的指令，这些指令对于 LCD1602 初始化是必须的。

（1）清屏指令

指令功能	指令编码										执行时间 /ms
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
清屏	0	0	0	0	0	0	0	0	0	1	1.64

功能：

<1> 清除液晶显示器，即将 DDRAM 的内容全部填入"空白"的 ASCII 码 20H;

<2> 光标归位，即将光标撤回液晶显示屏的左上方;

<3> 将地址计数器(AC)的值设为 0。

（2）模式设置指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
进入模式设置	0	0	0	0	0	0	0	1	I/D	S	40

功能：

设定每次写入 1 位数据后光标的移位方向，并且设定每次写入的一个字符是否移动。

I/D：0=写入新数据后光标左移　　1=写入新数据后光标右移

S：0=写入新数据后显示屏不移动　　1=写入新数据后显示屏整体右移 1 个字符

（3）显示开关控制指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
显示开关控制	0	0	0	0	0	0	1	D	C	B	40

功能：

控制显示器开/关、光标显示/关闭以及光标是否闪烁。

D：0=显示功能关　　1=显示功能开

C：0=无光标　　1=有光标

B：0=光标闪烁　　1=光标不闪烁

（4）功能设定指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
功能设定	0	0	0	0	1	DL	N	F	X	X	40

功能：

设定数据总线位数、显示的行数及字型。

DL：0=数据总线为 4 位　　1=数据总线为 8 位

N：0=显示 1 行　　1=显示 2 行

F：0=5×7 点阵/每字符　　1=5×10 点阵/每字符

29.1.3 LCD1602 使用

要使用 LCD1602，首先需要对其初始化，即通过写入一些特定的指令实现。然后选择要在 LCD1602 的哪个位置显示并将所要显示的数据发送到 LCD 的 DDRAM。使用 LCD1602 通常都是用于写数据进去，很少使用读功能。LCD1602 操作步骤如下所示：

（1）初始化

（2）写命令（RS=L），设置显示坐标

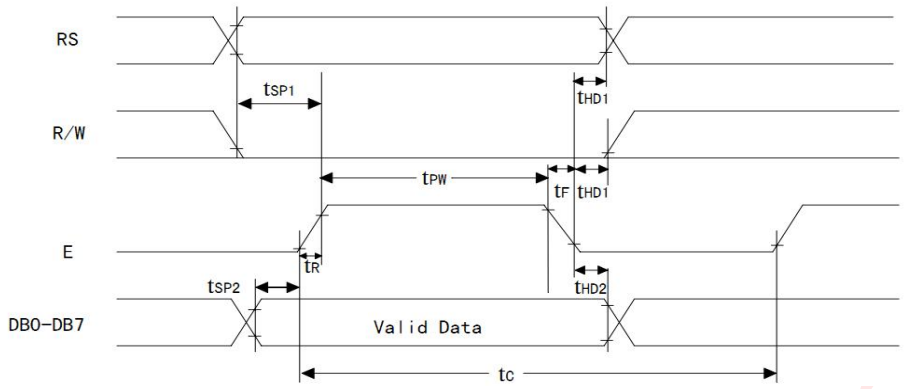
（3）写数据（RS=H）

在此，不需要读出它的数据的状态或者数据本身。所以只需要看两个写时序：

①当要写指令字，设置 LCD1602 的工作方式时：需要把 RS 置为低电平，RW 置为低电平，然后将数据送到数据口 D0~D7，最后 E 引脚一个高脉冲将数据写入。

②当要写入数据字，在 1602 上实现显示时：需要把 RS 置为高电平，RW 置为低电平，然后将数据送到数据口 D0~D7，最后 E 引脚一个高脉冲将数据写入。

写指令和写数据，差别仅仅在于 RS 的电平不一样而已。以下是 LCD1602 的时序图：



时序参数	符号	极限值			单位
		最小值	典型值	最大值	
E 信号周期	t_C	400	–	–	ns
E 脉冲宽度	t_{PW}	150	–	–	ns
E 上升沿/下降沿时间	t_R, t_F	–	–	25	ns
地址建立时间	t_{SP1}	30	–	–	ns
地址保持时间	t_{HD1}	10	–	–	ns
数据建立时间(读操作)	t_D	–	–	100	ns
数据保持时间(读操作)	t_{HD2}	20	–	–	ns
数据建立时间(写操作)	t_{SP2}	40	–	–	ns
数据保持时间(写操作)	t_{HD2}	10	–	–	ns

从上图可以看到，以上给的时间参数全部是 ns 级别的，而 51 单片机的机器周期是 1us，指令周期是 2-4 个机器周期，所以即便在程序里不加延时程序，也可以很好的配合 LCD1602 的时序要求了。

当要写命令字节的时候，时间由左往右，RS 变为低电平，R/W 变为低电平，注意看是 RS 的状态先变化完成。然后这时，DB0~DB7 上数据进入有效阶段，接着 E 引脚有一个整脉冲的跳变，接着要维持时间最小值为 tpw=400ns 的 E 脉冲宽度。然后 E 引脚负跳变，RS 电平变化，R/W 电平变化。这样便是一个完整的 LCD1602 写命令的时序。

注意：这里介绍的是 8 位 LCD1602，现在某些公司为简化引脚数，使用 4 位 LCD1602。使用 4 位 LCD1602 时，应该多看手册，找到不同点，对原有程序加以修改。我们例程也做了 4 位和 8 位 LCD1602 的兼容，具体后面会介绍。

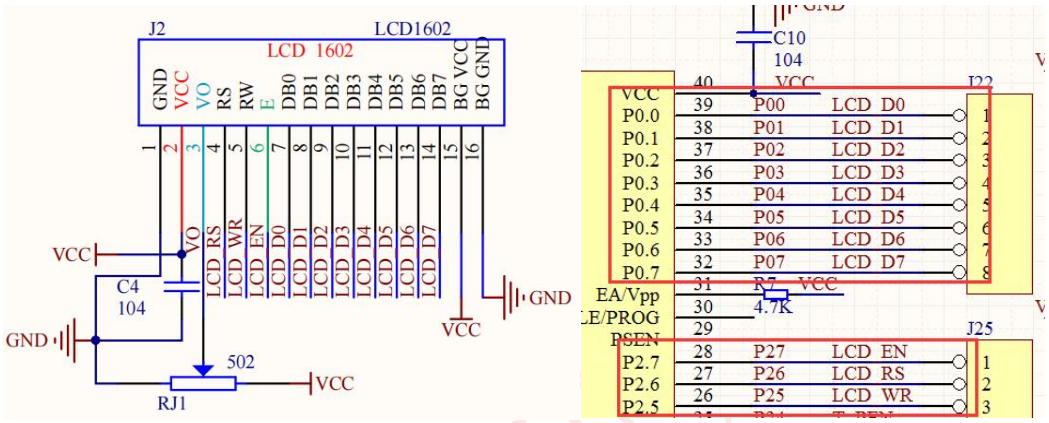
至此，我们就把 LCD1602 介绍完了，大家如果想要更详细的了解它，可以查看资料“\6--芯片资料\开发板芯片数据手册\LCD1602 液晶完整中文资料.pdf”。

29.2 硬件设计

本实验使用到硬件资源如下：

（1）LCD1602 液晶

开发板上集成了一个 LCD1602 液晶接口，下面来看下开发板上 LCD1602 液晶接口电路，如下图所示：



从上图中可知，LCD1602 的 8 位数据口 DB0-DB7 与单片机的 P0.0-P0.7 管脚连接，LCD1602 的 RS、RW、E 脚与单片机的 P2.6、P2.5、P2.7 管脚连接。RJ1 是一个电位器，用来调节 LCD1602 对比度即显示亮度。

注意：这里原理图是使用的 8 位 LCD1602 接口设计，是可以兼容 4 位 LCD1602 的。对于非标准接口的 LCD1602，我们也会通过转接板将其转接为对应开发板接口的。对于 4 位 LCD1602 在传输数据的时候需要将 8 位的数据截成两段，先发送高四位，在发送低四位。其它引脚操作方法不变。

29.3 软件设计

本章所要实现的功能是：在 LCD1602 液晶上显示字符信息。

程序框架如下：

（1）编写 LCD1602 显示函数

（2）编写主函数

本章软件的重点是如何对 LCD1602 进行写命令和数据。下面我们打开“\4--实验程序\1--基础实验\25-LCD1602 液晶显示实验”工程，在 App 工程组中可以看到新添加了 lcd1602.c 文件（里面包含了 LCD1602 驱动程序），还要包含对应

的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

29.3.1 LCD1602 驱动函数

打开 lcd1602.c 文件，代码如下：

```
#include "lcd1602.h"



/*********************************************************************
***********
* 函 数 名       : lcd1602_write_cmd
* 函数功能       : LCD1602 写命令
* 输    入       : cmd：指令
* 输    出       : 无
*********************************************************************
**********/
#if (LCD1602_4OR8_DATA_INTERFACE==0)//8 位 LCD
void lcd1602_write_cmd(u8 cmd)
{
    LCD1602_RS=0;//选择命令
    LCD1602_RW=0;//选择写
    LCD1602_E=0;
    LCD1602_DATAPORT=cmd;//准备命令
    delay_ms(1);
    LCD1602_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD1602_E=0;//使能脚 E 后负跳变完成写入
}
#else  //4 位 LCD
void lcd1602_write_cmd(u8 cmd)
{
    LCD1602_RS=0;//选择命令
    LCD1602_RW=0;//选择写
    LCD1602_E=0;
    LCD1602_DATAPORT=cmd;//准备命令
    delay_ms(1);
    LCD1602_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD1602_E=0;//使能脚 E 后负跳变完成写入

    LCD1602_DATAPORT=cmd<<4;//准备命令
```

```
    delay_ms(1);
    LCD1602_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD1602_E=0;//使能脚 E 后负跳变完成写入
}
#endif

/*********************************************************************
***********
* 函 数 名       : lcd1602_write_data
* 函数功能       : LCD1602 写数据
* 输    入       : dat: 数据
* 输    出       : 无
*********************************************************************
**********/
#if (LCD1602_4OR8_DATA_INTERFACE==0)//8 位 LCD
void lcd1602_write_data(u8 dat)
{
    LCD1602_RS=1;//选择数据
    LCD1602_RW=0;//选择写
    LCD1602_E=0;
    LCD1602_DATAPORT=dat;//准备数据
    delay_ms(1);
    LCD1602_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD1602_E=0;//使能脚 E 后负跳变完成写入
}
#else
void lcd1602_write_data(u8 dat)
{
    LCD1602_RS=1;//选择数据
    LCD1602_RW=0;//选择写
    LCD1602_E=0;
    LCD1602_DATAPORT=dat;//准备数据
    delay_ms(1);
    LCD1602_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD1602_E=0;//使能脚 E 后负跳变完成写入

    LCD1602_DATAPORT=dat<<4;//准备数据
    delay_ms(1);
    LCD1602_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD1602_E=0;//使能脚 E 后负跳变完成写入
```

```
}
#endif

/*********************************************************************
***********
* 函 数 名       : lcd1602_init
* 函数功能       : LCD1602 初始化
* 输    入       : 无
* 输    出       : 无
*********************************************************************
**********/
#if (LCD1602_4OR8_DATA_INTERFACE==0)//8 位 LCD
void lcd1602_init(void)
{
    lcd1602_write_cmd(0x38);//数据总线 8 位，显示 2 行，5*7 点阵/字符
    lcd1602_write_cmd(0x0c);//显示功能开，无光标，光标闪烁
    lcd1602_write_cmd(0x06);//写入新数据后光标右移，显示屏不移动
    lcd1602_write_cmd(0x01);//清屏
}
#else
void lcd1602_init(void)
{
    lcd1602_write_cmd(0x28);//数据总线 4 位，显示 2 行，5*7 点阵/字符
    lcd1602_write_cmd(0x0c);//显示功能开，无光标，光标闪烁
    lcd1602_write_cmd(0x06);//写入新数据后光标右移，显示屏不移动
    lcd1602_write_cmd(0x01);//清屏
}
#endif

/*********************************************************************
***********
* 函 数 名       : lcd1602_clear
* 函数功能       : LCD1602 清屏
* 输    入       : 无
* 输    出       : 无
*********************************************************************
**********/
void lcd1602_clear(void)
{
    lcd1602_write_cmd(0x01);
}

/*********************************************************************
***********
```

```
* 函 数 名         : lcd1602_show_string
* 函数功能         : LCD1602 显示字符
* 输    入         : x,y: 显示坐标，x=0~15，y=0~1;
                     str: 显示字符串
* 输    出         : 无
*******************************************************************************
**********/
void lcd1602_show_string(u8 x,u8 y,u8 *str)
{
    u8 i=0;

    if(y>1||x>15)return;//行列参数不对则强制退出

    if(y<1)    //第 1 行显示
    {
        while(*str!='\0')//字符串是以'\0'结尾，只要前面有内容就显示
        {
            if(i<16-x)//如果字符长度超过第一行显示范围，则在第二行继续显示
            {
                lcd1602_write_cmd(0x80+i+x);//第一行显示地址设置
            }
            else
            {
                lcd1602_write_cmd(0x40+0x80+i+x-16);//第二行显示地址设置

            }
            lcd1602_write_data(*str);//显示内容
            str++;//指针递增
            i++;
        }
    }
    else   //第 2 行显示
    {
        while(*str!='\0')
        {
            if(i<16-x) //如果字符长度超过第二行显示范围，则在第一行继续显示
            {
                lcd1602_write_cmd(0x80+0x40+i+x);
            }
            else
            {
                lcd1602_write_cmd(0x80+i+x-16);
            }
            lcd1602_write_data(*str);
```

```
            str++;
            i++;
        }
    }
}
```

在代码中可以看到有如下格式：

```
#if (LCD1602_4OR8_DATA_INTERFACE==0)
...（省略内容 1）
#else
...（省略内容 2）
#endif
```

这是 C 语言中的条件编译，如果 LCD1602_4OR8_DATA_INTERFACE 值等于 0，则执行省略内容 1 部分，否则执行省略内容 2 部分，这个和 if...else 是类似的。而 LCD1602_4OR8_DATA_INTERFACE 是在 lcd1602.h 中定义好的宏，用于选择使用的 LCD1602 是 8 位数据口还是 4 位数据口，通过这个宏就可以兼容 8 位和 4 位 2 款 LCD1602 驱动，非常方便。如果大家觉得代码过长，可以将兼容的一块去掉，这样代码就简短明了。

lcd1602_write_cmd 函数是 LCD1602 写命令，lcd1602_write_data 函数是 LCD1602 写数据，它们之间的区别仅在于控制脚 RS 的值，如果为 0 则写命令，为 1 则写数据。对于 8 位数据和 4 位数据口的驱动，不同之处在于 8 位数据是一次性传输到端口，而 4 位数据口需要将一个字节数据分两次传输，高 4 位和低 4 位。

lcd1602_init 函数是 LCD1602 初始化函数，根据其命令设置相应的值，开启 LCD1602 显示。

lcd1602_clear 函数是 LCD1602 清屏函数，实际上也是调用 lcd1602_write_cmd 函数写 0X01 命令。

lcd1602_show_string 函数是 LCD1602 显示字符函数，该函数有 3 个入口参数，x 和 y 用来确定显示的坐标，x 范围是 0-15，y 范围是 0-1。str 参数是一个指针类型变量，用于传递显示的内容。函数内首先判断 xy 坐标是否有效，无效则强制退出，然后根据 xy 值在对应的坐标位置显示内容。

29.3.2 主函数

打开 main.c 文件，代码如下：

```
/************************************************************************
******************
实验名称：LCD1602 液晶显示实验
接线说明：
实验现象：下载程序后，LCD1602 上显示字符信息
注意事项：

************************************************************************
******************/
#include "public.h"
#include "lcd1602.h"


/************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
************************************************************************
**********/
void main()
{
    lcd1602_init();//LCD1602 初始化
    lcd1602_show_string(0,0,"Hello World!");//第一行显示
    lcd1602_show_string(0,1,"0123456789");//第二行显示
    while(1)
    {

    }
}
```

主函数代码非常简单，首先调用外设驱动头文件，然后进入 main 函数初始化 LCD1602，并调用 LCD1602 显示函数让第一行显示"Hello World!"，第二行显示"0123456789"。

这里说明下，在开发中通常会显示数值，比如温度数据，程序中我们获取的温度数据通常存储在 float 变量中，并非字符串，此时需要想办法将 float 变量中的值转换为字符串，比如 float temp=31.2，那么我们可以再定义一个数组

temp_buf[]用于保存 temp 中的每一个位，操作如下：

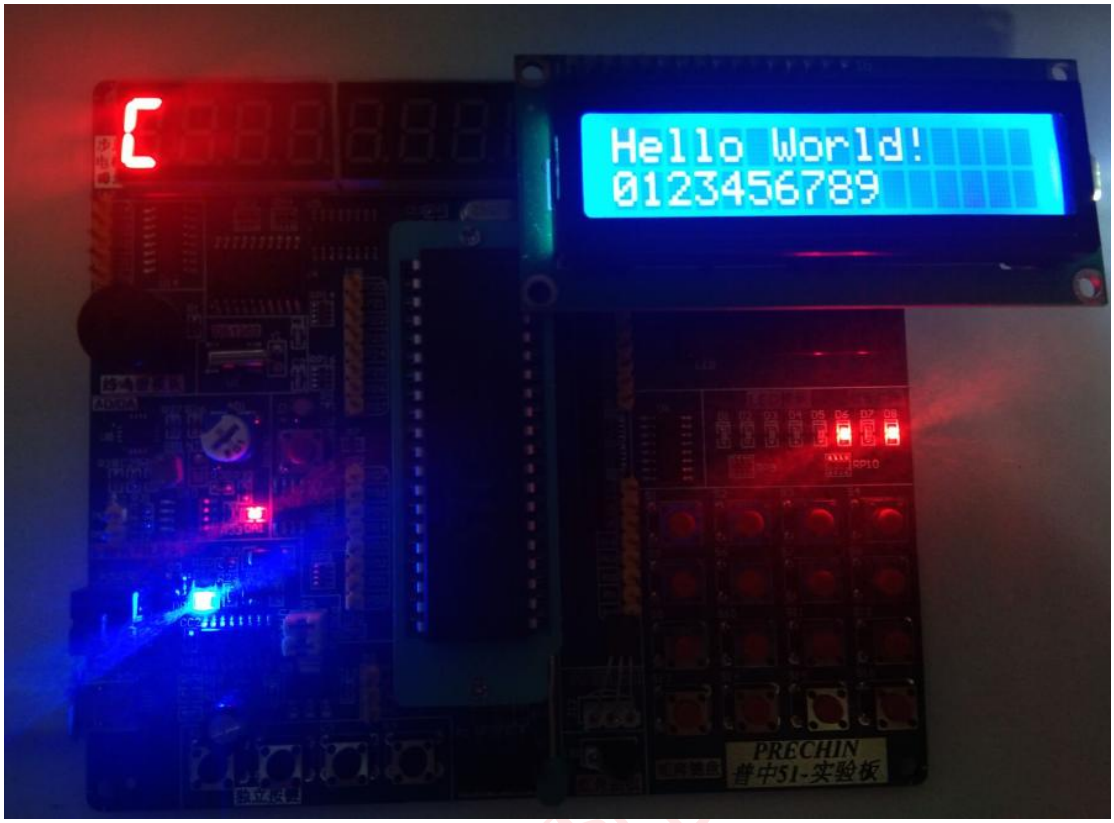
```
float temp=31.2;
u8 temp_buf[5];
u16 temp_value=temp*10;

temp_buf[0]=temp_value/100+0x30;
temp_buf[1]=temp_value%100/10+0x30;
temp_buf[2]='.';
temp_buf[3]=temp_value%100%10+0x30;
temp_buf[4]='\0';

lcd1602_show_string(0,0,temp_buf);
```

将浮点数转换为整型变量，然后将整型值的每一个位取出来存储在数组 temp_buf 中，细心的朋友会发现取出的每一位后面加了 0X30，这个值是将数字 0-9 转换为 ASCII 码，因为液晶是以 ASCII 码显示的。最后一个数组元素赋值了’\0’，这是为了让 LCD1602 显示函数知道数组内结束位置。

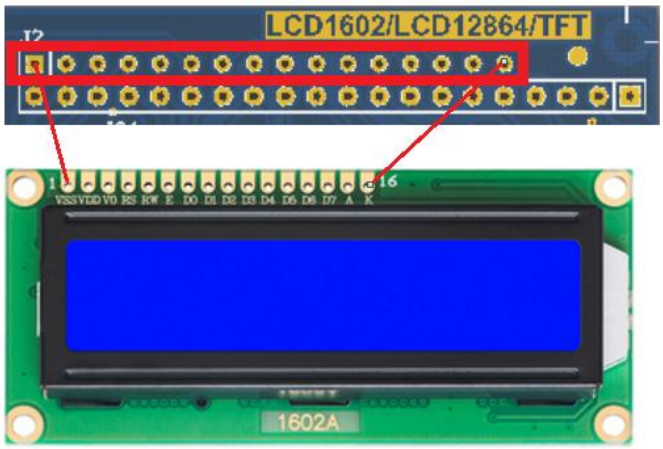
根据 C 语言指针的特点，此时调用 LCD1602 显示函数时，只需要将数组名传递进去即可，因为数组名即为该数组的起始地址，然后显示函数内部通过指针访问数组的每一位。

29.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：在 LCD1602 液晶上显示字符信息。



注意：LCD1602 液晶要正确插入到 LCD1602 接口 J2 位置，插反或差错都会导致显示不正常。如果出现显示看不清，可调节板子 LCD1602 接口下的 RJ1 电位器。如果 LCD1602 无法显示，可重启电源或复位。



课后作业

第 30 章 LCD12864 液晶显示实验

上一章我们学习了如何在 LCD1602 显示字符数据，我们知道 LCD1602 是字符型显示器，它不能显示汉字图形等。对于需要显示汉字或图形的项目中，LCD1602 无法实现，因此本章就来介绍一个可以实现字符、汉字、图形等显示的液晶屏--LCD12864。LCD12864 分为带字库和不带字库两种。我们开发板上集成了一个 LCD12864 液晶显示器接口，它可兼容带字库和不带字库两种屏，将 LCD12864 液晶对应插入即可。本章要实现的功能是：系统运行时，LCD12864 上显示汉字字符信息。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

30.1 LCD12864 介绍

30.2 硬件设计

30.3 软件设计

30.4 实验现象

普中51单片机开发板

30.1 LCD12864 介绍

LCD12864 液晶屏结构上与 LCD1602 一样，只是在行列数与显示像素上区别很大。LCD12864，以下简称 12864，注意区分 LCD1602 和 LCD12864。12864 是 64 行 128 列，当然也有可能会设计成 64 列 128 行，这里的行列不像 1602 那样，1602 是按照八行四列标准英文字符格式，以一行十六个字符，两列字符命名，而 12864 是以 128 列像素，64 行像素，也就是有 128*64 个像素点组成。就好比是 128 列 64 行的点阵。需要一行一列的去显示像素点。

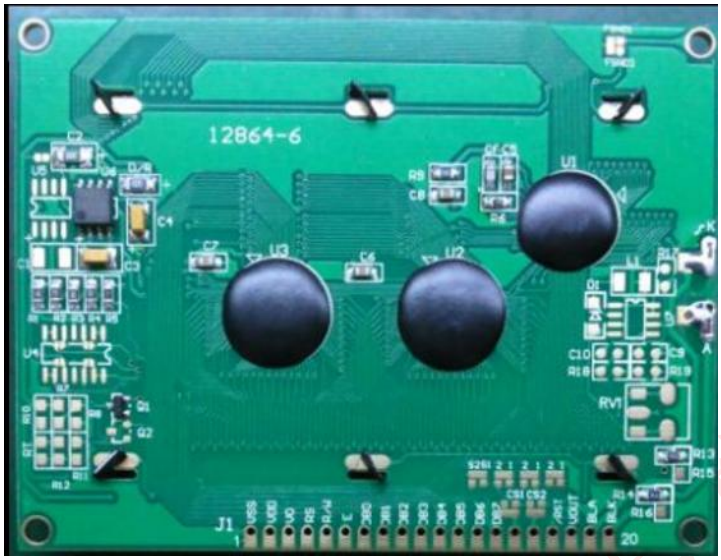
通常显示一个汉字需要 16*16 个像素点，所以 LCD12864 一行最多能显示 8 个汉字，最多能显示 4 行。通常显示一个字符需要 8*8 个像素点，所以 LCD12864 一行最多能显示 16 个字符，最多能显示 8 行。当然这是不依靠后期取模的显示情况，如果通过取模软件取模，然后将 LCD12864 按照 128*64 个像素点来显示，显示的内容就可能超过之前的，就拿字符来说，可以选择小号字体，通过取模软件将字符数据取出，然后将这些数据通过在对应的像素点上点亮或熄灭实现不同字体的显示。图像的显示原理也是这样。

常用的 12864 分为带字库和不带字库两种。

30.1.1 带字库 LCD12864

上图左侧为带字库 12864 显示字符和汉字，右侧为图形模式显示。注意：这里图形模式虽然显示了字符和汉字，但是并不是使用字库里的，显示的方法也是多样化的。

对于带字库 LCD12864，最常见的标志就是在屏幕背后，会有存放字库的芯片。如下图所示：



LCD12864 带字库屏共有 20 个引脚，管脚功能如下：

引脚号	引脚名称	功能说明
1	VSS	电源地（0V）
2	VDD	电压输入（+5V）
3	V0	LCD驱动电压输入端（对比度调节）
4	RS	寄存器选择端 高：数据寄存器 低：命令寄存器
5	R/W	读/写信号 高：读操作 低：写操作
6	E	使能信号
7-14	DB0-DB7	数据总线
15	CS1	片选信号1，低有效，对应左半屏64×64点
16	CS2	片选信号2，低有效，对应右半屏64×64点
17	/RST	复位信号，低有效
18	VOUT	负压输入输出端
19	LED-A	背光正极
20	LED-K	背光负极

LCD12864 的指令相对 LCD1602 要多一些，但常用指令基本上一样，下面来看下 LCD12864 带字库常用的指令。

（1）功能设定

6）功能设定(FUNCTION SET)

0	0	1	DL	X	0/RE	X	X

初始值：001 DL X100 B (DL=1,8BIT 并口;DL=0,4BIT 并口 X=0,1).
DL：8/4 位接口控制位：1—8 位接口；0—4 位接口.
RE：指令集选择控制位：1—扩充指令集；0—基本指令集.
同一指令的动作不能同时改变 DL 和 RE，需先改变 DL 再改变 RE 才能确保设置正确

（2）清屏指令

1）清除显示（CLEAR）

0	0	0	0	0	0	0	1

将 DDRAM 填满“20H” （空格）代码，并且设定 DDRAM 的地址计数器（AC）为 00H；更新设置进入设定点将 I/D 设为 1，游标右移 AC 加 1。

（3）进入设定点

3）进入设定点，初值：06H

0	0	0	0	0	1	I/D	S

指定在显示数据的读取与写入时，设定游标的移动方向及指定显示的移位
I/D＝1，游标右移，DDRAM 地址计数器（AC）加 1
I/D＝0，游标左移，DDRAM 地址计数器（AC）减 1
S:显示画面整体位移

S	S	功能描述
H	H	画面整体左移
H	L	画面整体右移

（4）显示状态开关设置

4）显示开关设置，初始值：08H

0	0	0	0	1	D	C	B

控制整体显示开关，游标开关，游标位置显示反白开关。
D=1,整体显示开；D=0，整体显示关，但是不改变 DDRAM 内容。
C=1,游标显示开；C=0，游标显示关。
B=1,游标位置显示反白开，将游标所在地址上的内容反白显示;B=0，正常显示。

同 LCD1602 一样，LCD12864 内部也含有 DDRAM，与其对应的地址如下：

从上图可知，LCD12864 能直接显示的地址范围如下：

第一行：0X80-0X87；（一个汉字占用 2 个字节，共 8 个汉字）

第二行：0X90-0X97；

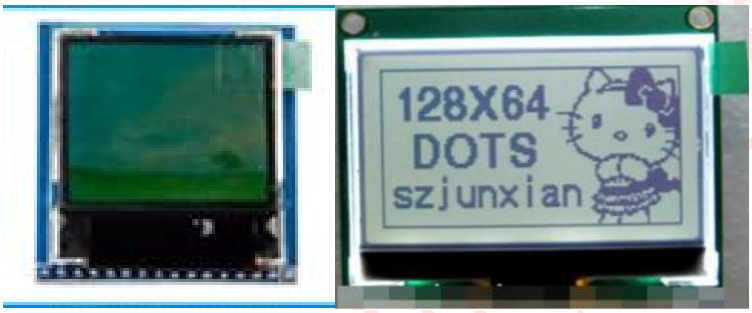
第三行：0X88-0X8F；

第四行：0X98-0X9F；

LCD12864 与 LCD1602 有太多的相似之处，包括通信时序，简直是一模一样，所以编写 LCD12864 的驱动程序时，可完全照搬 LCD1602 的。

带字库 LCD12864 就介绍到此，更多的内容可在网上查阅 LCD12864 相关资料。

30.1.2 不带字库 LCD12864



这种不带字库的 LCD12864 背面没有芯片，一般为转接板，将 12864 显示屏转接到开发板。这种显示屏的操作就像带字库的 12864 操作图片形式一样。 所有的显示都需要取模，取模方法和点阵取模一样。不带字库的 LCD12864，从外形上看其体积比带字库 LCD12864 要小很多，因此也常称之为 MiniLCD12864。

对于 MiniLCD12864，要想显示汉字或其他字符需通过取字模方式来实现。要让其显示首先还得初始化，这个和 LCD1602 原理类似，即通过对其内部一些特殊寄存器设置实现特定功能。MiniLCD12864 内部驱动芯片有的是 ST7565P，其数据手册以及转换板原理图在资料“\7--51 相关资料\MiniLCD12864”内可以查看。

ST7565P 数据手册是液晶屏内核手册，里面有驱动方法和指令和程序有关的数据。通常液晶屏的数据手册都是英文的，但是其内容并不复杂，我们要查看的通常都是某几个寄存器内容，没有必要把手册整篇都看完。而且寄存器的介绍都是一些较容易看懂的单词，如果不明白可以借助翻译软件理解。在这里要降调一下，既然打算进入嵌入式行业，一定要习惯看英文资料，以后进入高级单片机或者 ARM 的学习会遇到很多英文资料的查阅。关于 ST7565P 的介绍大家可以查阅手册，这里就不列举了。

本章实验，我们以简单易用的带字库 LCD12864 为例进行讲解，对于

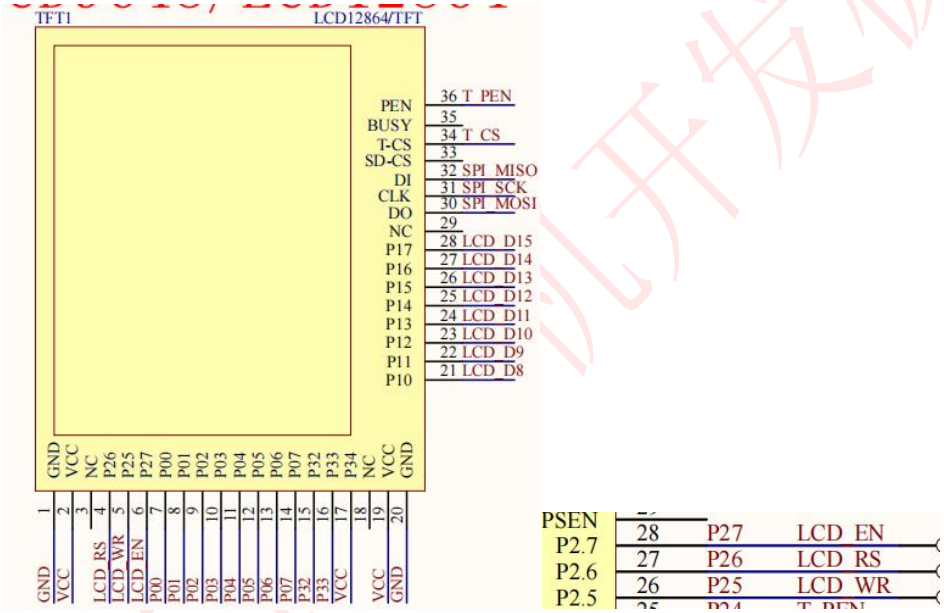
MiniLCD12864 等用到时再做辅助学习，不同品牌的 MiniLCD12864，驱动芯片也不一样，所以没有带字库的 LCD12864 通用。

30.2 硬件设计

本实验使用到硬件资源如下：

（1）带字库 LCD12864 液晶

开发板上集成了一个 LCD12864 液晶接口，下面我们来看下开发板上 LCD12864 液晶接口电路，如下图所示：



从上图中可知，该接口是 LCD12864 与 TFTLCD 彩屏共用的，LCD12864 占用下面 20 个管脚，MiniLCD12864 是 16 脚，而带字库的 LCD12864 通常是 20 脚。这样可以兼容带字库的 LCD12864 和不带字库的 LCD12864。至于 TFTLCD 后面章节会介绍。该电路中 LCD12864 的 8 位数据口 LCD_D0-LCD_D7 与单片机的 P0.0-P0.7 管脚连接，LCD12864 的 RS、RW、E 脚与单片机的 P2.6、P2.5、P2.7 管脚连接，LCD12864 的 PSB 脚与单片机的 P3.2 管脚连接，LCD12864 的 RESET 复位脚直接连接在 VCC 上。当使用 LCD12864 时，其他设备就不要占用这些管脚，即使要用也只能分时复用。

30.3 软件设计

本章所要实现的功能是：LCD12864 上显示汉字字符信息。

程序框架如下：

（1）编写 LCD12864 显示函数

（2）编写主函数

本章软件的重点是如何对 LCD12864 进行写命令和数据。下面我们打开“\4--实验程序\1--基础实验\26-LCD12864 液晶显示实验”工程，在 App 工程组中可以看到新添加了 lcd12864.c 文件（里面包含了 LCD12864 驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

30.3.1 LCD12864 显示函数

打开 lcd12864.c，代码如下：

```
#include "lcd12864.h"



/*************************************************************************
***********
* 函 数 名       : lcd12864_write_cmd
* 函数功能       : LCD12864 写命令
* 输    入       : cmd：指令
* 输    出       : 无
*************************************************************************
**********/
void lcd12864_write_cmd(u8 cmd)
{
    LCD12864_RS=0;//选择命令
    LCD12864_WR=0;//选择写
    LCD12864_E=0;
    LCD12864_DATAPORT=cmd;//准备命令
    delay_ms(1);
    LCD12864_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD12864_E=0;//使能脚 E 后负跳变完成写入
}
```

```
/*****************************************************************************
***********
* 函 数 名       : lcd12864_write_data
* 函数功能       : LCD12864 写数据
* 输    入       : dat: 数据
* 输    出       : 无
*****************************************************************************
**********/
void lcd12864_write_data(u8 dat)
{
    LCD12864_RS=1;//选择数据
    LCD12864_WR=0;//选择写
    LCD12864_E=0;
    LCD12864_DATAPORT=dat;//准备数据
    delay_ms(1);
    LCD12864_E=1;//使能脚 E 先上升沿写入
    delay_ms(1);
    LCD12864_E=0;//使能脚 E 后负跳变完成写入
}

/*****************************************************************************
***********
* 函 数 名       : lcd12864_init
* 函数功能       : LCD12864 初始化
* 输    入       : 无
* 输    出       : 无
*****************************************************************************
**********/
void lcd12864_init(void)
{
    LCD12864_PSB=1;//选择 8 位或 4 位并口方式
    lcd12864_write_cmd(0x30);//数据总线 8 位，基本指令操作
    lcd12864_write_cmd(0x0c);//整体显示关，游标显示关，游标正常显示
    lcd12864_write_cmd(0x06);//写入新数据后光标右移，显示屏不移动
    lcd12864_write_cmd(0x01);//清屏
}

/*****************************************************************************
***********
* 函 数 名       : lcd12864_clear
* 函数功能       : LCD12864 清屏
* 输    入       : 无
* 输    出       : 无
```

```
*****************************************************************************
**********/
void lcd12864_clear(void)
{
    lcd12864_write_cmd(0x01);
}

/*****************************************************************************
***********
* 函 数 名       : lcd12864_show_string
* 函数功能       : LCD12864 显示字符串
* 输    入       : x,y: 显示坐标，x=0~7，y=0~3;
                   str: 显示字符串数据
* 输    出       : 无
*****************************************************************************
**********/
void lcd12864_show_string(u8 x,u8 y,u8 *str)
{
    if(y<=0)y=0;
    if(y>3)y=3;
    x&=0x0f;   //限制 x,y 不能大于显示范围

    switch(y)
    {
        case 0: x|=0x80;break;//第 1 行地址+x 的偏移
        case 1: x|=0x90;break;//第 2 行地址+x 的偏移
        case 2: x|=0x88;break;//第 3 行地址+x 的偏移
        case 3: x|=0x98;break;//第 4 行地址+x 的偏移
    }
    lcd12864_write_cmd(x);
    while(*str!='\0')
    {
        lcd12864_write_data(*str);
        str++;
    }

}
```

上述代码非常简单，如果有认真学习 LCD1602，会发现 LCD12864 的写命令和写数据函数是一样的，而且 LCD12864 的初始化与 LCD1602 也是类似，不同之处 LCD12864 的命令与 LCD1602 有差别，对比注释可以知道。

lcd12864_show_string 函数是 LCD12864 显示字符串，可显示汉字、数字、字符等。函数实现过程大家可对照注释理解，相信到这里这种代码应该是能看懂

的，这里就不再叙述。

30.3.2 主函数

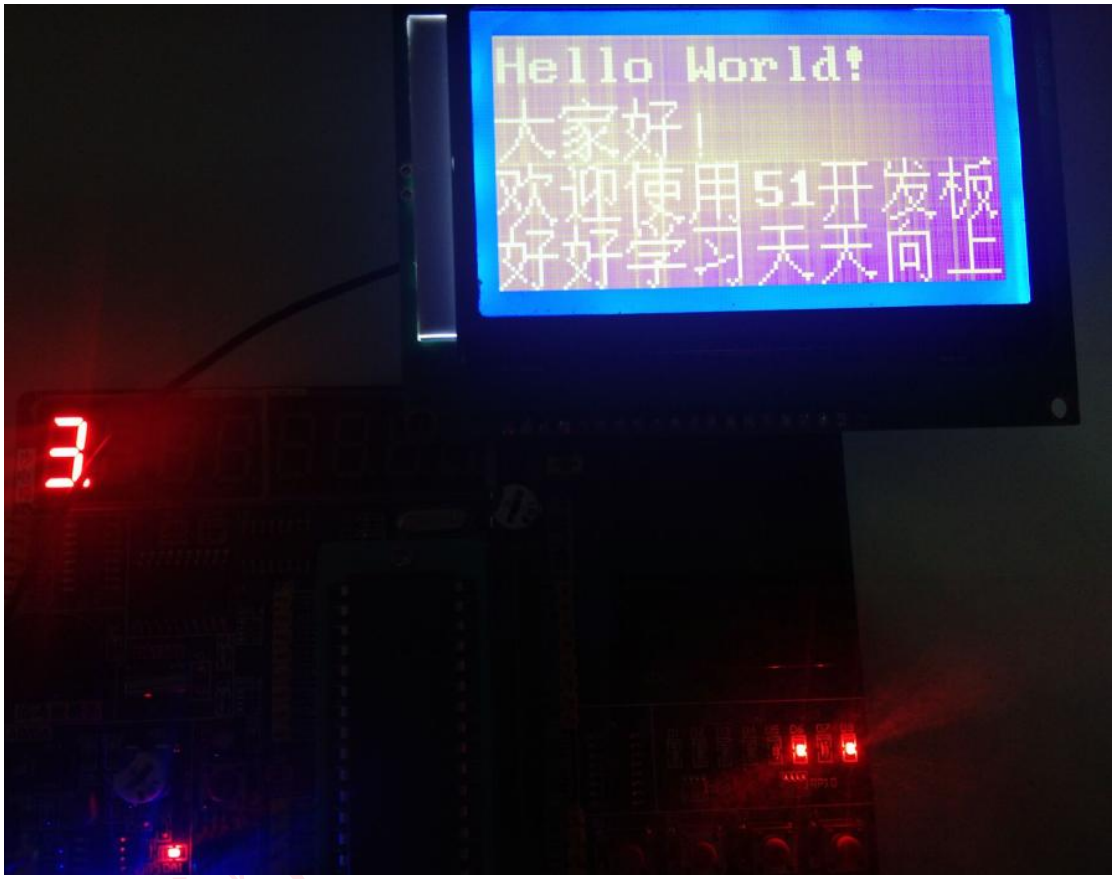
打开 main.c 文件，代码如下：

```
/*************************************************************************
******************
实验名称：LCD12864 液晶显示实验（带字库）
接线说明：
实验现象：下载程序后，LCD12864 上显示汉字字符信息
注意事项：

*************************************************************************
******************/
#include "public.h"
#include "lcd12864.h"


/*************************************************************************
***********
* 函 数 名       : main
* 函数功能       : 主函数
* 输    入       : 无
* 输    出       : 无
*************************************************************************
**********/
void main()
{
    lcd12864_init();//LCD12864 初始化

    lcd12864_show_string(0,0,"Hello World!");//第 1 行字符串显示
    lcd12864_show_string(0,1,"大家好！");//第 2 行字符串显示
    lcd12864_show_string(0,2,"欢迎使用 51 开发板");//第 3 行字符串显示
    lcd12864_show_string(0,3,"好好学习天天向上");//第 4 行字符串显示
    while(1)
    {

    }
}
```

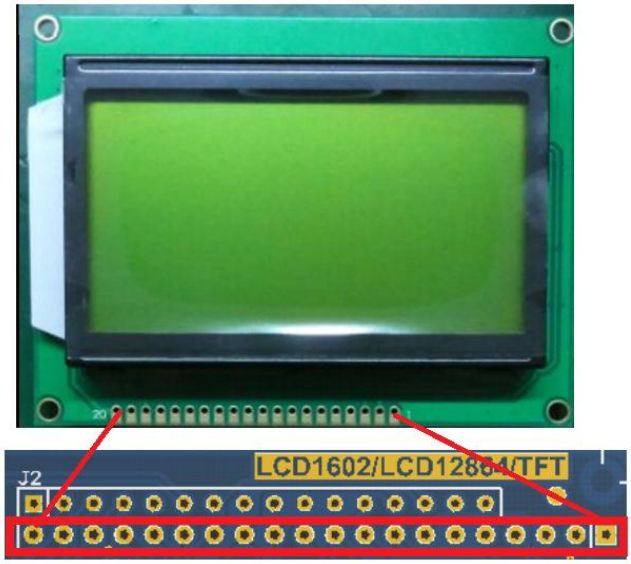
主函数代码非常简单，首先调用外设驱动头文件，然后进入 main 初始化 LCD12864，接着调用 lcd12864_show_string 函数显示内容。

30.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：LCD12864 上显示汉字字符信息。



注意：带字库 LCD12864 液晶要正确插入到 LCD12864 接口位置，插反或差错都会导致显示不正常。如果出现显示看不清，可调节 LCD12864 液晶屏幕背面的可调电位器（一般无需操作）。如果 LCD12864 无法显示，可重启电源或复位。



课后作业

普中51单片机开发板

第 31 章 TFTLCD 显示实验

在前面章节，我们介绍了 LCD1602 和 LCD12864 液晶显示器，它们只能用来显示字符、汉字和简单单色图片，不能显示彩色图片。这一章就来介绍一种彩色液晶显示装置--TFTLCD 薄膜晶体管液晶显示器，使用它不仅可以显示更多的汉字、字符数字信息，还可以显示 16 位色真彩图片，在一些高端设备中应用广泛，通常采用 8080 总线时序通信。我们开发板上集成了一个 TFTLCD 液晶显示器接口，将 TFTLCD 液晶对应插入即可。本章要实现的功能是：系统运行时，在 TFTLCD 上显示字符和汉字。学习本章可以参考前面的实验章节内容。本章分为如下几部分内容：

31.1 TFTLCD 介绍

31.2 硬件设计

31.3 软件设计

31.4 实验现象

普中51单片机开发板

31.1 TFTLCD 介绍

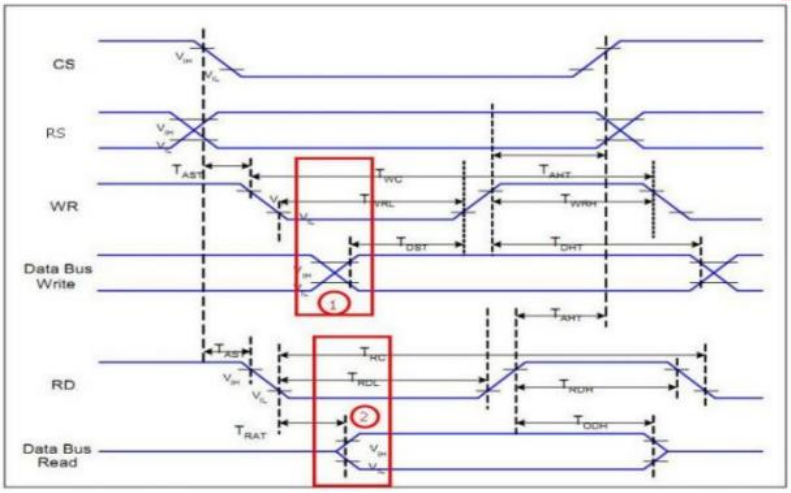
TFT-LCD 是薄膜晶体管液晶显示器英文 thin film transistor-liquid crystal display 字头的缩写。TFT 液晶为每个像素都设有一个薄膜晶体管（TFT），每个像素都可以通过点脉冲直接控制，因而每个节点都相对独立，并可以连续控制，不仅提高了显示屏的反应速度，同时可以精确控制显示色阶，所以 TFT 液晶的色彩更真，因此 TFT-LCD 也被叫做真彩液晶显示器。

常用的 TFT 液晶屏接口有很多种，8 位、9 位、16 位、18 位都有，这里的位数表示的是彩屏数据线的数量。常用的通信模式主要有 6800 模式和 8080 模式，对于 TFT 彩屏通常都使用 8080 并口（简称 80 并口）模式。

如果学习过 LCD1602 或者 LCD12864 等，那么就会发现 8080 模式的读写时序其实跟 LCD1602 或者 LCD12864 的读写时序是差不多的。8080 接口有 5 条基本的控制线和多条数据线，数据线的数量主要看液晶屏使用的是几位模式，有 8 根、9 根、16 根、18 根四种类型。它们的功能如下：

RST	0：复位选择 1：取消复位
CS	0：片选选择 1：取消片选
RS	0：控制寄存器 1：数据寄存器
RD	0：读选择 1：读取消
WR	0：写选择 1：写取消
DB0~DB8	数据线

接下来我们来看一下 8080 接口模式的时序，如下图：



从上图我们就可以得出液晶屏的读写时序：

①：在 WR 跳变为低电平之后，液晶屏开始读取总线上面的数据。如果使用 IO 口模拟写入的时候，可以先在总线上面写入数据，然后在跳变 WR，以保证当读取的时候，总线上面的数据是稳定的。

②：在 RD 跳变为低电平之后，液晶屏放置数据到总线上面。

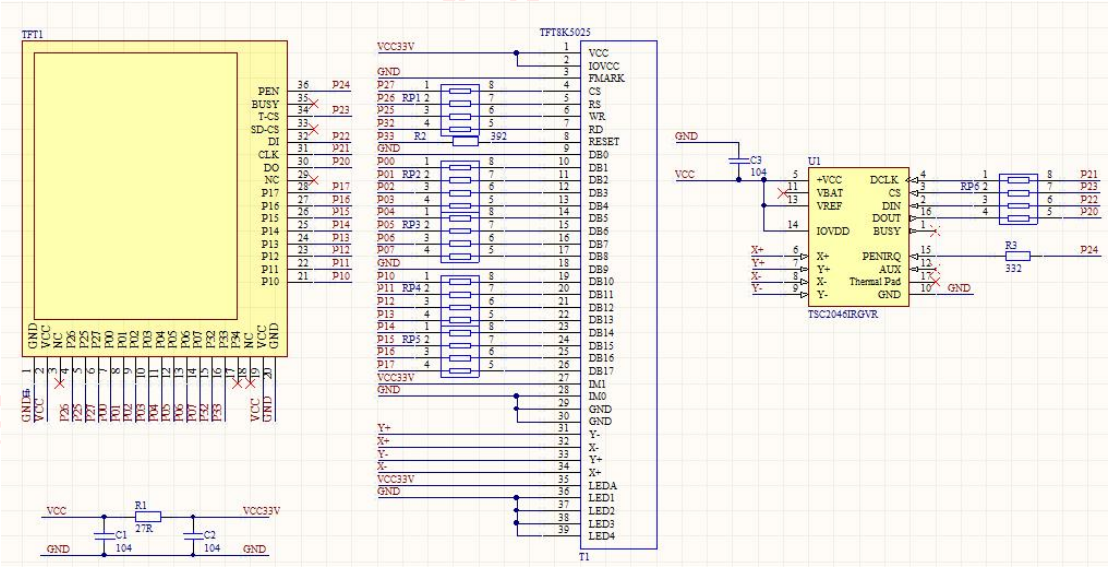
我们使用 51 单片机就是通过单片机的 IO 口模拟 8080 时序进行 TFT 彩屏控制。

下面我们来介绍下 TFTLCD 模块，我们公司推出的 TFTLCD 模块有很多种，按照屏幕大小的不同可分为 2.0、2.2、2.4、2.8、3.0、3.2、3.5、3.6、4.3、4.5、7 寸等，不同尺寸的彩屏对应的分辨率可能不同，比如说 3.5 寸的彩屏分辨率为 320*480（ 长*高），4.5 寸的为 480*854，当然这个具体要看对应彩屏的数据手册,彩屏数据手册和彩屏原理图在资料的“\2--开发板原理图\彩屏原理图”。 按照 TFT 彩屏驱动芯片的不同可分为海信 HX83xx、ILI93xx、R615xx、LG45xx、NT355 等等，大家手上的彩屏驱动芯片具体是哪一种，需要看彩屏板表面左上角或者背面型号，通常我们都会将彩屏的驱动芯片型号放在 TFTLCD 模块

的左上角或者背面。**我们的 TFTLCD 模块有带触摸和不带触摸两种，看具体产品型号所定，如果带触摸可用来做输入控制**。

本章我们就以 2.6 寸的 TFTLCD 模块为例来介绍（其他尺寸的彩屏和驱动芯片使用方法类似），该模块驱动芯片型号是 R61509V，分辨率为 240*320，接口为 16 位的 80 并口，自带触摸功能。该模块的外观图如下图所示：

该模块原理图如下图所示：



TFTLCD 模块采用垂直两排插针，横向是一个 20Pin 的插针，纵向是一个 16Pin 的插针，间距 2.54 公排针与开发板上 TFT/LCD12864 接口连接，从图中可以看出，此 TFTLCD 模块采用 16 位的并口方式与外部连接，之所以不采用 8 位的方式，是因为彩屏的数据量比较大，尤其在显示图片的时候，如果用 8 位数据线，就会比 16 位方式慢一倍以上，我们当然希望速度越快越好，所以选择 16

位的接口，当然不同 TFTLCD 数据位数不一样，如果彩屏是 8 位的同样也是接在 16 位的对应高 8 位或者低 8 位上，接口使用 16 位是方便兼容其他彩屏。图中还列出了触摸屏芯片的接口，关于触摸屏本章我们不多介绍，在后面的章节会有详细的介绍。该模块的 80 并口有如下一些信号线：

CS：TFTLCD 片选信号。

WR：向 TFTLCD 写入数据控制。

RD：从 TFTLCD 读取数据控制。

RS：命令/数据选择（0，读写命令；1，读写数据）。

DB[15：0]：16 位双向数据线。

RST：TFTLCD 复位。

80 并口的通信时序前面已经介绍，所以要控制 TFTLCD 模块显示，总共需要 21 个 IO 口（除触摸功能管脚）。

知道了模块的管脚功能及通信时序，接下来我们就来介绍下如何让液晶模块显示。通常按照以下几步即可实现 TFT 液晶显示：

（1）设置 51 单片机与 TFTLCD 模块相连接的 IO

要让 TFTLCD 模块显示，首先得定义 TFTLCD 模块与 51 单片机相连的 IO 口，以便控制 TFTLCD。

（2）初始化 TFTLCD 模块（写入一系列设置值）

初始化 IO 口，接着就是对 TFTLCD 进行配置，首先就是要复位下 LCD，然后就是初始化序列，即向 LCD 控制器写入一系列的设置值（比如 RGB 格式、LCD 显示方向、伽马校准等），这部分代码一般 LCD 厂商会提供，我们直接使用这些初始化序列即可，无需深入研究。关于这些设置值可以在你所使用的彩屏模块驱动芯片数据手册内查找到，只不过这些数据手册全是英文的，其实也不是很难，我们用到的只是几个设置值而已，不认识的可以百度翻译下。初始化完成之后，LCD 就可以正常使用了。

（3）将要显示的内容写到 TFTLCD 模块内

这一步需要按照：设置坐标→写 GRAM 指令→写 GRAM 来实现，但是这个步骤，只是一个点的处理，如果我们想要显示字符或数字，就必须要多次使用这个步骤，从而达到显示字符或数字的目的，一般我们会设计一个函数来封装这些过

程（实现字符或数字的显示），之后只需调用该函数，就可以实现字符或数字的显示了。

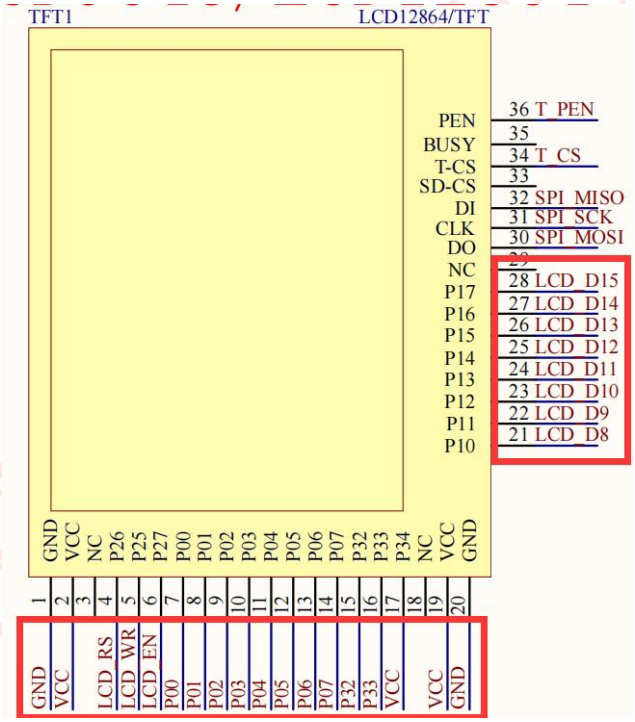
这一部分内容等到我们后面编写程序的时候大家就可以看到，其实还是比较简单的。接下来我们看下 TFTLCD 的硬件电路。

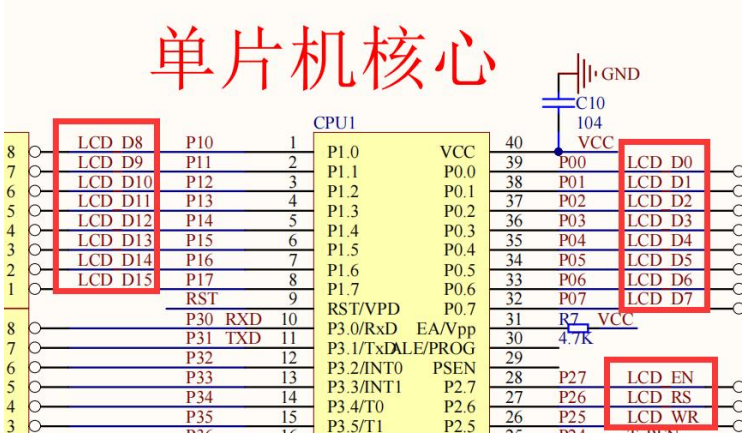
31.2 硬件设计

本实验使用到硬件资源如下：

（1）TFTLCD 液晶模块

开发板上集成了一个 TFTLCD 液晶接口，下面来看下 51 单片机与 TFTLCD 接口的连接关系，如下图所示：





在前面介绍 LCD12864 时我们知道 TFT1 接口可以兼容 LCD12864 和 TFTLCD 模块，LCD12864 占用 TFT1 下面 20 个管脚，而 TFTLCD 模块占用全部引脚。从图中可以看到，TFTLCD 模块的 16 位数据口的高八位连接在单片机的 P1 口，低八位连接在单片机的 P0 口。TFTLCD 模块的 CS、RS、RW、RD、RESET 脚分别连接在单片机的 P27、P26、P25、P32、P33 口。所以当使用 TFTLCD 时，其他设备就不要占用这些管脚，即使占用也只能分时复用。TFTLCD 接口上的 T_PEN、 T_CS、SPI_SCK 等引脚是用于控制触摸的，这些在后面触摸实验章节会介绍到

31.3 软件设计

本章所要实现的功能是：TFTLCD 上显示字符和汉字。

程序框架如下：

（1）编写 TFTLCD 初始化函数

（2）编写 TFTLCD 显示函数

（3）编写主函数

本章软件的重点是如何对 TFTLCD 进行写命令和数据以及初始化和显示函数。下面我们打开“\4--实验程序\1--基础实验\27-TFTLCD 显示实验”工程，在 App 工程组中可以看到新添加了 tftlcd.c 文件（里面包含了 TFTLCD 驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

31.3.1 TFTLCD 初始化函数

要让 TFTLCD 显示，需要初始化序列，即 TFT 彩屏厂家提供的 TFTLCD 寄存器设置值。将这些数值写入到 TFTLCD 内对应的命令寄存器中即可，所以还需要编写 TFTLCD 写命令和写数据等函数。TFTLCD 初始化代码如下：

```
void TFTLCD_Init(void)
{
    u16 i=0;
    TFT_RST=1;
    delay_ms(100);

    TFT_RST=0;
    delay_ms(100);

    TFT_RST=1;
    delay_ms(100);

#ifdef TFT20_HX8309
    ...（省略）
#endif

#ifdef TFT22_R61503B
    ...（省略）
#endif
#ifdef TFT22_ILI9225B
    ...（省略）
#endif

#ifdef TFT24_ST7781R
    ...（省略）
#endif

#ifdef TFT26_R61509V
    LCD_WriteCmd(0X0000);
    tftlcd_data.id=LCD_ReadData();
#endif

#ifdef TFT26_ILI9325D
    ...（省略）
#endif

#ifdef TFT24_ST7789S
```

```
    ...（省略）
#endif

#ifdef TFT24_SSD1297
    ...（省略）
#endif

#ifdef TFT20_ILI9225
    ...（省略）
#endif

#ifdef TFT20_ILI9341
    ...（省略）
#endif

#ifdef TFT20_ST7775R
    ...（省略）
#endif

#ifdef TFT20_ST7775RV
    ...（省略）
#endif

#ifdef TFT20_ILI9225C
    ...（省略）
#endif

    printf(" LCD ID:%x\r\n",tftlcd_data.id); //打印 LCD ID

#ifdef TFT20_HX8309
    ...（省略）
#endif

#ifdef TFT22_R61503B

    ...（省略）
#endif


#ifdef TFT22_ILI9225B
    ...（省略）

#endif
```

```
#ifdef TFT24_ST7781R
    ...（省略）
#endif

#ifdef TFT26_R61509V
    LCD_WriteCmd(0x0000);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0000);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0000);LCD_WriteData(0x0000);
    delay_ms(100);

    LCD_WriteCmd(0x0400);LCD_WriteData(0x6200);
    LCD_WriteCmd(0x0008);LCD_WriteData(0x0808);

    LCD_WriteCmd(0x0300);LCD_WriteData(0x0C00);
    LCD_WriteCmd(0x0301);LCD_WriteData(0x5A0B);
    LCD_WriteCmd(0x0302);LCD_WriteData(0x0906);
    LCD_WriteCmd(0x0303);LCD_WriteData(0x1017);
    LCD_WriteCmd(0x0304);LCD_WriteData(0x2300);
    LCD_WriteCmd(0x0305);LCD_WriteData(0x1700);
    LCD_WriteCmd(0x0306);LCD_WriteData(0x6309);
    LCD_WriteCmd(0x0307);LCD_WriteData(0x0C09);
    LCD_WriteCmd(0x0308);LCD_WriteData(0x010C);
    LCD_WriteCmd(0x0309);LCD_WriteData(0x2232);

    LCD_WriteCmd(0x0010);LCD_WriteData(0x0016);
    LCD_WriteCmd(0x0011);LCD_WriteData(0x0101);
    LCD_WriteCmd(0x0012);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0013);LCD_WriteData(0x0001);

    LCD_WriteCmd(0x0100);LCD_WriteData(0x0330);
    LCD_WriteCmd(0x0101);LCD_WriteData(0x0336);
    LCD_WriteCmd(0x0103);LCD_WriteData(0x1000);

    LCD_WriteCmd(0x0280);LCD_WriteData(0x6100);
    LCD_WriteCmd(0x0102);LCD_WriteData(0xBBB4);
    delay_ms(100);

    LCD_WriteCmd(0x0001);LCD_WriteData(0x0100);
    LCD_WriteCmd(0x0002);LCD_WriteData(0x0100);
    LCD_WriteCmd(0x0003);LCD_WriteData(0x1030);
    LCD_WriteCmd(0x0009);LCD_WriteData(0x0001);
    LCD_WriteCmd(0x000C);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0090);LCD_WriteData(0x0800);
    LCD_WriteCmd(0x000F);LCD_WriteData(0x0000);
```

```
    LCD_WriteCmd(0x0210);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0211);LCD_WriteData(0x00EF);
    LCD_WriteCmd(0x0212);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0213);LCD_WriteData(0x018F);        //;400


    LCD_WriteCmd(0x0500);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0501);LCD_WriteData(0x0000);
    LCD_WriteCmd(0x0502);LCD_WriteData(0x005F);

    LCD_WriteCmd(0x0401);LCD_WriteData(0x0001);
    LCD_WriteCmd(0x0404);LCD_WriteData(0x0000);
    delay_ms(100);

    LCD_WriteCmd(0x0007);LCD_WriteData(0x0100);
    delay_ms(100);

    LCD_WriteCmd(0x0202);
#endif

#ifdef TFT26_ILI9325D
    ...（省略）
#endif

#ifdef TFT24_ST7789S
    ...（省略）
#endif

#ifdef TFT24_SSD1297
    ...（省略）
#endif

#ifdef TFT20_ILI9225
    ...（省略）

#endif

#ifdef TFT20_ILI9341
    ...（省略）
#endif

#ifdef TFT20_ST7775R
    ...（省略）
```

```
#endif

#ifdef TFT20_ST7775RV
    ...（省略）
#endif

#ifdef TFT20_ILI9225C
    ...（省略）
#endif

    LCD_Display_Dir(TFTLCD_DIR);        //0：竖屏  1：横屏  默认竖屏
    LCD_Clear(BACK_COLOR);
}
```

该函数中的 LCD_WriteCmd、LCD_WriteData 和 LCD_WriteData_Color 为写命令、写数据函数，其实现过程很简单，代码如下：

```
//写寄存器函数
//cmd:寄存器值
void LCD_WriteCmd(u16 cmd)
{
#ifdef TFT20_HX8309
    ...（省略）
#endif

#ifdef TFT22_R61503B
    ...（省略）
#endif

#ifdef TFT22_ILI9225B
    ...（省略）
#endif

#ifdef TFT24_ST7781R
    ...（省略）
#endif

#ifdef TFT26_R61509V
    TFT_CS=0;
    TFT_RS=0;
    TFT_DATAPORTH=cmd>>8;
    TFT_DATAPORTL=cmd;
    TFT_WR=0;
    TFT_WR=1;
```

```
    TFT_CS=1;
#endif

#ifdef TFT26_ILI9325D
    ...（省略）
#endif

#ifdef TFT24_ST7789S
    ...（省略）
#endif

#ifdef TFT24_SSD1297
    ...（省略）
#endif

#ifdef TFT20_ILI9225
    ...（省略）
#endif

#ifdef TFT20_ILI9341
    ...（省略）
#endif

#ifdef TFT20_ST7775R
    ...（省略）
#endif

#ifdef TFT20_ST7775RV
    ...（省略）
#endif

#ifdef TFT20_ILI9225C
    ...（省略）
#endif
}

//写数据
//dat:要写入的值
void LCD_WriteData(u16 dat)
{
#ifdef TFT20_HX8309
    ...（省略）
#endif
```

```
#ifdef TFT22_R61503B
    ...（省略）
#endif
#ifdef TFT22_ILI9225B
    ...（省略）
#endif

#ifdef TFT24_ST7781R
    ...（省略）
#endif

#ifdef TFT26_R61509V
    TFT_CS=0;
    TFT_RS=1;
    TFT_DATAPORTH=dat>>8;
    TFT_DATAPORTL=dat;
    TFT_WR=0;
    TFT_WR=1;
    TFT_CS=1;
#endif

#ifdef TFT26_ILI9325D
    ...（省略）
#endif

#ifdef TFT24_ST7789S
    ...（省略）
#endif

#ifdef TFT24_SSD1297
    ...（省略）
#endif

#ifdef TFT20_ILI9225
    ...（省略）
#endif

#ifdef TFT20_ILI9341
    ...（省略）
#endif

#ifdef TFT20_ST7775R
    ...（省略）
#endif
```

```
#ifdef TFT20_ST7775RV
    ...（省略）
#endif

#ifdef TFT20_ILI9225C
    ...（省略）
#endif
}

void LCD_WriteData_Color(u16 color)
{
#ifdef TFT20_HX8309
    ...（省略）
#endif

#ifdef TFT22_R61503B
    ...（省略）
#endif
#ifdef TFT22_ILI9225B
    ...（省略）
#endif

#ifdef TFT24_ST7781R
    ...（省略）
#endif

#ifdef TFT26_R61509V
    TFT_CS=0;
    TFT_RS=1;
    TFT_DATAPORTH=color>>8;
    TFT_DATAPORTL=color;
    TFT_WR=0;
    TFT_WR=1;
    TFT_CS=1;
#endif

#ifdef TFT26_ILI9325D
    ...（省略）
#endif

#ifdef TFT24_ST7789S
    ...（省略）
#endif
```

```
#ifdef TFT24_SSD1297
    ...（省略）
#endif

#ifdef TFT20_ILI9225
    ...（省略）
#endif

#ifdef TFT20_ILI9341
    ...（省略）
#endif

#ifdef TFT20_ST7775R
    ...（省略）
#endif

#ifdef TFT20_ST7775RV
    ...（省略）
#endif

#ifdef TFT20_ILI9225C
    ...（省略）
#endif
}

void LCD_WriteCmdData(u16 cmd,u16 dat)
{
    LCD_WriteCmd(cmd);
    LCD_WriteData(dat);
}
```

程序中可以看到有很多是对管脚的操作，这些都在 tftlcd.h 文件中定义，如下所示：

```
//TFTLCD 彩屏数据控制端口定义
#define TFT_DATAPORTH P1
#define TFT_DATAPORTL P0


sbit   TFT_CS  = P2^7;
sbit   TFT_RST = P3^3;
```

```
sbit   TFT_RS  = P2^6;
sbit   TFT_WR  = P2^5;
sbit   TFT_RD  = P3^2;
```

下面我们再来介绍下 tftlcd.h 中另一个重要的结构体：

```
//LCD 重要参数集
typedef struct
{
    u16 width;        //LCD 宽度
    u16 height;          //LCD 高度
    u16 id;              //LCD ID
    u8  dir;          //横屏还是竖屏控制：0，竖屏；1，横屏。
}_tftlcd_data;
 extern _tftlcd_data tftlcd_data;   //管理 TFTLCD 重要参数
```

该结构体用于保存一些 TFTLCD 重要参数信息，比如 LCD 的尺寸、LCD ID（驱动 IC 型号）、LCD 显示方向，当然你也可以在我们这个结构体内添加其他 LCD 相关参数变量。通过对此结构体的管理可以让我们的驱动函数支持不同尺寸的 LCD，同时可以实现 LCD 横竖屏切换等重要功能。所以大家今后要多学习使用结构体管理不同属性。

初始化函数内我们还使用了条件编译语句来兼容多种彩屏驱动程序。比如：

```
#ifdef TFT22_ILI9225B
....对应 ILI9225B 驱动的操作代码
#endif
#ifdef TFT26_R61509V
....对应 R61509V 驱动的操作代码
#endif
#ifdef TFT20_HX8309
....对应 HX8309 驱动的操作代码
#endif
#ifdef TFT24_SSD1297
....对应 SSD1297 驱动的操作代码
#endif
```

本章所介绍的 TFTLCD 驱动 IC 是 R61509V，所以要对这部分代码操作，只需在 tftlcd.h 文件开始处开启这个宏定义，其他彩屏的宏定义就关闭它。如果你手上使用的是其他驱动类型屏，对应型号打开即可，如下：（后面如果又增加了屏，使用同样方法来兼容）

```
//定义 LCD 彩屏的驱动类型  可根据自己手上的彩屏背面型号来选择打开哪种驱动
```

```
//#define TFT22_ILI9225B
//#define TFT22_ILI9340
//#define TFT24_ST7781R
#define TFT26_R61509V
//#define TFT26_ILI9325D
//#define TFT22_R61503B
//#define TFT20_HX8309
//#define TFT24_ST7789S
//#define TFT24_SSD1297
//#define TFT20_ILI9225
//#define TFT20_ILI9341
//#define TFT20_ST7775R
//#define TFT20_ST7775RV
//#define TFT20_ILI9225C
```

在初始化函数最后部分还调用了 LCD_Display_Dir 和 LCD_Clear 函数，LCD_Display_Dir 函数用于切换 TFTLCD 显示方向（横屏和竖屏），默认我们设置为竖屏，代码如下：

```
//设置 LCD 显示方向
//dir:0,竖屏；1,横屏
void LCD_Display_Dir(u8 dir)
{
    tftlcd_data.dir=dir;
    if(dir==0)  //默认竖屏方向
    {
#ifdef TFT20_HX8309
        ...（省略）
#endif
#ifdef TFT22_R61503B
        ...（省略）
#endif
#ifdef TFT22_ILI9225B
        ...（省略）
#endif

#ifdef TFT24_ST7781R
        ...（省略）
#endif

#ifdef TFT26_R61509V
        LCD_WriteCmdData(0x0001, 0x0100); //Output Direct
        LCD_WriteCmdData(0x0003, 0x1030);   //设置彩屏显示方向的寄存器
#endif
```

```
#ifdef TFT26_ILI9325D
       ...（省略）
#endif

#ifdef TFT24_ST7789S
       ...（省略）
#endif

#ifdef TFT24_SSD1297
       ...（省略）
#endif

#ifdef TFT20_ILI9225
       ...（省略）
#endif

#ifdef TFT20_ILI9341
       ...（省略）
#endif

#ifdef TFT20_ST7775R
       ...（省略）
#endif

#ifdef TFT20_ST7775RV
       ...（省略）
#endif

#ifdef TFT20_ILI9225C
       ...（省略）
#endif

       tftlcd_data.height=HEIGHT;
       tftlcd_data.width=WIDTH;
   }
   else
   {
#ifdef TFT20_HX8309
   ...（省略）
#endif

#ifdef TFT22_R61503B
   ...（省略）
```

```
#endif

#ifdef TFT22_ILI9225B
	...（省略）
#endif

#ifdef TFT24_ST7781R
	...（省略）
#endif

#ifdef TFT26_R61509V
	LCD_WriteCmdData(0x0001, 0x0000); //Output Direct
	LCD_WriteCmdData(0x0003, 0x1038);   //设置彩屏显示方向的寄存器
#endif

#ifdef TFT26_ILI9325D
	...（省略）
#endif

#ifdef TFT24_ST7789S
	...（省略）
#endif

#ifdef TFT24_SSD1297
	...（省略）
#endif

#ifdef TFT20_ILI9225
	...（省略）
#endif

#ifdef TFT20_ILI9341
	...（省略）
#endif

#ifdef TFT20_ST7775R
	...（省略）
#endif

#ifdef TFT20_ST7775RV
	...（省略）
#endif

#ifdef TFT20_ILI9225C
```

```
        ...（省略）
#endif

        tftlcd_data.height=WIDTH;
        tftlcd_data.width=HEIGHT;
    }
}
```

横屏和竖屏的控制只需要对 TFTLCD 显示方向寄存器进行操作（寄存器可通过彩屏数据手册查找），然后再把他们的 X 和 Y 轴像素调换即可。

为了方便大家设置横竖屏，我们直接在初始化函数内调用 LCD_Display_Dir()函数时传递的是一个宏定义 TFTLCD_DIR，而该宏定义在 tftlcd.h 头文件开始处也定义了，其实就是数字 0 或者 1，如果要横屏显示的话该宏值为 1，如果要竖屏显示就为 0。如下所示：

```
    #define TFTLCD_DIR   0   //0：竖屏   1：横屏   默认竖屏
```

LCD_Clear 函数用于清屏，函数入口参数用于选择清屏的颜色，常用的颜色都在 tftlcd.h 文件内进行了宏定义，直接调用宏即可。清屏函数实现过程很简单，其实就是设置 TFTLCD 显示窗口，然后写入 tftlcd_data.width*tftlcd_data.height 个颜色数据即可。设置窗口函数代码如下：

```
//设置窗口,并自动设置画点坐标到窗口左上角(sx,sy).
//sx,sy:窗口起始坐标(左上角)
//width,height:窗口宽度和高度,必须大于 0!!
//窗体大小:width*height.
void LCD_Set_Window(u16 sx,u16 sy,u16 width,u16 height)
{
#ifdef TFT20_HX8309
    ...（省略）
#endif

#ifdef TFT22_R61503B
    ...（省略）
#endif

#ifdef TFT22_ILI9225B
    ...（省略）
#endif

#ifdef TFT24_ST7781R
```

```
    ...（省略）
#endif

#ifdef TFT26_R61509V
    if(tftlcd_data.dir==0)
    {
        LCD_WriteCmd(0x0210);   LCD_WriteData(sx);
        LCD_WriteCmd(0x0211);   LCD_WriteData(width);
        LCD_WriteCmd(0x0212);   LCD_WriteData(sy);
        LCD_WriteCmd(0x0213);   LCD_WriteData(height);

        LCD_WriteCmd(0x0200);   LCD_WriteData(sx);
        LCD_WriteCmd(0x0201);   LCD_WriteData(sy);

        LCD_WriteCmd(0x0202);
    }
    else
    {
        LCD_WriteCmd(0x0212);   LCD_WriteData(sx);
        LCD_WriteCmd(0x0213);   LCD_WriteData(width);
        LCD_WriteCmd(0x0210);   LCD_WriteData(sy);
        LCD_WriteCmd(0x0211);   LCD_WriteData(height);

        LCD_WriteCmd(0x0201);   LCD_WriteData(sx);
        LCD_WriteCmd(0x0200);   LCD_WriteData(sy);

        LCD_WriteCmd(0x0202);
    }
#endif

#ifdef TFT26_ILI9325D
    ...（省略）
#endif

#ifdef TFT24_ST7789S
    ...（省略）
#endif

#ifdef TFT24_SSD1297
    ...（省略）
#endif

#ifdef TFT20_ILI9225
    ...（省略）
```

```
#endif

#ifdef TFT20_ILI9341
    ...（省略）
#endif

#ifdef TFT20_ST7775R
    ...（省略）
#endif

#ifdef TFT20_ST7775RV
    ...（省略）
#endif

#ifdef TFT20_ILI9225C
    ...（省略）
#endif
}
```

其实就是在 TFTLCD 的 X 方向和 Y 方向寄存器内写入窗口值。我们讲解的 R61509V 的 X 方向和 Y 方向命令（寄存器）是 0x0210、0x0211、0x0212 和 0x0213，写完窗口值还需要将这些值写入到 TFTLCD 的 GRAM 内，命令是 0x0202。这些命令也是通过彩屏数据手册查找。

在初始化函数内使用了 printf 函数打印 TFTLCD 的 ID，所以在主函数内需要对串口初始化，否则将导致程序死在 printf 里面，如果不想用 printf，那么请注释掉它。

31.3.2 TFTLCD 显示函数

上述几个函数完成后，我们就可以使用 TFTLCD 了，要在 TFTLCD 上显示内容，我们需要编写对应的显示函数，比如要显示一个点，那么就编写一个点的函数。TFTLCD 显示函数如下：

```
//清屏函数
//color:要清屏的填充色
void LCD_Clear(u16 color)
{
    u16 i=0, j=0 ;

    LCD_Set_Window(0, 0, tftlcd_data.width-1, tftlcd_data.height-1);
```

```
    //作用区域
    for(i=0; i<tftlcd_data.width; i++)
    {
        for (j=0; j<tftlcd_data.height; j++)
        {
            LCD_WriteData_Color(color);
        }
    }
}

//在指定区域内填充单个颜色
//(sx,sy),(ex,ey):填充矩形对角坐标,区域大小为:(ex-sx+1)*(ey-sy+1)
//color:要填充的颜色
void LCD_Fill(u16 xState,u16 yState,u16 xEnd,u16 yEnd,u16 color)
{
    u16 temp=0;

    if((xState > xEnd) || (yState > yEnd))
    {
       return;
    }
    LCD_Set_Window(xState, yState, xEnd, yEnd);
    xState = xEnd - xState + 1;
    yState = yEnd - yState + 1;

    while(xState--)
    {
        temp = yState;
        while (temp--)
        {
            LCD_WriteData_Color(color);
        }
    }
}

//在指定区域内填充指定颜色块
//(sx,sy),(ex,ey):填充矩形对角坐标,区域大小为:(ex-sx+1)*(ey-sy+1)
//color:要填充的颜色
void LCD_Color_Fill(u16 sx,u16 sy,u16 ex,u16 ey,u16 *color)
{
    u16 height,width;
    u16 i,j;
    width=ex-sx+1;          //得到填充的宽度
    height=ey-sy+1;         //高度
```

```
    LCD_Set_Window(sx, sy,ex, ey);
    for(i=0;i<height;i++)
    {
        for(j=0;j<width;j++)
        {
            LCD_WriteData_Color(color[i*width+j]);
        }
    }
}

//画点
//x,y:坐标
//FRONT_COLOR:此点的颜色
void LCD_DrawPoint(u16 x,u16 y)
{
    LCD_Set_Window(x, y, x, y);  //设置点的位置
    LCD_WriteData_Color(FRONT_COLOR);
}

//快速画点
//x,y:坐标
//color:颜色
void LCD_DrawFRONT_COLOR(u16 x,u16 y,u16 color)
{
    LCD_Set_Window(x, y, x, y);
    LCD_WriteData_Color(color);
}

//画一个大点
//2*2 的点
void LCD_DrawBigPoint(u16 x,u16 y,u16 color)
{
    LCD_DrawFRONT_COLOR(x,y,color);//中心点
    LCD_DrawFRONT_COLOR(x+1,y,color);
    LCD_DrawFRONT_COLOR(x,y+1,color);
    LCD_DrawFRONT_COLOR(x+1,y+1,color);
}

//画线
//x1,y1:起点坐标
//x2,y2:终点坐标
void LCD_DrawLine(u16 x1, u16 y1, u16 x2, u16 y2)
{
    u16 t=0;
```

```
    int xerr=0,yerr=0,delta_x,delta_y,distance;
    int incx,incy,uRow,uCol;
    delta_x=x2-x1; //计算坐标增量
    delta_y=y2-y1;
    uRow=x1;
    uCol=y1;
    if(delta_x>0)incx=1; //设置单步方向
    else if(delta_x==0)incx=0;//垂直线
    else {incx=-1;delta_x=-delta_x;}
    if(delta_y>0)incy=1;
    else if(delta_y==0)incy=0;//水平线
    else{incy=-1;delta_y=-delta_y;}
    if( delta_x>delta_y)distance=delta_x; //选取基本增量坐标轴
    else distance=delta_y;
    for(t=0;t<=distance+1;t++ )//画线输出
    {
        LCD_DrawPoint(uRow,uCol);//画点
        xerr+=delta_x ;
        yerr+=delta_y ;
        if(xerr>distance)
        {
            xerr-=distance;
            uRow+=incx;
        }
        if(yerr>distance)
        {
            yerr-=distance;
            uCol+=incy;
        }
    }
}

void LCD_DrawLine_Color(u16 x1, u16 y1, u16 x2, u16 y2,u16 color)
{
    u16 t;
    int xerr=0,yerr=0,delta_x,delta_y,distance;
    int incx,incy,uRow,uCol;
    delta_x=x2-x1; //计算坐标增量
    delta_y=y2-y1;
    uRow=x1;
    uCol=y1;
    if(delta_x>0)incx=1; //设置单步方向
    else if(delta_x==0)incx=0;//垂直线
    else {incx=-1;delta_x=-delta_x;}
```

```
    if(delta_y>0)incy=1;
    else if(delta_y==0)incy=0;//水平线
    else{incy=-1;delta_y=-delta_y;}
    if( delta_x>delta_y)distance=delta_x; //选取基本增量坐标轴
    else distance=delta_y;
    for(t=0;t<=distance+1;t++ )//画线输出
    {
        LCD_DrawFRONT_COLOR(uRow,uCol,color);//画点
        xerr+=delta_x ;
        yerr+=delta_y ;
        if(xerr>distance)
        {
            xerr-=distance;
            uRow+=incx;
        }
        if(yerr>distance)
        {
            yerr-=distance;
            uCol+=incy;
        }
    }
}


// 画一个十字的标记
// x: 标记的 X 坐标
// y: 标记的 Y 坐标
// color: 标记的颜色
void LCD_DrowSign(u16 x, u16 y, u16 color)
{
    u8 i;

    /* 画点 */
    LCD_Set_Window(x-1, y-1, x+1, y+1);
    for(i=0; i<9; i++)
    {
        LCD_WriteData_Color(color);
    }

    /* 画竖 */
    LCD_Set_Window(x-4, y, x+4, y);
    for(i=0; i<9; i++)
    {
        LCD_WriteData_Color(color);
```

```
    }

    /* 画横 */
    LCD_Set_Window(x, y-4, x, y+4);
    for(i=0; i<9; i++)
    {
        LCD_WriteData_Color(color);
    }
}

//画矩形
//(x1,y1),(x2,y2):矩形的对角坐标
void LCD_DrawRectangle(u16 x1, u16 y1, u16 x2, u16 y2)
{
    LCD_DrawLine(x1,y1,x2,y1);
    LCD_DrawLine(x1,y1,x1,y2);
    LCD_DrawLine(x1,y2,x2,y2);
    LCD_DrawLine(x2,y1,x2,y2);
}
//在指定位置画一个指定大小的圆
//(x,y):中心点
//r    :半径
void LCD_Draw_Circle(u16 x0,u16 y0,u8 r)
{
    int a,b;
    int di;
    a=0;b=r;
    di=3-(r<<1);             //判断下个点位置的标志
    while(a<=b)
    {
        LCD_DrawPoint(x0+a,y0-b);             //5
        LCD_DrawPoint(x0+b,y0-a);             //0
        LCD_DrawPoint(x0+b,y0+a);             //4
        LCD_DrawPoint(x0+a,y0+b);             //6
        LCD_DrawPoint(x0-a,y0+b);             //1
        LCD_DrawPoint(x0-b,y0+a);
        LCD_DrawPoint(x0-a,y0-b);             //2
        LCD_DrawPoint(x0-b,y0-a);             //7
        a++;
        //使用 Bresenham 算法画圆
        if(di<0)di +=4*a+6;
        else
        {
            di+=10+4*(a-b);
```

```
            b--;
        }
    }
}


//在指定位置显示一个字符
//x,y:起始坐标
//num:要显示的字符:" "--->"~"
//size:字体大小 12/16/24
//mode:叠加方式(1)还是非叠加方式(0)
void LCD_ShowChar(u16 x,u16 y,u8 num,u8 size,u8 mode)
{
    u8 temp,t1,t;
    u16 y0=y;
    u8 csize=(size/8+((size%8)?1:0))*(size/2);        //得到字体一个字符
对应点阵集所占的字节数
    num=num-' ';//得到偏移后的值（ASCII 字库是从空格开始取模，所以-' '就是对应
字符的字库）
    for(t=0;t<csize;t++)
    {
        if(size==12)temp=ascii_1206[num][t];       //调用 1206 字体
        else if(size==16)temp=ascii_1608[num][t]; //调用 1608 字体
        else if(size==24)temp=ascii_2412[num][t]; //调用 2412 字体
        else return;                               //没有的字库
        for(t1=0;t1<8;t1++)
        {
            if(temp&0x80)LCD_DrawFRONT_COLOR(x,y,FRONT_COLOR);
            else if(mode==0)LCD_DrawFRONT_COLOR(x,y,BACK_COLOR);
            temp<<=1;
            y++;
            if(y>=tftlcd_data.height)return;       //超区域了
            if((y-y0)==size)
            {
                y=y0;
                x++;
                if(x>=tftlcd_data.width)return; //超区域了
                break;
            }
        }
    }
}
//m^n 函数
//返回值:m^n 次方.
```

```
u32 LCD_Pow(u8 m,u8 n)
{
    u32 result=1;
    while(n--)result*=m;
    return result;
}
//显示数字,高位为 0,则不显示
//x,y :起点坐标
//len :数字的位数
//size:字体大小
//color:颜色
//num:数值(0~4294967295);
void LCD_ShowNum(u16 x,u16 y,u32 num,u8 len,u8 size)
{
    u8 t,temp;
    u8 enshow=0;
    for(t=0;t<len;t++)
    {
        temp=(num/LCD_Pow(10,len-t-1))%10;
        if(enshow==0&&t<(len-1))
        {
            if(temp==0)
            {
                LCD_ShowChar(x+(size/2)*t,y,' ',size,0);
                continue;
            }else enshow=1;

        }
        LCD_ShowChar(x+(size/2)*t,y,temp+'0',size,0);
    }
}

//显示数字,高位为 0,还是显示
//x,y:起点坐标
//num:数值(0~999999999);
//len:长度(即要显示的位数)
//size:字体大小
//mode:
//[7]:0,不填充;1,填充 0.
//[6:1]:保留
//[0]:0,非叠加显示;1,叠加显示.
void LCD_ShowxNum(u16 x,u16 y,u32 num,u8 len,u8 size,u8 mode)
{
    u8 t,temp;
```

```
    u8 enshow=0;
    for(t=0;t<len;t++)
    {
        temp=(num/LCD_Pow(10,len-t-1))%10;
        if(enshow==0&&t<(len-1))
        {
            if(temp==0)
            {

    if(mode&0X80)LCD_ShowChar(x+(size/2)*t,y,'0',size,mode&0X01);
                else LCD_ShowChar(x+(size/2)*t,y,' ',size,mode&0X01);
                continue;
            }else enshow=1;

        }
        LCD_ShowChar(x+(size/2)*t,y,temp+'0',size,mode&0X01);
    }
}
//显示字符串
//x,y:起点坐标
//width,height:区域大小
//size:字体大小
//*p:字符串起始地址
void LCD_ShowString(u16 x,u16 y,u16 width,u16 height,u8 size,u8 *p)
{
    u8 x0=x;
    width+=x;
    height+=y;
    while((*p<='~')&&(*p>=' '))//判断是不是非法字符!
    {
        if(x>=width){x=x0;y+=size;}
        if(y>=height)break;//退出
        LCD_ShowChar(x,y,*p,size,0);
        x+=size/2;
        p++;
    }
}

/*****************************************************************************
********
*函数名：LCD_ShowFontHZ
*输　入：x：汉字显示的 X 坐标
*      * y：汉字显示的 Y 坐标
*      * cn：要显示的汉字
```

```
*      * wordColor：文字的颜色
*      * backColor：背景颜色
*输  出：
*功  能：写二号楷体汉字
*******************************************************************************
*******/
#if 1
void LCD_ShowFontHZ(u16 x, u16 y, u8 *cn)
{
    u8 i, j, wordNum;
    u16 color;
    while (*cn != '\0')
    {
        LCD_Set_Window(x, y, x+31, y+28);
        for (wordNum=0; wordNum<20; wordNum++)
        {   //wordNum 扫描字库的字数
            if ((CnChar32x29[wordNum].Index[0]==*cn)
                &&(CnChar32x29[wordNum].Index[1]==*(cn+1)))
            {

                for(i=0; i<116; i++)
                {   //MSK 的位数
                    color=CnChar32x29[wordNum].Msk[i];
                    for(j=0;j<8;j++)
                    {
                        if((color&0x80)==0x80)
                        {
                            LCD_WriteData_Color(FRONT_COLOR);

                        }
                        else
                        {
                            LCD_WriteData_Color(BACK_COLOR);
                        }
                        color<<=1;
                    }//for(j=0;j<8;j++)结束
                }
            }
        } //for (wordNum=0; wordNum<20; wordNum++)结束
        cn += 2;
        x += 32;
    }
}
#endif
```

```
#if 0
void LCD_ShowFontHZ(u16 x, u16 y, u8 *cn)
{
    u8 i, j, wordNum;
    u16 color;
    u16 x0=x;
    u16 y0=y;
    while (*cn != '\0')
    {
        for (wordNum=0; wordNum<20; wordNum++)
        {   //wordNum 扫描字库的字数
            if ((CnChar32x29[wordNum].Index[0]==*cn)
                &&(CnChar32x29[wordNum].Index[1]==*(cn+1)))
            {
                for(i=0; i<116; i++)
                {   //MSK 的位数
                    color=CnChar32x29[wordNum].Msk[i];
                    for(j=0;j<8;j++)
                    {
                        if((color&0x80)==0x80)
                        {
                            LCD_DrawFRONT_COLOR(x,y,FRONT_COLOR);
                        }
                        else
                        {
                            LCD_DrawFRONT_COLOR(x,y,BACK_COLOR);
                        }
                        color<<=1;
                        x++;
                        if((x-x0)==32)
                        {
                            x=x0;
                            y++;
                            if((y-y0)==29)
                            {
                                y=y0;
                            }
                        }
                    }//for(j=0;j<8;j++)结束
                }
            }
```

```
        } //for (wordNum=0; wordNum<20; wordNum++)结束
        cn += 2;
        x += 32;
        x0=x;
    }
}
#endif

void LCD_ShowPicture(u16 x, u16 y, u16 wide, u16 high,u8 *pic)
{
    u16 temp = 0;
    long tmp=0,num=0;
    LCD_Set_Window(x, y, x+wide-1, y+high-1);
    num = wide * high*2 ;
    do
    {
        temp = pic[tmp + 1];
        temp = temp << 8;
        temp = temp | pic[tmp];
        LCD_WriteData_Color(temp);//逐点显示
        tmp += 2;
    }
    while(tmp < num);
}
```

我们给大家提供很多 TFTLCD 操作的 API 函数，如下：

```
void TFTLCD_Init(void);//初始化
void LCD_Set_Window(u16 sx,u16 sy,u16 width,u16 height);//设置窗口
void LCD_Clear(u16 Color);  //清屏
void LCD_Fill(u16 xState,u16 yState,u16 xEnd,u16 yEnd,u16 color);//填充单色
void LCD_Color_Fill(u16 sx,u16 sy,u16 ex,u16 ey,u16 *color);//在指定区域内填充指定颜色块
void LCD_DrawPoint(u16 x,u16 y);//画点
void LCD_DrawFRONT_COLOR(u16 x,u16 y,u16 color);//指定颜色画点
void LCD_DrawBigPoint(u16 x,u16 y,u16 color);
u16 LCD_ReadPoint(u16 x,u16 y);//读点
void LCD_DrawLine(u16 x1, u16 y1, u16 x2, u16 y2);//画线
void LCD_DrawLine_Color(u16 x1, u16 y1, u16 x2, u16 y2,u16 color);//指定颜色画线
void LCD_DrowSign(u16 x, u16 y, u16 color);//画十字标记
void LCD_DrawRectangle(u16 x1, u16 y1, u16 x2, u16 y2);//画矩形
void LCD_Draw_Circle(u16 x0,u16 y0,u8 r);//画圆
void LCD_ShowChar(u16 x,u16 y,u8 num,u8 size,u8 mode);//显示一个字符
```

```
void LCD_ShowNum(u16 x,u16 y,u32 num,u8 len,u8 size);//显示一个数字
void LCD_ShowxNum(u16 x,u16 y,u32 num,u8 len,u8 size,u8 mode);//显示数字
void LCD_ShowString(u16 x,u16 y,u16 width,u16 height,u8 size,u8 *p);//显示字符串
void LCD_ShowFontHZ(u16 x, u16 y, u8 *cn);//显示汉字


void LCD_ShowPicture(u16 x, u16 y, u16 wide, u16 high,u8 *pic);//显示图片
```

我们提供的这么多函数，其实很多函数都是调用画点函数 LCD_DrawPoint 完成的，这些函数的使用方法也很简单，我们就以显示字符函数 LCD_ShowChar 为例进行介绍（其他函数大家可参考对应的源代码）：

```
//在指定位置显示一个字符
//x,y:起始坐标
//num:要显示的字符:" "--->"~"
//size:字体大小 12/16/24
//mode:叠加方式(1)还是非叠加方式(0)
void LCD_ShowChar(u16 x,u16 y,u8 num,u8 size,u8 mode)
{
    u8 temp,t1,t;
    u16 y0=y;
    u8 csize=(size/8+((size%8)?1:0))*(size/2);        //得到字体一个字符对应点阵集所占的字节数
    num=num-' ';//得到偏移后的值（ASCII 字库是从空格开始取模，所以-' '就是对应字符的字库）
    for(t=0;t<csize;t++)
    {
        if(size==12)temp=ascii_1206[num][t];       //调用 1206 字体
        else if(size==16)temp=ascii_1608[num][t]; //调用 1608 字体
        else if(size==24)temp=ascii_2412[num][t]; //调用 2412 字体
        else return;                               //没有的字库
        for(t1=0;t1<8;t1++)
        {
            if(temp&0x80)LCD_DrawFRONT_COLOR(x,y,FRONT_COLOR);
            else if(mode==0)LCD_DrawFRONT_COLOR(x,y,BACK_COLOR);
            temp<<=1;
            y++;
            if(y>=tftlcd_data.height)return;       //超区域了
            if((y-y0)==size)
            {
```

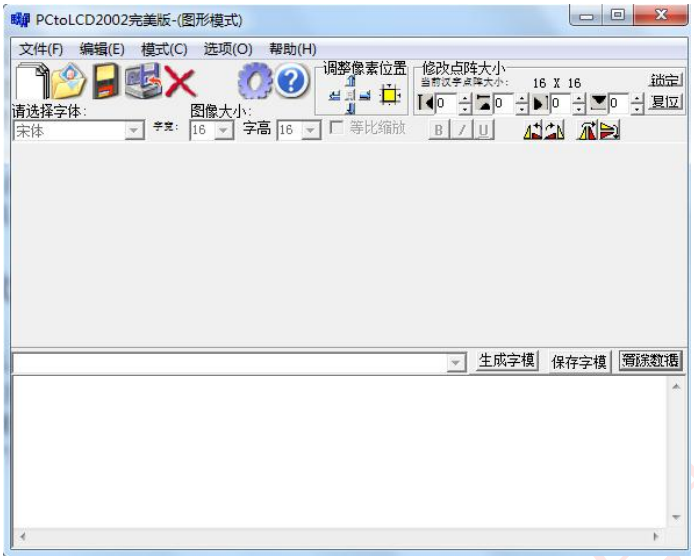
```
                y=y0;
                x++;
                if(x>=tftlcd_data.width)return; //超区域了
                break;
            }
        }
    }
}
```

函数入口参数 x 和 y 用来设置显示的起始位置，num 是要显示的字符，size 用来选择显示字体大小，本实验支持 12/16/24 号的 ASCII 字符显示。mode 用来设置是否支持叠加显示，为 0 表示不使用叠加，为 1 表示使用叠加显示。叠加方式显示多用于在显示的图片上显示字符，非叠加方式一般用于普通的显示。函数内我们用到了三个字符集点阵数据数组 ascii_1206、ascii_1608、ascii_2412。

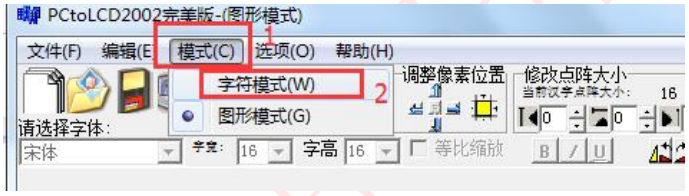
31.3.3 字符汉字图片提取软件使用

下面我们重点介绍下如何让字符显示在 TFTLCD 模块上。要显示字符，我们先要有字符的点阵数据，ASCII 常用的字符集总共有 95 个，从空格符开始，分为：!"#$%&'()*+,-0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~.

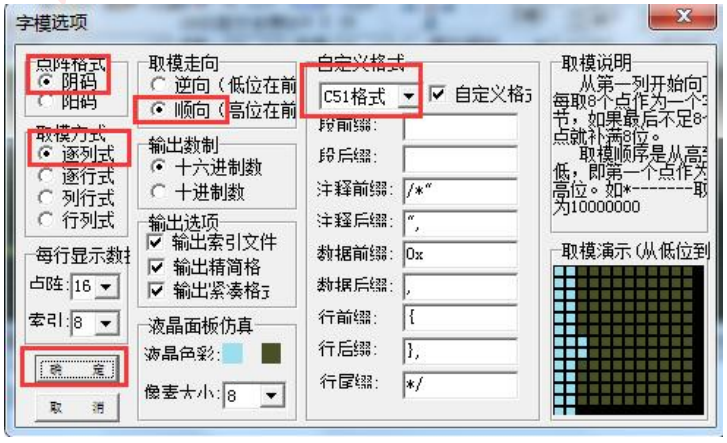
我们先要得到这个字符集的点阵数据，这里我们介绍一个款很好的字符提取软件：PCtoLCD2002 完美版，在资料的“\5--开发工具\4-常用辅助开发软件\PCtoLCD2002 完美版”位置。该软件可以提供各种字符，包括汉字（字体和大小都可以自己设置）阵提取，且取模方式可以设置好几种，常用的取模方式，该软件都支持。该软件还支持图形模式，也就是用户可以自己定义图片的大小，然后画图，根据所画的图形再生成点阵数据，这功能在制作图标或图片的时候很有用。双击这个软件，弹出界面如下：



这里我们介绍下如何取汉字和字符字模数据，点击“模式”，选择“字符模式”，操作如下：

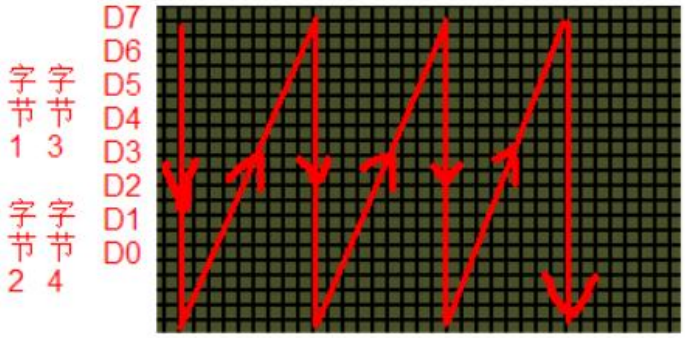


然后点击“选项”，会弹出一个字模选项对话框，将其设置为“阴码+逐列式+顺向+C51 格式”，操作如下：



上图设置的取模方式，在右上角的取模说明里面有，即：从第一列开始向下

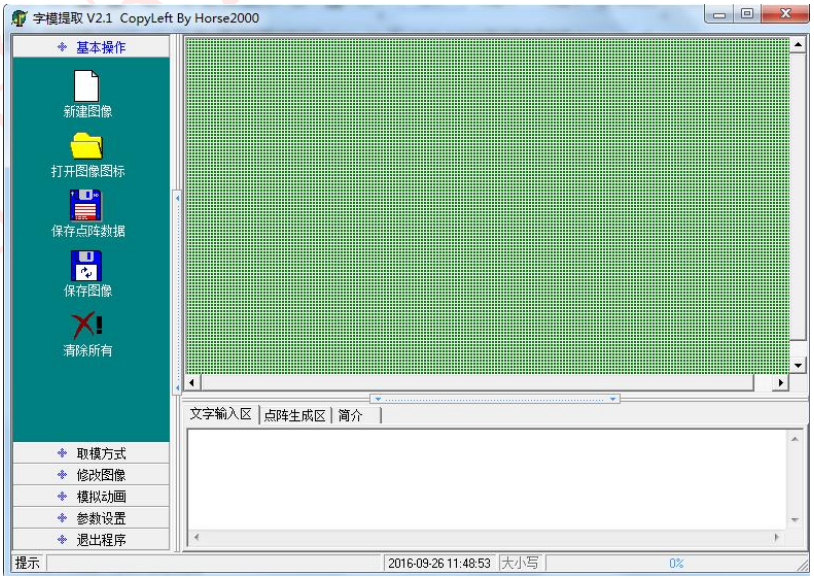
每取 8 个点作为一个字节，如果最后不足 8 个点就补满 8 位。取模顺序是从高到低，即第一个点作为最高位。如*-------取为 10000000。其实就是按如下图 的这种方式：



从上到下，从左到右，高位在前。我们按这样的取模方式，然后把 ASCII 字符集按 12*6 大小、16*8 、24*12 和 32*16 大小取模出来（对应汉字大小为 12*12、16*16 、24*24 和 32*32，字符只有汉字的一半大！），保存在 font.h 里面，每个 12*6 的字符占用 12 个字节，每个 16*8 的字符占用 16 个字节，每个 24*12 的字符占用 36 个字节。具体见 font.h 部分代码。

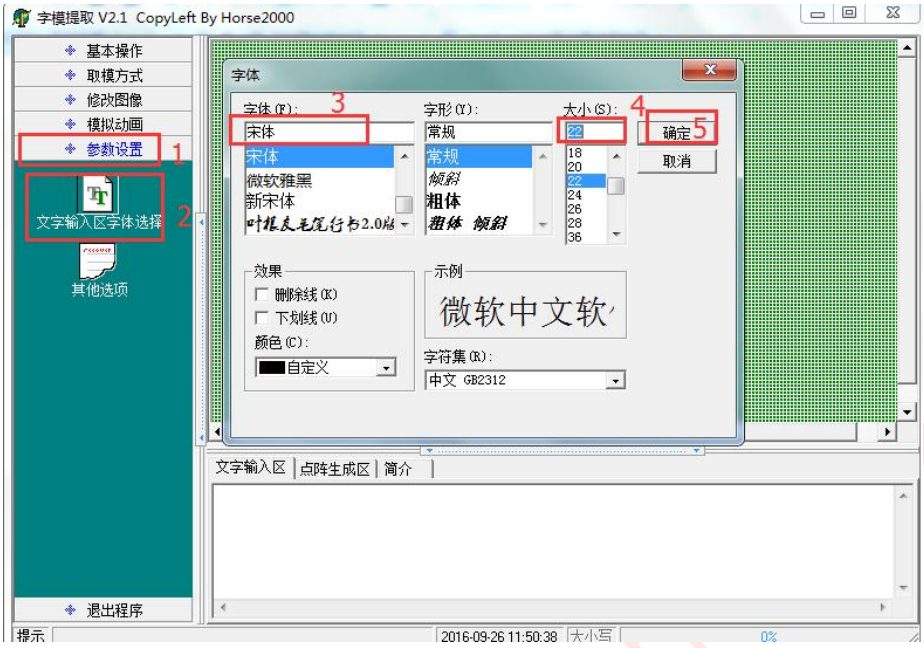
（1）汉字显示

本实验我们还给大家提供了一个汉字显示函数，这里再给大家介绍另一款取模软件，放在资料“\5--开发工具\4-常用辅助开发软件\文字取模软件”目录内，此软件用于汉字取模极其方便，下面打开这个软件，界面如下：

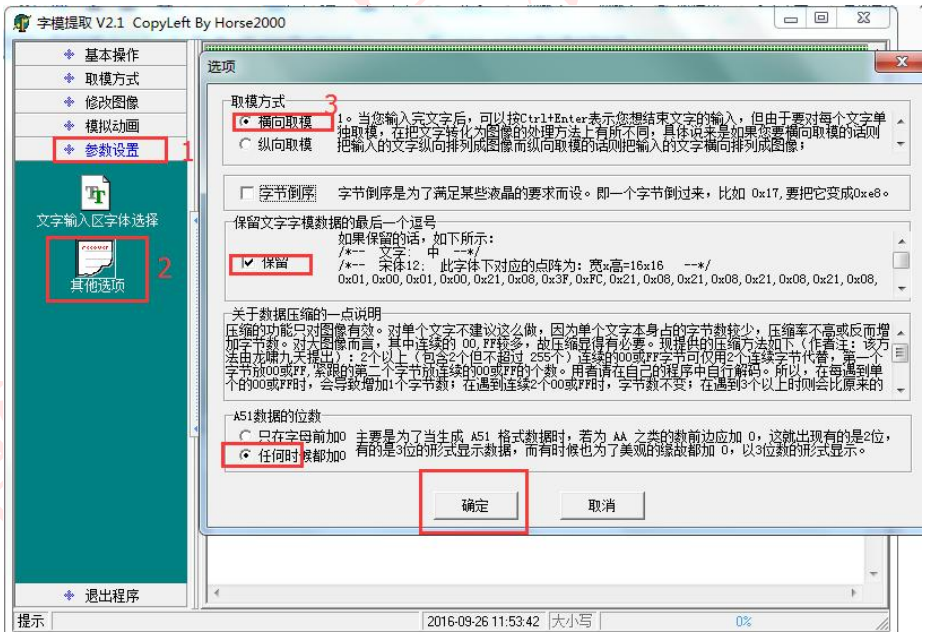


首先我们选择“参数设置”设定好字体类型及大小，本实验所取汉字大小为

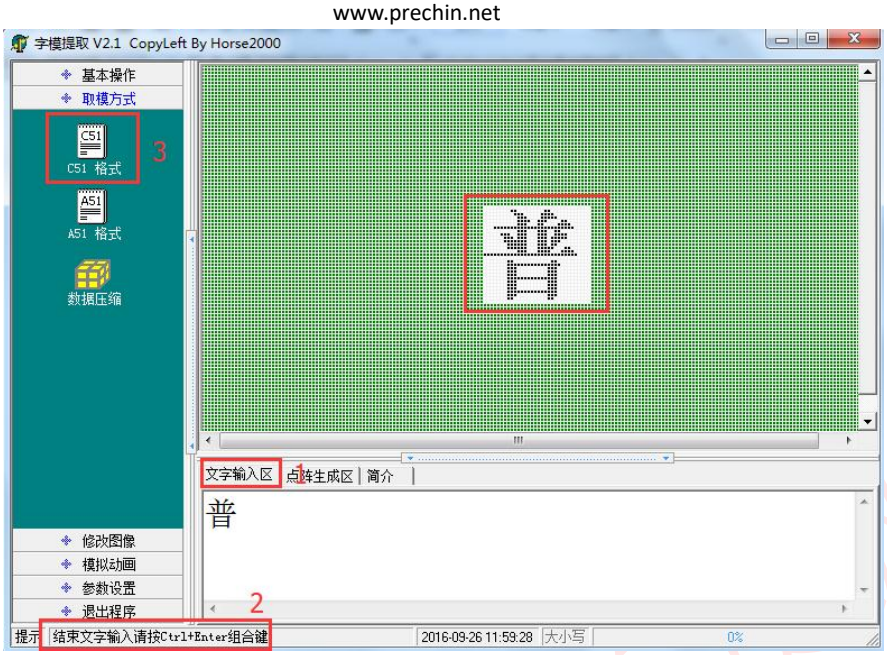
宋体 22 号，具体操作步骤如下：

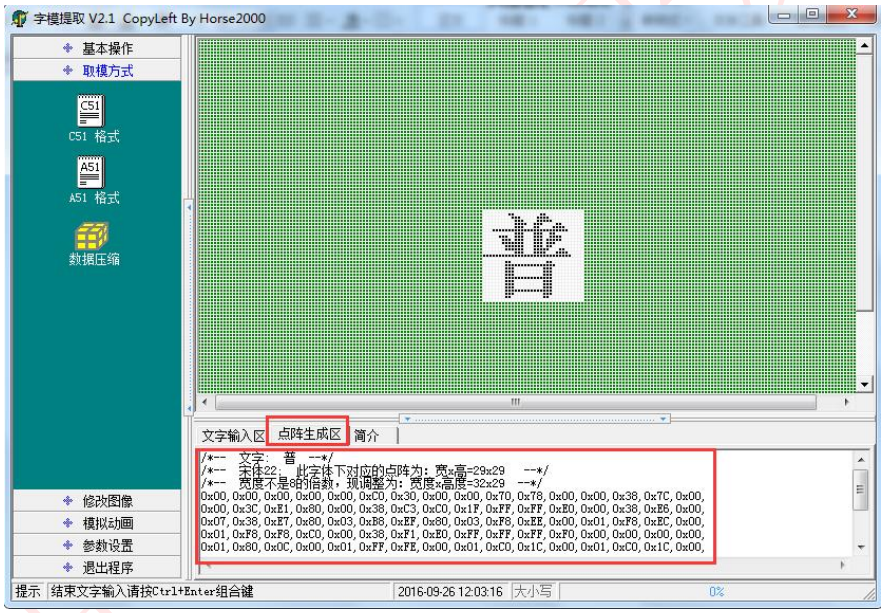


设置好字体大小后，我们再来设置下取模的方式，点击“其他选项”，选择“横向取模”，操作步骤如下：



设置好取模方式后，接下来就是取汉字字模数据了，我们直接在“文字输入区”输入所要显示的汉字，比如输入“普”，然后按下“Ctrl+Enter 组合键”就会在窗口显示输入的汉字，点击“取模方式”选择“C51 格式”，就会在“点阵生成区”显示汉字“普”的字模数据，操作步骤如下：





这样我们直接将其保存在 font.h 的 CnChar32x29 数组内，我们在定义显示汉字时还加了汉字内锁码，所以在保存这些汉字字模数据时，前面加上我们所输入的汉字“普”，如下：

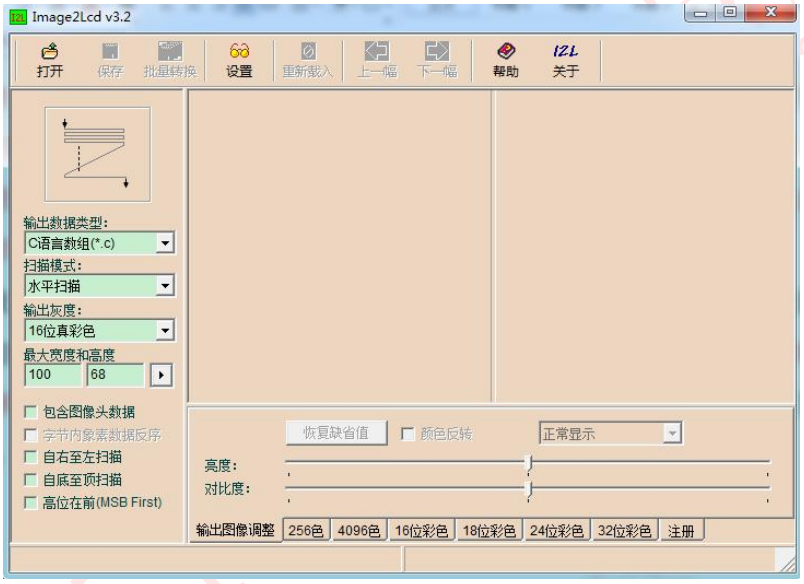
```
"普", 0x00, 0x00, 0x00, 0x00, 0x00, 0xC0, 0x30, 0x00, 0x00, 0x70, 0x78, 0x00, 0x00, 0x38, 0x7C, 0x00,
0x00, 0x3C, 0xE1, 0x80, 0x00, 0x38, 0xC3, 0xC0, 0x1F, 0xFF, 0xFF, 0xE0, 0x00, 0x38, 0xE6, 0x00,
0x07, 0x38, 0xE7, 0x80, 0x03, 0xB8, 0xEF, 0x80, 0x03, 0xF8, 0xEE, 0x00, 0x01, 0xF8, 0xEC, 0x00,
0x01, 0xF8, 0xF8, 0xC0, 0x00, 0x38, 0xF1, 0xE0, 0xFF, 0xFF, 0xFF, 0xF0, 0x00, 0x00, 0x00, 0x00,
0x01, 0x80, 0x0C, 0x00, 0x01, 0xFF, 0xFE, 0x00, 0x01, 0xC0, 0x1C, 0x00, 0x01, 0xC0, 0x1C, 0x00,
0x01, 0xC0, 0x1C, 0x00, 0x01, 0xFF, 0xFC, 0x00, 0x01, 0xC0, 0x1C, 0x00, 0x01, 0xC0, 0x1C, 0x00,
0x01, 0xC0, 0x1C, 0x00, 0x01, 0xFF, 0xFC, 0x00, 0x01, 0xC0, 0x1C, 0x00, 0x01, 0x80, 0x18, 0x00,
0x00, 0x00, 0x00, 0x00,
```

如果要显示其他的汉字，取模方法和上述一样，这些显示函数的具体实现过程，大家可以参考工程源代码，这里就不多介绍。

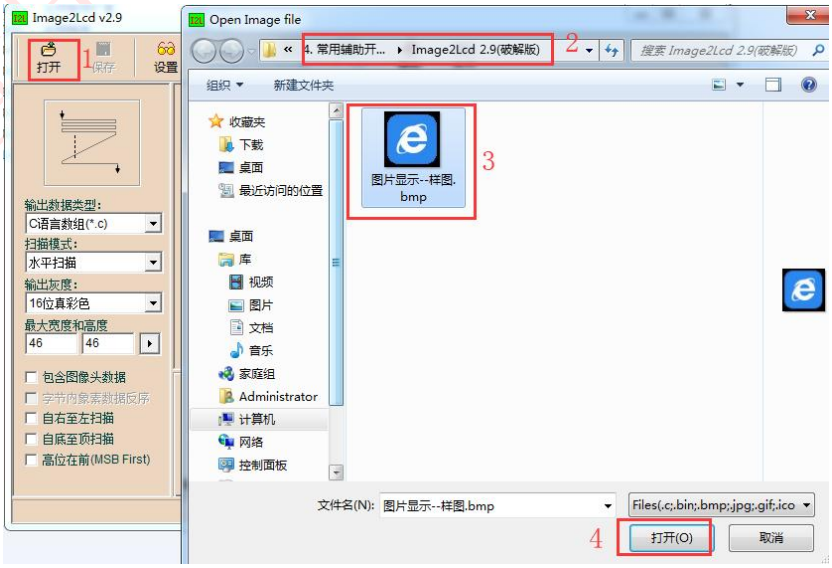
注意：有的朋友在使用取模软件取汉字时会出现这样一种情况，明明是按照教程来设置取模软件参数，但是最后取的汉字数据不是 16*16 大小的，这种情况是因为电脑系统内缺少某种字库，你可以换台电脑测试，如果可以就把系统内字库拷贝到你电脑中去，怎么查找系统字库，这个就需要自己百度。

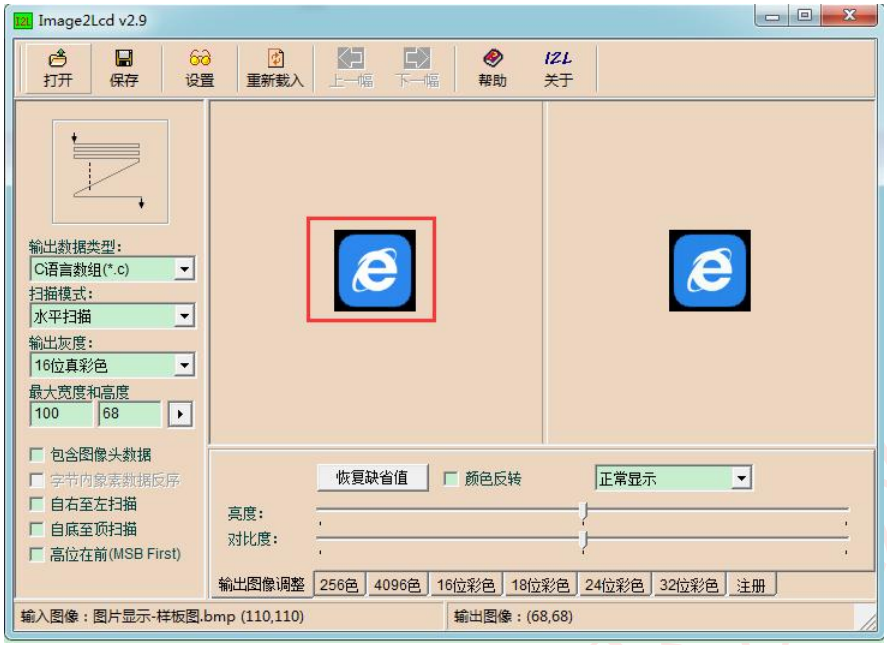
（2）图片显示

在我们的 tftlcd.c 文件内还提供了一个图片显示函数，那么怎么来显示图片呢？这里再给大家介绍另一款取模软件，放在资料“\5--开发工具\4-常用辅助开发软件\Image2Lcd 2.9(破解版)” 目录内，此软件用于图片取模极其方便，打开后界面如下：

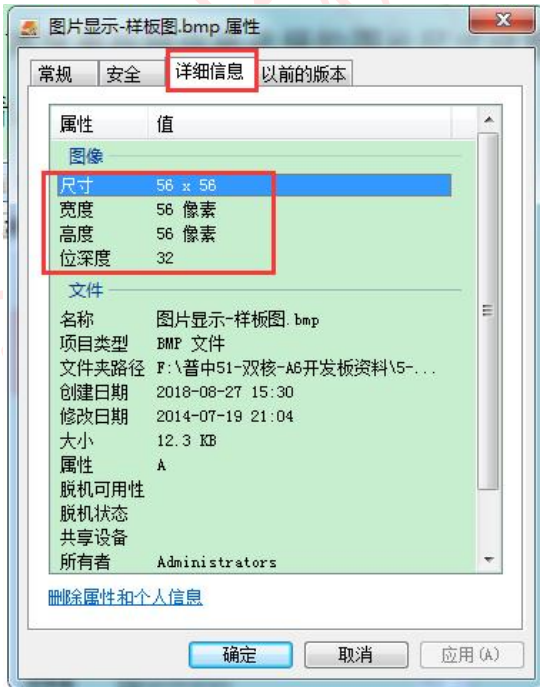


1，点击“打开”菜单按钮，选择你想要显示的图片，注意图片格式要是.BMP，如下所示：

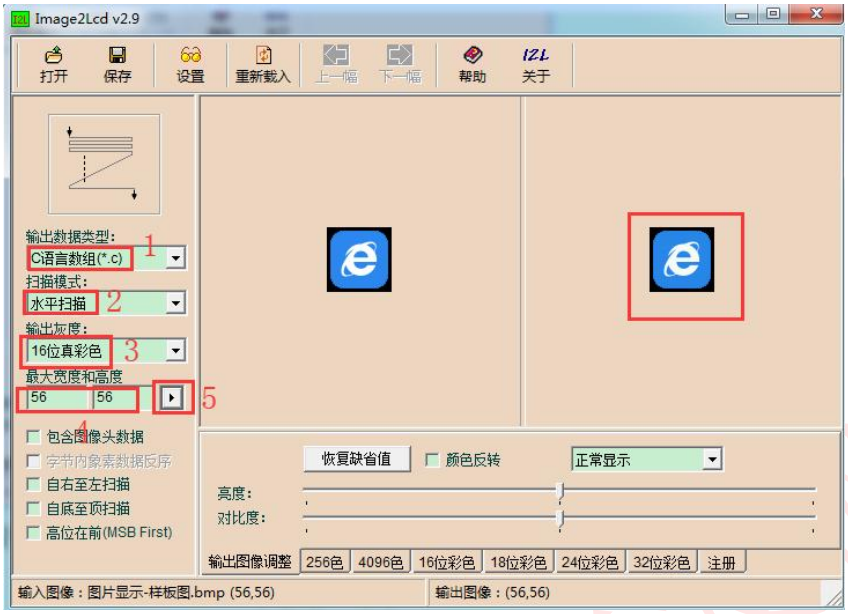




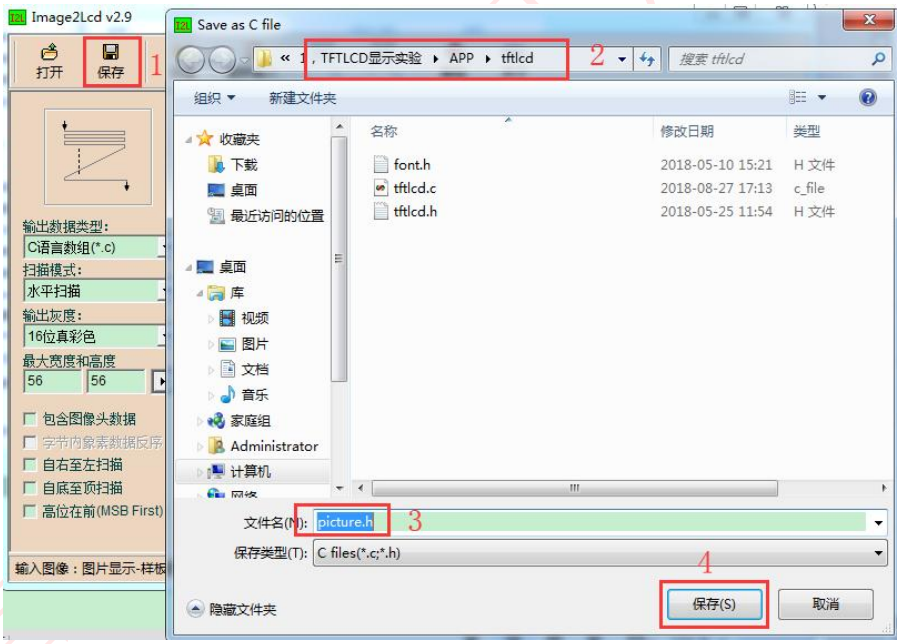
2，设置相关参数，包括输出数据类型、扫描模式、输出灰度、最大宽度和高度等，注意：最大宽度和高度要按照你所选择的图片尺寸设置。图片的实际尺寸大小可点击图片属性查看，如下：



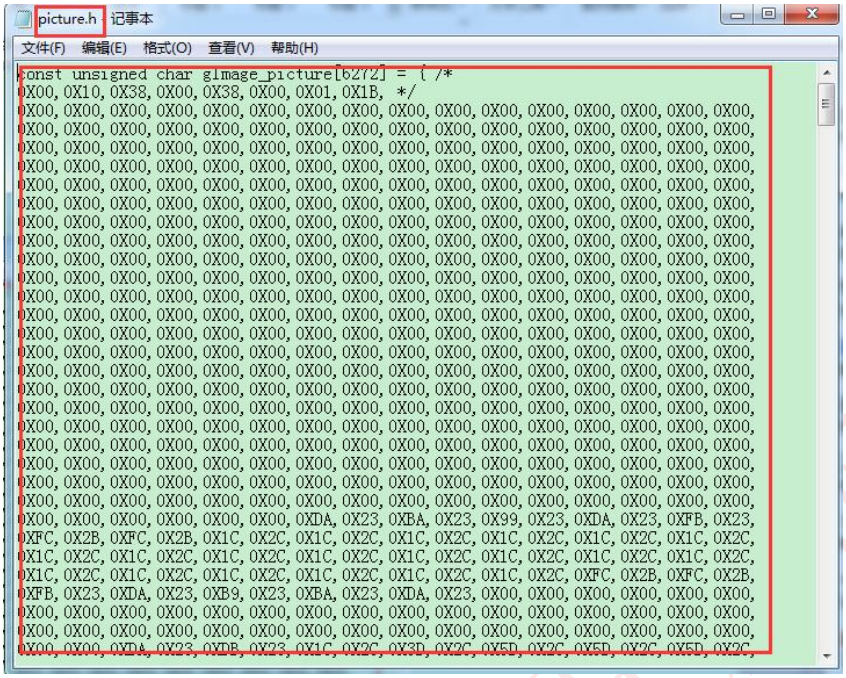
具体设置如下：其他未标注的保持默认即可。



4，点击“保存”菜单按钮，将输出的图片数据保存为 xxx.h 格式，存储到 tftlcd 文件夹内，这里我们就命名为 picture.h，如下：



保存好后就会弹出图片取模后的数据，如下：



上述 gImage_picture 数组即为该图片的数据。

然后在 main.c 文件内将 picture.h 头文件包含进来并且主函数内调用显示图片函数即可，如下：

```
LCD_ShowPicture(10,60,56,56,gImage_picture);
```

函数中前面两个参数 10,60 为图片显示的起始位置，后面两个参数 56,56 分别是图片的宽度和高度，这两个值一定要保证和前面取模时图片大小一致，否则显示出错。最后一个参数 gImage_picture 即为存储图片数据的数组。

31.3.4 主函数

编写好 TFTLCD 初始化和显示函数后，接下来就可以编写主函数了，代码如下：

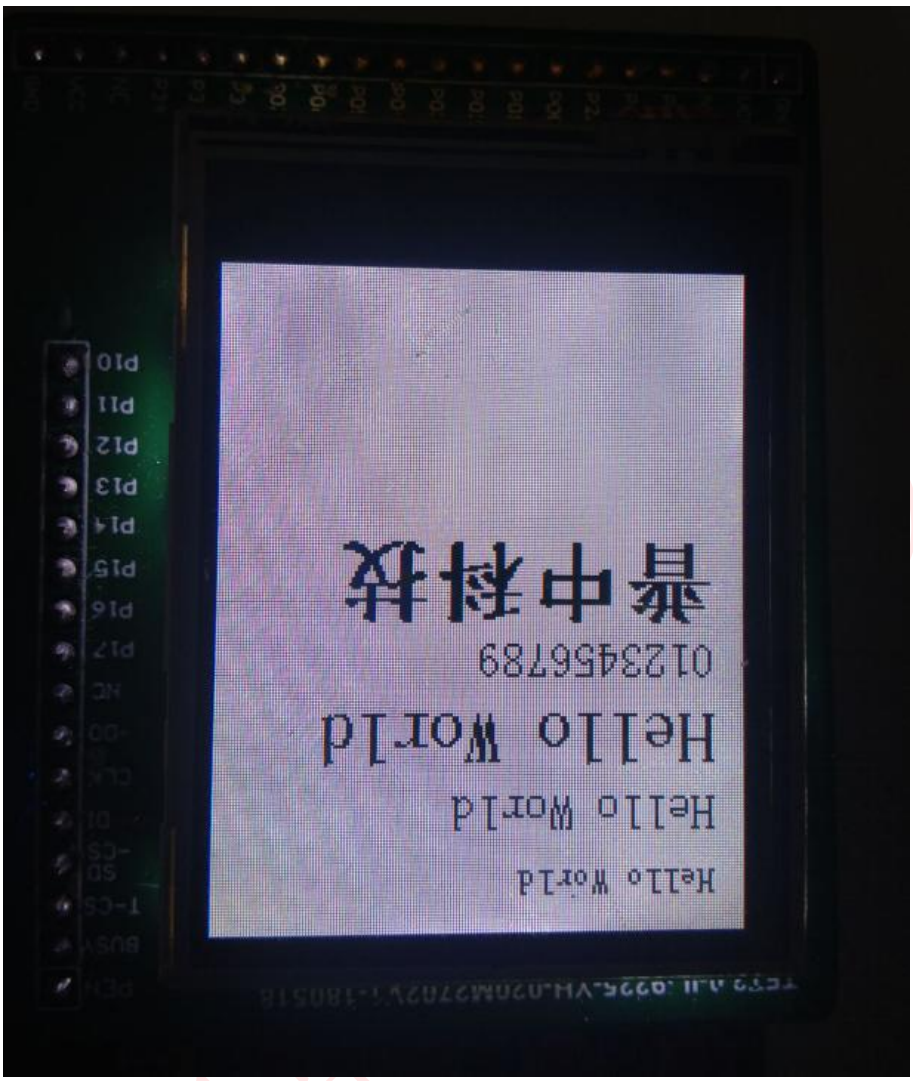
```
#include "public.h"
#include "uart.h"
#include "tftlcd.h"
#include "gui.h"
#include "touch.h"
//#include "picture.h"



void main()
```

```
{
    UART_Init();//串口初始化，方便使用 printf 调试
    TFTLCD_Init();//TFTLCD 初始化

    FRONT_COLOR=WHITE;
    LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,12,"Hel
lo World");
    LCD_ShowString(10,30,tftlcd_data.width,tftlcd_data.height,16,"Hel
lo World");
    LCD_ShowString(10,50,tftlcd_data.width,tftlcd_data.height,24,"Hel
lo World");
    LCD_ShowString(10,80,tftlcd_data.width,tftlcd_data.height,16,"012
3456789");
    LCD_ShowFontHZ(10,100,"普中科技");
    //LCD_ShowPicture(10,60,56,56,gImage_picture);

    while(1)
    {

    }
}
```

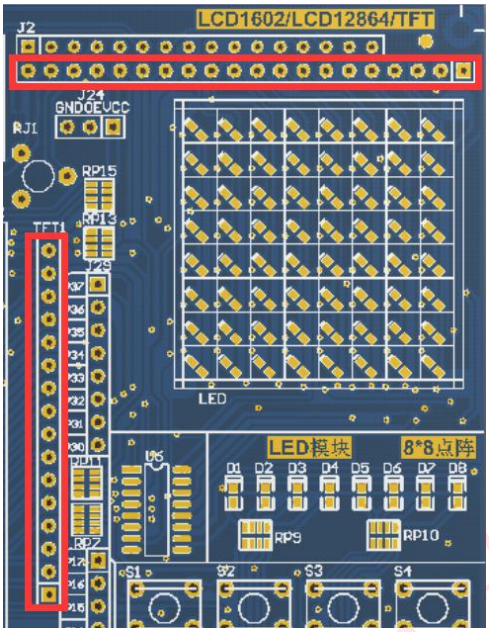
主函数实现的功能很简单，首先调用之前编写好的硬件初始化函数。然后调用我们前面编写的 TFTLCD_Init 函数，用来初始化 TFTLCD，默认设置为竖屏显示，背景颜色为白色。设置显示颜色为黑色后，调用字符串及汉字显示函数进行显示。

31.4 实验现象

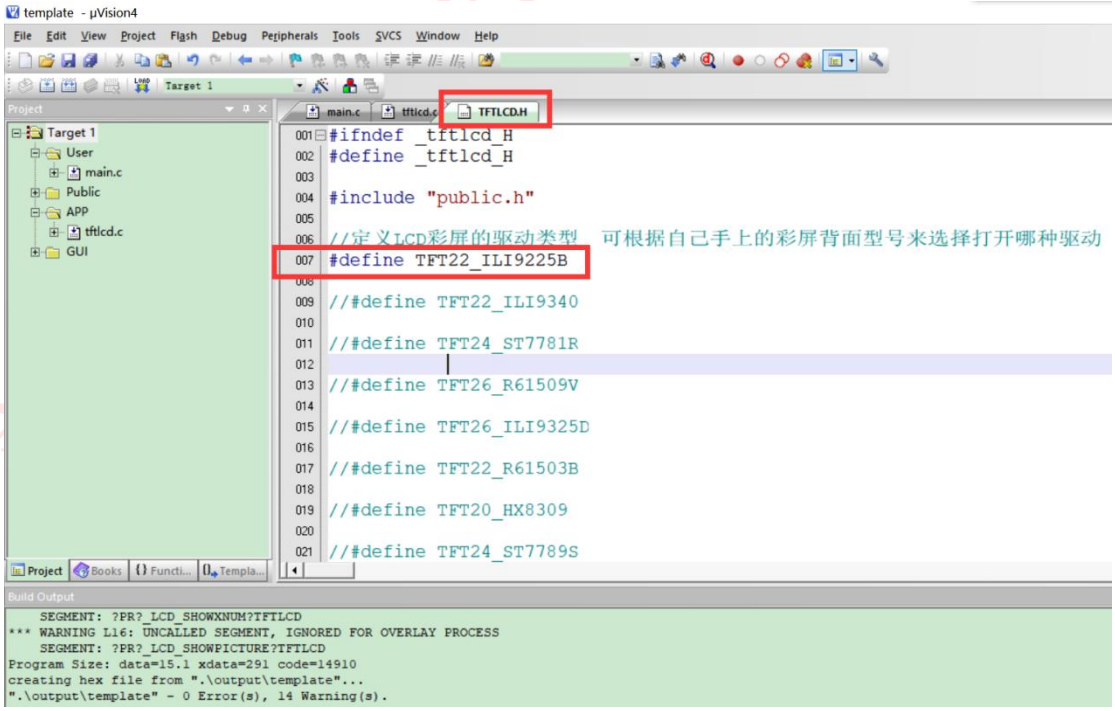
使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），把编译后产生的.hex 文件烧入到芯片内，实现现象如下：将 TFTLCD 模块插上开发板上的彩屏接口，按下复位键后，TFTLCD 即可显示字符及汉字信息。



注意：TFTLCD 显示屏要对应开发板彩屏接口插入，否则实验现象不正常，如下：



实验说明：下载有关 TFTLCD 显示实验时，请确保您所使用的 TFTLCD 彩屏驱动型号与程序内选择的一致，如何查看呢？首先看下您 TFTLCD 彩屏板左上角对应的驱动型号，比如 TFT22_ILI9225B。然后打开实验程序，查看 tftlcd.h 头文件开始处对应的 TFTLCD 驱动型号选择的是哪个？如下所示：



课后作业

第 32 章 触摸屏实验

在前面章节我们介绍了使用 TFTLCD 模块显示字符和汉字，利用 TFTLCD 模块，51 单片机系统就有了高级信息输出的功能，如果还希望有一个友好的用户输入的设备，触摸屏就是非常好的选择，现如今大多电子产品是将触摸屏配合液晶显示器组成人机交互系统，比如手机、平板等。因此很有必要学习下触摸屏的控制。这一章我们来学习如何使用 51 单片机来驱动触摸屏。我们开发板本身并没有触摸屏控制器，但是它支持触摸屏，可以通过外接带触摸屏的 LCD 模块（比如 TFTLCD 模块）来实现触摸屏控制。本章要实现的功能是：通过 TFTLCD 模块上的触摸板（包括电阻触摸和电容触摸）实现触摸功能，最终实现一个画板的功能。学习本章可以参考彩屏相关芯片数据手册（电阻触摸屏和电容触摸屏）。本章分为如下几部分内容：

32.1 触摸屏介绍

32.2 硬件设计

32.3 软件设计

32.4 实验现象

普中51单片机开发板

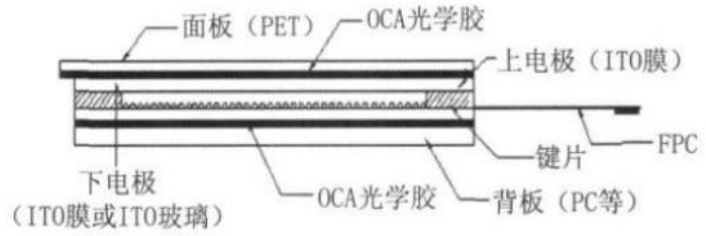
32.1 触摸屏介绍

触摸屏又称触控面板，它是一种把触摸位置转化成坐标数据的输入设备，根据触摸屏的检测原理，主要分为电阻式触摸屏和电容式触摸屏。下面我们就分别来介绍下这两种触摸屏。

32.1.1 电阻式触摸屏介绍

电阻式触摸屏是一种传感器，它将矩形区域中触摸点(X,Y)的物理位置转换为代表 X 坐标和 Y 坐标的电压。很多 LCD 模块都采用了电阻式触摸屏，比如我们 2.0/2.2/2.4/2.6/2.8/3.0/3.2/3.5 寸的 TFTLCD 模块都是采用电阻式触摸屏。使用时需要用一定的压力才会能检测到电压，即触摸。

电阻式触摸屏基本上是薄膜加上玻璃的结构，薄膜和玻璃相邻的一面上均涂有 ITO(纳米铟锡金属氧化物)涂层，ITO 具有很好的导电性和透明性。当触摸操作时，薄膜下层的 ITO 会接触到玻璃上层的 ITO，经由感应器传出相应的电信号，经过转换电路送到处理器，通过运算转化为屏幕上的 X、Y 值，而完成点选的动作，并呈现在屏幕上。电阻式触摸屏结构如下图所示：



电阻触摸屏的工作原理主要是通过压力感应原理来实现对屏幕内容的操作和控制的，这种触摸屏屏体部分是一块与显示器表面非常配合的多层复合薄膜，其中第一层为玻璃或有机玻璃底层，第二层为隔层，第三层为多元树脂表层，表面还涂有一层透明的导电层，上面再盖有一层外表面经硬化处理、光滑防刮的塑料层。在多元脂表层表面的传导层及玻璃层感应器是被许多微小的隔层所分隔电流通过表层，轻触表层压下时，接触到底层，控制器同时从四个角读出相称的电流及计算手指位置的距离。这种触摸屏利用两层高透明的导电层组成触摸屏，两

层之间距离仅为 2.5 微米。当手指触摸屏幕时，平常相互绝缘的两层导电层就在触摸点位置有了一个接触，因其中一面导电层接通 Y 轴方向的电源均匀

电压场，使得侦测层的电压由零变为非零，控制器侦测到这个接通后，进行 A/D 转换，并将得到的电压值与参考电压相比，即可得触摸点的 Y 轴坐标，同理得出 X 轴的坐标，这就是所有电阻技术触摸屏共同的最基本原理。

电阻触摸屏的优点：精度高、价格便宜、抗干扰能力强、稳定性好。

电阻触摸屏的缺点：容易被划伤、透光性不太好、不支持多点触摸。

从上面的简介，我们知道触摸屏都需要一个 AD 转换器，也就是要将电压变化读取出来，供主机求出触摸的位置。我们的 TFTLCD 模块使用的是四线电阻式触摸屏，这种触摸屏的控制芯片有很多，包括：ADS7843、ADS7846、TSC2046、XPT2046 和 AK4182 等。这几款芯片的驱动基本上是一样的，也就是说你只要写出了 XPT2046 的驱动，这个驱动对其他几个芯片也是有效的。而且封装也有一样的，而且管脚也完全兼容。所以在替换起来非常方便。

我们彩屏上面使用的触摸屏控制芯片是 XPT2046。该芯片在前面 ADC 章节介绍过，这里我们再回顾一下，XPT2046 的特点主要有：

1）一款 4 导线制触摸屏控制器，采用 SPI 模式进行通信。

2）内含 12 位分辨率 125KHz 转换速率逐步逼近型 A/D 转换器。

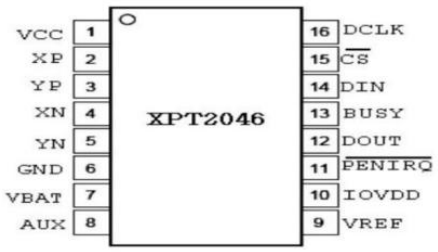
3）支持从 1.5V 到 5.25V 的低电压 I/O 接口。

4）只需执行两次 A/D 转换即可查出被按的屏幕位置。

5）可以测量加在触摸屏上的压力

6）芯片内部自带温度检测、电池电压（0-6V）监测等等

XPT2046 SOP 封装有 16 个引脚，如下图所示：



其引脚说明如下图所示：

QFN引脚号	TSSOP引脚号	VFBGA引脚号	名称	说明
1	13	A5	BUSY	忙时信号线，当$\overline{CS}$为高电平时为高阻状态
2	14	A4	DIN	串行数据输入端，当$\overline{CS}$为低电平时，数据在DCLK上升沿锁存进来
3	15	A3	$\overline{CS}$	片选信号，控制转换时序和使能串行输入输出寄存器，高电平时ADC掉电
4	16	A2	DCLK	外部时钟信号输入
5	1	B1和C1	VCC	电源输入端
6	2	D1	XP	XP位置输入端
7	3	E1	YP	YP位置输入端
8	4	G2	XN	XN位置输入端
9	5	G3	YN	YN位置输入端
10	6	G4和G5	GND	接地
11	7	G6	VBAT	电池监视输入端
12	8	E7	AUX	ADC辅助输入通道
13	9	D7	VREF	参考电压输入/输出
14	10	C7	IOVDD	数字电源输入端
15	11	B7	PENIRQ	笔接触中断引脚
16	12	A6	DOUT	串行数据输出端，数据在DCLK的下降沿移出，当$\overline{CS}$高电平时为高阻状态

芯片详细资料大家可以参考数据手册，在前面 ADC 模数转换实验章节也介绍过，可以返回看下。

32.1.2 电容式触摸屏介绍

现在几乎所有智能手机，包括平板电脑都是采用电容屏作为触摸屏，电容屏是利用人体感应进行触点检测控制，不需要直接接触或只需要轻微接触，通过检测感应电流来定位触摸坐标。

我们有的 3.5/4.3/4.5/7 寸 TFTLCD 模块上使用的触摸屏是电容式触摸屏，不过这些高端屏都是配置在 STM32 开发板上，下面简单介绍下电容式触摸屏的原理。电容式触摸屏主要分为两种：

（1）表面电容式电容触摸屏。

表面电容式触摸屏技术是利用 ITO(铟锡氧化物，是一种透明的导电材料)导电膜，通过电场感应方式感测屏幕表面的触摸行为进行。但是表面电容式触摸屏有一些局限性，它只能识别一个手指或者一次触摸。

（2）投射式电容触摸屏。

投射式电容触摸屏却具有多指触控的功能。这两种电容式触摸屏都具有透光率高、反应速度快、寿命长等优点，缺点是：随着温度、湿度的变化，电容值会发生变化，导致工作稳定性差，时常会有漂移现象，需要经常校对屏幕，且不可佩戴普通手套进行触摸定位。

投射电容式触摸屏是传感器利用触摸屏电极发射出静电场线。一般用于投射电容传感技术的电容类型有两种：自我电容和交互电容。

自我电容又称绝对电容，是最广为采用的一种方法，自我电容通常是指扫描电极与地构成的电容。在玻璃表面有用 ITO 制成的横向与纵向的扫描电极，这些电极和地之间就构成一个电容的两极。当用手或触摸笔触摸的时候就会并联一个电容到电路中去，从而使在该条扫描线上的总体的电容量有所改变。在扫描的时候，控制 IC 依次扫描纵向和横向电极，并根据扫描前后的电容变化来确定触摸点坐标位置。笔记本电脑触摸输入板就是采用的这种方式，笔记本电脑的输入板采用 X*Y 的传感电极阵列形成一个传感格子，当手指靠近触摸输入板时，在手指和传感电极之间产生一个小量电荷。采用特定的运算法则处理来自行、列传感器的信号来确定手指的位置。

交互电容又叫做跨越电容，它是在玻璃表面的横向和纵向的 ITO 电极的交叉处形成电容。交互电容的扫描方式就是扫描每个交叉处的电容变化，来判定触摸点的位置。当触摸的时候就会影响到相邻电极的耦合，从而改变交叉处的电容量，交互电容的扫面方法可以侦测到每个交叉点的电容值和触摸后电容变化，因而它需要的扫描时间与自我电容的扫描方式相比要长一些，需要扫描检测 X*Y 根电极。目前智能手机/平板电脑等的触摸屏，都是采用交互电容技术。

我们使用的电容屏也是采用投射式电容屏（交互电容类型），所以后面仅以投射式电容屏作为介绍。

投射式电容触摸屏（交互电容类型）内部由驱动电极与接收电极组成，驱动电极发出低电压高频信号投射到接收电极形成稳定的电流，当人体接触到电容屏时，由于人体接地，手指与电容屏就形成一个等效电容，而高频信号可以通过这一等效电容流入地线，这样，接收端所接收的电荷量减小，而当手指越靠近发射端时，电荷减小越明显，最后根据接收端所接收的电流强度来确定所触碰的点。以上就是电容触摸屏的基本原理，更多详细的资料大家可以百度了解下。

电容触摸屏通常也需要一个驱动 IC 来检测电容触摸，且一般是通过 I2C 接口输出触摸数据的。我们 4.5 寸电容触摸屏使用的驱动 IC 是 FT5336，不清楚自己彩屏驱动芯片及驱动触摸 IC 型号的可以看下彩屏表面及背面的型号。FT5336 支持最多 5 点触摸，这里我们以 FT5536 为例进行介绍，其他的驱动 IC 参考着学习。

FT5336 是台湾敦泰電子股份有限公司生产的一颗电容触摸屏驱动 IC，最多

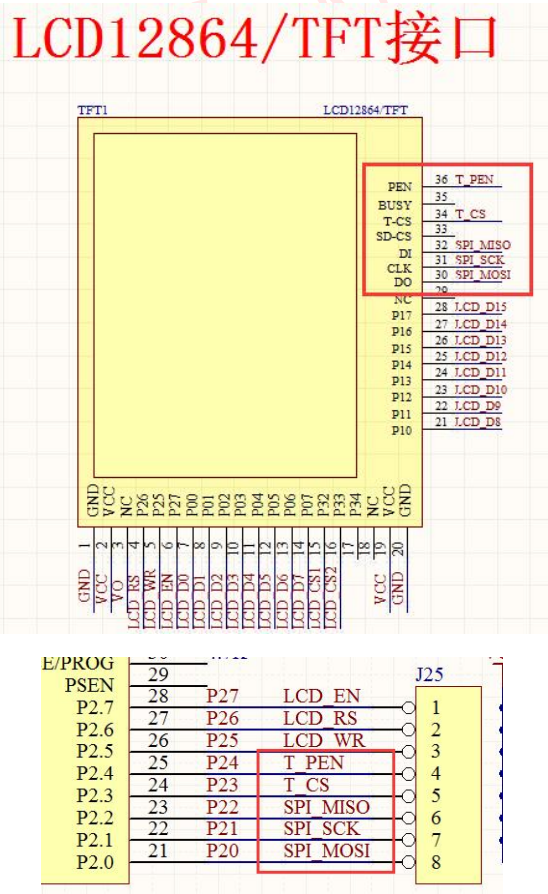
支持 13*24（314）个通道。支持 SPI/IIC 接口，我们使用的就是 IIC 接口。IIC 接口模式下，该驱动 IC 与 51 单片机的连接仅需要 4 根线：SDA、SCL、RST 和 INT，SDA 和 SCL 是 IIC 通信用的，RST 是复位脚（低电平有效），INT 是中断输出信号，关于 IIC 我们就不详细介绍了，在前面 I2C 实验时已经讲过。关于 FT5336 或者其他电容触摸驱动 IC 详细内容，大家可以在彩屏数据手册内查看，不过对于 51 开发板资料中我们未提供，仅仅顺带介绍下，让大家对电容屏有一个认识。这里不多说，我们主要还是看代码，从代码中体会它们的使用。

32.2 硬件设计

本实验使用到硬件资源如下：

（1）TFTLCD 液晶模块（带电阻触摸屏）

TFTLCD 液晶接口在前面章节已经介绍过，下面来看下 51 单片机与 TFTLCD 触摸屏的连接关系，如下图所示：



从电路图中可以看到，SPI_MOSI、SPI_MISO、SPI_SCK、T_CS 和 T_PEN 分别

连接在 51 单片机的 P20、P22、P21、P23 和 P24 上。 而 SPI_MOSI、 SPI_MISO、SPI_SCK、T_CS 和 T_PEN 引脚之前在介绍 TFTLCD 显示实验的时候就给大家介绍，如果是电阻触摸屏的话，它是直接连接在 XPT2046 触摸芯片 SPI 接口及笔中断引脚上的。如果是电容触摸屏的话，只需要 4 根线即可分别是 T_PEN(INT) 、T_CS(RST)、SPI_CLK(SCL) 和 SPI_MOSI(SDA)。其中：INT、RST、SCL 和 SDA 分别是 FT5336 的中断输出信号、复位信号，IIC 的 SCL 和 SDA 信号。 这里我们用查询的方式读取 FT5336 的数据， 没有用到中断信号（INT），所以同 51 单片机的连接，只需要 3 根线即可，不过 我们还是预留 INT 那根，方面扩展的其他驱动 IC 做 IIC 地址设定，所以保持 4 根线连接。因为 51 单片机开发板我们未配置电容屏，所以上述关于电容屏的硬件介绍大家可以不用管它。

32.3 软件设计

本章所要实现的功能是：通过 TFTLCD 模块上的触摸板实现触摸功能，最终实现一个画板的功能。

程序框架如下：

（1）编写触摸屏扫描函数

（2）编写主函数

本章软件的重点是如何对触摸屏进行写命令和数据以及触摸位置获取函数。下面我们打开“\4--实验程序\1--基础实验\28-触摸屏实验”工程，在 App 工程组中可以看到新添加了 touch.c 文件（里面包含了触摸屏驱动程序），还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

32.3.1 触摸屏扫描函数

打开 touch.c 文件，代码如下：

```
#include "touch.h"
#include "tftlcd.h"
#include "uart.h"
```

```
XPT_XY xpt_xy;     //定义一个全局变量保存 X、Y 的值



void TOUCH_SPI_Start(void)
{
    TOUCH_CLK = 0;
    TOUCH_CS  = 1;
    TOUCH_DIN = 1;
    TOUCH_CLK = 1;
    TOUCH_CS  = 0;
}

void TOUCH_SPI_Write(u8 dat)
{
    u8 i;
    TOUCH_CLK = 0;
    for(i=0; i<8; i++)
    {
        TOUCH_DIN = dat >>7;     //放置最高位
        dat <<= 1;
        TOUCH_CLK = 0;           //上升沿放置数据

        TOUCH_CLK = 1;

    }
}

u16 TOUCH_SPI_Read(void)
{
    u16 i, dat=0;
    TOUCH_CLK = 0;
    for(i=0; i<12; i++)      //接收 12 位数据
    {
        dat <<= 1;

        TOUCH_CLK = 1;
        TOUCH_CLK = 0;

        dat |= TOUCH_DOUT;

    }
    return dat;
}
```

```
#define XY_READ_TIMS  10        //读取的次数

u16 TOUCH_XPT_ReadData(u8 cmd)
{
    u8 i, j;
    u16 readValue[XY_READ_TIMS];
    long endValue;

    TOUCH_CLK = 0;    //先拉低时间
    TOUCH_CS  = 0;    //选中芯片

    for(i=0; i<XY_READ_TIMS; i++)       //读取 XY_READ_TIMS 次结果
    {
        TOUCH_SPI_Write(cmd);  //发送转换命令
        //delay_10us();
        for(j=6; j>0; j--);  //延时等待转换结果
        TOUCH_CLK = 1;      //发送一个时钟周期，清除 BUSY
        _nop_();
        _nop_();
        TOUCH_CLK = 0;
        _nop_();
        _nop_();

        readValue[i] = TOUCH_SPI_Read();
    }
    TOUCH_CS = 1; //释放片选

    //---软件滤波---//
    //---先大到小排序，除去最高值，除去最低值，求其平均值---//
    for(i=0; i<XY_READ_TIMS - 1; i++)   //从大到小排序
    {
        for(j= i+1; j<XY_READ_TIMS; j++)
        {
            if(readValue[i] < readValue[j])
            {
                endValue = readValue[i];
                readValue[i] = readValue[j];
                readValue[j] = endValue;
            }
        }
    }
//  if((readValue[2] - readValue[3]) > 5)
//  {
```

```
//      return 0;
//  }
    endValue = 0;
    for(i=2; i<XY_READ_TIMS-2; i++)
    {
        endValue += readValue[i];
    }
    endValue = endValue/ (XY_READ_TIMS - 4);//求平均值

    return endValue;
}



u8 TOUCH_XPT_ReadXY(void)
{
    u16 x1, x2, x, y1, y2, y;

    TOUCH_SPI_Start();
    //---分别读两次 X 值和 Y 值，交叉着读可以提高一些读取精度---//
    x1 = TOUCH_XPT_ReadData(XPT_CMD_X);
    y1 = TOUCH_XPT_ReadData(XPT_CMD_Y);
    x2 = TOUCH_XPT_ReadData(XPT_CMD_X);
    y2 = TOUCH_XPT_ReadData(XPT_CMD_Y);


    //---求取 X 值的差值---//
    if (x1 > x2)
    {
        x = x1 - x2;
    }
    else
    {
        x = x2 - x1;
    }

    //---求取 Y 值的差值---//
    if (y1 > y2)
    {
        y = y1 - y2;
    }
    else
    {
        y = y2 - y1;
```

```
    }

    //---判断差值是否大于 50，大于就返回 0，表示读取失败---//
    if((x > 50) || (y > 50))
    {
        return 0;
    }

    //---求取两次读取值的平均数作为读取到的 XY 值---//
    xpt_xy.x = (x1 + x2) / 2;
    xpt_xy.y = (y1 + y2) / 2;

    xpt_xy.x &= 0xFFF0;  //去掉低四位
    xpt_xy.y &= 0xFFF0;

    //---确定 XY 值的范围，用在触摸屏大于 TFT 时---//
    if((xpt_xy.x < 100) || (xpt_xy.y > 3800))
    {
        return 0;
    }

    return 1;  // 返回 1，表示读取成功
}

//返回 1：触摸按下
//0:无触摸
u8 TOUCH_Scan(void)
{
    u8 res=0;
    u32 temp;

    if(TOUCH_XPT_ReadXY())
    {
        //--如果触摸跟显示发生偏移，可以根据显示 AD 值--//
        //--调整下面公式里面的数值--//
        if(tftlcd_data.dir==0)
        {
#ifdef TFT20_HX8309
            xpt_xy.lcdx = xpt_xy.x;
            xpt_xy.lcdx = (xpt_xy.lcdx - 250) * 180 / 3700;
            xpt_xy.lcdy = xpt_xy.y;
            xpt_xy.lcdy = (xpt_xy.lcdy - 350) * 250 / 3500;
#endif
```

```
#ifdef TFT22_ILI9225B
          xpt_xy.lcdx = xpt_xy.x;
          xpt_xy.lcdx = (xpt_xy.lcdx - 250) * 180 / 3700;
          xpt_xy.lcdy = xpt_xy.y;
          xpt_xy.lcdy = (xpt_xy.lcdy - 350) * 250 / 3500;
#endif

#ifdef TFT22_R61503B
          xpt_xy.lcdx = xpt_xy.x;
          xpt_xy.lcdx = (xpt_xy.lcdx - 250) * 180 / 3700;
          xpt_xy.lcdy = xpt_xy.y;
          xpt_xy.lcdy = (xpt_xy.lcdy - 350) * 250 / 3500;
#endif

#ifdef TFT24_ST7781R
          xpt_xy.lcdx=xpt_xy.x;
          xpt_xy.lcdx=((xpt_xy.lcdx - 336)*240)/3328;
          xpt_xy.lcdy=xpt_xy.y;
          xpt_xy.lcdy =((xpt_xy.lcdy - 240) *320)/3412;
#endif

#ifdef TFT26_R61509V
          xpt_xy.lcdx=xpt_xy.x;
          xpt_xy.lcdx=((xpt_xy.lcdx - 240)*260)/3850;
          xpt_xy.lcdy=xpt_xy.y;
          xpt_xy.lcdy =((xpt_xy.lcdy - 200)*420)/3950;
#endif

#ifdef TFT26_ILI9325D
          xpt_xy.lcdx=xpt_xy.x;
          xpt_xy.lcdx=((xpt_xy.lcdx - 600)*297)/3550;
          xpt_xy.lcdy=xpt_xy.y;
          xpt_xy.lcdy =((xpt_xy.lcdy - 250)*362)/3850;
#endif

#ifdef TFT24_ST7789S
          xpt_xy.lcdx=xpt_xy.x;
          xpt_xy.lcdx=((xpt_xy.lcdx - 336)*240)/3328;
          xpt_xy.lcdy=xpt_xy.y;
          xpt_xy.lcdy =((xpt_xy.lcdy - 240) *320)/3412;
#endif

#ifdef TFT24_SSD1297
          xpt_xy.lcdx=xpt_xy.x;
```

```
            xpt_xy.lcdx=240-((xpt_xy.lcdx - 336)*240)/3328;
            xpt_xy.lcdy=xpt_xy.y;
            xpt_xy.lcdy =320-((xpt_xy.lcdy - 240) *320)/3412;
#endif

#ifdef TFT20_ILI9225
            xpt_xy.lcdx = xpt_xy.x;
            xpt_xy.lcdx = (xpt_xy.lcdx - 250) * 180 / 3700;
            xpt_xy.lcdy = xpt_xy.y;
            xpt_xy.lcdy = (xpt_xy.lcdy - 350) * 250 / 3500;
#endif

        }
        else
        {
#ifdef TFT20_HX8309
        xpt_xy.lcdx = 4096-xpt_xy.y;
        xpt_xy.lcdx = (xpt_xy.lcdx - 350) * 250 / 3500;
        xpt_xy.lcdy = xpt_xy.x;
        xpt_xy.lcdy = (xpt_xy.lcdy - 250) * 180 / 3700;
#endif

#ifdef TFT22_ILI9225B
        xpt_xy.lcdx = 4096-xpt_xy.y;
        xpt_xy.lcdx = (xpt_xy.lcdx - 350) * 250 / 3500;
        xpt_xy.lcdy = xpt_xy.x;
        xpt_xy.lcdy = (xpt_xy.lcdy - 250) * 180 / 3700;
#endif

#ifdef TFT22_R61503B
        xpt_xy.lcdx = 4096-xpt_xy.y;
        xpt_xy.lcdx = (xpt_xy.lcdx - 350) * 250 / 3500;
        xpt_xy.lcdy = xpt_xy.x;
        xpt_xy.lcdy = (xpt_xy.lcdy - 250) * 180 / 3700;
#endif

#ifdef TFT24_ST7781R
            xpt_xy.lcdx=xpt_xy.y;
            xpt_xy.lcdx =((xpt_xy.lcdx - 240) *320)/3412;
            xpt_xy.lcdy=4096-xpt_xy.x;
            xpt_xy.lcdy=((xpt_xy.lcdy - 336)*240)/3328;
#endif

#ifdef TFT26_R61509V
```

```
            xpt_xy.lcdx=xpt_xy.y;
            xpt_xy.lcdx=((xpt_xy.lcdx - 200)*420)/3950;
            xpt_xy.lcdy=4096 - xpt_xy.x;
            xpt_xy.lcdy =((xpt_xy.lcdy - 240)*260)/3850;
#endif

#ifdef TFT26_ILI9325D
            xpt_xy.lcdx=xpt_xy.y;
            xpt_xy.lcdx=((xpt_xy.lcdx - 250)*362)/3850;
            xpt_xy.lcdy=4096 - xpt_xy.x;
            xpt_xy.lcdy =((xpt_xy.lcdy - 600)*297)/3550;
#endif

#ifdef TFT24_ST7789S
            xpt_xy.lcdx=xpt_xy.y;
            xpt_xy.lcdx =((xpt_xy.lcdx - 240) *320)/3412;
            xpt_xy.lcdy=4096-xpt_xy.x;
            xpt_xy.lcdy=((xpt_xy.lcdy - 336)*240)/3328;
#endif

#ifdef TFT24_SSD1297
            xpt_xy.lcdx=xpt_xy.y;
            xpt_xy.lcdx =((xpt_xy.lcdx - 240) *320)/3412;
            xpt_xy.lcdy=4096-xpt_xy.x;
            xpt_xy.lcdy=((xpt_xy.lcdy - 336)*240)/3328;
#endif

#ifdef TFT20_ILI9225
        xpt_xy.lcdx = 4096-xpt_xy.y;
        xpt_xy.lcdx = (xpt_xy.lcdx - 350) * 250 / 3500;
        xpt_xy.lcdy = xpt_xy.x;
        xpt_xy.lcdy = (xpt_xy.lcdy - 250) * 180 / 3700;
#endif

        }


//      if(tftlcd_data.dir==0)
//      {
//          xpt_xy.lcdx=xpt_xy.lcdx;
//          xpt_xy.lcdy=xpt_xy.lcdy;
//      }
//      else
//      {
```

```
//          temp=xpt_xy.lcdx;
//          xpt_xy.lcdx=xpt_xy.lcdy;
//          xpt_xy.lcdy=tftlcd_data.height-temp;
//      }

//      printf("ad_x=%d   ad_y=%d\r\n",xpt_xy.x,xpt_xy.y);
//      printf("lcdx=%d   lcdy=%d\r\n",xpt_xy.lcdx,xpt_xy.lcdy);
      res=1;
      xpt_xy.sta=1;
    }
    else
        xpt_xy.sta=0;
    return res;
}
```

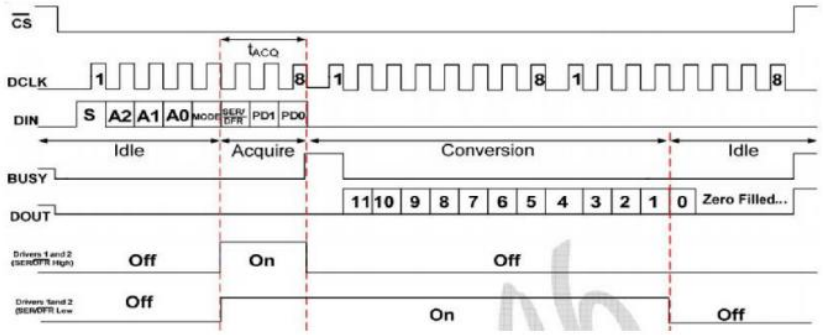
TOUCH_SPI_Write、TOUCH_SPI_Read 函数是 51 单片机 IO 口模拟 SPI 时序的读写函数程序，TOUCH_SPI_Start 函数是 IO 口初始化程序，TOUCH_XPT_ReadXY 是一个比较关键的函数，用来读取触摸屏 X/Y 轴的物理坐标，因为只有获取物理坐标后我们才能转换对应 LCD 的坐标，这部分后面会介绍。

TOUCH_XPT_ReadXY 内首先通过 TOUCH_ReadData 函数获取触摸屏物理坐标值，然后通过相应的程序滤波，保证数据的准确性，防止出现飞点等误差。比较常用的程序滤波的方法其实就是多次数据的读取，然后把最大最小值除去，算出平均值。这种方法读取的次数越多，得到的数据就越准确。不过为了更好的滤波，还使用了另外一种方式进行滤波，也就是当读取到两次数据之后，然后检查两个数据之间的差值，如果超过理想的误差，那么丢弃数据。这种方法也是处理飞点现象的常用方法。

该函数里面用到了很多的宏，比如 TOUCH_X_CMD、TOUCH_Y_CMD、TOUCH_MAX 等，这些都在 touch.h 文件内定义了。TOUCH_X_CMD=0xD0 和 TOUCH_Y_CMD=0X90 是 XPT2046 AD 芯片读取 X 和 Y 轴的命令，可通过《 XPT2046》 芯片数据手册查找到，如下图：

A2	A1	A0	V_{BAT}	AUX_{IN}	TEMP	YN	XP	YP	Y-位置	X-位置	Z_1-位置	Z_2-位置	X-驱动	Y-驱动
0	0	0			+IN (TEMP0)								off	off
0	0	1					+IN		测量				Off	On
0	1	0	+IN										Off	Off
0	1	1					+IN				测量		XN, On	YP, On
1	0	0				+IN						测量	XN, On	YP, On
1	0	1						+IN		测量			On	Off
1	1	0		+IN									Off	Off
1	1	1			+IN (TEMP1)								Off	Off

这里要特别说明下，要读取 XPT2046 的数据，根据其时序图可知，时序图如下：



XPT2046 完成一个完整的转换需要 24 个串行时钟，也就是需要 3 个字节的 SPI 时钟。对照上图，XPT2046 前 8 个串行时钟，是接收 1 个字节的转换命令，接收到转换命令了之后，然后使用 1 个串行时钟的时间来完成数据转换（当然在编写程序的时候，为了得到精确的数据，你可以适当的延时一下），然后返回 12 个字节长度（12 个字节长度也计时 12 个串行时钟）的转换结果。然后最后 4 个串行时钟返回 4 个无效数据。这一过程在 TOUCH_Read_AD 函数内实现。

下面我们来讲解如何将触摸屏物理坐标转换为 LCD 坐标。

我们使用 XPT2046 读取到了触摸屏的触摸位置之后，想要在 LCD 屏相对应的位置上进行操作，我们还要将它转换成 LCD 屏的坐标值。比如说，我们在 LCD 屏（0,0）坐标位置按下，而读取到的物理坐标值(也就是 AD 值)为（100，200），那么我们想要在 LCD 屏（0，0）位置进行处理，将要将物理坐标（100,200）转换成 LCD 屏坐标。

那如何转换呢？我们知道，XPT2046 的分辨率为 12 位，也就是说我们读取 X 轴的物理坐标值（这里我们假设为：Px）和 Y 轴的物理坐标值（这里我们假设为：Py）的值肯定是在 0~4096 之间。但是我们 LCD 彩屏 X 轴和 Y 轴的像

素坐标确是 240X400。（对应 TFT26_R61509V 彩屏的像素，不管多少，我们明白原理就行，为了更好的表示，在这里我们 LCD 彩屏 X 轴像素坐标我们假设为：Lcdx,LCD 彩屏 Y 轴像素坐标我们假设为:Lcdy)那么我们假设当(Px, Py) =(0,0)时，正好 LCD 彩屏像素坐标的起始坐标（0,0），当(Px,Py) = (4096,4096)时，正好 LCD 彩屏像素坐标的终止坐标（239, 399）。难么我们不难看出触摸屏的物理坐标跟 LCD 彩屏像素坐标的对应关系为：

```
Factorx = Lcdx / Px;
Factory = Lcdy / Py;
```

我们就可以求出 Factorx 和 Factory，然后每次读取到 Px 和 Py 之后就可以讲它很轻松的转换为 Lcdx 和 Lcdy。这是一个很简单的数学关系。

不过呢，事情没有那么理想化，我们在 LCD 像素坐标为（0, 0）读取的触摸屏物理坐标值不一定是（0，0）,在 LCD 像素坐标为最大时，也不一定读取到的是触摸屏的物理坐标最大值。所以我们要进行一些数据校正，这也是为什么电阻屏幕校正的原因。

什么意思呢？那我们在来解一个数学问题：

我们都知道每个触摸屏物理坐标值都能一一对应一个 LCD 彩屏上面的像素坐标值，也就是它们是成比例关系的。现在我们知道 LCD 彩屏的 X 轴像素坐标最小值为 Lcdx1,我们能显示的 LCD 彩屏的 X 轴像素坐标最大值为 Lcdx2。而我们在 LCD 彩屏像素坐标 X 轴最小值处读取的触摸屏 X 轴物理坐标为 Px1，在 LCD 彩屏 X 轴像素坐标最大值处读取的触摸屏 X 轴的物理坐标为 Px2。那么现在我们知道有一个触摸屏物理坐标值在 Px1 到 Px2 之间的坐标值为 Px，那么和它对应的 Lcdx 的值是多少呢？

我们可以这么解：

```
Factorx = (Lcdx2 - Lcdx1) / (Px2 - Px1);
Lcdx = (Px - Px1) * Factorx;
```

那么就求得出 Lcdx 是多少了。

现在我们把它分解出来：

```
Lcdx = Px * Factorx - Px1 * Factorx;
```

然后将 Px1*Factorx 替换成一个变量 Offsetx。 那么我们现在就可以得到

Lcdx 和 Px 之间的对应关系式。而关于 Y 轴也是同理，所以它们从物理坐标到像素坐标的转换关系式：

```
Lcdx = Px * Factorx  -  Offsetx;
Lcdy = Py * Factory  -  Offsety;
```

在程序中对应的如下所示：

```
xpt_xy.lcdx=xpt_xy.x;
xpt_xy.lcdx=((xpt_xy.lcdx - 240)*260)/3850;
xpt_xy.lcdy=xpt_xy.y;
xpt_xy.lcdy =((xpt_xy.lcdy - 200)*420)/3950;
```

xpt_xy.x 和 xpt_xy.y 是读取 XPT2046 的 X 和 Y 方向的 AD 值，而 xpt_xy.lcdx 和 xpt_xy.lcdy 是对应 TFTLCD 触摸屏实际的 XY 轴坐标值。如果大家不理解没有关系，这些都是固定公式，直接按照我们例程套用即可。如果大家发现手上的触摸偏差很大，可以通过修改(xpt_xy.lcdx - 240)*260)中的 240 值，和(xpt_xy.lcdy - 200)*420)中的 200 值。里面的 260 和 420 是 TFTLCD 分辨率，不过我们只用到了 240*400。

在该文件中还有一个函数是非常重要的，即触摸屏扫描函数。该函数检测是否有触摸，并在该函数中获取触摸屏的物理坐标和 LCD 坐标。

32.3.2 主函数

打开 main.c 文件，代码如下：

```
#include "public.h"
#include "uart.h"
#include "tftlcd.h"
#include "touch.h"
#include "gui.h"


u8 Touch_RST=0;


//触摸测试
void TOUCH_Test(void)
{
```

```
static u16 penColor = BLUE;

TOUCH_Scan();
if(xpt_xy.sta)
{

    if(xpt_xy.lcdx>tftlcd_data.width)
        xpt_xy.lcdx=tftlcd_data.width-1;
    if(xpt_xy.lcdy>tftlcd_data.height)
        xpt_xy.lcdy=tftlcd_data.height-1;
    if((xpt_xy.lcdx>=(tftlcd_data.width-3*12))&&(xpt_xy.lcdy<24))
        Touch_RST=1;
    if(xpt_xy.lcdy > tftlcd_data.height - 20)
    {
        if(xpt_xy.lcdx>100)
        {
            penColor = YELLOW;
        }
        else if(xpt_xy.lcdx>80)
        {
            penColor = CYAN;
        }
        else if(xpt_xy.lcdx>60)
        {
            penColor = GREEN;
        }
        else if(xpt_xy.lcdx>40)
        {
           penColor = MAGENTA;
        }
        else if(xpt_xy.lcdx>20)
        {
            penColor = RED;

        }
        else if(xpt_xy.lcdx>0)
        {
            penColor = BLUE;
        }
    }
    else
    {
```

```
    LCD_Fill(xpt_xy.lcdx-2,xpt_xy.lcdy-2,xpt_xy.lcdx+2,xpt_xy.lcdy+2,
penColor);
        }

    }
}

//GUI 测试
void GUI_Test(void)
{
    FRONT_COLOR=BLACK;
    LCD_ShowFontHZ(tftlcd_data.width/2-2*24-12, 0,"普中科技");
    LCD_ShowString(tftlcd_data.width/2-7*12,30,tftlcd_data.width,tftl
cd_data.height,24,"www.prechin.cn");
    LCD_ShowString(tftlcd_data.width/2-7*8,55,tftlcd_data.width,tftlc
d_data.height,24,"GUI Test");

    FRONT_COLOR=RED;
    gui_draw_bigpoint(10,55,GREEN);
    gui_draw_bline(10,80,120,80,10,GREEN);
    gui_draw_rectangle(10,95,30,30,GREEN);
    gui_draw_arcrectangle(50,95,30,30,5,1,BLUE,GREEN);
    gui_fill_rectangle(90,95,30,30,GREEN);
    //gui_fill_circle(90,120,20,GREEN);
    gui_fill_ellipse(30,150,20,10,GREEN);

    //更多的 GUI 等待大家来发掘和编写

    delay_ms(2000);
    LCD_Clear(BACK_COLOR);
}

void main()
{

    u16 color=0;

    UART_Init();
    TFTLCD_Init();

Start:
    //GUI_Test();

    FRONT_COLOR=BLACK;
```

```
    LCD_ShowFontHZ(tftlcd_data.width/2-2*24-12, 0,"普中科技");
    LCD_ShowString(tftlcd_data.width/2-7*12,30,tftlcd_data.width,tftl
cd_data.height,24,"www.prechin.cn");
    FRONT_COLOR=RED;
    LCD_ShowString(tftlcd_data.width-3*12,0,tftlcd_data.width,tftlcd_
data.height,24,"RST");
    LCD_Fill(0, tftlcd_data.height - 20, 20, tftlcd_data.height, BLUE);
    LCD_Fill(20, tftlcd_data.height - 20, 40, tftlcd_data.height, RED);
    LCD_Fill(40, tftlcd_data.height - 20, 60, tftlcd_data.height,
MAGENTA);
    LCD_Fill(60,tftlcd_data.height-20,80,tftlcd_data.height,GREEN);
    LCD_Fill(80,tftlcd_data.height-20,100,tftlcd_data.height,CYAN);
    LCD_Fill(100, tftlcd_data.height - 20, 120, tftlcd_data.height,
YELLOW);


    while(1)
    {
        if(Touch_RST)
        {
            Touch_RST=0;
            LCD_Clear(BACK_COLOR);
            goto Start;
        }

        TOUCH_Test();

    }
}
```

主函数实现的功能很简单，首先调用之前编写好的硬件初始化函数，包括串口初始化等。然后调用我们前面编写的 TFTLCD 汉字字符显示等函数完成信息显示，接着进入 while 循环，不断检测 Touch_RST 变量是否置 1，如果为 1 进行 TFTLCD 清屏，然后重新回到 Start 入口处运行程序。如果没有置 1，则执行 TOUCH_Test()函数实现触摸功能。通过触摸不同色块选择颜色，就可以画出不同颜色的图画。

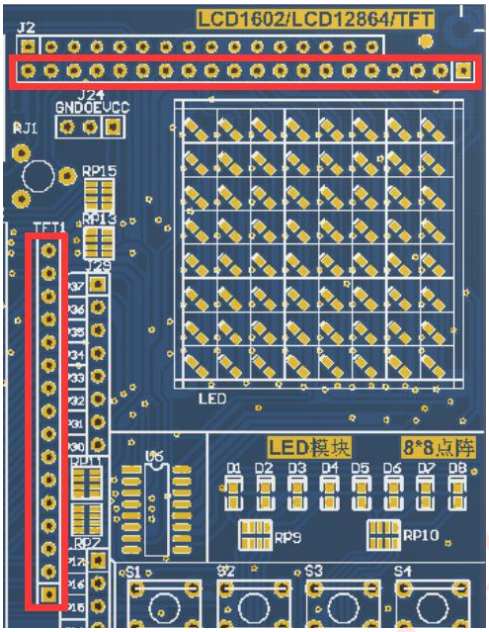
32.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 串口），

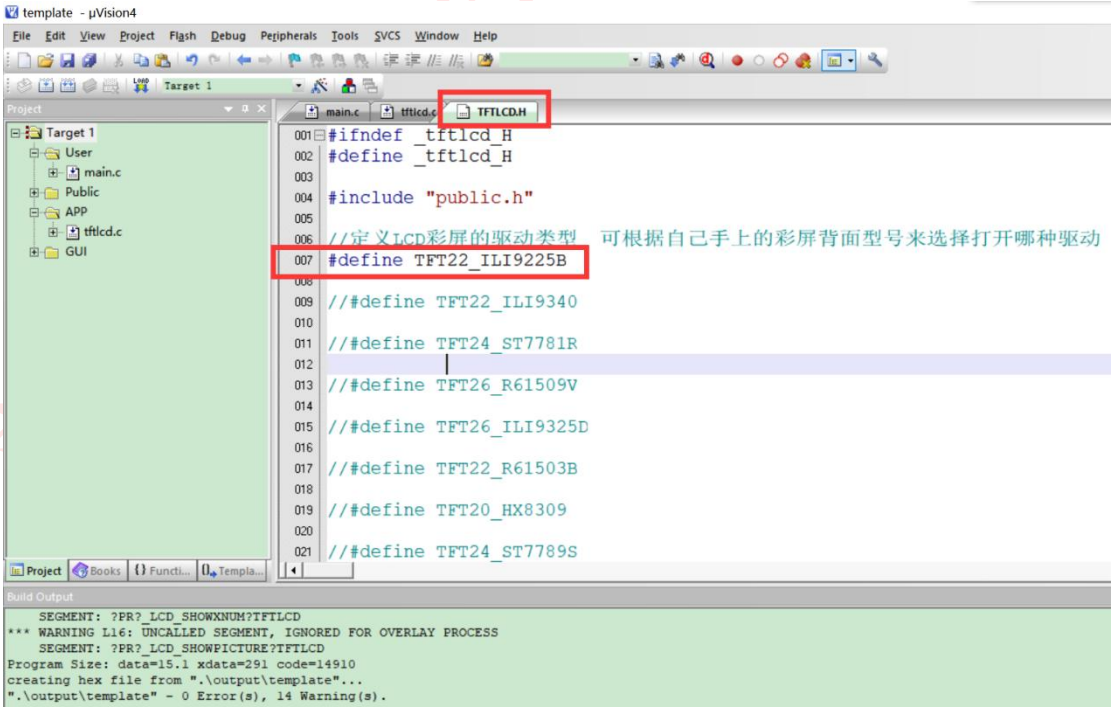
把编译后产生的.hex 文件烧入到芯片内，实现现象如下：将 TFTLCD 模块插上开发板上的彩屏接口，可以看到 TFTLCD 上显示提示信息，可通过触摸实现绘画功能。

注意：有的产品所配置的 TFTLCD 是不支持触摸功能，有的产品是支持触摸功能，所以根据自己产品所配置的 TFTLCD 来看，如果不支持触摸屏功能该实验无法实现。

注意：TFTLCD 显示屏要对应开发板彩屏接口插入，否则实验现象不正常，如下：



实验说明：下载有关 TFTLCD 显示实验时，请确保您所使用的 TFTLCD 彩屏驱动型号与程序内选择的一致，如何查看呢？首先看下您 TFTLCD 彩屏板左上角对应的驱动型号，比如 TFT22_ILI9225B。然后打开实验程序，查看 tftlcd.h 头文件开始处对应的 TFTLCD 驱动型号选择的是哪个？如下所示：



课后作业

普中51单片机开发板

普中51单片机开发板